

前端面试小册

关注公众号

“

关注公众号获取代码以及最新教程和文章，也可以联系作者，获取帮助

”



微信搜一搜



ByteTech

在线阅读

“

在线阅读体验更好，支持搜索，支持目录跳转，支持收藏

”

在线阅读 (<https://github.com/linwu-hi/code-interview/issues>)

前端面试小册

- 面试官：谈谈你对BFC的理解？ (<https://github.com/linwu-hi/code-interview#面试官：谈谈你对bfc的理解？>)
- 面试官：CSS如何画一个三角形？原理是什么？ (<https://github.com/linwu-hi/code-interview#面试官：css如何画一个三角形？原理是什么？>)
- 面试官：css选择器有哪些？优先级？哪些属性可以继承？ (<https://github.com/linwu-hi/code-interview#面试官：css选择器有哪些？优先级？哪些属性可以继承？>)
- 面试官：介绍一下grid网格布局 (<https://github.com/linwu-hi/code-interview#面试官：介绍一下grid网格布局>)
- 面试官：元素水平垂直居中的方法有哪些？如果元素不定宽高呢？ (<https://github.com/linwu-hi/code-interview#面试官：元素水平垂直居中的方法有哪些？如果元素不定宽高呢？>)
- 面试官：如何实现单行 / 多行文本溢出的省略样式？ (<https://github.com/linwu-hi/code-interview#面试官：如何实现单行 / 多行文本溢出的省略样式？>)
- 面试官：如果要做优化，CSS提高性能的方法有哪些？ (<https://github.com/linwu-hi/code-interview#面试官：如果要做优化，css提高性能的方法有哪些？>)
- 面试官：深入理解设备像素、CSS像素、设备独立像素、DPR、PPI之间的区别与适配方案 (<https://github.com/linwu-hi/code-interview#面试官：深入理解设备像素、css像素、设备独立像素、dpr、ppi之间的区别与适配方案>)
- 面试官：说说flexbox（弹性盒布局模型） (<https://github.com/linwu-hi/code-interview#面试官：说说flexbox（弹性盒布局模型）>)

[interview#面试官：说说flexbox（弹性盒布局模型）](#)）

- [面试官：深入理解CSS盒子模型 \(https://github.com/linwu-hi/code-interview#面试官：深入理解css盒子模型\)](#)
- [面试官：谈谈你对BFC的理解？ \(https://github.com/linwu-hi/code-interview#面试官：谈谈你对bfc的理解？\)](#)
- [面试官：CSS3动画有哪些？ \(https://github.com/linwu-hi/code-interview#面试官：css3动画有哪些？\)](#)
- [面试官：介绍一下grid网格布局 \(https://github.com/linwu-hi/code-interview#面试官：介绍一下grid网格布局\)](#)
- [面试官：DNS协议及完整的查询过程 \(https://github.com/linwu-hi/code-interview#面试官：dns协议及完整的查询过程\)](#)
- [面试官：为什么说HTTPS比HTTP安全？ HTTPS是如何保证安全的？ \(https://github.com/linwu-hi/code-interview#面试官：为什么说https比http安全？-https是如何保证安全的？\)](#)
- [面试官：什么是HTTP？ HTTP 和 HTTPS 的区别？ \(https://github.com/linwu-hi/code-interview#面试官：什么是http?-http-和-https-的区别?\)](#)
- [面试官：如何理解CDN？ 说说实现原理？ \(https://github.com/linwu-hi/code-interview#面试官：如何理解cdn？ 说说实现原理？\)](#)
- [面试官：如何理解OSI七层模型？ \(https://github.com/linwu-hi/code-interview#面试官：如何理解osi七层模型?\)](#)
- [面试官：如何理解TCP/IP协议？ \(https://github.com/linwu-hi/code-interview#面试官：如何理解tcp/ip协议?\)](#)
- [面试官：如何理解UDP 和 TCP？ 区别？ 应用场景？ \(https://github.com/linwu-hi/code-interview#面试官：如何理解udp-和-tcp?-区别?-应用场景?\)](#)
- [面试官：GET和POST的区别及使用场景 \(https://github.com/linwu-hi/code-interview#面试官：get和post的区别及使用场景\)](#)
- [面试官：说说 HTTP 常见的请求头有哪些？ 作用？ \(https://github.com/linwu-hi/code-interview#面试官：说说-http-常见的请求头有哪些?-作用？\)](#)
- [面试官：说说HTTP1.0/1.1/2.0的区别？ \(https://github.com/linwu-hi/code-interview#面试官：说说http1.0/1.1/2.0的区别?\)](#)
- [面试官：说说HTTP 常见的状态码有哪些，适用场景？ \(https://github.com/linwu-hi/code-interview#面试官：说说http-常见的状态码有哪些，适用场景?\)](#)
- [面试官：说说TCP为什么需要三次握手和四次挥手？ \(https://github.com/linwu-hi/code-interview#面试官：说说tcp为什么需要三次握手和四次挥手?\)](#)
- [面试官：说说地址栏输入 URL 敲下回车后发生了什么？ \(https://github.com/linwu-hi/code-interview#面试官：说说地址栏输入-url-敲下回车后发生了什么?\)](#)
- [面试官：说说对WebSocket的理解？ 应用场景？ \(https://github.com/linwu-hi/code-interview#面试官：说说对websocket的理解？ 应用场景?\)](#)
- [Date类：日期和时间处理 \(https://github.com/linwu-hi/code-interview#date类：日期和时间处理\)](#)
- [JS中的异步编程与Promise \(https://github.com/linwu-hi/code-interview#js中的异步编程与promise\)](#)
- [JavaScript中的Generator函数与其在实现Async/Await的应用 \(https://github.com/linwu-hi/code-interview#javascript中的generator函数与其在实现async/await的应用\)](#)
- [JavaScript中的编码 \(https://github.com/linwu-hi/code-interview#javascript中的编码\)](#)
- [JavaScript事件流：深入理解事件处理和传播机制 \(https://github.com/linwu-hi/code-interview#javascript事件流：深入理解事件处理和传播机制\)](#)
- [JavaScript修饰器：简化代码，增强功能 \(https://github.com/linwu-hi/code-interview#javascript修饰器：简化代码，增强功能\)](#)
- [JavaScript对象 \(https://github.com/linwu-hi/code-interview#javascript对象\)](#)
- [JavaScript数组 \(https://github.com/linwu-hi/code-interview#javascript数组\)](#)
- [JavaScript数据类型和类型转换 \(https://github.com/linwu-hi/code-interview#javascript数据类型和类型转换\)](#)
- [requestAnimationFrame：优化动画和渲染的利器 \(https://github.com/linwu-hi/code-interview#requestanimationframe：优化动画和渲染的利器\)](#)
- [作用域和作用域链 \(https://github.com/linwu-hi/code-interview#作用域和作用域链\)](#)
- [函数上下文和this关键字 \(https://github.com/linwu-hi/code-interview#函数上下文和this关键字\)](#)

- [前端跨页面通信：实现页面间的数据传递与交互 \(https://github.com/linwu-hi/code-interview#前端跨页面通信：实现页面间的数据传递与交互\)](https://github.com/linwu-hi/code-interview#前端跨页面通信：实现页面间的数据传递与交互)
- [原型和原型链 \(https://github.com/linwu-hi/code-interview#原型和原型链\)](https://github.com/linwu-hi/code-interview#原型和原型链)
- [实现符合Promise/A+规范的Promise \(https://github.com/linwu-hi/code-interview#实现符合promise/a+规范的promise\)](https://github.com/linwu-hi/code-interview#实现符合Promise/A+规范的Promise)
- [异步的终极解决方案：async/await \(https://github.com/linwu-hi/code-interview#异步的终极解决方案：async/await\)](https://github.com/linwu-hi/code-interview#异步的终极解决方案：async/await)
- [执行上下文与闭包 \(https://github.com/linwu-hi/code-interview#执行上下文与闭包\)](https://github.com/linwu-hi/code-interview#执行上下文与闭包)
- [深入理解Proxy \(https://github.com/linwu-hi/code-interview#深入理解proxy\)](https://github.com/linwu-hi/code-interview#深入理解Proxy)
- [详解Cookie, Session, sessionStorage, localStorage \(https://github.com/linwu-hi/code-interview#详解cookie,-session,-sessionstorage,-localstorage\)](https://github.com/linwu-hi/code-interview#详解Cookie, Session, sessionStorage, localStorage)
- [面向对象编程与Class \(https://github.com/linwu-hi/code-interview#面向对象编程与class\)](https://github.com/linwu-hi/code-interview#面向对象编程与Class)
- [面试官：说说linux系统下常用的命令有哪些？ \(https://github.com/linwu-hi/code-interview#面试官：说说-linux-系统下常用的命令有哪些？\)](https://github.com/linwu-hi/code-interview#面试官：说说linux系统下常用的命令有哪些？)
- [TS实战之扑克牌排序 \(https://github.com/linwu-hi/code-interview#ts实战之扑克牌排序\)](https://github.com/linwu-hi/code-interview#TS实战之扑克牌排序)
- [函数 \(https://github.com/linwu-hi/code-interview#函数\)](https://github.com/linwu-hi/code-interview#函数)
- [命名空间和模块 \(https://github.com/linwu-hi/code-interview#命名空间和模块\)](https://github.com/linwu-hi/code-interview#命名空间和模块)
- [扩展类型定义 \(https://github.com/linwu-hi/code-interview#扩展类型定义\)](https://github.com/linwu-hi/code-interview#扩展类型定义)
- [接口和类 \(https://github.com/linwu-hi/code-interview#接口和类\)](https://github.com/linwu-hi/code-interview#接口和类)
- [枚举和泛型 \(https://github.com/linwu-hi/code-interview#枚举和泛型\)](https://github.com/linwu-hi/code-interview#枚举和泛型)
- [泛型和类型体操 \(https://github.com/linwu-hi/code-interview#泛型和类型体操\)](https://github.com/linwu-hi/code-interview#泛型和类型体操)
- [类型 \(https://github.com/linwu-hi/code-interview#类型\)](https://github.com/linwu-hi/code-interview#类型)
- [类型兼容：协变和逆变 \(https://github.com/linwu-hi/code-interview#类型兼容：协变和逆变\)](https://github.com/linwu-hi/code-interview#类型兼容：协变和逆变)
- [类型兼容：结构化类型 \(https://github.com/linwu-hi/code-interview#类型兼容：结构化类型\)](https://github.com/linwu-hi/code-interview#类型兼容：结构化类型)
- [类型守卫 \(https://github.com/linwu-hi/code-interview#类型守卫\)](https://github.com/linwu-hi/code-interview#类型守卫)
- [类型推断 \(https://github.com/linwu-hi/code-interview#类型推断\)](https://github.com/linwu-hi/code-interview#类型推断)
- [类型系统层级 \(https://github.com/linwu-hi/code-interview#类型系统层级\)](https://github.com/linwu-hi/code-interview#类型系统层级)
- [装饰器与反射元数据 \(https://github.com/linwu-hi/code-interview#装饰器与反射元数据\)](https://github.com/linwu-hi/code-interview#装饰器与反射元数据)
- [解读TSConfig \(https://github.com/linwu-hi/code-interview#解读tsconfig\)](https://github.com/linwu-hi/code-interview#解读TSConfig)
- [面试官：说说你对TypeScript的认识？ \(https://github.com/linwu-hi/code-interview#面试官：说说你对typescript的认识？\)](https://github.com/linwu-hi/code-interview#面试官：说说你对TypeScript的认识？)
- [高级类型 \(https://github.com/linwu-hi/code-interview#高级类型\)](https://github.com/linwu-hi/code-interview#高级类型)
- [面试官：说说flexbox（弹性盒布局模型） \(https://github.com/linwu-hi/code-interview#面试官：说说flexbox（弹性盒布局模型）\)](https://github.com/linwu-hi/code-interview#面试官：说说flexbox（弹性盒布局模型）)
- [面试官：说说Git常用的命令有哪些？ \(https://github.com/linwu-hi/code-interview#面试官：说说git常用的命令有哪些？\)](https://github.com/linwu-hi/code-interview#面试官：说说Git常用的命令有哪些？)
- [面试官：探索现代图片格式：从GIF到HEIF，优势与适用场景一览 \(https://github.com/linwu-hi/code-interview#面试官：探索现代图片格式：从gif到heif，优势与适用场景一览\)](https://github.com/linwu-hi/code-interview#面试官：探索现代图片格式：从GIF到HEIF，优势与适用场景一览)
- [面试官：Node性能如何进行监控以及优化？ \(https://github.com/linwu-hi/code-interview#面试官：node性能如何进行监控以及优化？\)](https://github.com/linwu-hi/code-interview#面试官：Node性能如何进行监控以及优化？)
- [面试官：如何实现jwt鉴权机制？说说你的思路 \(https://github.com/linwu-hi/code-interview#面试官：如何实现jwt鉴权机制？说说你的思路\)](https://github.com/linwu-hi/code-interview#面试官：如何实现jwt鉴权机制？说说你的思路)
- [面试官：如何实现文件上传？说说你的思路 \(https://github.com/linwu-hi/code-interview#面试官：如何实现文件上传？说说你的思路\)](https://github.com/linwu-hi/code-interview#面试官：如何实现文件上传？说说你的思路)
- [面试官：如果让你来设计一个分页功能，你会怎么设计？前后端如何交互？ \(https://github.com/linwu-hi/code-interview#面试官：如果让你来设计一个分页功能，你会怎么设计？前后端如何交互？\)](https://github.com/linwu-hi/code-interview#面试官：如果让你来设计一个分页功能，你会怎么设计？前后端如何交互？)
- [面试官：Node文件查找的优先级与Require方法的文件查找策略 \(https://github.com/linwu-hi/code-interview#面试官：node文件查找的优先级与require方法的文件查找策略\)](https://github.com/linwu-hi/code-interview#面试官：Node文件查找的优先级与Require方法的文件查找策略)
- [面试官：Node.js中有哪些全局对象？ \(https://github.com/linwu-hi/code-interview#面试官：node.js-中有哪些全局对象？\)](https://github.com/linwu-hi/code-interview#面试官：Node.js中有哪些全局对象？)
- [面试官：说说Node中的EventEmitter？如何实现一个EventEmitter？](https://github.com/linwu-hi/code-interview#面试官：说说Node中的EventEmitter？如何实现一个EventEmitter？)

<https://github.com/linwu-hi/code-interview#面试官：说说node中的eventemitter?-如何实现一个eventemitter?>

- [面试官：说说你对Node.js的理解？优缺点？应用场景？](https://github.com/linwu-hi/code-interview#面试官：说说你对Node.js的理解？优缺点？应用场景？) (<https://github.com/linwu-hi/code-interview#面试官：说说你对node.js-的理解？优缺点？应用场景？>)
- [面试官：说说对Node中的Buffer的理解？应用场景？](https://github.com/linwu-hi/code-interview#面试官：说说对Node中的Buffer的理解？应用场景？) (<https://github.com/linwu-hi/code-interview#面试官：说说对-node-中的-buffer-的理解？应用场景？>)
- [面试官：说说对Node中的Stream的理解？应用场景？](https://github.com/linwu-hi/code-interview#面试官：说说对Node中的Stream的理解？应用场景？) (<https://github.com/linwu-hi/code-interview#面试官：说说对-node-中的-stream-的理解？应用场景？>)
- [面试官：说说对Node中的fs模块的理解？有哪些常用方法](https://github.com/linwu-hi/code-interview#面试官：说说对Node中的fs模块的理解？有哪些常用方法) (<https://github.com/linwu-hi/code-interview#面试官：说说对-node-中的-fs模块的理解?-有哪些常用方法>)
- [面试官：说说对Node中的process的理解？有哪些常用方法？](https://github.com/linwu-hi/code-interview#面试官：说说对Node中的process的理解？有哪些常用方法？) (<https://github.com/linwu-hi/code-interview#面试官：说说对-node-中的-process-的理解？有哪些常用方法？>)
- [面试官：说说对Nodejs中的事件循环机制理解？](https://github.com/linwu-hi/code-interview#面试官：说说对Nodejs中的事件循环机制理解？) (<https://github.com/linwu-hi/code-interview#面试官：说说对nodejs中的事件循环机制理解？>)
- [面试官：说说对中间件概念的理解，如何封装node中间件？](https://github.com/linwu-hi/code-interview#面试官：说说对中间件概念的理解，如何封装node中间件？) (<https://github.com/linwu-hi/code-interview#面试官：说说对中间件概念的理解，如何封装-node-中间件？>)
- [面试官：React中的key有什么作用？](https://github.com/linwu-hi/code-interview#面试官：React中的key有什么作用？) (<https://github.com/linwu-hi/code-interview#面试官：react中的key有什么作用？>)
- [面试官：React中组件之间如何通信？](https://github.com/linwu-hi/code-interview#面试官：React中组件之间如何通信？) (<https://github.com/linwu-hi/code-interview#面试官：react中组件之间如何通信？>)
- [面试官：React事件绑定的方式有哪些？区别？](https://github.com/linwu-hi/code-interview#面试官：React事件绑定的方式有哪些？区别？) (<https://github.com/linwu-hi/code-interview#面试官：react事件绑定的方式有哪些？区别？>)
- [面试官：React构建组件的方式有哪些？区别？](https://github.com/linwu-hi/code-interview#面试官：React构建组件的方式有哪些？区别？) (<https://github.com/linwu-hi/code-interview#面试官：react构建组件的方式有哪些？区别？>)
- [面试官：state和props有什么区别？](https://github.com/linwu-hi/code-interview#面试官：state和props有什么区别？) (<https://github.com/linwu-hi/code-interview#面试官：state-和-props-有什么区别？>)
- [面试官：super\(\)和super\(props\)有什么区别？](https://github.com/linwu-hi/code-interview#面试官：super()和super(props)有什么区别？) ([https://github.com/linwu-hi/code-interview#面试官：super\(\)-和-super\(props\)-有什么区别？](https://github.com/linwu-hi/code-interview#面试官：super()-和-super(props)-有什么区别？))
- [面试官：你在React项目中是如何使用Redux的？项目结构是如何划分的？](https://github.com/linwu-hi/code-interview#面试官：你在React项目中是如何使用Redux的？项目结构是如何划分的？) (<https://github.com/linwu-hi/code-interview#面试官：你在react项目中是如何使用redux的?-项目结构是如何划分的？>)
- [面试官：在react中组件间过渡动画如何实现？](https://github.com/linwu-hi/code-interview#面试官：在react中组件间过渡动画如何实现？) (<https://github.com/linwu-hi/code-interview#面试官：在react中组件间过渡动画如何实现？>)
- [面试官：说说React性能优化的手段有哪些？](https://github.com/linwu-hi/code-interview#面试官：说说React性能优化的手段有哪些？) (<https://github.com/linwu-hi/code-interview#面试官：说说-react-性能优化的手段有哪些？>)
- [面试官：说说React生命周期有哪些不同阶段？每个阶段对应的方法是？](https://github.com/linwu-hi/code-interview#面试官：说说React生命周期有哪些不同阶段？每个阶段对应的方法是？) (<https://github.com/linwu-hi/code-interview#面试官：说说-react-生命周期有哪些不同阶段？每个阶段对应的方法是？>)
- [面试官：说说React中的setState执行机制](https://github.com/linwu-hi/code-interview#面试官：说说React中的setState执行机制) (<https://github.com/linwu-hi/code-interview#面试官：说说-react中的setstate执行机制>)
- [面试官：说说Real DOM和Virtual DOM的区别？优缺点？](https://github.com/linwu-hi/code-interview#面试官：说说Real DOM和Virtual DOM的区别？优缺点？) (<https://github.com/linwu-hi/code-interview#面试官：说说-real-dom-和-virtual-dom-的区别？优缺点？>)
- [面试官：说说React JSX转换成真实DOM过程？](https://github.com/linwu-hi/code-interview#面试官：说说React JSX转换成真实DOM过程？) (<https://github.com/linwu-hi/code-interview#面试官：说说react-jsx转换成真实dom过程？>)
- [面试官：说说React Router有几种模式？实现原理？](https://github.com/linwu-hi/code-interview#面试官：说说React Router有几种模式？实现原理？) (<https://github.com/linwu-hi/code-interview#面试官：说说react-router有几种模式？实现原理？>)
- [面试官：说说React diff的原理是什么？](https://github.com/linwu-hi/code-interview#面试官：说说React diff的原理是什么？) (<https://github.com/linwu-hi/code-interview#面试官：说说react-diff的原理是什么？>)
- [面试官：说说React render方法的原理？在什么时候会被触发？](https://github.com/linwu-hi/code-interview#面试官：说说React render方法的原理？在什么时候会被触发？) (<https://github.com/linwu-hi/code-interview#面试官：说说react-render方法的原理？在什么时候会被触发？>)
- [面试官：说说React服务端渲染怎么做？原理是什么？](https://github.com/linwu-hi/code-interview#面试官：说说React服务端渲染怎么做？原理是什么？) (<https://github.com/linwu-hi/code-interview#面试官：说说react服务端渲染怎么做？原理是什么？>)
- [面试官：说说React的事件机制？](https://github.com/linwu-hi/code-interview#面试官：说说React的事件机制？) (<https://github.com/linwu-hi/code-interview#面试官：说说react的事件机制？>)
- [面试官：说说react中引入css的方式有哪几种？区别？](https://github.com/linwu-hi/code-interview#面试官：说说react中引入css的方式有哪几种？区别？) (<https://github.com/linwu-hi/code-interview#面试官：说说react中引入css的方式有哪几种？区别？>)

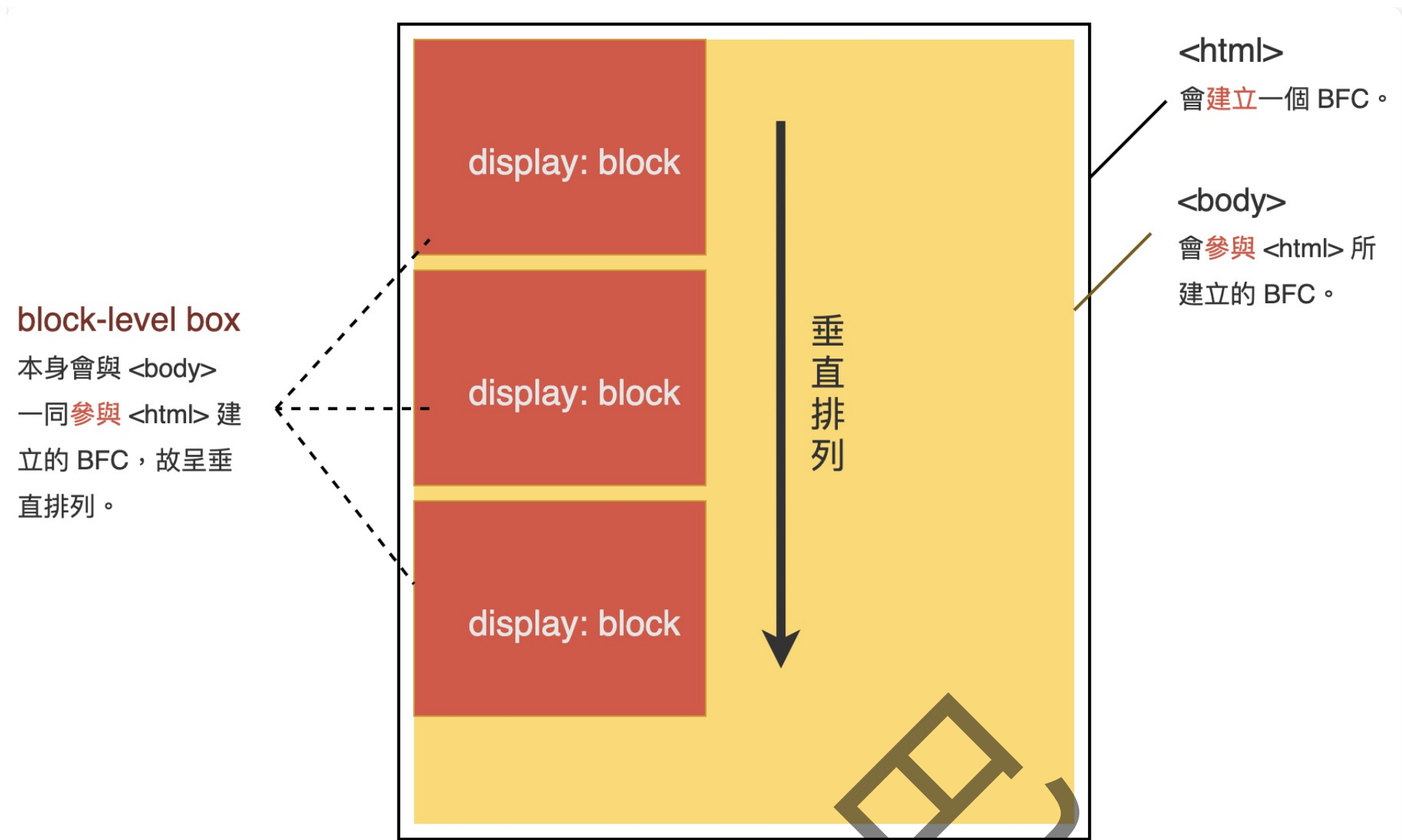
- [面试官：说说你在React项目是如何捕获错误的？](https://github.com/linwu-hi/code-interview#面试官：说说你在React项目是如何捕获错误的？) (<https://github.com/linwu-hi/code-interview#面试官：说说你在react项目是如何捕获错误的？>)
- [面试官：说说你在使用React过程中遇到的常见问题？如何解决？](https://github.com/linwu-hi/code-interview#面试官：说说你在使用React过程中遇到的常见问题？如何解决？) (<https://github.com/linwu-hi/code-interview#面试官：说说你在使用react-过程中遇到的常见问题？如何解决？>)
- [面试官：说说你对React Router的理解？常用的Router组件有哪些？](https://github.com/linwu-hi/code-interview#面试官：说说你对React Router的理解？常用的Router组件有哪些？) (<https://github.com/linwu-hi/code-interview#面试官：说说你对react-router的理解？常用的router组件有哪些？>)
- [说说你对Redux的理解？其工作原理？](https://github.com/linwu-hi/code-interview#面试官：说说你对Redux的理解？其工作原理？) (<https://github.com/linwu-hi/code-interview#面试官：说说你对redux的理解？其工作原理？>)
- [面试官：说说你对immutable的理解？如何应用在react项目中？](https://github.com/linwu-hi/code-interview#面试官：说说你对immutable的理解？如何应用在react项目中？) (<https://github.com/linwu-hi/code-interview#面试官：说说你对immutable的理解？如何应用在react项目中？>)
- [面试官：说说你是如何提高组件的渲染效率的？在React中如何避免不必要的render？](https://github.com/linwu-hi/code-interview#面试官：说说你是如何提高组件的渲染效率的？在React中如何避免不必要的render？) (<https://github.com/linwu-hi/code-interview#面试官：说说你是如何提高组件的渲染效率的？在react中如何避免不必要的render？>)
- [面试官：说说对React的理解？有哪些特性？](https://github.com/linwu-hi/code-interview#面试官：说说对React的理解？有哪些特性？) (<https://github.com/linwu-hi/code-interview#面试官：说说对-react-的理解？有哪些特性？>)
- [面试官：说说对Fiber架构的理解？解决了什么问题？](https://github.com/linwu-hi/code-interview#面试官：说说对Fiber架构的理解？解决了什么问题？) (<https://github.com/linwu-hi/code-interview#面试官：说说对fiber架构的理解？解决了什么问题？>)
- [面试官：说说对React Hooks的理解？解决了什么问题？](https://github.com/linwu-hi/code-interview#面试官：说说对React Hooks的理解？解决了什么问题？) (<https://github.com/linwu-hi/code-interview#面试官：说说对react-hooks的理解？解决了什么问题？>)
- [面试官：说说对React refs的理解？应用场景？](https://github.com/linwu-hi/code-interview#面试官：说说对React refs的理解？应用场景？) (<https://github.com/linwu-hi/code-interview#面试官：说说对react-refs-的理解？应用场景？>)
- [面试官：说说对React中类组件和函数组件的理解？有什么区别？](https://github.com/linwu-hi/code-interview#面试官：说说对React中类组件和函数组件的理解？有什么区别？) (<https://github.com/linwu-hi/code-interview#面试官：说说对react中类组件和函数组件的理解？有什么区别？>)
- [面试官：说说对Redux中间件的理解？常用的中间件有哪些？实现原理？](https://github.com/linwu-hi/code-interview#面试官：说说对Redux中间件的理解？常用的中间件有哪些？实现原理？) (<https://github.com/linwu-hi/code-interview#面试官：说说对redux中间件的理解？常用的中间件有哪些？实现原理？>)
- [面试官：说说对受控组件和非受控组件的理解？应用场景？](https://github.com/linwu-hi/code-interview#面试官：说说对受控组件和非受控组件的理解？应用场景？) (<https://github.com/linwu-hi/code-interview#面试官：说说对受控组件和非受控组件的理解？应用场景？>)
- [面试官：说说对高阶组件的理解？应用场景？](https://github.com/linwu-hi/code-interview#面试官：说说对高阶组件的理解？应用场景？) (<https://github.com/linwu-hi/code-interview#面试官：说说对高阶组件的理解？应用场景？>)
- [面试官：SPA首屏加载速度慢的怎么解决？](https://github.com/linwu-hi/code-interview#面试官：SPA首屏加载速度慢的怎么解决？) (<https://github.com/linwu-hi/code-interview#面试官：spa首屏加载速度慢的怎么解决？>)
- [面试官：SSR解决了什么问题？有做过SSR吗？你是怎么做的？](https://github.com/linwu-hi/code-interview#面试官：SSR解决了什么问题？有做过SSR吗？你是怎么做的？) (<https://github.com/linwu-hi/code-interview#面试官：ssr解决了什么问题？有做过ssr吗？你是怎么做的？>)
- [面试官：Vue.observable你有了解过吗？说说看](https://github.com/linwu-hi/code-interview#面试官：Vue.observable你有了解过吗？说说看) (<https://github.com/linwu-hi/code-interview#面试官：vue.observable你有了解过吗？说说看>)
- [面试官：Vue中的\\$nextTick有什么作用？](https://github.com/linwu-hi/code-interview#面试官：Vue中的$nextTick有什么作用？) ([https://github.com/linwu-hi/code-interview#面试官：vue中的\\$nexttick有什么作用？](https://github.com/linwu-hi/code-interview#面试官：vue中的$nexttick有什么作用？))
- [面试官：Vue中的过滤器了解吗？过滤器的应用场景有哪些？](https://github.com/linwu-hi/code-interview#面试官：Vue中的过滤器了解吗？过滤器的应用场景有哪些？) (<https://github.com/linwu-hi/code-interview#面试官：vue中的过滤器了解吗？过滤器的应用场景有哪些？>)
- [面试官：Vue中组件和插件有什么区别？](https://github.com/linwu-hi/code-interview#面试官：Vue中组件和插件有什么区别？) (<https://github.com/linwu-hi/code-interview#面试官：vue中组件和插件有什么区别？>)
- [面试官：Vue实例挂载的过程](https://github.com/linwu-hi/code-interview#面试官：Vue实例挂载的过程) (<https://github.com/linwu-hi/code-interview#面试官：vue实例挂载的过程>)
- [面试官：Vue常用的修饰符有哪些有什么应用场景](https://github.com/linwu-hi/code-interview#面试官：Vue常用的修饰符有哪些有什么应用场景) (<https://github.com/linwu-hi/code-interview#面试官：vue常用的修饰符有哪些有什么应用场景>)
- [面试官：Vue组件之间的通信方式都有哪些？](https://github.com/linwu-hi/code-interview#面试官：Vue组件之间的通信方式都有哪些？) (<https://github.com/linwu-hi/code-interview#面试官：vue组件之间的通信方式都有哪些？>)
- [面试官：Vue项目中你是如何解决跨域的呢？](https://github.com/linwu-hi/code-interview#面试官：Vue项目中你是如何解决跨域的呢？) (<https://github.com/linwu-hi/code-interview#面试官：vue项目中你是如何解决跨域的呢？>)
- [面试官：Vue项目中有封装过axios吗？主要是封装哪方面的？](https://github.com/linwu-hi/code-interview#面试官：Vue项目中有封装过axios吗？主要是封装哪方面的？) (<https://github.com/linwu-hi/code-interview#面试官：vue项目中有封装过axios吗？主要是封装哪方面的？>)
- [面试官：v-if和v-for的优先级是什么？](https://github.com/linwu-hi/code-interview#面试官：v-if和v-for的优先级是什么？) (<https://github.com/linwu-hi/code-interview#面试官：v-if和v-for的优先级是什么？>)

面试官：v-if和v-for的优先级是什么？)

- 面试官：v-show和v-if有什么区别？使用场景分别是什么？(<https://github.com/linwu-hi/code-interview#面试官：v-show和v-if有什么区别？使用场景分别是什么？>)
- 面试官：vue3有了解过吗？能说说跟vue2的区别吗？(<https://github.com/linwu-hi/code-interview#面试官：vue3有了解过吗？能说说跟vue2的区别吗？>)
- 面试官：vue要做权限管理该怎么做？如果控制到按钮级别的权限怎么做？(<https://github.com/linwu-hi/code-interview#面试官：vue要做权限管理该怎么做？如果控制到按钮级别的权限怎么做？>)
- 面试官：vue项目本地开发完成后部署到服务器后报404是什么原因呢？(<https://github.com/linwu-hi/code-interview#面试官：vue项目本地开发完成后部署到服务器后报404是什么原因呢？>)
- 面试官：为什么data属性是一个函数而不是一个对象？(<https://github.com/linwu-hi/code-interview#面试官：为什么data属性是一个函数而不是一个对象？>)
- 面试官：什么是虚拟DOM？如何实现一个虚拟DOM？说说你的思路(<https://github.com/linwu-hi/code-interview#面试官：什么是虚拟dom？如何实现一个虚拟dom？说说你的思路>)
- 面试官：你了解axios的原理吗？有看过它的源码吗？(<https://github.com/linwu-hi/code-interview#面试官：你了解axios的原理吗？有看过它的源码吗？>)
- 面试官：你了解vue的diff算法吗？说说看(<https://github.com/linwu-hi/code-interview#面试官：你了解vue的diff算法吗？说说看>)
- 面试官：你对SPA单页面的理解，它的优缺点分别是什么？如何实现SPA应用呢(<https://github.com/linwu-hi/code-interview#面试官：你对spa单页面的理解，它的优缺点分别是什么？如何实现spa应用呢>)
- 面试官：你是怎么处理vue项目中的错误的？(<https://github.com/linwu-hi/code-interview#面试官：你是怎么处理vue项目中的错误的？>)
- 面试官：你有写过自定义指令吗？自定义指令的应用场景有哪些？(<https://github.com/linwu-hi/code-interview#面试官：你有写过自定义指令吗？自定义指令的应用场景有哪些？>)
- 面试官：你知道vue中key的原理吗？说说你对它的理解(<https://github.com/linwu-hi/code-interview#面试官：你知道vue中key的原理吗？说说你对它的理解>)
- 面试官：动态给vue的data添加一个新的属性时会发生什么？怎样解决？(<https://github.com/linwu-hi/code-interview#面试官：动态给vue的data添加一个新的属性时会发生什么？怎样解决？>)
- 面试官：双向数据绑定是什么(<https://github.com/linwu-hi/code-interview#面试官：双向数据绑定是什么>)
- 面试官：有使用过vue吗？说说你对vue的理解(<https://github.com/linwu-hi/code-interview#面试官：有使用过vue吗？说说你对vue的理解>)
- 面试官：说下你的vue项目的目录结构，如果是大型项目你该怎么划分结构和划分组件呢？(<https://github.com/linwu-hi/code-interview#面试官：说下你的vue项目的目录结构，如果是大型项目你该怎么划分结构和划分组件呢？>)
- 面试官：说说你对keep-alive的理解是什么？(<https://github.com/linwu-hi/code-interview#面试官：说说你对keep-alive的理解是什么？>)
- 面试官：说说你对slot的理解？slot使用场景有哪些？(<https://github.com/linwu-hi/code-interview#面试官：说说你对slot的理解？slot使用场景有哪些？>)
- 面试官：说说你对vue的mixin的理解，有什么应用场景？(<https://github.com/linwu-hi/code-interview#面试官：说说你对vue的mixin的理解，有什么应用场景？>)
- 面试官：请描述下你对vue生命周期的理解？在created和mounted这两个生命周期中请求数据有什么区别呢？(<https://github.com/linwu-hi/code-interview#面试官：请描述下你对vue生命周期的理解？在created和mounted这两个生命周期中请求数据有什么区别呢？>)
- 面试官：Vue3.0 所采用的 Composition Api 与 Vue2.x 使用的 Options Api 有什么不同？(<https://github.com/linwu-hi/code-interview#面试官：vue3.0-所采用的-composition-api-与-vue2.x-使用的-options-api-有什么不同？>)
- 面试官：Vue3.0性能提升主要是通过哪几方面体现的？(<https://github.com/linwu-hi/code-interview#面试官：vue3.0性能提升主要是通过哪几方面体现的？>)
- 面试官：Vue3.0的设计目标是什么？做了哪些优化(<https://github.com/linwu-hi/code-interview#面试官：vue3.0的设计目标是什么？做了哪些优化>)
- 面试官：Vue3.0里为什么要用 Proxy API 替代 defineProperty API ？(<https://github.com/linwu-hi/code-interview#面试官：vue3.0里为什么要用-proxy-api-替代-defineproperty-api-？>)

- [面试官：用Vue3.0 写过组件吗？如果想实现一个 Modal你会怎么设计？
\(https://github.com/linwu-hi/code-interview#面试官：用vue3.0-写过组件吗？如果想实现一个-modal你会怎么设计？\)](https://github.com/linwu-hi/code-interview#面试官：用vue3.0-写过组件吗？如果想实现一个-modal你会怎么设计？)
- [面试官：说说Vue 3.0中Treeshaking特性？举例说明一下？
\(https://github.com/linwu-hi/code-interview#面试官：说说vue-3.0中treeshaking特性？举例说明一下？\)](https://github.com/linwu-hi/code-interview#面试官：说说vue-3.0中treeshaking特性？举例说明一下？)
- [面试官：与webpack类似的工具还有哪些？区别？
\(https://github.com/linwu-hi/code-interview#面试官：与webpack类似的工具还有哪些？区别？\)](https://github.com/linwu-hi/code-interview#面试官：与webpack类似的工具还有哪些？区别？)
- [面试官：如何提高webpack的构建速度？
\(https://github.com/linwu-hi/code-interview#面试官：如何提高webpack的构建速度？\)](https://github.com/linwu-hi/code-interview#面试官：如何提高webpack的构建速度？)
- [面试官：说说Loader和Plugin的区别？编写Loader，Plugin的思路？
\(https://github.com/linwu-hi/code-interview#面试官：说说loader和plugin的区别？编写loader，plugin的思路？\)](https://github.com/linwu-hi/code-interview#面试官：说说Loader和Plugin的区别？编写Loader，Plugin的思路？)
- [面试官：说说webpack proxy工作原理？为什么能解决跨域？
\(https://github.com/linwu-hi/code-interview#面试官：说说webpack-proxy工作原理？为什么能解决跨域？\)](https://github.com/linwu-hi/code-interview#面试官：说说webpack proxy工作原理？为什么能解决跨域？)
- [面试官：说说webpack中常见的Loader？解决了什么问题？
\(https://github.com/linwu-hi/code-interview#面试官：说说webpack中常见的loader？解决了什么问题？\)](https://github.com/linwu-hi/code-interview#面试官：说说webpack中常见的Loader？解决了什么问题？)
- [面试官：说说webpack中常见的Plugin？解决了什么问题？
\(https://github.com/linwu-hi/code-interview#面试官：说说webpack中常见的plugin？解决了什么问题？\)](https://github.com/linwu-hi/code-interview#面试官：说说webpack中常见的Plugin？解决了什么问题？)
- [面试官：说说webpack的构建流程？
\(https://github.com/linwu-hi/code-interview#面试官：说说webpack的构建流程？\)](https://github.com/linwu-hi/code-interview#面试官：说说webpack的构建流程？)
- [面试官：说说webpack的热更新是如何做到的？原理是什么？
\(https://github.com/linwu-hi/code-interview#面试官：说说webpack的热更新是如何做到的？原理是什么？\)](https://github.com/linwu-hi/code-interview#面试官：说说webpack的热更新是如何做到的？原理是什么？)
- [面试官：说说你对Webpack的理解？解决了什么问题？
\(https://github.com/linwu-hi/code-interview#面试官：说说你对webpack的理解？解决了什么问题？\)](https://github.com/linwu-hi/code-interview#面试官：说说你对Webpack的理解？解决了什么问题？)
- [面试官：说说如何借助webpack来优化前端性能？
\(https://github.com/linwu-hi/code-interview#面试官：说说如何借助webpack来优化前端性能？\)](https://github.com/linwu-hi/code-interview#面试官：说说如何借助webpack来优化前端性能？)
- [面试官：请你谈谈微信小程序的实现原理？
\(https://github.com/linwu-hi/code-interview#面试官：请你谈谈微信小程序的实现原理？\)](https://github.com/linwu-hi/code-interview#面试官：请你谈谈微信小程序的实现原理？)
- [面试官：说说微信小程序的生命周期函数有哪些？
\(https://github.com/linwu-hi/code-interview#面试官：说说微信小程序的生命周期函数有哪些？\)](https://github.com/linwu-hi/code-interview#面试官：说说微信小程序的生命周期函数有哪些？)
- [面试官：说说你对微信小程序的理解？优缺点？
\(https://github.com/linwu-hi/code-interview#面试官：说说你对微信小程序的理解？优缺点？\)](https://github.com/linwu-hi/code-interview#面试官：说说你对微信小程序的理解？优缺点？)
- [面试官：说说微信小程序中路由跳转的方式有哪些？区别？
\(https://github.com/linwu-hi/code-interview#面试官：说说微信小程序中路由跳转的方式有哪些？区别？\)](https://github.com/linwu-hi/code-interview#面试官：说说微信小程序中路由跳转的方式有哪些？区别？)
- [面试官：说说微信小程序的支付流程？
\(https://github.com/linwu-hi/code-interview#面试官：说说微信小程序的支付流程？\)](https://github.com/linwu-hi/code-interview#面试官：说说微信小程序的支付流程？)
- [面试官：说说微信小程序的登录流程？
\(https://github.com/linwu-hi/code-interview#面试官：说说微信小程序的登录流程？\)](https://github.com/linwu-hi/code-interview#面试官：说说微信小程序的登录流程？)
- [面试官：说说提高微信小程序的应用速度的手段有哪些？
\(https://github.com/linwu-hi/code-interview#面试官：说说提高微信小程序的应用速度的手段有哪些？\)](https://github.com/linwu-hi/code-interview#面试官：说说提高微信小程序的应用速度的手段有哪些？)
- [作为面试官，为什么我推荐组件库作为前端面试的亮点？
\(https://github.com/linwu-hi/code-interview#作为面试官，为什么我推荐组件库作为前端面试的亮点？\)](https://github.com/linwu-hi/code-interview#面试官：为什么我推荐组件库作为前端面试的亮点？)

面试官：谈谈你对BFC的理解？



一、BFC

在页面布局的过程中，我们经常会遇到一些奇怪的情况，比如元素的高度消失了、两栏布局无法自适应、元素间距出现异常等等。这些问题往往是由于元素之间相互影响而导致的。在这里，就涉及到了 BFC（Block Formatting Context）的概念。

BFC（块级格式化上下文）是页面中一块独立的渲染区域，具有一套独立的渲染规则：

- 内部的盒子会在垂直方向上一个接一个地放置。
- 同一个 BFC 的相邻盒子的 margin 会发生重叠，与方向无关。
- 每个元素的左外边距与包含块的左边界相接触（从左到右），即使浮动元素也是如此。
- BFC 的区域不会与 float 的元素区域重叠。
- 计算 BFC 的高度时，浮动子元素也参与计算。
- BFC 是页面上的一个隔离的独立容器，容器里面的子元素不会影响到外面的元素，反之亦然。

BFC 的目的是形成一个相对于外界完全独立的空间，使内部的子元素不会影响到外部的元素。

二、触发条件

触发 BFC 的条件包含但不限于：

- 根元素，即 HTML 元素
- 浮动元素：float 值为 left、right
- overflow 值不为 visible，为 auto、scroll、hidden
- display 的值为 inline-block、inline-table、table-cell、table-caption、flex、inline-flex、grid、inline-grid
- position 的值为 absolute 或 fixed

三、应用场景

利用 BFC 的特性，我们可以在以下场景中应用 BFC：

1. 防止margin重叠（塌陷）

```
<style>
  .container {
    overflow: hidden; /* 触发BFC */
  }
  p {
    color: #f55;
    background: #fcc;
    width: 200px;
    line-height: 100px;
    text-align: center;
    margin: 100px;
  }
</style>
<body>
  <div class="container">
    <p>Haha</p>
  </div>
  <p>Hehe</p>
</body>
```

在没有触发BFC的情况下，两个p元素之间的距离为100px，发生了margin重叠（塌陷）。通过给包含p元素的容器触发BFC，两个p元素不再属于同一个BFC，从而避免了margin重叠。

2. 清除内部浮动

```
<style>
  .parent {
    border: 5px solid #fcc;
    width: 300px;
  }

  .child {
    border: 5px solid #f66;
    width: 100px;
    height: 100px;
    float: left;
  }
</style>
<body>
  <div class="parent">
    <div class="child"></div>
    <div class="child"></div>
  </div>
</body>
```

通过触发parent元素生成BFC，可以清除内部浮动的影响，从而使parent元素具有高度。

```
.parent {
  overflow: hidden; /* 触发BFC */
}
```

3. 自适应多栏布局

```
<style>
  .wrapper {
    width: 300px;
  }

  .aside {
    width: 100px;
    height: 150px;
    float: left;
    background: #f66;
  }

  .main {
    height: 200px;
    background: #fcc;
  }
</style>
<body>
  <div class="wrapper">
    <div class="aside"></div>
    <div class="main"></div>
  </div>
</body>
```

在不触发BFC的情况下，.main元素的左边会与.wrapper元素的左边界相接触，从而无法实现两栏布局。

通过触发.main元素生成BFC，.main元素的左边不再与.wrapper元素的左边界相接触，从而实现自适应多栏布局。

```
.main {
  overflow: hidden; /* 触发BFC */
}
```

小结

通过上面的例子，我们可以看到BFC的实际效果就是形成一个页面中的独立容器，里面的子元素不影响外部的元素，反之亦然。BFC为我们解决了很多布局上的问题，提供了更灵活和可靠的布局手段。

参考文献

- [MDN Web Docs - Block Formatting Context\(https://developer.mozilla.org/zh-CN/docs/Web/Guide/CSS/Block_formatting_context\)](https://developer.mozilla.org/zh-CN/docs/Web/Guide/CSS/Block_formatting_context)
- [BFC原理剖析 - Github笔记\(https://github.com/zuopf769/notebook/blob/master/fe/BFC%E5%8E%9F%E7%90%8\)](https://github.com/zuopf769/notebook/blob/master/fe/BFC%E5%8E%9F%E7%90%8)

面试官：CSS如何画一个三角形？原理是什么？

在CSS中，我们可以通过一些技巧来绘制简单的形状，包括三角形。绘制三角形的常用方法有两种：利用边框和利用伪元素。

利用边框绘制三角形

绘制三角形最简单的方法是利用元素的边框。我们可以通过设置元素的宽度和高度为0，并设置对应的边框，然后隐藏不需要的边框，从而得到三角形的形状。

```
.triangle {  
  width: 0;  
  height: 0;  
  border-width: 20px;  
  border-style: solid;  
  border-color: transparent transparent red transparent;  
}
```

在这个例子中，我们设置了一个 `triangle` 类的元素，并且将其宽度和高度都设置为0。接着，通过 `border-width` 属性设置边框的宽度为20px，并设置边框的颜色。我们在这里选择了一个红色的边框，你可以根据需求设置不同的颜色。

`border-color` 属性的四个值分别表示四个边框的颜色，它们的顺序是：上、右、下、左。在这里，我们只让上边框显示出来，而其他三个边框设置为透明，从而形成一个三角形。

利用伪元素绘制三角形

除了利用边框，我们还可以利用伪元素来绘制三角形。伪元素可以在元素的内容前或内容后插入内容，利用这个特性，我们可以在元素的 `::before` 或 `::after` 伪元素中绘制三角形。

```
.triangle {  
  position: relative;  
  width: 0;  
  height: 0;  
}  
  
.triangle::before {  
  content: "";  
  position: absolute;  
  top: 0;  
  left: 0;  
  border-width: 20px;  
  border-style: solid;  
  border-color: transparent transparent red transparent;  
}
```

在这个例子中，我们同样创建了一个 `triangle` 类的元素，并将其宽度和高度都设置为0。然后，我们在 `::before` 伪元素中插入一个空内容，并将其设置为绝对定位，使得它可以覆盖在 `triangle` 元素的上方。

接下来，我们通过设置伪元素的边框属性来绘制三角形，方法与之前的方法相同，只显示上边框，并将其他三个边框设置为透明，形成三角形的形状。

原理

这两种方法的原理都是利用CSS的边框属性来绘制三角形。对于利用边框绘制三角形的方法，我们将元素的宽度和高度设置为0，然后利用边框的特性来实现三角形的形状。而对于利用伪元素绘制三角形的方法，我们在元素的 `::before` 或 `::after` 伪元素中插入一个空内容，并设置其边框属性来绘制三角形。

总结来说，绘制三角形的关键在于合理利用CSS的边框属性，通过设置不同边框的宽度和颜色，以及隐藏不需要的边框，来实现所需的形状。这样的技巧在实际开发中非常有用，能够减少额外的HTML标记和图像资源的使用，从而提高页面性能和加载速度。

面试官：css选择器有哪些？优先级？哪些属性可以继承？

一、选择器

CSS选择器是CSS规则的第一部分

它是元素和其他部分组合起来告诉浏览器哪个HTML元素应当是被选为应用规则中的CSS属性值的方式
选择器所选择的元素，叫做“选择器的对象”

我们从一个 `Html` 结构开始

```
<div id="box">
  <div class="one">
    <p class="one_1">
    </p>
    <p class="one_1">
    </p>
  </div>
  <div class="two"></div>
  <div class="two"></div>
  <div class="two"></div>
</div>
```

关于 `css` 属性选择器常用的有：

- id选择器（`#box`），选择id为box的元素
- 类选择器（`.one`），选择类名为one的所有元素
- 标签选择器（`div`），选择标签为div的所有元素
- 后代选择器（`#box div`），选择id为box元素内部所有的div元素
- 子选择器（`.one>one_1`），选择父元素为.one的所有.one_1的元素
- 相邻同胞选择器（`.one+.two`），选择紧接在.one之后的所有.two元素
- 群组选择器（`div,p`），选择div、p的所有元素

还有一些使用频率相对没那么多的选择器：

- 伪类选择器

:link：选择未被访问的链接
:visited：选取已被访问的链接
:active：选择活动链接
:hover：鼠标指针浮动在上面的元素
:focus：选择具有焦点的
:first-child：父元素的首个子元素

- 伪元素选择器

:first-letter：用于选取指定选择器的首字母
:first-line：选取指定选择器的首行
:before：选择器在被选元素的内容前面插入内容
:after：选择器在被选元素的内容后面插入内容

- 属性选择器

[attribute] 选择带有attribute属性的元素
[attribute=value] 选择所有使用attribute=value的元素
[attribute~value] 选择attribute属性包含value的元素
[attribute|=value]：选择attribute属性以value开头的元素

在 **CSS3** 中新增的选择器有如下：

- 层次选择器（**p~ul**），选择前面有p元素的每个ul元素
- 伪类选择器

:first-of-type 表示一组同级元素中其类型的第一个元素
:last-of-type 表示一组同级元素中其类型的最后一个元素
:only-of-type 表示没有同类型兄弟元素的元素
:only-child 表示没有任何兄弟的元素
:nth-child(n) 根据元素在一组同级中的位置匹配元素
:nth-last-of-type(n) 匹配给定类型的元素，基于它们在一组兄弟元素中的位置，从末尾开始计数
:last-child 表示一组兄弟元素中的最后一个元素
:root 设置**HTML**文档
:empty 指定空的元素
:enabled 选择可用元素
:disabled 选择被禁用元素
:checked 选择选中的元素
:not(selector) 选择与 **<selector>** 不匹配的所有元素

- 属性选择器

[attribute*=value]：选择attribute属性值包含value的所有元素
[attribute^=value]：选择attribute属性开头为value的所有元素
[attribute\$=value]：选择attribute属性结尾为value的所有元素

二、优先级

相信大家对 **CSS** 选择器的优先级都不陌生：

“

内联 > ID选择器 > 类选择器 > 标签选择器

”

到具体的计算层面，优先级是由 A 、 B、 C、 D 的值来决定的，其中它们的值计算规则如下：

- 如果存在内联样式，那么 $A = 1$ ，否则 $A = 0$
- B的值等于 ID选择器出现的次数
- C的值等于 类选择器 和 属性选择器 和 伪类 出现的总次数
- D 的值等于 标签选择器 和 伪元素 出现的总次数

这里举个例子：

```
#nav-global > ul > li > a.nav-link
```

套用上面的算法，依次求出 A B C D 的值：

- 因为没有内联样式，所以 $A = 0$
- ID选择器总共出现了1次， $B = 1$
- 类选择器出现了1次， 属性选择器出现了0次，伪类选择器出现0次，所以 $C = (1 + 0 + 0) = 1$
- 标签选择器出现了3次， 伪元素出现了0次，所以 $D = (3 + 0) = 3$

上面算出的 A 、 B、 C、 D 可以简记作：(0, 1, 1, 3)

知道了优先级是如何计算之后，就来看看比较规则：

- 从左往右依次进行比较，较大者优先级更高
- 如果相等，则继续往右移动一位进行比较
- 如果4位全部相等，则后面的会覆盖前面的

经过上面的优先级计算规则，我们知道内联样式的优先级最高，如果外部样式需要覆盖内联样式，就需要使用 `!important`

三、继承属性

在 `css` 中，继承是指的是给父元素设置一些属性，后代元素会自动拥有这些属性

关于继承属性，可以分成：

- 字体系列属性

```
font:组合字体
font-family:规定元素的字体系列
font-weight:设置字体的粗细
font-size:设置字体的尺寸
font-style:定义字体的风格
font-variant:偏大或偏小的字体
```

- 文本系列属性

text-indent: 文本缩进
text-align: 文本水平对刘
line-height: 行高
word-spacing: 增加或减少单词间的空白
letter-spacing: 增加或减少字符间的空白
text-transform: 控制文本大小写
direction: 规定文本的书写方向
color: 文本颜色

- 元素可见性

visibility

- 表格布局属性

caption-side: 定位表格标题位置
border-collapse: 合并表格边框
border-spacing: 设置相邻单元格的边框间的距离
empty-cells: 单元格的边框的出现与消失
table-layout: 表格的宽度由什么决定

- 列表属性

list-style-type: 文字前面的小点点样式
list-style-position: 小点点位置
list-style: 以上的属性可通过这属性集合

- 引用

quotes: 设置嵌套引用的引号类型

- 光标属性

cursor: 箭头可以变成需要的形状

继承中比较特殊的几点：

- a 标签的字体颜色不能被继承
- h1-h6标签字体的大小也是不能被继承的

无继承的属性

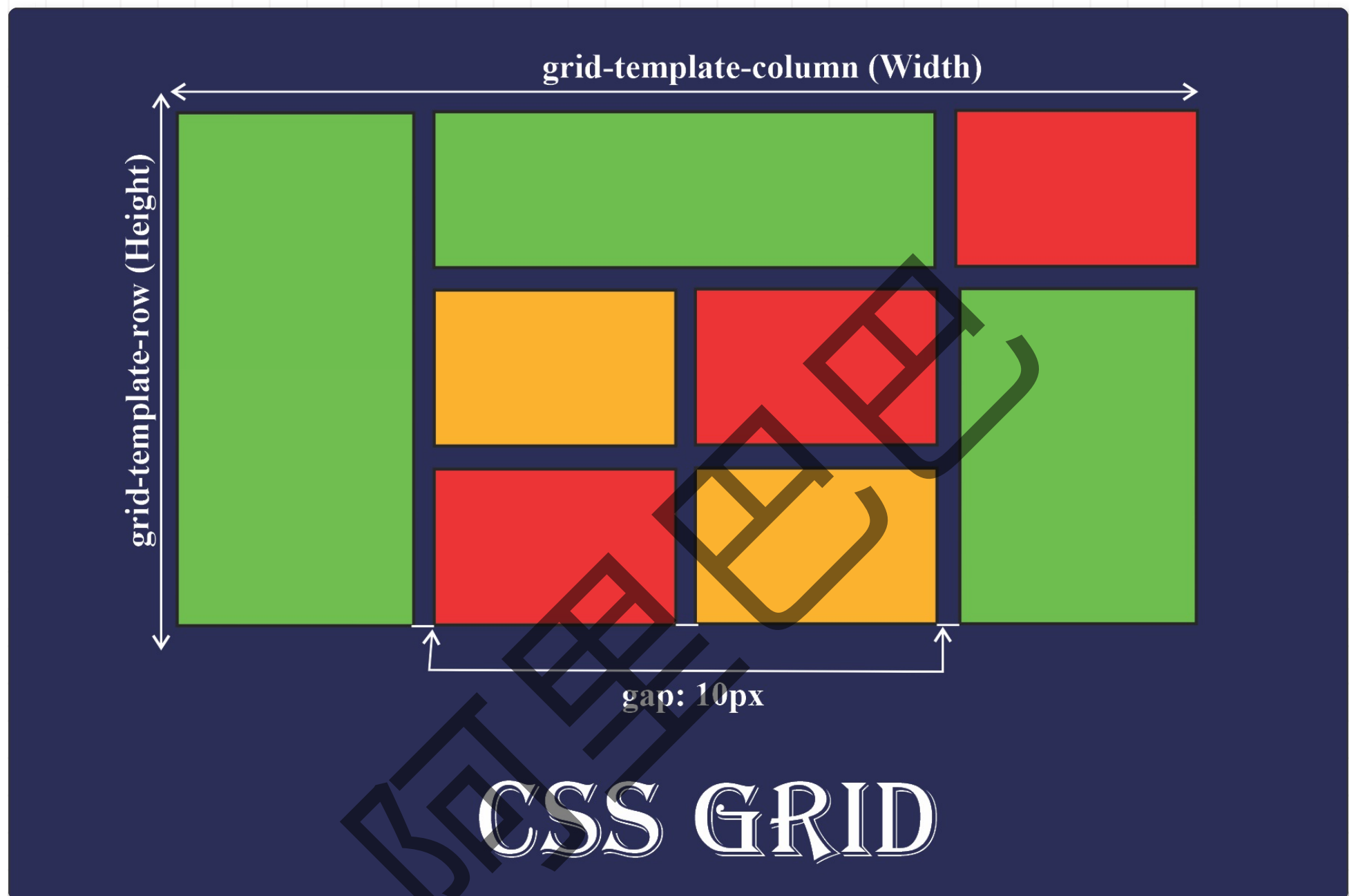
- display
- 文本属性：vertical-align、text-decoration
- 盒子模型的属性：宽度、高度、内外边距、边框等
- 背景属性：背景图片、颜色、位置等
- 定位属性：浮动、清除浮动、定位position等
- 生成内容属性：content、counter-reset、counter-increment

- 轮廓样式属性：outline-style、outline-width、outline-color、outline
- 页面样式属性：size、page-break-before、page-break-after

参考文献

- <https://www.html.cn/qa/css3/13444.html>
- https://developer.mozilla.org/zh-CN/docs/Learn/CSS/Building_blocks/Selectors

面试官：介绍一下grid网格布局



在现代Web开发中，页面布局是一个重要的任务，它决定了网页的结构和外观。在CSS中，我们通常使用Flexbox（弹性盒布局）和Grid布局来实现不同的布局需求。本文将重点介绍Grid布局，它是一种二维布局方式，通过纵横相交的网格线来创建布局结构，能够同时处理行和列，使得布局更加灵活多样。

一、什么是Grid布局？

Grid布局，又称为网格布局，是一种强大而灵活的CSS布局模型。它通过在容器中创建一个二维网格，将页面划分为多个主要区域，并定义这些区域的大小、位置、层次等关系。与Flexbox的一维布局不同，Grid布局是一个二维布局，它在排列元素时不仅考虑主轴方向（水平或垂直），还考虑交叉轴方向，使得布局更加精确和直观。

要使用Grid布局，我们需要将元素的 `display` 属性设置为 `grid` 或 `inline-grid`，这样浏览器渲染引擎会启动网格布局算法。


```
.container {  
  display: grid;  
  /* 或者 display: inline-grid; */  
}
```

二、Grid布局的属性

Grid布局的属性分为容器属性和项目属性两类，我们逐一介绍。

容器属性：

1. `grid-template-columns` 和 `grid-template-rows`：用于设置网格的列宽和行高。可以指定固定的宽度和高度，也可以使用 `repeat()` 函数简化重复的值。

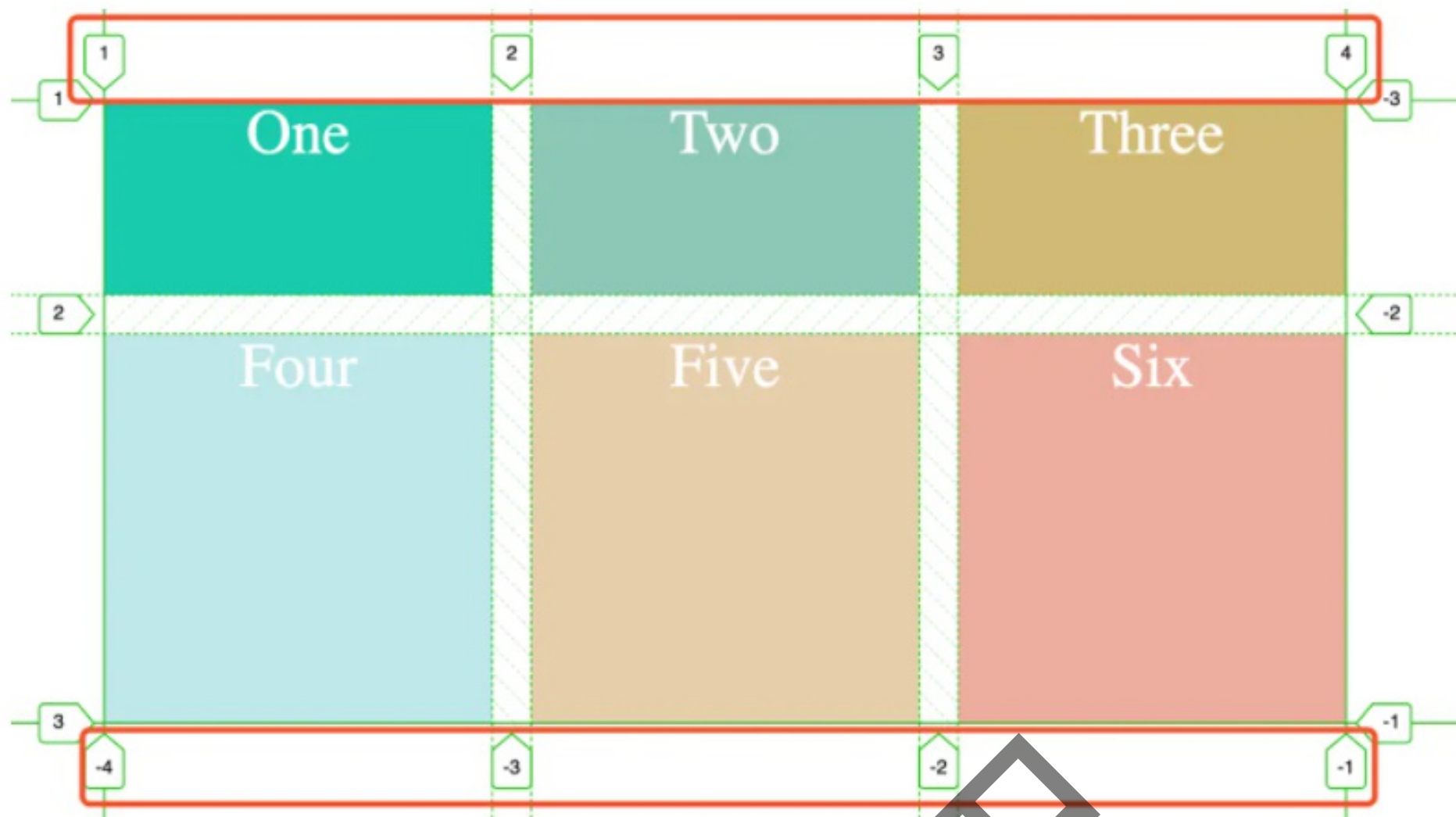
```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  grid-template-rows: 100px 100px;  
}
```

2. `grid-gap`：设置行间距和列间距的简写属性，相当于同时设置 `grid-row-gap` 和 `grid-column-gap`。

```
.container {  
  display: grid;  
  grid-gap: 10px 20px;  
}
```

3. `grid-template-areas`：用于定义网格区域，将多个单元格组成一个命名区域，通过字符串来表示。

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 100px 100px;  
  grid-template-areas:  
    'a b c'  
    'd e f';  
}
```



4. `grid-auto-flow`：用于指定项目的放置顺序，默认为 `row`，即先行后列。可以设置为 `column`，使项目先列后行排列。

```
.container {  
  display: grid;  
  grid-auto-flow: column;  
}
```

5. `justify-items` 和 `align-items`：分别用于设置单元格内容在水平和垂直方向上的对齐方式。

```
.item {  
  justify-items: center; /* left, right, stretch */  
  align-items: center; /* start, end, stretch */  
}
```

项目属性：

1. `grid-column-start` 和 `grid-column-end`：用于指定项目左边框和右边框所在的垂直网格线，定义项目在列方向的位置。

```
.item {  
  grid-column-start: 2;  
  grid-column-end: 4;  
}
```

2. `grid-row-start` 和 `grid-row-end`：用于指定项目上边框和下边框所在的水平网格线，定义项目在行方向的位置。

```
.item {  
  grid-row-start: 1;  
  grid-row-end: 3;  
}
```

3. `grid-area`：用于指定项目放在哪一个命名区域。

```
.item {  
  grid-area: e;  
}
```

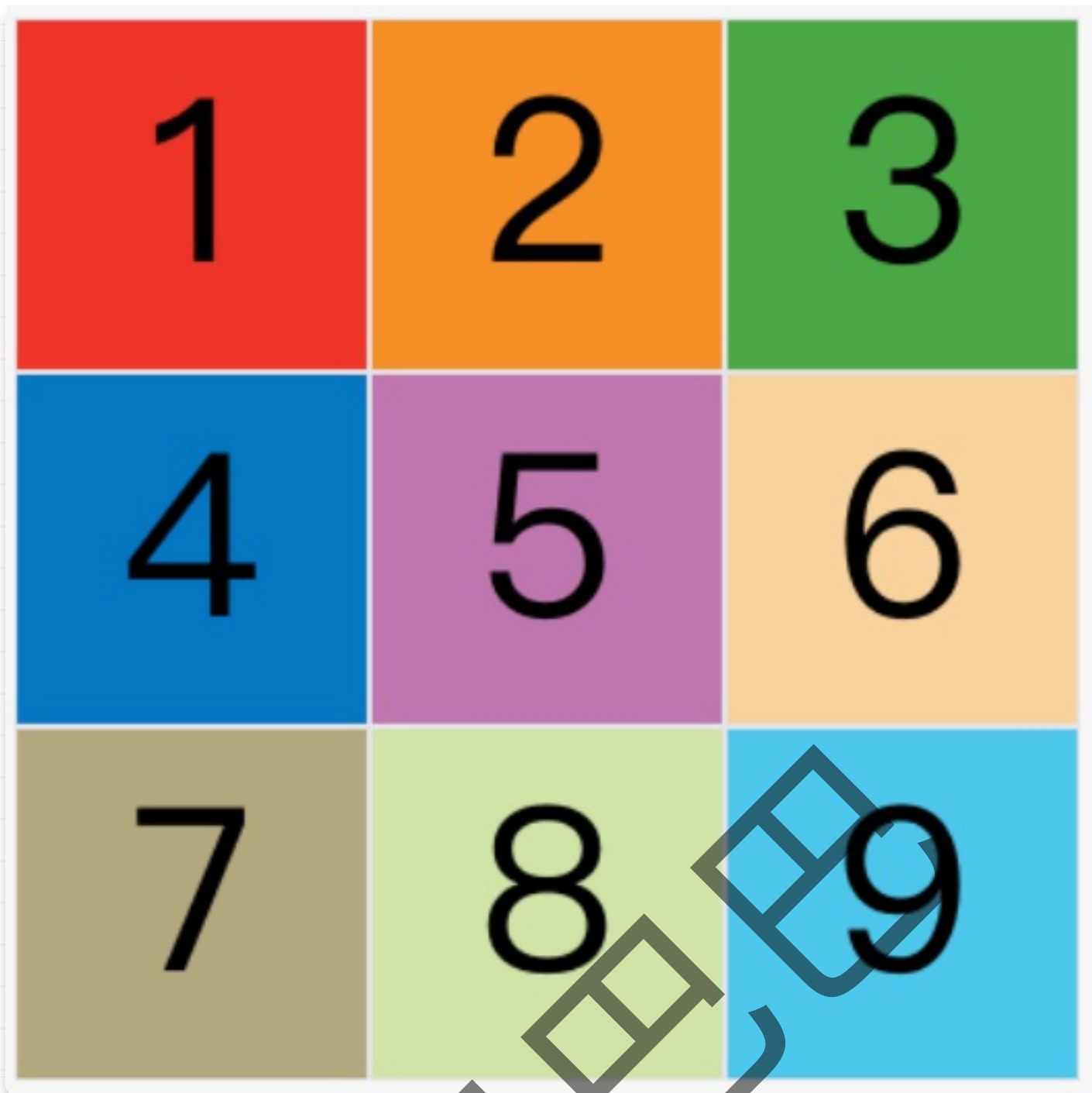
4. `justify-self` 和 `align-self`：分别用于设置单元格内容在水平和垂直方向上的对齐方式，只作用于单个项目。

```
.item {  
  justify-self: center; /* left, right, stretch */  
  align-self: center; /* start, end, stretch */  
}
```

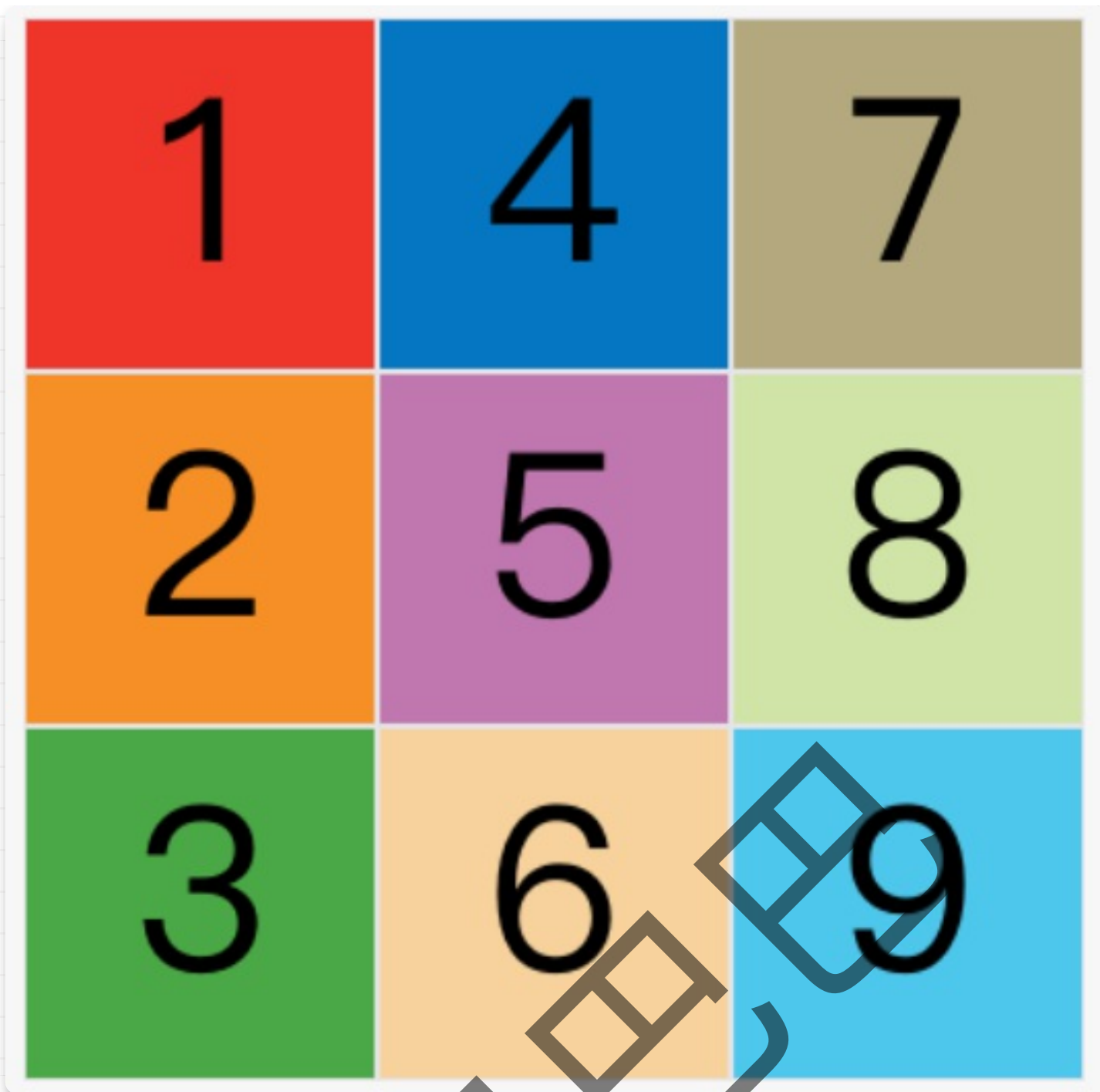
三、Grid布局的应用场景

Grid布局在实际开发中有着广泛的应用场景，以下是一些常见的应用示例：

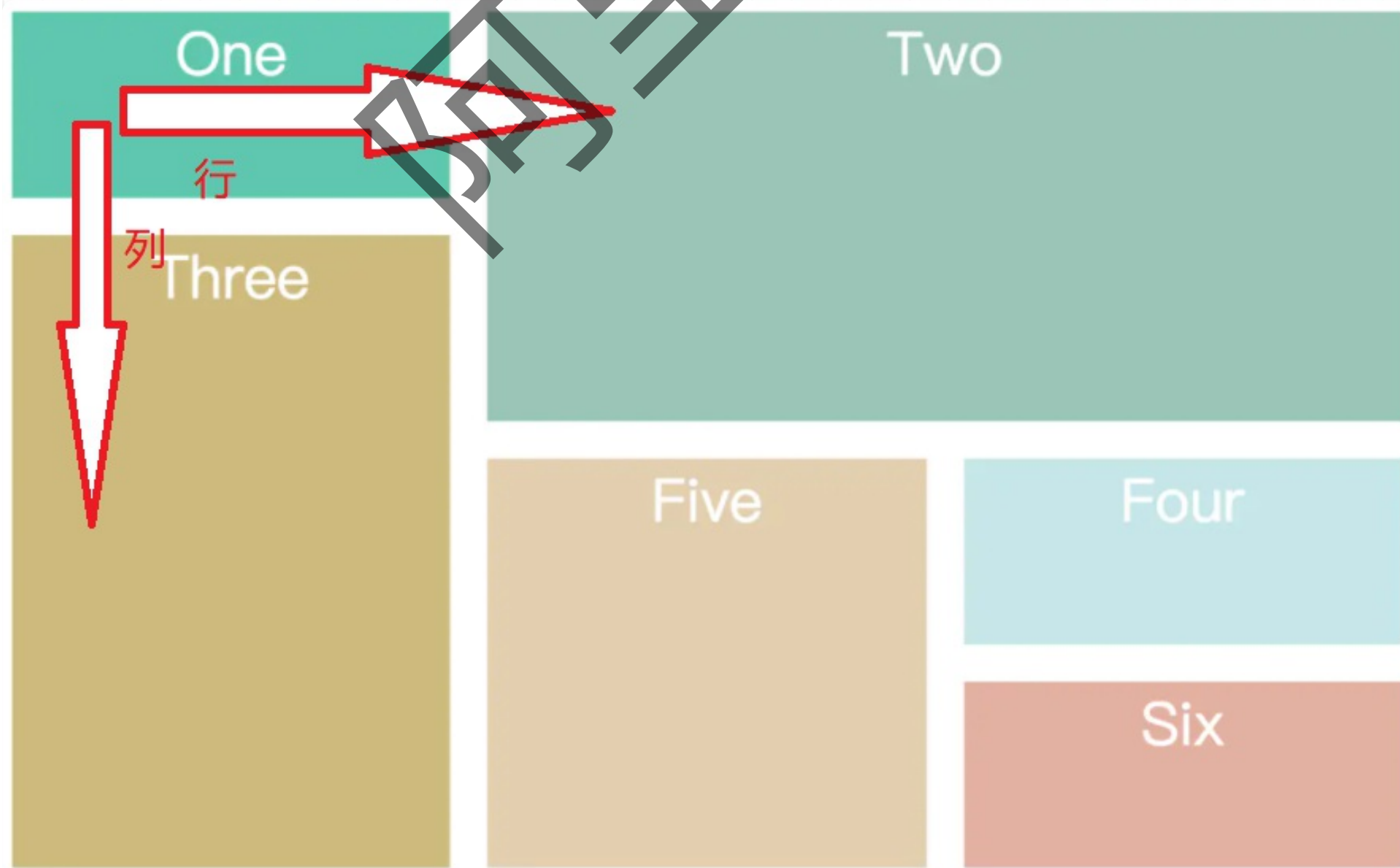
1. 网格导航：通过网格布局可以轻松实现水平或垂直的网格导航菜单，使得导航链接整齐排列。



- 居中布局：通过设置容器的对齐方式和项目的自身对齐方式，可以实现内容在容器中的水平和垂直居中，使得布局更加美观。



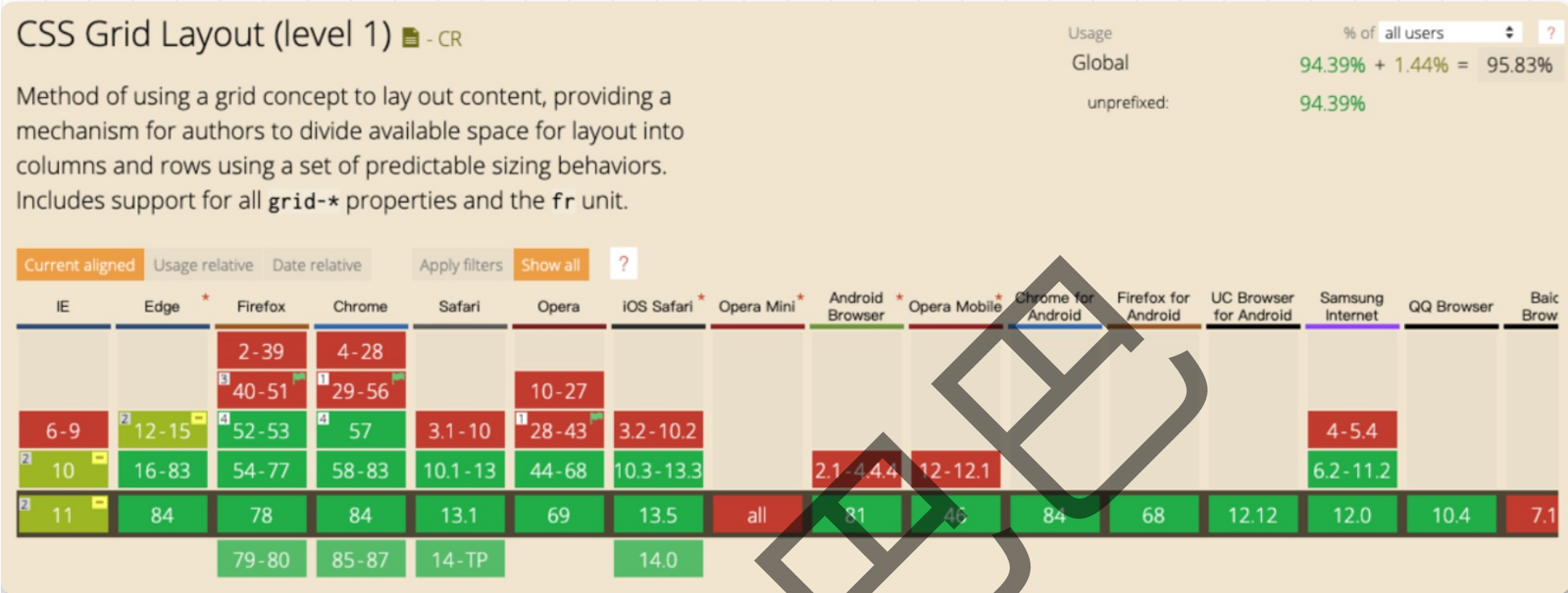
3. 多栏布局：Grid布局可以轻松实现多栏布局，使得栏目的宽度相等或不等，适用于新闻列表、产品展示等场景。



- 4. 自适应布局：通过设置项目的放大和缩小比例，可以实现根据容器大小自适应调整布局，使得页面在不同屏幕尺寸下都能良好展示。
- 5. 等高布局：通过设置项目的高度为 1fr，可以使得多个项目拥有相同的高度，适用于类似产品卡片的布局。
- 6. 响应式布局：在移动设备和桌面端的响应式布局中，Grid布局表现优秀，可以在不同尺寸的屏幕上显示不同的布局，提升用户体验。

四、兼容性和使用建议

目前，Grid布局在现代主流浏览器中具有良好的兼容性，支持情况如下：



综上所述，Grid布局是一种功能强大、灵活多样的布局方式，它在Web开发中有着广泛的应用场景。在实际开发中，我们可以灵活运用Grid布局来实现各种复杂的页面布局需求，提升网页的可读性和用户体验。然而，考虑到兼容性问题，建议在使用Grid布局时，及时做好兼容性处理，以确保网页在不同浏览器中都能正确显示和运行。希望本文能帮助读者深入理解Grid布局，并在实际项目中应用得心应手！

面试官：元素水平垂直居中的方法有哪些？如果元素不定宽高呢？

一、背景

在开发中经常遇到这个问题，即让某个元素的内容在水平和垂直方向上都居中，内容不仅限于文字，可能是图片或其他元素

居中是一个非常基础但又是非常重要的应用场景，实现居中的方法存在很多，可以将这些方法分成两大类：

- 居中元素（子元素）的宽高已知
- 居中元素宽高未知

二、实现方式

实现元素水平垂直居中的方式：

- 利用定位+margin:auto
- 利用定位+margin:负值
- 利用定位+transform

- table布局
- flex布局
- grid布局

利用定位+margin:auto

先上代码：

```
<style>
.father{
  width:500px;
  height:300px;
  border:1px solid #0a3b98;
  position: relative;
}
.son{
  width:100px;
  height:40px;
  background: #f0a238;
  position: absolute;
  top:0;
  left:0;
  right:0;
  bottom:0;
  margin:auto;
}
</style>
<div class="father">
  <div class="son"></div>
</div>
```

父级设置为相对定位，子级绝对定位，并且四个定位属性的值都设置了0，那么这时候如果子级没有设置宽高，则会被拉开到和父级一样宽高

这里子元素设置了宽高，所以宽高会按照我们的设置来显示，但是实际上子级的虚拟占位已经撑满了整个父级，这时候再给它一个 `margin: auto` 它就可以上下左右都居中了

利用定位+margin:负值

绝大多数情况下，设置父元素为相对定位，子元素移动自身50%实现水平垂直居中


```
<style>
  .father {
    position: relative;
    width: 200px;
    height: 200px;
    background: skyblue;
  }
  .son {
    position: absolute;
    top: 50%;
    left: 50%;
    margin-left: -50px;
    margin-top: -50px;
    width: 100px;
    height: 100px;
    background: red;
  }
</style>
<div class="father">
  <div class="son"></div>
</div>
```

- 初始位置为方块1的位置
- 当设置left、top为50%的时候，内部子元素为方块2的位置
- 设置margin为负数时，使内部子元素到方块3的位置，即中间位置

这种方案不要求父元素的高度，也就是即使父元素的高度变化了，仍然可以保持在父元素的垂直居中位置，水平方向上是一样的操作

但是该方案需要知道子元素自身的宽高，但是我们可以通过下面 `transform` 属性进行移动

利用定位+transform

实现代码如下：

```
<style>
  .father {
    position: relative;
    width: 200px;
    height: 200px;
    background: skyblue;
  }
  .son {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%,-50%);
    width: 100px;
    height: 100px;
    background: red;
  }
</style>
<div class="father">
  <div class="son"></div>
</div>
```

`translate(-50%, -50%)`将会将元素位移自己宽度和高度的-50%

这种方法其实和最上面被否定掉的margin负值用法一样，可以说是margin负值的替代方案，并不需要知道自身元素的宽高

table布局

设置父元素为 `display: table-cell`，子元素设置 `display: inline-block`。利用 `vertical` 和 `text-align` 可以让所有的行内块级元素水平垂直居中

```
<style>
  .father {
    display: table-cell;
    width: 200px;
    height: 200px;
    background: skyblue;
    vertical-align: middle;
    text-align: center;
  }
  .son {
    display: inline-block;
    width: 100px;
    height: 100px;
    background: red;
  }
</style>
<div class="father">
  <div class="son"></div>
</div>
```

flex弹性布局

还是看看实现的整体代码：

```
<style>
  .father {
    display: flex;
    justify-content: center;
    align-items: center;
    width: 200px;
    height: 200px;
    background: skyblue;
  }
  .son {
    width: 100px;
    height: 100px;
    background: red;
  }
</style>
<div class="father">
  <div class="son"></div>
</div>
```

css3 中了 flex 布局，可以非常简单实现垂直水平居中

这里可以简单看看 flex 布局的关键属性作用：

- display: flex 时，表示该容器内部的元素将按照 flex 进行布局
- align-items: center 表示这些元素将相对于本容器水平居中
- justify-content: center 也是同样的道理垂直居中

grid 网格布局

```
<style>
  .father {
    display: grid;
    align-items: center;
    justify-content: center;
    width: 200px;
    height: 200px;
    background: skyblue;
  }
  .son {
    width: 10px;
    height: 10px;
    border: 1px solid red;
  }
</style>
<div class="father">
  <div class="son"></div>
</div>
```

这里看到，grid 网格布局和 flex 弹性布局都简单粗暴

小结

上述方法中，不知道元素宽高大小仍能实现水平垂直居中的方法有：

- 利用定位+margin:auto
- 利用定位+transform
- flex布局
- grid布局

三、总结

根据元素标签的性质，可以分为：

- 内联元素居中布局
- 块级元素居中布局

内联元素居中布局

水平居中

- 行内元素可设置：text-align: center
- flex布局设置父元素：display: flex; justify-content: center

垂直居中

- 单行文本父元素确认高度：height === line-height
- 多行文本父元素确认高度：display: table-cell; vertical-align: middle

块级元素居中布局

水平居中

- 定宽：margin: 0 auto
- 绝对定位+left:50%+margin:负自身一半

垂直居中

- position: absolute设置left、top、margin-left、margin-top(定高)
- display: table-cell
- transform: translate(x, y)
- flex(不定高，不定宽)
- grid(不定高，不定宽)，兼容性相对较差

参考文献

- <https://juejin.cn/post/6844903982960214029#heading-10>

面试官：如何实现单行 / 多行文本溢出的省略样式？

“

实现单行或多行文本溢出时的省略样式是前端开发中常见的需求。CSS提供了 `text-overflow` 属性来实现单行文本溢出的省略样式，同时结合 `white-space` 和 `overflow` 属性，可以实现多行文本溢出的省略样式。

”

单行文本溢出省略样式

实现单行文本的省略样式需要用到以下三个属性：

1. `white-space`：用于控制元素内文本的换行方式。
2. `overflow`：用于控制元素内容溢出时的处理方式。
3. `text-overflow`：用于控制文本溢出时的省略样式。

```
.single-line-ellipsis {  
  white-space: nowrap; /* 文本不换行 */  
  overflow: hidden; /* 内容溢出时隐藏超出部分 */  
  text-overflow: ellipsis; /* 溢出部分用省略号代替 */  
}
```

在上面的例子中，我们创建了一个类名为 `.single-line-ellipsis` 的样式。设置 `white-space` 属性为 `nowrap`，表示文本不进行换行。设置 `overflow` 属性为 `hidden`，当文本内容超出容器时，隐藏超出部分。最后，设置 `text-overflow` 属性为 `ellipsis`，表示溢出部分用省略号（...）代替。

多行文本溢出省略样式

实现多行文本的省略样式需要用到以下三个属性：

1. `display`：用于控制元素的显示方式，必须设置为 `-webkit-box` 或 `-webkit-inline-box`，使得 `-webkit-line-clamp` 属性生效。
2. `-webkit-box-orient`：用于设置元素内部的排列方向，值为 `vertical` 表示垂直排列。
3. `-webkit-line-clamp`：用于设置显示的行数，超出行数的文本将被省略。

```
.multi-line-ellipsis {  
  display: -webkit-box;  
  -webkit-box-orient: vertical;  
  -webkit-line-clamp: 3; /* 显示的行数 */  
  overflow: hidden; /* 内容溢出时隐藏超出部分 */  
}
```

在上面的例子中，我们创建了一个类名为 `.multi-line-ellipsis` 的样式。通过设置 `display` 为 `-webkit-box`，并将 `-webkit-box-orient` 设置为 `vertical`，实现多行文本的垂直排列。然后，设置 `-webkit-line-clamp` 为希望显示的行数（例如 3），超出的文本将被省略。最后，设置 `overflow` 属性为 `hidden`，当文本内容超出容器时，隐藏超出部分。

需要注意的是，`-webkit-line-clamp` 是一个私有属性，目前主要用于WebKit浏览器（例如Safari和Chrome）。在其他浏览器中可能不被支持。因此，在使用多行省略样式时，建议考虑使用JavaScript等其他方案实现跨浏览器的兼容性。

面试官：如果要做优化，CSS提高性能的方法有哪些？

一、前言

每一个网页都离不开CSS，但是很多人又认为CSS主要是用来完成页面布局的，像一些细节或者优化，就不需要怎么考虑，实际上这种想法是不正确的。

作为页面渲染和内容展现的重要环节，CSS影响着用户对整个网站的第一体验。因此，在整个产品研发过程中，CSS性能优化同样需要贯穿全程。

二、实现方式

实现方式有很多种，主要有如下：

1. 内联首屏关键CSS

在打开一个页面时，页面首要内容出现在屏幕的时间影响着用户的体验，而通过内联CSS关键代码能够使浏览器在下载完HTML后就能立刻渲染。

而如果外部引用CSS代码，在解析HTML结构过程中遇到外部CSS文件，才会开始下载CSS代码，再渲染。

所以，CSS内联使用使渲染时间提前。

注意：但是较大的CSS代码并不合适内联（初始拥塞窗口、没有缓存），而其余代码则采取外部引用方式。

2. 异步加载CSS

在CSS文件请求、下载、解析完成之前，CSS会阻塞渲染，浏览器将不会渲染任何已处理的内容。

前面加载内联代码后，后面的外部引用CSS则没必要阻塞浏览器渲染。这时候就可以采取异步加载的方案，主要有如下：

- 使用JavaScript将link标签插到head标签最后

```
// 创建link标签
const myCSS = document.createElement("link");
myCSS.rel = "stylesheet";
myCSS.href = "mystyles.css";
// 插入到header的最后位置
document.head.insertBefore(myCSS,
document.head.childNodes[document.head.childNodes.length - 1].nextSibling);
```

- 设置link标签media属性为noexis，浏览器会认为当前样式表不适用当前类型，会在不阻塞页面渲染的情况下再进行下载。加载完成后，将media的值设为screen或all，从而让浏览器开始解析CSS

```
<link rel="stylesheet" href="mystyles.css" media="noexist" onload="this.media='all'">
```

- 通过rel属性将link元素标记为alternate可选样式表，也能实现浏览器异步加载。同样别忘了加载完成之后，将rel设回stylesheet

```
<link rel="alternate stylesheet" href="mystyles.css" onload="this.rel='stylesheet'">
```

3. 资源压缩

利用webpack、gulp/grunt、rollup等模块化工具，将CSS代码进行压缩，使文件变小，大大降低了浏览器的加载时间。

4. 合理使用选择器

CSS匹配的规则是从右往左开始匹配，例如 `#markdown .content h3` 匹配规则如下：

- 先找到h3标签元素
- 然后去除祖先不是.content的元素
- 最后去除祖先不是#markdown的元素

如果嵌套的层级更多，页面中的元素更多，那么匹配所要花费的时间代价自然更高。

所以我们在编写选择器的时候，可以遵循以下规则：

- 不要嵌套使用过多复杂选择器，最好不要三层以上
- 使用id选择器就没必要再进行嵌套
- 通配符和属性选择器效率最低，避免使用

5. 减少使用昂贵的属性

在页面发生重绘的时候，昂贵属性如 `box-shadow` / `border-radius` / `filter` / 透明度 / `:nth-child` 等，会降低浏览器的渲染性能。

6. 不要使用@import

CSS样式文件有两种引入方式，一种是 `link` 元素，另一种是 `@import`。

`@import` 会影响浏览器的并行下载，使得页面在加载时增加额外的延迟，增添了额外的往返耗时。

而且多个 `@import` 可能会导致下载顺序紊乱。

比如一个CSS文件 `index.css` 包含了以下内容：`@import url("reset.css")`。

那么浏览器就必须先把 `index.css` 下载、解析和执行后，才下载、解析和执行第二个文件 `reset.css`。

其他

- 减少重排操作，以及减少不必要的重绘
- 了解哪些属性可以继承而来，避免对这些属性重复编写
- CSS Sprite，合成所有icon图片，用宽高加上 `background-position` 的背景图方式显现出我们要的icon图，减少了HTTP请求
- 把小的icon图片转成Base64编码
- CSS3动画或者过渡尽量使用 `transform` 和 `opacity` 来实现动画，不要使用 `left` 和 `top` 属性

三、总结

CSS实现性能的方式可以从选择器嵌套、属性特性、减少HTTP这三面考虑，同时还要注意CSS代码的加载顺序。

参考文献

- 知乎 - 如何优化 CSS 性能? (<https://www.zhihu.com/question/19886806>)
- 掘金 - CSS性能优化，你真的了解吗? (<https://juejin.cn/post/6844903649605320711#heading-1>)

面试官：深入理解设备像素、CSS像素、设备独立像素、DPR、PPI之间的区别与适配方案

在前端开发中，经常会遇到关于像素的概念，如设备像素、CSS像素、设备独立像素、DPR（设备像素比）以及PPI（每英寸像素）等。这些概念之间的区别很重要，尤其在移动设备上开发时，需要进行适配，以确保页面在不同设备上显示良好。本文将深入探讨这些概念之间的区别，并介绍适配方案。

一、设备像素

设备像素（Device Pixel），也称为物理像素，是显示设备上的最小物理单位。它代表屏幕上的一个点，能够显示不同颜色和图像。设备像素是显示屏的固有属性，不可改变，每台设备在制造时都确定了其设备像素数量。

在高分辨率的显示屏上，设备像素更加密集，显示效果更加细腻。例如，若两台手机屏幕尺寸相同，但一台设备像素更多，则其显示效果更加清晰。

二、CSS像素

CSS像素（CSS Pixel, px）是Web开发中常用的长度单位。在CSS规范中，长度单位分为绝对单位和相对单位，而CSS像素是一种相对单位。大多数情况下，1个CSS像素对应1个设备独立像素，即1:1的关系。

然而，有时候1个CSS像素可能对应多个设备像素，这取决于设备的DPR（设备像素比）。

三、设备独立像素

设备独立像素（Device Independent Pixel），也称为逻辑像素或密度无关像素，是一个与设备无关的抽象单位。它是开发者可以通过程序控制使用的虚拟像素，是为了解决不同设备分辨率不同的问题。

设备独立像素可以通过 `window.screen.width` 和 `window.screen.height` 来获取。而设备像素比（DPR）可以通过 `window.devicePixelRatio` 来获取。

设备独立像素的引入使得开发者在不同设备上进行页面设计时更加灵活，不用考虑具体的物理像素数目，统一使用设备独立像素进行布局。

四、DPR（设备像素比）

DPR（Device Pixel Ratio）是设备像素和设备独立像素之间的转换关系。它表示1个设备独立像素由多少个设备像素组成。

计算公式如下：

$$DPR = \text{设备像素} / \text{设备独立像素}$$

举例来说，当DPR为2时，1个设备独立像素由2个设备像素组成；当DPR为3时，1个设备独立像素由3个设备像素组成。

DPR通常用于移动设备，不同的移动设备可能有不同的DPR，用于适配不同分辨率的屏幕。

五、PPI（每英寸像素）

PPI（Pixel Per Inch），也称为每英寸像素，表示屏幕每英寸所包含的像素点数目，也可以理解为屏幕的像素密度。PPI的数值越高，屏幕的显示越清晰。

PPI是一个指标，用来描述屏幕像素的密度，并与屏幕的尺寸相关。例如，同样尺寸的屏幕，PPI越高，说明屏幕像素更密集，显示效果更细腻。

六、适配方案

在移动设备开发中，为了确保页面在不同设备上显示良好，我们需要考虑设备的DPR和PPI，以及设备独立像素。以下是一些常用的适配方案：

1. 使用rem单位：rem单位是相对于根元素（html元素）的字体大小进行计算的长度单位。通过动态设置根元素的字体大小，可以根据不同设备的DPR进行适配。
2. 使用viewport标签：通过设置viewport标签，可以控制页面的缩放和布局。设置 `<meta name="viewport" content="width=device-width, initial-scale=1.0">` 可以让页面宽度等于设备宽度，并且不进行缩放。
3. 使用媒体查询：媒体查询可以根据不同设备的宽度、高度、DPR等条件来应用不同的CSS样式，从而实现页面的适配。
4. 使用flexbox和grid布局：flexbox和grid布局可以更加灵活地进行页面布局，适应不同设备的屏幕尺寸和分辨率。
5. 使用图片的@2x和@3x版本：对于高分辨率设备，提供相应的高清晰度图片，以确保图片显示效果清晰。

七、总结

- 设备像素是显示设备上的最小物理单位，代表一个点，无法通过代码改变。
- CSS像素是Web开发中常用的长度单位，大多数情况下1个CSS像素对应1个设备独立像素。

- 设备独立像素是与设备无关的抽象单位，通过程序控制使用，用于解决不同设备分辨率问题。
- DPR（设备像素比）表示1个设备独立像素由多少个设备像素组成，用于移动设备适配不同分辨率的屏幕。
- PPI（每英寸像素）是屏幕每英寸所包含的像素点数目，与屏幕的尺寸相关。

在移动设备开发中，我们需要考虑设备的DPR和PPI，以及设备独立像素，采取相应的适配方案，以确保页面在不同设备上显示效果一致和清晰。同时，灵活运用CSS单位和布局技巧，可以更好地适应不同设备的屏幕尺寸和分辨率。

面试官：说说flexbox（弹性盒布局模型）

Flex布局(Flexible Box Layout)是一种用于定义容器内部元素布局方式的CSS3布局模型。它可以使元素具有弹性,让元素在容器内自动排列,简化了元素的定位。

Flex布局的基本概念

采用Flex布局的元素可分为容器(container)和项目(item)。

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

容器通过设置 `display: flex` 生成,项目默认沿主轴排列。

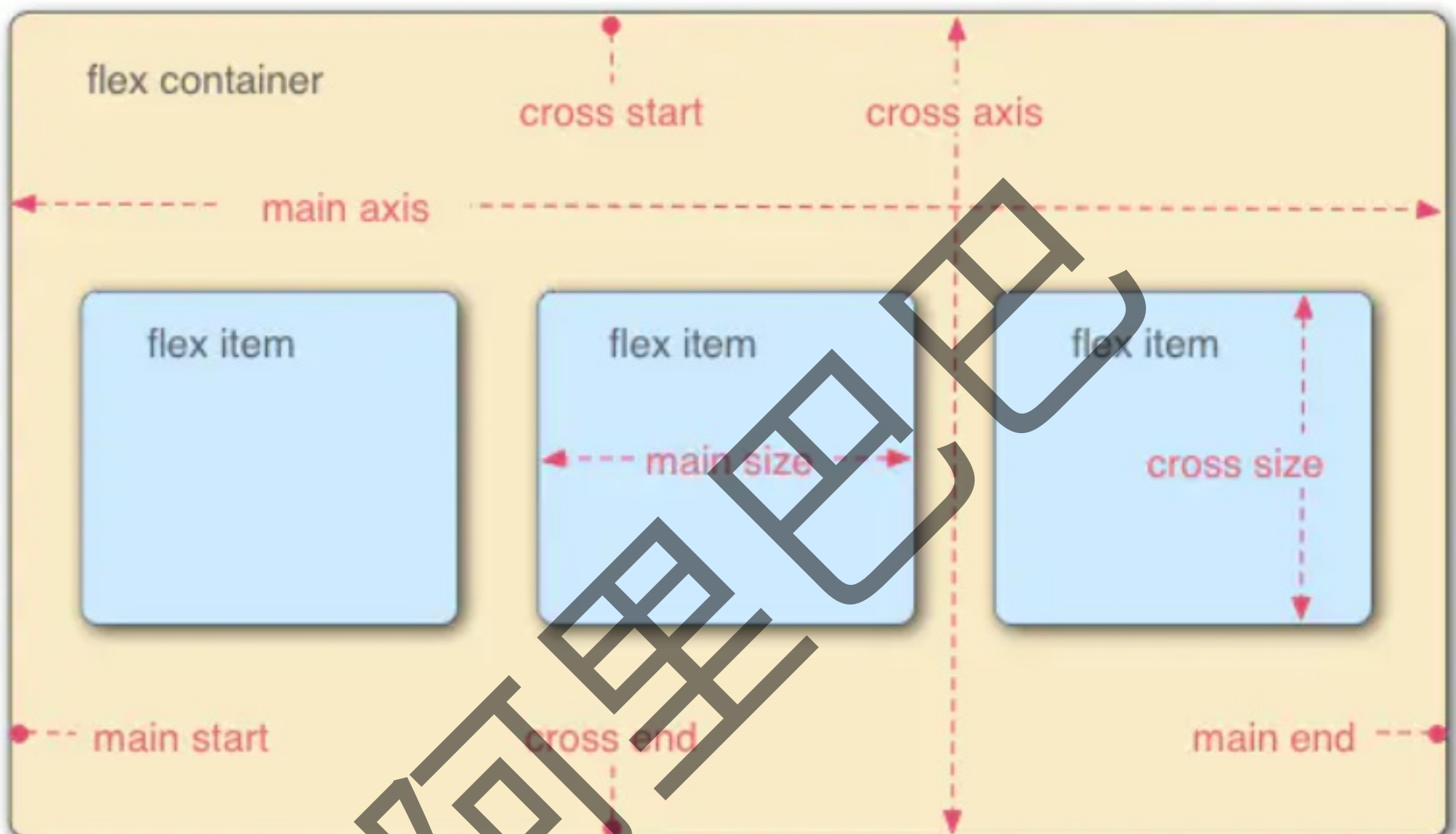
容器属性

flex-direction

决定主轴方向,即项目排列方向。

```
.container {  
  flex-direction: row | row-reverse | column | column-reverse;  
}
```

- row: 主轴为水平, 起点在左端
- row-reverse: 主轴为水平, 起点在右端
- column: 主轴为垂直, 起点在上沿
- column-reverse: 主轴为垂直, 起点在下沿



flex-wrap

定义容器内项目是否可换行。

```
.container {  
  flex-wrap: nowrap | wrap | wrap-reverse;  
}
```

- nowrap: 不换行
- wrap: 换行, 第一行在下方
- wrap-reverse: 换行, 第一行在上方

justify-content

定义项目在主轴上的对齐方式。

```
.container {  
  justify-content: flex-start | flex-end | center | space-between | space-around;  
}
```

- flex-start: 左对齐
- flex-end: 右对齐
- center: 居中
- space-between: 两端对齐, 项目之间间隔相等
- space-around: 每个项目两侧间隔相等

flex-start



flex-end



center



space-between



space-around

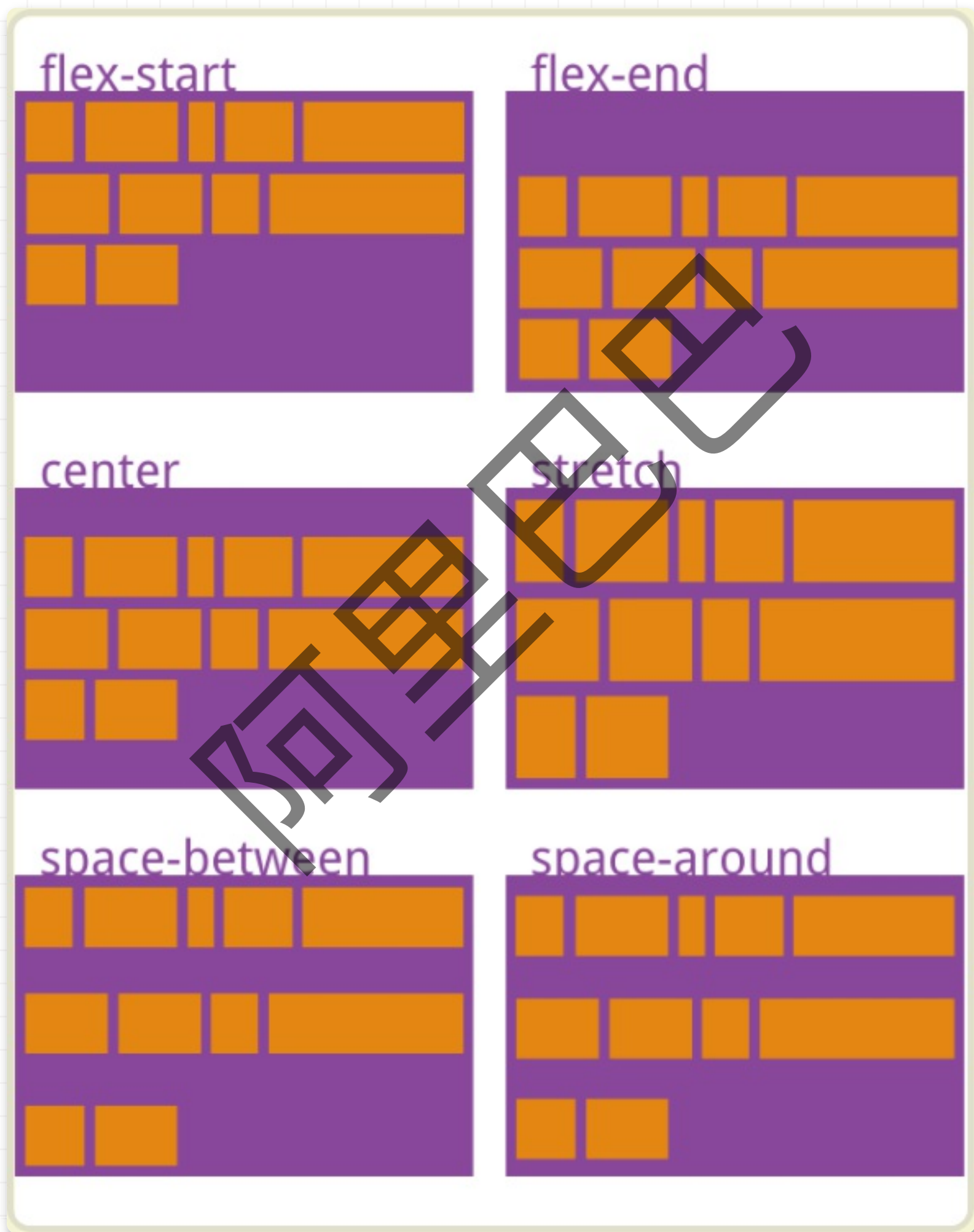


align-items

定义项目在交叉轴上如何对齐。

```
.container {  
  align-items: flex-start | flex-end | center | baseline | stretch;  
}
```

- flex-start:交叉轴起点对齐
- flex-end:交叉轴终点对齐
- center:交叉轴中点对齐
- baseline:项目第一行文字基线对齐
- stretch:项目高度撑满容器



项目属性

order

定义项目的排列顺序。

```
.item {  
  order: <integer>;  
}
```

数值越小,排列越靠前。

flex-grow

定义项目的放大比例。

```
.item {  
  flex-grow: <number>; /* default 0 */  
}
```

如果所有项目都为1,则它们将等分剩余空间。

flex-shrink

定义了项目的缩小比例。

```
.item {  
  flex-shrink: <number>; /* default 1 */  
}
```

如果空间不足,该项目将按比例缩小。

align-self

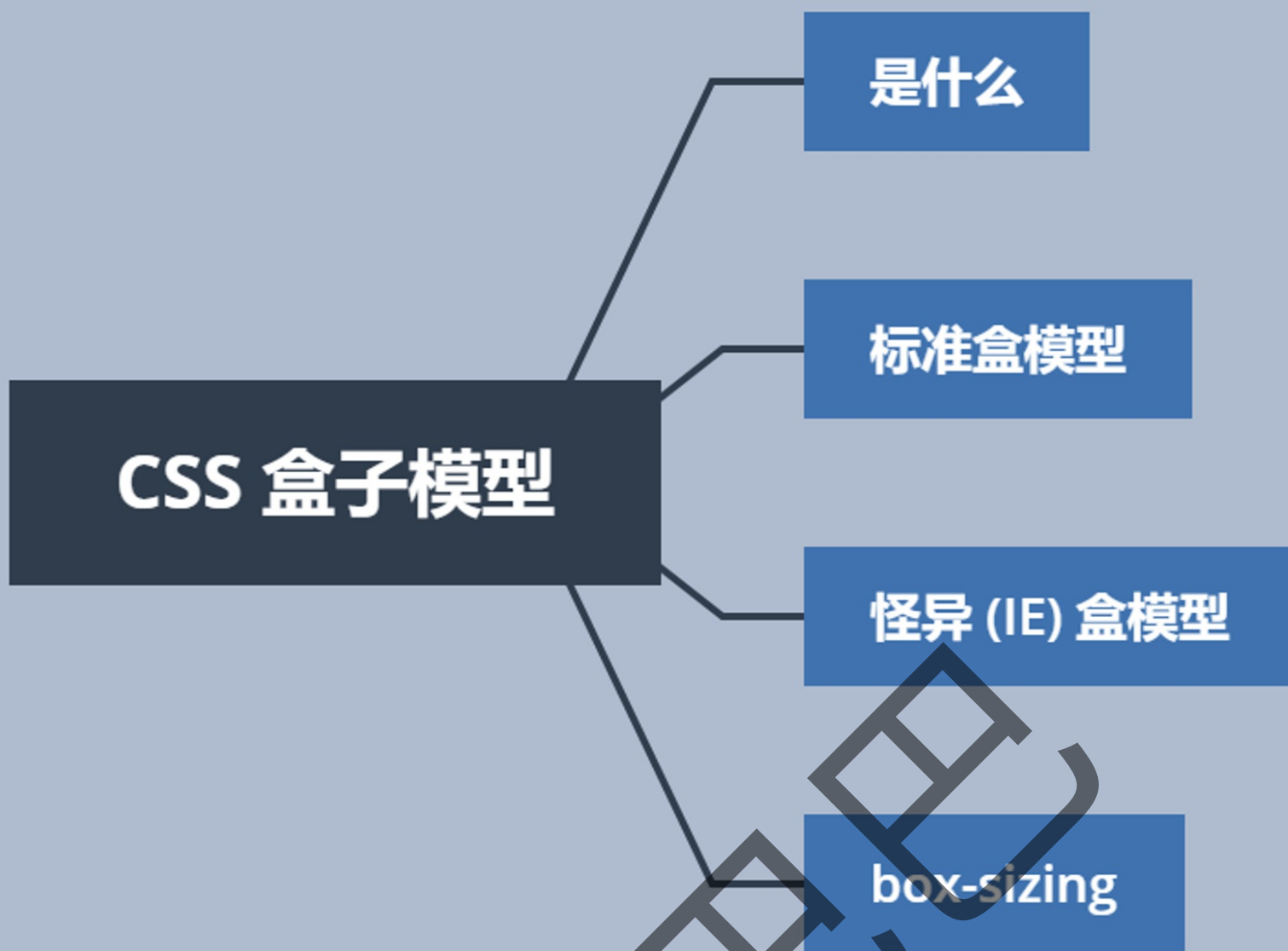
单个项目与其他项目对齐方式不同。

```
.item {  
  align-self: auto | flex-start | flex-end | center | baseline | stretch;  
}
```

总结

Flex布局提供了强大的容器属性,可以方便地控制项目布局、对齐和顺序,是实现响应式布局的首选方式。

面试官： 深入理解CSS盒子模型



一、引言

在前端开发中，了解CSS盒子模型是非常重要的基础知识。盒子模型描述了网页中的每个元素都是由一个矩形的盒子组成，这个盒子由内容、内边距、边框和外边距组成。正确理解盒子模型对于设计合理的布局和样式至关重要。让我们深入了解CSS盒子模型，理解其中的概念和计算方式。

二、盒子模型概述

1. 基本组成部分

CSS盒子模型由以下四个部分组成：

- **Content**：盒子内部的实际内容，例如文本、图像等。它的大小由 `width` 和 `height` 属性决定。
- **Padding**：围绕 **Content** 的内边距，用于控制内容与边框之间的空间。它的大小由 `padding-top`、`padding-right`、`padding-bottom` 和 `padding-left` 属性决定。
- **Border**：位于 **Padding** 之外的边框线，用于包围盒子。它的大小由 `border-width`、`border-style` 和 `border-color` 属性决定。
- **Margin**：位于 **Border** 之外的外边距，用于控制盒子与其他元素之间的空间。它的大小由 `margin-top`、`margin-right`、`margin-bottom` 和 `margin-left` 属性决定。

下图显示了盒子模型的基本结构：

2. 宽度和高度计算

在CSS盒子模型中，元素的总宽度和总高度是由上述四个部分共同决定的。

- 「总宽度」 = `width` + `padding-left` + `padding-right` + `border-left` + `border-right` + `margin-left` + `margin-right`
- 「总高度」 = `height` + `padding-top` + `padding-bottom` + `border-top` + `border-bottom` + `margin-top` + `margin-bottom`

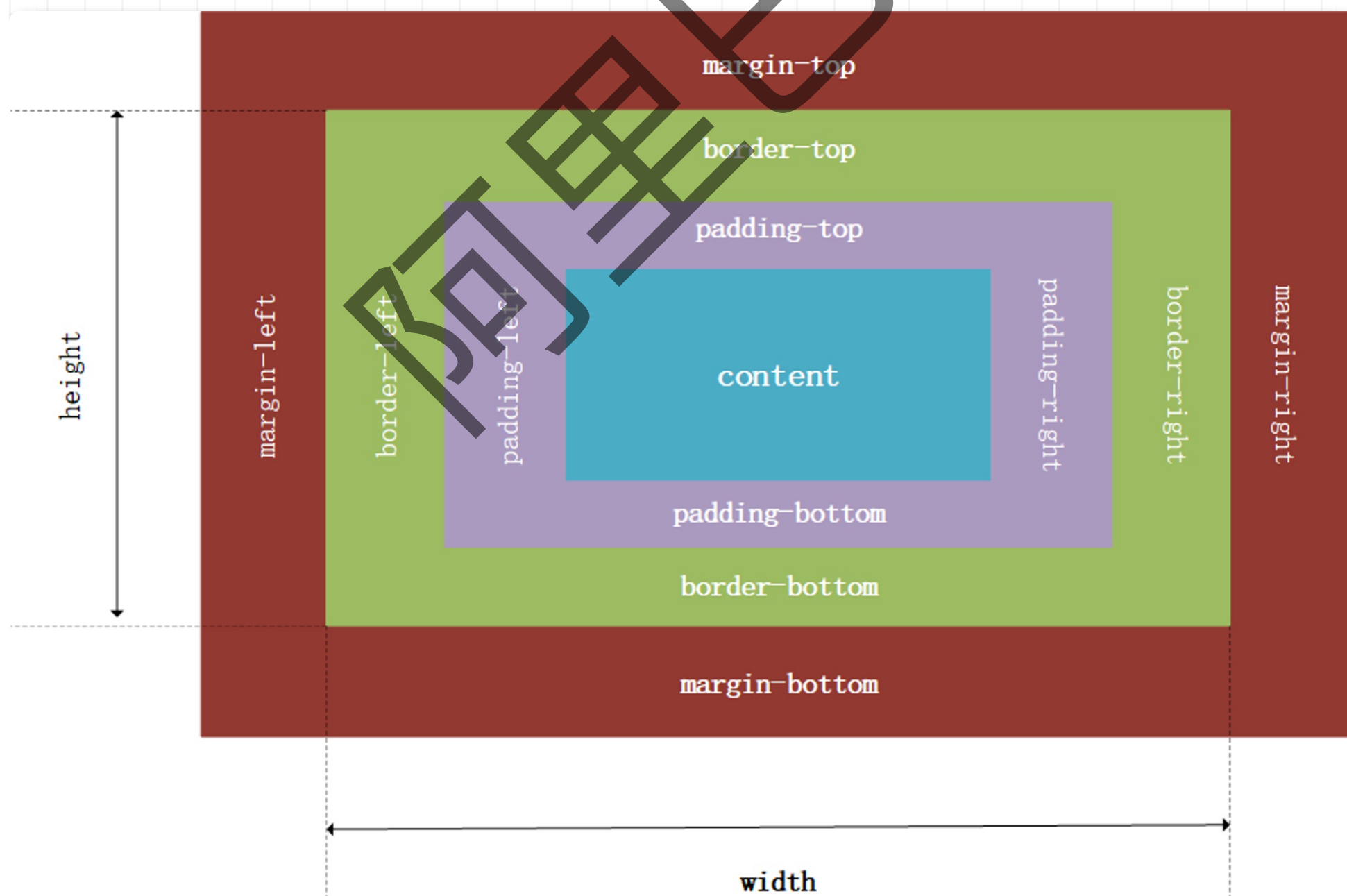
在默认情况下，CSS采用W3C标准盒子模型进行计算。也就是说，`width`和`height`属性只包含`Content`的宽度和高度，不包含`Padding`和`Border`的部分。

三、标准盒子模型与IE怪异盒子模型

1. 标准盒子模型 (content-box)

如上文所述，默认情况下，CSS采用W3C标准盒子模型，也称为`content-box`。在这种模型中，`width`和`height`属性只包含`Content`的宽度和高度。当给元素设置宽度和高度时，这个宽度和高度仅仅是内容的大小。

2. IE怪异盒子模型 (border-box)



除了标准盒子模型，还存在一种IE怪异盒子模型，也称为`border-box`。在这种模型中，`width`和`height`属性包含了`Content`、`Padding`和`Border`的宽度和高度。也就是说，设置了宽度和高度后，这个宽度和高度将包括内容、内边距和边框。

四、box-sizing属性

为了更好地控制元素的盒子模型，CSS提供了 `box-sizing` 属性。通过设置 `box-sizing` 属性，我们可以改变元素宽度和高度的计算方式。

- `box-sizing: content-box`：默认值，元素宽度和高度仅包含 `Content` 的部分（标准盒子模型）。
- `box-sizing: border-box`：元素宽度和高度包含 `Content`、`Padding` 和 `Border` 的部分（IE怪异盒子模型）。

```
<style>
.box {
  width: 200px;
  height: 100px;
  padding: 20px;
  box-sizing: border-box;
}
</style>
<div class="box">
  盒子模型
</div>
```

通过上面的代码，我们将 `.box` 元素的 `box-sizing` 属性设置为 `border-box`，这时候，它的总宽度和总高度将包括 `Content`、`Padding` 和 `Border` 的部分。因此，设置的 `width: 200px` 将完全生效，`padding` 和 `border` 部分不会再撑开盒子，实际占用的空间为 `200px`。

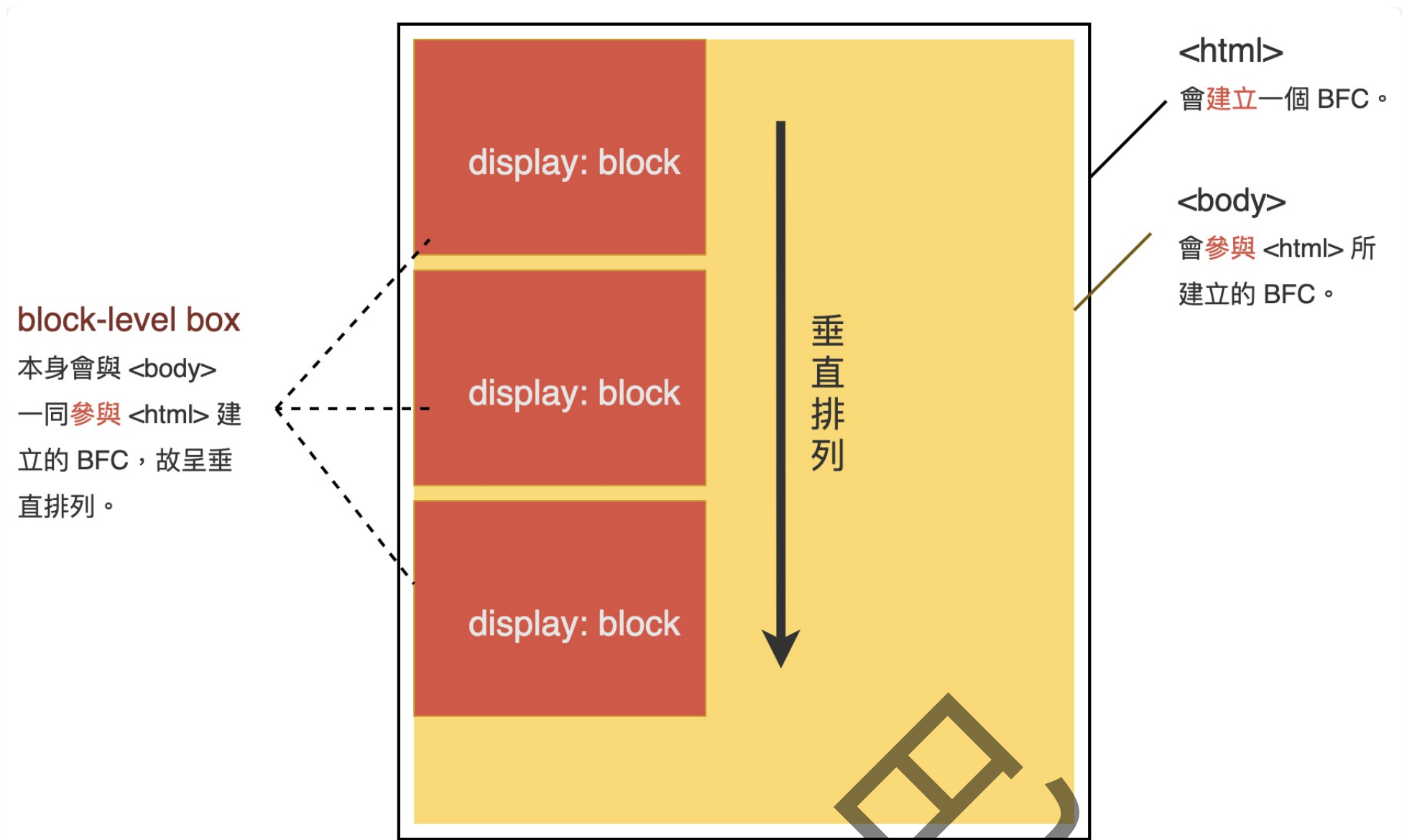
五、总结

CSS盒子模型是网页布局的基础，它决定了元素在页面中的大小和位置。了解盒子模型的概念和计算方式对于设计合理的布局和样式至关重要。在实际开发中，我们可以通过 `box-sizing` 属性来控制元素的宽度和高度的计算方式，从而更好地适应不同的布局需求。

参考文献

- [MDN Web Docs - Introduction to the CSS box model \(https://developer.mozilla.org/zh-CN/docs/Web/CSS/CSS_Box_Model/Introduction_to_the_CSS_box_model\)](https://developer.mozilla.org/zh-CN/docs/Web/CSS/CSS_Box_Model/Introduction_to_the_CSS_box_model)
- [MDN Web Docs - box-sizing \(https://developer.mozilla.org/zh-CN/docs/Web/CSS/box-sizing\)](https://developer.mozilla.org/zh-CN/docs/Web/CSS/box-sizing)

面试官：谈谈你对BFC的理解？



一、是什么

在页面布局的过程中，我们经常会遇到一些奇怪的情况，比如元素的高度消失了、两栏布局无法自适应、元素间距出现异常等等。这些问题往往是由于元素之间相互影响而导致的。在这里，就涉及到了 BFC (Block Formatting Context) 的概念。

BFC (块级格式化上下文) 是页面中一块独立的渲染区域，具有一套独立的渲染规则：

- 内部的盒子会在垂直方向上一个接一个地放置。
- 同一个 BFC 的相邻盒子的 margin 会发生重叠，与方向无关。
- 每个元素的左外边距与包含块的左边界相接触（从左到右），即使浮动元素也是如此。
- BFC 的区域不会与 float 的元素区域重叠。
- 计算 BFC 的高度时，浮动子元素也参与计算。
- BFC 是页面上的一个隔离的独立容器，容器里面的子元素不会影响到外面的元素，反之亦然。

BFC 的目的是形成一个相对于外界完全独立的空间，使内部的子元素不会影响到外部的元素。

二、触发条件

触发 BFC 的条件包含但不限于：

- 根元素，即 HTML 元素
- 浮动元素：float 值为 left、right
- overflow 值不为 visible，为 auto、scroll、hidden
- display 的值为 inline-block、inline-table、table-cell、table-caption、flex、inline-flex、grid、inline-grid
- position 的值为 absolute 或 fixed

三、应用场景

利用 BFC 的特性，我们可以在以下场景中应用 BFC：

1. 防止margin重叠（塌陷）

```
<style>
  .container {
    overflow: hidden; /* 触发BFC */
  }
  p {
    color: #f55;
    background: #fcc;
    width: 200px;
    line-height: 100px;
    text-align: center;
    margin: 100px;
  }
</style>
<body>
  <div class="container">
    <p>Haha</p>
  </div>
  <p>Hehe</p>
</body>
```

在没有触发BFC的情况下，两个p元素之间的距离为100px，发生了margin重叠（塌陷）。通过给包含p元素的容器触发BFC，两个p元素不再属于同一个BFC，从而避免了margin重叠。

2. 清除内部浮动

```
<style>
  .parent {
    border: 5px solid #fcc;
    width: 300px;
  }

  .child {
    border: 5px solid #f66;
    width: 100px;
    height: 100px;
    float: left;
  }
</style>
<body>
  <div class="parent">
    <div class="child"></div>
    <div class="child"></div>
  </div>
</body>
```

通过触发parent元素生成BFC，可以清除内部浮动的影响，从而使parent元素具有高度。

```
.parent {
  overflow: hidden; /* 触发BFC */
}
```

3. 自适应多栏布局

```
<style>
  .wrapper {
    width: 300px;
  }

  .aside {
    width: 100px;
    height: 150px;
    float: left;
    background: #f66;
  }

  .main {
    height: 200px;
    background: #fcc;
  }
</style>
<body>
  <div class="wrapper">
    <div class="aside"></div>
    <div class="main"></div>
  </div>
</body>
```

在不触发BFC的情况下，.main元素的左边会与.wrapper元素的左边界相接触，从而无法实现两栏布局。

通过触发.main元素生成BFC，.main元素的左边不再与.wrapper元素的左边界相接触，从而实现自适应多栏布局。

```
.main {
  overflow: hidden; /* 触发BFC */
}
```

小结

通过上面的例子，我们可以看到BFC的实际效果就是形成一个页面中的独立容器，里面的子元素不影响外部的元素，反之亦然。BFC为我们解决了很多布局上的问题，提供了更灵活和可靠的布局手段。

参考文献

- [MDN Web Docs - Block Formatting Context\(https://developer.mozilla.org/zh-CN/docs/Web/Guide/CSS/Block_formatting_context\)](https://developer.mozilla.org/zh-CN/docs/Web/Guide/CSS/Block_formatting_context)
- [BFC原理剖析 - Github笔记\(https://github.com/zuopf769/notebook/blob/master/fe/BFC%E5%8E%9F%E7%90%8\)](https://github.com/zuopf769/notebook/blob/master/fe/BFC%E5%8E%9F%E7%90%8)

面试官：CSS3动画有哪些？

一、是什么？

CSS动画（CSS Animations）是一种使用CSS来实现动画效果的技术，它允许通过对元素样式的逐步变化，从而产生平滑过渡的动画效果。

常见的动画效果有很多，例如平移、旋转、缩放等等，复杂的动画可以由多个简单动画组合而成。

CSS提供了多种实现动画的方式，包括：

- Transition（过渡）：用于设置元素的样式过渡效果。
- Transform（变形）：用于对元素进行旋转、缩放、移动或倾斜等变形操作。
- Animation（动画）：用于自定义复杂的动画效果。

二、实现方式

Transition 实现渐变动画

Transition通过定义元素属性的变化过程来实现动画效果，常用的属性有：

- property：填写需要变化的CSS属性。
- duration：完成过渡效果需要的时间，单位为秒（s）或毫秒（ms）。
- timing-function：完成效果的速度曲线，可选值有linear、ease、ease-in、ease-out、ease-in-out等。
- delay：动画效果的延迟触发时间，单位为秒（s）或毫秒（ms）。

举个例子，实现鼠标移动上去时发生变化的动画效果：


```
<style>
  .base {
    width: 100px;
    height: 100px;
    display: inline-block;
    background-color: #0EA9FF;
    border-width: 5px;
    border-style: solid;
    border-color: #5daf34;
    transition-property: width, height, background-color, border-width;
    transition-duration: 2s;
    transition-timing-function: ease-in;
    transition-delay: 500ms;
  }

  /*简写*/
  /*transition: all 2s ease-in 500ms;*/
  .base:hover {
    width: 200px;
    height: 200px;
    background-color: #5daf34;
    border-width: 10px;
    border-color: #3a8ee6;
  }
</style>
<div class="base"></div>
```

Transform 转变动画

Transform 用于对元素进行位移、缩放、旋转和倾斜等变换操作，一般与 **Transition** 结合使用实现动画效果。

Transform 包含四个常用的变换功能：

- translate：位移
- scale：缩放
- rotate：旋转
- skew：倾斜

注意的是，**Transform** 不支持 **inline** 元素，需要先将其设置为 **block**。

举个例子，实现元素在鼠标悬停时的动画效果：

```
<style>
  .base {
    width: 100px;
    height: 100px;
    display: inline-block;
    background-color: #0EA9FF;
    border-width: 5px;
    border-style: solid;
    border-color: #5daf34;
    transition-property: width, height, background-color, border-width;
    transition-duration: 2s;
    transition-timing-function: ease-in;
    transition-delay: 500ms;
  }

  .base2 {
    transform: none;
    transition-property: transform;
    transition-delay: 5ms;
  }

  .base2:hover {
    transform: scale(0.8) rotate(35deg) skew(5deg) translate(15px, 25px);
  }
</style>
<div class="base base2"></div>
```

在这个例子中，盒子会发生旋转、倾斜、平移和缩小的效果。

Animation 实现自定义动画

Animation 是一种用于实现自定义动画效果的属性，它是由8个子属性组成：

- **animation-name**：指定 **@keyframes** 动画的名称。
- **animation-duration**：指定动画完成一个周期所需要的时间，单位为秒（s）或毫秒（ms）。
- **animation-timing-function**：指定动画计时函数，即动画的速度曲线。
- **animation-delay**：指定动画延迟时间，即动画何时开始。
- **animation-iteration-count**：指定动画播放的次数。
- **animation-direction**：指定动画播放的方向。
- **animation-fill-mode**：指定动画填充模式。
- **animation-play-state**：指定动画播放状态，正在运行或暂停。

CSS 动画通过定义一些关键帧，浏览器会根据计时函数插值计算出中间帧，从而形成动画效果。关键帧使用 **@keyframes** 来定义，可以使用 **from** 和 **to** 关键字或百分比来刻画动画生命周期。

举个例子，实现一个元素旋转一圈的动画效果：

```
@keyframes rotate {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

/* 或者使用百分比刻画生命周期 */
/*
@keyframes rotate {
  0% {
    transform: rotate(0deg);
  }
  100% {
    transform: rotate(360deg);
  }
}
*/

.element {
  animation: rotate 2s;
}
```

三、总结

在本文中，我们介绍了CSS3中三种实现动画的方式：

- **Transition**（过渡）：用于设置元素的样式过渡效果。
- **Transform**（变形）：用于对元素进行旋转、缩放、移动或倾斜等变形操作。
- **Animation**（动画）：用于自定义复杂

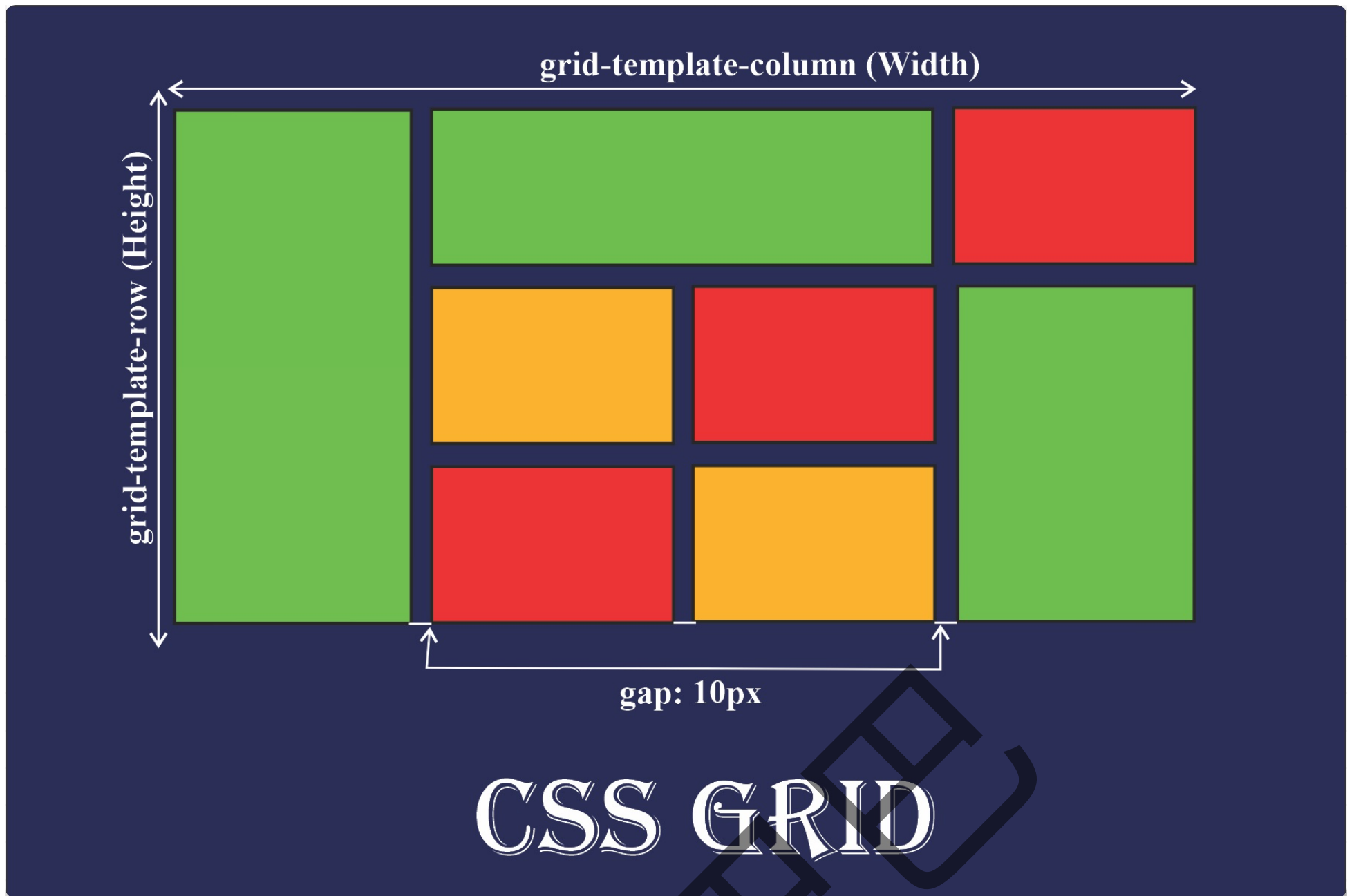
的动画效果。

每种方式在实现动画效果时有各自的特点和用途，具体应用时根据需求选择合适的方式来实现动画效果。

参考文献

- [CSS3 动画与过渡 \(https://segmentfault.com/a/1190000022540857\)](https://segmentfault.com/a/1190000022540857)
- [CSS 动画 - 维基百科 \(https://zh.m.wikipedia.org/wiki/CSS%E5%8A%A8%E7%94%BB\)](https://zh.m.wikipedia.org/wiki/CSS%E5%8A%A8%E7%94%BB)

面试官：介绍一下grid网格布局



在现代Web开发中，页面布局是一个重要的任务，它决定了网页的结构和外观。在CSS中，我们通常使用Flexbox（弹性盒布局）和Grid布局来实现不同的布局需求。本文将重点介绍Grid布局，它是一种二维布局方式，通过纵横相交的网格线来创建布局结构，能够同时处理行和列，使得布局更加灵活多样。

一、什么是Grid布局？

Grid布局，又称为网格布局，是一种强大而灵活的CSS布局模型。它通过在容器中创建一个二维网格，将页面划分为多个主要区域，并定义这些区域的大小、位置、层次等关系。与Flexbox的一维布局不同，Grid布局是一个二维布局，它在排列元素时不仅考虑主轴方向（水平或垂直），还考虑交叉轴方向，使得布局更加精确和直观。

要使用Grid布局，我们需要将元素的 `display` 属性设置为 `grid` 或 `inline-grid`，这样浏览器渲染引擎会启动网格布局算法。

```
.container {  
  display: grid;  
  /* 或者 display: inline-grid; */  
}
```

二、Grid布局的属性

Grid布局的属性分为容器属性和项目属性两类，我们逐一介绍。

容器属性：

- `grid-template-columns` 和 `grid-template-rows`：用于设置网格的列宽和行高。可以指定固定的宽度和高度，也可以使用 `repeat()` 函数简化重复的值。

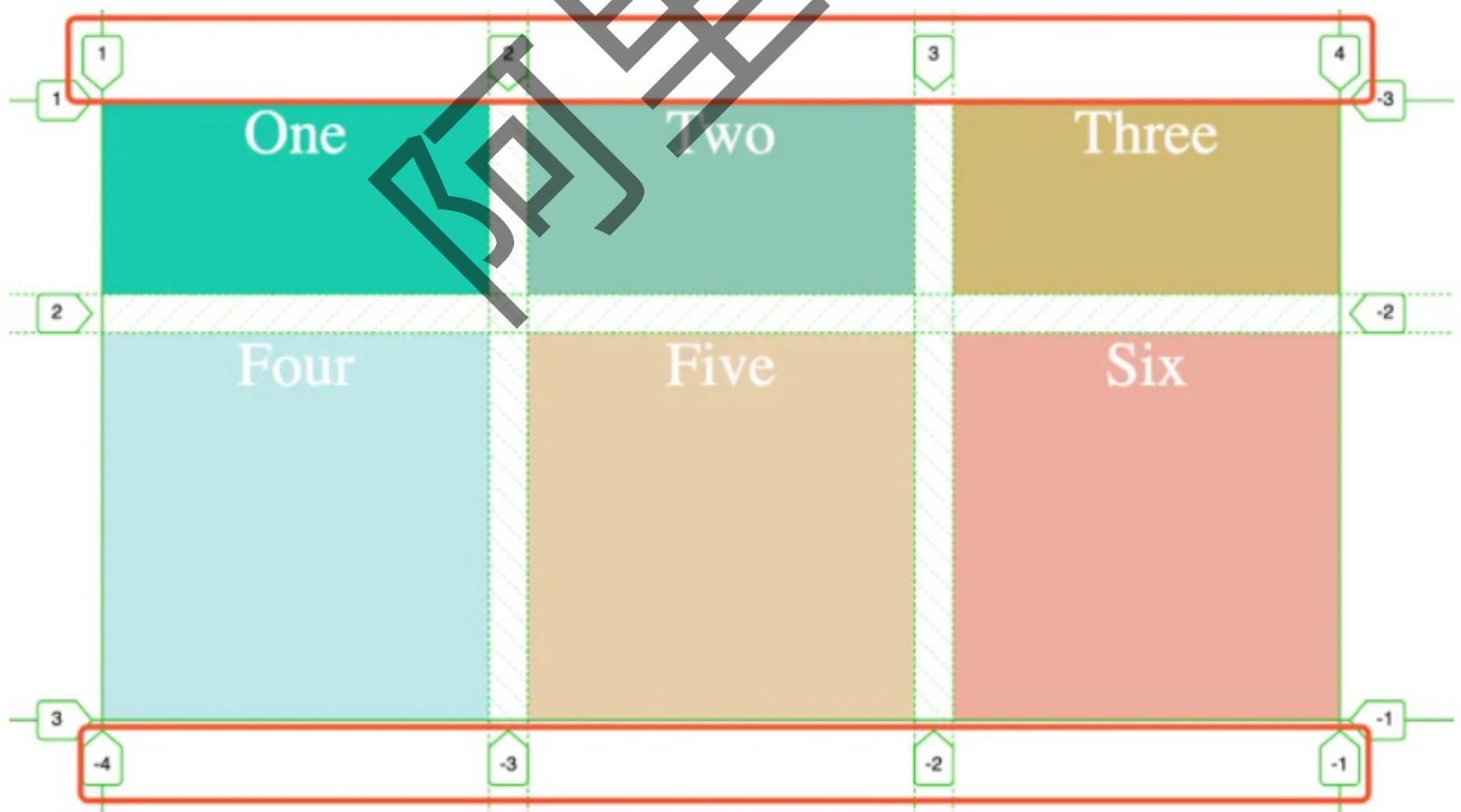
```
.container {
  display: grid;
  grid-template-columns: 200px 200px 200px;
  grid-template-rows: 100px 100px;
}
```

2. `grid-gap`：设置行间距和列间距的简写属性，相当于同时设置 `grid-row-gap` 和 `grid-column-gap`。

```
.container {
  display: grid;
  grid-gap: 10px 20px;
}
```

3. `grid-template-areas`：用于定义网格区域，将多个单元格组成一个命名区域，通过字符串来表示。

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 100px 100px;
  grid-template-areas:
    'a b c'
    'd e f';
}
```



4. `grid-auto-flow`：用于指定项目的放置顺序，默认为 `row`，即先行后列。可以设置为 `column`，使项目先列后行排列。


```
.container {  
  display: grid;  
  grid-auto-flow: column;  
}
```

5. `justify-items` 和 `align-items`：分别用于设置单元格内容在水平和垂直方向上的对齐方式。

```
.item {  
  justify-items: center; /* left, right, stretch */  
  align-items: center; /* start, end, stretch */  
}
```

项目属性：

1. `grid-column-start` 和 `grid-column-end`：用于指定项目左边框和右边框所在的垂直网格线，定义项目在列方向的位置。

```
.item {  
  grid-column-start: 2;  
  grid-column-end: 4;  
}
```

2. `grid-row-start` 和 `grid-row-end`：用于指定项目上边框和下边框所在的水平网格线，定义项目在行方向的位置。

```
.item {  
  grid-row-start: 1;  
  grid-row-end: 3;  
}
```

3. `grid-area`：用于指定项目放在哪一个命名区域。

```
.item {  
  grid-area: e;  
}
```

4. `justify-self` 和 `align-self`：分别用于设置单元格内容在水平和垂直方向上的对齐方式，只作用于单个项目。

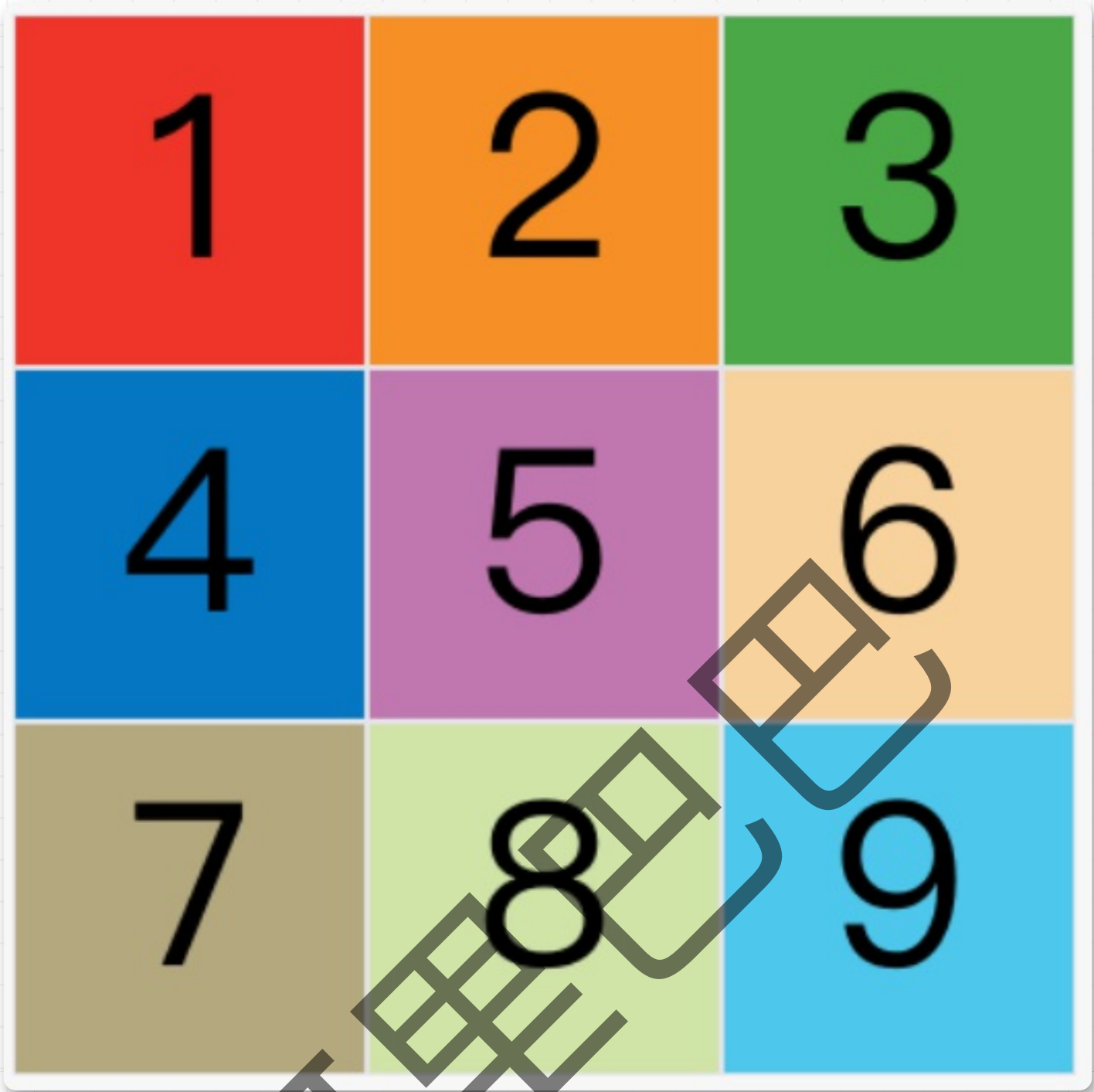
```
.item {  
  justify-self: center; /* left, right, stretch */  
  align-self: center; /* start, end, stretch */  
}
```

三、Grid布局的应用场景

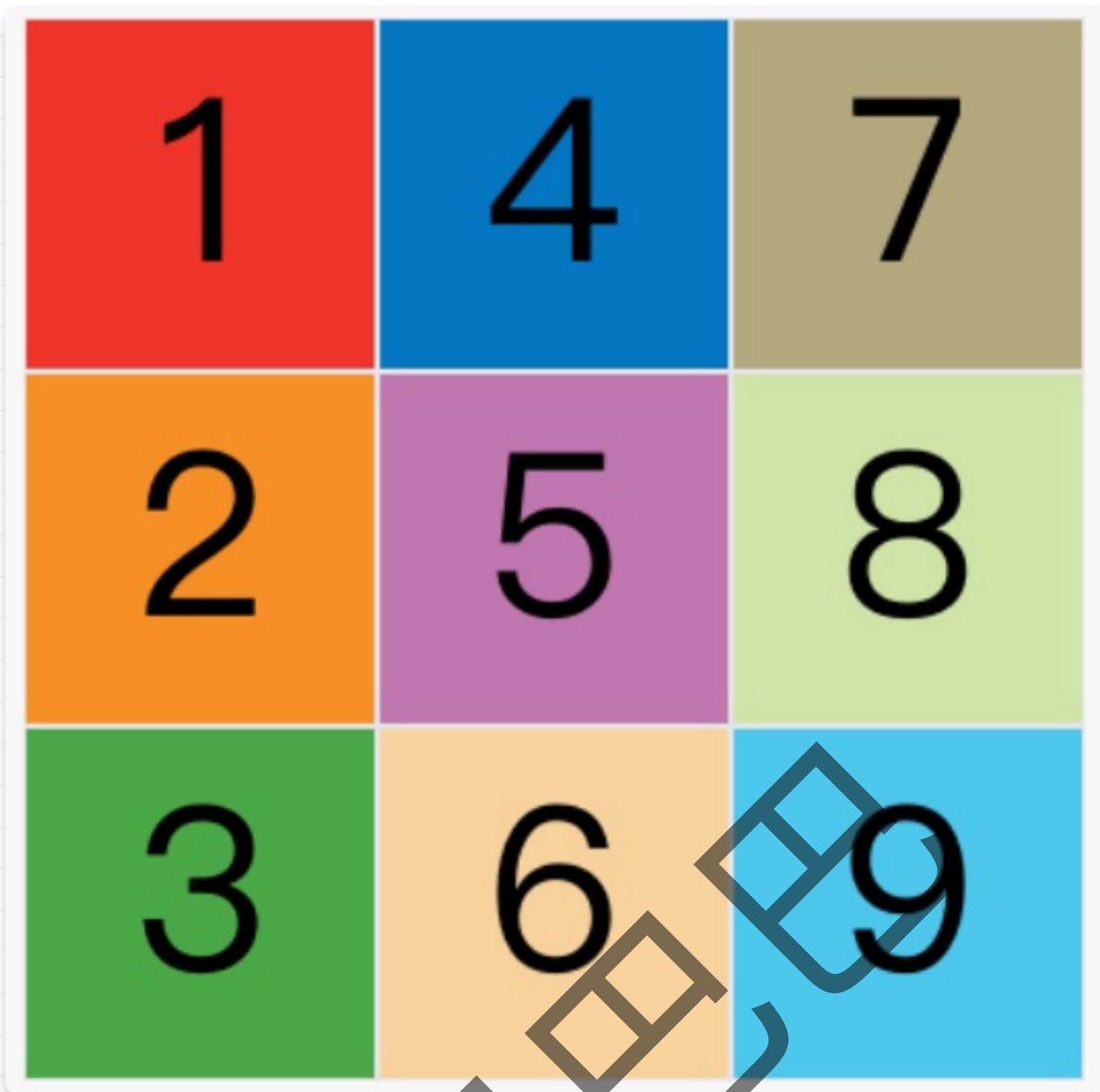
Grid布局在实际开发中有着广

泛的应用场景，以下是一些常见的应用示例：

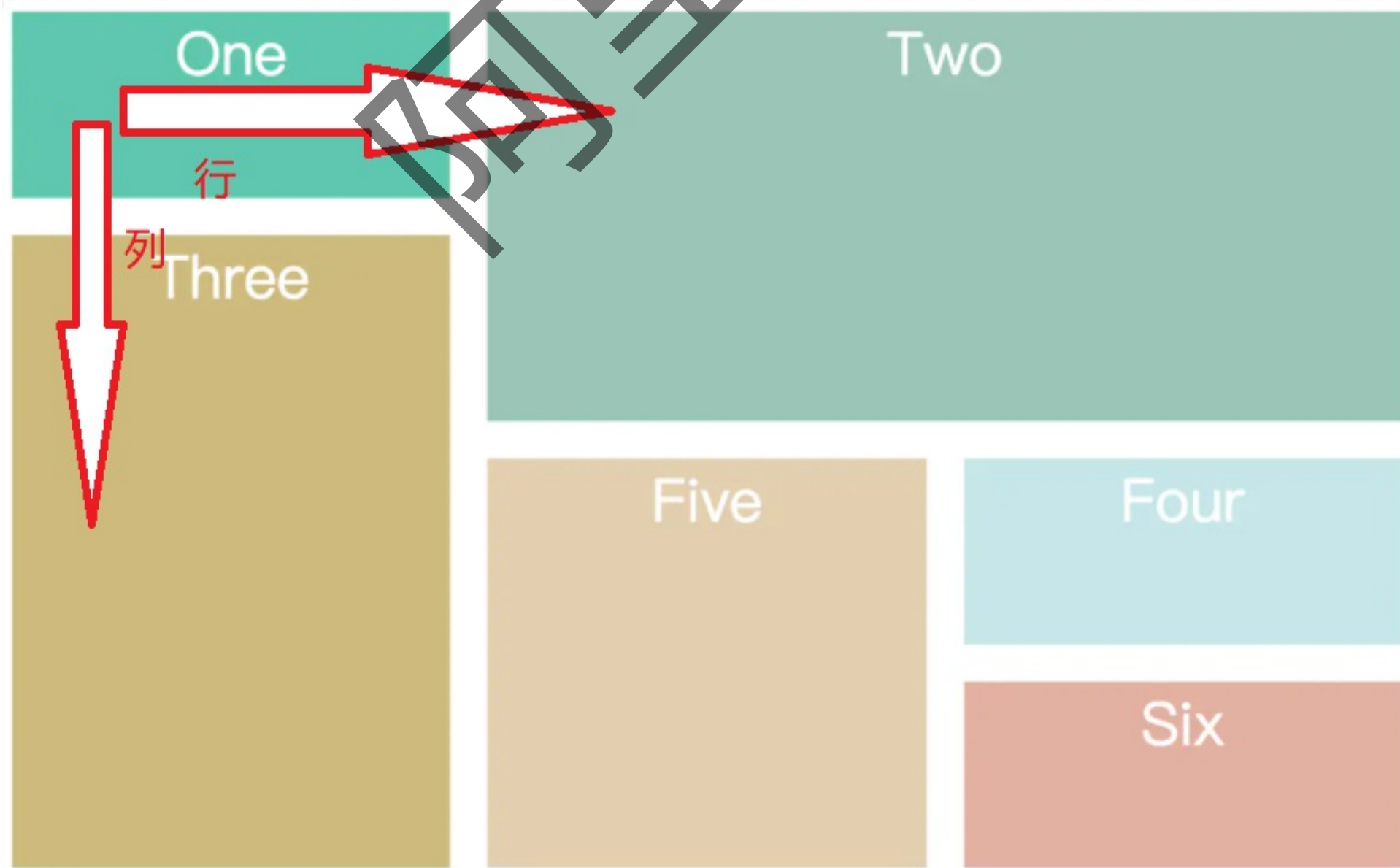
- 1. 网格导航：通过网格布局可以轻松实现水平或垂直的网格导航菜单，使得导航链接整齐排列。



- 2. 居中布局：通过设置容器的对齐方式和项目的自身对齐方式，可以实现内容在容器中的水平和垂直居中，使得布局更加美观。



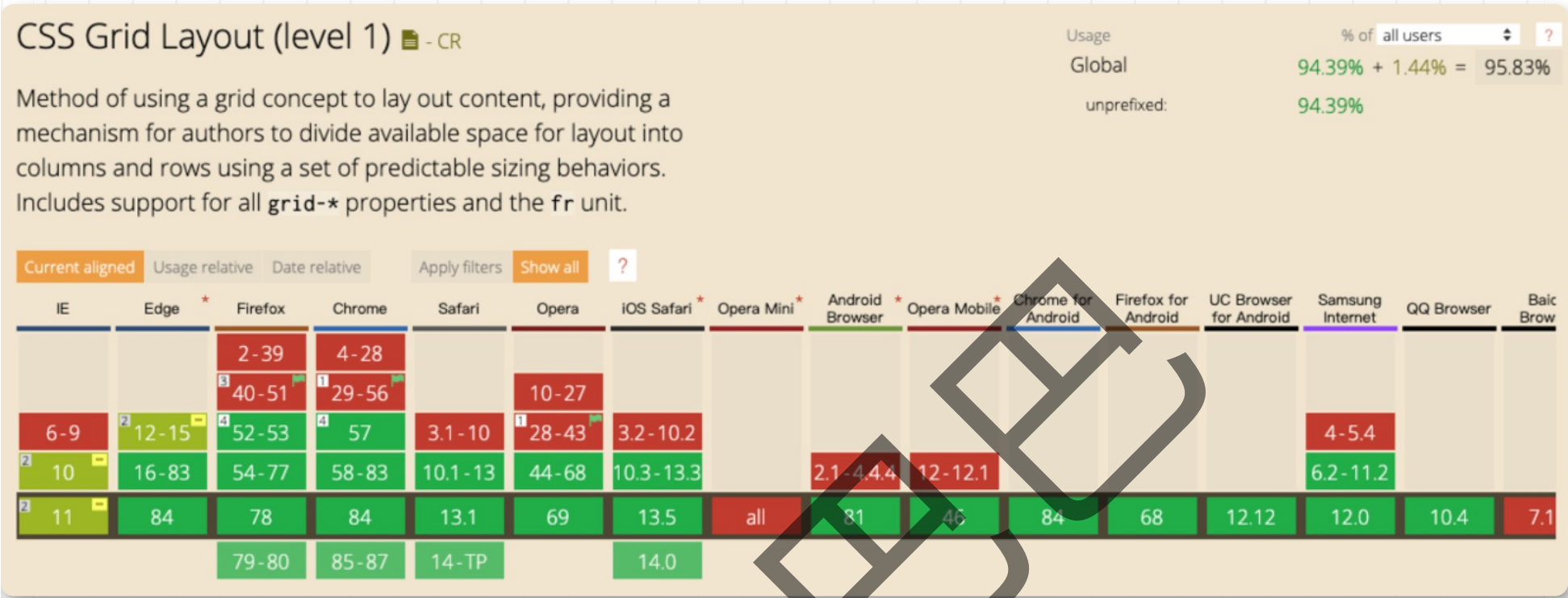
3. 多栏布局：Grid布局可以轻松实现多栏布局，使得栏目的宽度相等或不等，适用于新闻列表、产品展示等场景。



- 4. 自适应布局：通过设置项目的放大和缩小比例，可以实现根据容器大小自适应调整布局，使得页面在不同屏幕尺寸下都能良好展示。
- 5. 等高布局：通过设置项目的高度为 1fr，可以使得多个项目拥有相同的高度，适用于类似产品卡片的布局。
- 6. 响应式布局：在移动设备和桌面端的响应式布局中，Grid布局表现优秀，可以在不同尺寸的屏幕上显示不同的布局，提升用户体验。

四、兼容性和使用建议

目前，Grid布局在现代主流浏览器中具有良好的兼容性，支持情况如下：



综上所述，Grid布局是一种功能强大、灵活多样的布局方式，它在Web开发中有着广泛的应用场景。在实际开发中，我们可以灵活运用Grid布局来实现各种复杂的页面布局需求，提升网页的可读性和用户体验。然而，考虑到兼容性问题，建议在使用Grid布局时，及时做好兼容性处理，以确保网页在不同浏览器中都能正确显示和运行。希望本文能帮助读者深入理解Grid布局，并在实际项目中应用得心应手！

面试官：DNS协议及完整的查询过程

一、DNS协议是什么？

DNS (Domain Name System)，域名系统，是互联网中的一项服务，用于将域名（例如 www.example.com）解析为对应的IP地址（例如192.0.2.1）。DNS相当于一个翻译官，负责将人类可读的域名翻译成计算机可理解的IP地址，从而实现网络通信。

在计算机网络中，数据通信的核心是IP地址，但是IP地址是一串数字，不方便人们记忆和使用。因此，DNS就像一个电话簿一样，通过域名来查找相应的IP地址，使得人们可以使用便于记忆的域名来访问网站，而不必记住复杂的IP地址。

二、域名结构

域名是一个具有层次结构的标识系统，由一系列以点分隔的名字组成。从右向左，域名的层级结构由低级到高级，例如： www.example.com，其中.com为顶级域名，example为二级域名，www为三级域名。

域名系统的结构如下图所示：

```
. (根域名)
|
|-- com (顶级域名)
|   |
|   |-- example (二级域名)
|       |
|       |-- www (三级域名)
|           |
|           |-- ... (更多子域名)
|
|-- net (顶级域名)
|   |
|   |-- ...
|
|-- org (顶级域名)
|   |
|   |-- ...
|
... (更多顶级域名)
```

三、DNS查询的方式

DNS查询的方式有两种：

1. 递归查询

递归查询是一种由客户端向DNS服务器发起的查询方式，客户端会一直迭代查询直至得到最终的查询结果。在递归查询中，如果目标DNS服务器不知道某个域名的IP地址，它会向其他DNS服务器发起请求，直到找到能够解析该域名的DNS服务器为止。

2. 迭代查询

迭代查询是一种DNS服务器之间的查询方式，DNS服务器之间会相互请求信息，但不会一直迭代查询直至得到最终结果。每个DNS服务器只会返回下一级的DNS服务器地址给客户端，让客户端继续查询。

四、域名缓存

DNS查询过程中，为了减少查询时间和减轻DNS服务器负担，会在客户端和DNS服务器上设置缓存。查询过的域名和对应的IP地址会被缓存一段时间，以便下次查询时可以直接使用缓存结果，加快查询速度。

客户端和DNS服务器的缓存分别为：

- 浏览器缓存：浏览器在获取网站域名的实际IP地址后会对其进行缓存，减少网络请求的损耗。
- 操作系统缓存：操作系统的缓存其实是用户自己配置的 `hosts` 文件，可以手动添加一些常用域名和对应的IP地址，避免每次都进行DNS查询。

五、完整的DNS查询过程

下面是一个完整的DNS查询过程：

- 首先搜索浏览器的DNS缓存，如果存在目标域名的缓存结果，则直接返回IP地址，查询结束。

2. 如果浏览器缓存中没有目标域名的结果，继续搜索操作系统的DNS缓存，如果存在目标域名的缓存结果，则直接返回IP地址，查询结束。
3. 如果操作系统缓存中没有目标域名的结果，将域名发送给本地域名服务器。
4. 本地域名服务器首先搜索自己的缓存，如果存在目标域名的缓存结果，则直接返回IP地址，查询结束。
5. 如果本地域名服务器的缓存中没有目标域名的结果，本地域名服务器会按照递归查询的方式，向更高级的DNS服务器发起请求。
6. 递归查询中，如果某个DNS服务器无法解析目标域名，则该DNS服务器会向更高级的DNS服务器请求解析结果，直至找到能够解析目标域名的DNS服务器。
7. 最终，本地域名服务器得到目标域名对应的IP地址，并将IP地址返回给操作系统，并缓存该结果。
8. 操作系统将IP地址返回给浏览器，并缓存该结果。
9. 浏览器得到目标域名对应的IP地址，开始与服务器建立连接，进行后续的网络通信。

整个DNS查询过程涉及多个级别的DNS服务器的协作和查询，直至找到目标域名对应的IP地址。

六、DNS查询过程中的代码示例

下面是一个使用Python编写的简单DNS查询的代码示例，使用socket库进行DNS查询：

```
const dns = require('dns');

function dnsLookup(domain) {
  return new Promise((resolve, reject) => {
    dns.lookup(domain, (err, address, family) => {
      if (err) {
        reject(err);
      } else {
        resolve(address);
      }
    });
  });
}

const domainName = 'www.example.com';
dnsLookup(domainName)
  .then(ipAddress => {
    console.log(`The IP address of ${domainName} is ${ipAddress}`);
  })
  .catch(error => {
    console.error(`DNS lookup failed: ${error.message}`);
  });
```

在这个示例中，我们定义了一个 `dns_lookup` 函数来进行DNS查询，传入域名作为参数，然后使用 `socket.gethostbyname()` 函数来获取域名对应的IP地址。如果查询失败，会捕获 `socket.gaierror` 异常，并返回相应的错误信息。

参考文献

- <https://en.wikipedia.org/wiki/>

面试官：为什么说HTTPS比HTTP安全？ HTTPS是如何保证安全的？

一、安全特性

在上篇文章中，我们了解到 HTTP 在通信过程中，存在以下问题：

- 通信使用明文（不加密），内容可能被窃听
- 不验证通信方的身份，因此有可能遭遇伪装

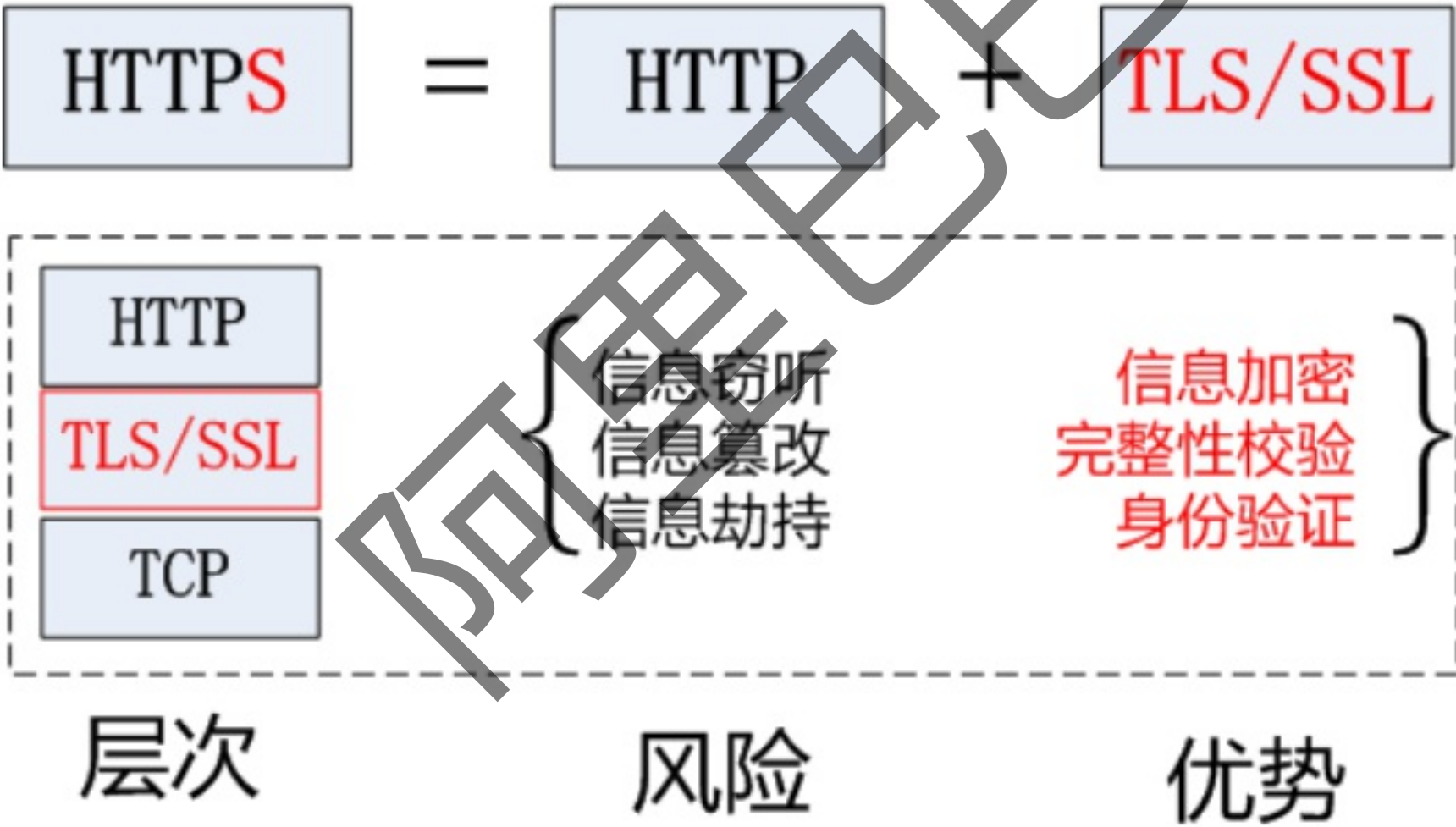
而 HTTPS 的出现正是解决这些问题， HTTPS 是建立在 SSL 之上，其安全性由 SSL 来保证

在采用 SSL 后， HTTP 就拥有了 HTTPS 的加密、证书和完整性保护这些功能

“

SSL(Secure Sockets Layer 安全套接字协议),及其继任者传输层安全 (Transport Layer Security, TLS) 是为网络通信提供安全及数据完整性的一种安全协议

”



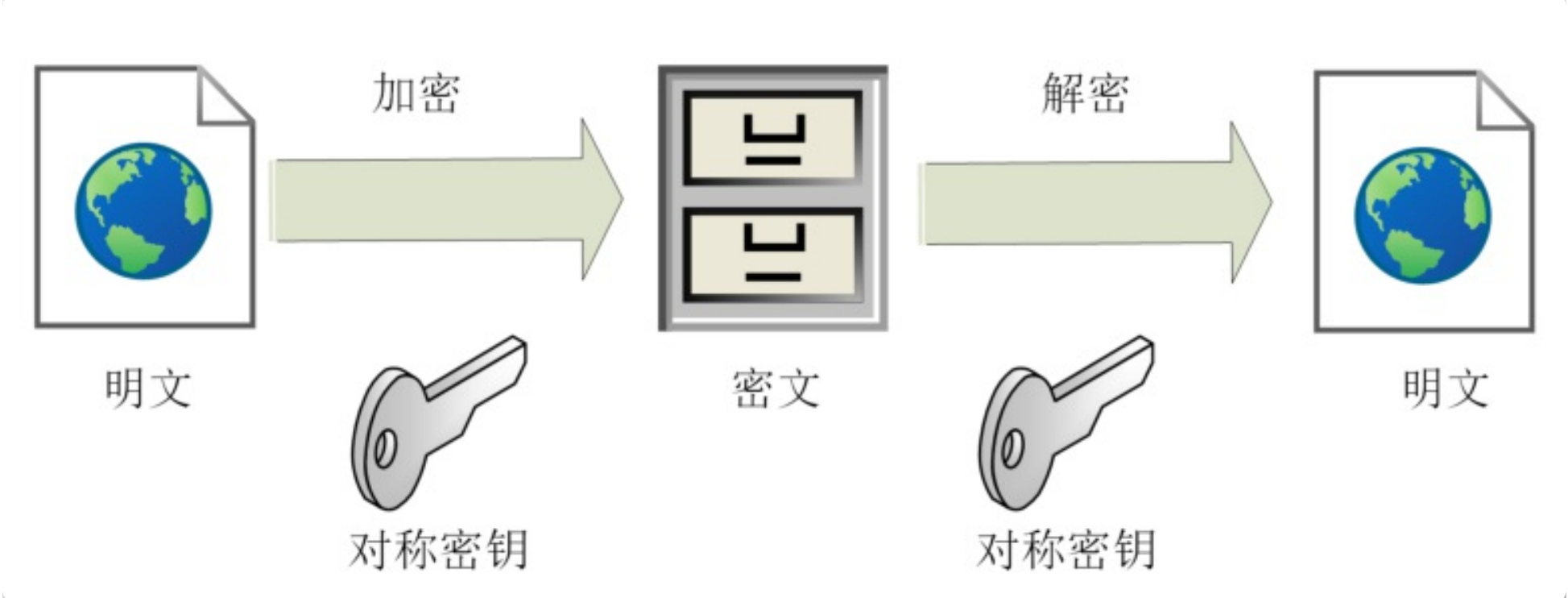
二、如何做

SSL 的实现这些功能主要依赖于三种手段：

- 对称加密：采用协商的密钥对数据加密
- 非对称加密：实现身份认证和密钥协商
- 摘要算法：验证信息的完整性
- 数字签名：身份验证

对称加密

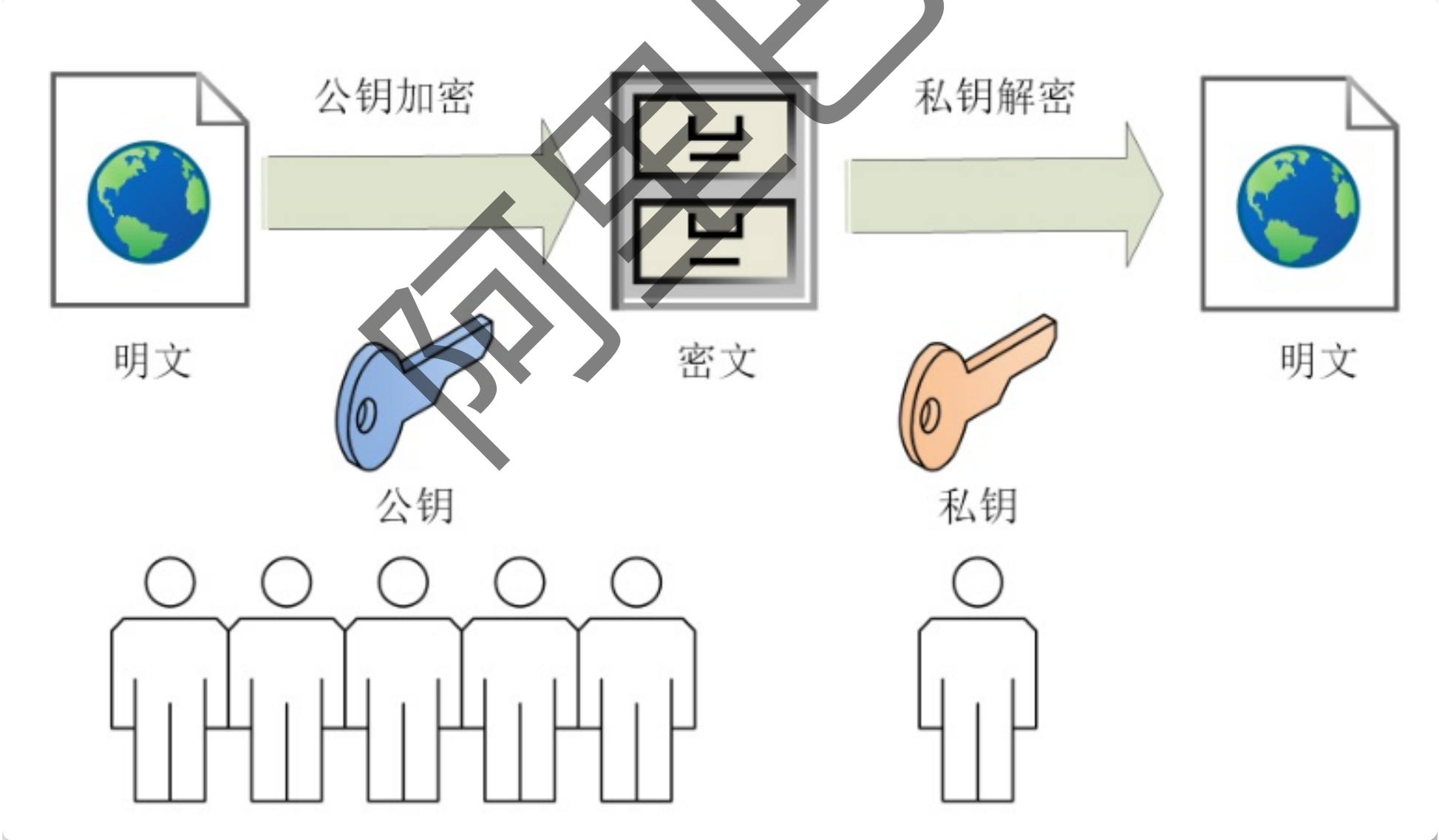
对称加密指的是加密和解密使用的密钥都是同一个，是对称的。只要保证了密钥的安全，那整个通信过程就可以说具有了机密性



非对称加密

非对称加密，存在两个密钥，一个叫公钥，一个叫私钥。两个密钥是不同的，公钥可以公开给任何人使用，私钥则需要保密

公钥和私钥都可以用来加密解密，但公钥加密后只能用私钥解密，反过来，私钥加密后也只能用公钥解密

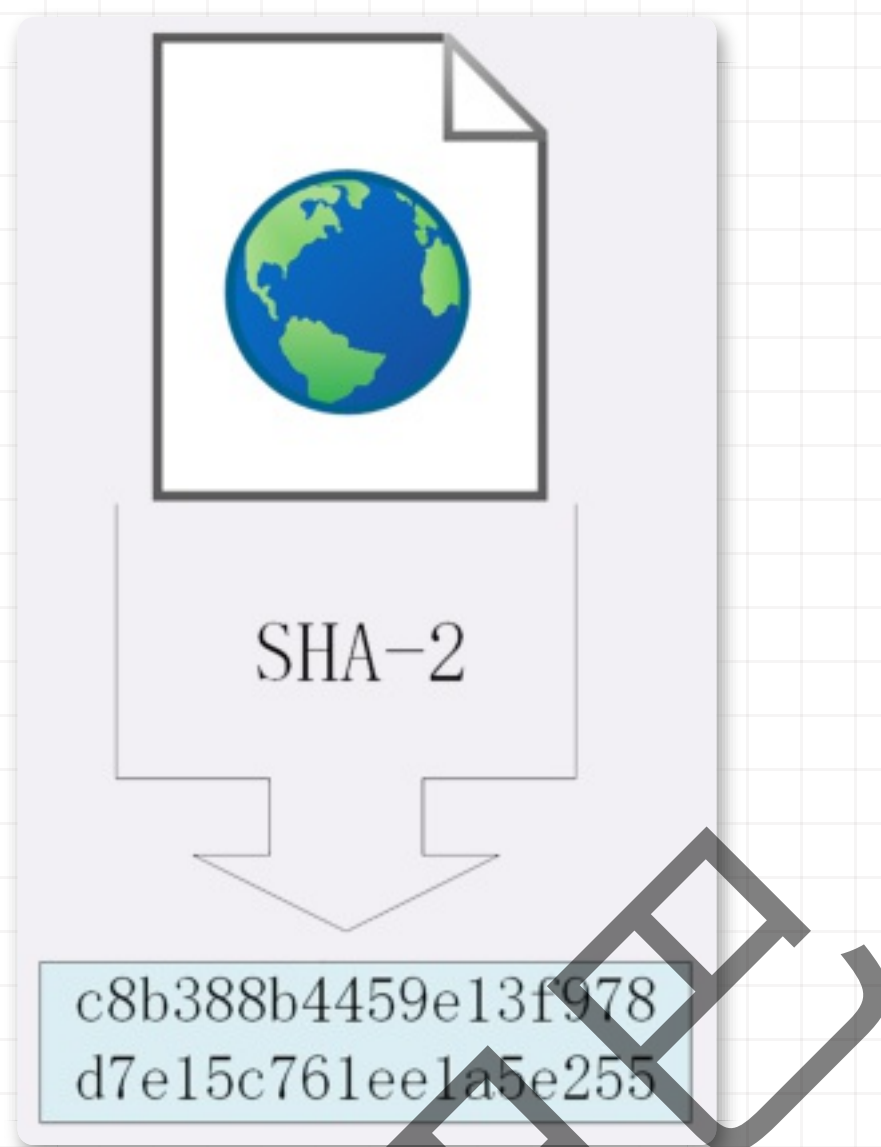


混合加密

在 [HTTPS](#) 通信过程中，采用的是对称加密+非对称加密，也就是混合加密

在对称加密中讲到，如果能够保证了密钥的安全，那整个通信过程就可以说具有了机密性
而 [HTTPS](#) 采用非对称加密解决密钥交换的问题

具体做法是发送密文的一方使用对方的公钥进行加密处理“对称的密钥”，然后对方用自己的私钥解密拿到“对称的密钥”



这样可以确保交换的密钥是安全的前提下，使用对称加密方式进行通信

举个例子：

网站秘密保管私钥，在网上任意分发公钥，你想要登录网站只要用公钥加密就行了，密文只能由私钥持有者才能解密。而黑客因为没有私钥，所以就无法破解密文

上述的方法解决了数据加密，在网络传输过程中，数据有可能被篡改，并且黑客可以伪造身份发布公钥，如果你获取到假的公钥，那么混合加密也并无多大用处，你的数据仍被黑客解决

因此，在上述加密的基础上仍需加上完整性、身份验证的特性，来实现真正的安全，实现这一功能则是摘要算法

摘要算法

实现完整性的手段主要是摘要算法，也就是常说的散列函数、哈希函数

可以理解成一种特殊的压缩算法，它能够把任意长度的数据“压缩”成固定长度、而且独一无二的“摘要”字符串，就好像是给这段数据生成了一个数字“指纹”

摘要算法保证了“数字摘要”和原文是完全等价的。所以，我们只要在原文后附上它的摘要，就能够保证数据的完整性

比如，你发了条消息：“转账 1000 元”，然后再加上一个 SHA-2 的摘要。网站收到后也计算一下消息的摘要，把这两份“指纹”做个对比，如果一致，就说明消息是完整可信的，没有被修改

数字签名

数字签名能确定消息确实是由发送方签名并发出来的，因为别人假冒不了发送方的签名

原理其实很简单，就是用私钥加密，公钥解密

签名和公钥一样完全公开，任何人都可以获取。但这个签名只有用私钥对应的公钥才能解开，拿到摘要后，再比对原文验证完整性，就可以像签署文件一样证明消息确实是你发的

和消息本身一样，因为谁都可以发布公钥，我们还缺少防止黑客伪造公钥的手段，也就是说，怎么判断这个公钥就是你的公钥

这时候就需要一个第三方，就是证书验证机构

CA验证机构

数字证书认证机构处于客户端与服务器双方都可信赖的第三方机构的立场

CA 对公钥的签名认证要求包括序列号、用途、颁发者、有效时间等等，把这些打成一个包再签名，完整地证明公钥关联的各种信息，形成“数字证书”

流程如下图：

- 服务器的运营人员向数字证书认证机构提出公开密钥的申请
- 数字证书认证机构在判明提出申请者的身份之后，会对已申请的公开密钥做数字签名
- 然后分配这个已签名的公开密钥，并将该公开密钥放入公钥证书后绑定在一起
- 服务器会将这份由数字证书认证机构颁发的数字证书发送给客户端，以进行非对称加密方式通信

接到证书的客户端可使用数字证书认证机构的公开密钥，对那张证书上的数字签名进行验证，一旦验证通过，则证明：

- 认证服务器的公开密钥的是真实有效的数字证书认证机构
- 服务器的公开密钥是值得信赖的

三、总结

可以看到，**HTTPS**与**HTTP**虽然只差一个**SSL**，但是通信安全得到了大大的保障，通信的四大特性都以解决，解决方式如下：

- 机密性：混合算法
- 完整性：摘要算法
- 身份认证：数字签名
- 不可否定：数字签名

同时引入第三方证书机构，确保公开密钥的安全性

参考文献

- <https://zhuanlan.zhihu.com/p/100657391>
- <https://juejin.cn/post/6844903830987997197#heading-7>
- <https://cloud.tencent.com/developer/article/1748862>

面试官：什么是HTTP? HTTP 和 HTTPS 的区别?

一、HTTP

HTTP (HyperText Transfer Protocol)，即超文本运输协议，是实现网络通信的一种规范

在计算机和网络世界有，存在不同的协议，如广播协议、寻址协议、路由协议等等.....

而 **HTTP** 是一个传输协议，即将数据由A传到B或将B传输到A，并且 A 与 B 之间能够存放很多第三方，如： $A \rightleftharpoons X \rightleftharpoons Y \rightleftharpoons Z \rightleftharpoons B$

传输的数据并不是计算机底层中的二进制包，而是完整的、有意义的数据，如HTML 文件，图片文件，查询结果等超文本，能够被上层应用识别

在实际应用中， **HTTP** 常被用于在 **Web** 浏览器和网站服务器之间传递信息，以明文方式发送内容，不提供任何方式的数据加密

特点如下：

- 支持客户/服务器模式
- 简单快速：客户向服务器请求服务时，只需传送请求方法和路径。由于HTTP协议简单，使得HTTP服务器的程序规模小，因而通信速度很快
- 灵活：HTTP允许传输任意类型的数据对象。正在传输的类型由Content-Type加以标记
- 无连接：无连接的含义是限制每次连接只处理一个请求。服务器处理完客户的请求，并收到客户的应答后，即断开连接。采用这种方式可以节省传输时间
- 无状态：HTTP协议无法根据之前的状态进行本次的请求处理

二、HTTPS

在上述介绍 **HTTP** 中，了解到 **HTTP** 传递信息是以明文的形式发送内容，这并不安全。而 **HTTPS** 出现正是为了解决 **HTTP** 不安全的特性

为了保证这些隐私数据能加密传输，让 **HTTP** 运行安全的 **SSL/TLS** 协议上，即 $HTTPS = HTTP + SSL/TLS$ ，通过 **SSL** 证书来验证服务器的身份，并为浏览器和服务器之间的通信进行加密

SSL 协议位于 **TCP/IP** 协议与各种应用层协议之间，浏览器和服务器在使用 **SSL** 建立连接时需要选择一组恰当的加密算法来实现安全通信，为数据通讯提供安全支持

流程图如下所示：

- 首先客户端通过URL访问服务器建立SSL连接
- 服务端收到客户端请求后，会将网站支持的证书信息（证书中包含公钥）传送一份给客户端
- 客户端的服务器开始协商SSL连接的安全等级，也就是信息加密的等级
- 客户端的浏览器根据双方同意的安全等级，建立会话密钥，然后利用网站的公钥将会话密钥加密，并传送给网站
- 服务器利用自己的私钥解密出会话密钥
- 服务器利用会话密钥加密与客户端之间的通信

三、区别

- HTTPS是HTTP协议的安全版本，HTTP协议的数据传输是明文的，是不安全的，HTTPS使用了SSL/TLS协议进行了加密处理，相对更安全
- HTTP 和 HTTPS 使用连接方式不同，默认端口也不一样，HTTP是80，HTTPS是443
- HTTPS 由于需要设计加密以及多次握手，性能方面不如 HTTP
- HTTPS需要SSL，SSL 证书需要钱，功能越强大的证书费用越高

参考文献

- <https://www.cnblogs.com/klb561/p/10289199.html>
- <https://www.jianshu.com/p/205c0fc51c97>

面试官：如何理解CDN？说说实现原理？

一、CDN是什么？

CDN (全称 Content Delivery Network)，即内容分发网络。

CDN是构建在现有网络基础之上的智能虚拟网络，通过部署在各地的边缘服务器，通过中心平台的负载均衡、内容分发、调度等功能模块，使用户能够就近获取所需内容，从而降低网络拥塞，提高用户访问响应速度和命中率。CDN的关键技术主要有内容存储和分发技术。

简单来说，CDN就是根据用户位置分配最近的资源，让用户在上网的时候不用直接访问源站，而是访问离他“最近的”一个CDN节点，该节点其实是缓存了源站内容的代理服务器。如下图所示：

二、CDN实现原理

在没有应用CDN时，我们使用域名访问某个站点的路径为：

“
用户提交域名→浏览器对域名进行解释→DNS解析得到目的主机的IP地址→根据IP地址发出请求→得到请求数据并回复
”

应用CDN后，DNS解析不再返回IP地址，而是一个CNAME (Canonical Name) 别名记录，指向CDN的全局负载均衡。

CNAME实际上在域名解析的过程中承担了中间人（或者说代理）的角色，这是CDN实现的关键。

负载均衡系统

由于没有返回IP地址，本地DNS会向负载均衡系统发送请求，然后进入CDN的全局负载均衡系统进行智能调度：

- 看用户的IP地址，查表得知地理位置，找相对最近的边缘节点。
- 看用户所在的运营商网络，找相同网络的边缘节点。
- 检查边缘节点的负载情况，找负载较轻的节点。
- 其他因素，如节点的“健康状况”、服务能力、带宽、响应时间等。

综合上述因素，找到最合适的边缘节点，然后将该节点返回给用户，用户就能够就近访问CDN的缓存代理。

整体流程如下图所示：

缓存代理

缓存系统是CDN的另一个关键组成部分，缓存系统会有选择地缓存那些最常用的资源。

其中有两个衡量CDN服务质量的指标：

- 命中率：用户访问的资源恰好在缓存系统里，可以直接返回给用户，命中次数与所有访问次数之比。
- 回源率：缓存里没有，必须用代理的方式回源站取，回源次数与所有访问次数之比。

缓存系统也可以划分成层次，分为一级缓存节点和二级缓存节点。一级缓存配置高一些，直连源站，二级缓存配置低一些，直连用户。

回源的时候二级缓存只找一级缓存，一级缓存没有才回源站，这样可以有效地减少真正的回源。

现在的商业CDN命中率通常在90%以上，相当于把源站的服务能力放大了10倍以上。

三、总结

CDN的目的是为了改善互联网的服务质量，通俗一点说其实就是提高访问速度。
CDN构建了全国、全球级别的专网，让用户就近访问专网里的边缘节点，降低了传输延迟，实现了网站加速。
通过CDN的负载均衡系统，智能调度边缘节点提供服务，相当于CDN服务的大脑，而缓存系统相当于CDN的心脏，缓存命中直接返回给用户，否则回源。

参考文献

- <https://zh.wikipedia.org/wiki/内容傳遞網路>
- <https://juejin.cn/post/6844903890706661389#heading-5>
- <https://blog.csdn.net/lxx309707872/article/details/109078783>

面试官：如何理解OSI七层模型？

一、是什么

OSI（Open System Interconnect）模型全称为开放式通信系统互连参考模型，是国际标准化组织（ISO）提出的一个试图使各种计算机在世界范围内互连为网络的标准框架
OSI将计算机网络体系结构划分为七层，每一层实现各自的功能和协议，并完成与相邻层的接口通信。即每一层扮演固定的角色，互不打扰

二、划分

OSI主要划分了七层，如下图所示：

应用层

应用层位于OSI参考模型的第七层，其作用是通过应用程序间的交互来完成特定的网络应用
该层协议定义了应用进程之间的交互规则，通过不同的应用层协议为不同的网络应用提供服务。例如域名系统DNS，支持万维网应用的HTTP协议，电子邮件系统采用的SMTP协议等
在应用层交互的数据单元我们称之为报文

表示层

表示层的作用是使通信的应用程序能够解释交换数据的含义，其位于OSI参考模型的第六层，向上为应用层提供服务，向下接收来自会话层的服务
该层提供的服务主要包括数据压缩，数据加密以及数据描述，使应用程序不必担心在各台计算机中表示和存储的内部格式差异

会话层

会话层就是负责建立、管理和终止表示层实体之间的通信会话
该层提供了数据交换的定界和同步功能，包括了建立检查点和恢复方案的方法

传输层

传输层的主要任务是为两台主机进程之间的通信提供服务，处理数据包错误、数据包次序，以及其他一些关键传输问题

传输层向高层屏蔽了下层数据通信的细节。因此，它是计算机通信体系结构中关键的一层

其中，主要的传输层协议是 TCP 和 UDP

网络层

两台计算机之间传送数据时其通信链路往往不止一条，所传输的信息甚至可能经过很多通信子网

网络层的主要任务就是选择合适的网间路由和交换节点，确保数据按时成功传送

在发送数据时，网络层把传输层产生的报文或用户数据报封装成分组和包，向下传输到数据链路层

在网络层使用的协议是无连接的网际协议（Internet Protocol）和许多路由协议，因此我们通常把该层简单地称为 IP 层

数据链路层

数据链路层通常也叫做链路层，在物理层和网络层之间。两台主机之间的数据传输，总是在一段一段的链路上传送的，这就需要使用专门的链路层协议

在两个相邻节点之间传送数据时，数据链路层将网络层交下来的 IP 数据报组装成帧，在两个相邻节点间的链路上传送帧

每一帧的数据可以分成：报头 head 和数据 data 两部分：

- head 标明数据发送者、接受者、数据类型，如 MAC 地址
- data 存储了计算机之间交互的数据

通过控制信息我们可以知道一个帧的起止比特位置，此外，也能使接收端检测出所收到的帧有无差错，如果发现差错，数据链路层能够简单的丢弃掉这个帧，以避免继续占用网络资源

物理层

作为 OSI 参考模型中最低的一层，物理层的作用是实现计算机节点之间比特流的透明传送

该层的主要任务是确定与传输媒体的接口的一些特性（机械特性、电气特性、功能特性，过程特性）

该层主要是和硬件有关，与软件关系不大

三、传输过程

数据在各层之间的传输如下图所示：

- 应用层报文被传送到运输层
- 在最简单的情况下，运输层收取到报文并附上附加信息，该首部将被接收端的运输层使用
- 应用层报文和运输层首部信息一道构成了运输层报文段。附加的信息可能包括：允许接收端运输层向上向适当的应用程序交付报文的信息以及差错检测位信息。该信息让接收端能够判断报文中的比特是否在途中已被改变
- 运输层则向网络层传递该报文段，网络层增加了如源和目的端系统地址等网络层首部信息，生成了网络层数据报
- 网络层数据报接下来被传递给链路层，在数据链路层数据包添加发送端 MAC 地址和接收端 MAC 地址后被封装成数据帧
- 在物理层数据帧被封装成比特流，之后通过传输介质传送到对端
- 对端再一步步解开封装，获取到传送的数据

参考文献

- <https://zh.wikipedia.org/wiki/OSI%E6%A8%A1%E5%9E%8B>
- <https://zhuanlan.zhihu.com/p/32059190>
- <https://leetcode-cn.com/leetbook/detail/networks-interview-highlights/>

面试官：如何理解TCP/IP协议？

一、是什么

TCP/IP，「**传输控制协议**」/「**网际协议**」，是指能够在多个不同网络间实现信息传输的协议簇

- TCP（传输控制协议）

一种面向连接的、可靠的、基于字节流的传输层通信协议

- IP（网际协议）

用于封包交换数据网络的协议

TCP/IP协议不仅仅指的是**TCP**和**IP**两个协议，而是指一个由**FTP**、**SMTP**、**TCP**、**UDP**、**IP**等协议构成的协议簇，

只是因为**TCP/IP**协议中**TCP**协议和**IP**协议最具代表性，所以通称为TCP/IP协议族（英语：TCP/IP Protocol Suite，或TCP/IP Protocols）

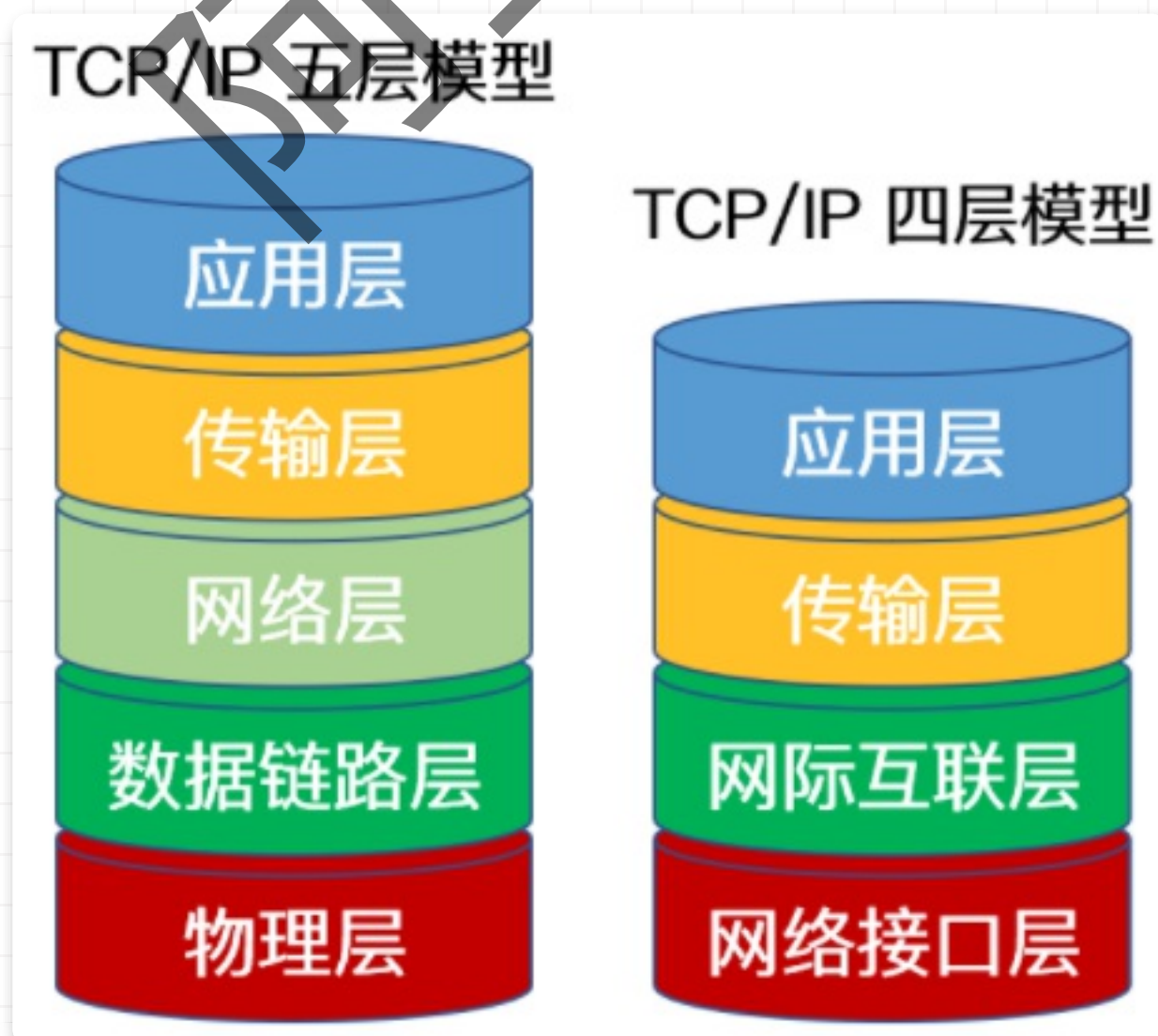
二、划分

TCP/IP协议族按层次分别有五层体系或者四层体系

五层体系的协议结构是综合了OSI和TCP/IP优点的一种协议，包括应用层、传输层、网络层、数据链路层和物理层

五层协议的体系结构只是为介绍网络原理而设计的，实际应用还是TCP/IP四层体系结构，包括应用层、传输层、网络层（网际互联层）、网络接口层

如下图所示：



五层体系

应用层

TCP/IP 模型将 OSI 参考模型中的会话层、表示层和应用层的功能合并到一个应用层实现，通过不同的应用层协议为不同的应用提供服务

如：FTP、Telnet、DNS、SMTP 等

传输层

该层对应于 OSI 参考模型的传输层，为上层实体提供源端到对端主机的通信功能

传输层定义了两个主要协议：传输控制协议（TCP）和用户数据报协议（UDP）

其中面向连接的 TCP 协议保证了数据的传输可靠性，面向无连接的 UDP 协议能够实现数据包简单、快速地传输

网络层

负责为分组网络中的不同主机提供通信服务，并通过选择合适的路由将数据传递到目标主机

在发送数据时，网络层把运输层产生的报文段或用户数据封装成分组或包进行传送

数据链路层

数据链路层在两个相邻节点传输数据时，将网络层交下来的IP数据报组装成帧，在两个相邻节点之间的链路上传送帧

物理层

保数据可以在各种物理媒介上进行传输，为数据的传输提供可靠的环境

四层体系

TCP/IP 的四层结构则如下表所示：

层次名称	单位	功 能	协 议
网络接口层	帧	负责实际数据的传输，对应OSI参考模型的下两层	HDLC（高级链路控制协议）PPP（点对点协议）SLIP（串行线路接口协议）
网络层	数据报	负责网络间的寻址数据传输，对应OSI参考模型的第三层	IP（网际协议）ICMP（网际控制消息协议）ARP（地址解析协议）RARP（反向地址解析协议）
传输层	报文段	负责提供可靠的传输服务，对应OSI参考模型的第四层	TCP（控制传输协议）UDP（用户数据报协议）
应用层		负责实现一切与应用程序相关的功能，对应OSI参考模型的上三层	FTP（文件传输协议）HTTP（超文本传输协议）DNS（域名服务器协议）SMTP（简单邮件传输协议）NFS（网络文件系统协议）

三、总结

OSI 参考模型与 TCP/IP 参考模型区别如下：

相同点：

- OSI 参考模型与 TCP/IP 参考模型都采用了层次结构

◦ 都能够提供面向连接和无连接两种通信服务机制

不同点：

- OSI 采用的七层模型； TCP/IP 是四层或五层结构
- TCP/IP 参考模型没有对网络接口层进行细分，只是一些概念性的描述； OSI 参考模型对服务和协议做了明确的区分
- OSI 参考模型虽然网络划分为七层，但实现起来较困难。TCP/IP 参考模型作为一种简化的分层结构是可以的
- TCP/IP协议去掉表示层和会话层的原因在于会话层、表示层、应用层都是在应用程序内部实现的，最终产出的是一个应用数据包，而应用程序之间是几乎无法实现代码的抽象共享的，这也就造成 OSI 设想中的应用程序维度的分层是无法实现的

三种模型对应关系如下图所示：

区域	TCP/IP 四层模型	TCP/IP 五层模型	OSI 七层模型	单位	地址	功能	对应设备	协议
计算机 高层	应用层	应用层	应用层	应用进程	进程号	应用程序与协议	应用程序（eg: FTP、HTTP）	FTP、NFS
			表示层			数据加密、压缩	编码解码、加密解密	Telnet、SNMP
			会话层			会话的开始、恢复、释放、同步	建立会话，session验证、断点传输	SMTP、DNS
网络 低层	传输层	传输层	传输层	报文/数据段	端口号	端到端的可靠透明传输、保证数据完整性	进程与端口	TCP、UDP
	网络层	网络层	网络层	包/分组	IP地址	服务选择、路径选择、多路复用等（如何选择发送路径、方式）	路由器、防火墙、多层交换机	IP、ICMP、ARP
	网络接口层	数据链路层	数据链路层	帧	mac地址	差错控制、流量控制（规定如何进行01发送不会造成错误）	网卡、网桥、交换机	PPP、SLIP
		物理层	物理层	比特流	bit	光纤、电缆、双绞线连接，传送0/1电信号	中继器、集线器、网线	IEEExxxx

参考文献

- <https://zh.wikipedia.org/wiki/TCP/IP%E5%8D%8F%E8%AE%AE%E6%97%8F>
- <https://zhuanlan.zhihu.com/p/103162095>
- <https://segmentfault.com/a/1190000039204681>
- <https://leetcode-cn.com/leetbook/detail/networks-interview-highlights/>

面试官： 如何理解UDP 和 TCP？ 区别？ 应用场景？

一、UDP

UDP (User Datagram Protocol)，用户数据包协议，是一个简单的「面向数据报的通信协议」，即对应用层交下来的报文，不合并，不拆分，只是在其上面加上首部后就交给了下面的网络层

也就是说无论应用层交给 UDP 多长的报文，它统统发送，一次发送一个报文

而对接收方，接到后直接去除首部，交给上面的应用层就完成任务

UDP 报头包括4个字段，每个字段占用2个字节（即16个二进制位），标题短，开销小

特点如下：

- UDP 不提供复杂的控制机制，利用 IP 提供面向无连接的通信服务
- 传输途中出现丢包，UDP 也不负责重发
- 当包的到达顺序出现乱序时，UDP没有纠正的功能。
- 并且它是将应用程序发来的数据在收到的那一刻，立即按照原样发送到网络上的一种机制。即使是出现网络拥堵的情况，UDP 也无法进行流量控制等避免网络拥塞行为

二、TCP

TCP (Transmission Control Protocol)，传输控制协议，是一种可靠、**「面向字节流的通信协议」**，把上面应用层交下来的数据看成无结构的字节流来发送

可以想象成流水形式的，发送方TCP会将数据放入“蓄水池”（缓存区），等到可以发送的时候就发送，不能发送就等着，TCP会根据当前网络的拥塞状态来确定每个报文段的大小

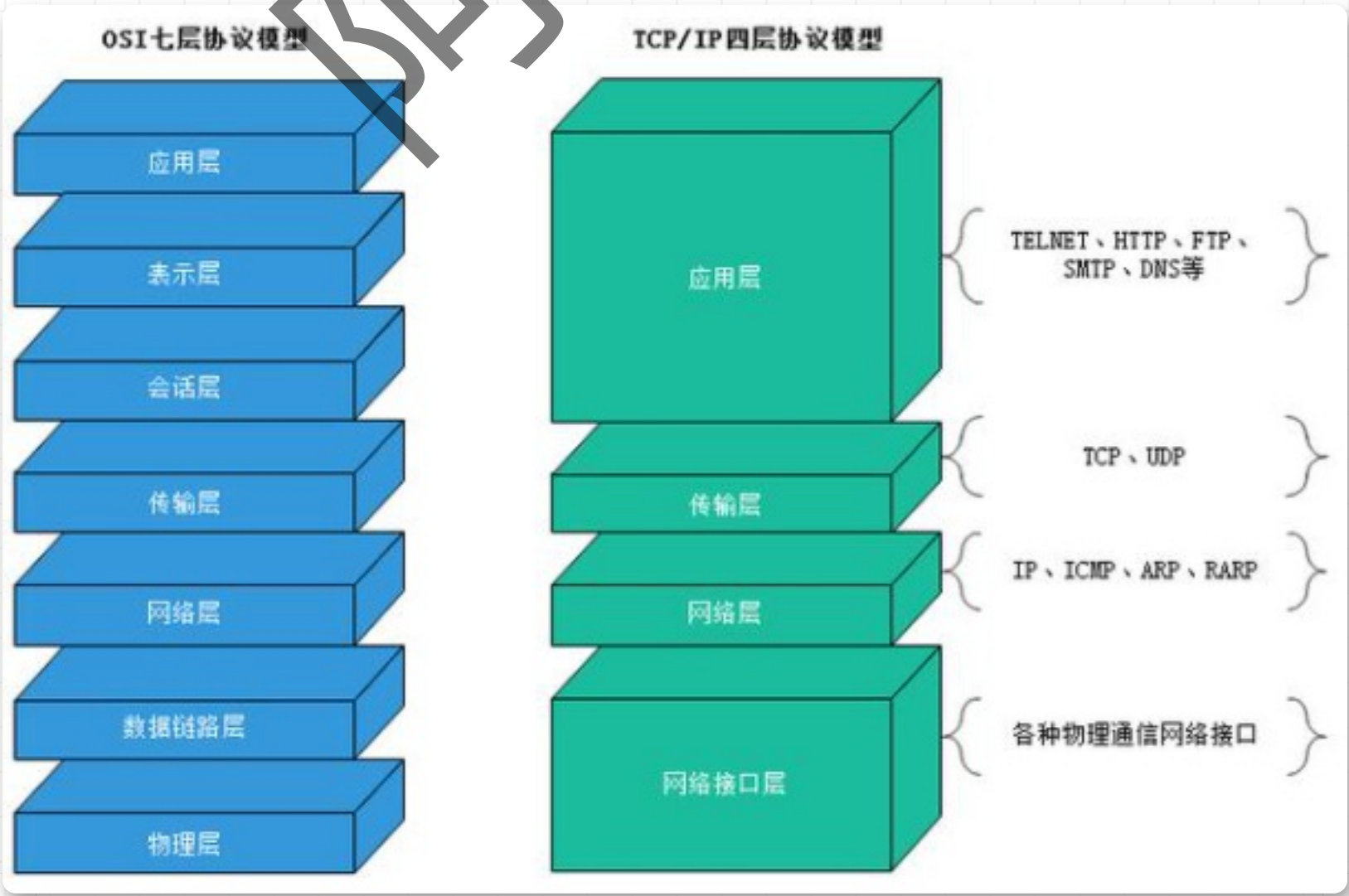
TCP 报文首部有20个字节，额外开销大

特点如下：

- TCP充分地实现了数据传输时各种控制功能，可以进行丢包时的重发控制，还可以对次序乱掉的分包进行顺序控制。而这些在 UDP 中都没有。
- 此外，TCP 作为一种面向有连接的协议，只有在确认通信对端存在时才会发送数据，从而可以控制通信流量的浪费。
- 根据 TCP 的这些机制，在 IP 这种无连接的网络上也能够实现高可靠性的通信（主要通过检验和、序列号、确认应答、重发控制、连接管理以及窗口控制等机制实现）

三、区别

UDP 与 TCP 两者的都位于传输层，如下图所示：



两者区别如下表所示：

	TCP	UDP
可靠性	可靠	不可靠
连接性	面向连接	无连接
报文	面向字节流	面向报文
效率	传输效率低	传输效率高
双共性	全双工	一对一、一对多、多对一、多对多
流量控制	滑动窗口	无
拥塞控制	慢开始、拥塞避免、快重传、快恢复	无
传输效率	慢	快

- TCP 是面向连接的协议，建立连接3次握手、断开连接四次挥手，UDP是面向无连接，数据传输前后不连接连接，发送端只负责将数据发送到网络，接收端从消息队列读取
- TCP 提供可靠的服务，传输过程采用流量控制、编号与确认、计时器等手段确保数据无差错，不丢失。UDP 则尽可能传递数据，但不保证传递交付给对方
- TCP 面向字节流，将应用层报文看成一串无结构的字节流，分解为多个TCP报文段传输后，在目的站重新装配。UDP协议面向报文，不拆分应用层报文，只保留报文边界，一次发送一个报文，接收方去除报文首部后，原封不动将报文交给上层应用
- TCP 只能点对点全双工通信。UDP 支持一对一、一对多、多对一和多对多的交互通信

两者应用场景如下图：

可以看到，TCP 应用场景适用于对效率要求低，对准确性要求高或者要求有链接的场景，而UDP 适用场景为对效率要求高，对准确性要求低的场景

参考文献

- <https://zh.wikipedia.org>
- <https://www.shangmayuan.com/a/a1e3ceb218284cefb95de7fd.html>
- <https://segmentfault.com/a/1190000021815671>

面试官：GET和POST的区别及使用场景

一、GET和POST是什么？

在HTTP协议中，GET和POST是两种常用的请求方法，用于向服务器发送请求并获取响应。它们有以下特点：

GET方法

GET方法用于请求一个指定资源的表示形式，通常用于获取数据，不应该对服务器产生副作用。GET请求的参数会附加在URL后面。

POST方法

POST方法用于向指定的资源提交要被处理的数据，通常会导致服务器上的状态变化或产生副作用。POST请求的参数会包含在请求的正文中。

二、GET和POST的区别

虽然GET和POST都是使用HTTP协议传输数据，但它们在有一些方面有明显的区别：

1. 参数位置

GET请求的参数附加在URL的末尾，形式为：<http://example.com/path?param1=value1¶m2=value2>

POST请求的参数包含在请求的正文中，不会显示在URL中。

2. 数据长度限制

GET请求的参数附加在URL后面，受限于URL的长度限制，一般由浏览器限制，不同浏览器可能有不同的限制，例如IE限制为2083字节。

POST请求的参数在请求的正文中，没有明确的长度限制，一般受到服务器端的配置限制。

3. 安全性

GET请求的参数会显示在URL中，因此对于敏感信息不宜使用GET请求，因为敏感信息可能会被浏览器保存、历史记录或被其他人看到。

POST请求的参数在请求的正文中，相对来说更加安全，但仍然不是绝对安全，因为HTTP是明文传输的，只有使用HTTPS才能加密传输数据。

4. 缓存与历史记录

GET请求会被浏览器主动缓存，而POST请求不会被缓存，除非手动设置。因为GET请求的参数附加在URL后面，浏览器会将URL作为缓存的标识。

5. 幂等性

GET请求是幂等的，多次执行相同的GET请求，服务器的状态不会发生变化。

POST请求不是幂等的，多次执行相同的POST请求可能会产生不同的结果，因为可能会导致服务器状态的变化。

三、GET和POST的使用场景

GET请求主要用于获取数据，因为它是幂等的、可缓存的。常见的使用场景包括：

- 搜索引擎的搜索请求
- 获取网页内容
- 获取资源文件等

POST请求主要用于向服务器提交数据，因为它可能会导致状态变化或产生副作用。常见的使用场景包括：

- 提交表单数据
- 上传文件
- 发表评论等

在实际应用中，要根据具体的需求选择GET或POST方法，合理地使用它们可以提升应用的性能和安全性。

四、示例代码

GET请求示例 (JavaScript)


```
const url = "https://api.example.com/data?id=123";
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error("Error:", error);
  });
```

POST请求示例 (JavaScript)

```
const url = "https://api.example.com/submit";
const data = {
  username: "user123",
  password: "pass456"
};
fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(data)
})
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error("Error:", error);
  });
```

以上示例使用了 `fetch` 函数来发送GET和POST请求，通过Promise链式调用处理响应结果。

参考文献

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- https://www.w3schools.com/tags/ref_httpmethods.asp
- <https://stackoverflow.com/questions/3477333/what-is-the-difference-between-post-and-get>

面试官：说说 HTTP 常见的请求头有哪些？作用？

一、什么是HTTP头字段？

HTTP头字段 (HTTP header fields) 是指在超文本传输协议 (HTTP) 的请求和响应消息中的消息头部分。它们定义了一个HTTP事务中的操作参数。HTTP头部字段可以根据需要自定义，因此可能在Web服务器和浏览器上发现非标准的头字段。

下面是一个HTTP请求的请求头示例：

```
GET /home.html HTTP/1.1
Host: developer.mozilla.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/testpage.html
Connection: keep-alive
Upgrade-Insecure-Requests: 1
If-Modified-Since: Mon, 18 Jul 2016 02:36:04 GMT
If-None-Match: "c561c68d0ba92bb8b0ff2a9199f722e3a621a"
Cache-Control: max-age=0
```

二、常见的请求头字段

以下是一些常见的HTTP请求头字段及其说明：

1. Accept

用于指定能够接受的回应内容类型（Content-Types）。

2. Accept-Charset

用于指定能够接受的字符集。

3. Accept-Encoding

用于指定能够接受的编码方式列表，常见的有gzip、deflate等。

4. Accept-Language

用于指定能够接受的回应内容的自然语言列表，例如en-US表示英语（美国）。

5. Authorization

用于超文本传输协议的认证的认证信息，通常用于进行用户身份验证。

6. Cache-Control

用来指定在请求/响应链中的所有缓存机制都必须遵守的指令，例如no-cache表示不缓存。

7. Cookie

用于服务器通过Set-Cookie发送的一个HTTP Cookie。

8. Content-Length

表示请求体的长度，以八位字节数组（8位的字节）表示。

9. Content-Type

用于指定请求体的多媒体类型，例如application/x-www-form-urlencoded表示表单数据。

10. User-Agent

表示浏览器的浏览器身份标识字符串，用于标识用户使用的浏览器信息。

11. Host

表示服务器的域名，以及服务器所监听的传输控制协议端口号。

12. Referer

表示请求的来源页面，即用户是从哪个页面跳转过来的。

13. Origin

用于发起一个针对跨来源资源共享的请求，表示允许跨域请求。

三、使用场景

通过配合请求头和响应头，可以实现一些常见的功能：

1. 协商缓存

通过请求头的 `If-Modified-Since`、`If-None-Match` 和响应头的 `Last-Modified`、`ETag` 实现缓存的协商，减少不必要的数据传输。

2. 会话状态管理

通过请求头的 `Cookie` 字段，服务器可以在客户端维护会话状态，实现用户登录状态、购物车、个性化设置等功能。

以上是HTTP常见请求头及其作用的简要介绍，HTTP头字段在HTTP通信中扮演着重要的角色，帮助服务器和浏览器进行信息交换和功能实现。

参考文献

- [HTTP头字段 - 维基百科](https://zh.wikipedia.org/wiki/HTTP%E5%A4%B4%E5%AD%97%E6%AE%B5) (<https://zh.wikipedia.org/wiki/HTTP%E5%A4%B4%E5%AD%97%E6%AE%B5>)
- [HTTP Request Headers - MDN Web Docs](https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers) (<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>)

面试官：说说HTTP1.0/1.1/2.0的区别？

一、HTTP1.0

HTTP1.0是HTTP协议的第一个版本，也是第一个在通信中指定版本号的HTTP协议版本。

在HTTP1.0中，浏览器与服务器之间只保持短暂的连接，每次请求都需要与服务器建立一个TCP连接。服务器在完成请求处理后立即断开TCP连接，不保持与客户端的长连接，也不跟踪每个客户端的状态或记录过去的请求。

简单来说，每次与服务器交互都需要新开一个连接，如下图所示：

例如，在解析HTML文件时，如果文件中存在资源文件（如CSS、JS等），就需要创建单独的连接来获取这些资源，最终导致一个HTML文件的访问涉及多次请求和响应，每次请求都需要创建连接、建立连接和关闭连接，这种形式明显造成了性能上的缺陷。

如果需要建立长连接，需要设置一个非标准的Connection字段 `Connection: keep-alive`。

二、HTTP1.1

HTTP1.1在HTTP1.0的基础上进行了改进和优化。

在HTTP1.1中，默认支持长连接（`Connection: keep-alive`），即在一个TCP连接上可以传送多个HTTP请求和响应，减少了建立和关闭连接的消耗和延迟。

通过建立一次连接，多次请求可以由这个连接完成，如下图所示：

这样，在加载HTML文件的时候，文件中的多个请求和响应可以在一个连接中传输，提高了性能。

此外，HTTP1.1还引入了更多的请求头和响应头来完善功能，例如：

- 引入了更多的缓存控制策略，如If-Unmodified-Since、If-Match、If-None-Match等缓存头来控制缓存策略。
- 引入了range头，允许请求资源的某个部分。
- 引入了host头，实现在一台WEB服务器上可以在同一个IP地址和端口号上使用不同的主机名来创建多个虚拟WEB站点。

同时，HTTP1.1还新增了其他的请求方法：PUT、DELETE、OPTIONS等。

三、HTTP2.0

HTTP2.0在相比之前版本，性能上有很大的提升，主要体现在以下几个方面：

1. 多路复用

HTTP2.0复用TCP连接，在一个连接上可以同时发送多个请求或回应，而且不需要按照顺序一一对应，避免了"队头堵塞"的问题。

通过复用连接，在同一个TCP连接里面，客户端可以同时发送多个请求，服务器也可以同时返回多个响应，这样大大减少了连接的建立和关闭次数，提高了并发性能，如下图所示：

2. 二进制分帧

HTTP2.0采用二进制格式传输数据，而非HTTP1.x的文本格式，解析起来更高效。

将请求和响应数据分割为更小的帧，并且它们采用二进制编码，这样数据传输更加紧凑高效。

HTTP2.0中，同域名下所有通信都在单个连接上完成，该连接可以承载任意数量的双向数据流。每个数据流都以消息的形式发送，而消息又由一个或多个帧组成。多个帧之间可以乱序发送，根据帧首部的流标识可以重新组装，这也是多路复用同时发送数据的实现条件。

3. 首部压缩

HTTP2.0使用首部表来跟踪和存储之前发送的键值对，对于相同的数据，不再通过每次请求和响应发送。

首部表在HTTP2.0的连接存续期内始终存在，由客户端和服务端共同渐进地更新。这样就实现了对HTTP1.x中冗余头部信息的压缩，减少了数据传输量，降低了开销。

例如，下图中的两个请求，请求一发送了所有的头部字段，第二个请求则只需要发送差异数据，这样可以减少冗余数据，降低开销。

4. 服务器推送

HTTP2.0引入了服务器推送，允许服务器将一些客户端需要的资源预先推送到客户端。

服务器可以顺便把一些客户端需要的资源一起推送到客户端，如在响应一个页面请求时，可以随同页面的其他资源一起推送。这样免得客户端再次创建连接发送请求到服务器端获取资源，提高了性能。

服务器推送适合用于加载静态资源，如图片、CSS、JS等，如下图所示：

四、总结

HTTP1.0、HTTP1.1和HTTP2.0在性能和功能方面都有很大的区别。

HTTP1.0通过短连接来实现请求和响应，HTTP1.1引

入了长连接和更多的请求头和响应头来优化。

而HTTP2.0则进一步提升了性能，通过多路复用、二进制分帧、首部压缩和服务器推送等特性来优化数据传输和性能表现。

随着HTTP2.0的逐渐普及，网络通信将更加高效快速，为用户提供更好的上网体验。

参考文献

- <https://zh.wikipedia.org/wiki/超文本传输协议#HTTP/1.0>
- <https://www.jianshu.com/p/52d86558ca57>
- <https://segmentfault.com/a/1190000016496448>
- <https://zhuanlan.zhihu.com/p/26559480>

面试官：说说HTTP 常见的状态码有哪些，适用场景？

一、HTTP状态码

HTTP状态码（HTTP Status Code）是用来表示网页服务器超文本传输协议响应状态的3位数字代码。它由RFC 2616规范定义，并得到RFC 2518、RFC 2817、RFC 2295、RFC 2774与RFC 4918等规范扩展。简单来说，HTTP状态码是服务器告诉客户端当前请求响应的状态，通过状态码可以判断和分析服务器的运行状态。

二、常见状态码

HTTP状态码分为五类，每类以不同的数字开头表示不同的响应状态：

1xx - 信息类

1xx状态码表示请求已被接受，需要继续处理。这类响应是临时响应，只包含状态行和某些可选的响应头信息，并以空行结束。常见状态码有：

- 100（继续）：客户端继续发送请求，这是临时响应，用来通知客户端部分请求已经被服务器接收，且仍未被拒绝。客户端应当继续发送请求的剩余部分，或者如果请求已经完成，忽略这个响应。服务器必须在请求完成后向客户端发送一个最终响应。
- 101（切换协议）：服务器根据客户端的请求切换协议，主要用于WebSocket或HTTP/2升级。

2xx - 成功类

2xx状态码表示请求已成功被服务器接收、理解，并接受。常见状态码有：

- 200（成功）：请求已成功，请求所希望的响应头或数据体将随此响应返回。
- 201（已创建）：请求成功并且服务器创建了新的资源。
- 204（无内容）：服务器成功处理请求，但没有返回任何内容。

3xx - 重定向类

3xx状态码表示要完成请求，需要进一步操作。通常，这些状态代码用来重定向。常见状态码有：

- 301（永久重定向）：请求的网页已永久移动到新位置。服务器返回此响应时，会自动将请求者转到新位置。
- 302（临时重定向）：服务器目前从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。
- 304（未修改）：服务器告诉客户端可以直接使用缓存的版本，无需重新请求。
- 307（临时重定向）：服务器目前从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。

4xx - 客户端错误类

4xx状态码代表了客户端看起来可能发生了错误，妨碍了服务器的处理。常见状态码有：

- 400（错误请求）：服务器不理解请求的语法。
- 401（未授权）：请求要求身份验证。对于需要登录的网页，服务器可能返回此响应。
- 403（禁止）：服务器拒绝请求。
- 404（未找到）：服务器找不到请求的网页。

5xx - 服务器错误类

5xx状态码表示服务器无法完成明显有效的请求。这类状态码代表了服务器在处理请求的过程中有错误或者异常状态发生。常见状态码有：

- 500（服务器内部错误）：服务器遇到错误，无法完成请求。
- 502（错误网关）：服务器作为网关或代理，从上游服务器收到无效响应。
- 503（服务不可用）：服务器目前无法使用（由于超载或停机维护）。

三、适用场景

不同的状态码适用于不同的场景，下面给出一些常见的适用场景：

- 200：表示请求成功，服务器正确处理了请求。
- 204：表示服务器正确处理了请求，但没有返回任何内容，通常用于删除操作等。
- 301：表示永久重定向，请求的资源已经被永久移动到新的位置。
- 400：表示客户端请求错误，服务器无法理解请求的语法。
- 403：表示请求被服务器拒绝，没有权限访问该资源。
- 404：表示请求的资源不存在，服务器找不到请求的网页。

不同的状态码对应不同的请求处理情况，客户端可以根据状态码来判断请求是否成功，并做相应处理。

参考文献

- [HTTP状态码 - 维基百科 \(https://zh.wikipedia.org/wiki/HTTP状态码\)](https://zh.wikipedia.org/wiki/HTTP状态码)
- [HTTP状态码详解 \(https://kebingzao.com/2018/10/05/http-status-code/\)](https://kebingzao.com/2018/10/05/http-status-code/)

面试官：说说TCP为什么需要三次握手和四次挥手？

一、TCP三次握手

TCP三次握手是在建立TCP连接时需要进行的过程，用于确认双方的接收能力和发送能力是否正常，以及指定序列号等参数。

握手过程：

1. 第一次握手：客户端发送一个SYN包，指定自己的初始序列号ISN(c)，此时客户端处于SYN_SENT状态。
2. 第二次握手：服务端收到客户端的SYN包后，以自己的SYN包作为应答，确认客户端的SYN，并将客户端的ISN+1作为ACK的值，此时服务端处于SYN_RCVD状态。
3. 第三次握手：客户端收到服务端的SYN包后，发送一个ACK包，值为服务端的ISN+1。此时客户端处于ESTABLISHED状态，服务端也处于ESTABLISHED状态，双方连接建立完成。

通过三次握手，双方确认了彼此的接收和发送能力正常，可以进行正常的数据传输。

为什么需要三次握手？

如果只有两次握手，发送端可以确认自己发送的信息对方能收到，也可以确认对方发送的信息自己能收到，但接收端无法确认自己发送的信息对方是否能收到。因此，需要三次握手来确认双方的接收和发送能力都正常。

另外，三次握手可以防止已失效的连接请求报文段再次传到服务端，导致服务器资源浪费。

二、TCP四次挥手

TCP四次挥手是在断开TCP连接时需要进行的过程，用于正常关闭连接。

挥手过程：

1. 第一次挥手：客户端发送一个FIN包，指定一个序列号。此时客户端处于FIN_WAIT1状态，停止发送数据，等待服务端的确认。
2. 第二次挥手：服务端收到FIN包后，发送一个ACK包，且将客户端的序列号值+1作为ACK包的序列号值，表明已经收到客户端的FIN包。此时服务端处于CLOSE_WAIT状态。
3. 第三次挥手：如果服务端也想断开连接，服务端发送一个FIN包，指定一个序列号。此时服务端处于LAST_ACK状态。
4. 第四次挥手：客户端收到服务端的FIN包后，发送一个ACK包作为应答，且将服务端的序列号值+1作为ACK包的序列号值，此时客户端处于TIME_WAIT状态。需要等待一段时间以确保服务端收到自己的ACK包之后才会进入CLOSED状态，服务端收到ACK包后，就处于关闭连接了，处于CLOSED状态。

四次挥手原因

服务端在收到客户端断开连接FIN包后，并不会立即关闭连接，而是先发送一个ACK包先告诉客户端收到关闭连接的请求，只有当服务器的所有报文发送完毕之后，才发送FIN包断开连接，因此需要四次挥手。

三、总结

TCP三次握手和四次挥手是TCP连接建立和断开过程中必要的步骤，通过这些步骤，可以保证连接的可靠性和正常关闭，确保数据的准确传输和释放服务器资源。

参考文献

- [TCP三次握手与四次挥手 - 知乎 \(https://zhuanlan.zhihu.com/p/53374516\)](https://zhuanlan.zhihu.com/p/53374516)

- [图解TCP三次握手和四次挥手 - SegmentFault \(https://segmentfault.com/a/1190000020610336\)](https://segmentfault.com/a/1190000020610336)

面试官：说说地址栏输入 URL 敲下回车后发生了什么？

一、URL解析和DNS查询

当你在浏览器的地址栏中输入一个URL并敲下回车时，浏览器会首先对URL进行解析。URL通常由多个部分组成，包括协议（例如http或https）、域名或IP地址、端口号（如果未指定则使用默认端口）、路径和查询参数等。

浏览器解析URL后，会提取出其中的域名部分，然后进行DNS查询以将域名转换为对应的IP地址。DNS查询是将域名映射到IP地址的过程，浏览器向本地DNS服务器发送查询请求，本地DNS服务器会向根DNS服务器逐级查询，直到找到目标域名对应的IP地址。一旦得到IP地址，浏览器就可以向服务器发送请求。

二、TCP连接和HTTP请求

接下来，浏览器需要与目标服务器建立TCP连接。TCP连接是一种面向连接的传输协议，确保数据的可靠传输。在建立TCP连接时，浏览器会经历一个三次握手的过程，确保客户端和服务端之间的连接状态正常。

当TCP连接建立后，浏览器会发送HTTP请求到服务器。HTTP请求包括请求行、请求头和请求体。请求行包含请求方法（例如GET、POST等）和路径信息。请求头包含一些额外的信息，例如浏览器的User-Agent、Accept-Encoding等。请求体包含一些提交的数据，例如表单数据或JSON数据等。

三、服务器处理和HTTP响应

服务器收到HTTP请求后，会根据请求的内容进行处理。服务器可能会读取请求体中的数据，查询数据库，执行业务逻辑等操作。

处理完成后，服务器会生成HTTP响应并发送回浏览器。HTTP响应包括状态行、响应头和响应体。状态行包含状态码，表示服务器处理请求的结果。响应头包含一些附加信息，例如服务器类型、响应时间等。响应体包含实际返回的数据，例如HTML页面、JSON数据等。

四、页面渲染

浏览器收到服务器的响应后，会对返回的资源进行解析和渲染。如果响应是HTML页面，浏览器会构建DOM树和CSS规则树，并最终生成渲染树。然后浏览器会对渲染树进行布局和绘制，最终将页面呈现给用户。

在页面渲染的过程中，浏览器还会下载页面中引用的其他资源，例如CSS文件、JavaScript文件、图片等，并对这些资源进行相同的解析和渲染过程。

1. 解析HTML

浏览器从服务器接收到HTML文档后，会进行解析，构建DOM树（文档对象模型）。DOM树是文档的逻辑结构表示，每个HTML标签都会被解析成DOM树的一个节点，标签之间的嵌套关系则对应着DOM树中的父子关系。

2. 解析CSS

解析CSS样式表，生成CSS规则树（样式表对象模型）。CSS规则树描述了文档中各个元素应用的样式信息。DOM树和CSS规则树结合后形成渲染树（Render Tree），渲染树只包含需要显示的节点和其对应的样式信息。

3. 布局（Layout）

渲染树中的每个节点都有对应的几何信息，例如位置、大小等。布局阶段会根据渲染树中每个节点的几何信息计算节点在页面中的精确位置，确定每个元素在屏幕上的显示位置。

4. 绘制（Paint）

在布局完成后，浏览器将根据渲染树和布局阶段得到的几何信息来绘制页面。这个过程将把页面的每个节点转换为屏幕上的实际像素，创建一个包含所有像素信息的位图。

5. 合成（Composite）

绘制阶段完成后，浏览器会将各个图层（Layer）按照合成顺序进行合并，形成最终的页面图像。合成是将各个图层的像素信息进行组合，以最终形成用户所见的页面。

值得注意的是，现代浏览器为了提高性能和用户体验，会采用一些优化技术，例如异步加载脚本、懒加载图片、预渲染等。这些优化技术可以减少页面加载时间和渲染时间，提高页面的显示速度和响应性。

页面渲染是整个过程中较为复杂的一部分，它涉及到浏览器的多个模块协同工作，确保用户能够快速、流畅地浏览网页内容。

五、总结

整个过程可以总结为：

1. URL解析和DNS查询，将域名解析为IP地址；
2. 建立TCP连接，确保数据传输的可靠性；
3. 发送HTTP请求，向服务器请求资源；
4. 服务器处理请求并发送HTTP响应；
5. 浏览器解析和渲染页面，显示给用户。

这些步骤使得用户可以通过浏览器访问互联网上的各种资源，从而实现网页浏览、数据传输等功能。

参考文献

- <https://github.com/febobo/web-interview/issues/141>
- <https://zhuanlan.zhihu.com/p/80551769>

面试官：说说对WebSocket的理解？应用场景？

一、WebSocket 是什么？

WebSocket 是一种网络传输协议，位于 OSI 模型的应用层。它允许在单个 TCP 连接上进行全双工通信，实现了客户端与服务器之间的双向数据传输，从而能更好地节省服务器资源和带宽，并达到实时通信的效果。

在传统的 HTTP 协议中，客户端需要不断发送请求以获取服务器的响应数据，这种轮询的方式对服务器和网络资源产生较大的负担。而 WebSocket 的出现改变了这种情况，通过一次握手连接，客户端和服务器之间可以创建持久性的连接，随时进行双向数据交换，大大降低了通信的延迟和资源消耗。

二、WebSocket 的特点

1. 全双工通信

WebSocket 支持全双工通信，允许数据在两个方向上同时传输，客户端和服务端可以实时地发送和接收数据，达到瞬时同步的效果。

2. 二进制帧结构

WebSocket 采用二进制帧结构，与 HTTP 完全不兼容。相比起 HTTP/2，WebSocket 更侧重于实时通信，不像 HTTP/2 那样定义流和多路复用等特性，因为 WebSocket 自身已经是全双工通信，不需要这些特性。

3. 协议名和握手

WebSocket 引入了 ws 和 wss 分别代表明文和密文的 WebSocket 协议，且默认端口使用 80 或 443，与 HTTP 的端口一致。在连接建立时，客户端需要发送协议升级请求并进行握手，服务端返回接受握手请求的响应，完成连接的建立。

4. 优点

- 较少的控制开销：相对于 HTTP 每次请求都需要携带完整的头部，WebSocket 的数据包头部较小，减少了控制开销。
- 更强的实时性：相对于 HTTP 请求需要等待客户端发起请求才能响应，WebSocket 实现了持久连接，实时性更好。
- 保持连接状态：WebSocket 连接建立后，客户端和服务端之间可以保持连接状态，省去了每次请求都要携带身份验证的过程。
- 更好的二进制支持：WebSocket 定义了二进制帧，更好地处理二进制内容。
- 支持扩展：用户可以扩展 WebSocket 协议，实现自定义的子协议，增加了灵活性。
- 更好的压缩效果：在适当的扩展支持下，WebSocket 可以沿用之前内容的上下文，在传递类似的数据时，可以显著提高压缩率。

三、应用场景

WebSocket 的实时通信特性使得它在许多场景下得到广泛应用：

- 弹幕：在直播或视频网站中，用户可以实时发送弹幕评论，通过 WebSocket 实时将弹幕内容显示在视频画面上。
- 媒体聊天：实现在线即时通信功能，例如在线聊天室、即时消息应用等。
- 协同编辑：多个用户可以同时编辑同一个文档，实现实时协作编辑。
- 基于位置的应用：例如地图应用中，实时显示用户的位置信息。
- 体育实况更新：在体育比赛进行时，实时更新比分和比赛进展。
- 股票基金报价实时更新：股票和基金价格实时更新，及时推送最新的行情信息。

总的来说，WebSocket 在需要实时通信和即时更新数据的场景下都能发挥出很大的作用，提升用户体验，减少服务器压力，逐渐成为现代 Web 应用中不可或缺的技术之一。

使用

当使用 WebSocket 进行通信时，通常需要在客户端和服务端分别实现 WebSocket 的功能。下面我将给出一个简单的示例，展示如何使用 JavaScript 在客户端建立 WebSocket 连接并发送和接收消息，以及如何在 Python 中创建一个简单的 WebSocket 服务器。

在客户端使用 WebSocket

```
<!DOCTYPE html>
<html>
<head>
  <title>WebSocket Client</title>
</head>
<body>
  <input type="text" id="messageInput" placeholder="Enter your message">
  <button onclick="sendMessage()">Send</button>
  <div id="messageArea"></div>

  <script>
    const socket = new WebSocket("ws://your_server_address");

    socket.onopen = function() {
      showMessage("WebSocket connection established.");
    };

    socket.onmessage = function(event) {
      showMessage("Received: " + event.data);
    };

    function sendMessage() {
      const messageInput = document.getElementById("messageInput");
      const message = messageInput.value;
      socket.send(message);
      showMessage("Sent: " + message);
      messageInput.value = "";
    }

    function showMessage(message) {
      const messageArea = document.getElementById("messageArea");
      messageArea.innerHTML += "<p>" + message + "</p>";
    }
  </script>
</body>
</html>
```

在上面的示例中，首先我们创建了一个 `WebSocket` 对象，传入服务器地址 `ws://your_server_address`。然后，我们监听 `WebSocket` 的 `onopen` 事件，当连接建立时会显示连接已经建立。监听 `onmessage` 事件，当接收到消息时会在页面上显示收到的消息。通过输入框和按钮可以向服务器发送消息，点击按钮时调用 `sendMessage()` 函数将输入框中的消息发送给服务器，并将消息显示在页面上。

在服务器端使用 WebSocket

在 `Node.js` 中，你可以使用 `ws` 模块来创建 `WebSocket` 服务器。这个模块提供了 `WebSocket` 的实现，使你能够轻松地创建一个 `WebSocket` 服务器，并与客户端进行实时通信。

下面是使用 `Node.js` 创建 `WebSocket` 服务器的代码示例：

首先，确保你已经安装了 `ws` 模块：

```
npm install ws
```

然后，创建一个 Node.js 文件 `websocket_server.js`，包含以下代码：

```
const WebSocket = require('ws');

const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  console.log('Client connected');

  ws.on('message', (message) => {
    console.log('Received: ' + message);
    // 在这里你可以对接收到的消息进行处理
    // 例如，将消息广播给所有连接的客户端
    wss.clients.forEach((client) => {
      if (client !== ws && client.readyState === WebSocket.OPEN) {
        client.send('Broadcast: ' + message);
      }
    });
  });

  ws.on('close', () => {
    console.log('Client disconnected');
  });
});

console.log('WebSocket server started at ws://localhost:8080');
```

在上面的代码中，我们使用 `ws` 模块创建了一个 WebSocket 服务器，监听在本地地址 `ws://localhost:8080` 上。当有客户端连接到服务器时，`connection` 事件会被触发，并创建一个 WebSocket 实例 `ws`。然后，我们监听 `message` 事件，当客户端发送消息时会收到该事件，并在控制台上显示接收到的消息。在这个示例中，我们将收到的消息广播给所有连接的客户端，除了发送消息的客户端本身。

运行 WebSocket 服务器：

```
node websocket_server.js
```

现在，服务器已经在本地地址 `ws://localhost:8080` 上运行，客户端可以连接到该地址，并与服务器进行实时通信。

参考文献

- [WebSocket - Wikipedia \(https://en.wikipedia.org/wiki/WebSocket\)](https://en.wikipedia.org/wiki/WebSocket)
- [9 Killer Uses for WebSockets - Open Source China \(https://www.oschina.net/translate/9-killer-uses-for-websockets\)](https://www.oschina.net/translate/9-killer-uses-for-websockets)

Date类：日期和时间处理

引言

在JavaScript中，`Date`类是用于处理日期和时间的内置类。它提供了一系列属性和方法，使我们能够操作和管理日期、时间、时区等相关信息。本文将详细介绍 `Date` 类的属性、常用方法以及应用场景，并提供相应的代码示例。

1. Date类的属性

`Date` 类具有以下常用属性：

- `Date.prototype.constructor`：返回创建对象实例的构造函数。对于 `Date` 类实例，该属性始终指向 `Date` 构造函数。
- `Date.prototype.toString()`：返回一个表示日期和时间的字符串，通常以本地时间格式显示。
- `Date.prototype.toISOString()`：返回一个符合ISO 8601标准的日期和时间字符串，格式为 `YYYY-MM-DDTHH:mm:ss.sssZ`。
- `Date.prototype.valueOf()`：返回一个表示日期对象的原始值的数值，即自1970年1月1日午夜（格林威治时间）以来经过的毫秒数。

2. Date类的常用方法

2.1 日期和时间获取方法

- `Date.prototype.getFullYear()`：获取年份（四位数）。
- `Date.prototype.getMonth()`：获取月份，返回值范围为0（一月）到11（十二月）。
- `Date.prototype.getDate()`：获取日期，返回值范围为1到31。
- `Date.prototype.getHours()`：获取小时数，返回值范围为0到23。
- `Date.prototype.getMinutes()`：获取分钟数，返回值范围为0到59。
- `Date.prototype.getSeconds()`：获取秒数，返回值范围为0到59。
- `Date.prototype.getMilliseconds()`：获取毫秒数，返回值范围为0到999。

2.2 日期和时间设置方法

- `Date.prototype.setFullYear(year[, month[, day]])`：设置年份。
- `Date.prototype.setMonth(month[, day])`：设置月份。
- `Date.prototype.setDate(day)`：设置日期。
- `Date.prototype.setHours(hour[, min[, sec[, ms]]])`：设置小时数。
- `Date.prototype.setMinutes(min[, sec[, ms]])`：设置分钟数。
- `Date.prototype.setSeconds(sec[, ms])`：设置秒数。
- `Date.prototype.setMilliseconds(ms)`：设置毫秒数。

2.3 格式化方法

- `Date.prototype.toLocaleDateString()`：返回一个表示日期部分的字符串，根据本地时间格式化。

- `Date.prototype.toLocaleTimeString()`：返回一个表示时间部分的字符串，根据本地时间格式化。
- `Date.prototype.toLocaleString()`：返回一个表示日期和时间的字符串，根据本地时间格式化。

2.4 日期和时间计算方法

- `Date.prototype.getTime()`：返回一个表示日期对象的时间值，即自1970年1月1日午夜（格林威治时间）以来经过的毫秒数。
- `Date.prototype.setTime(timeValue)`：设置日期对象的时间值。
- `Date.prototype.getTimezoneOffset()`：返回当前系统时区与

UTC之间的时间差，以分钟为单位。

- `Date.prototype.addDays(days)`：在当前日期基础上增加指定天数。
- `Date.prototype.addMonths(months)`：在当前日期基础上增加指定月份数。
- `Date.prototype.addYears(years)`：在当前日期基础上增加指定年份数。

3. Date类的应用场景

`Date`类在JavaScript中广泛应用于以下场景：

- 「日期和时间处理」：`Date`类提供了丰富的方法来处理日期和时间，包括日期格式化、日期比较、日期计算等。这在开发中经常需要对日期和时间进行操作的场景中非常有用，如日历应用、倒计时、时间轴等。
- 「时区处理」：`Date`类支持获取当前系统时区与UTC之间的时间差，以及设置特定时区的日期和时间。这对于全球化的应用、跨时区的事件调度、时区转换等非常重要。
- 「日期和时间展示」：通过`Date`类提供的方法，我们可以根据本地时间格式将日期和时间展示给用户。这在用户界面的日期选择、消息时间显示等场景中非常常见。
- 「日期的存储和传输」：在与服务器进行数据交互时，常常需要将日期数据存储或传输。`Date`类提供了获取日期的时间值、转换为ISO字符串等方法，方便数据的存储和传输。

Date类：日期和时间处理

4. 常用的Date方法实现

下面是一些常用的`Date`方法的实现代码示例，以展示它们的基本用法：

4.1 格式化日期和时间

4.1.1 实现format方法

```
Date.prototype.format = function(format) {  
  const year = this.getFullYear();  
  const month = String(this.getMonth() + 1).padStart(2, '0');  
  const day = String(this.getDate()).padStart(2, '0');  
  const hours = String(this.getHours()).padStart(2, '0');  
  const minutes = String(this.getMinutes()).padStart(2, '0');  
  const seconds = String(this.getSeconds()).pad  
  
  Start(2, '0');  
  
  format = format.replace('YYYY', year);  
  format = format.replace('MM', month);  
  format = format.replace('DD', day);  
  format = format.replace('HH', hours);  
  format = format.replace('mm', minutes);  
  format = format.replace('ss', seconds);  
  
  return format;  
};  
  
// 使用示例  
const date = new Date();  
const formattedDate = date.format('YYYY-MM-DD HH:mm:ss');  
console.log(formattedDate);
```

4.1.2 实现toISOString方法

```
Date.prototype.toISOString = function() {  
  const year = this.getFullYear();  
  const month = String(this.getMonth() + 1).padStart(2, '0');  
  const day = String(this.getDate()).padStart(2, '0');  
  
  return `${year}-${month}-${day}`;  
};  
  
// 使用示例  
const date = new Date();  
const isoDate = date.toISOString();  
console.log(isoDate);
```

4.2 计算两个日期之间的天数差

```
Date.prototype.getDaysDiff = function(otherDate) {  
  const oneDay = 24 * 60 * 60 * 1000; // 一天的毫秒数  
  const diffInTime = Math.abs(this - otherDate);  
  const diffInDays = Math.round(diffInTime / oneDay);  
  
  return diffInDays;  
};
```

// 使用示例

```
const date1 = new Date('2022-01-01');  
const date2 = new Date('2022-01-10');  
const daysDiff = date1.getDaysDiff(date2);  
console.log(daysDiff); // 输出 9
```

4.3 获取当前月份的第一天和最后一天

```
Date.prototype.getFirstDayOfMonth = function() {  
  const year = this.getFullYear();  
  const month = this.getMonth();  
  
  return new Date(year, month, 1);  
};
```

```
Date.prototype.getLastDayOfMonth = function() {  
  const year = this.getFullYear();  
  const month = this.getMonth() + 1;  
  
  return new Date(year, month, 0);  
};
```

// 使用示例

```
const date = new Date();  
const firstDayOfMonth = date.getFirstDayOfMonth();  
const lastDayOfMonth = date.getLastDayOfMonth();  
console.log(firstDayOfMonth);  
console.log(lastDayOfMonth);
```

总结

本文介绍了 `Date` 类的属性、应用场景，并提供了一些常用的 `Date` 方法的实现代码示例。`Date` 类在 JavaScript 中用于处理日期和时间相关的操作非常重要，掌握其基本用法能够帮助我们更好地处理和管理日期和时间。通过逐步学习和实践，我们可以在实际项目中灵活运用 `Date` 类，满足各种日期和时间处理的需求。

参考资料

- [MDN Web Docs: Date](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date) (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date)
- [JavaScript Date Object](https://www.w3schools.com/jsref/jsref_obj_date.asp) (https://www.w3schools.com/jsref/jsref_obj_date.asp)
- [ECMAScript® 2021 Language Specification - Date Objects](#)

<https://tc39.es/ecma262/#sec-date-objects>

JS中的异步编程与Promise

一、JavaScript的异步编程机制

在了解JavaScript的异步机制之前，我们首先需要理解JavaScript是一种单线程语言。单线程就意味着所有的任务需要按照顺序一次执行，如果前一个任务没有完成，后一个任务就无法开始。这个特性在执行大量或耗时任务时可能会导致阻塞或者界面卡死，这显然是不可取的。

为了解决这个问题，JavaScript引入了异步编程的机制。简单地说，异步就是你现在发出了一个“命令”，但是并不等待这个“命令”完成，而是继续执行下一个“命令”。只有在“听到”之前的那个“命令”完成了的消息时，才会回过头来处理这个“命令”的结果。这就是所谓的异步编程。

二、事件循环（Event Loop）和任务队列（Task Queue）

这种异步的机制是如何实现的呢？关键在于事件循环（Event Loop）和任务队列（Task Queue）。

事件循环是 JavaScript 内部的一个处理过程，系统会在此处不断地循环等待，检查任务队列中是否有任务，如果有，就处理它。

而任务队列，就是一个存储待处理任务的队列，当我们使用 `setTimeout`、`setInterval`、`ajax`等API时，实际上是向任务队列中添加了一个任务。

当主线程空闲时（也就是同步任务都执行完毕），便会去看任务队列里有没有任务，如果有，便将其取出执行；没有的话，则继续等待。

这个模型可以简单地用下面的代码表示：

```
while (true) {  
  let task = taskQueue.pop();  
  execute(task);  
}
```

三、宏任务和微任务

在任务队列中，任务被分为两类：宏任务（MacroTask）和微任务（MicroTask）。两者的区别在于，宏任务在下一轮事件循环开始时执行，微任务在本轮事件循环结束时执行。这意味着微任务的优先级高于宏任务。

常见的宏任务有：`script`全文（可以看作一种宏任务）、`setTimeout`、`setInterval`、`setImmediate`（Node.js 环境）、`I/O`、`UI渲染`。

常见的微任务有：`Promise`、`process.nextTick`（Node.js环境）、`MutationObserver`(html5新特性)。

事件循环的顺序，决定了 JavaScript 代码的执行顺序。过程如下：

- 执行同步代码，这属于宏任务
- 执行栈为空，查询是否有微任务需要执行
- 执行所有微任务
- 必要的话渲染UI
- 然后开始下一轮 Event loop，执行宏任务中的异步代码

代码示例如下：

```
console.log('script start'); // 宏任务

setTimeout(function() {
  console.log('setTimeout'); // 宏任务
}, 0);

Promise.resolve().then(function() {
  console.log('promise1'); // 微任务
}).then(function() {
  console.log('promise2'); // 微任务
});

console.log('script end'); // 宏任务
```

输出顺序为：script start -> script end -> promise1 -> promise2 -> setTimeout。这是因为 JavaScript 执行机制决定了微任务比宏任务优先执行。

四、requestAnimationFrame

requestAnimationFrame 是一个优化动画效果的函数，也有它在事件循环中的位置。

requestAnimationFrame 的调用是有频率限制的，在大多数浏览器里，这个频率是 60Hz，也就是说，每一次刷新闻隔为 $1000/60 \approx 16.7\text{ms}$ 。requestAnimationFrame 的执行时机是在下一次重绘之前，而不是立即执行。

requestAnimationFrame 的优点是由系统来决定回调函数的执行时机。如果系统忙到一定程度，可能会两次“刷新”之间多次执行回调函数，这时就可以省略掉一些回调函数的执行。这种机制可以有效节省 CPU 开销，提高系统的性能。

requestAnimationFrame 的位置在事件循环中的具体位置是视浏览器的实现而定，但一般来说，它在宏任务执行完，渲染之前，这使得其可以获取到最新的布局 and 样式信息。

五、Promise 的发展

Promise 对象代表一个异步操作的最终完成（或失败）及其结果值。一个 Promise 处于以下状态之一：

- pending: 初始状态，既不是成功，也不是失败状态。
- fulfilled: 意味着操作成功完成。
- rejected: 意味着操作失败。

一个 promise 必须处于一种状态：fulfilled、rejected 或 pending。一个 promise 的状态在 settle 之后就不能再改变。

Promise 起初是由社区提出并实现的，最早的版本是由 Kris Kowal 提出的 Q 库，后来被 ES6 正式接受，并成为了浏览器的原生对象。

Promise 主要解决了两类问题：

- 异步操作的一致性問題：无论异步操作是同步完成还是异步完成，使用 Promise 对象的 then 方法都可以以同样的方式进行处理。
- 回调地狱问题：回调地狱指的是多层嵌套的回调函数，导致代码难以维护和理解。Promise 可以通过链式调用的方式，解决回调地狱问题。

我们可以通过下面的代码示例来看一下 Promise 是如何工作的：

```
let promise = new Promise(function(resolve, reject) {  
  // 异步处理  
  // 处理结束后、调用resolve 或 reject  
});  
  
promise.then(function(value) {  
  // success  
}, function(error) {  
  // failure  
});
```

Promise 的状态一旦改变，就会一直保持那个状态，不会再次改变。这个特性可以让我们有序地处理异步操作的结果，避免出现复杂的状态判断。

以上是关于 JavaScript 中异步编程、事件循环、任务队列、宏任务、微任务，以及 requestAnimationFrame 在事件循环的位置，Promise 的发展和如何解决回调地狱的详细介绍。对于 JavaScript 的异步编程机制，我们应该有了全面深入的了解。

参考资料

1. [MDN文档 - 使用 Promises \(https://developer.mozilla.org/zh-CN/docs/Web/JavaScript/Guide/Using_promises\)](https://developer.mozilla.org/zh-CN/docs/Web/JavaScript/Guide/Using_promises)
2. [MDN文档 - Window.requestAnimationFrame\(\) \(https://developer.mozilla.org/zh-CN/docs/Web/API/Window/requestAnimationFrame\)](https://developer.mozilla.org/zh-CN/docs/Web/API/Window/requestAnimationFrame)

JavaScript中的Generator函数与其在实现Async/Await的应用

在JavaScript的世界里，异步编程是一个核心的主题，而Generator函数和Async/Await则是它的重要组成部分。这篇文章将深入讨论Generator函数和它在实现Async/Await中的作用，帮助你更深入的理解这两个重要概念。

1. Generator函数的基础

在ES6（ECMAScript 2015）中，JavaScript引入了一种新的函数类型：Generator函数。Generator函数是可以暂停执行并在稍后恢复的特殊函数，这种行为由yield关键字控制。当函数遇到yield语句时，它将暂停执行，并将紧跟在yield后的值作为返回结果。下面是一个简单的示例：

```
function* generatorFunction() {  
  yield 'Hello, '  
  yield 'World!'  
}  
  
const generator = generatorFunction();  
  
console.log(generator.next().value); // 'Hello, '  
console.log(generator.next().value); // 'World!'  
console.log(generator.next().done); // true
```

在这个例子中，generatorFunction 是一个Generator函数。调用这个函数不会直接执行函数体内的代码，而是返回一个Generator对象。调用Generator对象的next方法，函数体内的代码将从头开始执行，或者从上一次yield语句处继续执行，直到遇到下一个yield语句。每次调用next方法，都会返回

一个对象，包含 `value` 和 `done` 两个属性。`value` 属性是 `yield` 语句后面的值，`done` 属性表示函数是否执行完成。

这种暂停执行的特性使得 Generator 函数能够以一种完全不同的方式来编写和理解代码，尤其是在处理复杂的异步逻辑时。

2. Generator函数与异步操作

Generator 函数的真正威力在于它能以同步的方式来编写异步代码。通过使用 `yield` 关键字，我们可以暂停函数的执行，等待异步操作完成，然后再继续执行。

这是一个使用 Generator 函数处理异步操作的例子：

```
function* fetchUser(userId) {
  const response = yield fetch(`https://api.example.com/users/${userId}`);
  const user = yield response.json();
  return user;
}

const generator = fetchUser(1);
const responsePromise = generator.next().value;

responsePromise.then(response => {
  const userPromise = generator.next(response).value;
  userPromise.then(user => generator.next(user));
});
```

在这个例子中，我们首先发起一个网络请求来获取用户信息。这是一个异步操作，但是使用 `yield` 关键字，我们可以将其转化为一个同步操作。网络请求完成后，我们获取响应并解析为 JSON。这也是一个异步操作，但是我们同样可以使用 `yield` 关键字来将其转化为同步操作。

3. 使用Generator函数实现Async/Await

在 JavaScript 中，Async/Await 是一种处理异步操作的新方法，它基于 Promise 和 Generator 函数。实际上，我们可以使用 Generator 函数来模拟 Async/Await 的行为。

首先，我们需要一个处理 Generator 函数的辅助函数，来自动执行 Generator 函数：

```
function asyncGenerator(generatorFunc) {
  return function (...args) {
    const generator = generatorFunc(...args);

    function handle(result) {
      if (result.done) return Promise.resolve(result.value);
      return Promise.resolve(result.value)
        .then(res => handle(generator.next(res)))
        .catch(err => handle(generator.throw(err)));
    }

    return handle(generator.next());
  };
}
```


这个 `asyncGenerator` 函数接受一个 Generator 函数作为参数，返回一个新的函数。当这个新的函数被调用时，它首先创建一个 Generator 对象。然后，它定义了一个 `handle` 函数来处理 Generator 对象的返回结果。如果 Generator 函数已经执行完毕，它将返回一个解析为最后返回值的 Promise；如果 Generator 函数还未执行完毕，它将处理当前的 Promise，等待 Promise 解析完成后再次调用 `handle` 函数。这样，我们就可以像使用 Async/Await 那样使用 Generator 函数。

接下来，我们可以使用 `asyncGenerator` 函数来改写前面的 `fetchUser` 函数：

```
const fetchUser = asyncGenerator(function* (userId) {
  const response = yield fetch(`https://api.example.com/users/${userId}`);
  const user = yield response.json();
  return user;
});

fetchUser(1).then(user => console.log(user));
```

这段代码的行为与使用 Async/Await 完全相同。实际上，Async/Await 在底层就是使用了类似的机制。

以上就是关于 JavaScript 中的 Generator 函数以及其在实现 Async/Await 中的应用的详细讨论。理解和掌握这些概念对于编写高效、易读的 JavaScript 代码具有重要的意义。

JavaScript 中的编码

在编写 JavaScript 代码时，我们常常需要处理 URLs，这时候理解 JavaScript 中的 `escape`，`encodeURIComponent` 和 `encodeURIComponent` 函数就显得尤为重要。这些函数用于将特殊字符转化为能在 URL 中安全传输的形式。本文将详细介绍这三个函数的用法和区别，以帮助你更准确的处理 URL 编码问题。

1. escape 函数

首先，我们来了解一下 `escape` 函数。这是一个老旧的函数，现在已经不再推荐使用，因为它不能处理所有的 Unicode 字符。`escape` 函数会将传入的字符串转化为十六进制的 escape 序列，这样的序列以 `%` 开头。

然而，这个函数只能正确处理 ASCII 字符（字符代码小于等于 255 的字符）。对于 ASCII 字符代码大于 255 的字符，`escape` 函数会先将其转化为 Unicode 转义序列（例如，`\u20AC`），然后再对这个转义序列进行编码。这种处理方式会导致一些问题。比如，对于欧元符号（`€`），它的 Unicode 代码是 `20AC`，`escape` 函数会将其转化为 `%u20AC`，而不是正确的 `%E2%82%AC`。

因此，我们不应该再使用 `escape` 函数来处理 URL 编码。

2. encodeURIComponent 函数

接下来，我们来看看 `encodeURIComponent` 函数。这个函数用于编码完整的 URL。它会将非法的 URL 字符转化为各自的十六进制表示，以 `%` 开头。

然而，`encodeURIComponent` 函数并不会对所有的字符进行编码。一些在 URL 中有特殊含义的字符，例如 `/`，`:`，`#` 等，以及 ASCII 字母，数字和一些符号（`- _ . ! ~ * ' ( )`），不会被 `encodeURIComponent` 函数编码。这是因为这些字符在 URL 中是合法的，可以直接使用。

下面是一个 `encodeURIComponent` 函数的例子：

```
const url = 'https://example.com/Hello World!';
console.log(encodeURIComponent(url)); // https://example.com/Hello%20World!
```

在这个例子中，`encodeURIComponent` 函数将空格字符编码为 `%20`，因为空格在URL中是不合法的。而其他的字符，如 `/` 和 `:` 等，都没有被编码。

3. encodeURIComponent函数

最后，我们来看看 `encodeURIComponent` 函数。这个函数用于编码URL的组成部分，比如查询参数。它会将所有非法的URL字符以及一些有特殊含义的字符（如 `/`，`:`，`#` 等）转化为各自的十六进制表示。

这意味着 `encodeURIComponent` 函数会对更多的字符进行编码。在大多数情况下，我们都应该使用 `encodeURIComponent`

函数来编码URL的组成部分。

下面是一个 `encodeURIComponent` 函数的例子：

```
const query = '/Hello World!';
console.log(encodeURIComponent(query)); // %2FHello%20World%21
```

在这个例子中，`encodeURIComponent` 函数将 `/` 和空格字符都编码了，因为这些字符在URL的查询参数中都是不合法的。

4. 总结

总的来说，当我们需要编码完整的URL时，应该使用 `encodeURI` 函数；而当我们需要编码URL的组成部分，比如查询参数，应该使用 `encodeURIComponent` 函数。不再推荐使用 `escape` 函数，因为它不能正确处理所有的字符。

理解和掌握这些函数的用法和区别对于正确处理URL编码问题来说是非常重要的。

JavaScript事件流：深入理解事件处理和传播机制

引言

JavaScript中的事件流是一种机制，用于描述和处理事件在DOM树中的传播过程。了解事件流的属性和工作原理对于编写高效的事件处理代码和实现复杂的交互功能至关重要。本文将详细介绍JavaScript事件流的发展流程、属性以及应用场景，并提供一些代码示例和引用资料，帮助读者深入理解并应用这一重要的前端技术。

1. 事件流的发展流程

事件流在前端的发展过程中经历了一些变化和演进。下面简要介绍了事件流的发展历程：

1.1 传统的DOM0级事件

在早期的JavaScript中，事件处理是通过在DOM元素上直接定义事件处理属性来实现的，称为DOM0级事件。例如，可以通过为按钮元素的 `onclick` 属性赋值一个函数来定义点击事件的处理程序。

```
const button = document.getElementById('myButton');
button.onclick = function() {
  console.log('按钮被点击');
};
```

这种方式简单直接，但是有一个缺点是无法同时为一个元素的同一个事件类型添加多个处理程序。

1.2 DOM2级事件和addEventListener方法

随着DOM2级事件的引入，新增了 `addEventListener` 方法，提供了更强大和灵活的事件处理能力。`addEventListener` 方法允许为一个元素的同一个事件类型添加多个处理程序，并且可以控制事件的捕获阶段。

```
const button = document.getElementById('myButton');
button.addEventListener('click', function() {
  console.log('按钮被点击');
});
```

通过 `addEventListener` 方法，可以在一个元素上同时添加多个处理程序，而且可以使用 `removeEventListener` 方法移除指定的处理程序。

1.3 W3C DOM3级事件

W3C DOM3级事件进一步扩展了事件的类型和属性，引入了更多的事件类型和特性，以满足不断增长的前端开发需求。DOM3级事件规范定义了新的事件类型，如滚动事件、触摸事件、过渡事件等，以及一些新的事件属性和方法，提供更丰富的事件处理能力。

```
const element = document.getElementById('myElement');
element.addEventListener('scroll', function(event) {
  console.log('元素滚动事件');
});
```

DOM3级事件的引入丰富了事件处理的能力，使得开发者可以更灵活地响应各种类型的事件。

1.4 React与Virtual DOM

随着React等前端框架的出现，事件处理机制也发生了一些变化。React通过Virtual DOM的概念，将事件处理从直接操作DOM转移到组件层面进行管理。React利用了合成事件（

SyntheticEvent）来处理事件，实现了跨浏览器的一致性和性能优化。

在React中，事件处理程序是通过特定的语法和属性绑定到组件的，而不是直接操作DOM元素。

```
class MyComponent extends React.Component {
  handleClick() {
    console.log('按钮被点击');
  }

  render() {
    return <button onClick={this.handleClick}>点击按钮</button>;
  }
}
```

通过使用合成事件，React能够更高效地管理事件处理，并提供了更好的性能和开发体验。

2. 事件流的属性

事件流涉及到三个主要的概念：事件捕获阶段、目标阶段和事件冒泡阶段。了解这些阶段和相关的属性对于理解事件流的机制至关重要。

2.1 事件捕获阶段

事件捕获阶段是事件流的第一个阶段，从根节点开始向下传播到目标元素。在事件捕获阶段中，事件依次经过每个父元素，直到达到目标元素。

在事件捕获阶段，可以使用 `addEventListener` 的第三个参数指定事件处理程序在捕获阶段中执行。

```
element.addEventListener('click', handler, true);
```

2.2 目标阶段

目标阶段是事件流的第二个阶段，事件到达目标元素后被触发执行事件处理程序。

2.3 事件冒泡阶段

事件冒泡阶段是事件流的最后一个阶段，事件从目标元素开始向上冒泡，依次经过每个父元素，直到达到根节点。

在事件冒泡阶段，可以使用 `addEventListener` 的第三个参数设置为 `false` 或省略来指定事件处理程序在冒泡阶段中执行（默认值）。

```
element.addEventListener('click', handler, false);  
// 或  
element.addEventListener('click', handler);
```

2.4 事件对象

在事件处理程序中，可以通过事件对象访问和操作相关的事件信息。事件对象提供了一些属性和方法，可以获取事件的类型、目标元素、鼠标坐标等信息。

例如，可以通过事件对象的 `type` 属性获取事件类型：

```
element.addEventListener('click', function(event) {  
  console.log(event.type); // 输出 'click'  
});
```

3. 事件流的应用场景

事件流在前端开发中具有广泛的应用场景，下面介绍几个常见的应用场景：

3.1 事件处理

事件流提供了一种机制，用于处理和响应用户的交互操作。通过在目标元素上注册事件处理程序，可以捕获和处理用户触发的事件，实现交互功能。

例如，通过在按钮上注册 `click` 事件处理程序，可以

在按钮被点击时执行相应的代码逻辑。

```
const button = document.getElementById('myButton');
button.addEventListener('click', function(event) {
  console.log('按钮被点击');
});
```

3.2 事件代理

事件代理（Event Delegation）是一种常见的优化技术，用于处理大量具有相似行为的子元素事件。通过在父元素上注册事件处理程序，可以利用事件冒泡机制，统一管理子元素的事件处理。

例如，可以在父元素上注册 `click` 事件处理程序，根据触发事件的具体子元素进行不同的操作。

```
const list = document.getElementById('myList');
list.addEventListener('click', function(event) {
  if (event.target.tagName === 'LI') {
    console.log('项目被点击');
  }
});
```

3.3 事件委托

事件委托是一种通过将事件处理委托给父元素来提高性能和简化代码的技术。它利用事件冒泡机制，在父元素上注册一个事件处理程序，处理多个子元素的相同事件。

例如，可以在父元素上注册 `click` 事件处理程序，根据触发事件的子元素的不同类别执行不同的操作。

```
const container = document.getElementById('myContainer');
container.addEventListener('click', function(event) {
  if (event.target.classList.contains('button')) {
    console.log('按钮被点击');
  } else if (event.target.classList.contains('link')) {
    console.log('链接被点击');
  }
});
```

4. 示例代码

下面是一些示例代码，演示了事件流的应用和相关的属性：

```
<button id="myButton">点击按钮</button>
<ul id="myList">
  <li>项目1</li>
  <li>项目2</li>
  <li>项目3</li>
</ul>
<div id="myContainer">
  <button class="button">按钮</button>
  <a href="#" class="link">链接</a>
</div>

<script>
  // 事件处理示例
  const button = document.getElementById('myButton');
  button.addEventListener('click', function(event) {
    console.log('按钮被点击');
  });

  // 事件代理示例
  const list = document.getElementById('myList');
  list.addEventListener('click', function(event) {
    if (event.target.tagName === 'LI') {
      console.log('项目被点击');
    }
  });

  // 事件委托示例
  const container = document.getElementById('myContainer');
  container.addEventListener('click', function(event) {
    if (event.target.classList.contains('button')) {
      console.log('按钮被点击');
    } else if (event.target.classList.contains('link')) {
      console.log('链接被点击');
    }
  });
</script>
```

5. 参考资料

- [MDN Web Docs - Event reference](https://developer.mozilla.org/en-US/docs/Web/Events) (https://developer.mozilla.org/en-US/docs/Web/Events)
- [MDN Web Docs - Introduction to events](https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events) (https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events)
- [JavaScript.info - Bubbling and capturing](https://javascript.info/bubbling-and-capturing) (https://javascript.info/bubbling-and-capturing)
- [W3Schools - JavaScript HTML DOM EventListener](https://www.w3schools.com/js/js_htmlDOM_eventlistener.asp) (https://www.w3schools.com/js/js_htmlDOM_eventlistener.asp)

JavaScript修饰器：简化代码，增强功能

引言

在JavaScript中，修饰器（Decorator）是一种特殊的语法，用于修改类、方法或属性的行为。修饰器提供了一种简洁而灵活的方式来扩展和定制代码功能。本文将详细介绍JavaScript修饰器的概念、语法和应用场景，并提供相关的代码示例。

1. 修饰器简介

修饰器是一种用于修改类、方法或属性的语法，它可以在不修改原始代码的情况下增强其功能。修饰器可以实现横切关注点（cross-cutting concerns）的功能，例如日志记录、性能分析、缓存等。通过将这些功能与原始代码分离，我们可以更好地组织和维护代码，并实现更高的可重用性和可扩展性。

2. 修饰器语法

修饰器使用 `@` 符号作为前缀，紧跟着修饰器函数或类。修饰器可以接收不同的参数，根据修饰的目标不同，参数也会有所区别。修饰器可以单独使用，也可以通过组合多个修饰器来实现更复杂的功能。

下面是一个基本的修饰器语法示例：

```
@decorator
class MyClass {
  @propertyDecorator
  myProperty = 123;

  @methodDecorator
  myMethod() {
    // 代码逻辑
  }
}
```

3. 类修饰器

应用场景

类修饰器用于修改类的行为和属性。它可以在类定义之前应用，以修改类的构造函数或原型。

常见的应用场景包括：

- 「日志记录」：在类的方法执行前后记录日志信息。
- 「验证和授权」：对类的方法进行验证和授权操作。
- 「性能分析」：测量类的方法执行时间，进行性能分析。
- 「依赖注入」：为类的构造函数注入依赖项。

示例代码

下面是一个使用类修饰器实现日志记录的示例：

```
function log(target) {
  const originalConstructor = target;

  function newConstructor(...args) {
    console.log(`Creating instance of ${originalConstructor.name}`);
    return new originalConstructor(...args);
  }

  return newConstructor;
}

@log
class MyClass {
  constructor(name) {
    this.name = name;
  }
}

const myObj = new MyClass("John");
```

在上面的示例中，我们定义了一个名为 `log` 的修饰器函数。该修饰器函数接收一个参数 `target`，表示要修饰的类构造函数。在修饰器函数内部，我们将原始的构造函数保存到 `originalConstructor` 中，并创建一个新的构造函数 `newConstructor`，该构造函数在创建实例前打印日志信息。最后，我们将新的构造函数返回作为修饰后的类构造函数。

4. 方法修饰器

应用场景

方法修饰器用于修改类的方法行为。它可以在方法定义之前应用，以修改方法的特性和行为。

常见的应用场景包括：

- **「日志记录」**：在方法执行前后记录日志信息。
- **「验证和授权」**：对方法进行验证和授权操作。
- **「性能分析」**：测量方法执行时间，进行性能分析。
- **「缓存」**：为方法添加缓存功能，提高性能。

示例代码

下面是一个使用方法修饰器实现日志记录的示例：


```
function log(target, name, descriptor) {
  const originalMethod = descriptor.value;

  descriptor.value = function(...args) {
    console.log(`Executing method ${name}`);
    const result = originalMethod.apply(this, args);
    console.log(`Method ${name} executed`);
    return result;
  };

  return descriptor;
}

class MyClass {
  @log
  myMethod() {
    // 代码逻辑
  }
}

const myObj = new MyClass();
myObj.myMethod();
```

在上面的示例中，我们定义了一个名为 `log` 的修饰器函数。该修饰器函数接收三个参数，分别是 `target`（类的原型或构造函数）、`name`（方法名）和 `descriptor`（方法的属性描述符）。在修饰器函数内部，我们获取原始方法并将其保存到 `originalMethod` 中。然后，我们修改 `descriptor.value`，将其替换为一个新的函数，该函数在执行原始方法前后打印日志信息。最后，我们返回修改后的属性描述符。

5. 属性修饰器

应用场景

属性修饰器用于修改类的属性行为。它可以在属性定义之前应用，以修改属性的特性和行为。

常见的应用场景包括：

- **「日志记录」**：在属性读取或写入时记录日志信息。
- **「验证和授权」**：对属性进行验证和授权操作。
- **「计算属性」**：根据其他属性的值计算属性的值。
- **「缓存」**：为属性添加

缓存功能，提高性能。

示例代码

下面是一个使用属性修饰器实现日志记录的示例：

```
function log(target, name) {
  let value;

  const getter = function() {
    console.log(`Getting value of property ${name}`);
    return value;
  };

  const setter = function(newValue) {
    console.log(`Setting value of property ${name}`);
    value = newValue;
  };

  Object.defineProperty(target, name, {
    get: getter,
    set: setter,
    enumerable: true,
    configurable: true
  });
}

class MyClass {
  @log
  myProperty;
}

const myObj = new MyClass();
myObj.myProperty = 123;
const value = myObj.myProperty;
```

在上面的示例中，我们定义了一个名为 `log` 的修饰器函数。该修饰器函数接收两个参数，分别是 `target`（类的原型或构造函数）和 `name`（属性名）。在修饰器函数内部，我们定义了一个名为 `getter` 的函数，用于获取属性值，并在获取属性值时打印日志信息。我们还定义了一个名为 `setter` 的函数，用于设置属性值，并在设置属性值时打印日志信息。最后，我们使用 `Object.defineProperty` 方法将修饰后的属性定义到类的原型上。

6. 参数修饰器

应用场景

参数修饰器用于修改方法的参数行为。它可以在方法参数声明之前应用，以修改参数的特性和行为。常见的应用场景包括：

- 「验证和授权」：对方法的参数进行验证和授权操作。
- 「日志记录」：在方法执行前后记录参数信息。
- 「参数转换」：对方法的参数进行类型转换或格式化操作。

示例代码

下面是一个使用参数修饰器实现参数验证的示例：

```
function validate(target, name, index, validator) {
  const originalMethod = target[name];

  target[name] = function(...args) {
    const value = args[index];
    if (validator(value)) {
      return originalMethod.apply(this, args);
    } else {
      throw new Error(`Invalid value for parameter ${index} of method ${name}`);
    }
  };
}

class MyClass {
  myMethod(@validate isNumber) {
    // 代码逻辑
  }
}

function isNumber(value) {
  return typeof value === "number";
}

const myObj = new MyClass();
myObj.myMethod(123);
```

在上面的示例中，我们定义了一个名为 `validate` 的修饰器函数。该修饰器函数接收四个参数，分别是 `target`（类的原型或构造函数）、`name`（方法名）、`index`（参数索引）和 `validator`（验证函数）。在修饰器函数内部，我们获取原始方法并将其保存到 `originalMethod` 中。然后，我们修改 `target[name]`，将其替换为一个新的函数，该函数在执行原始方法之前对指定参数进行验证。如果参数通过验证，就继续执行原始方法；否则，抛出一个错误。

最后，我们使用 `@validate` 修饰器应用参数验证。

7. 修饰器组合和执行顺序

可以通过组合多个修饰器来实现更复杂的功能。修饰器的执行顺序从上到下，从右到左。

以下是一个使用多个修饰器组合的示例：

```
function log(target, name, descriptor) {  
  // 日志记录逻辑  
}  
  
function validate(target, name, index, validator) {  
  // 参数验证逻辑  
}  
  
class MyClass {  
  @log  
  @validate(isNumber)  
  myMethod(@validate(isString) param1, @validate(isBoolean) param2) {  
    // 代码逻辑  
  }  
}
```

在上面的示例中，我们通过使用 `@log` 修饰器和 `@validate` 修饰器组合，为类的方法和参数添加日志记录和验证功能。修饰器的执行顺序是从上到下，从右到左。

8. 常用修饰器库和工具

除了原生的修饰器语法，还有许多优秀的修饰器库和工具可供使用。一些常见的库和工具包括：

- **「core-decorators」**：提供了一组常用的修饰器，如 `@readonly`、`@debounce`、`@throttle` 等。[GitHub 地址 \(https://github.com/jayphelps/core-decorators\)](https://github.com/jayphelps/core-decorators)
- **「lodash-decorators」**：基于Lodash库的修饰器集合，提供了许多实用的修饰器。[GitHub 地址 \(https://github.com/steelsojka/lodash-decorators\)](https://github.com/steelsojka/lodash-decorators)
- **「mobx」**：流行的状态管理库MobX使用修饰器来实现响应式数据和自动触发更新。[官方文档 \(https://mobx.js.org/README.html\)](https://mobx.js.org/README.html)
- **「nestjs」**：基于Node.js的框架NestJS使用修饰器来实现依赖注入、路由定义等功能。[官方文档 \(https://docs.nestjs.com/\)](https://docs.nestjs.com/)

9. 结论

JavaScript修饰器是一种强大的语法，它能够简化代码、增强功能，并提高代码的可维护性和可扩展性。通过使用修饰器，我们可以轻松地实现日志记录、验证和授权、性能分析等常见的功能，同时保持代码的整洁和可读性。修饰器在许多库和框架中得到了广泛的应用，为开发者提供了更好的开发体验和工具支持。

10. 参考资料

- [MDN Web Docs - Decorators \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Decorators\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Decorators)
- [JavaScript Decorators: What They Are and When to Use Them \(https://www.sitepoint.com/javascript-decorators-what-they-are/\)](https://www.sitepoint.com/javascript-decorators-what-they-are/)

JavaScript对象

引言

在 JavaScript 中，对象是一种非常重要的数据类型，它允许我们以键值对的形式组织和存储数据。对象提供了丰富的属性和方法，使得我们能够创建、操作和管理复杂的数据结构。本文将详细介绍 JavaScript 对象的属性和常用 API，并提供一个模拟实现对象的示例。同时，还将探讨对象的应用场景和一些相关的参考资料。

1. 对象属性

JavaScript 对象的属性是以键值对的形式存储的。对象属性可以是任意类型的值，包括基本数据类型（如字符串、数字、布尔值）和其他对象。

访问属性

我们可以使用点符号或方括号来访问对象的属性。例如：

```
const person = {  
  name: 'John',  
  age: 25,  
};  
  
console.log(person.name); // 输出: John  
console.log(person['age']); // 输出: 25
```

修改属性

可以通过赋值运算符来修改对象的属性值。例如：

```
person.age = 30;  
console.log(person.age); // 输出: 30
```

删除属性

可以使用 `delete` 关键字来删除对象的属性。例如：

```
delete person.age;  
console.log(person.age); // 输出: undefined
```

动态添加属性

JavaScript 对象是动态的，意味着我们可以在运行时动态添加新的属性。例如：

```
person.address = '123 Main Street';  
console.log(person.address); // 输出: 123 Main Street
```

属性枚举

JavaScript 对象的属性默认可枚举，即可以通过 `for...in` 循环遍历对象的属性。可以使用 `Object.defineProperty()` 方法来定义不可枚举的属性。例如：

```
const car = {
  brand: 'Toyota',
  model: 'Camry',
};

Object.defineProperty(car, 'color', {
  value: 'blue',
  enumerable: false,
});

for (let key in car) {
  console.log(key); // 输出: brand, model
}
```

在上面的示例中，我们使用 `Object.defineProperty()` 定义了一个不可枚举的 `color` 属性，因此在 `for...in` 循环中不会被遍历到。

属性描述符

每个属性都有一个与之关联的属性描述符，描述了属性的各种特性。可以使用 `Object.getOwnPropertyDescriptor()` 方法获取属性的描述符。例如：

```
const descriptor = Object.getOwnPropertyDescriptor(person, 'name');
console.log(descriptor);
// 输出: { value: 'John', writable: true, enumerable: true, configurable: true }
```

在上面的示例中，我们获取了 `person` 对象的 `name` 属性的描述符。

2. 对象 API

JavaScript 对象提供了许多常用的 API，用于操作和管理对象的属性和行为。

Object.keys()

`Object.keys()` 方法返回一个包含对象所有可枚举属性的数组。

```
const person = {
  name: 'John',
  age: 25,
};

const keys = Object.keys(person);
console.log(keys); // 输出: ['name', 'age']
```

Object.values()

`Object.values()` 方法返回一个包含对象所有可枚举属性值的数组。

```
const person = {
  name: 'John',
  age: 25,
};

const values = Object.values(person);
console.log(values); // 输出: ['John', 25]
```

Object.entries()

`Object.entries()` 方法返回一个包含对象所有可枚举属性键值对的数组。

```
const person = {
  name: 'John',
  age: 25,
};

const entries = Object.entries(person);
console.log(entries);
// 输出: [['name', 'John'], ['age', 25]]
```

Object.assign()

`Object.assign()` 方法用于将一个或多个源对象的属性复制到目标对象中。

```
const target = {
  name: 'John',
};

const source = {
  age: 25,
};

Object.assign(target, source);
console.log(target); // 输出: { name: 'John', age: 25 }
```

Object.freeze()

`Object.freeze()` 方法冻结一个对象，使其属性不可修改。

```
const person = {
  name: 'John',
};

Object.freeze(person);

person.age = 25; // 操作无效，没有修改属性的权限

console.log(person); // 输出: { name: 'John' }
```

3. 实现对象 API

下面是一个简单的示例，展示了如何模拟实现几个常用的对象

API：`Object.keys()`、`Object.values()` 和 `Object.entries()`。

```
// 模拟实现 Object.keys()
function getKeys(obj) {
  const keys = [];
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      keys.push(key);
    }
  }
  return keys;
}

// 模拟实现 Object.values()
function getValues(obj) {
  const values = [];
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      values.push(obj[key]);
    }
  }
  return values;
}

// 模拟实现 Object.entries()
function getEntries(obj) {
  const entries = [];
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      entries.push([key, obj[key]]);
    }
  }
  return entries;
}

const person = {
  name: 'John',
  age: 25,
};

console.log(getKeys(person)); // 输出: ['name', 'age']
console.log(getValues(person)); // 输出: ['John', 25]
console.log(getEntries(person)); // 输出: [['name', 'John'], ['age', 25]]
```

在上面的示例中，我们使用自定义函数 `getKeys()`、`getValues()` 和 `getEntries()` 来模拟实现了 `Object.keys()`、`Object.values()` 和 `Object.entries()` 的功能。

4. 应用场景

JavaScript 对象在前端开发中有广泛的应用场景，包括但不限于以下几个方面：

- 「数据存储和操作」：对象允许我们以键值对的形式存储和操作数据，非常适合表示复杂的数据结构。
- 「面向对象编程」：对象是面向对象编程的核心概念，允许我们创建和管理对象的行为和属性。
- 「DOM 操作」：在前端开发中，我们经常需要操作网页的 DOM 元素，使用对象可以更方便地访问和操作 DOM。
- 「数据交互和传输」：在与后端进行数据交互时，常常使用对象来传输和接收数据，例如通过 AJAX 请求返回的 JSON 数据。

5. 参考资料

- [MDN Web Docs - Working with objects](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_Objects)(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_Objects)
- [MDN Web Docs - Object](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object)(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object)

JavaScript数组

引言

在JavaScript中，数组（Array）是一种重要且广泛应用的数据结构，用于存储和操作一组有序的数据。JavaScript提供了丰富的数组方法和属性，使我们能够方便地对数组进行增删改查等操作。本文将详细介绍JavaScript数组的方法API、属性，并探讨如何模拟实现数组的API。此外，还将介绍数组的应用场景，帮助读者更好地理解和应用数组。

1. 数组简介

数组是一种有序的数据集合，它可以存储多个值，并根据索引访问和操作这些值。在JavaScript中，数组是一种动态类型的数据结构，可以容纳任意类型的数据，包括基本类型和对象。

JavaScript数组的特点包括：

- 数组的长度是动态可变的，可以根据需要随时添加或删除元素。
- 数组的索引是从0开始的，通过索引可以快速访问和修改数组中的元素。
- 数组可以包含不同类型的元素，甚至可以嵌套包含其他数组。

JavaScript提供了许多方法和属性来操作和处理数组，使得数组成为处理数据的强大工具。

2. 数组方法API

JavaScript数组提供了丰富的方法来操作数组。以下是一些常用的方法API：

添加和删除元素

- 「push()」：在数组末尾添加一个或多个元素，并返回新数组的长度。
- 「pop()」：移除并返回数组的最后一个元素。
- 「unshift()」：在数组开头添加一个或多个元素，并返回新数组的长度。
- 「shift()」：移除并返回数组的第一个元素。
- 「splice()」：从指定位置添加或删除元素。

修改和访问元素

- 「slice()」：返回数组的一部分，不改变原数组。
- 「concat()」：将多个数组合并为一个新数组。
- 「join()」：将数组的元素连接成一个字符串。
- 「reverse()」：颠倒数组中元素的顺序。
- 「sort()」：对数组元素进行排序。

数组遍历

- 「forEach()」：对数组的每个元素执行指定的操作。
- 「map()」：创建一个新数组，其中的元素是原始数组经过指定操作后的结果。
- 「filter()」：创建一个新数组，其中的元素是符合指定条件的原始数组元素。
- 「reduce()」：对数组的元素进行累加或合并操作。

数组转换和连接

- 「toString()」：将数组转换为字符串。
- 「toLocaleString()」：将数组转换为本地化的字符串。
- 「join()」：将数组的元素连接成一个字符串。

数组排序和搜索

- 「sort()」：对数组元素进行排序。
- 「reverse()」：颠倒数组中元素的顺序。
- 「indexOf()」：返回指定元素在数组中首次出现的索引。
- 「lastIndexOf()」：返回指定元素在数组中最后一次出现的索引。

其他常用方法

- 「isArray()」：检测一个值是否为数组。
- 「find()」：返回数组中符合指定条件的第一个元素。
- 「findIndex()」：返回数组中符合指定条件的第一个元素的索引。
- 「some()」：检测数组中是否至少有一个元素符合指定条件。
- 「every()」：检测数组中是否所有元素都符合指定条件。

以上仅是JavaScript数组方法API的部分常用示例，更多详细的方法和用法请参考[MDN Web Docs \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array)。

3. 数组属性

JavaScript数组还有一些常用的属性，用于描述和操作数组的特性和状态。

- 「length」：返回数组的长度。
- 「constructor」：返回创建数组对象的原型函数。
- 「prototype」：数组对象的原型对象，用于添加新的方法和属性。

这些属性可以帮助我们了解数组的结构和信息，以便更好地处理和操作数组。

4. 实现数组API

为了更好地理解数组的方法和实现原理，我们可以尝试自己模拟实现一些数组API的方法。以下是一些常用的数组方法的简单模拟实现示例：

实现添加和删除元素的方法

// 模拟实现 push() 方法

```
Array.prototype.myPush = function(...elements) {  
  const len = this.length;  
  let i = 0;  
  while (i < elements.length) {  
    this[len + i] = elements[i];  
    i++;  
  }  
  return this.length;  
};
```

// 模拟实现 pop() 方法

```
Array.prototype.myPop = function() {  
  if (this  
  
.length === 0) return undefined;  
  const lastElement = this[this.length - 1];  
  delete this[this.length - 1];  
  this.length--;  
  return lastElement;  
};
```

// 模拟实现 unshift() 方法

```
Array.prototype.myUnshift = function(...elements) {  
  const originalLength = this.length;  
  for (let i = originalLength - 1; i >= 0; i--) {  
    this[i + elements.length] = this[i];  
  }  
  for (let i = 0; i < elements.length; i++) {  
    this[i] = elements[i];  
  }  
  this.length = originalLength + elements.length;  
  return this.length;  
};
```

// 模拟实现 shift() 方法

```
Array.prototype.myShift = function() {  
  if (this.length === 0) return undefined;  
  const firstElement = this[0];  
  for (let i = 0; i < this.length - 1; i++) {  
    this[i] = this[i + 1];  
  }  
  delete this[this.length - 1];  
  this.length--;  
  return firstElement;  
};
```

// 示例使用

```
const myArray = [1, 2, 3];  
console.log(myArray.myPush(4, 5)); // 输出: 5  
console.log(myArray); // 输出: [1, 2, 3, 4, 5]  
console.log(myArray.myPop()); // 输出: 5  
console.log(myArray); // 输出: [1, 2, 3, 4]
```

```
console.log(myArray.myUnshift(0)); // 输出: 5
console.log(myArray);              // 输出: [0, 1, 2, 3, 4]
console.log(myArray.myShift());    // 输出: 0
console.log(myArray);              // 输出: [1, 2, 3, 4]
```

实现修改和访问元素的方法

```
// 模拟实现 splice() 方法
Array.prototype.mySplice = function(startIndex, deleteCount, ...elements) {
  const removedElements = [];
  const len = this.length;
  const deleteEndIndex = Math.min(startIndex + deleteCount, len);
  const moveCount = len - deleteEndIndex;

  // 保存删除的元素
  for (let i = startIndex; i < deleteEndIndex; i++) {
    removedElements.push(this[i]);
  }

  // 移动剩余元素
  for (let i = 0; i < moveCount; i++) {
    this[startIndex + deleteCount + i] = this[startIndex + deleteCount + i + moveCount];
  }

  // 插入新元素
  for (let i = 0; i < elements.length; i++) {
    this[startIndex + i] = elements[i];
  }

  // 更新长度
  this.length = len - deleteCount + elements.length;

  return removedElements;
};

// 示例使用
const myArray = [1, 2, 3, 4, 5];
console.log(myArray.mySplice(2, 2, 'a', 'b')); // 输出: [3, 4]
console.log(myArray);                          // 输出: [1, 2, 'a', 'b', 5]
```

实现数组遍历的方法


```
// 模拟实现 forEach() 方法
Array.prototype.myForEach = function(callbackFn) {
  for (let i = 0;

i < this.length; i++) {
    callbackFn(this[i], i, this);
  }
};

// 模拟实现 map() 方法
Array.prototype.myMap = function(callbackFn) {
  const mappedArray = [];
  for (let i = 0; i < this.length; i++) {
    mappedArray.push(callbackFn(this[i], i, this));
  }
  return mappedArray;
};

// 示例使用
const myArray = [1, 2, 3];
myArray.myForEach((value, index) => {
  console.log(`Element at index ${index} is ${value}`);
});

const doubledArray = myArray.myMap(value => value * 2);
console.log(doubledArray); // 输出: [2, 4, 6]
```

实现数组转换和连接的方法

// 模拟实现 toString() 方法

```
Array.prototype.myToString = function() {  
  let result = "";  
  for (let i = 0; i < this.length; i++) {  
    if (i > 0) {  
      result += ',';  
    }  
    result += this[i];  
  }  
  return result;  
};
```

// 模拟实现 join() 方法

```
Array.prototype.myJoin = function(separator = ',') {  
  let result = "";  
  for (let i = 0; i < this.length; i++) {  
    if (i > 0) {  
      result += separator;  
    }  
    result += this[i];  
  }  
  return result;  
};
```

// 示例使用

```
const myArray = [1, 2, 3];  
console.log(myArray.myToString()); // 输出: '1, 2, 3'  
console.log(myArray.myJoin('-')); // 输出: '1-2-3'
```

实现数组排序和搜索的方法

// 模拟实现 sort() 方法

```
Array.prototype.mySort = function(compareFn) {  
  const length = this.length;  
  for (let i = 0; i < length - 1; i++) {  
    for (let j = 0; j < length - 1 - i; j++) {  
      if (compareFn(this[j], this[j + 1]) > 0) {  
        [this[j], this[j + 1]] = [this[j + 1], this[j]];  
      }  
    }  
  }  
  return this;  
};
```

// 模拟实现 indexOf() 方法

```
Array.prototype.myIndexOf = function(searchElement, fromIndex = 0) {  
  const length = this.length;  
  for (let i = Math.max(fromIndex, 0); i < length; i++) {  
    if (this[i] === searchElement) {  
      return i;  
    }  
  }  
  return -1;  
};
```

// 示例使用

```
const myArray = [5, 2, 1, 4, 3];  
console.log(myArray.mySort()); // 输出: [1, 2, 3, 4, 5]  
console.log(myArray.myIndexOf(4)); // 输出: 3
```

实现其他常用方法

```
// 模拟实现 isArray() 方法
Array.myIsArray = function(obj) {
  return Object.prototype.toString.call(obj) === '[object Array]';
};

// 模拟实现 find() 方法
Array.prototype.myFind = function(callbackFn) {
  for (let i = 0; i < this.length; i++) {
    if (callbackFn(this[i], i, this)) {
      return this[i];
    }
  }
  return undefined;
};

// 示例使用
const myArray = [1, 2, 3, 4, 5];
console.log(Array.myIsArray(myArray)); // 输出: true
console.log(myArray.myFind(value => value > 3)); // 输出: 4
```

以上是一些简单的模拟实现示例，用于帮助理解数组方法的实现原理。

5. 数组的应用场景

数组作为一种常见的数据结构，在前端开发中有许多应用场景。以下是一些常见的应用场景：

- **「数据存储和管理」**：数组可用于存储和管理数据集合，如用户列表、商品列表等。可以通过数组的增删改查操作，对数据进行增删改查、排序和搜索等操作。
- **「数据筛选和过滤」**：使用数组的过滤方法（如 `filter()`）可以方便地筛选和过滤数据，根据指定条件获取符合条件的数据子集。
- **「数据统计和计算」**：通过数组的迭代方法（如 `reduce()`）可以对数据进行统计和计算操作，如求和、平均值、最大值、最小值等。
- **「数据展示和渲染」**：使用数组和模板引擎可以方便地进行数据的展示和渲染，如动态生成列表、表格等页面元素。

数组在前端开发中的应用非常广泛，几乎涉及到数据的存储、处理和展示等方方面面。

6. 参考资料

- [MDN Web Docs - Array](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array) (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array)
- [JavaScript Arrays: Understanding the Basics](https://www.sitepoint.com/javascript-arrays-understanding-the-basics/) (<https://www.sitepoint.com/javascript-arrays-understanding-the-basics/>)
- [JavaScript Array Methods Every Developer Should Know](https://dev.to/frugencefidel/10-javascript-array-methods-you-should-know-4lk3) (<https://dev.to/frugencefidel/10-javascript-array-methods-you-should-know-4lk3>)
- [Understanding JavaScript Array Series](https://codeburst.io/understanding-javascript-array-series-part-1-376cdac94213) (<https://codeburst.io/understanding-javascript-array-series-part-1-376cdac94213>)

Javascript数据类型和类型转换

在JavaScript中，理解数据类型，如何区分它们，以及它们如何被转换是至关重要的。在这篇文章中，我们将探讨这些主题，以帮助巩固你的JavaScript基础。

基础数据类型和引用数据类型

当涉及JavaScript的数据类型时，我们可以将其分为两类：基本数据类型和引用数据类型。

1. 基本数据类型 (Primitive Types) :

- 数字 (Number) : 表示数值，可以包含整数和浮点数。例如： `let age = 25;`
- 字符串 (String) : 表示文本数据，由一串字符组成。可以使用单引号或双引号包裹。例如： `let name = 'John';`
- 布尔 (Boolean) : 表示逻辑值，只有两个可能的值： `true` (真) 和 `false` (假) 。例如： `let isStudent = true;`
- 空值 (Null) : 表示空值或无值。它是一个特殊的关键字 `null` 。例如： `let myVariable = null;`
- 未定义 (Undefined) : 表示变量声明但未赋值的值。它是一个特殊的关键字 `undefined` 。例如： `let myVariable;`
- 符号 (Symbol) : 表示唯一且不可变的值，用于创建对象属性的唯一标识符。在ES6中引入。例如： `let id = Symbol('id');`

2. 引用数据类型 (Reference Types) :

- 对象 (Object) : 表示复杂的数据结构，可以包含多个键值对。对象可以通过大括号 `{}` 创建，或者通过构造函数创建。例如：

```
let person = {  
  name: 'John',  
  age: 25,  
  city: 'New York'  
};
```

- 数组 (Array) : 表示有序的数据集合，可以包含任意类型的数据。数组可以通过方括号 `[]` 创建。例如：

```
let numbers = [1, 2, 3, 4, 5];
```

- 函数 (Function) : 是一段可执行的代码块，可以接收参数并返回值。函数可以作为变量、参数传递、存储在对象属性中等。例如：

```
function greet(name) {  
  console.log('Hello, ' + name + '!');  
}
```

基本数据类型在JavaScript中是按值传递的，而引用数据类型则是按引用传递的。这意味着基本数据类型的值在传递过程中是复制的，而引用数据类型的值在传递过程中是共享的。

了解这些基本数据类型和引用数据类型，为后续讲解类型转换提供了基本的背景知识。它们在JavaScript中的不同行为和用法对于理解类型转换的概念和机制非常重要。

使用typeof操作符

在JavaScript中，我们可以使用 `typeof` 操作符来获取一个值的数据类型。下面是一些例子：

```
console.log(typeof undefined); // 'undefined'
console.log(typeof true); // 'boolean'
console.log(typeof 78); // 'number'
console.log(typeof 'hey'); // 'string'
console.log(typeof Symbol()); // 'symbol'
console.log(typeof BigInt(1)); // 'bigint'
console.log(typeof new String('abc')); // 'object'
console.log(typeof null); // 'object'
console.log(typeof function(){}); // 'function'
console.log(typeof {name: 'Jack'}); // 'object'
```

注意，`typeof` 返回的是值的类型，而不是变量的类型。因为在JavaScript中，变量本身并没有类型，它们可以持有任何类型的值。

对大多数对象使用 `typeof` 时，返回的结果是 `'object'`，对于函数则返回 `'function'`。特别的，对 `null` 使用 `typeof` 返回的也是 `'object'`，这是一个历史遗留的bug，我们无法改正。所以，如果我们需要检查一个值是否为 `null`，我们可以使用以下方式：

```
var a = null;
console.log(!a && typeof a === "object"); // true
```

包装类型

在JavaScript中，基本数据类型有对应的包装对象，这样我们就可以在基本数据类型上调用方法了。例如，字符串有对应的 `String` 包装对象，我们就可以在字符串上调用 `String` 对象的方法：

```
let s = 'Hello, world!';
console.log(s.length); // 13
```

这里，`length` 是 `String` 对象的一个属性，我们可以在字符串 `s` 上访问它。这是如何做到的呢？当我们在一个字符串上调用一个方法或者访问一个属性时，JavaScript会将字符串自动转换为一个临时的 `String` 对象，然后在这个临时对象上调用方法或者访问属性。完成后，临时对象就会被销毁。

其他的基本数据类型，如 `Number`，`Boolean`，也有对应的包装对象，操作方式类似。

隐式类型转换

在JavaScript中，隐式类型转换是指在特定的上下文中，JavaScript自动将一个数据类型转换为另一个数据类型，而无需显式地编写转换代码。以下是一些常见的隐式类型转换示例：

1. 数字转字符串：

```
let num = 10;
let str = num + ''; // 将数字转换为字符串
console.log(str); // 输出: "10"
```

在这个例子中，通过将数字与一个空字符串相加，JavaScript会将数字隐式转换为字符串。

2. 字符串转数字：

```
let str = '20';
let num = +str; // 将字符串转换为数字
console.log(num); // 输出: 20
```

在这个例子中，通过使用一元加号操作符（+）对字符串进行操作，JavaScript会将字符串隐式转换为数字。

3. 布尔值转数字：

```
let bool = true;
let num = +bool; // 将布尔值转换为数字
console.log(num); // 输出: 1
```

在这个例子中，通过使用一元加号操作符（+）对布尔值进行操作，JavaScript会将布尔值隐式转换为数字，`true`转换为1，`false`转换为0。

4. 字符串转布尔值：

```
let str = 'true';
let bool = !!str; // 将字符串转换为布尔值
console.log(bool); // 输出: true
```

在这个例子中，通过使用两个逻辑非操作符（!!）对字符串进行操作，JavaScript会将字符串隐式转换为布尔值，非空字符串转换为`true`，空字符串转换为`false`。

需要注意的是，隐式类型转换在某些情况下可能会导致意外的结果。因此，在进行类型转换时，特别是涉及不同的数据类型之间的运算时，要注意确保结果符合预期。

理解隐式类型转换的规则和机制可以帮助我们更好地理解JavaScript代码中的行为，并在需要时正确地处理数据类型转换。

5. 对象的隐式转换

在JavaScript中，对象在进行隐式类型转换时会根据一定的规则进行处理。对象的隐式类型转换通常涉及将对象转换为字符串或将对象转换为数字。

1. 对象转换为字符串：

当一个对象需要被隐式转换为字符串时，JavaScript会尝试调用对象的`toString()`方法。`toString()`方法是一个内置方法，它返回表示对象的字符串形式。

```
let obj = { name: "John", age: 25 };
let str = obj.toString();
console.log(str); // 输出: "[object Object]"
```

在上述例子中，对象`obj`会被隐式转换为字符串形式，调用了`toString()`方法并返回了`"[object Object]"`。

需要注意的是，`toString()` 方法的默认实现返回 "[object Object]"，这对于大多数对象来说并不是非常有用。因此，可以通过重写对象的 `toString()` 方法来自定义对象转换为字符串的行为。

```
let person = {
  name: "John",
  age: 25,
  toString() {
    return this.name + " - " + this.age;
  }
};
let str = person.toString();
console.log(str); // 输出: "John - 25"
```

在这个例子中，我们重写了 `person` 对象的 `toString()` 方法，使其返回自定义的字符串形式。

2. 对象转换为数字：

在JavaScript中，当一个对象需要被隐式转换为数字时，会首先尝试调用对象的 `valueOf()` 方法，如果该方法返回的不是原始值（例如数字），则会接着尝试调用对象的 `toString()` 方法，将返回值转换为数字。

```
let obj = { value: 42 };
let num = obj.valueOf();
console.log(num); // 输出: { value: 42 }
```

需要注意的是，与日期对象的 `valueOf()` 方法不同，大多数对象的默认 `valueOf()` 方法的行为通常并不有用。因此，可以通过重写对象的 `valueOf()` 方法来自定义对象转换为数字的行为。

```
let counter = {
  value: 0,
  valueOf() {
    return this.value++;
  }
};
let num = counter.valueOf();
console.log(num); // 输出: 0
console.log(counter.value); // 输出: 1
```

在这个例子中，我们重写了 `counter` 对象的 `valueOf()` 方法，使其每次调用时返回一个递增的值。

需要注意的是，对象的隐式类型转换的行为和结果可能会因对象的类型、实现方式以及具体的上下文而有所不同。在编写代码时，建议根据实际需求和预期结果来处理对象的隐式类型转换，并确保理解和掌握对象的 `toString()` 和 `valueOf()` 方法的使用。

显式类型转换

在JavaScript中，我们可以使用一些内置函数和操作符来进行显式类型转换，以将一个值转换为特定的数据类型。下面是一些常用的类型转换函数和操作符以及它们的用法和注意事项：

1. `String()` 函数：用于将一个值转换为字符串类型。


```
let num = 10;
let str = String(num); // 将数字转换为字符串
console.log(str); // 输出: "10"
```

需要注意的是，使用String()函数进行转换时，对于 null 和 undefined 值会分别得到 "null" 和 "undefined" 字符串。

2. Number() 函数：用于将一个值转换为数字类型。

```
let str = "20";
let num = Number(str); // 将字符串转换为数字
console.log(num); // 输出: 20
```

需要注意的是，使用Number()函数进行转换时，如果传入的字符串无法解析为有效的数字，将返回 NaN (Not a Number) 。

3. Boolean() 函数：用于将一个值转换为布尔类型。

```
let num = 0;
let bool = Boolean(num); // 将数字转换为布尔值
console.log(bool); // 输出: false
```

需要注意的是，使用Boolean()函数进行转换时，对于 0、-0、null、undefined、NaN 和空字符串会返回 false，其他值都会返回 true。

4. parseInt() 和 parseFloat() 函数：用于将字符串转换为整数和浮点数类型。

```
let str = "123";
let num = parseInt(str); // 将字符串转换为整数
console.log(num); // 输出: 123

let floatStr = "3.14";
let floatNum = parseFloat(floatStr); // 将字符串转换为浮点数
console.log(floatNum); // 输出: 3.14
```

需要注意的是，使用 parseInt() 和 parseFloat() 函数进行转换时，它们会尝试解析字符串的开头部分，直到遇到非数字字符为止。

除了上述函数，还有一些常用的操作符也可以进行显式类型转换：

- 加号操作符 (+)：用于将值转换为数字类型。

```
let str = "20";
let num = +str; // 将字符串转换为数字
console.log(num); // 输出: 20
```

- 双重取反操作符 (!!): 用于将值转换为布尔类型。

```
let num = 0;
let bool = !!num; // 将数字转换为布尔值
console.log(bool); // 输出: false
```


在进行显式类型转换时，需要注意以下几点：

- 了解转换函数和操作符的行为和规则，以避免出现意外的结果。
- 特别注意在将字符串转换为数字时，确保字符串能够正确解析为有效的数字，以避免得到NaN。
- 注意处理 null 和 undefined 值时的类型转换结果。
- 在类型转换场景中，根据具体需求选择合适的函数或操作符。

通过显式类型转换，我们可以将值从一个数据类型转换为另一个数据类型，以满足具体的需求和逻辑。

类型转换规则

了解类型转换的规则和注意事项是非常重要的，可以帮助我们避免出现意外的结果和错误的行为。下面是一些类型转换的规则和需要注意的情况：

1. 类型转换的优先级：在JavaScript中，类型转换有一定的优先级。从高到低的优先级顺序是：

- 布尔值 -> 数字 -> 字符串

这意味着在进行混合类型的操作时，JavaScript会首先尝试将值转换为布尔值，然后是数字，最后是字符串。

2. 字符串拼接优先：在涉及字符串和其他数据类型的操作中，字符串拼接的优先级最高。这意味着如果一个操作符是字符串拼接操作符（+），那么其他操作数将被隐式转换为字符串。

```
let num = 10;
let str = "The number is: " + num;
console.log(str); // 输出: "The number is: 10"
```

在这个例子中，数字 `num` 会被隐式转换为字符串，然后与其他字符串进行拼接。

3. NaN (Not a Number)：当涉及无法进行有效数值计算的情况时，JavaScript会返回NaN。NaN是一个特殊的数字值，表示不是一个有效的数字。

```
let result = 10 / "hello";
console.log(result); // 输出: NaN
```

在这个例子中，字符串"hello"无法被解析为有效的数字，所以计算结果为NaN。

4. null和undefined的类型转换：null和undefined在进行类型转换时有一些特殊规则：

- null在进行数字转换时会被转换为0，而在进行字符串转换时会被转换为"null"。
- undefined在进行数字转换时会被转换为NaN，而在进行字符串转换时会被转换为"undefined"。

```
let num = Number(null);
console.log(num); // 输出: 0

let str = String(undefined);
console.log(str); // 输出: "undefined"
```

在这个例子中，`null`在数字转换时被转换为`0`，`undefined`在字符串转换时被转换为`"undefined"`。

5. 注意一元加号操作符（`+`）的行为：一元加号操作符可以用于将值转换为数字类型，但需要注意一些情况。当应用于字符串时，一元加号操作符会尝试将字符串解析为数字。

```
let str = "123";
let num = +str;
console.log(num); // 输出: 123

let invalidStr = "hello";
let invalidNum = +invalidStr;
console.log(invalidNum); // 输出: NaN
```

在这个例子中，有效的数字字符串可以成功转换为数字，而无法解析为数字的字符串会转换为`NaN`。

了解这些规则和注意事项可以帮助我们更好地理解类型转换的行为，并在编写代码时避免潜在的错误和意外结果。同时，在进行类型转换时，要根据具体的需求选择合适的方法和操作符，并进行适当的错误处理和边界检查。

最佳实践

在JavaScript中，以下是一些类型转换的最佳实践和常见应用场景，以帮助我们编写更安全、清晰和高效的代码：

- 避免意外的类型转换：隐式类型转换可能导致意外的结果和错误的行为。为了避免这种情况，可以遵循以下实践：
 - 显式地使用适当的类型转换函数或操作符，明确指定期望的转换结果。
 - 在涉及类型转换的操作中，添加适当的错误处理机制，以防止无效的转换。
- 类型安全的比较：在条件语句中，确保进行类型安全的比较，避免因类型转换而导致的问题。使用恰当的比较操作符（如`===`和`!==`）可以同时比较值和类型，确保比较的准确性。

```
let num = "10";
if (num === 10) {
  // 正确的比较方式，值和类型都匹配
  console.log("The number is 10.");
} else {
  console.log("The number is not 10.");
}
```

在这个例子中，使用`===`进行比较可以避免字符串与数字的隐式转换，确保比较的准确性。

- 使用适当的类型转换技巧：在某些情况下，可以使用类型转换来解决问题或优化代码逻辑。以下是一些常见的类型转换技巧：

- 将字符串转换为数字或反之：使用 `Number()` 函数或一元加号操作符 (+) 进行转换。
- 将字符串转换为数组：使用 `split()` 函数将字符串拆分为数组。
- 将对象转换为字符串：使用 `JSON.stringify()` 函数将对象转换为字符串表示。
- 将数字转换为字符串并添加特定格式：使用字符串模板或字符串拼接操作符 (+) 。

4. 考虑性能和可读性：尽管类型转换是一种强大的工具，但过度使用或滥用可能会影响代码的性能和可读性。在进行类型转换时，要权衡利弊，并确保代码易于理解和维护。

总之，掌握类型转换的最佳实践可以帮助我们编写更健壮和高效的代码。遵循类型安全的比较、避免意外的类型转换、选择适当的类型转换技巧，并在性能和可读性之间找到平衡，都是编写优质JavaScript代码的重要因素。

参考资料

1. [MDN Web Docs - Type Conversion \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Type_conversion\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Type_conversion): MDN Web Docs中关于JavaScript中类型转换的官方文档，提供了关于隐式类型转换和显式类型转换的详细解释和示例。
2. [MDN Web Docs - toString\(\) \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/toString\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/toString): MDN Web Docs中关于 `toString()` 方法的官方文档，提供了有关对象的 `toString()` 方法的详细解释和用法示例。
3. [MDN Web Docs - valueOf\(\) \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/valueOf\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/valueOf): MDN Web Docs中关于 `valueOf()` 方法的官方文档，提供了有关对象的 `valueOf()` 方法的详细解释和用法示例。

requestAnimationFrame：优化动画和渲染的利器

引言

在Web开发中，实现平滑且高性能的动画和渲染是一个关键的需求。而requestAnimationFrame是浏览器提供的一个用于优化动画和渲染的API。它可以协调浏览器的刷新率，帮助开发者实现流畅的动画效果，并提供更高效的渲染方式。本文将详细介绍requestAnimationFrame的属性、应用场景以及使用示例，帮助读者深入理解和应用这一强大的工具。

1. requestAnimationFrame简介

requestAnimationFrame是浏览器提供的一个用于优化动画和渲染的API。它基于浏览器的刷新率，调度回调函数的执行，以确保动画和渲染的流畅性和高性能。

使用requestAnimationFrame，开发者可以在每个浏览器刷新帧之前请求执行一个函数。浏览器会在适当的时机调用这个函数，以保证动画和渲染的协调性。通过与浏览器的合作，requestAnimationFrame可以避免不必要的渲染操作，并确保动画的效果更加平滑。

requestAnimationFrame在现代浏览器中得到广泛支持，并成为实现高性能动画和渲染的首选方式。

2. requestAnimationFrame的属性

requestAnimationFrame提供了一些属性，用于控制和管理动画和渲染的执行。下面是一些常用的属性：

- **「callback」**：一个函数，表示要在下一次浏览器刷新帧之前执行的回调函数。
- **「id」**：一个整数，表示回调函数的唯一标识符。可以用于取消回调函数的执行。

通过这些属性，开发者可以精确地控制和管理动画和渲染的执行过程。

3. requestAnimationFrame的应用场景

requestAnimationFrame在许多场景下都能发挥重要作用。下面是一些常见的应用场景：

3.1 动画效果

当需要实现平滑的动画效果时，requestAnimationFrame是一个理想的选择。通过使用requestAnimationFrame，可以在每个浏览器刷新帧之前更新动画的状态，并在合适的时机进行渲染。这样可以确保动画的流畅性，并减少不必要的渲染操作。例如，实现平滑的过渡效果、动态的图表展示等都可以使用requestAnimationFrame来实现。

3.2 游戏开发

在游戏开发中，高性能和流畅的渲染是至关重要的。requestAnimationFrame提供了一种高效的渲染方式，可以与游戏引擎

配合使用，实现流畅的游戏画面和良好的用户体验。通过在每个浏览器刷新帧之前更新游戏的状态并进行渲染，可以实现高性能的游戏效果。例如，实时的射击游戏、跑酷游戏等都可以使用requestAnimationFrame来实现。

3.3 数据可视化

在数据可视化的场景中，展示大量的数据并实时更新是一项挑战。使用requestAnimationFrame，可以在每个浏览器刷新帧之前更新数据的可视化状态，并进行相应的渲染。这样可以实现高效的数据可视化，并保持良好的性能和交互性。例如，绘制实时图表、展示动态地图等都可以使用requestAnimationFrame来实现。

3.4 UI动效

在网页开发中，为用户提供吸引人的UI动效是一种常见的需求。使用requestAnimationFrame，可以实现各种各样的UI动效，如平滑的滚动效果、渐变动画、拖拽效果等。通过在每个浏览器刷新帧之前更新UI状态并进行渲染，可以实现流畅和高性能的UI动效。

4. 使用requestAnimationFrame的示例

下面通过几个示例来演示如何使用requestAnimationFrame来实现动画和渲染效果。

4.1 实现平滑的滚动效果

下面的示例代码演示了如何使用requestAnimationFrame实现平滑的滚动效果：


```
function smoothScrollTo(targetY, duration) {
  const startY = window.pageYOffset;
  const distance = targetY - startY;
  const startTime = performance.now();

  function step(currentTime) {
    const elapsedTime = currentTime - startTime;
    const progress = Math.min(elapsedTime / duration, 1);
    const ease = easingFunction(progress);
    window.scrollTo(0, startY + distance * ease);

    if (elapsedTime < duration) {
      requestAnimationFrame(step);
    }
  }

  requestAnimationFrame(step);
}

function easingFunction(t) {
  return t * t * t;
}

// 使用示例
const button = document.querySelector('#scrollButton');
button.addEventListener('click', () => {
  smoothScrollTo(1000, 1000);
});
```

在上述代码中，我们定义了一个 `smoothScrollTo` 函数，用于实现平滑的滚动效果。该函数接收目标位置 `targetY` 和滚动的持续时间 `duration` 作为参数。在函数内部，我们获取当前的滚动位置 `startY` 和目标位置与起始位置之间的距离 `distance`。然后，我们使用 `performance.now()` 获取当前的时间戳 `startTime`，并定义一个 `step` 函数用于更新滚动位置。在 `step` 函数中，我们根据时间的流逝计算出进度 `progress`，并使用缓动函数 `easingFunction` 来调整进度。最后，我们使用 `requestAnimationFrame` 调度 `step` 函数的执行，并在滚动动画完成之前不断更新滚动位置。

4.2 实现粒子动画效果

下面的示例代码演示了如何使用 `requestAnimationFrame` 实现粒子动画效果：

```
const canvas = document.querySelector('#canvas');
const ctx = canvas.getContext('2d');

const particles = [];

function Particle(x, y, speedX, speedY, radius, color) {
  this.x = x;
  this.y = y;
  this.speedX = speedX;
  this.speedY = speedY;
  this.radius = radius;
```



```
this.color = color;
}

Particle.prototype.update = function() {
  this.x += this.speedX;
  this.y += this.speedY;

  if (this.x + this.radius < 0 || this.x - this.radius > canvas.width) {
    this.speedX = -this.speedX;
  }

  if (this.y + this.radius < 0 || this.y - this.radius > canvas.height) {
    this.speedY = -this.speedY;
  }
};

Particle.prototype.draw = function() {
  ctx.beginPath();
  ctx.arc(this.x, this.y, this.radius, 0, Math.PI * 2, false);
  ctx.fillStyle = this.color;
  ctx.fill();
  ctx.closePath();
};

function createParticles() {
  for (let i = 0; i < 100; i++) {
    const x = Math.random() * canvas.width;
    const y = Math.random() * canvas.height;
    const speedX = Math.random() * 4 - 2;
    const speedY = Math.random() * 4 - 2;
    const radius = Math.random() * 5 + 1;
    const color = getRandomColor();

    particles.push(new Particle(x, y, speedX, speedY, radius, color));
  }
}

function updateParticles() {
  particles.forEach((particle) => {
    particle.update();
  });
}

function drawParticles() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  particles.forEach((particle) => {
    particle.draw();
  });

  requestAnimationFrame(drawParticles);
}
```

```
// 使用示例
createParticles();
drawParticles();

function getRandomColor() {
  const letters = '0123456789ABCDEF';
  let color = '#';

  for (let i = 0; i < 6; i++) {
    color += letters[Math.floor(Math.random() * 16)];
  }

  return color;
}
```

在上述代码中，我们定义了一个 `Particle` 构造函数，用于创建粒子对象。粒子对象包含位置坐标 `x` 和 `y`、速度 `speedX` 和 `speedY`、半径 `radius` 和颜色 `color` 等属性。我们还为 `Particle` 对象添加了 `update` 方法和 `draw` 方法，用于更新粒子的位置和绘制粒子的图形。

我们还定义了 `createParticles` 函数，用于创建一定数量的粒子，并随机生成它们的初始位置、速度、半径和颜色。在 `drawParticles` 函数中，我们使用 `requestAnimationFrame` 调度 `drawParticles` 函数的执行，并在每一帧清空画布、更新粒子的位置和绘制粒子的图形。

通过上述示例，我们可以看到使用 `requestAnimationFrame` 可以轻松实现平滑的动画效果和高性能的渲染。

5. 总结

`requestAnimationFrame` 是浏览器提供的用于优化动画和渲染的 API，它通过与浏览器的合作，协调刷新率并在合适的时机执行回调函数，从而实现流畅的动画效果和高性能的渲染。

本文详细介绍了 `requestAnimationFrame` 的属性、应用场景以及使用示例。通过使用 `requestAnimationFrame`，开发者可以实现平滑的滚动效果、高性能的游戏渲染、复杂的数据可视化和吸引人的 UI 动效等。同时，本文提供了几个示例代码，帮助读者更好地理解和应用 `requestAnimationFrame`。

请记住，使用 `requestAnimationFrame` 时应注意避免过度使用和滥用，以免对浏览器性能造成负面影响。合理利用 `requestAnimationFrame`，结合适当的优化和控制，能够提供更好的用户体验和更高效的渲染方式。

6. 参考资料

- [MDN Web Docs - requestAnimationFrame \(https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame\)](https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame)
- [Using requestAnimationFrame \(https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame/Using_requestAnimationFrame\)](https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame/Using_requestAnimationFrame)
- [W3C - Timing control for script-based animations \(https://www.w3.org/TR/animation-timing/\)](https://www.w3.org/TR/animation-timing/)
- [High Performance Animations \(https://www.html5rocks.com/en/tutorials/speed/high-performance-animations/\)](https://www.html5rocks.com/en/tutorials/speed/high-performance-animations/)
- [Animating with requestAnimationFrame \(https://css-tricks.com/animating-with-requestanimationframe/\)](https://css-tricks.com/animating-with-requestanimationframe/)
- [Creating smooth animations with requestAnimationFrame \(https://www.javascripttutorial.net/web-animations-api/creating-smooth-animations-with-requestanimationframe/\)](https://www.javascripttutorial.net/web-animations-api/creating-smooth-animations-with-requestanimationframe/)

作用域和作用域链

引言

在 JavaScript 中，作用域是指变量在代码中可访问的范围。理解 JavaScript 的作用域和作用域链对于编写高质量的代码至关重要。本文将详细介绍 JavaScript 中的词法作用域、作用域链和闭包的概念，并探讨它们在实际开发中的应用场景。

1. 词法作用域

1.1 概念

词法作用域是 JavaScript 中最常见的作用域类型。它是在代码编写阶段确定的，而不是在代码执行阶段确定的。在词法作用域中，变量的访问权限是由它们在代码中的位置决定的。

1.2 示例

```
function outer() {  
  var outerVariable = "Hello";  
  
  function inner() {  
    var innerVariable = "World";  
    console.log(outerVariable + " " + innerVariable);  
  }  
  
  inner();  
}  
  
outer(); // 输出: Hello World
```

在上面的示例中，函数 `inner` 内部可以访问外部函数 `outer` 中定义的变量 `outerVariable`，这是因为它们处于词法作用域中。词法作用域确保了变量在代码编写阶段就能够正确地被访问。

1.3 词法作用域的应用场景

词法作用域在 JavaScript 中有广泛的应用场景，包括：

- 「变量访问控制」：词法作用域使得我们可以控制变量的可见性和访问权限，避免命名冲突和变量污染。
- 「模块化开发」：通过使用函数和闭包，可以实现模块化的代码组织，将变量和函数封装在私有作用域中，提供了良好的封装性和代码组织性。
- 「函数嵌套」：函数嵌套是 JavaScript 中常见的编程模式，词法作用域确保了内部函数可以访问外部函数的变量，实现了信息的隐藏和封装。

2. 作用域链

2.1 概念

作用域链是 JavaScript 中用于查找变量的一种机制。它由当前作用域和所有父级作用域的变量对象组成。当访问一个变量时，JavaScript 引擎会首先在当前作用域的变量对象中查找，如果找不到，则沿着作用域链向上查找，直到找到变量或者到达全局作用域。

Global Execution Context

```
|
+-- Function Execution Context 1
|   |
|   +-- Function Execution Context 2
|       |
|       +-- Function Execution Context 3
|
+-- Function Execution Context 4
```

2.2 示例

```
var globalVariable = "Global";

function outer() {
  var outerVariable = "Hello";

  function inner() {
    var innerVariable = "World";
    console.log(globalVariable + " " + outerVariable + " " + innerVariable);
  }

  inner();
}

outer(); // 输出: Global Hello World
```

在上面的示例中，函数 `inner` 内部可以访问全局作用域中定义的变量 `globalVariable`，以及外部函数 `outer` 中定义的变量 `outerVariable`，这是因为 JavaScript 引擎按照作用域链的顺序查找变量。

2.3 作用域链的应用场景

作用域链在 JavaScript 中有多种应用场景，包括：

- **「变量查找」**：作用域链决定了变量的查找顺序，使得 JavaScript 可以正确地找到并访问变量。
- **「闭包」**：通过创建闭包，内部函数可以访问外部函数的变量，实现了信息的保留和共享。
- **「模块化开发」**：作用域链的特性使得我们可以实现模块化的代码组织，将变量和函数封装在私有作用域中，提供了良好的封装性和代码组织性。

3. 闭包

3.1 概念

闭包是指函数和其词法环境的组合。它可以访问其词法作用域中定义的变量，即使在函数外部也可以访问这些变量。闭包在 JavaScript 中常用于创建私有变量和实现模块化开发。

3.2 示例

```
function createCounter() {  
  var count = 0;  
  
  return function() {  
    count++;  
    console.log(count);  
  };  
}  
  
var counter = createCounter();  
counter(); // 输出: 1  
counter(); // 输出: 2
```

在上面的示例中，函数 `createCounter` 返回一个内部函数，该内部函数引用了外部函数 `createCounter` 的变量 `count`。即使在外部函数执行完毕后，内部函数依然可以访问并修改变量 `count`，这就是闭包的特性。

3.3 闭包的应用场景

闭包在 JavaScript 中有多种应用场景，包括：

- 「私有变量」：闭包提供了一种实现私有变量的机制，可以隐藏变量并提供访问控制。
- 「模块化开发」：通过创建闭包，可以实现模块化的代码组织，将变量和函数封装在私有作用域中，提供了良好的封装性和代码组织性。
- 「延迟执行」：通过使用闭包，可以延迟执行函数，实现异步操作和事件处理。

4. 总结

作用域、作用域链和闭包是 JavaScript 中重要的概念，它们相互关联，共同构建了 JavaScript 的变量访问和代码组织机制。理解这些概念的原理和应用场景对于编写高质量的 JavaScript 代码至关重要。

通过词法作用域，我们可以控制变量的可见性和访问权限，实现模块化的代码组织，避免命名冲突和变量污染。

作用域链决定了变量的查找顺序，使得 JavaScript 可以正确地找到并访问变量。同时，作用域链的特性也为闭包的创建提供了基础，通过闭包，我们可以创建私有变量，实现模块化的代码组织以及延迟执行函数等。

深入理解作用域、作用域链和闭包，能够帮助我们更好地编写可维护、高效的 JavaScript 代码。

5. 参考资料

- [MDN Web Docs: Closures](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures) (<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures>)
- [MDN Web Docs: Scopes and closures](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_Objects) ([https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working with Objects](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_Objects))
- [JavaScript: Understanding Scope, Context, and Closures](https://www.sitepoint.com/javascript-scope-context-closures/) (<https://www.sitepoint.com/javascript-scope-context-closures/>)
- [JavaScript Closures: A Comprehensive Guide](https://www.freecodecamp.org/news/javascript-closures-a-comprehensive-guide/) (<https://www.freecodecamp.org/news/javascript-closures-a-comprehensive-guide/>)

函数上下文和this关键字

函数是 JavaScript 中最重要的概念之一，理解函数的定义和调用方式涉及到多个知识点，特别是函数的上下文，即函数中的 `this` 关键字，是前端面试中必考的知识点。本文将介绍函数上下文、箭头函数以及修正 `this` 指向的方法。

1. 函数作为独立函数调用

考虑以下脚本：

```
function printThis() {  
  return this;  
}  
  
console.log(printThis() === window);
```

输出结果：

- 在严格模式下：`false`
- 在非严格模式下：`true`

解析：

当函数被作为独立函数调用时，`this` 指向不同，严格模式和非严格模式下有区别。在严格模式下，`this` 指向 `undefined`；在非严格模式下，`this` 指向全局对象 `window`。

2. 函数作为对象方法调用

考虑以下脚本：

```
function printThis() {  
  return this;  
}  
  
const obj = { printThis };  
console.log(obj.printThis() === obj);
```

输出结果：`true`

解析：

当函数被作为对象的方法调用时，其中的 `this` 指向该对象本身。在上述代码中，`printThis` 函数被作为 `obj` 对象的一个方法调用，所以 `printThis` 中的 `this` 指向 `obj`，而不是全局对象 `window`。

3. 构造函数调用

考虑以下脚本：

```
function Dog() {  
  this.name = 'Puppy';  
}  
  
const dog = new Dog();  
console.log(dog.name);
```

输出结果：Puppy

解析：

在这段代码中，Dog 函数被当作构造函数调用，通过 new 关键字创建实例时，this 关键字会指向新创建的对象。因此，this.name = 'Puppy' 将在新创建的对象上设置 name 属性，最后打印出 Puppy。

4. 构造函数返回对象

考虑以下脚本：

```
const puppet = {
  rules: false
};

function Emperor() {
  this.rules = true;
  return puppet;
}

const emperor = new Emperor();
console.log(emperor.rules);
```

输出结果：false

解析：

尽管构造函数的 this 关键字指向通过构造函数构建的实例，但如果构造函数中使用 return 语句返回一个对象，则返回的对象将取代通过构造函数创建的实例。在上述代码中，Emperor 构造函数返回了 puppet 对象，因此 emperor 实例实际上就是 puppet 对象，其中的 rules 属性值为 false。

5. 函数调用时使用 call 或 apply

考虑以下脚本：

```
function greet() {
  return `Hello, ${this.name}`;
}

const person = {
  name: 'Alice'
};

console.log(greet.call(person)); // 使用call
console.log(greet.apply(person)); // 使用apply
```

输出结果：

- 使用 call：Hello, Alice!
- 使用 apply：Hello, Alice!

解析：
通过使用函数的 `call` 或 `apply` 方法，可以显式地指定函数执行时的上下文，即 `this` 的值。在上述代码中，`greet.call(person)` 和 `greet.apply(person)` 中的 `this` 都被绑定到了 `person` 对象，所以打印出的结果都是 `Hello, Alice!`。

6. 箭头函数的上下文

箭头函数的 `this` 绑定与常规函数不同，箭头函数没有自己的 `this` 值，而是捕获了封闭上下文的 `this` 值。考虑以下脚本：

```
const obj = {
  name: 'Bob',
  greet: function() {
    const arrowFunc = () => {
      return `Hello, ${this.name}!`;
    };

    return arrowFunc();
  }
};

console.log(obj.greet());
```

输出结果：`Hello, Bob!`

解析：
在这段代码中，箭头函数 `arrowFunc` 没有自己的 `this` 值，而是捕获了封闭上下文 `greet` 函数中的 `this` 值，即 `obj` 对象。所以 `this.name` 实际上指向 `obj.name`，打印出 `Hello, Bob!`。
这就是 JavaScript 函数上下文和 `this` 关键字的一些重要概念和用法。

前端跨页面通信：实现页面间的数据传递与交互

引言

在前端开发中，有时我们需要在不同的页面之间进行数据传递和交互。这种场景下，前端跨页面通信就显得尤为重要。前端跨页面通信是指在不同的页面之间传递数据、发送消息以及实现页面间的交互操作。本文将详细介绍前端跨页面通信的属性、应用场景以及实现方法，并提供一些代码示例和引用资料，帮助读者深入了解并应用这一重要的技术。

1. 前端跨页面通信的概述

前端跨页面通信是指在不同的浏览器页面或标签页之间进行数据传递和交互的过程。在单页面应用（Single-Page Application）中，这种通信往往是在同一页面的不同组件之间进行的，而在多页面应用（Multi-Page Application）中，通信涉及到不同的页面之间的数据传递和交互。

前端跨页面通信的目的是实现不同页面之间的信息共享和协作，使得用户在不同页面间的操作能够产生相应的效果和影响。通过跨页面通信，我们可以实现以下功能：

- 在不同页面之间传递数据和状态。
- 发送消息和通知。
- 同步数据和状态的更新。
- 实现页面间的协作和交互操作。

了解前端跨页面通信的属性、应用场景和实现方法对于构建复杂的前端应用和提供良好的用户体验至关重要。

2. 前端跨页面通信的属性

前端跨页面通信具有以下几个重要的属性：

2.1 双向通信

前端跨页面通信是双向的，即页面之间可以相互发送和接收消息。不仅可以从一个页面向另一个页面发送数据和消息，还可以接收来自其他页面的数据和消息。这种双向通信使得页面之间可以实现实时的数据交互和状态同步。

2.2 异步通信

前端跨页面通信是异步的，即数据和消息的传递是非阻塞的。不同页面之间可以同时发送和接收消息，不需要等待对方的响应。这种异步通信的特性使得页面间的交互能够更加流畅和高效。

2.3 安全性

前端跨页面通信的安全性是一个重要的考虑因素。由于涉及到不同页面之间的数据传递，我们需要确保通信过程的安全性，防止恶意攻击和数据泄露。

。在设计和实现跨页面通信时，需要注意采取安全的策略和机制，如数据加密、身份验证等。

2.4 可靠性

前端跨页面通信需要具备一定的可靠性，即保证消息的准确传递和接收。在网络不稳定或通信中断的情况下，应该能够恢复通信并确保数据的完整性。为了实现可靠的跨页面通信，我们可以使用合适的机制，如消息确认、重试机制等。

3. 前端跨页面通信的应用场景

前端跨页面通信可以应用于各种场景，满足不同的需求。下面介绍几个常见的应用场景：

3.1 多标签页间的数据共享

在多标签页的应用中，不同的标签页可能需要共享一些数据或状态。通过跨页面通信，可以在不同的标签页之间传递数据，使得数据的更新能够在各个标签页中同步。

例如，一个电子商务网站中的购物车功能，用户可以在一个标签页中添加商品到购物车，而在另一个标签页中也能够实时看到购物车的变化。这就需要通过跨页面通信将购物车的数据在不同标签页之间进行同步。

3.2 页面间的消息通知和事件触发

在页面间进行消息通知和事件触发是前端跨页面通信的常见应用场景之一。通过跨页面通信，可以向其他页面发送消息，通知它们发生了某个事件或状态的改变。

例如，一个在线聊天应用中，当用户在一个页面发送消息时，需要通过跨页面通信将消息发送给其他页面，以实现实时的消息同步和通知。

3.3 页面间的数据传递和共享

页面间的数据传递和共享是前端跨页面通信的核心应用场景之一。通过跨页面通信，可以在不同的页面之间传递数据，实现数据的共享和交互。

例如，一个表单提交页面和一个结果展示页面之间需要传递数据。可以通过跨页面通信将表单提交的数据传递给结果展示页面，以便展示提交结果。

3.4 协同编辑和实时协作

前端跨页面通信还可以用于实现协同编辑和实时协作的功能。通过跨页面通信，多个用户可以同时编辑同一个文档或画布，并实时看到其他用户的编辑内容。

例如，一个协同编辑的文档应用中，多个用户可以同时编辑同一个文档，并实时看到其他用户的编辑操作。这就需要通过跨页面通信将用户的编辑内容进行同步和交互。

4. 前端跨页面通信的实现方法

在前端中，有多种方法可以实现跨页面通信。下面介绍几种常用的实现方法：

4.1 Cookie

Cookie是一种在浏览器中存储数据的机制，可以通过设置Cookie的值在不同页面之间传递数据。通过设置相同的Cookie名称和值，不同的页面可以读取和修改Cookie的值，实现跨页面数据的传递和共享。

使用Cookie进行跨页面通信的示例代码如下：

```
// 在页面 A 中设置 Cookie
document.cookie = "data=example";

// 在页面 B 中读取 Cookie
const cookies = document.cookie.split(";");
for (let i = 0; i < cookies.length; i++) {
  const [name, value] = cookies[i].split("=");
  if (name === "data") {
    console.log(value); // 输出 "example"
    break;
  }
}
```

4.2 LocalStorage 和 SessionStorage

LocalStorage和SessionStorage是浏览器提供的本地存储机制，可以在不同页面之间存储和读取数据。它们的区别在于数据的生命周期，LocalStorage中的数据在浏览器关闭后仍然保留，而SessionStorage中的数据在会话结束后被清除。

使用LocalStorage进行跨页面通信的示例代码如下：

```
// 在页面 A 中存储数据到 LocalStorage
localStorage.setItem("data", "example");

// 在页面 B 中读取 LocalStorage 的数据
const data = localStorage.getItem("data");
console.log(data); // 输出 "example"
```

4.3 Broadcast Channel

Broadcast Channel是浏览器提供的API，用于在不同页面之间进行消息广播和接收。通过Broadcast Channel，我们可以创建一个频道，并在不同的页面之间发送和接收消息。

使用Broadcast Channel进行跨页面通信的示例代码如下：

```
// 在页面 A 中创建 Broadcast Channel
const channel = new BroadcastChannel("myChannel");

// 在页面 B 中监听消息
channel.addEventListener("message", (event) => {
  console.log(event.data); // 输出接收到的消息
});

// 在页面 A 中发送消息
channel.postMessage("Hello from Page A");
```

4.4 Window.postMessage

Window.postMessage是浏览器提供的API，用于在不同窗口或框架之间进行安全的跨页面通信。通过Window.postMessage，我们可以向其他窗口发送消息，并接收其他窗口发送的消息。

使用Window.postMessage进行跨页面通信的示例代码如下：

```
// 在页面 A 中发送消息给页面 B
window.opener.postMessage("Hello from Page A", "https://www.example.com");

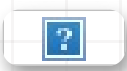
// 在页面 B 中监听消息
window.addEventListener("message", (event) => {
  if (event.origin === "https://www.example.com") {
    console.log(event.data); // 输出接收到的消息
  }
});
```

5. 参考资料

- [MDN Web Docs - Window.postMessage\(\) \(https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage\)](https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage)
- [MDN Web Docs - BroadcastChannel \(https://developer.mozilla.org/en-US/docs/Web/API/Broadcast_Channel_API\)](https://developer.mozilla.org/en-US/docs/Web/API/Broadcast_Channel_API)
- [MDN Web Docs - Cookie \(https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies)
- [MDN Web Docs - Web Storage API \(https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API\)](https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API)
- [Using the Broadcast Channel API for JavaScript communication between tabs \(https://blog.logrocket.com/using-the-broadcast-channel-api-for-javascript-communication-between-tabs/\)](https://blog.logrocket.com/using-the-broadcast-channel-api-for-javascript-communication-between-tabs/)
- [Cross-window communication using postMessage \(https://javascript.info/cross-window-communication\)](https://javascript.info/cross-window-communication)
- [Window Communication in JavaScript: A Comprehensive Guide \(https://www.telerik.com/blogs/window-communication-in-javascript-a-comprehensive-guide\)](https://www.telerik.com/blogs/window-communication-in-javascript-a-comprehensive-guide)

原型和原型链

JavaScript是一门支持面向对象编程的语言，它的函数是第一公民，同时也拥有类的概念。不同于传统的基于类的继承，JavaScript的类和继承是基于原型链模型的。在ES2015/ES6中引入了 `class` 关键字，但其本质仍然是基于原型链的语法糖。



原型（Prototype）

原型（Prototype）是JavaScript中对象的一个特殊属性，它用于实现属性和方法的继承。在JavaScript中，每个对象都有一个原型属性，它指向另一个对象，这个对象被称为原型对象。通过原型链，对象可以从原型对象继承属性和方法。

原型的概念可以用以下方式解释：每个JavaScript对象都是基于一个构造函数创建的，构造函数是对象的模板或蓝图。在创建对象时，构造函数会创建一个关联的原型对象，对象通过原型链继承原型对象上的属性和方法。原型对象是一个普通的JavaScript对象，它具有自己的属性和方法。

让我们以一个示例来说明原型的概念和作用：

```
// 构造函数
function Person(name) {
  this.name = name;
}

// 在原型对象上添加方法
Person.prototype.sayHello = function() {
  console.log("Hello, my name is " + this.name);
};

// 创建实例
var person1 = new Person("John");
var person2 = new Person("Alice");

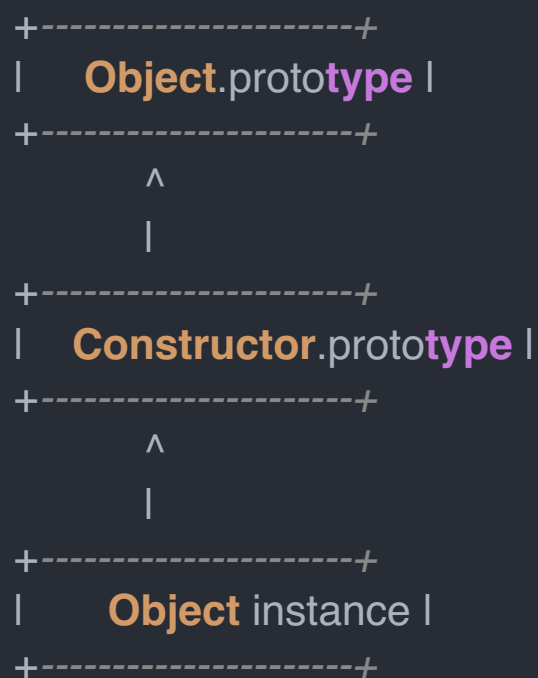
// 调用原型对象上的方法
person1.sayHello(); // 输出: "Hello, my name is John"
person2.sayHello(); // 输出: "Hello, my name is Alice"
```

在这个示例中，我们定义了一个构造函数 `Person`，它有一个 `name` 属性。然后，我们通过给原型对象 `Person.prototype` 添加一个 `sayHello` 方法，使得所有通过 `Person` 构造函数创建的实例都可以访问该方法。我们创建了两个实例 `person1` 和 `person2`，并分别调用了 `sayHello` 方法。

原型的重要性体现在以下几个方面：

- 1. 继承：原型链允许对象继承其原型对象上的属性和方法。通过原型链，子对象可以访问和复用父对象的属性和方法，实现了继承的概念。
- 2. 代码复用和共享：通过将方法和属性定义在原型对象上，可以实现多个对象共享相同的方法和属性。这样可以节省内存空间，提高性能，同时也方便了代码的维护和扩展。

下面是一个简单的原型链示意图：



在这个示意图中，`Object.prototype` 是所有对象的原型，`Constructor.prototype` 是构造函数的原型，`Object instance` 是基于构造函数创建的对象实例。

构造函数和原型对象

构造函数是用于创建对象的特殊函数。它通常以大写字母开头，通过使用 `new` 关键字来调用构造函数，我们可以创建一个新的对象实例。构造函数在创建对象时可以执行一些初始化操作，并为对象添加属性和方法。

原型对象是构造函数的一个属性，它是一个普通的 JavaScript 对象。原型对象上的属性和方法可以被通过构造函数创建的对象实例所继承。通过将属性和方法定义在原型对象上，我们可以实现方法的共享和节省内存空间。

让我们通过一个示例来说明构造函数和原型对象的概念以及如何使用它们来创建对象和共享方法：

```
// 构造函数
function Person(name, age) {
  this.name = name;
  this.age = age;
}

// 在原型对象上添加方法
Person.prototype.sayHello = function() {
  console.log("Hello, my name is " + this.name + " and I am " + this.age + " years old.");
};

// 创建对象实例
var person1 = new Person("John", 25);
var person2 = new Person("Alice", 30);

// 调用共享的方法
person1.sayHello(); // 输出: "Hello, my name is John and I am 25 years old."
person2.sayHello(); // 输出: "Hello, my name is Alice and I am 30 years old."
```

在这个示例中，我们定义了一个构造函数 `Person`，它接受 `name` 和 `age` 参数，并将它们赋值给对象的属性。然后，我们通过在原型对象 `Person.prototype` 上添加一个方法 `sayHello`，使得通过 `Person` 构造函数创建的对象实例可以访问该方法。我们创建了两个对象实例 `person1` 和 `person2`，并分别调用了共享的方法 `sayHello`。

以下是一个简单的构造函数和原型对象的流程图示意图：



在这个示意图中，构造函数和原型对象之间存在关联，构造函数拥有原型对象的引用。通过构造函数，我们可以创建对象实例，并且这些实例可以通过原型对象继承原型上的属性和方法。

原型链

原型链是 JavaScript 中对象之间通过原型链接起来的机制，用于实现属性和方法的继承。它是由一系列的原型对象组成，每个对象都有一个指向其原型对象的连接，形成了一条链式结构。

原型链的概念可以通过以下方式解释：在 JavaScript 中，每个对象都有一个内部属性 `[[Prototype]]` (`__proto__`)，它指向该对象的原型。当我们访问一个对象的属性或方法时，如果该对象本身没有这个属性或方法，JavaScript 引擎会自动沿着原型链向上查找，直到找到匹配的属性或方法或者到达原型链的顶部 (`Object.prototype`)。

让我们通过一个示例来说明原型链的概念和工作原理：

```
// 父对象构造函数
function Parent() {
  this.name = "Parent";
}

// 在父对象原型上添加方法
Parent.prototype.sayHello = function() {
  console.log("Hello, I am " + this.name);
};

// 子对象构造函数
function Child() {
  this.name = "Child";
}

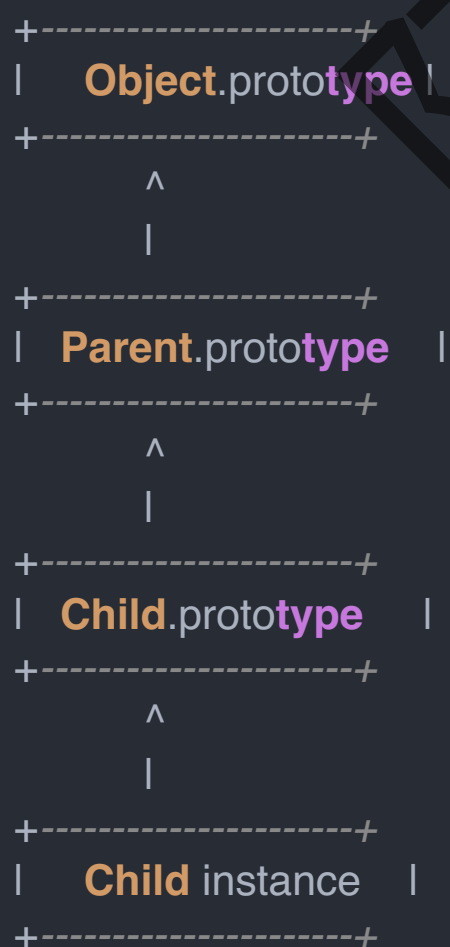
// 通过原型继承建立子对象和父对象的连接
Child.prototype = Object.create(Parent.prototype);

// 创建子对象实例
var child = new Child();

// 调用父对象原型上的方法
child.sayHello(); // 输出: "Hello, I am Child"
```

在这个示例中，我们定义了一个父对象构造函数 `Parent`，它有一个属性 `name` 和一个原型方法 `sayHello`。然后，我们定义了一个子对象构造函数 `Child`，它也有一个属性 `name`。通过 `Object.create()` 方法，我们将子对象的原型连接到父对象的原型上，建立了子对象和父对象之间的原型链关系。最后，我们创建了子对象实例 `child`，并调用了父对象原型上的方法 `sayHello`。

以下是一个简单的原型链示意图：



<!--



-->

在这个示意图中，`Object.prototype` 是所有对象的顶层原型，`Parent.prototype` 是父对象的原型，`Child.prototype` 是子对象的原型，`Child instance` 是基于子对象构造函数创建的对象实例。

原型链的重要性体现在以下几个方面：

1. 继承：原型链允许对象通过继承获取其他对象的属性和方法。子对象可以继承父对象的属性和方法，而父对象又可以继承更上层对象的属性和方法，以此类推。
2. 代码复用和共享：通过原型链，我们可以在原型对象

上定义方法和属性，从而实现多个对象之间的方法共享和代码复用。这样可以节省内存空间，提高性能，并减少代码的冗余。

3. 扩展和修改：通过在原型对象上添加新的方法和属性，我们可以在整个原型链中的所有对象实例上访问和使用这些扩展。这样可以方便地对现有对象进行功能扩展和修改。

原型继承

原型继承是一种通过继承原型对象来创建新对象的方式。在 JavaScript 中，我们可以使用多种方式实现原型继承。原型继承的概念是通过将一个对象作为另一个对象的原型来实现继承，从而使新对象可以共享原型对象的属性和方法。

1. 对象字面量和 `Object.create()`：可以使用字面量对象定义属性和方法，并使用 `Object.create()` 方法创建一个新对象，并将其原型设置为现有对象的原型。

```
var parent = {
  name: "Parent",
  sayHello: function() {
    console.log("Hello, I am " + this.name);
  }
};

var child = Object.create(parent);
child.name = "Child";
```

2. 构造函数和 `Object.create()`：可以使用构造函数定义对象，并通过 `Object.create()` 方法将新对象的原型连接到现有对象的原型上。

```
function Parent(name) {
  this.name = name;
}

Parent.prototype.sayHello = function() {
  console.log("Hello, I am " + this.name);
};

function Child(name) {
  Parent.call(this, name);
}

Child.prototype = Object.create(Parent.prototype);
Child.prototype.constructor = Child;

var child = new Child("Child");
```

3. 构造函数和 **new** 关键字：可以使用构造函数创建对象实例，并使用 **new** 关键字进行实例化。

```
function Parent(name) {
  this.name = name;
}

Parent.prototype.sayHello = function() {
  console.log("Hello, I am " + this.name);
};

function Child(name) {
  Parent.call(this, name);
}

Child.prototype = new Parent();
Child.prototype.constructor = Child;

var child = new Child("Child");
```

4. 寄生组合继承

寄生组合继承是一种常用的原型继承方式，结合了构造函数继承和原型链继承的优点，避免了原型链中不必要的属性复制和方法重复定义的问题。这种方式先通过构造函数继承属性，然后通过设置原型链继承方法。

```
function Parent(name) {
  this.name = name;
}

Parent.prototype.sayHello = function() {
  console.log("Hello, I am " + this.name);
};

function Child(name, age) {
  Parent.call(this, name);
  this.age = age;
}

Child.prototype = Object.create(Parent.prototype);
Child.prototype.constructor = Child;

var child = new Child("Child", 10);
```

以上是常用的原型继承实现方式，每种方式都有其特点和适用场景。根据具体的需求和代码结构，可以选择最适合的方式来实现原型继承。

参考资料

- [MDN Web Docs - Object.create\(\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create) (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create)
- [MDN Web Docs - Inheritance and the prototype chain](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain) (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain)
- [JavaScript.info - Prototypal Inheritance](https://javascript.info/prototype-inheritance) (<https://javascript.info/prototype-inheritance>)
- [Eloquent JavaScript - Object-Oriented Programming](https://eloquentjavascript.net/06_object.html) (https://eloquentjavascript.net/06_object.html)

实现符合Promise/A+规范的Promise

介绍：

Promise是JavaScript中处理异步操作的重要工具之一。Promise/A+规范是一种关于Promise实现的标准，它定义了Promise的行为和方法。本文将详细介绍如何实现Promise/A+规范，让你了解Promise的工作原理并能够自己实现一个符合规范的Promise。

Promise/A+规范简介

1. Promise的三种状态：

- pending（进行中）：Promise的初始状态，表示异步操作正在执行。
- fulfilled（已完成）：异步操作成功完成，并返回一个值，称为解决值（fulfillment value）。
- rejected（已拒绝）：异步操作失败或被拒绝，并返回一个原因（reason），通常是一个错误对象。

2. 状态转换：

- Promise的状态只能从pending转变为fulfilled或rejected，一旦转变就不可逆转。
- 状态转换是由异步操作的结果决定的。如果异步操作成功完成，Promise的状态会转变为fulfilled；如果异步操作失败或被拒绝，Promise的状态会转变为rejected。

3. Promise的基本方法：

- **then** 方法：用于注册异步操作成功完成时的回调函数，并返回一个新的Promise对象。它接受两个参数：onFulfilled（可选，异步操作成功时的回调函数）和onRejected（可选，异步操作失败时的回调函数）。
- **catch** 方法：用于注册异步操作失败时的回调函数，并返回一个新的Promise对象。它是 **then** 方法的一个特殊形式，仅用于捕获异常。
- **finally** 方法：无论异步操作成功或失败，都会执行的回调函数，并返回一个新的Promise对象。它在Promise链中的最后执行，并且不接收任何参数。

4. 错误冒泡和异常传递：

- Promise/A+规范要求Promise的错误能够被适当地捕获和处理。当一个Promise发生错误时，它会向下传播，直到找到最近的错误处理函数为止。
- 在Promise链中的任何一个Promise发生错误，都会导致整个链上的错误处理函数被调用，以便进行错误处理和恢复。

遵循Promise/A+规范的实现应该具备上述特性，以确保一致的Promise行为和接口。这样，开发者可以编写通用的异步代码，而无需担心特定Promise实现的差异性。

实现Promise

当从零开始实现 Promise/A+ 规范的 Promise，我们需要逐步构建 Promise 的核心功能，包括状态管理、状态转换、回调处理和错误处理。

步骤 1: 创建 Promise 构造函数

首先，我们需要创建一个 Promise 构造函数，它接受一个执行器函数作为参数。执行器函数接受两个参数，即 resolve 和 reject 函数，用于控制 Promise 的状态转换。

```
function MyPromise(executor) {  
  // TODO: 实现构造函数  
}
```

步骤 2: 初始化 Promise 状态和回调

在构造函数中，我们需要初始化 Promise 的状态和回调数组。状态可以使用一个变量来表示，初始值为 'pending'。回调数组用于存储注册的成功和失败回调函数。

```
function MyPromise(executor) {  
  var self = this;  
  self.state = 'pending';  
  self.value = undefined;  
  self.reason = undefined;  
  self.onFulfilledCallbacks = [];  
  self.onRejectedCallbacks = [];  
  
  // TODO: 实现构造函数的其余部分  
}
```

步骤 3: 实现 resolve 和 reject 函数

我们需要实现 resolve 和 reject 函数，用于将 Promise 的状态从 'pending' 转换为 'fulfilled' 或 'rejected'。resolve 函数将传递一个值来兑现 Promise，而 reject 函数将传递一个原因来拒绝 Promise。

```
function MyPromise(executor) {
  var self = this;
  self.state = 'pending';
  self.value = undefined;
  self.reason = undefined;
  self.onFulfilledCallbacks = [];
  self.onRejectedCallbacks = [];

  function resolve(value) {
    if (self.state === 'pending') {
      self.state = 'fulfilled';
      self.value = value;
      self.onFulfilledCallbacks.forEach(function(callback) {
        callback(self.value);
      });
    }
  }

  function reject(reason) {
    if (self.state === 'pending') {
      self.state = 'rejected';
      self.reason = reason;
      self.onRejectedCallbacks.forEach(function(callback) {
        callback(self.reason);
      });
    }
  }

  try {
    executor(resolve, reject);
  } catch (e) {
    reject(e);
  }
}
```

步骤 4: 实现 then 方法

接下来，我们需要实现 then 方法，用于注册成功和失败的回调函数，并返回一个新的 Promise。then 方法接受两个参数：成功回调函数和失败回调函数。


```

function MyPromise(executor) {
  var self = this;
  self.state = 'pending';
  self.value = undefined;
  self.reason = undefined;
  self.onFulfilledCallbacks = [];
  self.onRejectedCallbacks = [];

  function resolve(value) {
    if (self.state === 'pending') {
      self.state = 'fulfilled';
      self.value = value;
      self.onFulfilledCallbacks.forEach(function(callback) {
        callback(self.value);
      });
    }
  }

  function reject(reason) {
    if (self.state === 'pending') {
      self.state = 'rejected';
      self.reason = reason;
      self.onRejectedCallbacks.forEach(function(callback) {
        callback(self.reason);
      });
    }
  }

  try {
    executor
  } catch (e) {
    reject(e);
  }
}

MyPromise.prototype.then = function(onFulfilled, onRejected) {
  var self = this;
  onFulfilled = typeof onFulfilled === 'function' ? onFulfilled : function(value) { return value; };
  onRejected = typeof onRejected === 'function' ? onRejected : function(reason) { throw reason; };

  var newPromise = new MyPromise(function(resolve, reject) {
    // TODO: 实现 then 方法的其余部分
  });

  return newPromise;
};

```

步骤 5: 处理 Promise 状态转换和回调执行

我们需要在 `then` 方法中处理 Promise 的状态转换和回调的执行。根据当前 Promise 的状态，我们可以立即执行回调函数或将回调函数添加到相应的回调数组中。

```

MyPromise.prototype.then = function(onFulfilled, onRejected) {
  var self = this;
  onFulfilled = typeof onFulfilled === 'function' ? onFulfilled : function(value) { return value; };
  onRejected = typeof onRejected === 'function' ? onRejected : function(reason) { throw reason; };

  var newPromise = new MyPromise(function(resolve, reject) {
    function handleFulfilled(value) {
      try {
        var x = onFulfilled(value);
        resolvePromise(newPromise, x, resolve, reject);
      } catch (e) {
        reject(e);
      }
    }

    function handleRejected(reason) {
      try {
        var x = onRejected(reason);
        resolvePromise(newPromise, x, resolve, reject);
      } catch (e) {
        reject(e);
      }
    }

    if (self.state === 'fulfilled') {
      setTimeout(function() {
        handleFulfilled(self.value);
      }, 0);
    } else if (self.state === 'rejected') {
      setTimeout(function() {
        handleRejected(self.reason);
      }, 0);
    } else if (self.state === 'pending') {
      self.onFulfilledCallbacks.push(function(value) {
        setTimeout(function() {
          handleFulfilled(value);
        }, 0);
      });
      self.onRejectedCallbacks.push(function(reason) {
        setTimeout(function() {
          handleRejected(reason);
        }, 0);
      });
    }
  });

  return newPromise;
};

```

步骤 6: 解析 Promise

最后，我们需要实现 `resolvePromise` 函数，用于解析 `Promise`。它会处理 `thenable` 和非 `thenable` 值，并根据其状态执行相应的处理。

```
function resolvePromise(promise, x, resolve, reject) {
  if (promise === x) {
    reject(new TypeError('Circular reference detected.'));
  }

  if (x && typeof x === 'object' || typeof x === 'function') {
    var called = false;

    try {
      var then = x.then;

      if (typeof then === 'function') {
        then.call(
          x,
          function(y) {
            if (!called) {
              called = true;
              resolvePromise(promise, y, resolve, reject);
            }
          },
          function(r) {
            if (!called) {
              called = true;
              reject(r);
            }
          }
        );
      } else {
        resolve(x);
      }
    } catch (e) {
      if (!called) {
        called = true;
        reject(e);
      }
    }
  } else {
    resolve(x);
  }
}
```

Promise的测试与调试

1. 安装Jest：

确保在项目中安装了Jest。可以使用npm或yarn进行安装。

```
npm install jest --save-dev
```

2. 编写单元测试：

在项目中创建一个测试文件，以 `.test.js` 为后缀，编写单元测试用例来验证Promise的各个功能和方法的正确性。例如，可以编写测试用例来验证状态转换、回调函数的执行、链式调用等方面的行为是否符合预期。

```
// promise.test.js

const { MyPromise } = require('./promise');

describe('MyPromise', () => {
  it('should fulfill with correct value', () => {
    const promise = new MyPromise((resolve, reject) => {
      setTimeout(() => {
        resolve('success');
      }, 100);
    });

    return promise.then((value) => {
      expect(value).toBe('success');
    });
  });

  it('should reject with correct reason', () => {
    const promise = new MyPromise((resolve, reject) => {
      setTimeout(() => {
        reject(new Error('failure'));
      }, 100);
    });

    return promise.catch((reason) => {
      expect(reason).toBeInstanceOf(Error);
      expect(reason.message).toBe('failure');
    });
  });

  // 更多测试用例...
});
```

3. 运行测试：

使用Jest运行编写的测试用例，执行Promise的测试。

```
npx jest
```

4. 模拟异步操作：

使用 `setTimeout` 函数或 `Promise.resolve` 等方法来模拟异步操作，并确保Promise在正确的时机进行状态转换和回调函数的执行。例如，可以使用 `setTimeout` 来模拟异步操作的完成。

```
const promise = new MyPromise((resolve, reject) => {
  setTimeout(() => {
    resolve('success');
  }, 100);
});

promise.then((value) => {
  console.log(value); // 输出: success
});
```

5. 调试Promise链：

在开发过程中，可能会遇到Promise链上的问题，如回调函数不执行、结果不符合预期等。可以使用 `console.log` 或 `debugger` 语句来打印中间变量的值，或者使用Jest的调试功能来逐步跟踪Promise链的执行过程。

```
const promise1 = new MyPromise((resolve, reject) => {
  setTimeout(() => {
    resolve('success');
  }, 100);
});

const promise2 = promise1.then((value) => {
  console.log(value); // 输出: success
  return value.toUpperCase();
});

promise2.then((value) => {
  console.log(value); // 输出: SUCCESS
});
```

可以使用Jest的 `--inspect` 参数进行调试，或者在代码中添加 `debugger` 语句来触发断点。

```
npx jest --inspect
```

使用Promise/A+测试套件

使用Promise/A+测试套件是确保Promise实现符合规范的重要步骤。Promise/A+测试套件是一组针对Promise实现的测试用例，可用于验证Promise是否符合Promise/A+规范的要求。

以下是使用Promise/A+测试套件的步骤：

1. 下载Promise/A+测试套件：
首先，从Promise/A+官方的GitHub仓库（<https://github.com/promises-aplus/promises-tests>）下载Promise/A+测试套件的代码。将其保存到项目的测试目录中。
2. 集成测试套件：
将Promise/A+测试套件集成到项目的测试环境中，确保可以运行测试套件并获得结果。
3. 实现Promise接口：
根据Promise/A+规范的要求，实现一个符合规范的Promise类。确保Promise类的行为和接口与规范一致。

- 运行测试套件：
使用测试框架（如Mocha、Jest等）运行Promise/A+测试套件。在测试套件的配置中，指定测试文件为Promise/A+测试套件的入口文件。
- 验证结果：
查看测试套件的运行结果。如果所有的测试用例都通过，表示Promise实现符合Promise/A+规范。如果有测试用例失败，根据测试结果来调试和修复Promise实现中的问题。

下面是一个示例，展示如何使用Promise/A+测试套件进行测试：

```
// 安装Promise/A+测试套件
npm install promises-aplus-tests --save-dev

// 集成Promise/A+测试套件到测试环境中
const promisesAplusTests = require('promises-aplus-tests');
const { MyPromise } = require('./promise');

// 运行Promise/A+测试套件
promisesAplusTests(MyPromise, function (err) {
  // 测试完成后的回调函数
  if (err) {
    console.error('Promise/A+测试失败: ', err);
  } else {
    console.log('Promise/A+测试通过! ');
  }
});
```

在上述代码中，`MyPromise` 是自己实现的Promise类。通过将`MyPromise`传递给`promisesAplusTests`函数，将Promise类集成到Promise/A+测试套件中。运行测试后，将会得到测试结果。

通过使用Promise/A+测试套件，可以验证自己实现的Promise是否符合Promise/A+规范的要求，确保Promise的行为和接口的一致性。

Promise其它API

要实现`Promise.all`和`Promise.race`等其他API，可以根据Promise的规范和功能需求来编写相应的代码。以下是对这两个API的实现进行展开讲解的代码示例：

- 实现`Promise.all`：
`Promise.all`方法接收一个可迭代对象（如数组或类数组对象），并返回一个新的Promise，该Promise在所有输入的Promise都成功完成时才会成功，否则将会失败。返回的Promise的解决值是一个由所有输入Promise解决值组成的数组。

```
Promise.all = function (promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let completedCount = 0;

    const checkCompletion = () => {
      if (completedCount === promises.length) {
        resolve(results);
      }
    };

    for (let i = 0; i < promises.length; i++) {
      promises[i]
        .then((value) => {
          results[i] = value;
          completedCount++;
          checkCompletion();
        })
        .catch((reason) => {
          reject(reason);
        });
    }

    if (promises.length === 0) {
      resolve(results);
    }
  });
};
```

使用示例：

```
const promise1 = new Promise((resolve) => setTimeout(() => resolve(1), 1000));
const promise2 = new Promise((resolve) => setTimeout(() => resolve(2), 2000));
const promise3 = new Promise((resolve) => setTimeout(() => resolve(3), 1500));

Promise.all([promise1, promise2, promise3])
  .then((results) => {
    console.log(results); // 输出: [1, 2, 3]
  })
  .catch((reason) => {
    console.error(reason);
  });
```

2. 实现 `Promise.race`：

`Promise.race` 方法接收一个可迭代对象（如数组或类数组对象），并返回一个新的 Promise，该 Promise 将与最先解决或拒绝的输入 Promise 具有相同的状态。

```
Promise.race = function (promises) {
  return new Promise((resolve, reject) => {
    for (let i = 0; i < promises.length; i++) {
      promises[i]
        .then((value) => {
          resolve(value);
        })
        .catch((reason) => {
          reject(reason);
        });
    }
  });
};
```

使用示例：

```
const promise1 = new Promise((resolve) => setTimeout(() => resolve(1), 1000));
const promise2 = new Promise((resolve) => setTimeout(() => resolve(2), 2000));
const promise3 = new Promise((resolve) => setTimeout(() => resolve(3), 1500));

Promise.race([promise1, promise2, promise3])
  .then((value) => {
    console.log(value); // 输出: 1
  })
  .catch((reason) => {
    console.error(reason);
  });
```

参考资料

- Promise/A+ 规范官方文档：<https://promisesaplus.com/>

异步的终极解决方案：async/await

1. 背景

在深入讨论 async/await 之前，我们需要了解一下 JavaScript 的单线程和非阻塞的特性。JavaScript 是单线程的，也就是说在任何给定的时间点，只能执行一个操作。然而，对于需要大量时间的操作（例如从服务器获取数据），如果没有适当的管理机制，这种单线程特性可能会导致应用程序的阻塞。为了解决这个问题，JavaScript 引入了回调函数和后来的 Promise，用来管理这些异步操作。

然而，回调函数和 Promise 还是存在一些问题。回调函数很容易导致 "回调地狱"，因为每个异步操作都需要一个回调函数，如果有很多这样的操作，代码就会变得非常混乱。Promise 解决了这个问题，让异步代码更加直观，但是，Promise 的链式调用有时候还是显得不够直观。

为了结合 Promise 和生成器的优势，Async/await 在 ECMAScript 2017 (ES8) 中被引入。它通过 async 函数和 await 表达式提供了一种更加直观和简洁的方式来编写异步代码，消除了回调函数和手动管理 Promise 的需要。

2. 使用方法

Async/await的使用方法非常简单明了，主要涉及两个关键字：`async`和`await`。

- `async` 关键字：用于声明一个`async`函数，它返回一个`Promise`对象。在`async`函数内部，我们可以使用`await`关键字来暂停函数的执行，等待一个异步操作的完成，并获得其结果。在这个过程中，`async`函数会暂时释放线程的控制权，使其他代码可以继续执行。
- `await` 关键字：用于暂停`async`函数的执行，等待一个`Promise`对象的完成，并返回其解析的值。它只能在`async`函数内部使用。当使用`await`表达式时，代码的执行会暂停，直到`Promise`对象被解析或拒绝。

下面是一个示例，展示了Async/await的使用方法：

```
async function getData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Error:', error);
    throw error;
  }
}

getData()
  .then(data => console.log('Data:', data))
  .catch(error => console.error('Error:', error));
```

在上面的示例中，`getData`函数是一个`async`函数，它等待`fetch`函数返回的`Promise`对象，并使用`await`关键字获取响应的数据。最后，我们使用`.then`方法处理返回的数据，或使用`.catch`方法处理可能发生的错误。

3. 实现原理

Async/Await 的实现原理其实就是 Generator + Promise。我们知道 Generator 可以在 `yield` 关键字处暂停和恢复执行，Promise 可以处理异步操作，两者结合在一起，就可以实现一个类似于 `async/await` 的功能。

```
function promiseFn() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve('promise resolved');
    }, 2000);
  });
}

function* genFn() {
  let result = yield promiseFn();
  console.log(result);
}

function asyncToGenerator(generator) {
  let gen = generator();
  return new Promise((resolve, reject) => {
    function step(key, arg) {
      let result;
      try {
        result = gen[key](arg);
      } catch (error) {
        return reject(error);
      }
      const { value, done } = result;
      if (done) {
        return resolve(value);
      } else {
        return Promise.resolve(value).then(val => {
          step('next', val);
        }, err => {
          step('throw', err);
        });
      }
    }
    step('next');
  });
}

asyncToGenerator(genFn);
```

在上述代码中，我们首先创建了一个 `promiseFn` 函数，该函数返回一个在 2 秒后解析的 `Promise`。然后，我们创建了一个 `Generator` 函数 `genFn`，在该函数内部，我们使用 `yield` 关键字暂停执行并等待 `promiseFn` 的结果。最后，我们创建了一个 `asyncToGenerator` 函数，该函数接受一个 `Generator` 函数作为参数，并返回一个新的 `Promise`，这个 `Promise` 的解析值就是 `Generator` 函数的返回值。

4. 参考资料

- [MDN Web Docs: Async functions](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function) (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function)
- [MDN Web Docs: await](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await) (<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await>)

执行上下文与闭包

一、由来

JavaScript中的闭包源于计算机科学中的一种理论概念，称为“λ演算”（Lambda Calculus）。λ演算是计算机科学的基础之一，1930年由Alonzo Church提出，它是一种用于描述计算过程的数学抽象模型，也是函数式编程语言的基础。

在JavaScript中，闭包是函数和声明该函数的词法环境的组合。这个环境包含了闭包创建时所能访问的所有局部变量。

理解闭包，需要理解JavaScript的特性和工作原理。JavaScript的函数在创建时，就确定了其操作的上下文环境，即词法作用域。这是因为JavaScript采用的是静态作用域，也叫词法作用域，函数的作用域在函数定义的时候就决定了。

例如：

```
function outer() {  
  var name = 'JavaScript';  
  function inner() {  
    console.log(name);  
  }  
  return inner;  
}  
  
var innerFunc = outer();  
innerFunc(); // 输出 'JavaScript'
```

在这个例子中，`outer`函数返回了`inner`函数。`inner`函数访问了`outer`函数的局部变量`name`，因此形成了一个闭包。即使`outer`函数执行完毕，`name`变量的引用仍然被保留，因此`innerFunc`在执行时仍然能够输出 `'JavaScript'`。

闭包的概念虽然来自计算机科学的深层理论，但在日常的JavaScript编程中，它是一个非常实用且常见的特性，被广泛用于如数据隐藏和封装、模块化编程、回调函数和计时器等许多场景中。

二、JavaScript中的闭包

在JavaScript中，闭包是一个强大而复杂的特性，理解和利用好闭包对于编写高效且安全的代码至关重要。下面就让我们深入地了解一下JavaScript的闭包。

闭包是指那些能够访问自由变量的函数。什么是自由变量呢？如果一个变量在函数内部被引用，但它既不是函数的参数也不是函数的局部变量，那么就称之为“自由变量”。

例如，我们有一个外部函数和一个内部函数：

```
function outerFunction(outerVariable) {  
  function innerFunction() {  
    console.log(outerVariable);  
  }  
  return innerFunction;  
}  
  
var inner = outerFunction('Hello Closure');  
inner(); // 输出 'Hello Closure'
```

在这个例子中，`outerFunction`是一个外部函数，接受一个参数`outerVariable`。它包含一个内部函数`innerFunction`，这个内部函数没有自己的参数或局部变量，但却引用了外部函数的变量`outerVariable`。所以，我们说`innerFunction`是一个闭包，而`outerVariable`就是它的自由变量。

需要注意的是，由于JavaScript的垃圾回收机制，如果一个变量离开了它的作用域，那么这个变量就会被回收。但是，由于 `innerFunction` 是一个闭包，它引用了 `outerVariable`，所以即使 `outerFunction` 执行完毕，`outerVariable` 离开了它的作用域，但仍然不会被垃圾回收机制回收。

再者，每次调用外部函数，都会为内部的闭包创建一个新的作用域。例如：

```
var inner1 = outerFunction('Hello Closure 1');
var inner2 = outerFunction('Hello Closure 2');
inner1(); // 输出 'Hello Closure 1'
inner2(); // 输出 'Hello Closure 2'
```

这里，`inner1` 和 `inner2` 是两个不同的闭包。他们分别有自己的作用域，储存了不同的 `outerVariable`。

三、执行上下文与闭包

在JavaScript中，执行上下文（execution context）是一个关键概念，与闭包（closure）密切相关。理解执行上下文如何与闭包交互可以帮助我们深入理解闭包的工作原理和行为。

执行上下文是JavaScript代码执行时的环境。它包含了变量、函数声明、作用域链等信息，用于管理和跟踪代码的执行过程。当一个函数被调用时，就会创建一个新的执行上下文。每个执行上下文都有自己的词法环境（Lexical Environment），用于存储变量和函数的声明。

在理解闭包之前，让我们先了解一下执行上下文的创建和销毁过程。当函数被调用时，会创建一个新的执行上下文，并将其推入执行上下文栈（execution context stack）中。当函数执行完毕后，其执行上下文会从栈中弹出并销毁。

现在，让我们通过一个例子来更具体地了解执行上下文和闭包之间的关系：

```
function outerFunction(outerVariable) {
  function innerFunction(innerVariable) {
    console.log('outerVariable:', outerVariable);
    console.log('innerVariable:', innerVariable);
  }
  return innerFunction;
}

var newFunction = outerFunction('outside');
newFunction('inside'); // 输出: outerVariable: outside innerVariable: inside
```

在这个例子中，当调用 `outerFunction` 时，会创建一个新的执行上下文，其中包含了 `outerVariable` 参数和 `innerFunction` 函数声明。然后，`outerFunction` 返回了 `innerFunction`，并将其赋值给变量 `newFunction`。

现在让我们来看看闭包是如何形成的。当 `innerFunction` 被返回时，它会携带其词法环境（包含 `outerVariable`）一起返回。这意味着 `innerFunction` 保持对 `outerVariable` 的引用，即使 `outerFunction` 执行完毕并且其执行上下文已经销毁。

这就是闭包的力量所在。它允许内部函数（`innerFunction`）访问其词法环境中的变量（`outerVariable`），即使这些变量在其创建时的执行上下文已经不存在。

在这个例子中，`newFunction` 就是一个闭包。它引用了外部函数 `outerFunction` 的词法环境，其中包含了 `outerVariable` 变量。因此，当我们调用 `newFunction` 时，它可以访问并打印出 `outerVariable` 和 `innerVariable` 的值。

执行上下文和闭包的关系是密不可分的。闭包是由执行上下文中的变量引用形成的，而这些变量保留在闭包的作用域中。这使得闭包能够在函数执行完成后继续访问这些变量，实现了JavaScript中非常重要的特性。

理解执行上下文和闭包的交互对于编写复杂的JavaScript代码非常重要。它有助于我们更好地理解作用域、变量的生命周期以及如何正确使用闭包来解决问题。同时，它也帮助我们避免一些潜在的问题，如内存泄漏和不必要的资源消耗。

四、闭包的应用场景

闭包在JavaScript中有广泛的应用场景，它是一种强大的编程工具，可以解决许多常见的问题。下面我们来介绍一些常见的闭包应用场景。

1. 数据封装和私有性

闭包可以用于创建私有变量，将变量隐藏在函数作用域内部，从而实现数据的封装和私有性。通过闭包，我们可以控制变量的访问权限，只暴露需要暴露的接口。这种封装机制可以防止外部代码直接访问和修改内部数据，增加代码的安全性。

```
function createCounter() {
  let count = 0;
  return {
    increment: function () {
      count++;
    },
    decrement: function () {
      count--;
    },
    getCount: function () {
      return count;
    }
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 输出: 2
```

在这个例子中，`createCounter`函数返回一个对象，该对象包含了三个闭包函数，分别用于增加计数、减少计数和获取计数值。通过闭包，我们可以将`count`变量隐藏在函数内部，并通过闭包函数来操作和访问这个变量。

2. 模块化编程

闭包可以用于实现模块化编程，将相关的变量和函数组织在一个闭包内部，形成一个模块。这样可以避免全局命名冲突，提供命名空间，并且允许模块内部的函数相互调用和共享数据。

```
var myModule = (function () {
  var privateVariable = '私有变量';

  function privateFunction() {
    console.log('私有函数');
  }

  return {
    publicMethod: function () {
      console.log(privateVariable);
    },
    publicFunction: function () {
      privateFunction();
    }
  };
})();

myModule.publicMethod(); // 输出: 私有变量
myModule.publicFunction(); // 输出: 私有函数
```

在这个例子中，我们使用了立即调用函数表达式（IIFE）来创建一个闭包，形成一个独立的模块。模块内部的变量和函数对外部是不可见的，只有通过公共接口才能访问。

3. 回调函数和事件处理

闭包常常用于处理回调函数和事件处理，特别是在异步编程中。由于闭包的特性，它可以捕获外部函数的上下文，并在内部函数被调用时保留这个上下文，从而实现对异步操作的响应。

```
function fetchData(url, callback) {
  fetch(url).then(function (response) {
    return response.json();
  }).then(function (data) {
    callback(data);
  });
}

function processData(data) {
  console
  .log(data);
}

fetchData('https://api.example.com/data', processData);
```

在这个例子中，`fetchData` 函数通过闭包捕获了 `processData` 函数作为回调函数。当异步操作完成时，它会调用回调函数并传递数据给它。闭包保持了回调函数的上下文，使得回调函数可以访问外部的 `processData` 函数。

4. 缓存和记忆化

闭包还可以用于实现缓存和记忆化功能。通过闭包，我们可以在函数内部维护一个缓存，避免重复计算相同的结果，提高函数执行的性能。


```
function memoizedFunction() {
  var cache = {};
  return function (arg) {
    if (cache[arg]) {
      return cache[arg];
    }
    // 计算结果
    var result = // ...
    cache[arg] = result;
    return result;
  };
}

var memoized = memoizedFunction();
console.log(memoized('value')); // 第一次计算并缓存结果
console.log(memoized('value')); // 直接从缓存中读取结果
```

在这个例子中，`memoizedFunction` 返回一个闭包函数，用于记忆化计算结果。闭包内部维护了一个缓存对象 `cache`，当输入相同的参数时，直接从缓存中读取结果，避免重复计算。

闭包在JavaScript中有许多其他的应用场景，如实现延迟执行、函数柯里化、实现迭代器等。了解闭包的应用场景可以帮助我们写出更加优雅、高效的代码，并利用闭包的强大能力解决问题。

五、闭包的优缺点

当谈到闭包的缺点时，主要涉及内存消耗、内存泄漏和性能影响。下面是一些代码示例，帮助我们理解这些缺点。

「1. 内存消耗」

闭包会导致内存占用增加，因为它们会保留对外部变量的引用，即使外部函数执行完毕。这可能会导致内存占用过高。

```
function createHugeArray() {
  var arr = new Array(1000000).fill('Huge Data');
  return function() {
    console.log(arr.length);
  };
}

var bigDataFunc = createHugeArray();
bigDataFunc(); // 输出: 1000000
```

在这个例子中，`createHugeArray` 函数返回一个闭包函数，它引用了一个巨大的数组 `arr`。即使 `createHugeArray` 执行完毕，`arr` 仍然被闭包引用，无法被垃圾回收机制回收，从而导致内存占用增加。

「2. 内存泄漏」

由于闭包会持有对外部变量的引用，如果不正确地处理闭包的使用，可能会导致内存泄漏。如果一个闭包长时间存在，但不再需要，它会一直持有对外部变量的引用，使这些变量无法被垃圾回收。


```
function leakMemory() {  
  var data = 'Sensitive Data';  
  var timer = setInterval(function() {  
    console.log(data);  
  }, 1000);  
}  
  
leakMemory();
```

在这个例子中，`leakMemory` 函数创建了一个闭包，它引用了一个定时器内部的函数。即使 `leakMemory` 执行完毕，定时器仍然在持续执行，因此闭包会一直存在并引用 `data` 变量，导致 `data` 无法被垃圾回收。

「3. 性能影响」

闭包可能对性能产生一定的影响，特别是在涉及大量变量或复杂词法环境的情况下。闭包的创建和执行可能消耗更多的时间和资源。

```
function calculate() {  
  var result = 0;  
  for (var i = 0; i < 1000000; i++) {  
    result += i;  
  }  
  return function() {  
    console.log(result);  
  };  
}  
  
var expensiveFunc = calculate();  
expensiveFunc(); // 输出: 499999500000
```

在这个例子中，`calculate` 函数返回一个闭包函数，它引用了一个在循环中计算的结果。由于闭包保留了这个结果，闭包的执行可能会耗费更多的时间和资源。

为了减少闭包的缺点，我们可以采取以下措施：

- 优化内存使用：在闭包中避免持有大量数据或不必要的引用。确保只保留必要的变量和引用。
- 及时清理闭包：在不需要使用闭包时，手动解除对闭包的引用，以便垃圾回收机制可以回收闭包相关的资源。
- 避免滥用闭包：只在必要的情况下使用闭包，避免在不必要的场景中使用闭包。
- 优化性能：在闭包的创建和使用过程中，尽量避免不必要的计算或资源消耗，以提高性能。

通过合理使用和处理闭包，我们可以最大限度地减少其缺点，同时享受闭包在JavaScript中带来的强大功能。

深入理解Proxy

在现代JavaScript中，Proxy是一种非常有用的特性，它允许我们在许多常规操作中插入自定义行为。然而，由于其深度和复杂性，很多开发者可能会对如何使用它或它的工作原理感到困惑。在本篇文章中，我们将详细讨论JavaScript Proxy，并通过代码示例演示其使用。

Proxy是什么？

在JavaScript中，Proxy是一个特殊的“包装器”对象，它可以用于修改或扩展某些基本操作的行为，比如属性读取、函数调用等。这种修改或扩展的行为是通过所谓的"traps"实现的，这些"traps"定义了如何拦截和改变基本操作。

以下是一个简单的例子，显示了如何使用Proxy拦截对象的属性读取操作：

```
let target = {
  name: "target"
};

let proxy = new Proxy(target, {
  get: function(target, property) {
    return property in target ? target[property] : "Default";
  }
});

console.log(proxy.name); // 输出 "target"
console.log(proxy.unknown); // 输出 "Default"
```

在上面的例子中，当我们尝试从proxy读取不存在的属性时，我们得到了"default"，而不是通常的"undefined"。这是因为我们的"get" trap拦截了读取操作，并返回了默认值。

Proxy的用途

Proxy有许多用途，下面是一些常见的例子：

数据校验

Proxy可以用于校验设置对象属性的值：

```
let validator = {
  set: function(target, property, value) {
    if (property === "age") {
      if (!Number.isInteger(value)) {
        throw new TypeError("The age is not an integer");
      }
      if (value < 0 || value > 200) {
        throw new RangeError("The age is invalid");
      }
    }

    target[property] = value;
    return true;
  }
};

let person = new Proxy({}, validator);

person.age = 100; // 正常
console.log(person.age); // 输出 100
person.age = "young"; // 抛出 TypeError: The age is not an integer
person.age = 300; // 抛出 RangeError: The age is invalid
```

数据绑定和观察

Proxy可以用于实现数据绑定和观察（数据变化的监听）：

```
let handler = {
  set: function(target, property, value) {
    console.log( `${property} is set to ${value}` );
    target[property] = value;
    return true;
  }
};

let proxy = new Proxy({}, handler);

proxy.name = "proxy"; // 输出 "name is set to proxy"
```

函数参数的默认值

Proxy可以用于给函数参数设置默认值：

```
function defaultValues(target, defaults) {
  return new Proxy(target, {
    apply: function(target, thisArg, args) {
      args = args.map((arg, index) => arg === undefined ? defaults[index] : arg);

      return target.apply(thisArg, args);
    }
  });
}

let add = defaultValues(function(x, y) {
  return x + y;
}, [0, 0]);

console.log(add(1, 1)); // 输出 2
console.log(add(undefined, 1)); // 输出 1
console.log(add(1)); // 输出 1
```

以上仅仅是Proxy能做的事情的一部分。在实际开发中，你可以根据需要灵活使用Proxy。

Proxy vs Reflect

在ES6中引入了另一个新的全局对象 [Reflect](#)，它提供了一组用于执行JavaScript基本操作的方法，例如 [Reflect.get\(\)](#)，[Reflect.set\(\)](#) 等。这些方法与Proxy的traps一一对应。这使得Proxy的traps可以使用对应的Reflect方法来执行被拦截的操作：

```
let proxy = new Proxy(target, {
  get: function(target, property) {
    return Reflect.get(target, property);
  }
});
```

Reflect的方法有许多优点。首先，它们总是返回一个期望的值，使得代码更易于理解和调试。其次，它们提供了一种正确处理JavaScript基本操作的方法。例如，使用 [Reflect.set\(\)](#) 可以正确处理设置只读属性的情况。

结论

JavaScript Proxy是一个非常强大的工具，它为修改和扩展基本操作提供了可能性。虽然在某些情况下，使用Proxy可能会让代码变得更复杂，但在处理某些复杂问题时，如数据绑定和观察、操作拦截和校验等，它的优势就显现出来了。理解和掌握Proxy可以让你的JavaScript代码更具有扩展性和灵活性。

详解Cookie, Session, sessionStorage, localStorage

引言

在Web开发中，数据的存储和管理是非常重要的。Cookie、Session、sessionStorage和localStorage是常见的Web存储解决方案。本文将详细介绍这些概念，比较它们的特点和用法，并提供相关的代码示例。

1. 什么是Cookie?

属性

Cookie是一种在客户端存储数据的机制，它将数据以键值对的形式存储在用户的浏览器中。Cookie具有以下属性：

- 「名称和值」：每个Cookie都有一个名称和对应的值，以键值对的形式表示。
- 「域（Domain）」：Cookie的域属性指定了可以访问Cookie的域名。默认情况下，Cookie的域属性设置为创建Cookie的页面的域名。
- 「路径（Path）」：Cookie的路径属性指定了可以访问Cookie的路径。默认情况下，Cookie的路径属性设置为创建Cookie的页面的路径。
- 「过期时间（Expires/Max-Age）」：Cookie的过期时间属性指定了Cookie的有效期限。可以通过设置 Expires 或 Max-Age 属性来定义过期时间。过期时间可以是一个具体的日期和时间，也可以是一个从当前时间开始的时间段。
- 「安全标志（Secure）」：Cookie的安全标志属性指定了是否只在通过HTTPS协议发送请求时才发送Cookie。
- 「同站点标志（SameSite）」：Cookie的同站点标志属性指定了是否限制Cookie只能在同一站点发送。可以设置为 Strict （仅允许来自当前站点的请求携带Cookie）或 Lax （允许部分跨站点请求携带Cookie）。

应用场景

Cookie在Web开发中有多种应用场景，包括：

- 「会话管理」：Cookie常用于存储会话标识符，以便在用户访问不同页面时保持会话状态。
- 「身份验证」：Cookie可以用于存储用户的身份验证凭证或令牌，以便在用户下次访问时自动登录。
- 「个性化设置」：Cookie可以用于存储用户的个性化首选项，例如语言偏好、主题设置等。
- 「追踪和分析」：Cookie可以用于追踪用户的行为和进行网站分析，例如记录用户访问的页面、点击的链接等。

以下是一个使用JavaScript创建和读取Cookie的示例：

```
// 设置Cookie
document.cookie = "username=John Doe; expires=Fri, 31 Dec 2023 23:59:59 GMT; path=/; secure; SameSite=Strict";

// 读取Cookie
const cookies = document.cookie.split("; ");
for (let i = 0; i < cookies.length; i++) {
  const cookie = cookies[i].split("=");
  const name = cookie[0];
  const value = cookie[1];
  console.log(name + ": " + value);
}
```

2. 什么是Session?

属性

Session是一种在服务器端存储和跟踪用户会话状态的机制。Session具有以下属性：

- 「存储位置」：Session数据存储在服务器端的内存或持久化介质中，而不是存储在客户

端。

- 「会话ID」：每个会话都有一个唯一的会话ID，用于标识该会话。会话ID通常通过Cookie或URL参数发送给客户端，并在后续请求中用于识别会话。
- 「过期时间」：Session可以设置过期时间，以控制会话的有效期。过期时间可以是一个具体的日期和时间，也可以是一个从会话创建时开始的时间段。
- 「安全性」：Session的会话ID需要进行保护，以防止会话劫持和其他安全问题。

应用场景

Session在Web开发中有多种应用场景，包括：

- 「用户身份验证」：Session用于存储用户的身份验证状态，以便在用户访问需要验证的资源时进行验证。
- 「购物车」：Session用于存储用户的购物车内容，以便在用户进行结账或继续购物时保持购物车状态。
- 「个性化设置」：Session可以用于存储用户的个性化首选项，例如语言偏好、主题设置等。

以下是一个使用Express.js处理Session的示例：

```
const express = require("express");
const session = require("express-session");

const app = express();

app.use(session({
  secret: "mysecret",
  resave: false,
  saveUninitialized: true,
  cookie: { secure: true, sameSite: "strict", httpOnly: true }
}));

app.get("/", (req, res) => {
  req.session.username = "John Doe";
  res.send("Session is set.");
});

app.get("/profile", (req, res) => {
  const username = req.session.username;
  res.send("Welcome, " + username);
});

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

3. 什么是SessionStorage?

属性

SessionStorage是一种在客户端存储临时数据的机制。SessionStorage具有以下属性：

- 「存储位置」：SessionStorage数据存储在客户端的内存中，与当前会话关联。
- 「会话范围」：SessionStorage数据仅在浏览器会话期间保留，当用户关闭标签页或浏览器

时数据将被清除。

- 「域和协议限制」：SessionStorage数据只能在同一域和协议下访问。

应用场景

SessionStorage在Web开发中有多种应用场景，包括：

- 「临时数据存储」：SessionStorage可用于在页面之间传递临时数据，例如表单数据、临时状态等。
- 「表单数据保存」：SessionStorage可用于保存用户填写的表单数据，以便在刷新页面或返回页面时恢复数据，防止数据丢失。
- 「单页应用状态管理」：在单页应用中，可以使用SessionStorage来存储和管理应用的状态，例如当前选中的标签、展开/收起的面板等。

以下是一个使用JavaScript操作SessionStorage的示例：

```
// 设置SessionStorage
sessionStorage.setItem("username", "John Doe");

// 读取SessionStorage
const username = sessionStorage.getItem("username");
console.log(username);
```

4. 什么是LocalStorage?

属性

LocalStorage是一种在客户端存储持久性数据的机制。LocalStorage具有以下属性：

- 「存储位置」：LocalStorage数据存储在客户端的持久化介质中，与浏览器相关联。
- 「持久性」：LocalStorage数据不受会话结束或浏览器关闭的影响，会一直保留在浏览器中，除非被显式删除。
- 「域和协议限制」：LocalStorage数据只能在同一域和协议下访问。

应用场景

LocalStorage在Web开发中有多种应用场景，包括：

- 「本地数据存储」：LocalStorage可用于在客户端存储持久性数据，如用户首选项、缓存的数据等。
- 「离线应用」：LocalStorage可用于存储离线应用所需的资源，例如HTML、CSS和JavaScript文件，以实现离线访问能力。
- 「单页应用状态管理」：在单页应用中，可以使用LocalStorage来存储和管理应用的状态，例如当前选中的标签、展开/收起的面板等。

以下是一个使用JavaScript操作LocalStorage的示例：

```
// 设置LocalStorage
localStorage.setItem("username", "John Doe");

// 读取LocalStorage
const username = localStorage.getItem("username");
console.log(username);
```

5. Cookie vs. Session vs. SessionStorage vs. LocalStorage

	属性	存储位置	生命周期	安全性	大小限制	跨域限制
Cookie	键值对	客户端	可配置	受同源策略限制	约4KB	是
Session	会话ID和服务器端存储	服务器端	可配置	较高（会话ID保护）	无	否
SessionStorage	键值对	客户端	浏览器会话期间	同源	约5MB	否
LocalStorage	键值对	客户端	永久（需显式删除）	同源	约5MB	否

Cookie、Session、SessionStorage和LocalStorage都是常见的Web存储解决方案，每种方案都有其适用的场景和特点。

- 使用Cookie可以在客户端存储数据，适用于存储会话标识符、用户首选项和追踪用户行为等场景。
- Session用于在服务器端存储和管理用户的会话状态，适用于身份验证、购物车和个性化设置等场景。
- SessionStorage用于在浏览器会话期间存储临时数据，适用于传递数据、保存表单数据和单页应用状态管理等场景。
- LocalStorage用于在客户端存储持久性数据，适用于本地数据存储、离线应用和单页应用状态管理等场景。

根据具体的需求和场景，选择合适的存储方案可以更好地管理和使用数据。

6. 参考资料

- [MDN Web Docs - HTTP Cookies](https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies) (https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies)
- [MDN Web Docs - Web Storage API](https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API) (https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API)
- [MDN Web Docs - SameSite attribute](https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite) (https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite)
- [MDN Web Docs - HttpOnly attribute](https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies#restrict_access_to_cookies) (https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies#restrict_access_to_cookies)
- [W3Schools - JavaScript Cookies](https://www.w3schools.com/js/js_cookies.asp) (https://www.w3schools.com/js/js_cookies.asp)
- [W3Schools - HTML Web Storage Objects](https://www.w3schools.com/html/html5_webstorage.asp) (https://www.w3schools.com/html/html5_webstorage.asp)

面向对象编程与Class

引言

随着JavaScript的发展，ECMAScript 6（ES6）引入了许多新的语言特性和语法糖，其中包括了面向对象编程的Class（类）机制。Class提供了一种更简洁、更直观的方式来定义对象和操作对象的行为。本文将介绍ES6中Class的概念、语法和特性，并通过示例代码来说明其实际应用。

1. 什么是面向对象编程？

面向对象编程（Object-Oriented Programming，简称OOP）是一种编程范式，它将程序中的对象作为基本单元，通过封装、继承和多态等机制来组织和管理代码。面向对象编程将现实世界中的实体抽象为代码中的对象，对象拥有自己的状态（属性）和行为（方法），并与其他对象进行交互。

面向对象编程有以下几个核心概念：

- 「封装（Encapsulation）」：将数据和操作数据的方法封装在一个对象中，使其成为一个

独立的实体，外部无法直接访问对象的内部实现细节。

- 「**继承 (Inheritance)**」：通过定义一个基类（父类），其他类可以继承该基类的属性和方法，并可以在此基础上进行扩展或覆盖。
- 「**多态 (Polymorphism)**」：不同对象可以对相同的方法做出不同的响应，即同一个方法可以根据调用对象的不同而具有不同的行为。

面向对象编程的优势包括代码的可重用性、可维护性、扩展性和灵活性等。

2. Class的基本概念

在ES6之前，JavaScript中的对象和面向对象编程的概念相对比较模糊。ES6引入了Class机制，使得JavaScript可以更加直观地定义和使用类。Class是一种特殊的函数，通过Class关键字定义。Class中可以定义构造函数、属性和方法等。

一个简单的Class示例如下：

```
class Rectangle {  
  constructor(width, height) {  
    this.width = width;  
    this.height = height;  
  }  
  
  area() {  
    return this.width * this.height;  
  }  
  
  perimeter() {  
    return 2 * (this.width + this.height);  
  }  
}
```

在上述示例中，我们定义了一个名为 `Rectangle` 的类，

它具有 `width` 和 `height` 两个属性，以及 `area()` 和 `perimeter()` 两个方法。通过Class定义的类可以通过实例化来创建具体的对象，并调用其属性和方法。

```
const rect = new Rectangle(5, 3);  
console.log(rect.area());    // 输出: 15  
console.log(rect.perimeter()); // 输出: 16
```

3. Class的语法

ES6中Class的语法相对简洁明了。一个Class可以包含构造函数、属性和方法等。下面介绍一些常用的语法规则：

3.1 构造函数

在Class中使用 `constructor` 关键字定义构造函数。构造函数用于创建对象时进行初始化操作，通过 `new` 关键字实例化类时会自动调用构造函数。

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }
}
```

构造函数中的 `this` 关键字表示当前实例化的对象。

3.2 属性

在Class中可以定义各种属性。属性可以直接定义在Class的内部，也可以在构造函数中通过 `this` 关键字进行定义。

```
class Rectangle {
  width = 0; // 直接定义属性
  height = 0;

  constructor(width, height) {
    this.width = width; // 在构造函数中定义属性
    this.height = height;
  }
}
```

3.3 方法

在Class中定义的函数称为方法。可以直接在Class的内部定义方法，也可以使用ES6的简写形式。

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  area() { // 定义方法
    return this.width * this.height;
  }

  perimeter() {
    return 2 * (this.width + this.height);
  }
}
```

3.4 方法的访问修饰符

在Class中，可以使用访问修饰符来限制方法的访问权限。ES6中的Class默认所有方法都是公共的，可以被外部调用。但我们可以使用 `static`、`get`、`set`、`private` 和 `protected` 等修饰符来控制方法的访问。

- `static`：定义静态方法，只能通过类本身调用，不能通过类的实例调用。
- `get` 和 `set`：定义属性的读取和设置方法，使用类似访问属性的语法进行调用。

- `private`：定义私有方法，只能在类的内部被访问，外部无法访问。
- `protected`：定义受保护方法，只能在类的内部和子类中被访问，外部无法访问。

```
class Rectangle {
  static description = 'This is a rectangle'; // 静态属性

  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  static createSquare(side) { // 静态方法
    return new Rectangle(side, side);
  }

  get area() { // Getter方法
    return this.width * this.height;
  }

  set area(value) { // Setter方法
    this.width = Math.sqrt(value);
    this.height = Math.sqrt(value);
  }

  #privateMethod() { // 私有方法
    console.log('This is a private method');
  }

  protectedMethod() { // 受保护方法
    console.log('This is a protected method');
  }

  publicMethod() { // 公共方法
    console.log('This is a public method');
    this.#privateMethod();
    this.protectedMethod();
  }
}
```

在上述示例中，我们定义了一个 `Square` 类，它继承自 `Rectangle` 类。通过 `super` 关键字调用父类的构造函数，确保父类的属性被正确初始化。子类可以新增或覆盖父类的方法。

```
const square = new Square(5);
console.log(square.area()); // 输出：25
console.log(square.perimeter()); // 输出：20
```

4. 类的静态方法和属性

静态方法和属性属于类本身，而不是类的实例。静态方法和属性可以通过类名直接访问，无需实例化类。

```
class MathUtil {  
  static PI = 3.14159; // 静态属性  
  
  static square(number) { // 静态方法  
    return number * number;  
  }  
}
```

在上述示例中，我们定义了一个 `MathUtil` 类，它具有一个静态属性 `PI` 和一个静态方法 `square()`。可以通过类名直接访问静态属性和方法。

```
console.log(MathUtil.PI); // 输出: 3.14159  
console.log(MathUtil.square(5)); // 输出: 25
```

5. Getter和Setter方法

Getter和Setter方法用于对类的属性进行读取和设置操作，可以通过类似访问属性的语法进行调用。

```
class Circle {  
  constructor(radius) {  
    this.radius = radius;  
  }  
  
  get diameter() {  
    return 2 * this.radius;  
  }  
  
  set diameter(value) {  
    this.radius = value / 2;  
  }  
}
```

在上述示例中，我们定义了一个 `Circle` 类，它具有一个属性 `radius`。通过定义 `get diameter()` 方法和 `set diameter()` 方法，我们可以通过类似访问属性的方式来读取和设置直径（`diameter`）属性，而不需要直接访问 `radius` 属性。

```
const circle = new Circle(5);  
console.log(circle.diameter); //  
  
输出: 10  
circle.diameter = 12;  
console.log(circle.radius); // 输出: 6
```

6. 类的私有属性和方法

在ES6中，可以使用 `#` 作为前缀来定义私有属性和方法。私有属性和方法只能在类的内部被访问，外部无法访问。

```
class Person {  
  #name; // 私有属性  
  
  constructor(name) {  
    this.#name = name;  
  }  
  
  #privateMethod() { // 私有方法  
    console.log('This is a private method');  
  }  
  
  publicMethod() { // 公共方法  
    console.log('Hello, my name is ${this.#name}');  
    this.#privateMethod();  
  }  
}
```

在上述示例中，我们定义了一个 `Person` 类，它具有一个私有属性 `#name` 和一个私有方法 `#privateMethod()`。私有属性和方法只能在类的内部访问。

```
const person = new Person('John');  
person.publicMethod(); // 输出: Hello, my name is John  
person.#name;          // 报错: SyntaxError: Private field '#name' must be declared in an enclosing class  
person.#privateMethod(); // 报错: SyntaxError: Private field '#privateMethod' must be declared in an enclosing class
```

7. 类的实例和构造函数

在ES6中，类的实例通过 `new` 关键字进行创建，并自动调用类的构造函数进行初始化。

```
const rect = new Rectangle(5, 3);  
console.log(rect.area()); // 输出: 15  
console.log(rect.perimeter()); // 输出: 16
```

可以使用 `instanceof` 运算符来判断一个对象是否是某个类的实例。

```
console.log(rect instanceof Rectangle); // 输出: true  
console.log(rect instanceof Object); // 输出: true
```

8. 类的继承

继承是面向对象编程中的重要概念之一，它允许我们创建一个基类（父类），其他类可以继承该基类并扩展或覆盖其中的属性和方法。ES6中使用 `extends` 关键字实现类的继承。

```
class Square extends Rectangle {
  constructor(side) {
    super(side, side); // 调用父类的构造函数
  }
}
```

9. 类的封装

封装通过将数据和操作数据的方法封装在一个对象中，实现了数据的保护和访问的控制。类的属性和方法可以使用不同的访问修饰符来控制其可见性。

```
class Rectangle {
  #width; // 私有属性
  #height;

  constructor(width, height) {
    this.#width = width;
    this.#height = height;
  }

  getArea() { // 公共方法
    return this.#width * this.#height;
  }
}

const rect = new Rectangle(5, 3);
console.log(rect.#width); // 报错: SyntaxError: Private field '#width' must be declared in an enclosing class
console.log(rect.getArea()); // 输出: 15
```

在上述示例中，Rectangle类具有私有属性#width和#height，只能在类的内部被访问。通过定义公共方法getArea()来访问私有属性，从而实现了封装。

10. 类的多态

多态允许不同的对象对相同的消息作出不同的响应。通过继承和方法的覆盖，不同的子类可以对父类的方法进行不同的实现，从而实现多态性。

```
class Animal {
  makeSound() {
    console.log('Animal makes sound');
  }
}

class Dog extends Animal {
  makeSound() {
    console.log('Dog barks');
  }
}

class Cat extends Animal {
  makeSound() {
    console.log('Cat meows');
  }
}

const animal = new Animal();
const dog = new Dog();
const cat = new Cat();

animal.makeSound(); // 输出: Animal makes sound
dog.makeSound();    // 输出: Dog barks
cat.makeSound();    // 输出: Cat meows
```

在上述示例中，Animal类是基类，Dog和Cat类是子类。它们都具有makeSound()方法，但不同的子类对该方法进行了不同的实现，实现了多态性。

通过封装、继承和多态，面向对象编程提供了一种更加灵活和可扩展的编程方式，使得代码的组织和管理更加直观和高效。

11. 结语

ES6引入的Class机制为JavaScript提供了一种更直观、更简洁的面向对象编程方式。通过Class，我们可以更方便地定义和使用类，实现封装、继承和多态等面向对象编程的基本原理。同时，ES6还提供了许多其他的语法糖和特性，使得JavaScript在面向对象编程方面更加强大和灵活。

12. 参考资料

- [MDN Web Docs - Classes \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes)
- [ECMAScript 6 入门 - Class \(http://es6.ruanyifeng.com/#docs/class\)](http://es6.ruanyifeng.com/#docs/class)
- [Understanding ECMAScript 6 - Classes \(https://leanpub.com/understandingses6/read/#leanpub-auto-classes\)](https://leanpub.com/understandingses6/read/#leanpub-auto-classes)
- [Exploring ES6 - Classes \(https://exploringjs.com/es6/ch_classes.html\)](https://exploringjs.com/es6/ch_classes.html)

面试官：说说 linux 系统下常用的命令有哪些？

前言

“

总结一下我工作常用的linux命令，当做备忘录了，比如 `pwd` , `mv` , `history` ，用好这些命令，工作就轻松多了

”

☆ 1. ls - 列出目录内容

“

列出当前目录中的文件和子目录。

”

```
ls
```

☆ 2. cd - 切换目录

“

切换到指定目录。

”

```
cd directory_path
```

☆ 3. pwd - 显示当前工作目录

“

显示当前所在的工作目录的路径。

”

```
pwd
```

☆ 4. mkdir - 创建新目录

“

创建一个新的目录。

”

```
mkdir directory_name
```

☆ 5. rm - 删除文件或目录

“

删除指定的文件或目录。

”

```
rm file_name
```

☆ 6. cp - 复制文件或目录

“

将文件或目录复制到指定位置。

”

```
cp source_file destination_file
```

☆ 7. mv - 移动文件或目录

“

将文件或目录移动到指定位置。

”

```
mv source_file destination_file
```

☆ 8. cat - 查看文件内容

“

显示文件的全部内容。

”

```
cat file_name
```

☆ 9. less - 分页查看文件内容

“

分页显示文件的内容。

”

```
less file_name
```

☆ 10. head - 查看文件开头内容

“

显示文件的开头部分内容。

”

```
head file_name
```

☆ 11. tail - 查看文件结尾内容

“

显示文件的结尾部分内容。

”

```
tail file_name
```

☆ 12. grep - 在文件中搜索文本

“

在文件中搜索指定的文本。

”

```
grep "search_text" file_name
```

☆ 13. find - 在目录中搜索文件

“

在指定目录及其子目录中搜索文件。

”

```
find directory_path -name "file_name"
```

☆ 14. chmod - 修改文件权限

“

修改文件的权限。

”

```
chmod permissions file_name
```

☆ 15. chown - 修改文件所有者

“

修改文件的所有者。

”

```
chown new_owner file_name
```

☆ 16. chgrp - 修改文件所属组

“

修改文件的所属组。

”

```
chgrp new_group file_name
```

☆ 17. tar - 创建或提取归档文件

“

创建或提取 tar 归档文件。

”

```
tar options file_name
```

☆ 18. gzip - 压缩文件

“

使用 gzip 压缩文件。

”

```
gzip file_name
```

☆ 19. gunzip - 解压缩文件

“

解压缩使用 gzip 压缩的文件。

”

```
gunzip file_name
```

☆ 20. tar -xvf - 解压缩 tar 归档文件

“

解压缩 tar 归档文件。

”

```
tar -xvf file_name.tar
```

☆ 21. wget - 下载文件

“

从指定 URL 下载文件。

”

```
wget file_url
```

☆ 22. curl - 发送 HTTP 请求

“

使用各种协议发送 HTTP 请求。

”

```
curl options url
```

☆ 23. ssh - 远程登录

“

通过 SSH 协议远程登录到另一台计算机。

”

```
ssh username@hostname
```

☆ 24. scp - 复制文件到远程计算机

“

将文件复制到远程计算机。

”

```
scp source_file username@hostname:destination_path
```

☆ 25. grep -r - 递归搜索文件内容

“

递归搜索指定目录中的文件内容。

”

```
grep -r "search_text" directory_path
```

☆ 26. ps - 查看进程状态

“

显示系统中运行的进程状态。

”

```
ps
```

☆ 27. top - 监视系统资源

“

实时监视系统资源使用情况。

”

```
top
```

☆ 28. kill - 终止进程

“

终止指定进程。

”

```
kill process_id
```

☆ 29. ifconfig - 查看网络接口配置

“

显示网络接口的配置信息。

”

```
ifconfig
```

☆ 30. ping - 测试网络连接

“

向指定主机发送 ICMP 回显请求以测试网络连接。

”

```
ping hostname
```

☆ 31. netstat - 查看网络状态

“

显示网络连接、路由表和网络接口信息。

”

```
netstat
```

☆ 32. iptables - 配置防火墙规则

“

配置 Linux 防火墙规则。

”

```
iptables options
```

☆ 33. history - 查看命令历史记录

“

显示之前执行的命令历史记录。

”

```
history
```

☆ 34. useradd - 创建新用户

“

创建新的系统用户。

”

```
useradd username
```

☆ 35. usermod - 修改用户属性

“

修改现有用户的属性。

”

```
usermod options username
```

☆ 36. userdel - 删除用户

“

删除系统中的用户。

”

```
userdel username
```

☆ 37. su - 切换用户

“

切换到其他用户账户。

”

```
su username
```

☆ 38. sudo - 以超级用户权限执行命令

“

以超级用户权限执行指定命令。

”

```
sudo command
```

☆ 39. du - 查看文件和目录大小

“

显示文件和目录的磁盘使用情况。

”

```
du options file_or_directory
```

☆ 40. df - 查看磁盘空间使用情况

“

显示文件系统的磁盘空间使用情况。

”

```
df
```

☆ 41. passwd - 修改用户密码

“

用于修改指定用户的密码。

”

```
passwd username
```

☆ 42. history -c - 清除命令历史记录

“

清除之前执行的命令历史记录。

”

```
history -c
```

☆ 43. alias - 创建命令别名

“

创建自定义的命令别名。

”

```
alias alias_name='command'
```

☆ 44. unalias - 删除命令别名

“

删除已创建的命令别名。

”

```
unalias alias_name
```

☆ 45. chmod -R - 递归修改文件权限

“

递归修改文件及子目录的权限。

”

```
chmod -R permissions directory_path
```

☆ 46. chown -R - 递归修改文件所有者

“

递归修改文件及子目录的所有者。

”

```
chown -R new_owner directory_path
```

☆ 47. chgrp -R - 递归修改文件所属组

“

递归修改文件及子目录的所属组。

”

```
chgrp -R new_group directory_path
```

☆ 48. tar -czvf - 压缩文件和目录

“

压缩文件和目录为 tar 归档文件。

”

```
tar -czvf archive_name.tar.gz file_or_directory
```

☆ 49. tar -xvzf - 解压缩 tar.gz 归档文件

“

解压缩 tar.gz 归档文件。

”

```
tar -xvzf archive_name.tar.gz
```

☆ 50. grep -i - 忽略大小写搜索文本

“

在文件中忽略大小写搜索指定的文本。

”

```
grep -i "search_text" file_name
```

☆ 51. find -type - 按类型搜索文件

“

按指定的类型在目录中搜索文件。

”

```
find directory_path -type f
```

☆ 52. ps aux - 查看详细进程信息

“

显示详细的进程信息，包括所有用户的进程。

”

```
ps aux
```

☆ 53. top -n 1 -b - 以非交互方式显示系统资源

“

以非交互方式一次性显示系统资源的使用情况。

”

```
top -n 1 -b
```

☆ 54. killall - 终止特定名称的进程

“

终止所有具有指定名称的进程。

”

```
killall process_name
```

☆ 55. ifconfig interface up/down - 启用/禁用网络接口

“

启用或禁用指定的网络接口。

”

```
ifconfig interface up  
ifconfig interface down
```

☆ 56. ping -c - 指定 ICMP 回显请求次数

“

指定发送 ICMP 回显请求的次数。

”

```
ping -c count hostname
```

☆ 57. netstat -tunlp - 查看网络端口和进程信息

“

显示网络端口和与之关联的进程信息。

”

```
netstat -tunlp
```

☆ 58. iptables-save - 保存防火墙规则

“

将当前防火墙规则保存到文件中。

”

```
iptables-save > rules_file
```

☆ 59. iptables-restore - 恢复防火墙规则

“

从文件中恢复防火墙规则。

”

```
iptables-restore < rules_file
```

☆ 60. adduser - 创建新用户（交互式）

“

以交互方式创建新的系统用户。

”

```
adduser username
```

☆ 61. deluser - 删除用户（同时删除用户目录）

“

删除系统中的用户，并删除其关联的用户目录。

”

```
deluser --remove-home username
```

☆ 62. groups - 查看用户所属组

“

显示指定用户所属的组列表。

”

```
groups username
```

☆ 63. whoami - 查看当前用户名

“

显示当前登录用户的用户名。

”

```
whoami
```

☆ 64. uname - 显示系统信息

“

显示当前系统的信息。

”

```
uname -a
```

☆ 65. date - 显示系统日期和时间

“

显示当前日期和时间。

”

```
date
```

☆ 66. cal - 显示日历

“

显示当前月份的日历。

”

```
cal
```

☆ 67. uptime - 查看系统运行时间

“

显示系统的运行时间和平均负载。

”

```
uptime
```

☆ 68. free - 查看内存使用情况

“

显示系统的内存使用情况。

”

```
free -h
```

☆ 69. df -h - 查看磁盘空间使用情况（人类可读格式）

“

显示文件系统的磁盘空间使用情况（以人类可读的格式）。

”

```
df -h
```

☆ 70. du -sh - 查看目录大小（总计）

“

显示目录的总大小。

”

```
du -sh directory_path
```

☆ 71. ifconfig -a - 查看所有网络接口配置

“

显示所有网络接口的配置信息。

”

```
ifconfig -a
```

☆ 72. route -n - 查看网络路由表

“

显示系统的网络路由表。

”

```
route -n
```

☆ 73. history - 查看最近执行的命令历史记录

“

显示最近执行的命令历史记录。

”

```
history
```

☆ 74. alias - 查看当前命令别名

“

显示当前定义的命令别名。

”

```
alias
```

☆ 75. echo - 输出文本

“

输出指定的文本内容。

”

```
echo "text"
```

☆ 76. touch - 创建空文件或修改时间戳

“

创建一个新的空文件或修改文件的访问和修改时间戳。

”

```
touch file_name
```

☆ 77. file - 查看文件类型

“

确定文件的类型。

”

```
file file_name
```

☆ 78. find -mtime - 按修改时间搜索文件

“

在指定的天数范围内按修改时间搜索文件。

”

```
find directory_path -mtime n
```

☆ 79. locate - 快速定位文件

“

根据数据库快速定位文件。

”

```
locate file_name
```

☆ 80. which - 查找命令的路径

“

查找指定命令的完整路径。

”

```
which command_name
```

☆ 81. wc - 统计文件行数、字数和字节数

“

统计文件的行数、字数和字节数。

”

```
wc file_name
```

☆ 82. sed - 文本流编辑器

“

对文本进行流式编辑和替换操作。

”

```
sed options file_name
```

☆ 83. awk - 文本处理工具

“

对文本进行处理和分析。

”

```
awk 'pattern { action }' file_name
```

☆ 84. cut - 提取文件的列

“

从文件中提取指定的列。

”

```
cut -f columns file_name
```

☆ 85. paste - 合并文件的列

“

将多个文件的列合并到一个文件中。

”

```
paste file1 file2
```

☆ 86. sort - 对文件进行排序

“

对文件的行进行排序。

”

```
sort file_name
```

☆ 87. uniq - 去重文件的行

“

去除文件中的重复行。

”

```
uniq file_name
```

☆ 88. diff - 比较文件的差异

“

比较两个文件的差异。

”

```
diff file1 file2
```

☆ 89. patch - 应用补丁文件

“

根据补丁文件更新源文件。

”

```
patch file < patch_file
```

☆ 90. wget -c - 断点续传下载文件

“

断点续传方式下载文件。

”

```
wget -c file_url
```

☆ 91. curl -O - 下载文件并保留原始文件名

“

下载文件并保留原始文件名。

”

```
curl -O file_url
```

☆ 92. tar -tvf - 查看 tar 归档文件内容

“

查看 tar 归档文件的内容列表。

”

```
tar -tvf archive_name.tar
```

☆ 93. tar -rvf - 向 tar 归档文件添加文件

“

向 tar 归档文件中添加文件。

”

```
tar -rvf archive_name.tar file1 file2
```

☆ 94. tar -cvf - 创建 tar 归档文件

“

创建新的 tar 归档文件。

”

```
tar -cvf archive_name.tar file1 file2
```

☆ 95. tar -xf - 提取 tar 归档文件

“

从 tar 归档文件中提取文件。

”

```
tar -xf archive_name.tar
```

☆ 96. gzip -d - 解压缩 .gz 文件

“

解压缩 .gz 压缩文件。

”

```
gzip -d file_name.gz
```

☆ 97. bzip2 - 压缩文件

“

压缩文件为 .bz2 格式。

”

```
bzip2 file_name
```

☆ 98. bunzip2 - 解压缩 .bz2 文件

“

解压缩 .bz2 压缩文件。

”

```
bunzip2 file_name.bz2
```

☆ 99. zip -r - 压缩文件和目录为 .zip 文件

“

压缩文件和目录为 .zip 格式。

”

```
zip -r archive_name.zip file_or_directory
```

☆ 100. unzip - 解压缩 .zip 文件

“

解压缩 .zip 压缩文件。

”

```
unzip archive_name.zip
```

TS实战之扑克牌排序

[在线运行 \(https://code.juejin.cn/pen/7254739493366333499\)](https://code.juejin.cn/pen/7254739493366333499)

我们用 [ts实现扑克牌排序问题](#)，首先，我们将定义所需的数据类型，然后专注于模式查找算法，该算法有几个有趣的要点。

类型和转换

定义一些我们需要的类型。[Rank](#) 和 [Suit](#) 是明显的[联合类型](#)

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#union-types>。

```
type Rank =  
  | 'A' | '2' | '3' | '4' | '5' | '6' | '7'  
  | '8' | '9' | '10' | 'J' | 'Q' | 'K'  
  
type Suit = '♥' | '♦' | '♠' | '♣';
```

我们将使用 [Card](#) 对象进行处理，将rank和suit转换为数字。卡片将用从1（Ace）到13（King）的值表示，花色从1（红心）到4（梅花）。[rankToNumber\(\)](#) 和 [suitToNumber\(\)](#) 函数处理从 [Rank](#) 和 [Suit](#) 值到数字的转换。

```
type Card = { rank: number; suit: number };  
  
const rankToNumber = (rank: Rank): number =>  
  rank === 'A' ? 1  
  : rank === 'J' ? 11  
  : rank === 'Q' ? 12  
  : rank === 'K' ? 13  
  : Number(rank);  
  
const suitToNumber = (suit: Suit): number =>  
  suit === '♥' ? 1  
  : suit === '♦' ? 2  
  : suit === '♠' ? 3  
  : /* suit === "♣" */ 4;
```



这些类型用于内部工作；我们还必须定义手牌检测算法的结果类型。我们需要一个[枚举](#) <https://www.typescriptlang.org/docs/handbook/enums.html> 类型来表示手牌的可能值。这些值按照从最低（"高牌"）到最高（"皇家同花顺"）的顺序排列。

```
enum Hand {
  HighCard, // 高牌
  OnePair, // 一对
  TwoPairs, // 两对
  ThreeOfAKind, // 三条
  Straight, // 顺子
  Flush, // 同花
  FullHouse, // 葫芦
  FourOfAKind, // 四条
  StraightFlush, // 同花顺
  RoyalFlush // 皇家同花顺
}
```

我们有什么手牌？

让我们首先定义我们将要构建的 `handRank()` 函数。我们的函数将接收一个包含 `五张牌的元组`，并返回一个 `Hand` 结果。

```
export function handRank(
  cardStrings: [string, string, string, string, string]
): Hand {
  .
  .
  .
}
```

由于处理字符串比我们需要的要困难，我们将把牌字符串转换为具有数字 `rank` 和 `suit` 值的 `Card` 对象，以便更容易编写。

```
const cards: Card[] = cardStrings.map((str: string) => ({
  rank: rankToNumber(
    str.substring(0, str.length - 1) as Rank
  ),
  suit: suitToNumber(str.at(-1) as Suit)
}));
.
.
.
// 继续...
```



确定玩家手牌的价值的关键在于知道每个等级的牌有多少张，以及我们有多少计数。例如，如果我们有三张J和两张K，J的计数为3，K的计数为2。然后，知道我们有一个计数为三和一个计数为两的计数，我们可以确定我们有一个葫芦。另一个例子：如果我们有三个Q，两个A和一个5，我们会得到三个计数为两和一个计数为一；我们有三对。

生成计数很简单。我们希望A的计数在 `countByRank[1]` 处，因此我们不会使用 `countByRank` 数组的初始位置。类似地，花色的计数将位于 `countBySuit[1]` 到 `countBySuit[4]` 之间，因此我们也不会使用该数组的初始位置。

```
// ...继续
.
.
.
const countBySuit = new Array(5).fill(0);
const countByRank = new Array(15).fill(0);
const countBySet = new Array(5).fill(0);

cards.forEach((card: Card) => {
  countByRank[card.rank]++;
  countBySuit[card.suit]++;
});
countByRank.forEach(
  (count: number) => count && countBySet[count]++
);
.
.
.
// 继续...
```

我们不要忘记A可能位于顺子的开头（A-2-3-4-5）或结尾（10-J-Q-K-A）。我们可以通过在K之后复制Aces计数来处理这个问题。

```
// ...继续
.
.
.
countByRank[14] = countByRank[1];
.
.
.
// 继续...
```

现在我们可以开始识别手牌了。我们只需要查看按等级计数即可识别几种手牌：

```

// ...继续
.
.
.
if (count
BySet[4] === 1 && countBySet[1] === 1)
  return Hand.FourOfAKind;
else if (countBySet[3] && countBySet[2] === 1)
  return Hand.FullHouse;
else if (countBySet[3] && countBySet[1] === 2)
  return Hand.ThreeOfAKind;
else if (countBySet[2] === 2 && countBySet[1] === 1)
  return Hand.TwoPairs;
else if (countBySet[2] === 1 && countBySet[1] === 3)
  return Hand.OnePair;
.
.
.
// 继续...

```

例如，如果有四张相同等级的牌，我们知道玩家将获得“四条”。可能会问：如果 `countBySet[4] === 1`，为什么还要测试 `countBySet[1] === 1`？如果四张牌的等级相同，应该只有一张其他牌，对吗？答案是“防御性编程” (https://en.wikipedia.org/wiki/Defensive_programming)——在开发代码时，有时会出现错误，通过在测试中更加具体，有助于排查错误。

上面的情况包括了所有某个等级出现多次的可能性。我们必须处理其他情况，包括顺子、同花和“高牌”。

```

// ...继续
.
.
.
else if (countBySet[1] === 5) {
  if (countByRank.join("").includes('11111'))
    return !countBySuit.includes(5)
      ? Hand.Straight
      : countByRank.slice(10).join("") === '11111'
      ? Hand.RoyalFlush
      : Hand.StraightFlush;
  else {
    return countBySuit.includes(5)
      ? Hand.Flush
      : Hand.HighCard;
  }
} else {
  throw new Error(
    'Unknown hand! This cannot happen! Bad logic!'
  );
}

```

这里我们再次进行防御性编程；即使我们知道我们有五个不同的等级，我们也确保逻辑工作良好，甚至在出现问题时抛出一个 `throw`。

我们如何测试顺子？我们应该有五个连续的等级。如果我们查看 `countByRank` 数组，它应该有五个连续的1，所以通过执行 `countByRank.join()` 并检查生成的字符串是否包含 `11111`，我们可以确定是顺子。



我们必须区分几种情况：

- 如果没有五张相同花色的牌，那么它是一个普通的顺子
- 如果所有牌都是相同花色，如果顺子以一张A结束，则为皇家同花顺
- 如果所有牌都是相同花色，但我们不以A结束，那么我们有一个同花顺

如果我们没有顺子，只有两种可能性：

- 如果所有牌都是相同花色，我们有一个同花
- 如果不是所有牌都是相同花色，我们有一个“高牌”

完整的函数如下所示：

```
export function handRank(
  cardStrings: [string, string, string, string, string]
): Hand {
  const cards: Card[] = cardStrings.map((str: string) => ({
    rank: rankToNumber(
      str.substring(0, str.length - 1) as Rank
    ),
    suit: suitToNumber(str.at(-1) as Suit)
  }));

  // We won't use the [0] place in the following arrays
  const countBySuit = new Array(5).fill(0);
  const countByRank = new Array(15).fill(0);
  const countBySet = new Array(5).fill(0);

  cards.forEach((card: Card) => {
    countByRank[card.rank]++;
    countBySuit[card.suit]++;
  });
  countByRank.forEach(
    (count: number) => count && countBySet[count]++
  );

  // count the A also as a 14, for straights
  countByRank[14] = countByRank[1];

  if (countBySet[4] === 1 && countBySet[1] === 1)
    return Hand.FourOfAKind;
  else if (countBySet[3] && countBySet[2] === 1)
    return Hand.FullHouse;
  else if (countBySet[3] && countBySet[1] === 2)
    return Hand.ThreeOfAKind;
  else if (countBySet[2] === 2 && countBySet[1] === 1)
    return Hand.TwoPairs;
  else if (countBySet[2] === 1 && countBySet[1] === 3)
    return Hand.OnePair;
```



```

else if (countBySet[1] === 5) {
  if (countByRank.join("").includes('11111'))
    return !countBySuit.includes(5)
      ? Hand.Straight
      : countByRank.slice(10).join("") === '11111'
        ? Hand.RoyalFlush
        : Hand.StraightFlush;
  else {
    /* !countByRank.join("").includes("11111") */
    return countBySuit.includes(5)
      ? Hand.Flush
      : Hand.HighCard;
  }
} else {
  throw new Error(
    'Unknown hand! This cannot happen! Bad logic!'
  );
}
}

```

测试代码

```

console.log(handRank(['3♥', '5♦', '8♣', 'A♥', '6♠'])); // 0
console.log(handRank(['3♥', '5♦', '8♣', 'A♥', '5♠'])); // 1
console.log(handRank(['3♥', '5♦', '3♣', 'A♥', '5♠'])); // 2
console.log(handRank(['3♥', '5♦', '8♣', '5♥', '5♠'])); // 3
console.log(handRank(['3♥', '2♦', 'A♠', '5♥', '4♠'])); // 4
console.log(handRank(['J♥', '10♦', 'A♠', 'Q♥', 'K♠'])); // 4
console.log(handRank(['3♥', '4♦', '7♣', '5♥', '6♦'])); // 4
console.log(handRank(['3♥', '4♥', '9♥', '5♥', '6♥'])); // 5
console.log(handRank(['3♥', '5♦', '3♣', '5♥', '3♠'])); // 6
console.log(handRank(['3♥', '3♦', '3♣', '5♥', '3♠'])); // 7
console.log(handRank(['3♥', '4♦', '7♥', '5♥', '6♥'])); // 8
console.log(handRank(['K♥', 'Q♥', 'A♥', '10♥', 'J♥'])); // 9

```

在线运行 (<https://code.juejin.cn/pen/7254739493366333499>)

函数

TypeScript提供了丰富的函数类型定义方式，可以对函数参数、返回值进行类型注解，从而提供了更为强大的类型检查。

函数声明

在TypeScript中，你可以在函数声明中对函数的参数和返回值进行类型注解。以下是一个例子：

```

function add(x: number, y: number): number {
  return x + y;
}

```

在这个例子中，我们定义了一个 `add` 函数，它接受两个参数 `x` 和 `y`，这两个参数都是 `number` 类型，函数的返回值也是 `number` 类型。

如果你尝试调用这个函数并传入一个非数字类型的参数，TypeScript编译器会报错：

```
add("Hello", 1); // Error: Argument of type '"Hello"' is not assignable to parameter of type 'number'.
```

函数表达式

在JavaScript中，函数不仅可以通过函数声明的方式定义，还可以通过函数表达式定义。在TypeScript中，函数表达式也可以使用类型注解：

```
let myAdd: (x: number, y: number) => number = function(x: number, y: number): number {  
    return x + y;  
};
```

在上面的例子中，我们首先定义了 `myAdd` 变量的类型为一个函数类型 `(x: number, y: number) => number`，然后将一个匿名函数赋值给 `myAdd`。这个匿名函数的参数 `x` 和 `y` 的类型是 `number`，返回值的类型也是 `number`。

可选参数和默认参数

TypeScript支持可选参数和默认参数。你可以使用 `?` 来标记可选参数，或者使用 `=` 来指定参数的默认值：

```
function buildName(firstName: string, lastName?: string) {  
    if (lastName)  
        return firstName + " " + lastName;  
    else  
        return firstName;  
}  
  
let result1 = buildName("Bob"); // works correctly now  
let result2 = buildName("Bob", "Adams"); // ah, just right
```

在上面的例子中，`lastName` 是一个可选参数。你可以不传这个参数调用 `buildName` 函数。

```
function buildName(firstName: string, lastName = "Smith") {  
    return firstName + " " + lastName;  
}  
  
let result1 = buildName("Bob"); // returns "Bob Smith"  
let result2 = buildName("Bob", "Adams"); // returns "Bob Adams"
```

在上面的例子中，`lastName` 有一个默认值 `"Smith"`。如果你不传这个参数调用 `buildName` 函数，`lastName` 的值将是 `"Smith"`。

剩余参数（Rest Parameters）

当你不知道要操作的函数会有多少个参数时，TypeScript提供了剩余参数的概念。与JavaScript一样，你可以使用三个

点 `...` 来定义剩余参数：

```
function buildName(firstName: string, ...restOfName: string[]) {  
    return firstName + " " + restOfName.join(" ");  
}  
  
let employeeName = buildName("Joseph", "Samuel", "Lucas", "MacKinzie");
```

在上面的例子中，`restOfName` 就是剩余参数，它可以接受任意数量的参数。

this和箭头函数

箭头函数可以保留函数创建时的 `this` 值，而不是调用时的值。在TypeScript中，你可以使用箭头函数来确保 `this` 的值：

```
let deck = {  
    suits: ["hearts", "spades", "clubs", "diamonds"],  
    cards: Array(52),  
    createCardPicker: function() {  
        return () => {  
            let pickedCard = Math.floor(Math.random() * 52);  
            let pickedSuit = Math.floor(pickedCard / 13);  
  
            return {suit: this.suits[pickedSuit], card: pickedCard % 13};  
        }  
    }  
}  
  
let cardPicker = deck.createCardPicker();  
let pickedCard = cardPicker();  
  
alert("card: " + pickedCard.card + " of " + pickedCard.suit);
```

在上面的例子中，`createCardPicker` 函数返回一个箭头函数，这个箭头函数可以访问创建时的 `this` 值。

重载

在JavaScript中，根据传入不同的参数调用同一个函数，返回不同类型的值是常见的情况。TypeScript通过为同一个函数提供多个函数类型定义来实现这个功能：

```
function reverse(x: number): number;
function reverse(x: string): string;
function reverse(x: number | string): number | string {
  if (typeof x === 'number') {
    return Number(x.toString().split('').reverse().join(''));
  } else if (typeof x === 'string') {
    return x.split('').reverse().join('');
  }
}

reverse(12345); // returns 54321
reverse('hello'); // returns 'olleh'
```

在上面的例子中，我们定义了两个重载：一个是接受 `number` 类型的参数，另一个是接受 `string` 类型的参数。然后我们在实现函数中处理了这两种情况。

命名空间和模块

命名空间（Namespace）

在 TypeScript 中，命名空间是一种将代码封装在一个特定名称下的方式，以防止全局作用域污染并避免命名冲突。命名空间在 TypeScript 中非常重要，因为它们为模块化和封装提供了灵活的选项。

创建命名空间的语法如下：

```
namespace MyNamespace {
  export const myVar: number = 10;
  export function myFunction(): void {
    console.log("Hello from MyNamespace");
  }
}
```

在此例中，我们创建了一个名为 `MyNamespace` 的命名空间，该命名空间内有一个变量 `myVar` 和一个函数 `myFunction`。`export` 关键字允许我们从命名空间外部访问这些元素。

命名空间中的元素可以通过以下方式访问：

```
console.log(MyNamespace.myVar); // 输出: 10
MyNamespace.myFunction(); // 输出: Hello from MyNamespace
```

我们也可以使用嵌套的命名空间：

```
namespace ParentNamespace {
  export namespace ChildNamespace {
    export const myVar: number = 20;
  }
}

console.log(ParentNamespace.ChildNamespace.myVar); // 输出: 20
```

命名空间（Namespace）使用场景

“

在 TypeScript 的早期版本中，命名空间被广泛地使用来组织和包装一组相关的代码。然而，随着 ES6 模块系统（ES6 Modules）的出现和广泛使用，命名空间的用法变得越来越少，现在被视为一种遗留的方式来组织代码。

”

第三方库

一些第三方库仍然使用命名空间来组织自己的代码，并提供命名空间作为库的入口点。在这种情况下，我们需要使用命名空间来访问和使用库中的类型和函数。

```
namespace MyLibrary {  
  export function myFunction() {  
    // ...  
  }  
}  
  
MyLibrary.myFunction();
```

兼容性

在一些遗留的 JavaScript 代码或库中，命名空间仍然是一种常见的组织代码的方式。如果我们需要与这些代码进行交互，我们可能需要创建命名空间来适应它们。

```
// legacy.js  
var MyNamespace = {  
  myFunction: function() {  
    // ...  
  }  
};  
  
MyNamespace.myFunction();
```

在上面的示例中，我们演示了命名空间的几个使用场景。第一个示例展示了如何使用命名空间访问和使用第三方库的函数。第二个示例展示了如何使用命名空间来管理全局状态。第三个示例展示了如何在与遗留 JavaScript 代码进行交互时创建命名空间。

虽然在现代 TypeScript 开发中，模块是更常见和推荐的代码组织方式，但命名空间仍然在特定的情况下具有一定的用处，并且在与一些特定的库或代码进行交互时可能是必需的。

模块

在 TypeScript 中，模块是另一种组织代码的方式，但它们更关注的是依赖管理。每个模块都有其自己的作用域，并且只有明确地导出的部分才能在其他模块中访问。

创建和使用模块的方式如下：

在 `myModule.ts` 文件中：


```
export const myVar: number = 10;
export function myFunction(): void {
  console.log("Hello from myModule");
}
```

在另一个文件中导入和使用模块：

```
import { myVar, myFunction } from './myModule';

console.log(myVar); // 输出: 10
myFunction(); // 输出: Hello from myModule
```

在 TypeScript 中，我们可以使用模块解析策略（如 Node 或 Classic），以确定如何查找模块。这些策略在 `tsconfig.json` 文件的 `compilerOptions` 选项下的 `moduleResolution` 选项中定义。

3. 命名空间与模块的对比

虽然命名空间和模块在某种程度上有所相似，但它们有以下几个关键区别：

1. **「作用域」**：命名空间是在全局作用域中定义的，而模块则在自己的作用域中定义。这意味着，在模块内部定义的所有内容默认情况下在模块外部是不可见的，除非显式地导出它们。
2. **「文件组织」**：命名空间通常用于组织在同一文件中的代码，而模块则是跨文件进行组织。
3. **「依赖管理」**：模块关注的是如何导入和导出功能，以便管理代码之间的依赖关系。相比之下，命名空间并未对依赖管理提供明确的支持。
4. **「使用场景」**：随着 ES6 模块语法的普及，现代 JavaScript 项目通常更倾向于使用模块来组织代码。然而，对于一些遗留项目或那些需要将多个文件合并为一个全局可用的库的场景，命名空间可能更为合适。

扩展类型定义

在 TypeScript 中，我们可以通过声明文件（`.d.ts` 文件）来为现有的 JavaScript 库提供类型定义，或者为现有的类型添加额外的属性和方法。这个过程通常被称为“类型声明扩展”。在这篇文章中，我们将详细探讨如何通过声明文件扩展类型定义。

什么是声明文件？

在 TypeScript 中，声明文件是一种以 `.d.ts` 为扩展名的特殊文件，它不包含具体的实现，只包含类型声明。这些文件通常用来为已有的 JavaScript 库提供类型定义，使得我们可以在 TypeScript 代码中更安全、更方便地使用这些库。

声明文件的主要内容是类型声明，包括变量、函数、类、接口等的类型定义。这些类型声明提供了一种描述 JavaScript 代码的结构和行为的方式，使得 TypeScript 编译器能够理解和检查 JavaScript 代码。

例如，以下是一个简单的声明文件的例子：

```
// types.d.ts
declare var foo: string;
declare function bar(baz: number): boolean;
```

在上面的声明文件中，我们声明了一个全局变量 `foo` 和一个全局函数 `bar`，并分别给它们提供了类型声明。

declare

当我们在 TypeScript 中编写声明文件时，我们使用 `declare` 关键字来声明全局变量、函数、类、接口等类型。

`declare` 关键字用于告诉 TypeScript 编译器某个标识符的类型，而不需要实际的实现代码。它用于在声明文件中描述 JavaScript 代码的类型。

下面是一些常见的用法：

1. 声明全局变量：

```
declare const myGlobal: string;
```

这个声明告诉 TypeScript 编译器，存在一个名为 `myGlobal` 的全局变量，它的类型是 `string`。

2. 声明全局函数：

```
declare function myFunction(arg: number): string;
```

这个声明告诉 TypeScript 编译器，存在一个名为 `myFunction` 的全局函数，它接受一个 `number` 类型的参数，并返回一个 `string` 类型的值。

3. 声明全局类：

```
declare class MyClass {  
  constructor(name: string);  
  getName(): string;  
}
```

这个声明告诉 TypeScript 编译器，存在一个名为 `MyClass` 的全局类，它有一个接受 `string` 类型参数的构造函数，并且有一个返回 `string` 类型的 `getName` 方法。

4. 声明命名空间

```
declare namespace MyNamespace {  
  export const myVariable: number;  
  export function myFunction(): void;  
}
```

这个声明告诉 TypeScript 编译器，存在一个名为 `MyNamespace` 的全局模块/命名空间，它包含一个名为 `myVariable` 的变量和一个名为 `myFunction` 的函数。

通过使用 `declare` 关键字，我们可以在声明文件中描述出我们所需要的类型信息，以便 TypeScript 编译器进行类型检查和类型推断。

需要注意的是，`declare` 关键字只用于类型声明，不包含具体的实现代码。在使用声明文件时，我们需要确保提供了实际的实现代码，以便程序在运行时可以访问到所声明的类型。

5. 声明模块

当我们在声明文件中使用 `declare module` 时，我们可以定义一个模块，并在其中声明模块内部的类型。这样，其他文件在导入该模块时，就可以按照模块的名称来引用其中的类型。

```
declare module 'my-module' {  
  export const myVariable: string;  
  export function myFunction(): void;  
}
```

在这个示例中，我们在 `my-module` 模块中声明了一个名为 `myVariable` 的变量和一个名为 `myFunction` 的函数，并通过 `export` 关键字将它们导出，使其在导入该模块时可见。

通过声明文件扩展类型定义

在某些情况下，我们可能需要为已有的类型添加额外的属性或方法。比如，我们可能在使用一个库时发现它缺少一些我们需要的类型定义，或者我们可能想要为一些内置类型（如 `string` 或 `Array`）添加一些自定义的方法。

这时，我们可以通过在声明文件中使用“声明合并”（Declaration Merging）来扩展类型定义。声明合并是 TypeScript 的一项特性，它允许我们在多个位置声明同名的类型，然后 TypeScript 会将这些声明合并为一个类型。

例如，假设我们想要为所有的数组添加一个 `last` 属性，该属性返回数组的最后一个元素。我们可以在声明文件中为 `Array` 类型添加一个新的声明：

```
// types.d.ts  
interface Array<T> {  
  last: T;  
}
```

在上面的代码中，我们通过声明一个同名的 `Array` 接口来为 `Array` 类型添加一个新的 `last` 属性。这样，我们在 TypeScript 代码中使用数组时，就可以访问这个新的 `last` 属性：

```
let nums: number[] = [1, 2, 3];  
console.log(nums.last); // 3
```

注意事项

虽然通过声明文件扩展类型定义可以让我们更灵活地使用类型，但也需要注意一些事项。

首先，声明文件只提供类型信息，不包含实现。也就是说，如果我们为一个类型添加了新的属性或方法，我们还需要在实际的代码中提供这些属性或方法的实现。

其次，尽管 TypeScript 允许我们为内置类型添加自定义的属性和方法，但这并不意味着这是一个好的做法。在很多情况下，过度修改内置类型可能会导致代码难以理解和维护。因此，我们应该谨慎使用这种特性，尽可能地遵循库或语言的原始设计。

最后，当我们在一个项目中使用多个声明文件时，需要注意文件的加载顺序和作用域问题。因为声明文件中的类型声明会影响整个项目，所以我们需要确保所有的声明文件都被正确地加载，并且不会互相冲突。

为第三方库创建声明文件

当我们在使用第三方库时，通常会遇到缺乏类型声明的情况。我们可以通过创建一个声明文件来为该库添加类型声明，以便在 TypeScript 代码中使用该库的时候获得类型检查和自动完成的支持。

以下是一个实际的示例，假设我们使用的是一个名为 `axios` 的库，它是一个流行的用于发起 HTTP 请求的库。假设 `axios` 库没有提供类型声明文件，我们可以创建一个声明文件 `axios.d.ts` 来为它添加类型声明：

```
declare module 'axios' {
  export interface AxiosRequestConfig {
    url: string;
    method?: string;
    data?: any;
    headers?: any;
  }

  export interface AxiosResponse<T = any> {
    data: T;
    status: number;
    statusText: string;
    headers: any;
    config: AxiosRequestConfig;
  }

  export function request<T = any>(config: AxiosRequestConfig):
    Promise<AxiosResponse<T>>;
  export function get<T = any>(url: string, config?: AxiosRequestConfig):
    Promise<AxiosResponse<T>>;
  export function post<T = any>(url: string, data?: any, config?: AxiosRequestConfig):
    Promise<AxiosResponse<T>>;
  // ... 其他请求方法的类型声明 ...
}
```

在这个声明文件中，我们使用 `declare module` 来声明一个名为 `axios` 的模块，并在其中定义了与 `axios` 相关的类型声明。

我们定义了 `AxiosRequestConfig` 接口，它描述了发起请求时的配置选项；定义了 `AxiosResponse` 接口，它描述了请求返回的响应数据的结构。

然后，我们通过 `export` 关键字将 `request`、`get` 和 `post` 等函数导出为模块的公共 API，以便在其他文件中使用这些函数。

现在，在我们的 TypeScript 代码中，我们可以通过导入 `axios` 模块来使用这些类型声明，以及使用 `axios` 库的方法：

```
import axios, { AxiosResponse, AxiosRequestConfig } from 'axios';

const config: AxiosRequestConfig = {
  url: 'https://api.example.com',
  method: 'GET',
};

axios.get(config)
  .then((response: AxiosResponse) => {
    console.log(response.data);
  })
  .catch((error) => {
    console.error(error);
  });
```

通过这种方式，我们可以为第三方库创建声明文件，并在 TypeScript 代码中使用它们来获得类型检查和自动完成的支持，提高代码的可靠性和开发效率。

接口和类

在 TypeScript 中，接口（Interfaces）和类（Classes）是实现面向对象编程（Object-Oriented Programming, OOP）的基础工具。这些工具提供了一种方式来定义和组织复杂的数据结构和行为。

接口

接口在 TypeScript 中扮演着关键的角色，用于强类型系统的支持。接口可以描述对象的形状，使我们可以编写出预期的行为。接口可用于描述对象、函数或者类的公共部分。

以下是一个基本的接口示例：

```
interface LabelledValue {
  label: string;
}

function printLabel(labelledObj: LabelledValue) {
  console.log(labelledObj.label);
}

let myObj = { size: 10, label: "Size 10 Object" };
printLabel(myObj);
```

在这个例子中，`LabelledValue` 接口就像一个名片，告诉其他代码，只要一个对象有 `label` 属性，并且类型为 `string`，那么就可以认为它是 `LabelledValue`。

接口也可以描述函数类型：


```
interface SearchFunc {
  (source: string, subString: string): boolean;
}

let mySearch: SearchFunc;
mySearch = function(src: string, sub: string): boolean {
  let result = src.search(sub);
  return result > -1;
}
```

此外，接口还能用于描述数组和索引类型：

```
interface StringArray {
  [index: number]: string;
}

let myArray: StringArray;
myArray = ["Bob", "Fred"];

interface Dictionary {
  [index: string]: string;
}

let myDict: Dictionary;
myDict = { "key": "value" };
```

类

与传统的 JavaScript 一样，TypeScript 也使用类（Classes）来定义对象的行为。然而，TypeScript 的类具有一些额外的特性，如访问修饰符（Access Modifiers）、静态属性（Static Properties）、抽象类（Abstract Classes）等。

以下是一个基本的类示例：

```
class Greeter {
  greeting: string;
  constructor(message: string) {
    this.greeting = message;
  }
  greet() {
    return "Hello, " + this.greeting;
  }
}

let greeter = new Greeter("world");
```

TypeScript 还引入了访问修饰符 `public`、`private` 和 `protected`。如果没有指定访问修饰符，则默认为 `public`。

```
class Animal {  
  private name: string;  
  constructor(theName: string) { this.name = theName; }  
}
```

TypeScript 类还支持继承，通过 `extends` 关键字可以创建一个子类。子类可以访问和改变父类的属性和方法：

```
class Animal {  
  name: string;  
  constructor(theName: string) { this.name = theName; }  
  move(distanceInMeters: number = 0) {  
    console.log(`${this.name} moved ${distanceInMeters}m.`);  
  }  
}  
  
class Dog extends Animal {  
  constructor(name: string) { super(name); }  
  bark() {  
    console.log('Woof! Woof!');  
  }  
}  
  
const dog = new Dog('Tom');  
dog.bark();  
dog.move(10);  
dog.bark();
```

为了实现多态，TypeScript 提供了抽象类的概念。抽象类是不能被实例化的类，只能作为其他类的基类。抽象类中可以定义抽象方法，抽象方法必须在派生类中实现：

```
abstract class Animal {  
  abstract makeSound(): void;  
  move(): void {  
    console.log('roaming the earth...');  
  }  
}  
  
class Dog extends Animal {  
  makeSound() {  
    console.log('Woof! Woof!');  
  }  
}  
  
const myDog = new Dog();  
myDog.makeSound();  
myDog.move();
```

枚举和泛型

接下来我们将学习 TypeScript 中的两个重要主题：枚举（Enums）和泛型（Generics）。这两个特性能大大提高代码的可重用性和安全性。

枚举

枚举是 TypeScript 中一种特殊的数据类型，允许我们为的一组数值设定友好的名字。枚举的定义使用 `enum` 关键字。

```
enum Direction {  
  Up = 1,  
  Down,  
  Left,  
  Right,  
}
```

在这个例子中，我们定义了一个名为 `Direction` 的枚举，它有四个成员：`Up`、`Down`、`Left` 和 `Right`。`Up` 的初始值为 1，其余成员的值会自动递增。

除了使用数值，我们也可以使用字符串：

```
enum Direction {  
  Up = "UP",  
  Down = "DOWN",  
  Left = "LEFT",  
  Right = "RIGHT",  
}
```

此外，TypeScript 还支持计算的和常量成员。常量枚举通过 `const enum` 进行定义，TypeScript 会在编译阶段进行优化：

```
const enum Enum {  
  A = 1,  
  B = A * 2  
}
```

「异构枚举」

TypeScript 支持数字和字符串混用的枚举，这种类型的枚举被称为异构枚举：

```
enum BooleanLikeHeterogeneousEnum {  
  No = 0,  
  Yes = "YES",  
}
```

尽管 TypeScript 支持这种用法，但我们在实际项目中应尽可能避免使用异构枚举，因为这会引入不必要的复杂性。

「枚举成员的类型」

在某些特殊的情况下，枚举成员本身也可以作为一种类型：

```
enum ShapeKind {  
  Circle,  
  Square,  
}  
  
interface Circle {  
  kind: ShapeKind.Circle;  
  radius: number;  
}  
  
interface Square {  
  kind: ShapeKind.Square;  
  sideLength: number;  
}
```

泛型

在 TypeScript 中，泛型（Generics）是一种强大的类型工具，它允许我们编写可重用、灵活和类型安全的代码。泛型允许我们在定义函数、类或接口时使用类型参数，这些类型参数在使用时可以被动态地指定具体的类型。

以下是泛型在 TypeScript 中的几个常见应用场景：

1. 「函数泛型」

函数泛型允许我们编写可适用于多种类型的函数，提高代码的重用性和灵活性。例如：

```
function identity<T>(arg: T): T {  
  return arg;  
}  
  
let result = identity<number>(42); // result的类型为number
```

在上面的示例中，`identity` 函数接受一个类型参数 `T`，表示输入和输出的类型。通过在函数调用时显式指定类型参数为 `number`，我们可以将 `42` 传递给 `identity` 函数并推断出结果的类型为 `number`。

2. 「接口泛型」

接口泛型允许我们创建可适用于不同类型的接口定义。例如：

```
interface Pair<T, U> {  
  first: T;  
  second: U;  
}  
  
let pair: Pair<number, string> = { first: 42, second: "hello" };
```

在上面的示例中，我们定义了一个 `Pair` 接口，它接受两个类型参数 `T` 和 `U`，表示 `first` 和 `second` 属性的类型。通过指定类型参数为 `number` 和 `string`，我们创建了一个具体的 `pair` 对象，它的 `first` 属性类型为 `number`，`second` 属性类型为 `string`。

3. 「类泛型」

类泛型允许我们创建可适用于不同类型的类定义。例如：

```
class Container<T> {  
  private value: T;  
  
  constructor(value: T) {  
    this.value = value;  
  }  
  
  getValue(): T {  
    return this.value;  
  }  
}  
  
let container = new Container<number>(42);  
let value = container.getValue(); // value的类型为number
```

在上面的示例中，我们定义了一个 `Container` 类，它接受一个类型参数 `T`，表示类的内部值的类型。通过在创建类的实例时显式指定类型参数为 `number`，我们创建了一个具体的 `container` 对象，它的 `value` 属性类型为 `number`，并可以使用 `getValue` 方法获取该值。

泛型还支持约束（Constraints）的概念，通过使用约束，我们可以限制泛型的类型范围，使其满足特定的条件。

泛型在 TypeScript 中广泛应用于函数、类、接口和类型别名的定义中，它提供了一种灵活、类型安全且可重用的方式来处理不同类型的数据。通过使用泛型，我们可以在编写代码时提供更强大的类型支持，从而减少错误并提高代码的可维护性和可读性。

泛型和类型体操

泛型和类型体操（Type Gymnastics）是 TypeScript 中高级类型系统的重要组成部分。它们提供了强大的工具和技巧，用于处理复杂的类型操作和转换。

泛型（Generics）

1. 泛型函数

泛型函数允许我们在函数定义中使用类型参数，以便在函数调用时动态指定类型。例如：

```
function identity<T>(arg: T): T {  
  return arg;  
}  
  
let result = identity<number>(42); // result 的类型为 number
```

在上面的示例中，`identity` 函数使用类型参数 `T`，并返回与输入类型相同的值。通过显式传递泛型参数，我们可以确保在函数调用时指定了具体的类型。

2. 泛型接口

泛型接口允许我们在接口定义中使用类型参数，以便在实现该接口时指定具体的类型。例如：


```
interface Container<T> {  
  value: T;  
}  
  
let container: Container<number> = { value: 42 };
```

在上面的示例中，我们定义了一个泛型接口 `Container`，它包含一个类型参数 `T`。通过指定 `Container<number>`，我们创建了一个具体的实现，其中的 `value` 属性类型为 `number`。

3. 泛型类

泛型类允许我们在类定义中使用类型参数，以便在创建类的实例时指定具体的类型。例如：

```
class Stack<T> {  
  private items: T[] = [];  
  
  push(item: T) {  
    this.items.push(item);  
  }  
  
  pop(): T | undefined {  
    return this.items.pop();  
  }  
}  
  
let stack = new Stack<number>();  
stack.push(1);  
stack.push(2);  
let item = stack.pop(); // item 的类型为 number | undefined
```

在上面的示例中，我们定义了一个泛型类 `Stack`，它使用类型参数 `T` 来表示堆栈中的元素类型。通过创建 `Stack<number>` 的实例，我们限制了堆栈中的元素必须为 `number` 类型。

类型体操（Type Gymnastics）

1. 条件类型（Conditional Types）

条件类型允许我们根据输入类型的条件判断结果来选择不同的类型。条件类型的语法形式为：

```
T extends U ? X : Y
```

其中，`T` 是待检查的类型，`U` 是条件类型，`X` 是满足条件时返回的类型，`Y` 是不满足条件时返回的类型。

下面是一个使用条件类型的示例：

```
type Check<T> = T extends string ? true : false;  
  
type Result = Check<string>; // Result 的类型为 true
```

在上面的示例中，我们定义了一个条件

类型 `Check<T>`，它接受一个类型参数 `T`。如果 `T` 是 `string` 类型，那么 `Check<T>` 的类型将是 `true`，否则为 `false`。

2. keyof 操作符和索引访问类型

`keyof` 操作符用于获取类型的所有属性名，结合索引访问类型可以从一个类型中获取属性的具体类型。

```
interface Person {
  name: string;
  age: number;
}

type PersonKeys = keyof Person; // "name" | "age"
type PersonNameType = Person['name']; // string
```

在上面的示例中，我们使用 `keyof` 操作符获取了 `Person` 接口的属性名集合，并通过索引访问类型获取了 `Person` 接口中 `name` 属性的类型。

3. infer 关键字

`infer` 关键字用于在条件类型中推断类型，并将其赋值给一个类型变量。

```
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function add(a: number, b: number): number {
  return a + b;
}

type AddReturnValue = ReturnType<typeof add>; // 类型为 number
```

在上面的示例中，`ReturnType` 类型接受一个类型参数 `T`，并使用条件类型和 `infer` 关键字推断函数类型的返回类型。通过调用 `ReturnType<typeof add>`，我们推断出 `add` 函数的返回类型为 `number`。

当涉及到泛型时，还有一些重要的概念和内置泛型函数可以深入分析。让我们继续探讨 `extends` 关键字、TS 官方内置的一些泛型函数以及它们的使用。

extends 关键字和类型约束

在泛型中，我们可以使用 `extends` 关键字来对泛型类型进行约束。这样可以确保传递给泛型的类型满足特定条件。

```
function printProperty<T extends { name: string }>(obj: T): void {
  console.log(obj.name);
}

printProperty({ name: 'John', age: 25 }); // 输出 'John'
```

在上面的示例中，`printProperty` 函数接受一个泛型参数 `T`，该参数必须满足一个约束条件：具有 `name` 属性，且 `name` 的类型为 `string`。通过使用 `extends` 关键字和类型约束，我们可以确保 `obj` 参数具有所需的属性和类型，从而避免出现错误。

泛型函数Util

TypeScript 提供了一些内置的泛型函数，这些函数被广泛用于处理各种类型操作。以下是一些常见的官方内置泛型函数：

Partial<T>

`Partial<T>` 是 TypeScript 中的一个内置泛型类型，它可以将给定类型 `T` 中的所有属性转换为可选属性。这对于创建部分完整的对象非常有用。

```
interface Person {
  name: string;
  age: number;
}

type PartialPerson = Partial<Person>;

const partialPerson: PartialPerson = { name: 'John' }; // age 属性是可选的
```

在上面的示例中，`Partial<Person>` 将 `Person` 接口中的所有属性变为可选属性，从而创建了一个部分完整的 `PartialPerson` 类型。

Required<T>

`Required<T>` 是 TypeScript 中的另一个内置泛型类型，它可以将给定类型 `T` 中的所有可选属性转换为必需属性。这对于确保对象的完整性非常有用。

```
interface Person {
  name?: string;
  age?: number;
}

type RequiredPerson = Required<Person>;

const requiredPerson: RequiredPerson = { name: 'John', age: 25 }; // name 和 age 属性是必需的
```

在上面的示例中，`Required<Person>` 将 `Person` 接口中的所有可选属性变为必需属性，从而创建了一个要求完整性的 `RequiredPerson` 类型。

Pick<T, K>

`Pick<T, K>` 是 TypeScript 中的另一个内置泛型函数，它可以从给定类型 `T` 中选择指定的属性 `K` 组成一个新的类型。

```
interface Person {
  name: string;
  age: number;
  address: string;
}

type NameAndAge = Pick<Person, 'name' | 'age'>;

const person: NameAndAge = { name:
  'John', age: 25 }; // 只包含 name 和 age 属性
```

在上面的示例中，`Pick<Person, 'name' | 'age'>` 从 `Person` 接口中选择了 `'name'` 和 `'age'` 属性，创建了一个新的类型 `NameAndAge`。

我们还可以结合泛型和内置泛型函数来实现更复杂的类型操作。以下是一个示例，展示了如何使用 `Pick` 和泛型来创建一个函数，该函数从给定对象中选择指定属性，并返回一个新的对象。

```
function pickProperties<T, K extends keyof T>(obj: T, keys: K[]): Pick<T, K> {
  const result: Partial<T> = {};
  for (const key of keys) {
    result[key] = obj[key];
  }
  return result as Pick<T, K>;
}

interface Person {
  name: string;
  age: number;
  address: string;
}

const person: Person = {
  name: 'John',
  age: 25,
  address: '123 Main St'
};

const nameAndAge = pickProperties(person, ['name', 'age']); // 只包含 name 和 age 属性
console.log(nameAndAge); // 输出: { name: 'John', age: 25 }
```

在上面的示例中，`pickProperties` 函数接受一个泛型参数 `T` 和一个属性数组 `keys`。通过使用 `Pick<T, K>`，我们将从给定对象 `obj` 中选择指定的属性 `keys`，并创建一个新的对象。

这个例子结合了泛型、内置泛型函数 `Pick`、`keyof` 操作符和 `extends` 关键字，展示了如何在 TypeScript 中处理复杂的类型操作和转换。

当涉及到官方内置的泛型函数时，还有一些重要的函数值得分析。让我们继续探讨一些常用的官方内置泛型函数以及它们的使用。

Exclude<T, U>

`Exclude<T, U>` 是 TypeScript 中的一个内置泛型函数，用于从类型 `T` 中排除类型 `U`。它返回一个新类型，该新类型包含在 `T` 中存在但不在 `U` 中存在的成员类型。

```
type T = Exclude<"a" | "b" | "c", "a" | "b">; // T 的类型为 "c"
```

在上面的示例中，`Exclude<"a" | "b" | "c", "a" | "b">` 排除了类型 `"a"` 和 `"b"`，返回类型为 `"c"`。

Omit<T, K>

`Omit<T, K>` 是 TypeScript 中的另一个内置泛型函数，它返回一个新类型，该新类型排除了类型 `T` 中指定的属性 `K`。

```
interface Person {  
  name: string;  
  age: number;  
  address: string;  
}  
  
type PersonWithoutAddress = Omit<Person, "address">;
```

在上面的示例中，`Omit<Person, "address">` 返回了一个新类型 `PersonWithoutAddress`，该类型排除了 `Person` 接口中的 `address` 属性。

Readonly<T>

`Readonly<T>` 是 TypeScript 中的另一个内置泛型函数，它将类型 `T` 中的所有属性转换为只读属性。

```
interface Person {  
  name: string;  
  age: number;  
}  
  
type ReadonlyPerson = Readonly<Person>;
```

在上面的示例中，`Readonly<Person>` 将 `Person` 接口中的所有属性变为只读属性，创建了一个新类型 `ReadonlyPerson`。

总结

泛型和类型体操是 TypeScript 中强大的类型系统的关键组成部分。通过使用泛型，我们可以创建可重用、灵活和类型安全的代码。内置泛型函数提供了一些常用的类型转换工具，如 `Partial`、`Required` 和 `Pick`，可以帮助我们更方便地处理类型操作。

通过结合泛型、`extends` 关键字、内置泛型函数和其他高级类型概念，我们能够在 TypeScript 中编写更复杂、类型安全的代码，并利用 TypeScript 的强大类型系统来提高代码的可读性、可维护性和可扩展性。

类型

TypeScript提供了JavaScript的所有基本数据类型，如：`number`、`string`、`boolean`等。它还增加了额外的类型，比如`any`、`unknown`、`never`、`void`等。

number

在TypeScript中，所有的数字都是浮点数。这些数字的类型是`number`。下面是一些例子：

```
let decimal: number = 6; // 十进制
let hex: number = 0xf00d; // 十六进制
let binary: number = 0b1010; // 二进制
let octal: number = 0o744; // 八进制
```

string

`string`类型表示文本数据。你可以使用单引号（`'`）或双引号（`"`）定义字符串，也可以使用反引号（```）定义模板字符串：

```
let color: string = "blue";
color = 'red';

let fullName: string = `Bob Bobbington`;
let age: number = 37;
let sentence: string = `Hello, my name is ${fullName}. I'll be ${age + 1} years old next month.`;
```

boolean

`boolean`类型有两个值：`true`和`false`：

```
let isDone: boolean = false;
```

Array

在TypeScript中，数组类型有两种表达方式。一种是在元素类型后面加上`[]`，表示由此类型元素组成的一个数组。另一种方式是使用数组泛型，`Array<元素类型>`：

```
let list: number[] = [1, 2, 3];
// 或
let list: Array<number> = [1, 2, 3];
```

Tuple

元组类型允许表示一个已知元素数量和类型的数组，各元素的类型不必相同。比如，你可以定义一对值分别为`string`和`number`的元组：

```
let x: [string, number];  
x = ['hello', 10]; // OK
```

以上是TypeScript的一些基本类型。在接下来的对话中，我们可以进一步讨论其他的TypeScript类型，比如枚举（`enum`）、`null`、`undefined`、`never`、`void`以及对象类型。

Enum

Enum是一种特殊的类型，它可以更容易地处理一组有名字的常量。在TypeScript中，enum的默认起始值是0，然后每个成员的值都会依次增加。你也可以手动为enum的成员指定值：

```
enum Color {Red, Green, Blue}  
let c: Color = Color.Green;  
  
// 手动指定成员的数值  
enum Color {Red = 1, Green = 2, Blue = 4}  
let c: Color = Color.Green;
```

Null and Undefined

在TypeScript中，`undefined`和`null`各自有自己的类型，分别是`undefined`和`null`。默认情况下，它们是所有类型的子类型。这意味着你可以把`null`和`undefined`赋值给`number`类型的变量。

然而，当你指定了`--strictNullChecks`标记，`null`和`undefined`只能赋值给`void`和它们各自的类型：

```
let u: undefined = undefined;  
let n: null = null;
```

Any

当你不确定一个变量应该是什么类型的时候，你可能需要用到`any`类型。`any`类型的变量允许你对它进行任何操作，它就像是TypeScript类型系统的一个逃生窗口：

```
let notSure: any = 4;  
notSure = "maybe a string instead";  
notSure = false; // okay, definitely a boolean
```

Unknown

`unknown`类型是TypeScript 3.0中引入的一种新类型，它是`any`类型对应的安全类型。`unknown`类型的变量只能被赋值给`any`类型和`unknown`类型本身：

```
let value: unknown;

value = true;      // OK
value = 42;        // OK
value = "Hello World"; // OK
value = [];        // OK
value = {};        // OK
```

Never

`never` 类型表示的是那些永不存在的值的类型。例如，`never` 类型是那些总是会抛出异常或者根本就不会有返回值的函数表达式或箭头函数表达式的返回值类型：

```
function error(message: string): never {
    throw new Error(message);
}
```

void

在TypeScript中，`void` 类型用于表示没有返回值的函数的返回类型。在JavaScript中，当一个函数没有返回任何值时，它会隐式地返回 `undefined`。`void` 类型就是用来表达这种情况的：

```
function warnUser(): void {
    console.log("This is my warning message");
}
```

在这个例子中，`warnUser` 函数没有返回任何值，所以它的返回类型是 `void`。

需要注意的是，在TypeScript中，你可以声明一个 `void` 类型的变量，但是你只能为它赋予 `undefined` 和 `null`（在非严格null检查模式下）：

```
let unusable: void = undefined;
```

通常情况下，我们不会这样使用 `void` 类型，因为除了 `undefined` 和 `null` 之外，你不能将任何值赋给 `void` 类型的变量。

联合类型（Union Types）

TypeScript联合类型是一种将多种类型组合到一起的方式，表示一个值可以是多种类型之一。你可以使用管道符（`|`）来分隔每个类型。这可以让你在不确定一个值是什么类型的时候，为它选择多个可能的类型。

例如，假设我们有一个函数，这个函数的参数可以是 `number` 类型或者 `string` 类型：

```
function display(input: string | number) {
    console.log(input);
}
```

在上面的函数中，我们声明了 `input` 参数可以是 `string` 类型或者 `number` 类型。你可以传递一个 `string` 类型或者 `number` 类型的值给 `display` 函数，而 TypeScript 编译器不会报错：

```
display(1); // OK
display("Hello"); // OK
```

你也可以将联合类型用于变量和属性。例如：

```
let variable: string | number;

variable = "Hello"; // OK
variable = 1; // OK
```

在上面的代码中，我们声明了 `variable` 可以是 `string` 类型或者 `number` 类型。然后我们可以安全地将一个字符串或者数字赋值给 `variable`。

联合类型在 TypeScript 中非常常用，因为它们可以帮助你编写更灵活的代码。

交叉类型（Intersection Types）

交叉类型是将多个类型合并为一个类型。这让我们可以把现有的多种类型叠加到一起获得所需的能力。它被定义为 `Type1 & Type2 & Type3 & ... & TypeN`。它包含了所需的所有类型的成员。

```
interface Part1 {
  a: string;
}

interface Part2 {
  b: number;
}

type Combined = Part1 & Part2;

let obj: Combined = {
  a: 'hello',
  b: 42,
};
```

类型别名（Type Aliases）

类型别名是给一个类型起个新名字。类型别名有时和接口很相似，但可以作用于原始值，联合类型，交叉类型等任何我们需要手写的地方。

```
type MyBool = true | false;
type StringOrNumber = string | number;
```

字符串字面量类型（String Literal Types）

字符串字面量类型允许你指定字符串必须的固定值。在实践中，这种类型常与联合类型、类型别名和类型保护结合使用。

```
type Easing = "ease-in" | "ease-out" | "ease-in-out";

class UIElement {
  animate(dx: number, dy: number, easing: Easing) {
    // ...
  }
}

let button = new UIElement();
button.animate(0, 0, "ease-in"); // OK
button.animate(0, 0, "uneasy"); // Error: "uneasy" is not allowed here
```

类型兼容：协变和逆变

引言

在类型系统中，协变和逆变是对类型比较(类型兼容)一种形式化描述。在一些类型系统中，例如 Java，这些概念是显式嵌入到语言中的，例如使用 `extends` 关键字表示协变，使用 `super` 关键字表示逆变。在其他一些类型系统中，例如 TypeScript，协变和逆变的规则是隐式嵌入的，通过类型兼容性检查来实现。

协变和逆变的存在使得类型系统具有更大的灵活性。例如，如果你有一个 `Animal` 类型的数组，并且你有一个 `Dog` 类型的对象（假设 `Dog` 是 `Animal` 的子类型），那么你应该能够将 `Dog` 对象添加到 `Animal` 数组中。这就是协变。反过来，如果你有一个处理 `Animal` 类型对象的函数，并且你有一个 `Dog` 类型的对象，你应该可以使用这个函数来处理 `Dog` 对象。这就是逆变。

协变和逆变还可以帮助我们创建更通用的代码。例如，如果你有一个可以处理任何 `Animal` 的函数，那么这个函数应该能够处理任何 `Animal` 的子类型。这意味着，你可以编写一段只依赖于 `Animal` 类型的代码，然后使用这段代码处理任何 `Animal` 的子类型。

协变（Covariance）

协变描述的是如果存在类型 A 和 B，并且 A 是 B 的子类型，那么我们就可以说由 A 组成的复合类型（例如 `Array<A>` 或者 `(a: A) => void`）也是由 B 组成的相应复合类型（例如 `Array<B>` 或者 `(b: B) => void`）的子类型。

让我们通过一个例子来理解协变。假设我们有两个类型 `Animal` 和 `Dog`，其中 `Dog` 是 `Animal` 的子类型。

```
type Animal = { name: string };
type Dog = Animal & { breed: string };

let dogs: Dog[] = [{ name: "Fido", breed: "Poodle" }];
let animals: Animal[] = dogs; // OK because Dog extends Animal, Dog[] is a subtype of Animal[]
```

这里我们可以将类型为 `Dog[]` 的 `dogs` 赋值给类型为 `Animal[]` 的 `animals`，因为 `Dog[]` 是 `Animal[]` 的子类型，所以数组是协变的。

协变：类型的向下兼容性

协变是类型系统中的一个基本概念，它描述的是类型的“向下兼容性”。如果一个类型A可以被看作是另一个类型B的子类型（即A可以被安全地用在期望B的任何地方），那么我们就说A到B是协变的。这是类型系统中最常见和直观的一种关系，例如在面向对象编程中的继承就是协变的一种表现。

在TypeScript中，所有的类型都是自身的子类型（即每个类型到自身是协变的），并且 `null` 和 `undefined` 类型是所有类型的子类型。除此之外，接口和类也可以通过继承来形成协变关系。

```
class Animal {
  name: string;
}

class Dog extends Animal {
  breed: string;
}

let myDog: Dog = new Dog();
let myAnimal: Animal = myDog; // OK, 因为Dog是Animal的子类型
```

这个例子中，我们可以将一个 `Dog` 对象赋值给一个 `Animal` 类型的变量，因为 `Dog` 到 `Animal` 是协变的。

在TypeScript中，泛型类型也是协变的。例如，如果类型A是类型B的子类型，那么 `Array<A>` 就是 `Array<B>` 的子类型。

```
let dogs: Array<Dog> = [new Dog()];
let animals: Array<Animal> = dogs; // OK, 因为Array<Dog>是Array<Animal>的子类型
```

逆变（Contravariance）

逆变是协变的反面。如果存在类型A和B，并且A是B的子类型，那么我们就可以说由B组成的某些复合类型是由A组成的相应复合类型的子类型。

这在函数参数中最常见。让我们来看一个例子：

```
type Animal = { name: string };
type Dog = Animal & { breed: string };

let dogHandler = (dog: Dog) => { console.log(dog.breed); }
let animalHandler: (animal: Animal) => void = dogHandler; // Error!
```

在这个例子中，我们不能将类型为 `(dog: Dog) => void` 的 `dogHandler` 赋值给类型为 `(animal: Animal) => void` 的 `animalHandler`。因为如果我们传递一个 `Animal`（并非所有的 `Animal` 都是 `Dog`）给 `animalHandler`，那么在执行 `dogHandler` 函数的时候，就可能会引用不存在的 `breed` 属性。因此，函数的参数类型是逆变的。

逆变：类型的向上兼容性

逆变描述的是类型的“向上兼容性”。如果一个类型A可以被看作是另一个类型B的超类型（即B可以被安全地用在期望A的任何地方），那么我们就说A到B是逆变的。在函数参数类型的兼容性检查中，TypeScript使用了逆变。

```
type Handler = (arg: Animal) => void;
```

```
let animalHandler: Handler = (animal: Animal) => { /* ... */};
```

```
let dogHandler: Handler = (dog: Dog) => { /* ... */}; // OK, 因为Animal是Dog的超类型
```

这个例子中，我们可以将一个处理`

Dog 的函数赋值给一个处理 Animal 的函数类型的变量，因为 Animal 是 Dog 的超类型，所以 (dog: Dog) => void 类型是 (animal: Animal) => void 类型的子类型。

这看起来可能有些反直觉，但实际上是为了保证类型安全。因为在执行 dogHandler 函数时，我们可以安全地传入一个 Animal 对象，而不需要担心它可能不是 Dog 类型。

协变与逆变的平衡

协变和逆变在大多数情况下都可以提供合适的类型检查，但是它们并非完美无缺。在实际应用中，我们必须关注可能的边界情况，以避免运行时错误。在某些情况下，我们甚至需要主动破坏类型的协变或逆变，以获得更强的类型安全。例如，如果我们需要向一个 Dog[] 数组中添加 Animal 对象，我们可能需要将这个数组的类型声明为 Animal[]，以防止添加不兼容的类型。

总的来说，协变和逆变是理解和应用 TypeScript 类型系统的重要工具，但我们必须在灵活性和类型安全之间找到合适的平衡。

类型兼容：结构化类型

TypeScript 是一种基于 JavaScript 的静态类型语言，它为 JavaScript 添加了类型系统，并提供了强大的类型检查和自动补全功能。TypeScript 的类型系统有一个非常重要的特性，那就是 "鸭子类型" (Duck Typing) 或 "结构化类型" (Structural Typing) (文章会以 "鸭子类型" (Duck Typing) 作为简称)。这种特性有时会让人感到惊讶，但它是 TypeScript 增强 JavaScript 开发体验的重要方式之一。

鸭子类型的概念来自一个古老的英语成语："如果它走起路来像一只鸭子，叫起来也像一只鸭子，那么它就是一只鸭子。"在 TypeScript (或更一般地说，静态类型语言) 的上下文中，鸭子类型意味着一个对象的类型不是由它继承或实现的具体类别决定的，而是由它具有的结构决定的。

本文将全面深入地探讨 TypeScript 中的鸭子类型，以及如何在实际的开发中应用和利用鸭子类型。

1. 鸭子类型：定义和示例

鸭子类型的概念来自一个古老的英语成语："如果它走起路来像一只鸭子，叫起来也像一只鸭子，那么它就是一只鸭子。"在 TypeScript (或更一般地说，静态类型语言) 的上下文中，鸭子类型意味着一个对象的类型不是由它继承或实现的具体类别决定的，而是由它具有的结构决定的。

这是一个简单的鸭子类型示例：

```
interface Duck {
  walk: () => void;
  quack: () => void;
}

function doDuckThings(duck: Duck) {
  duck.walk();
  duck.quack();
}

const myDuck = {
  walk: () => console.log('Walking like a duck'),
  quack: () => console.log('Quacking like a duck'),
  swim: () => console.log('Swimming like a duck')
};

doDuckThings(myDuck); // OK
```

在这个例子中，我们定义了一个 `Duck` 接口和一个 `doDuckThings` 函数，这个函数需要一个 `Duck` 类型的参数。然后我们创建了一个 `myDuck` 对象，它有 `walk`、`quack` 和 `swim` 这三个方法。尽管 `myDuck` 并没有显式地声明它实现了 `Duck` 接口，但是由于 `myDuck` 的结构满足了 `Duck` 接口的要求（即 `myDuck` 有 `walk` 和 `quack` 这两个方法），我们可以将 `myDuck` 作为参数传递给 `doDuckThings` 函数。

这就是鸭子类型的基本概念：只要一个对象的结构满足了接口的要求，我们就可以把这个对象看作是这个接口的实例，而不管这个对象的实际类型是什么。

2. 鸭子类型的优点

鸭子类型有许多优点，特别是在编写更灵活和更通用的代码方面。

2.1 代码的灵活性

鸭子类型增加了代码的灵活性。我们可以创建和使用满足特定接口的任何对象，而不必担心它们的具体类型。这使得我们可以更容易地编写通用的代码，因为我们的代码只依赖于对象的结构，而不是对象的具体类型。

2.2 代码的复用

鸭子类型有助于代码的复用。由于我们的函数和方法只依赖于对象的结构，我们可以在不同的上下文中重用这些函数和方法，只要传入的对象满足所需的结构。

例如，我们可以写一个函数，它接受一个具有 `toString` 方法的任何对象，然后返回这个对象的字符串表示。由于几乎所有的 JavaScript 对象都有 `toString` 方法，我们可以在许多不同的上下文中重用这个函数。

```
function toString(obj: { toString: () => string }) {  
    return obj.toString();  
}  
  
console.log(toString(123)); // "123"  
console.log(toString([1, 2, 3])); // "1,2,3"  
console.log(toString({ a: 1, b: 2 })); // "[object Object]"
```

2.3 与 JavaScript 的互操作性

鸭子类型提高了 TypeScript 与 JavaScript 的互操作性。由于 JavaScript 是一种动态类型语言，我们经常需要处理的对象可能没有明确的类型。鸭子类型使我们能够在 TypeScript 中安全地处理这些对象，只要它们的结构满足我们的需求。

例如，我们可能从一个 JavaScript 库获取一个对象，这个对象有一个 `forEach` 方法。我们不关心这个对象的具体类型，我们只关心它是否有 `forEach` 方法。使用鸭子类型，我们可以定义一个接口来描述这个对象的结构，然后在 TypeScript 中安全地使用这个对象。

```
interface Iterable {  
    forEach: (callback: (item: any) => void) => void;  
}  
  
function processItems(iterable: Iterable) {  
    iterable.forEach(item => console.log(item));  
}  
  
const jsArray = [1, 2, 3]; // From a JavaScript library  
processItems(jsArray); // OK
```

3. 鸭子类型的局限性

尽管鸭子类型有许多优点，但它也有一些局限性。

3.1 类型安全

鸭子类型可能会降低代码的类型安全性。因为 TypeScript 的类型检查器只检查对象是否满足接口的结构，而不检查对象是否真的是接口所期望的类型。如果一个对象恰好有与接口相同的属性和方法，但实际上它并不是接口所期望的类型，TypeScript 的类型检查器可能无法发现这个错误。

例如，我们可能有一个 `Dog` 类型和一个 `Cat` 类型，它们都有一个 `bark` 方法。我们可能会错误地将一个 `Cat`

对象传递给一个期望 `Dog` 对象的函数，而 TypeScript 的类型检查器无法发现这个错误。


```
interface Dog {
  bark: () => void;
}

function letDogBark(dog: Dog) {
  dog.bark();
}

const cat = {
  bark: () => console.log('Meow...'), // Cats don't bark!
  purr: () => console.log('Purr...')
};

letDogBark(cat); // No error, but it's wrong!
```

在这种情况下，我们需要更仔细地设计我们的类型和接口，以避免混淆。

3.2 易读性和可维护性

鸭子类型可能会降低代码的易读性和可维护性。因为我们的代码只依赖于对象的结构，而不是对象的具体类型，这可能会使代码更难理解和维护。

为了提高易读性和可维护性，我们需要清晰地记录我们的接口和函数期望的对象结构。TypeScript 的类型注解和接口提供了一种强大的工具来实现这一点。

4. 使用鸭子类型的最佳实践

在使用鸭子类型时，有一些最佳实践可以帮助我们避免上述问题，并充分利用鸭子类型的优点。

4.1 清晰地定义接口

我们应该清晰地定义我们的接口，以描述我们的函数和方法期望的对象结构。这有助于提高代码的易读性和可维护性。

例如，如果我们有一个函数，它期望一个具有 `name` 和 `age` 属性的对象，我们应该定义一个接口来描述这个结构。

```
interface Person {
  name: string;
  age: number;
}

function greet(person: Person) {
  console.log(`Hello, my name is ${person.name} and I'm ${person.age} years old.`);
}
```

4.2 适度使用鸭子类型

我们应该适度地使用鸭子类型。虽然鸭子类型有许多优点，但如果过度使用，可能会导致类型安全性的问题，以及易读性和可维护性的降低。我们应该在类型安全性、易读性、可维护性和灵活性之间找到一个平衡。

在某些情况下，我们可能更希望使用类和继承，而不是鸭子类型。例如，如果我们有一组紧密相关的类型，它们有共享的行为和状态，使用类和继承可能更合适。


```
interface Named {
  name: string;
}

class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }
}

let p: Named;
// OK, because of structural typing
p = new Person('mike');
```

在这个例子中，尽管 `Person` 类并没有显式地实现 `Named` 接口，但是因为 `Person` 类有一个 `name` 属性，所以我们可以把 `Person` 的实例赋值给 `Named` 类型的变量。这是由于 TypeScript 的 "鸭子类型" 或 "结构化类型" 系统导致的。

类型守卫

概述

在 TypeScript 中，类型守卫可以用于在运行时检查变量的类型，并在代码块内部将变量的类型范围缩小到更具体的类型。这种类型收窄可以让 TypeScript 编译器更好地理解我们代码的意图，从而提供更准确的类型推断和类型检查。

类型守卫通常使用类型断言、类型谓词、`typeof` 操作符、`instanceof` 操作符或自定义的谓词函数来判断变量的具体类型，并根据判断结果收窄变量的类型范围。

typeof 类型守卫

`typeof` 类型守卫允许我们使用 `typeof` 操作符来在代码中根据变量的类型范围进行条件判断。以下是一个示例：

```
function printValue(value: string | number) {
  if (typeof value === 'string') {
    console.log(value.toUpperCase());
  } else {
    console.log(value.toFixed(2));
  }
}

printValue('hello'); // 输出: HELLO
printValue(3.1415); // 输出: 3.14
```

在上面的示例中，我们使用 `typeof` 操作符在条件语句中检查变量 `value` 的类型。如果它的类型是 `'string'`，则调用 `toUpperCase` 方法；如果是 `'number'`，则调用 `toFixed` 方法。通过使用 `typeof` 类型守卫，我们能够根据不同的类型执行不同的代码逻辑。

instanceof 类型守卫

instanceof 类型守卫允许我们使用 instanceof 操作符来检查对象的类型，并在代码块内部收窄对象的类型范围。以下是一个示例：

```
class Animal {
  move() {
    console.log('Animal is moving');
  }
}

class Dog extends Animal {
  bark() {
    console.log('Dog is barking');
  }
}

function performAction(animal: Animal) {
  if (animal instanceof Dog) {
    animal.bark();
  } else {
    animal.move();
  }
}

const animal1 = new Animal();
const animal2 = new Dog();

performAction(animal1); // 输出: Animal is moving
performAction(animal2); // 输出: Dog is barking
```

在上面的示例中，我们使用 instanceof 操作符在条件语句中检查变量 `animal` 的类型。如果它是 Dog 类的实例，则调用 `bark` 方法；否则调用 `move` 方法。通过使用 instanceof 类型守卫，我们可以根据对象的具体类型执行不同的代码逻辑。

使用自定义谓词函数类型守卫

自定义谓词函数类型守卫允许我们定义自己的函数，根据特定条件判断变量的类型，并在代码块内部收窄变量的类型范围。以下是一个示例：

```
interface Circle {
  kind: 'circle';
  radius: number;
}

interface Rectangle {
  kind: 'rectangle';
  width: number;
  height: number;
}

type Shape = Circle | Rectangle;

function calculateArea(shape: Shape) {
  if (isCircle(shape)) {
    console.log(Math.PI * shape.radius ** 2);
  } else {
    console.log(shape.width * shape.height);
  }
}

function isCircle(shape: Shape): shape is Circle {
  return shape.kind === 'circle';
}

const circle: Circle = { kind: 'circle', radius: 5 };
const rectangle: Rectangle = { kind: 'rectangle', width: 10, height: 20 };

calculateArea(circle); // 输出: 78.53981633974483
calculateArea(rectangle); // 输出: 200
```

在上面的示例中，我们定义了 `Shape` 类型，它可以是 `Circle` 或 `Rectangle`。通过自定义的谓词函数 `isCircle`，我们判断变量 `shape` 的类型是否是 `Circle`，并在条件语句内部收窄变量的类型范围。通过使用自定义谓词函数类型守卫，我们能够根据特定的谓词条件执行相应的代码逻辑。

联合类型守卫

类型守卫最常用于联合类型中，因为联合类型可能包含多个不同的类型选项。以下是一个更复杂的示例，展示了如何使用类型守卫和联合类型来提供更精确的类型推断和类型检查：

```
interface Car {
  type: 'car';
  brand: string;
  wheels: number;
}

interface Bicycle {
  type: 'bicycle';
  color: string;
}

interface Motorcycle {
  type: 'motorcycle';
  engine: number;
}

type Vehicle = Car | Bicycle | Motorcycle;

function printVehicleInfo(vehicle: Vehicle) {
  switch (vehicle.type) {
    case 'car':
      console.log(`Brand: ${vehicle.brand}, Wheels: ${vehicle.wheels}`);
      break;
    case 'bicycle':
      console.log(`Color: ${vehicle.color}`);
      break;
    case 'motorcycle':
      console.log(`Engine: ${vehicle.engine}`);
      break;
    default:
      const _exhaustiveCheck: never = vehicle;
  }
}

const car: Car = { type: 'car', brand: 'Toyota', wheels: 4 };
const bicycle: Bicycle = { type: 'bicycle', color: 'red' };
const motorcycle: Motorcycle = { type: 'motorcycle', engine: 1000 };

printVehicleInfo(car);      // 输出: Brand: Toyota, Wheels: 4
printVehicleInfo(bicycle);  // 输出: Color: red
printVehicleInfo(motorcycle); // 输出: Engine: 1000
```

在上面的示例中，我们定义了 `Vehicle` 类型，它是 `Car`、`Bicycle` 和 `Motorcycle` 的联合类型。通过使用 `switch` 语句和根据 `vehicle.type` 的不同值进行类型守卫，我们可以在每个 `case` 分支中收窄 `vehicle`` 的类型范围，并执行相应的代码逻辑。通过这种方式，我们能够更准确地推断和检查联合类型的变量。

非常抱歉之前的回答没有覆盖到你提到的其他重要概念。下面我将继续解析这些概念并提供相应的代码示例。

使用 `in` 操作符进行类型守卫

`in` 操作符可以用于在 TypeScript 中判断一个属性是否存在于对象中，从而进行类型判断和类型收窄。以下是一个示例：

```
interface Circle {
  kind: 'circle';
  radius: number;
}

interface Rectangle {
  kind: 'rectangle';
  width: number;
  height: number;
}

type Shape = Circle | Rectangle;

function printArea(shape: Shape) {
  if ('radius' in shape) {
    console.log(Math.PI * shape.radius ** 2);
  } else {
    console.log(shape.width * shape.height);
  }
}

const circle: Circle = { kind: 'circle', radius: 5 };
const rectangle: Rectangle = { kind: 'rectangle', width: 10, height: 20 };

printArea(circle); // 输出: 78.53981633974483
printArea(rectangle); // 输出: 200
```

在上面的示例中，我们使用 `in` 操作符来检查属性 `'radius'` 是否存在于 `shape` 对象中。如果存在，则收窄 `shape` 的类型为 `Circle`，并执行相应的代码逻辑。通过使用 `in` 操作符进行类型判断，我们可以根据属性的存在与否进行类型收窄。

控制流类型守卫

在 TypeScript 中，当执行特定的操作后，编译器会智能地调整变量的类型范围，这被称为控制流类型收窄。以下是一些常见的控制流类型收窄情况：

「if 语句的条件判断」

```
function printValue(value: string | number) {
  if (typeof value === 'string') {
    console.log(value.toUpperCase());
  } else {
    console.log(value.toFixed(2));
  }
}
```

在上面的示例中，当执行 `typeof value === 'string'` 的条件判断时，TypeScript 编译器会收窄 `value` 的类型为 `string`，从而在代码块内部提供相应的智能提示和类型检查。

「switch 语句的 case 判断」


```
type Fruit = 'apple' | 'banana' | 'orange';

function getFruitColor(fruit: Fruit) {
  let color: string;
  switch (fruit) {
    case 'apple':
      color = 'red';
      break;
    case 'banana':
      color = 'yellow';
      break;
    default:
      color = 'orange';
  }
  console.log(`The color of ${fruit} is ${color}`);
}
```

在上面的示例中，根据 `switch` 语句中的 `case` 判断，TypeScript 编译器会智能地收窄 `color` 的类型为相应的颜色字符串。

真值类型守卫

真值收窄是一种在条件表达式中进行类型收窄的机制。当条件表达式的结果是真值时，TypeScript 编译器

会将变量的类型范围缩小为 `true` 的类型。以下是一个示例：

```
function processValue(value: string | null) {
  if (value) {
    console.log(value.toUpperCase());
  } else {
    console.log('Value is null or empty');
  }
}
```

在上面的示例中，当条件表达式 `value` 的结果是真值（即不为 `null` 或空字符串）时，TypeScript 编译器会将 `value` 的类型范围缩小为 `string`。

自定义类型判断式（Type Predicates）守卫

TypeScript 提供了自定义类型判断式的功能，它允许我们定义自己的谓词函数来进行类型判断和类型收窄。以下是一个示例：

```
interface Bird {
  fly(): void;
}

interface Fish {
  swim(): void;
}

function isBird(animal: Bird | Fish): animal is Bird {
  return (animal as Bird).fly !== undefined;
}

function processAnimal(animal: Bird | Fish) {
  if (isBird(animal)) {
    animal.fly();
  } else {
    animal.swim();
  }
}
```

在上面的示例中，我们定义了 `isBird` 谓词函数来判断参数 `animal` 是否属于 `Bird` 类型。在 `processAnimal` 函数中，通过使用自定义谓词函数 `isBird`，我们能够根据 `animal` 的具体类型执行相应的代码逻辑，并在代码块内部收窄 `animal` 的类型范围。

类型推断

TypeScript通过类型推断可以自动推导出变量和表达式的类型，提高代码的可读性和可维护性。

类型推断

类型推断是 TypeScript 在编译时根据上下文自动推导变量和表达式的类型。它根据变量的赋值、函数的返回值、表达式的操作等信息来确定变量或表达式的最佳类型。

1. 基础类型推断

TypeScript根据变量的初始值来推断基础类型，包括字符串、数字、布尔值等。

```
let name = "John"; // 推断为string类型
let age = 30; // 推断为number类型
let isStudent = true; // 推断为boolean类型
```

2. 最佳公共类型推断

当我们将不同类型的值赋给一个变量或数组时，TypeScript会根据这些值的类型推断出一个最佳公共类型。

```
let values = [1, 2, "three", true]; // 推断为(number | string | boolean)[]
```

在上面的示例中，数组 `values` 包含了数字、字符串和布尔值，TypeScript推断出这个数组的类型为 `(number | string | boolean)[]`，即联合类型。

3. 上下文类型推断

TypeScript会根据上下文中的预期类型推断变量的类型。这种上下文可以是函数参数、赋值语句等。

```
function greet(person: string) {  
  console.log(`Hello, ${person}!`);  
}  
  
greet("John"); // person的类型推断为string
```

在上面的示例中，函数 `greet` 的参数 `person` 的类型被推断为 `string`，因为在函数调用时传入的实参是一个字符串。

4. 类型断言

如果我们希望手动指定一个变量或表达式的类型，可以使用类型断言（Type Assertion）来告诉 TypeScript 我们的意图。

```
let value = "Hello, TypeScript!";  
let length = (value as string).length; // 类型断言为string
```

在上面的示例中，我们使用类型断言 `as string` 将变量 `value` 的类型指定为 `string`，以便在后面获取其长度时，TypeScript 能正确推断出类型。

5. 类型推断和泛型

在使用泛型时，TypeScript 会根据传入的参数类型推断泛型类型的具体类型。

```
function identity<T>(value: T): T {  
  return value;  
}  
  
let result = identity("Hello, TypeScript!"); // result的类型推断为string
```

在上面的示例中，泛型函数 `identity` 的参数 `value` 的类型被推断为传入的实参类型，因此返回值的类型也被推断为 `string`。

总结

类型推断是 TypeScript 中的一个重要特性，通过自动推导变量和表达式的类型，可以提高代码的可读性和可维护性。TypeScript 根据赋值、返回值、上下文等信息进行类型推断，并在需要时允许手动指定类型。在编写现代化高级 TypeScript 代码时，深入了解和应用类型推断是非常重要的部分。通过结合最新的 TypeScript 语法和类型推断，我们可以编写更具表达力和类型安全的代码。

类型系统层级

TypeScript 的类型系统是强类型和静态类型的，这为开发者提供了强大的类型检查和类型安全保障，同时也增加了一定的学习复杂性。为了更好地理解 TypeScript 的类型系统，本文将全面介绍其类型系统层级，包括顶层类型（Top Type）和底层类型（Bottom Type），以及在这个层次结构中如何处理和操作各种类型。理解 TypeScript 的类型系统层级有助于我们更好地使用和掌握 TypeScript，写出更健壮、可维护的代码。

1. 顶层类型 (Top Type)

顶层类型是所有其他类型的父类型，这意味着在 TypeScript 中的任何类型都可以看作是顶层类型的子类型。TypeScript 中有两个特殊的顶层类型：`any` 和 `unknown`。

1.1 any 类型

`any` 类型是 TypeScript 的一个逃生窗口，它可以接受任意类型的值，并且对 `any` 类型的值进行的任何操作都是允许的。使用 `any` 类型，可以使我们绕过 TypeScript 的类型检查。下面的例子展示了 `any` 类型的灵活性：

```
let a: any = 123; // OK
a = 'hello'; // OK
a = true; // OK
a = { id: 1, name: 'Tom' }; // OK

a.foo(); // OK
```

我们可以看到，我们可以将任何类型的值赋给 `any` 类型的变量，甚至可以对 `any` 类型的值进行我们想要的任何操作，而 TypeScript 编译器并不会对此做出任何投诉。然而，正是由于其超高的灵活性，使得 `any` 类型在一定程度上削弱了 TypeScript 的类型安全性，因此在我们编写 TypeScript 代码时，应尽量避免使用 `any` 类型。

1.2 unknown 类型

`unknown` 类型与 `any` 类型在接受任何类型的值这一点上是一样的，但 `unknown` 类型却不能像 `any` 类型那样对其进行任何操作。我们在对 `unknown` 类型的值进行操作之前，必须进行类型检查或类型断言，确保操作的安全性。

下面的例子展示了 `unknown` 类型的使用：

```
let u: unknown = 123; // OK
u = 'hello'; // OK
u = true; // OK
u = { id: 1, name: 'Tom' }; // OK

// Error: Object is of type 'unknown'.
// u.foo();

if (typeof u === 'object' && u !== null) {
  // OK after type check
  console.log((u as { id: number,
    name: string }).name);
}
```

在这个例子中，我们对 `unknown` 类型的值 `u` 进行了类型检查，然后通过类型断言安全地访问了其 `name` 属性。

2. 底层类型 (Bottom Type)

与顶层类型相对，底层类型是所有类型的子类型。这意味着，在类型系统的层次结构中，任何类型都可以被看作是底层类型的超类型。在 TypeScript 中，`never` 类型是唯一的底层类型。

`never` 类型用来表示永远不可能存在的值的类型。比如，一个永远抛出错误或者永远处于死循环的函数的返回类型就是 `never`。

```
function error(message: string): never {
  throw new Error(message);
}

function infiniteLoop(): never {
  while (true) {}
}
```

在上面的代码中，函数 `error` 和 `infiniteLoop` 的返回类型都是 `never`，这是因为这两个函数都永远不会有返回值。

3. 对比：顶层类型 vs 底层类型

顶层类型和底层类型是 TypeScript 类型系统的两个重要组成部分，它们各自扮演着不同的角色。

顶层类型 `any` 和 `unknown` 能够接受任何类型的值，这使得我们可以灵活地处理不确定类型的数据。然而，`any` 类型和 `unknown` 类型在使用上有着重要的区别：`any` 类型允许我们对其进行任何操作，而 `unknown` 类型则要求我们在操作之前进行类型检查或类型断言，以确保类型的安全性。

底层类型 `never` 有点特殊，它表示一个永远不会有值的类型。在实际开发中，我们可能很少直接使用 `never` 类型，但是它在 TypeScript 的类型推断和控制流分析中起着非常重要的作用。

理解 TypeScript 的类型系统层级有助于我们编写更健壮、可维护的 TypeScript 代码。尽管 `any` 类型提供了很大的灵活性，但是它的滥用可能会削弱 TypeScript 的类型安全性。因此，我们应尽量避免使用 `any` 类型，而优先使用 `unknown` 类型和类型断言、类型保护等方式来处理不确定类型的数据。同时，虽然我们可能很少直接使用 `never` 类型，但是理解它的含义和用法，对于我们理解 TypeScript 的类型推断和控制流分析也是非常有帮助的。

装饰器与反射元数据

介绍

在过去的几年中，JavaScript 及其生态系统发生了快速的变化。其中，TypeScript 已成为许多开发人员的首选语言。其主要优势在于其静态类型系统，它使我们可以在编译时捕获错误，而不是在运行时。除此之外，TypeScript 还为我们提供了许多 ES6+ 特性以及一些其他的独有特性，例如枚举、命名空间和装饰器。

装饰器简介

在 TypeScript 中，装饰器是一种特殊类型的声明，可以被附加到类声明，方法，属性，访问器或参数上。装饰器的核心思想是增强已经存在的类、方法、属性等的行为，或者添加新的行为。通过装饰器，我们可以在不改变原始类的定义的情况下，为类添加新的特性。

在 TypeScript 中，装饰器使用 `@expression` 的形式。其中，`expression` 必须为一个返回函数的表达式，这个函数在运行时会被调用，传入相关的装饰器参数。

TypeScript 支持以下几种类型的装饰器：

- 类装饰器
- 方法装饰器
- 访问器装饰器
- 属性装饰器
- 参数装饰器

类装饰器

类装饰器应用于类的构造函数，用于观察、修改或替换类定义。类装饰器在应用时，会作为函数调用，并将构造函数作为其唯一的参数。

```
function Sealed(constructor: Function) {
    Object.seal(constructor);
    Object.seal(constructor.prototype);
}

@Sealed
class Greeter {
    constructor(public greeting: string) {}
    greet() {
        return "Hello, " + this.greeting;
    }
}
```

方法装饰器

方法装饰器应用于方法的属性描述符，并可以用于观察、修改或替换方法定义。当装饰器被调用时，它会接收到三个参数：当前类的原型，方法名，以及该方法的属性描述符。

```
function Log(target: Object, propertyKey: string, descriptor: TypedPropertyDescriptor<any>) {
    let originalMethod = descriptor.value; // 保存原始函数
    descriptor.value = function (...args: any[]) {
        console.log("Arguments: ", JSON.stringify(args));
        let result = originalMethod.apply(this, args);

        console.log("Result: ", result);
        return result;
    }
}

class Calculator {
    @Log
    add(x: number, y: number): number {
        return x + y;
    }
}
```

访问器装饰器

访问器装饰器可以应用于访问器的属性描述符，并可以用于观察、修改或替换访问器的定义。访问器装饰器和方法装饰器有相似的语法。

```
function ReadOnly(target: any, key: string, descriptor: PropertyDescriptor) {
  descriptor.writable = false;
  return descriptor;
}

class Circle {
  private _radius: number;

  constructor(radius: number) {
    this._radius = radius;
  }

  @ReadOnly
  get radius() {
    return this._radius;
  }
}
```

装饰器与反射元数据

为了让装饰器能够更好地工作，TypeScript 提供了反射元数据 API。这是一个实验性的 API，它允许装饰器在声明时添加元数据。可以使用 npm 来安装反射元数据 API：

“

反射元数据（Reflect Metadata）是一个实验性的 API，用于在声明装饰器时执行元数据类型注解和元数据反射。

”

```
npm install reflect-metadata --save
```

然后，你需要在全局范围内导入反射 API：

```
import "reflect-metadata";
```

在 TypeScript 配置文件 `tsconfig.json` 中，你需要开启 `emitDecoratorMetadata` 选项：

```
{
  "compilerOptions": {
    "emitDecoratorMetadata": true,
    "experimentalDecorators": true,
    "target": "ES5",
    "module": "commonjs"
  }
}
```

然后你就可以在装饰器中使用反射 API 了：

```
function logType(target : any, key : string) {
  var t = Reflect.getMetadata("design:type", target, key);
  console.log(` ${key} type: ${t.name}`);
}

class Demo{
  @logType
  public attr1 : string;
}
```

以上就是装饰器的基本原理与应用。装饰器可以使我们的代码更简洁，更易读，也使得我们的代码更易于管理和维护。但是，需要注意的是，装饰器目前还处于实验阶段，如果你决定在生产环境中使用装饰器，你需要了解使用装饰器可能带来的风险。

结论

TypeScript的装饰器为我们提供了一种强大的工具，可以在运行时改变类的行为。通过理解装饰器的工作原理，我们可以创造更加强大、灵活且易于维护的应用。

解读TSConfig

TypeScript 配置文件（tsconfig.json）是用于配置 TypeScript 项目的重要文件。它允许开发者自定义 TypeScript 编译器的行为，指定编译选项、文件包含与排除规则、输出目录等。通过合理配置 tsconfig.json，我们可以根据项目需求进行灵活的 TypeScript 编译设置。

本文将全面解读 tsconfig.json 的各个配置选项，并提供一些常见的使用场景和示例代码，以及封装定制化自己 tsconfig.base 配置

创建和基本配置

要使用 TypeScript 配置文件，我们首先需要创建一个名为 `tsconfig.json` 的文件，并将其放置在项目的根目录下。

下面是一个基本的 tsconfig.json 配置示例：

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "commonjs",
    "outDir": "dist"
  },
  "include": [
    "src/**/*.ts"
  ],
  "exclude": [
    "node_modules",
    "dist"
  ]
}
```

在上述示例中，我们使用 `compilerOptions` 配置选项指定了 TypeScript 编译器的选项。其中：

- `"target": "es6"` 指定编译的目标 JavaScript 版本为 ES6。

- `"module": "commonjs"` 指定模块的生成方式为 CommonJS。
- `"outDir": "dist"` 指定输出目录为 "dist"。

同时，我们使用 `include` 和 `exclude` 配置选项分别指定了需要编译的源文件的包含规则和排除规则。

compilerOptions

`compilerOptions` 是 `tsconfig.json` 中最重要的配置选项之一，它允许我们指定 TypeScript 编译器的各种行为和设置。以下是一些常用的 `compilerOptions` 配置选项：

target

`target` 选项指定了编译后的 JavaScript 代码所要遵循的 ECMAScript 标准。常见的选项包括 `"es5"`、`"es6"`、`"es2015"`、`"es2016"` 等。

```
"compilerOptions": {  
  "target": "es6"  
}
```

module

`module` 选项用于指定生成的模块化代码的模块系统。常见的选项包括 `"commonjs"`、`"amd"`、`"es2015"`、`"system"` 等。

```
"compilerOptions": {  
  "module": "commonjs"  
}
```

outDir

`outDir` 选项指定了编译输出的目录路径。

```
"compilerOptions": {  
  "outDir": "dist"  
}
```

strict

`strict` 选项用于启用严格的类型检查和更严格的编码规范。

```
"compilerOptions": {  
  "strict": true  
}
```

lib

`lib` 选项用于指定 TypeScript 编译器可以使用的 JavaScript 标准库的列表。默认情况下，

TypeScript 编译器会根据目标版本自动选择合适的库。

```
"compilerOptions": {  
  "lib": ["es6", "dom"]  
}
```

sourceMap

sourceMap 选项用于生成与源代码对应的源映射文件（.map 文件），以便在调试过程中可以将编译后的 JavaScript 代码映射回原始 TypeScript 代码。

```
"compilerOptions": {  
  "sourceMap": true  
}
```

paths

paths 选项用于配置模块解析时的路径映射，可以帮助我们简化模块导入的路径。

```
"compilerOptions": {  
  "paths": {  
    "@/*": ["src/*"]  
  }  
}
```

allowJs

allowJs 选项允许在 TypeScript 项目中引入 JavaScript 文件，使得我们可以混合使用 TypeScript 和 JavaScript。

```
"compilerOptions": {  
  "allowJs": true  
}
```

esModuleInterop 和 allowSyntheticDefaultImports

esModuleInterop 属性用于提供对 ES 模块的兼容性支持。当我们在 TypeScript 项目中引入 CommonJS 模块时，可以通过设置 **esModuleInterop** 为 **true** 来避免引入时的错误。


```
{
  "compilerOptions": {
    "target": "es5",
    "module": "commonjs",
    "lib": ["es6", "dom"],
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true
  }
}
```

在上面的示例中，我们设置了 `esModuleInterop` 和 `allowSyntheticDefaultImports` 属性为 `true`，以支持对 ES 模块的兼容性导入。

更多的 `compilerOptions` 配置选项可以参考 TypeScript 官方文档：[Compiler Options \(https://www.typescriptlang.org/tsconfig#compiler-options\)](https://www.typescriptlang.org/tsconfig#compiler-options)。

include 和 exclude

`include` 和 `exclude` 配置选项用于指定哪些文件应该包含在编译过程中，以及哪些文件应该排除在编译过程之外。

`include` 是一个文件或者文件夹的数组，用于指定需要编译的文件或文件夹的路径模式。

```
"include": [
  "src/**/*.ts",
  "test/**/*.ts"
]
```

`exclude` 是一个文件或者文件夹的数组，用于指定需要排除的文件或文件夹的路径模式。

```
"exclude": [
  "node_modules",
  "dist"
]
```

在上述示例中，我们将 `src` 文件夹和 `test` 文件夹下的所有 TypeScript 文件包含在编译过程中，并排除了 `node_modules` 文件夹和 `dist` 文件夹。

文件引用和 composite

`files` 配置选项允许我们显式列出需要编译的文件路径。

```
"files": [  
  "src/main.ts",  
  "src/utils.ts"  
]
```

`composite` 配置选项用于启用 TypeScript 的项目引用功能，允许我们将一个 TypeScript 项目作为另一个项目的依赖。

```
"composite": true
```

declaration

`declaration` 配置选项用于生成声明文件（.d.ts 文件），它们包含了编译后 JavaScript 代码的类型信息。

```
"declaration": true
```

tsconfig.json 继承

TypeScript 支持通过 `extends` 配置选项从其他的 `tsconfig.json` 文件中继承配置。

```
{  
  "extends": ".tsconfig.base.json",  
  "compilerOptions": {  
    "outDir": "dist"  
  },  
  "include": [  
    "src/**/*.ts"  
  ]  
}
```

在上述示例中，我们通过 `extends` 指定了一个基础配置文件 `tsconfig.base.json`，然后在当前的 `tsconfig.json` 中添加了额外的编译选项和文件包含规则。

定制化tsconfig.base

制化`tsconfig.base`可以让我们在多个项目中共享和复用配置，提高开发效率。下面是一些步骤来封装自己的 `TSConfig` 为一个库：

首先，我们需要创建一个新的 TypeScript 项目作为我们的库项目。可以使用以下命令初始化一个新的项目：

```
$ mkdir my-tsconfig-lib  
$ cd my-tsconfig-lib  
$ npm init -y
```

2. 创建 tsconfig.json 文件

在项目根目录下创建一个名为 `tsconfig.json` 的文件，并将 `TSCConfig` 的配置内容添加到其中。

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "commonjs",
    "outDir": "dist"
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

这是一个示例的 `TSCConfig` 配置，你可以根据自己的需求进行相应的修改。

3. 创建包入口文件

为了能够在其他项目中使用我们的库，我们需要创建一个入口文件来导出我们的 `TSCConfig`。

在项目根目录下创建一个名为 `index.ts` 的文件，并添加以下代码：

```
import * as tsconfig from './tsconfig.json';

export default tsconfig;
```

在上述代码中，我们将 `tsconfig.json` 导入为一个模块，并使用 `export default` 将其导出。

4. 构建和发布

现在我们可以使用 `TypeScript` 编译器将我们的代码构建为 `JavaScript`，以便在其他项目中使用。

首先，确保你已经在项目中安装了 `TypeScript`：

```
$ npm install typescript --save-dev
```

然后，在 `package.json` 中添加构建脚本：

```
{
  "scripts": {
    "build": "tsc"
  }
}
```

最后，运行构建命令进行构建：

```
$ npm run build
```

构建完成后，我们的库文件将位于 `dist` 目录下。

5. 发布到 NPM

要将我们的 TSConfig 封装为一个库，并使其可供其他项目使用，我们可以将其发布到 NPM。
首先，创建一个 NPM 账号，并登录到 NPM：

```
$ npm login
```

然后，在项目根目录下运行以下命令发布库：

```
$ npm publish
```

发布成功后，我们的 TSConfig 库就可以在其他项目中使用。

6. 在其他项目中使用

在其他项目中使用我们的 TSConfig 库非常简单。首先，在目标项目中安装我们的库：

```
$ npm install my-tsconfig-lib --save-dev
```

然后，在目标项目的 `tsconfig.json` 文件中使用我们的 TSConfig：

```
{
  "extends": "my-tsconfig-lib"
}
```

通过 `extends` 配置选项，我们可以继承和使用我们的 TSConfig。

总结

通过 `tsconfig.json` 文件，我们可以配置 TypeScript 编译器的行为，包括编译选项、文件包含与排除规则、输出目录等。合理配置 `tsconfig.json` 可以帮助我们根据项目进行灵活的 TypeScript 编译设置。

详细的 TypeScript 配置文件的参考信息可以在 TypeScript 官方文档中找到：[tsconfig.json](https://www.typescriptlang.org/tsconfig)
(<https://www.typescriptlang.org/tsconfig>)

面试官：说说你对TypeScript的认识？

TypeScript是一种由微软开发的开源编程语言，它是JavaScript的超集，意味着所有JavaScript代码都是合法的TypeScript代码。TypeScript添加了静态类型检查和一些新的特性，以帮助开发者更好地组织和维护大型的JavaScript应用程序。

我对TypeScript的认识可以总结为以下几点：

1. **「静态类型系统：」** TypeScript引入了静态类型检查，允许在编码阶段捕获潜在的错误，例如类型不匹配、未定义的变量等。这可以大大提高代码质量和可维护性，减少在运行时出现的错误。
2. **「类型注解：」** TypeScript允许开发者为变量、函数参数、返回值等添加类型注解，帮助编译器进行类型检查。通过类型注解，我们可以清楚地了解代码中数据的类型，提高代码可读性。
3. **「类和接口：」** TypeScript支持面向对象编程，可以使用类和接口来创建对象、继承和实现接口。这使得代码的结构更加清晰，易于维护。
4. **「泛型：」** TypeScript支持泛型编程，允许我们编写可以适用于多种类型的代码。泛型提高了代码的重用性和灵活性。

5. 「ES6+支持:」 TypeScript是JavaScript的超集，支持ES6+的语法和特性。我们可以在TypeScript中使用箭头函数、解构赋值、Promise等新特性，提高开发效率。
6. 「工具支持:」 TypeScript具有强大的开发工具支持，如代码自动补全、错误提示、重构等。IDE和编辑器可以根据类型信息提供更好的代码提示和错误检查，帮助开发者更快地编写高质量的代码。
7. 「逐步迁移:」 由于TypeScript是JavaScript的超集，我们可以逐步将现有的JavaScript项目迁移到TypeScript，不需要一次性重写整个代码。这为大型项目的逐步升级提供了便利。

总的来说，TypeScript是一个强大且灵活的编程语言，通过引入静态类型检查和其他新特性，它帮助我们在大型项目中编写更安全、更健壮的代码，并提高了开发效率和可维护性。它适用于任何规模的项目，并且在现代前端开发中越来越受欢迎。

高级类型

映射类型 (Mapped Types)

映射类型 (Mapped Types) 是 TypeScript 中一种强大的类型操作工具，它允许我们在编译时转换已知类型的属性，并创建一个新的类型。通过映射类型，我们可以对已有类型的属性进行转换、修改或添加新的属性。这在许多情况下都非常有用，例如将属性变为只读或可选，从现有属性中选择一部分属性等。

映射类型的语法形式为：

```
type NewType = {  
  [Property in keyof ExistingType]: TransformType;  
};
```

其中，`NewType` 是我们要创建的新类型，`Property` 是 `ExistingType` 的键，`TransformType` 是对应属性的转换类型。

下面是一些常见的映射类型的示例：

1. Readonly

`Readonly` 是 TypeScript 内置的一个映射类型，它将给定类型的所有属性变为只读。

```
type Readonly<T> = {  
  readonly [P in keyof T]: T[P];  
};
```

示例使用：


```
interface Person {
  name: string;
  age: number;
}

type ReadonlyPerson = Readonly<Person>;

const person: ReadonlyPerson = {
  name: "John",
  age: 30,
};

person.name = "Alice"; // Error: Cannot assign to 'name' because it is a read-only property.
```

2. Partial

Partial 是另一个内置的映射类型，它将给定类型的所有属性变为可选。

```
type Partial<T> = {
  [P in keyof T]?: T[P];
};
```

示例使用：

```
interface Person {
  name: string;
  age: number;
}

type PartialPerson = Partial<Person>;

const person: PartialPerson = {
  name: "John",
};

person.age = 30; // Valid: age is optional
```

3. Pick

Pick 是一个映射类型，它从给定类型中选择一部分属性来创建新类型。

```
type Pick<T, K extends keyof T> = {
  [P in K]: T[P];
};
```

示例使用：

```
interface Person {
  name: string;
  age: number;
  occupation: string;
}

type PersonInfo = Pick<Person, "name" | "age">;

const info: PersonInfo = {
  name: "John",
  age: 30,
};
```

4. Record

Record 是一个映射类型，它根据指定的键类型和值类型创建一个新的对象类型。

```
type Record<K extends keyof any, T> = {
  [P in K]: T;
};
```

示例使用：

```
type Weekday = "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday";

type WorkingHours = Record<Weekday, string>;

const hours: WorkingHours = {
  Monday: "9am-6pm",
  Tuesday: "9am-6pm",
  Wednesday: "9am-6pm",
  Thursday: "9am-6pm",
  Friday: "9am-5pm",
};
```

条件类型（Conditional Types）

它允许我们根据类型的条件判断结果来选择不同的类型。条件类型的语法形式为：

```
T extends U ? X : Y
```

其中，T 是待检查的类型，U 是条件类型，X 是满足条件时返回的类型，Y 是不满足条件时返回的类型。条件类型通常与泛型一起使用，以便根据不同的类型参数值进行类型推断和转换。

条件类型与infer

当我们在 TypeScript 中使用条件类型时，有时候我们希望从某个类型中提取出一个部分类型并进行推断。这时就可以使用 **infer** 关键字。

`infer` 关键字用于声明一个类型变量，在条件类型中表示待推断的部分类型。它通常在条件类型的分支中使用，以便从给定类型中提取和推断出某些信息。

下面是一个示例，展示了如何使用 `infer` 关键字：

```
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function add(a: number, b: number): number {
  return a + b;
}

type AddReturnValue = ReturnType<typeof add>; // 推断为number类型
```

在上面的示例中，我们定义了一个条件类型 `ReturnType<T>`，它接受一个函数类型 `T` 作为输入。当 `T` 是一个函数类型时，我们使用 `infer R` 声明一个类型变量 `R` 来推断函数的返回类型，并将其作为结果返回。

通过调用 `ReturnType<typeof add>`，我们将函数 `add` 的类型传递给 `ReturnType<T>`，从而提取并推断出其返回类型。结果 `AddReturnValue` 的类型被推断为 `number`，因为 `add` 函数返回一个数字。

`infer` 关键字的作用是告诉 TypeScript 编译器在条件类型中推断一个待定的类型，并将其赋值给声明的类型变量。这使得我们可以在条件类型中使用这个推断出的类型进行进一步的类型操作。

需要注意的是，`infer` 关键字只能在条件类型的右侧使用，用于声明一个待推断的类型变量，而不能在其他地方使用。此外，每个条件类型只能使用一次 `infer` 关键字，并且通常与泛型一起使用。

`infer` 关键字是 TypeScript 中用于提取并推断待定类型的工具。它允许我们在条件类型中声明一个类型变量，用于在类型推断过程中捕获和使用待推断的类型，从而使类型系统更加灵活和强大。

模板字面量类型（Template Literal Types）

模板字面量类型（Template Literal Types）是 TypeScript 4.1 引入的新特性，它允许我们在类型级别上操作字符串字面量类型。通过使用模板字面量类型，我们可以创建基于字符串模板的复杂类型。

下面是一个使用模板字面量类型的示例：

```
type Greeting<T extends string> = `Hello, ${T}!`;

type GreetingWorld = Greeting<'World'>; // GreetingWorld的类型为"Hello, World!"
```

在上面的示例中，我们定义了一个模板字面量类型 `Greeting<T>`，它接受一个字符串类型参数 `T`，并使用字符串模板将其包装在 `Hello,` 和 `!` 之间。通过使用 `Greeting<'World'>`，我们可以将字符串字面量类型 `'World'` 传递给模板字面量类型，从而创建一个具体的类型 `GreetingWorld`，它的类型被推断为 `"Hello, World!"`。

模板字面量类型还支持模板字符串的拼接、条件语句、循环等操作，使得我们可以在类型级别上创建更加动态和复杂的类型。

```
type Pluralize<T extends string> = `${T}s`;
type Message<T extends boolean> = T extends true ? 'Enabled' : 'Disabled';

type Plural = Pluralize<'apple'>; // Plural的类型为"apples"
type EnabledMessage = Message<true>; // EnabledMessage的类型为'Enabled'
```

在上面的示例中，我们定义了两个模板字面量类型， `Pluralize<T>` 用于将字符串类型 `T` 转换为其复数形式， `Message<T>` 用于根据布尔类型参数 `T` 返回不同的消息。

面试官：说说flexbox（弹性盒布局模型）

Flex布局(Flexible Box Layout)是一种用于定义容器内部元素布局方式的CSS3布局模型。它可以让元素具有弹性,让元素在容器内自动排列,简化了元素的定位。

Flex布局的基本概念

采用Flex布局的元素可分为容器(container)和项目(item)。

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

容器通过设置 `display: flex` 生成,项目默认沿主轴排列。

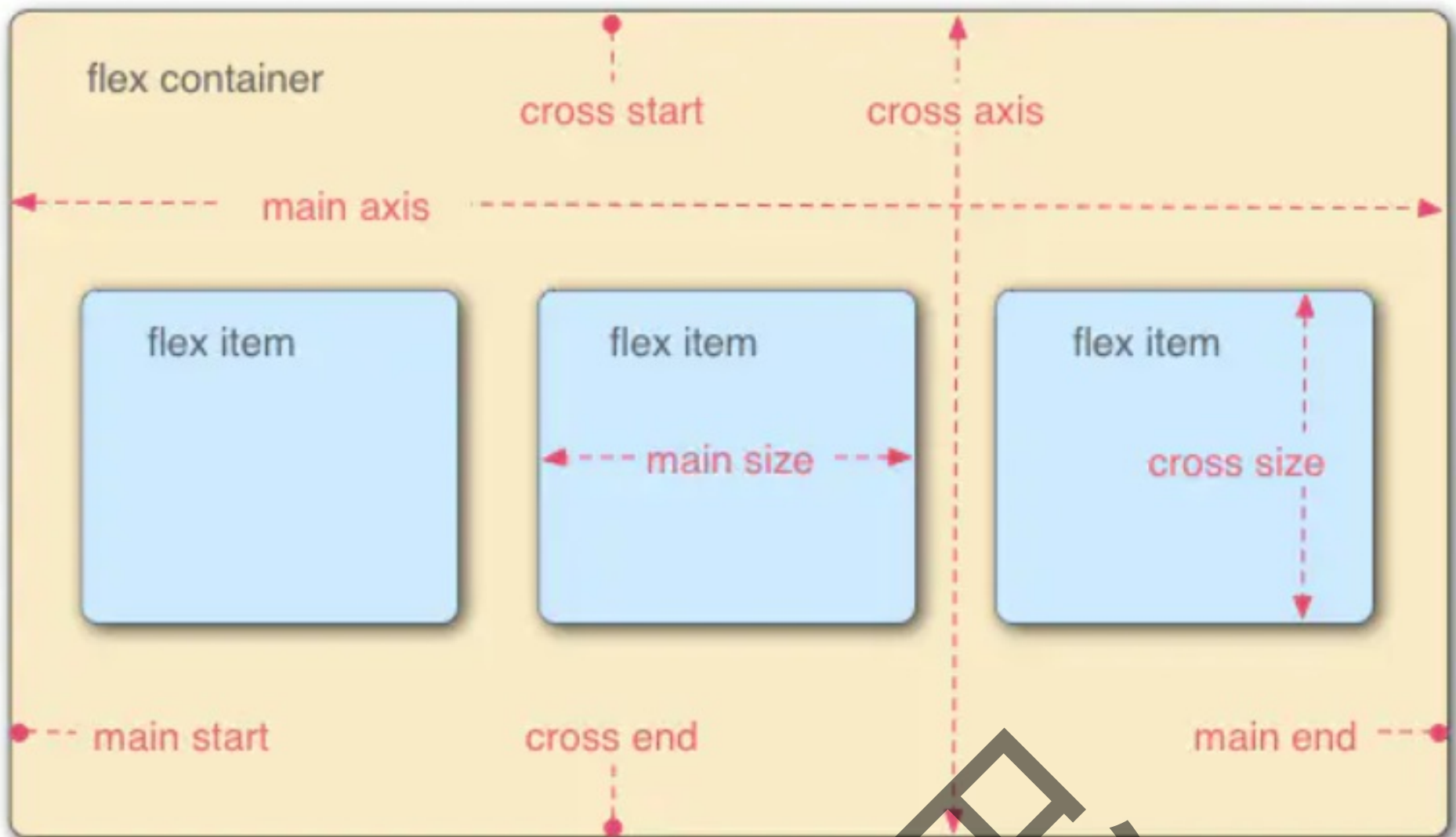
容器属性

flex-direction

决定主轴方向,即项目排列方向。

```
.container {
  flex-direction: row | row-reverse | column | column-reverse;
}
```

- row: 主轴为水平,起点在左端
- row-reverse: 主轴为水平,起点在右端
- column: 主轴为垂直,起点在上沿
- column-reverse: 主轴为垂直,起点在下沿



flex-wrap

定义容器内项目是否可换行。

```
.container{  
  flex-wrap: nowrap | wrap | wrap-reverse;  
}
```

- nowrap: 不换行
- wrap: 换行, 第一行在下方
- wrap-reverse: 换行, 第一行在上方

justify-content

定义项目在主轴上的对齐方式。

```
.container {  
  justify-content: flex-start | flex-end | center | space-between | space-around;  
}
```

- flex-start: 左对齐
- flex-end: 右对齐
- center: 居中
- space-between: 两端对齐, 项目之间间隔相等
- space-around: 每个项目两侧间隔相等

flex-start



flex-end



center



space-between



space-around

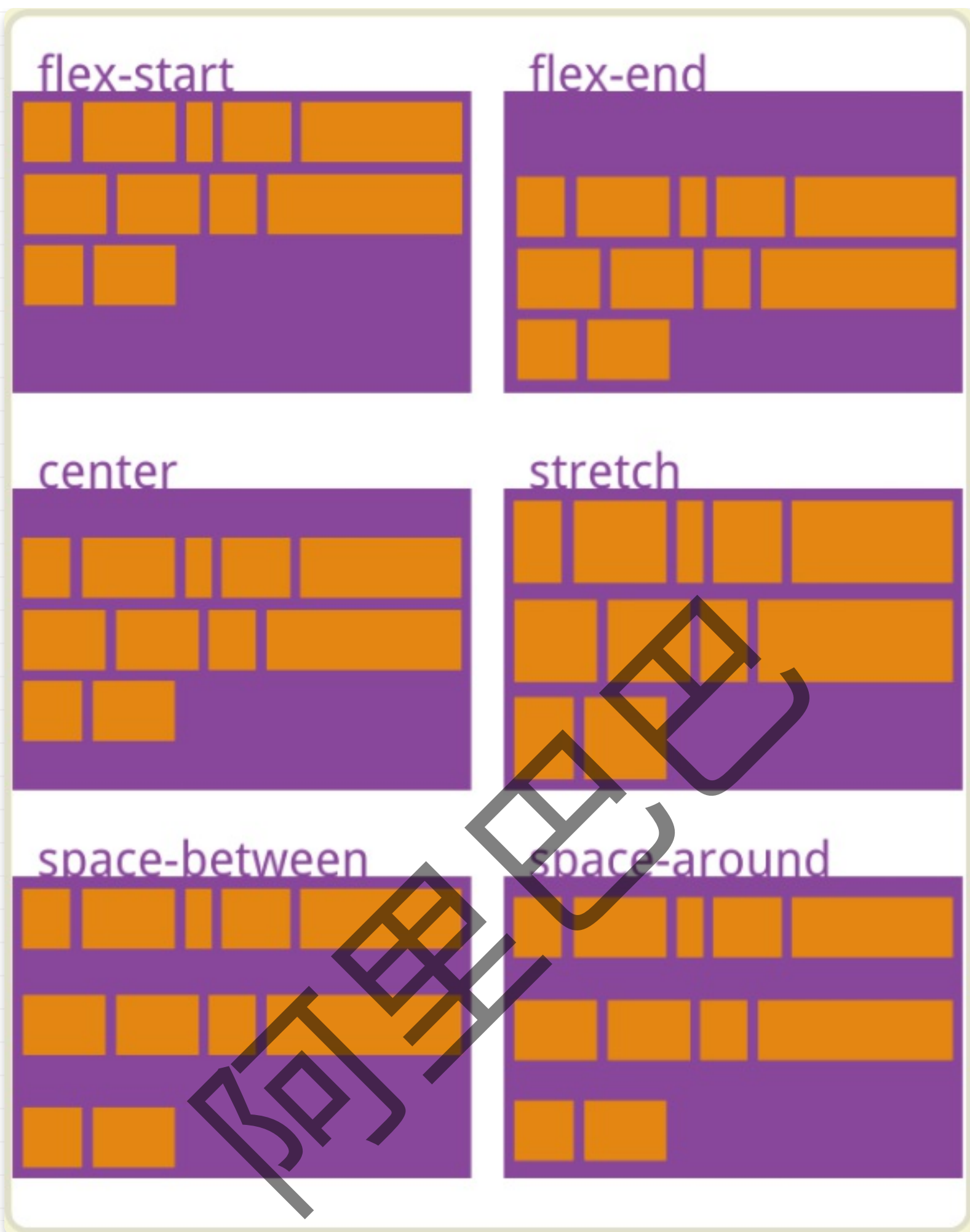


align-items

定义项目在交叉轴上如何对齐。

```
.container {  
  align-items: flex-start | flex-end | center | baseline | stretch;  
}
```

- flex-start: 交叉轴起点对齐
- flex-end: 交叉轴终点对齐
- center: 交叉轴中点对齐
- baseline: 项目第一行文字基线对齐
- stretch: 项目高度撑满容器



项目属性

order

定义项目的排列顺序。

```
.item {  
  order: <integer>;  
}
```

数值越小,排列越靠前。

flex-grow

定义项目的放大比例。

```
.item {
  flex-grow: <number>; /* default 0 */
}
```

如果所有项目都为1,则它们将等分剩余空间。

flex-shrink

定义了项目的缩小比例。

```
.item {
  flex-shrink: <number>; /* default 1 */
}
```

如果空间不足,该项目将按比例缩小。

align-self

单个项目与其他项目对齐方式不同。

```
.item {
  align-self: auto | flex-start | flex-end | center | baseline | stretch;
}
```

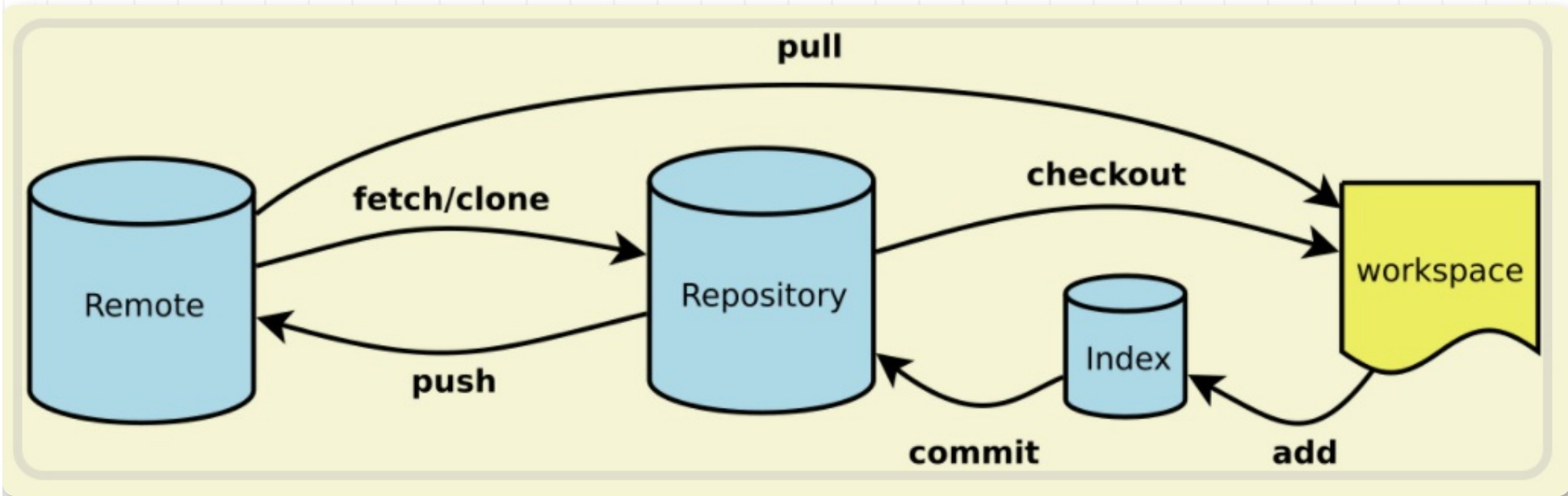
总结

Flex布局提供了强大的容器属性,可以方便地控制项目布局、对齐和顺序,是实现响应式布局的首选方式。

面试官：说说Git常用的命令有哪些？

一、前言

在日常的Git操作中，我们通常只需要掌握一小部分简单的命令，如下图所示的六个命令即可：



然而，要熟练使用Git，了解超过60多个命令将是非常有帮助的。接下来，我们将介绍一些常见的Git命令供参考。

二、常用的Git命令

1. 配置

Git提供了一个名为 `git config` 的工具，用于设置控制Git外观和行为的配置变量。安装完Git后，第一件事就是设置用户名和邮件地址，这些信息会被写入每次的提交中，且不可更改。

设置提交代码时的用户信息命令如下：

- `git config [--global] user.name "[name]"`
- `git config [--global] user.email "[email address]"`

2. 启动

一个Git项目可以通过两种方式初始化：

- `git init [project-name]`：在当前目录创建或初始化一个Git代码库。
- `git clone url`：克隆一个项目及其整个代码历史。

3. 日常基本操作

在日常工作中，常用的基本操作如下：

```
git init                # 初始化仓库，默认为 master 分支
git add .               # 提交全部文件修改到缓存区
git add <具体某个文件路径+全名> # 提交某些文件到缓存区
git diff                # 查看当前代码 add 后，会 add 哪些内容
git diff --staged       # 查看现在 commit 提交后，会提交哪些内容
git status              # 查看当前分支状态
git pull <远程仓库名> <远程分支名> # 拉取远程仓库的分支与本地当前分支合并
git pull <远程仓库名> <远程分支名>:<本地分支名> # 拉取远程仓库的分支与本地某个分支合并
git commit -m "<注释>"  # 提交代码到本地仓库，并写提交注释
git commit -v           # 提交时显示所有diff信息
git commit --amend [file1] [file2] # 重做上一次commit，并包括指定文件的新变化
```

关于提交信息的格式，可以遵循以下规则：

- `feat`：新特性，添加功能
- `fix`：修改bug
- `refactor`：代码重构
- `docs`：文档修改
- `style`：代码格式修改，注意不是CSS修改
- `test`：测试用例修改
- `chore`：其他修改，比如构建流程，依赖管理

4. 分支操作

常用的分支操作如下：

```
git branch          # 查看本地所有分支
git branch -r       # 查看远程所有分支
git branch -a       # 查看本地和远程所有分支
git merge <分支名>   # 合并分支
git merge --abort    # 合并分支出现冲突时，取消合并，回到合并前的状态
git branch <新分支名> # 基于当前分支，新建一个分支
git checkout --orphan <新分支名> # 新建一个空分支（会保留之前分支的所有文件）
git branch -D <分支名> # 删除本地某个分支
git push <远程库名> :<分支名> # 删除远程某个分支
git branch <新分支名称> <提交ID> # 从提交历史恢复某个删除的分支
git branch -m <原分支名> <新分支名> # 分支更名
git checkout <分支名> # 切换到本地某个分支
git checkout <远程库名>/<分支名> # 切换到线上某个分支
git checkout -b <新分支名> # 基于当前分支新建分支，并切换为这个分支
```

5. 远程同步

常见的远程操作命令：

```
git fetch [remote]    # 下载远程仓库的所有变动
git remote -v         # 显示所有远程仓库
git pull [remote] [branch] # 拉取远程仓库的分支与本地当前分支合并
git fetch             # 获取线上最新版信息记录，不合并
git push [remote] [branch] # 上传本地指定分支到远程仓库
git push [remote] --force # 强行推送当前分支到远程仓库，即使有冲突
git push [remote] --all  # 推送所有分支到远程仓库
```

6. 撤销

```
git checkout [file]          # 恢复暂存区的指定文件到工作区
git checkout [commit] [file] # 恢复某个commit的指定文件到暂存区和工作区
git checkout .               # 恢复暂存区的所有文件到工作区
git reset [commit]           # 重置当前分支的指针为指定commit，同时重置暂存区，但工作区不变
git reset --hard             # 重置暂存区与工作区，与上一次commit保持一致
git reset [file]             # 重置暂存区的指定文件，与上一次commit保持一致，但工作区不
```

变

```
git revert [commit]          # 后者的所有变化都将被前者抵消，并应用到当前分支
```

“

注意：`reset`是真实硬性回滚，目标版本后面的提交记录全部丢失了；`revert`也是回滚操作，但这个回滚操作相当于一个新的提交，目标版本后面的提交记录也全部被保留。

”

7. 存储操作

当你正在进行项目中某一部分的工作，里面的东西处于一个比较杂乱的状态，而你想转到其他分支上进行一些工作，但又不想提交这些杂乱的代码，这时候可以将代码进行存储。


```
git stash           # 暂时将未提交的变化移除
git stash pop       # 取出储藏中最后存入的工作状态进行恢复，并删除储藏
git stash list      # 查看所有储藏中的工作
git stash apply <储藏的名称> # 取出储藏中对应的工作状态进行恢复，不会删除储藏
git stash clear     # 清空所有储藏中的工作
git stash drop <储藏的名称> # 删除对应的某个储藏
```

三、总结

以下是Git常用命令速查表：

Git 常用命令速查表

master :默认开发分支
origin :默认远程版本库

Head :默认开发分支
Head^ :Head 的父提交

创建版本库

```
$ git clone <url>           #克隆远程版本库
$ git init                  #初始化本地版本库
```

修改和提交

```
$ git status                #查看状态
$ git diff                  #查看变更内容
$ git add .                 #跟踪所有改动过的文件
$ git add <file>            #跟踪指定的文件
$ git mv <old> <new>        #文件改名
$ git rm <file>             #删除文件
$ git rm --cached <file>    #停止跟踪文件但不删除
$ git commit -m "commit message"
                               #提交所有更新过的文件
$ git commit --amend        #修改最后一次提交
```

查看提交历史

```
$ git log                   #查看提交历史
$ git log -p <file>         #查看指定文件的提交历史
$ git blame <file>          #以列表方式查看指定文件的提交历史
```

撤销

```
$ git reset --hard HEAD     #撤销工作目录中所有未提交文件的修改内容
$ git checkout HEAD <file>  #撤销指定的未提交文件的修改内容
$ git revert <commit>        #撤销指定的提交
```

分支与标签

```
$ git branch                #显示所有本地分支
$ git checkout <branch/tag> #切换到指定分支或标签
$ git branch <new-branch>   #创建新分支
$ git branch -d <branch>    #删除本地分支
$ git tag                   #列出所有本地标签
$ git tag <tagname>          #基于最新提交创建标签
$ git tag -d <tagname>      #删除标签
```

合并与衍合

```
$ git merge <branch>        #合并指定分支到当前分支
$ git rebase <branch>       #衍合指定分支到当前分支
```

远程操作

```
$ git remote -v             #查看远程版本库信息
$ git remote show <remote>  #查看指定远程版本库信息
$ git remote add <remote> <url>
                               #添加远程版本库
$ git fetch <remote>        #从远程库获取代码
$ git pull <remote> <branch> #下载代码及快速合并
$ git push <remote> <branch> #上传代码及快速合并
$ git push <remote> :<branch/tag-name>
                               #删除远程分支或标签
$ git push --tags           #上传所有标签
```

参考文献

- <https://www.ruanyifeng.com/blog/2015/12/git-cheat-sheet.html>
(<https://www.ruanyifeng.com/blog/2015/12/git-cheat-sheet.html>)
- <https://segmentfault.com/a/1190000017875714>
(<https://segmentfault.com/a/1190000017875714>)

面试官：探索现代图片格式：从GIF到HEIF，优势与适用场景一览

点击在线阅读，体验更好 (https://www.coding-time.cn)	链接 (https://www.coding-time.cn)
「现代JavaScript高级小册 (https://www.coding-time.cn)」	链接 (https://www.coding-time.cn)
「深入浅出Dart (https://www.coding-time.cn)」	链接 (https://www.coding-time.cn)
「现代TypeScript高级小册 (https://www.coding-time.cn)」	链接 (https://www.coding-time.cn)

引言

最近在掘金看到很多动图头像，但都失真了，好奇就研究了下前端各种图片格式



视觉属性

“

图片的透明性和色彩丰富程度是两个重要的视觉属性，对于图像的呈现和使用都有着关键的影响。

”

「色彩 (Color)」：

- 色彩丰富程度是指图像中可以显示的颜色数量。它通常以位 (bit) 为单位表示，例如8位色彩意味着可以显示 $2^8 = 256$ 种颜色。更高的色彩丰富程度意味着图像可以呈现更多的颜色细节和平滑的渐变效果。对于照片、图形和图像处理等领域，较高的色彩丰富程度非常重要，因为它可以确保图像在不同设备上显示更真实和细腻。
- 常见的色彩丰富程度有8位 (256色)、16位 (65,536色)、24位 (约1600万色) 和32位 (约1677万色) 等。关注图片的透明性和色彩丰富程度对于选择正确的图像格式以及在设计和处理图像时都至关重要。对于需要有透明效果或需要在不同背景上使用的图像，选择支持透明性的格式 (如PNG或GIF) 是很重要的。而对于需要更高质量和更真实颜色表现的图像，选择具有较高色彩丰富程度的格式 (如24位或32位的PNG或JPEG) 是明智的选择。

8位色

每个像素所能显示的彩色数为2的8次方，即256种颜色。这种彩色深度适用于较古老的显示设备和简单的图像场景。它在色彩表现方面相对较弱，颜色过渡可能显得不够平滑，导致图像呈现出颗粒感，不适合表现细腻的色彩变化。

16位增强色

每个像素所能显示的彩色数为2的16次方，即65536种颜色。这种彩色深度相对于8位色有了明显的提升，能够更好地表现色彩细节和平滑的过渡效果。它适用于一些对色彩表现要求较高的场景，但仍然有一定的局限性。

24位真彩色

每个像素所能显示的彩色数为24位，即2的24次方，约为1680万种颜色。真彩色是最常见的彩色深度标准，它具备较好的色彩表现能力，可以呈现丰富的色彩细节和平滑的过渡效果，适用于绝大多数图像场景。

32位真彩色

32位真彩色在24位真彩色的基础上再增加了一个表示图像透明度信息的Alpha通道，通常称为32位色。这使得图像可以支持半透明效果，使得图像在叠加时能够更好地融合。32位真彩色并非是2的32次方的色数，它实际上也是约为1680万种颜色，但因为增加了Alpha通道，为了方便称呼，就规定它为32位色。

不同彩色深度的选择取决于图像的具体需求和展示平台的支持能力。较为复杂的图像和对色彩表现要求高的场景通常会选择较高的彩色深度，而简单的图像或者需要考虑性能的场景可能会选择较低的彩色深度。

「对比度（Contrast）」：

- 图像中不同颜色或亮度之间的差异程度，影响图像的明暗分界
- 高对比度和低对比度的区别，对图像整体视觉效果的影响

「亮度（Brightness）」：

- 图像的整体明暗程度，决定图像的整体明亮度
- 明亮图像和较暗图像的效果，影响图像的视觉表现和情感传递

「清晰度（Sharpness）」：

- 图像中细节的清晰程度，关系到图像的清晰度和细节表现
- 清晰度对图像质量的影响，影响图像的真实感和清晰感

「透明性（Transparency）」：

- 图片的透明性决定了图像中的哪些部分是透明的，即允许背景或下层图像透过。有透明性的图像可以让您将图像放置在其他背景上，形成无缝融合的效果。这在图标、徽标和图形设计中非常常见。
- 常见的支持透明性的图像格式是PNG和GIF，它们允许指定图像的某些区域为透明。例如，一个PNG图像可以有部分是不透明的，部分是透明的。

「色调（Hue）」：

- 图像中不同色彩的种类，涉及不同颜色的命名和区分
- 色调对图像氛围和情绪的影响，色调的选择传递特定的感觉和意境

「饱和度（Saturation）」：

- 图像中颜色的纯度和鲜艳程度，决定颜色的饱满度
- 高饱和度和低饱和度的视觉效果，影响图像的色彩感受和视觉强度

「纹理（Texture）」：

- 图像中表面的质感或细节，包括粗糙、光滑、木纹等
- 纹理在艺术创作中的应用，可以增加图像的趣味和观赏性

「形状（Shape）」：

- 图像中物体或对象的外形特征，涉及基本几何形状和轮廓
- 形状在视觉感知和图像识别中的重要性，影响图像的认知和理解

「动态（Motion）」：

- 动态图像和动画的视觉效果，涉及图像中运动和变化的表现
- 动态图像传递的信息和视觉冲击，对视觉表达具有独特影响

这些视觉属性涵盖了图片中常见的各种特征和效果，对于图像设计、图像处理、艺术创作以及视觉感知都有着重要的作用。深入了解和掌握这些属性有助于更好地理解图像，进行更有意义的图像创作和处理。

图的分类

光栅图和矢量图

图片一般分为两类：光栅图和矢量图。

光栅图：

光栅图是基于像素构成的图像。每个像素都有自己的颜色值，是图像最小的可见单元。光栅图格式常见的有JPEG、PNG、WEBP等。

矢量图：

矢量图使用点、线、多边形等几何形状来构图，而不是像素。矢量图具有高分辨率和无损缩放的特点，适合用于图标、徽标和需要频繁缩放的设计。SVG是一种常见的矢量图格式。

几何元素：

几何元素就像是用几何图形来拼接的图像。假设你想要画一条线，你可以用两个点来定义这条线的起点和终点，而这两个点就是几何元素。同样，你还可以用一些点来定义一个矩形、圆形、三角形等。这些点和形状的坐标信息被保存在计算机中，就像一张图纸一样，它们可以组合成更复杂的图形。

矢量图像与栅格化：

当矢量图像显示在屏幕上时，计算机会将矢量图形转换为像素图像的过程称为栅格化（Rasterization）。栅格化是将矢量图像中的几何元素和路径等转换为像素的过程。计算机会根据屏幕的分辨率和显示大小，将图像中的几何元素映射到屏幕上的像素点。

在栅格化过程中，计算机会根据矢量图像的数学公式和几何信息，决定每个像素点的颜色和亮度，然后在屏幕上以像素为单位绘制图像。这个过程中，矢量图像的优势在于可以无损地缩放，但在栅格化时，由于像素的有限性，可能会导致细节丢失或锯齿状的边缘（锯齿效应），尤其是在图像放大时。

在矢量图像经过栅格化后，实际上仍然可以无损地缩放而不会失真。栅格化的过程将矢量图像中的几何元素和路径转换为像素图像，但这并不改变矢量图形的数学描述和几何信息。所以，当您放大矢量图像时，计算机重新栅格化图像，并且会根据新的分辨率和显示大小生成更多的像素点，从而保持图像的清晰度和质量。

这是矢量图像的一个重要优点：在任何缩放级别下，它们都能保持较好的质量。因为矢量图像是使用数学公式描述的，所以不受像素数量的限制，可以无限地放大或缩小而不会出现像素化或失真问题。

对比光栅图像（像素图像），它们在放大时会出现像素化问题，因为它们的像素数量是固定的。在放大光栅图像时，计算机只是简单地放大像素，导致图像变得模糊，并且在极端情况下，可能会出现马赛克效果。

压缩分类

图片的压缩方式可以分为三类：无压缩、无损压缩和有损压缩。

无压缩：

无压缩的图片格式不对图片数据进行压缩处理，能准确地呈现原图片。其中，BMP是一种无压缩格式，保留了图像的所有信息，但文件体积较大。

无损压缩：

无损压缩算法对图片的所有数据进行编码压缩，能在保证图片质量的同时降低图片的尺寸。无损压缩格式中，PNG是常见的代表。PNG文件通常比BMP小，而且保持了图像的质量，适用于需要高质量和透明背景的情况。

有损压缩：

有损压缩算法会在压缩过程中去除图片中人眼无法察觉的细节，从而降低图片的尺寸。虽然有损压缩会导致一定程度的图像质量损失，但对于一般应用来说，这些损失是可以接受的。有损压缩格式中，JPEG是最常见的代表。JPEG文件通常较小，适用于在网页上显示图片和储存大量照片。

图片格式

GIF (Graphics Interchange Format) :

“

GIF（图形交换格式）是一种常见的图片格式，具有以下特点：

”

「优点：」

- 「支持动画和透明度：」 GIF支持多帧动画，使其在展示简单动画和表情包等方面非常适用。同时，它也支持透明度，可以创建带有透明背景的图像。
- 「广泛兼容性：」 GIF格式被广泛支持，几乎所有的现代浏览器和图像编辑软件都可以正常显示GIF图像。
- 「无损压缩：」 GIF使用无损压缩算法，保持图像质量，适用于一些需要保留细节的图像场景。
- 「适用于简单图标和动画表情包：」 由于其支持动画和透明度，GIF常用于制作简单的图标和有趣的动画表情包。

「缺点：」

- 「有限的颜色深度：」 GIF仅支持256色，对于照片等颜色丰富的图像，会出现色彩丢失和较低的图像质量。
- 「不适合保存高质量照片：」 由于有限的颜色深度，GIF不适合保存高质量的照片，特别是那些需要更多色彩细节的照片。
- 「文件较大：」 由于无损压缩，GIF图像文件较大，不适合在网页上频繁使用或作为高质量图片的存储格式。

「使用场景：」

GIF适用于需要展示简单动画和透明背景的场景，例如动画表情包、简单图标、以及一些简易的动画图像。



JPEG (Joint Photographic Experts Group) :

“

JPEG是一种广泛使用的图片格式，具有以下特点：

”

「优点：」

1. **「广泛兼容性：」** JPEG是一种通用的图片格式，在几乎所有现代设备和软件上都得到支持，包括电脑、手机、相机等。
2. **「有损压缩：」** JPEG使用有损压缩算法，可以显著减小图像文件的大小，同时保持适度的图像质量。这使得JPEG成为存储和传输照片的理想格式。
3. **「适合保存照片和复杂图像：」** JPEG对颜色细节的损失通常不会对照片和复杂图像造成明显影响，使其成为摄影、艺术品和复杂图像的首选格式。

「缺点：」

1. **「不支持透明度：」** JPEG不支持透明度，因此不适合用于制作带有透明背景的图像。
2. **「压缩损失：」** JPEG使用有损压缩，图像质量会在一定程度上受损。较高的压缩率会导致更明显的图像失真。
3. **「不适合无损图像需求：」** 由于压缩损失，JPEG不适合对图像完整性要求极高的场景，如医学图像、图像编辑等领域。

「使用场景：」

JPEG适用于各种需要高度压缩、广泛兼容性以及不需要透明背景的场景。它特别适用于摄影、照片、艺术品等复杂图像的存储和展示。



PNG (Portable Network Graphics) :

“

PNG (便携式网络图形) 是一种常见的无损图片格式, 具有以下特点:

”

「优点:」

- 「支持透明度:」** PNG支持完整的透明度, 可以创建带有透明背景的图像, 使其在网页设计和图标制作等场景中非常有用。
- 「无损压缩:」** PNG使用无损压缩算法, 保留了图像的完整性和质量, 不会引起图像失真。这使得PNG适用于需要保持高质量图像的需求。
- 「保留图像质量:」** PNG在压缩图像的同时, 不会丢失图像细节和色彩, 适用于需要高质量图像的应用场景。

「缺点:」

- 「文件较大:」** 由于使用无损压缩, PNG图像文件通常较大, 相比JPEG格式的压缩率较低, 不适合在网页上频繁使用大尺寸的PNG图像。
- 「不适合保存动画:」** PNG不支持动画功能, 因此对于动画图像, 不如GIF或APNG (Animated Portable Network Graphics) 这样的格式。

「使用场景:」

PNG适用于需要透明背景、保持图像质量以及不需要动画的场景。它特别适用于图标、网页上需要透明背景的图像、以及图像编辑中的临时存储格式。



APNG (Animated Portable Network Graphics) :

APNG (动画便携式网络图形) 是一种支持动画和透明度的图片格式，它是PNG的扩展版本，具有以下特点：

「优点：」

- 「支持动画和透明度：」** APNG能够实现多帧动画和完整的透明度，使其成为一种用于制作动画图像的高级格式。
- 「文件较小：」** 相较于GIF格式，APNG在保持相似动画效果的情况下，通常具有更小的文件大小，因为它采用了更高效的压缩算法。
- 「质量较好：」** 由于使用PNG的无损压缩，APNG保持了较好的图像质量，不会引起显著的图像失真。

「缺点：」

- 「浏览器兼容性不完善：」** 尽管APNG具有更好的动画效果和文件大小，但并不是所有的浏览器都完全支持它。在某些旧版本的浏览器中，APNG可能无法正确加载或播放。

「使用场景：」

APNG适用于制作带有透明背景和动画效果的图像，特别是替代GIF的情况。它可以用于制作复杂的动画表情包、网页上的动画图像等。



WebP:

“

WebP是一种现代的图片格式，由Google推出，具有以下特点：

”

「优点：」

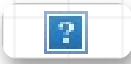
1. **「有损和无损压缩两种模式：」** WebP支持有损压缩和无损压缩两种模式，可以根据需要选择压缩程度，实现更高的压缩率或更高的图像质量。
2. **「文件较小：」** WebP采用先进的压缩算法，通常比JPEG和PNG具有更小的文件大小，可以减少网页加载时间，提高网页性能。
3. **「支持透明度和动画：」** WebP支持完整的透明度和动画功能，使其成为制作动画和带有透明背景的图像的理想格式。

「缺点：」

1. **「浏览器兼容性较差：」** 尽管WebP有许多优点，但并不是所有的浏览器都完全支持它。在一些旧版本的浏览器中，可能无法正确加载或显示WebP格式的图片。

「使用场景：」

WebP适用于需要高度压缩和带有透明背景或动画效果的图像。它特别适用于网页图像、动画图像，以及对文件大小敏感的场景。



SVG (Scalable Vector Graphics):

“

SVG 是一种基于 xml 的矢量图形格式，它将图像的内容指定为一组绘图命令，这些命令创建形状、线条、应用颜色、过滤器等等。SVG 文件是理想的图表，图标和其他图像，可以准确地绘制在任何大小。因此，SVG 是现代 Web 设计中用户界面元素的流行选择。

”

「优点：」

1. **「基于矢量图形：」** SVG是基于矢量图形的格式，它使用数学方程式来描述图像，因此不会因放大或缩小而失去清晰度，适合各种尺寸的显示。
2. **「文件小：」** SVG文件通常很小，因为它只保存描述图像的数学坐标和指令，而不保存像素信息。这有助于减少文件大小和加快加载速度。
3. **「广泛兼容性：」** SVG格式在各种设备和浏览器上具有广泛的兼容性，因为它是一种开放标准，并且被主流浏览器支持。
4. **「可无限缩放：」** 由于是矢量图形，SVG图像可以无限缩放而不失真，使其适用于需要高质量图形的场景。

「缺点：」

1. **「不适合保存复杂的光栅图像：」** SVG适合保存简单的图形和图标，但不适合保存复杂的光栅图像（例如照片），因为它会导致文件过大。

「使用场景：」

SVG适用于需要无限缩放、高度保持清晰度的图标、线条图形，以及在网页上实现矢量图形效果的场景。它特别适合用于制作网页图标、图表、地图等。



AVIF (AV1 Image File Format):

AVIF (AV1 Image File Format) 是一种现代的图片格式，它是由 Alliance for Open Media (开放媒体联盟) 开发的，使用了高级视频编码 AV1 来进行图像压缩。AV1 是一种开放、免版税的视频编码格式，因为其高效的压缩性能，也被应用于图像编码领域，形成了 AVIF 图片格式。

「优点：」

1. **「高效的压缩：」** AVIF 使用 AV1 编码技术，可以显著减小文件大小，尤其在保持良好图像质量的同时，能比其他图片格式更高效地压缩图像。
2. **「支持高级功能：」** 类似于 HEIF，AVIF 也支持多帧图像、动画、深度图像以及高动态范围图像 (HDR) 等高级功能，使其成为一种功能丰富的图片格式。
3. **「免费开放：」** AVIF 和 AV1 都是开放标准，没有专利费用，任何人都可以使用和部署，这促进了其广泛应用和推广。
4. **「未来性：」** 由于 AV1 技术的优越性，AVIF 可能成为未来图片压缩的主流格式之一，逐渐取代一些传统格式。

当然！补充AVIF图片格式的缺点：

「缺点：」

- 「浏览器兼容性不完善：」** 目前，尽管AVIF是一种高效的图片格式，但并不是所有的浏览器都完全支持它。一些旧版本的浏览器可能无法正确加载或显示AVIF格式的图片，这可能会限制其在某些环境中的使用。
- 「编码速度较慢：」** 由于AV1编码技术的复杂性，AVIF图片格式的编码速度相对较慢。在处理大量图片或需要实时编码的情况下，可能会导致一些性能方面的挑战。
- 「解码要求较高：」** 虽然AVIF可以实现高效的压缩，但解码过程对设备性能要求较高。在一些低端设备上，可能会导致图片加载速度较慢或增加能耗。
- 「文件格式复杂性：」** AVIF图片格式采用AV1视频编码作为基础，因此文件格式相对复杂。这可能导致一些图片编辑或处理软件在处理AVIF格式时遇到困难或需要进行适当的更新。

「使用场景：」

AVIF 图片格式适用于各种用途，特别是在需要高质量图像的情况下，如摄影、艺术品展示、专业图形设计等。它也可用于网络上的图像，可以显著减少图片加载时间，提高网页性能。

HEIF (High Efficiency Image File Format)

“

它是一种现代的图像文件格式，由Moving Picture Experts Group (MPEG) 开发。它采用了先进的编码技术，如高级视频编码 (H.265/HEVC) 和多帧图像 (MIAF)，以实现高效的图像压缩和更好的图像质量。

”

HEIF的特点和优势包括：

- 「1. 高效的压缩：」** HEIF使用H.265/HEVC编码，相较于传统的JPEG图像，可以显著减小文件大小，节省存储空间和网络传输带宽。
- 「2. 支持多种图像类型：」** HEIF不仅支持单个静态图像，还可以包含序列图像、深度图像和高动态范围图像 (HDR)，使其成为一种功能丰富的图像格式。
- 「3. 动态图像：」** HEIF可以保存多帧图像，从而支持动态图像、短视频和动画，类似于GIF或APNG，但具有更好的压缩性能。
- 「4. 兼容性：」** HEIF在现代设备和平台上得到广泛支持，包括iOS、Android、macOS、Windows等操作系统，并被多个主流浏览器支持。
- 「5. 缺失部分补偿：」** HEIF可以在图像数据丢失时通过参考其他帧进行补偿，提高了容错性和恢复能力。

「缺点：」

- 「兼容性问题：」** 尽管HEIF在现代设备和平台上得到广泛支持，但仍然存在一些旧版本的软件、操作系统或设备不支持HEIF格式的问题。这可能导致在某些环境中无法正确显示或处理HEIF图像。
- 「解码复杂性：」** HEIF使用H.265/HEVC编码技术，这是一种相对复杂的编码标准。因此，在一些较低性能的设备上，解码HEIF图像可能会导致一定的处理延迟或资源占用。
- 「专利费用：」** HEIF使用了H.265/HEVC编码技术，这是一个受专利保护的标准，因此在一些情况下可能需要支付专利费用，特别是在商业使用时。

「使用场景：」

使用场景

HEIF适用于多种用途，特别是在需要高效压缩和丰富图像功能的场景：

- 「移动设备和社交媒体：」 由于HEIF可以在相同质量下实现更小的文件大小，它在移动设备上的应用广泛。在社交媒体平台上，HEIF可以减少图像上传和下载时间，提高用户体验。
- 「摄影领域：」 对于摄影师和摄影爱好者来说，HEIF是一种优秀的格式，可以在保持图像质量的同时减少照片占用的存储空间，方便图片的管理和传输。
- 「动态图像和动画：」 HEIF支持多帧图像和动态图像，适用于制作GIF或APNG类似的动画效果，但在文件大小和图像质量上具有更好的表现。
- 「在线共享图像：」 在网络共享图像的情况下，HEIF可以减少图像加载时间，提高网页性能，特别是对于移动端用户来说更加重要。
- 「深度图像和HDR图像：」 HEIF支持深度图像和高动态范围图像，可以在显示设备支持时提供更好的图像效果和视觉体验。



面试官：Node性能如何进行监控以及优化？

1. 什么是Node性能监控？

Node作为一门服务端语言，性能方面尤为重要。在监控Node性能时，需要关注以下指标：

- CPU占用率：CPU负载和使用率，反映系统CPU繁忙程度。
- 内存占用率：监控Node进程的内存使用情况，避免内存泄漏和过度消耗。
- 磁盘I/O：监控硬盘的读写操作，避免I/O瓶颈影响性能。
- 网络：监控网络流量，确保网络连接稳定。

2、如何监控Node性能？

1. 使用Node.js内置工具

Node.js内置了一些工具和模块，可以用于监控和分析Node应用的性能。其中最常用的工具是 `util`、`perf_hooks` 和 `v8` 模块。

- `util`：提供了一些实用函数，包括 `util.promisify` 用于将回调函数转换为Promise形式，方便异步编程。

- `perf_hooks`: 是一个性能度量和监控的工具, 提供了 `performance` 和 `performanceObserver` 等API, 可以用于测量Node应用的性能指标。
- `v8`: 提供了许多V8引擎的信息和统计数据, 可以通过 `v8.getHeapStatistics()` 来获取堆内存使用情况。

```
// 使用util.promisify将回调函数转换为Promise形式
const util = require('util');
const fs = require('fs');
const readFileAsync = util.promisify(fs.readFile);

// 使用perf_hooks测量函数执行时间
const { performance } = require('perf_hooks');

async function readLargeFile() {
  const start = performance.now();
  const data = await readFileAsync(__dirname + '/large-file.txt');
  const end = performance.now();
  console.log(`读取文件耗时: ${end - start} 毫秒`);
  return data;
}

readLargeFile();
```

2. 使用开源监控工具

除了Node.js内置的工具, 还有许多优秀的开源监控工具可供选择。这些工具可以帮助我们更全面地监控Node应用的性能和状态。

- 「**PM2**」: 是一个成熟的Node.js进程管理器, 提供了丰富的监控和管理功能, 包括CPU和内存监控、进程状态查看、错误日志记录等。
- 「**New Relic**」: 是一款功能强大的应用性能监控工具, 支持多种语言和平台, 提供实时性能数据和报告, 帮助定位性能问题。
- 「**AppDynamics**」: 是一种全栈的应用性能管理解决方案, 可监控分布式应用、云基础架构和网络性能。

3. 自定义监控指标

除了使用现有的监控工具, 有时候我们需要根据应用的具体需求自定义监控指标。这可以通过日志记录、指标上报等方式实现。

// 使用日志记录自定义监控指标

```
const fs = require('fs');
const http = require('http');

function logMetrics(metric) {
  fs.appendFile('metrics.log', `${new Date().toISOString()} - ${metric}\n`, (err) => {
    if (err) {
      console.error('写入日志失败', err);
    }
  });
}

// 读取大文件并返回
http.createServer((req, res) => {
  const start = process.hrtime.bigint();
  const buffer = fs.readFileSync(__dirname + '/large-file.txt');
  const end = process.hrtime.bigint();
  const duration = Number(end - start) / 1e6; // 转换为毫秒
  logMetrics(`读取文件耗时: ${duration} 毫秒`);
  res.end(buffer);
}).listen(3000);
```

3、如何优化Node性能？

1. 使用异步操作

Node.js的异步非阻塞特性是其高性能的重要原因之一。在编写Node应用时，应尽可能使用异步操作，避免阻塞事件循环。

// 使用异步操作优化性能

```
const http = require('http');
const fs = require('fs');

http.createServer((req, res) => {
  fs.readFile(__dirname + '/large-file.txt', (err, data) => {
    if (err) {
      console.error('读取文件失败', err);
      res.statusCode = 500;
      res.end('读取文件失败');
    } else {
      res.end(data);
    }
  });
}).listen(3000);
```

2. 内存管理优化

Node.js中的内存管理是关键性能优化的一部分。合理使用内存可以减少垃圾回收的频率，提高性能。

- 使用对象池：通过重复使用对象，避免频繁的对象创建和销毁，减少内存开销。
- 控制内存使用：限制单个请求或任务的内存使用，防止出现内存泄漏和堆栈溢出。

```
// 使用对象池优化内存管理
const fs = require('fs');
const http = require('http');

const bufferPool = [];

function createBuffer() {
  return fs.readFileSync(__dirname + '/large-file.txt');
}

http.createServer((req, res) => {
  let buffer = bufferPool.pop();
  if (!buffer) {
    buffer = createBuffer();
  }
  res.end(buffer);
  bufferPool.push(buffer);
}).listen(3000);
```

3. 使用缓存

缓存是提高Node应用性能的有效方式之一。可以使用内存缓存（如Redis、Memcached）或本地缓存（如Node缓存模块）来存储经常使用的数据，减少重复计算和查询。

```
// 使用内存缓存优化性能
const http = require('http');
const fs = require('fs');
const NodeCache = require('node-cache');

const cache = new NodeCache();

http.createServer((req, res) => {
  const key = 'large-file-data';
  let data = cache.get(key);
  if (!data) {
    data = fs.readFileSync(__dirname + '/large-file.txt');
    cache.set(key, data);
  }
  res.end(data);
}).listen(3000);
```

4. 使用最新版本Node.js

Node.js团队持续改进V8引擎和Node.js内部代码，每个版本都会带来性能的提升。使用最新版本的Node.js可以获得更好的性能和稳定性。

5. 使用CDN加速

如果Node.js应用涉及到静态资源（如CSS、JavaScript、图片等），可以考虑使用CDN（内容分发网络）来加速资源的传输，提高页面加载速度。

总结

Node性能监控和优化是保证服务稳定性和性能的重要手段。通过使用内置工具、开源监控工具和自定义监控指标，可以全面监控Node应用的性能。同时，通过使用异步操作、内存管理优化、缓存和使用最新版本Node.js等方法，可以进一步提高Node应用的性能和稳定性。

参考文献：

- <https://segmentfault.com/a/1190000039327565>
- <https://segmentfault.com/a/1190000039327565>
- <https://zhuanlan.zhihu.com/p/50055740> (<https://zhuanlan.zhihu.com/p/50055740>)
- <https://segmentfault.com/a/1190000010231628>
- <https://segmentfault.com/a/1190000010231628>

面试官：如何实现jwt鉴权机制？说说你的思路

一、JWT简介

JSON Web Token (JWT) 是一种用于在用户和服务器之间传递安全可靠信息的字符串书写规范。它由三部分组成：头部 (Header)、载荷 (Payload)、签名 (Signature)，并以.进行拼接。JWT在前后端分离的开发中常用于身份验证。

头部 (Header)

头部指定使用的加密算法，一般使用HMAC SHA256。头部信息经过Base64编码，例如：

```
{ "alg": "HS256", "typ": "JWT" }
```

编码后为：

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

载荷 (Payload)

载荷用于存放实际的内容，例如用户的ID和名称，还可以设置过期时间等信息。载荷信息也经过Base64编码，例如：

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "iat": 1516239022  
}
```

编码后为：

```
eyJzdWliOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ
```

签名 (Signature)

签名是对头部和载荷进行签名，使用密钥对数据进行加密。签名可以确保数据在传输过程中没有被篡改。

二、实现JWT鉴权机制

1. 生成Token

使用第三方库 `jsonwebtoken` 来生成JWT Token。在用户登录成功后，颁发Token作为后续接口访问的凭证。

```
const crypto = require("crypto");
const jwt = require("jsonwebtoken");

// 用户列表 (应该使用数据库存储, 这里仅作演示)
let userList = [];

class UserController {
  static async login(ctx) {
    const data = ctx.request.body;
    if (!data.name || !data.password) {
      return (ctx.body = {
        code: "000002",
        message: "参数不合法",
      });
    }
    const result = userList.find(
      (item) =>
        item.name === data.name &&
        item.password === crypto.createHash("md5").update(data.password).digest("hex")
    );
    if (result) {
      const token = jwt.sign(
        {
          name: result.name,
        },
        "test_token", // 密钥
        { expiresIn: 60 * 60 } // 过期时间: 60 * 60秒
      );
      return (ctx.body = {
        code: "0",
        message: "登录成功",
        data: {
          token,
        },
      });
    } else {
      return (ctx.body = {
        code: "000002",
        message: "用户名或密码错误",
      });
    }
  }
}
```

前端接收到Token后，通常会使用 `localStorage` 进行缓存，并将Token放在HTTP请求头的 `Authorization` 字段中。注意在 `Authorization` 字段中加上 `Bearer` 前缀，后面跟一个空格。

```
axios.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  config.headers.common["Authorization"] = "Bearer " + token; // 注意这里的Authorization
  return config;
});
```

2. 校验Token

使用 `koa-jwt` 中间件进行Token验证，配置白名单（接口白名单不需要校验）。

```
const koajwt = require("koa-jwt");

app.use(
  koajwt({
    secret: "test_token", // 密钥，与签发时保持一致
  }).unless({
    path: ["/api/register", "/api/login"], // 接口白名单
  })
);
```

通过中间件校验后，可以在后续路由中获取用户信息。

```
router.get("/api/userInfo", async (ctx, next) => {
  const authorization = ctx.header.authorization;
  const token = authorization.replace("Bearer ", "");
  const result = jwt.verify(token, "test_token");
  ctx.body = result;
});
```

三、优缺点

优点：

- JSON具有通用性，可跨语言使用。
- JWT结构简单，字节占用小，便于传输。
- 服务器无需保存会话信息，容易进行水平扩展。
- 一处生成，多处使用，可解决分布式系统中的单点登录问题。
- 可防护CSRF攻击。

缺点：

- 载荷部分仅进行简单编码，只能存储逻辑必需的非敏感信息。
- 需要保护好加密密钥，一旦泄露后果严重。
- 为避免Token被劫持，建议使用HTTPS协议保护通信安全。

参考文献

- [理解JSON Web Token \(http://www.ruanyifeng.com/blog/2018/07/json_web_token-tutorial.html\)](http://www.ruanyifeng.com/blog/2018/07/json_web_token-tutorial.html)
- [Golang JWT详解 \(https://blog.wangjunfeng.com/post/golang-jwt/\)](https://blog.wangjunfeng.com/post/golang-jwt/)

面试官：如何实现文件上传？说说你的思路

一、什么是文件上传？

文件上传在日常开发中应用很广泛，比如在发微博、发微信朋友圈等操作中都会用到图片上传功能。由于浏览器的安全限制，浏览器不能直接操作文件系统，因此需要通过浏览器暴露出来的统一接口来访问文件，然后将文件内容读取到指定的内存中，并执行提交请求操作，将内存中的文件内容上传到服务端，服务端再解析前端传来的数据信息，最终将文件保存到服务器的文件系统中。

文件上传需要设置请求头为 `content-type: multipart/form-data`，其中 `multipart` 表示互联网上的混合资源，即资源由多种元素组成，而 `form-data` 表示可以使用HTML Forms和POST方法上传文件。

文件上传的请求结构如下：

```
POST /upload HTTP/1.1
Host: example.com
Content-Type: multipart/form-data; boundary=-----1234567890

-----1234567890
Content-Disposition: form-data; name="file"; filename="example.jpg"
Content-Type: image/jpeg

... binary data of the image ...
-----1234567890--
```

其中，`boundary` 表示分隔符，用于分隔不同的表单项。每个表单项必须包含 `Content-Disposition` 头，其他头信息如 `Content-Type` 为可选项。

二、如何实现文件上传？

文件上传可以分为两个步骤：

1. 文件的上传
2. 文件的解析

1. 文件上传

在前端，我们可以通过HTML的 `<input type="file">` 元素来实现文件上传的表单：

```
<form action="http://localhost:8080/api/upload" method="post" enctype="multipart/form-data">
  <input type="file" name="file" id="file" value="" multiple="multiple" />
  <input type="submit" value="提交"/>
</form>
```

在后端，我们可以使用Koa框架中的 `koa-body` 中间件来解析上传的文件数据：

```
npm install koa-body
```

```
const Koa = require('koa');
const koaBody = require('koa-body');
const app = new Koa();

app.use(koaBody({
  multipart: true,
  formidable: {
    maxFileSize: 200 * 1024 * 1024 // 设置上传文件大小最大限制, 默认2M
  }
}));

app.use(async (ctx) => {
  // 获取上传的文件
  const file = ctx.request.files.file;
  // 创建可读流
  const reader = fs.createReadStream(file.path);
  let filePath = path.join(__dirname, 'public/upload/') + `${file.name}`;
  // 创建可写流
  const upStream = fs.createWriteStream(filePath);
  // 可读流通过管道写入可写流
  reader.pipe(upStream);
  ctx.body = "上传成功! ";
});

app.listen(8080, () => {
  console.log('Server started on http://localhost:8080');
});
```

2. 文件解析

在服务器端，我们需要将上传的文件进行解析，保存到指定的目录中。在上述代码中，我们通过使用 `fs` 模块来处理文件的保存操作。

除了使用 `koa-body`，我们还可以使用 `koa-multer` 中间件来实现文件上传：

```
npm install koa-multer
```



```
const Koa = require('koa');
const Router = require('koa-router');
const multer = require('koa-multer');
const path = require('path');

const app = new Koa();
const router = new Router();

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, './upload/')
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname))
  }
})

const upload = multer({
  storage
});

router.post('/upload', upload.single('file'), (ctx, next) => {
  console.log(ctx.req.file); // 获取上传文件
})

app.use(router.routes());

app.listen(8080, () => {
  console.log('Server started on http://localhost:8080');
});
```

以上代码中，我们使用 `multer` 中间件来实现文件上传，`storage` 选项用于设置文件的保存路径和文件名，`upload.single('file')` 表示只上传单个文件。

参考文献

- [基于koa-body和koa-multer的文件上传 \(https://segmentfault.com/a/1190000037411957\)](https://segmentfault.com/a/1190000037411957)
- [koa-body文档 \(https://github.com/dlau/koa-body\)](https://github.com/dlau/koa-body)
- [koa-multer文档 \(https://github.com/koa-modules/multer\)](https://github.com/koa-modules/multer)

面试官：如果让你来设计一个分页功能，你会怎么设计？前后端如何交互？

一、什么是分页功能

在我们进行数据查询时，如果数据量很大，例如几万条数据，将其全部显示在一页上显然不友好。这时候我们需要采用分页显示的形式，每次只显示一部分数据，如每页显示10条数据。

实现分页功能，实际上就是从结果集中截取出第M~N条记录进行显示。

二、如何实现分页功能

后端实现

后端需要返回一些必要的分页信息给前端，例如总的数量、总页数、当前页数、当前页的数据。

首先，前端向后端发送目标的页码 `page` 以及每页显示的数据数量 `pageSize`。后端根据这些参数确定每页显示数据的起始位置 `start`，公式为：

```
const start = (page - 1) * pageSize;
```

然后，利用数据库的 `limit` 和 `OFFSET` 关键字进行分页查询：

```
const sql = `SELECT * FROM record LIMIT ${pageSize} OFFSET ${start};`;
```

其中，`LIMIT` 表示每页显示的数据条数，`OFFSET` 表示数据的偏移量。

另外，还需要查询数据库得到总的数量，用于计算总页数：

```
SELECT COUNT(*) FROM record;
```

最后，后端将获取的总数量、总页数、当前页数、当前页的数据发送给前端。

前端实现

前端需要发送请求给后端，传递目标页码 `page` 和每页显示数据的数量 `pageSize`。默认情况下，每次取10条数据。

三、代码实现

下面是一个使用 `Node.js` 和 `mysql` 数据库实现分页功能的示例代码：

```
const express = require('express');
const mysql = require('mysql');
const app = express();

const pool = mysql.createPool({
  host: 'localhost',
  user: 'your_mysql_username',
  password: 'your_mysql_password',
  database: 'your_database_name',
});

app.get('/api', (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const pageSize = parseInt(req.query.pageSize) || 10;
  const start = (page - 1) * pageSize;

  const sql = `SELECT * FROM record LIMIT ${pageSize} OFFSET ${start};`;
```

```
pool.getConnection((err, connection) => {
  if (err) {
    console.error('Error getting database connection: ', err);
    return res.status(500).json({ error: 'Internal server error' });
  }

  connection.query(sql, (err, results) => {
    connection.release();

    if (err) {
      console.error('Error executing SQL query: ', err);
      return res.status(500).json({ error: 'Internal server error' });
    }

    // 获取总数据量
    const countSql = 'SELECT COUNT(*) as total FROM record;';
    connection.query(countSql, (err, countResults) => {
      if (err) {
        console.error('Error executing count SQL query: ', err);
        return res.status(500).json({ error: 'Internal server error' });
      }

      const totalCount = countResults[0].total;
      const totalPages = Math.ceil(totalCount / pageSize);

      return res.status(200).json({
        totalCount,
        totalPages,
        currentPage: page,
        data: results,
      });
    });
  });
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log('Server started on port ${PORT}');
});
```

在上面的代码中，我们使用了 `express` 库和 `mysql` 库来实现服务器和数据库的交互。首先，获取前端传递的目标页码 `page` 和每页显示数据数量 `pageSize`，然后根据这些参数构造 `SQL` 语句进行分页查询。同时，还查询数据库得到总数据量，用于计算总页数。最后，将总数据量、总页数、当前页数、当前页的数据发送给前端。

四、总结

分页功能是在大量数据查询时提高用户体验的常见手段。通过在前后端交互中传递必要的分页信息，我们可以实现一个简单有效的分页功能。

参考文献

面试官：Node文件查找的优先级与Require方法的文件查找策略

在Node.js中，模块化是一个重要的特性，它允许我们将代码组织成独立的、可重用的模块。模块之间的导入和导出通过 `exports`、`module.exports` 和 `require` 来实现。让我们来深入了解Node文件查找的优先级和 `require` 方法的文件查找策略。

1. 模块查找规则

在理解模块查找优先级之前，我们先来了解Node.js是如何查找模块的。当使用 `require` 方法导入一个模块时，Node.js将按照以下顺序查找模块：

- 「**核心模块 (Built-in Modules)**」：首先，Node.js会尝试查找是否为内置模块，这些模块是Node.js自带的，无需安装即可使用，例如 `http`、`fs` 和 `path` 等。
- 「**文件模块 (File Modules)**」：如果模块不是核心模块，Node.js将尝试查找文件模块。文件模块可以是相对路径的文件模块（以 `./` 或 `../` 开头）、绝对路径的文件模块（以 `/` 开头）或者目录作为模块（以 `./dirname` 形式）。
- 「**目录作为模块**」：如果使用了目录作为模块名，并且目录中包含一个 `package.json` 文件，则Node.js会查找该文件中指定的 `main` 入口文件。如果 `package.json` 不存在或者未指定 `main`，则Node.js会尝试加载目录下的 `index.js` 或 `index.node` 文件作为入口。
- 「**非原生模块**」：如果以上查找都没有找到模块，Node.js会将模块名解析为绝对路径，并按照一定的路径顺序在文件系统中查找 `node_modules` 目录。Node.js会从当前模块的目录开始查找，然后逐级向上查找父目录的 `node_modules`，直到根目录。如果找到了 `node_modules` 目录，则进入其中查找对应模块。

2. 模块查找优先级

模块查找的优先级如下图所示：

在进行模块查找时，Node.js会将已经加载的模块缓存起来，以避免重复加载相同的模块，这也是优化加载速度的一种策略。

3. 示例代码

为了更好地理解模块查找的过程，让我们通过一些示例代码来演示：

「示例 1：导入核心模块」

```
const http = require('http');  
// Node.js会直接从核心模块中加载http模块，无需进一步查找
```

「示例 2：导入文件模块」

假设我们有一个文件模块 `myModule.js`，它的内容如下：

```
// myModule.js
exports.message = "Hello, I am a file module.";
```

现在我们在另一个文件中导入这个模块：

```
// app.js
const myModule = require('./myModule');
console.log(myModule.message);
```

在这个示例中，Node.js 会从当前文件所在目录查找 `myModule.js` 文件并加载它。

「示例 3：导入目录作为模块」

假设我们有一个目录 `myPackage`，它的结构如下：

```
myPackage
├── package.json
└── index.js
```

`package.json` 的内容如下：

```
{
  "name": "my-package",
  "main": "index.js"
}
```

`index.js` 的内容如下：

```
// index.js
exports.greet = "Hello, I am a package module.";
```

现在我们在另一个文件中导入这个目录作为模块：

```
// app.js
const myPackage = require('./myPackage');
console.log(myPackage.greet);
```

在这个示例中，Node.js 会先查找 `myPackage` 目录下的 `package.json`，然后根据其中指定的 `main` 入口文件 `index.js` 来加载模块。

「示例 4：导入非原生模块」

假设我们在项目中安装了一个第三方模块 `lodash`，现在我们在文件中导入它：

```
const _ = require('lodash');
// Node.js 会按照查找规则，从当前目录开始向上级目录查找 node_modules，直到找到 lodash 模块
```

4. 总结

Node.js 的模块查找规则和优先级确保了模块可以按照一定的路径顺序被找到并加载。核心模块和已加载的模块会被缓存，避免了重复加载，提高了应用程序的性能。在编写 Node.js 应用程序时，我们可以根据这些规则来组织和导入模块，实现代码的模块化和复用。

参考文献

- [Node.js Documentation: Modules \(http://nodejs.cn/api/modules.html#modules_file_modules\)](http://nodejs.cn/api/modules.html#modules_file_modules)
- [Node.js 模块加载机制详解 \(https://blog.csdn.net/qq_36801250/article/details/106352686\)](https://blog.csdn.net/qq_36801250/article/details/106352686)
- [Node.js 模块机制及 require 源码解析 \(https://www.cnblogs.com/samve/p/10805908.html\)](https://www.cnblogs.com/samve/p/10805908.html)

面试官：Node.js 中有哪些全局对象？

一、什么是全局对象

在浏览器 JavaScript 中，通常 `window` 是全局对象，而在 Node.js 中，全局对象是 `global`。

在 Node.js 里，不能在最外层定义一个变量，因为所有的用户代码都是当前模块的，只在当前模块里可用，但可以通过 `exports` 对象将变量传递给模块外部。

在 Node.js 中，用 `var` 声明的变量并不属于全局的变量，它们只在当前模块生效。真正的全局变量是挂载在 `global` 对象上的属性。

像上述的 `global` 全局对象则在全局作用域中，任何全局变量、函数、对象都是该对象的一个属性值。

二、哪些是全局对象

我们可以将全局对象分为两类：

- 真正的全局对象
- 模块级别的全局变量

真正的全局对象

下面给出一些常见的真正的全局对象：

- `Class: Buffer`
- `process`
- `console`
- `clearInterval`、`setInterval`
- `clearTimeout`、`setTimeout`
- `global`

Class: Buffer

`Buffer` 类用于处理二进制数据以及非 `Unicode` 编码的数据。

在 `Buffer` 类的实例化中存储了原始数据。`Buffer` 类似于一个整数数组，在 V8 堆原始存储空间给它分配了内存。

一旦创建了 `Buffer` 实例，则无法改变大小。

```
// 创建一个包含字节 [1, 2, 3] 的 Buffer。  
const buf = Buffer.from([1, 2, 3]);  
  
// 输出: <Buffer 01 02 03>  
console.log(buf);
```

process

`process` 对象提供有关当前进程的信息和控制。它是一个全局变量，无需使用 `require()` 就可以访问它。

`process` 对象包含了许多有用的属性，比如 `process.argv`，用于获取执行脚本时的命令行参数。

启动进程：

```
node index.js 参数1 参数2 参数3
```

index.js 文件如下：

```
process.argv.forEach((val, index) => {  
  console.log(` ${index}: ${val}`);  
});
```

输出如下：

```
/usr/local/bin/node  
/Users/mjr/work/node/process-args.js  
参数1  
参数2  
参数3
```

除此之外，`process` 对象还包含其他信息，如版本、操作系统等。

console

`console` 对象用于打印到标准输出和标准错误输出。它是 `stdout` 和 `stderr` 的简单封装。

最常用的打印函数：`console.log`

```
console.log("hello");
```

清空控制台：`console.clear`

```
console.clear
```

打印函数的调用栈：`console.trace`

```
function test() {
  demo();
}

function demo() {
  foo();
}

function foo() {
  console.trace();
}

test();
```

clearInterval、setInterval

`setInterval()` 方法用于按指定的周期调用函数。`clearInterval()` 方法用于取消由 `setInterval()` 方法设置的定时器。

```
setInterval(callback, delay[, ...args])
```

`callback` 每 `delay` 毫秒重复执行一次。

```
// 每 1000 毫秒输出一次 "Hello"
const intervalId = setInterval(() => {
  console.log("Hello");
}, 1000);
```

`clearInterval` 用于取消定时器的调用。

```
clearInterval(intervalId);
```

clearTimeout、setTimeout

`setTimeout()` 方法用于在指定的毫秒数后调用函数。`clearTimeout()` 方法用于取消由 `setTimeout()` 方法设置的定时器。

```
setTimeout(callback, delay[, ...args])
```

`callback` 在 `delay` 毫秒后执行一次。

```
// 3000 毫秒后输出 "Hello"
const timeoutId = setTimeout(() => {
  console.log("Hello");
}, 3000);
```

`clearTimeout` 用于取消延时器的调用。

```
clearTimeout(timeoutId);
```

global

全局命名空间对象，前面提到的 `process`、`console`、`setTimeout` 等都是 `global` 对象的属性。

```
console.log(process === global.process); // true
```

模块级别的全局变量

这些全局变量是模块中的变量，每个模块都有，看起来像全局变量

。但在命令交互中无法使用。

这些变量包括：

- `__dirname`
- `__filename`
- `exports`
- `module`
- `require`

`__dirname`

获取当前文件所在的路径，不包括后续的文件名。

```
console.log(__dirname);  
// 输出: /Users/mjr
```

`__filename`

获取当前文件的路径和文件名，包括后续的文件名。

```
console.log(__filename);  
// 输出: /Users/mjr/example.js
```

exports

`exports` 对象用于指定一个模块所导出的内容，即可以通过 `require()` 访问的内容。

```
exports.name = "John Doe";  
exports.age = 30;  
exports.sayHello = function() {  
  console.log("Hello");  
};
```

module

`module` 是对当前模块的引用。通过 `module.exports` 用于指定一个模块所导出的内容，即可以通过 `require()` 访问的内容。

require

`require()` 函数用于引入模块、JSON、或本地文件。可以使用相对路径引入本地模块或 JSON 文件，路径会根据 `__dirname` 定义的目录名或当前工作目录进行处理。

```
const myModule = require("./myModule");
```

三、优缺点

优点：

- JSON 具有通用性，所以可以跨语言
- 组成简单，字节占用小，便于传输
- 服务端无需保存会话信息，很容易进行水平扩展

缺点：

- 没有内置支持广泛数据类型（如日期、正则表达式）
- 无法处理循环引用，会导致序列化失败
- JSON 不支持注释，可读性较差

补充代码示例：

```
// 例1：演示模块级别的全局变量 __dirname 和 __filename
console.log(__dirname); // 输出当前文件所在的路径，如：/Users/mjr
console.log(__filename); // 输出当前文件的路径和文件名，如：/Users/mjr/example.js

// 例2：演示模块导出和引入
// myModule.js
exports.name = "John Doe";
exports.age = 30;
exports.sayHello = function() {
  console.log("Hello");
};

// app.js
const myModule = require("./myModule");
console.log(myModule.name); // 输出: John Doe
myModule.sayHello(); // 输出: Hello
```

参考文献

- [Node.js 官方文档 \(https://nodejs.org/api/globals.html\)](https://nodejs.org/api/globals.html)

面试官：说说Node中的EventEmitter？如何实现一个EventEmitter？

一、什么是EventEmitter？

在Node.js中，`EventEmitter`是事件驱动的基础，几乎所有模块都继承自它。它实现了观察者模式，其中被观察者维护一组观察者，并在更新时通知观察者。

`EventEmitter`允许对象绑定和触发事件监听器，实现异步操作。在Node.js中，许多对象都会分发事件，例如`fs.readStream`对象会在文件被打开时触发一个事件。

二、使用EventEmitter

Node.js的`events`模块提供了一个`EventEmitter`类，可以通过继承它创建自定义事件对象。

基本使用方法如下：

```
const EventEmitter = require('events');

class MyEmitter extends EventEmitter {}

const myEmitter = new MyEmitter();

function callback() {
  console.log('触发了event事件! ');
}

myEmitter.on('event', callback);
myEmitter.emit('event');
myEmitter.removeListener('event', callback);
```

`EventEmitter`的常用方法有：

- `on(eventName, listener)`: 添加类型为`eventName`的事件监听器。
- `once(eventName, listener)`: 添加类型为`eventName`的事件监听器，但只能执行一次，执行后将被删除。
- `prependListener(eventName, listener)`: 添加类型为`eventName`的事件监听器到事件数组头部。
- `emit(eventName, ...args)`: 触发类型为`eventName`的事件监听器。
- `removeListener(eventName, listener)`: 移除类型为`eventName`的事件监听器。

三、实现自定义EventEmitter

我们可以简单实现一个`EventEmitter`类，了解其基本原理。在实现过程中，需要维护一个包含所有事件的对象`events`。

```
class EventEmitter {
  constructor() {
    this.events = {};
  }

  on(type, handler) {
    if (!this.events[type]) {
      this.events[type] = [];
    }
    this.events[type].push(handler);
  }

  emit(type, ...args) {
    if (!this.events[type]) {
      return;
    }
    this.events[type].forEach((handler) => {
      handler.apply(this, args);
    });
  }

  removeListener(type, handler) {
    if (!this.events[type]) {
      return;
    }
    this.events[type] = this.events[type].filter((item) => item !== handler);
  }
}
```

以上代码实现了 `on`、`emit`、`removeListener` 方法，分别用于添加事件监听器、触发事件和移除事件监听器。

四、测试自定义EventEmitter

```
const bus = new EventEmitter();

bus.on('event', () => {
  console.log('Event 1');
});

function event2() {
  console.log('Event 2');
}

bus.on('event', event2);

bus.emit('event');
// Output:
// Event 1
// Event 2

bus.removeListener('event', event2);
bus.emit('event');
// Output:
// Event 1
```

五、总结

`EventEmitter` 是 Node.js 中非常重要的一个模块，它实现了事件驱动的基本模式，让 Node.js 具备了处理异步操作的能力。我们也可以通过简单实现一个自定义的 `EventEmitter` 来更好地理解其原理和用法。

参考文献

- [Node.js 官方文档 - events 模块 \(http://nodejs.cn/api/events.html#events_class_eventemitter\)](http://nodejs.cn/api/events.html#events_class_eventemitter)
- [Node.js EventEmitter 源码解析 \(https://segmentfault.com/a/1190000015762318\)](https://segmentfault.com/a/1190000015762318)
- [Node.js 事件模块 EventEmitter 的简单实现 \(https://juejin.cn/post/6844903781230968845\)](https://juejin.cn/post/6844903781230968845)

面试官：说说你对 Node.js 的理解？优缺点？应用场景？

一、Node.js 的概述

Node.js 是一个开源且跨平台的 JavaScript 运行时环境，它在浏览器外运行 V8 JavaScript 引擎（Google Chrome 的内核），利用事件驱动、非阻塞和异步输入输出模型等技术来提高性能，使得 JavaScript 可以用于服务器端开发。Node.js 采用非阻塞型 I/O 机制，在处理 I/O 操作时不会阻塞进程，而是通过事件通知的方式处理操作结果。这种机制使得 Node.js 在处理高并发场景时表现出色。

非阻塞异步

Node.js 采用了非阻塞型 I/O 机制，在执行 I/O 操作时不会造成阻塞，当操作完成后通过回调函数以事件的形式通知执行结果。这样可以提高程序的执行效率，使得在执行高并发任务时表现出色。

事件驱动

Node.js使用事件驱动的编程模型。当有新的请求进来时，请求会被压入一个事件队列中，然后通过事件循环来检测队列中的事件状态变化，一旦有状态变化，就会执行对应的回调函数来处理事件。这使得Node.js能够高效处理I/O密集型应用。

二、Node.js的优缺点

优点

- 处理高并发场景性能更佳：Node.js适用于处理大量并发请求的场景，由于其非阻塞I/O和事件驱动模型，能够高效地处理并发请求。
- 适合I/O密集型应用：Node.js擅长处理I/O密集型应用，其执行效率仍然较高，即使在运行极限时，CPU占用率仍然较低。
- 轻量快速：Node.js采用V8引擎作为底层，使得其执行速度快，且具有较小的内存占用。

缺点

- 不适合CPU密集型应用：由于Node.js是单线程的，如果应用程序中有大量计算密集型任务，会导致阻塞，影响整个应用的性能。
- 单线程：Node.js采用单线程事件循环，虽然能够高效处理I/O密集型应用，但是无法充分利用多核CPU，不能充分发挥多核处理器的优势。
- 可靠性低：由于Node.js是单线程的，一旦某个环节崩溃，整个系统都会崩溃，可靠性较低。

三、Node.js的应用场景

Node.js的应用场景主要分为以下几个类别：

1. Web应用程序：对于高并发的网络应用程序，如用户表单收集系统、后台管理系统、实时交互系统、考试系统等，Node.js表现优越。
2. 多人联网游戏：基于Web、Canvas等技术的多人联网游戏能够充分利用Node.js的高并发特性。
3. 实时聊天和直播：基于Web的多人实时聊天客户端、聊天室、图文直播等应用。
4. 单页面应用程序：对于单页面应用程序，Node.js能够处理前端与后端的数据交互，提供高效的API服务。
5. 数据库操作和API服务：Node.js可以用于操作数据库，并为前端和移动端提供基于JSON的API服务。

虽然Node.js在上述场景中表现优秀，但在选择使用Node.js时，需要根据具体的业务需求和技术要求来判断其适用性。

参考文献

- [Node.js 官方网站 \(http://nodejs.cn/\)](http://nodejs.cn/)
- [深入理解Node.js的优势与不足 \(https://segmentfault.com/a/1190000019854308\)](https://segmentfault.com/a/1190000019854308)
- [Node.js的优势和劣势 \(https://segmentfault.com/a/1190000005173218\)](https://segmentfault.com/a/1190000005173218)

面试官：说说对 Node 中的 Buffer 的理解？应用场景？

一、什么是Buffer？

在Node.js应用中，我们常常需要处理网络协议、操作数据库、处理图片或接收上传文件等任务。在这些任务中，需要处理大量的二进制数据，而Node.js提供了一个专门用于处理二进制数据的类——`Buffer`。

Buffer 是一种在内存中分配的用于存储二进制数据的缓冲区，它类似于整数数组，但专门用于存储字节。Buffer 的大小在创建时确定，并且不能更改。它是 Node.js 中处理二进制数据的核心工具之一。

二、使用 Buffer

Node.js 提供了多种创建 Buffer 的方式，常用的有 Buffer.from() 和 Buffer.alloc()。

Buffer.from()

```
const b1 = Buffer.from('hello');
const b2 = Buffer.from('hello', 'utf8');
const b3 = Buffer.from([104, 101, 108, 108, 111]);
const b4 = Buffer.from(b3);

console.log(b1, b2, b3, b4); // <Buffer 68 65 6c 6c 6f> <Buffer 68 65 6c 6c 6f> <Buffer 68 65 6c 6c 6f> <Buffer 68 65 6c 6c 6f>
```

Buffer.alloc()

```
const bAlloc1 = Buffer.alloc(10); // 创建一个大小为 10 个字节的缓冲区，会清空之前分配的内存
const bAlloc2 = Buffer.alloc(10, 1); // 创建一个长度为 10 的 Buffer，其中全部填充了值为 `1` 的字节
console.log(bAlloc1); // <Buffer 00 00 00 00 00 00 00 00 00 00>
console.log(bAlloc2); // <Buffer 01 01 01 01 01 01 01 01 01 01>
```

通过 Buffer，我们可以将二进制数据转换成不同编码的字符串，例如：

```
const buffer = Buffer.from('hello');
console.log(buffer); // <Buffer 68 65 6c 6c 6f>
const str = buffer.toString();
console.log(str); // hello
```

三、应用场景

Buffer 在 Node.js 中有广泛的应用场景，特别是与流的概念相关的操作。

I/O 操作

通过流的形式，可以将一个文件的内容读取到另外一个文件。

```
const fs = require('fs');

const inputStream = fs.createReadStream('input.txt'); // 创建可读流
const outputStream = fs.createWriteStream('output.txt'); // 创建可写流

inputStream.pipe(outputStream); // 管道读写
```

加解密

在一些加解密算法中，我们会使用 `Buffer`，例如 `crypto.createCipheriv` 的第二个参数 `key` 可以为 `string` 或 `Buffer` 类型。

zlib.js

`zlib.js` 是 Node.js 的核心库之一，它利用了 `Buffer` 的功能来操作二进制数据流，提供了压缩或解压功能。

`Buffer` 的强大功能使得它在处理二进制数据时非常高效和灵活，适用于许多不同的应用场景。

参考文献

- [Node.js 官方文档 - Buffer](http://nodejs.cn/api/buffer.html) (<http://nodejs.cn/api/buffer.html>)
- [Buffer 用法详解](https://segmentfault.com/a/1190000019894714) (<https://segmentfault.com/a/1190000019894714>)

面试官：说说对 Node 中的 Stream 的理解？应用场景？

一、什么是流？

在 Node.js 中，流 (Stream) 是一种数据传输的手段，用于端到端的信息交换，数据是按照顺序逐块读取和处理的。相比于传统的一次性将整个文件读入内存的方式，流是逐块处理数据，不将全部数据保存在内存中。

流可以分为四种种类：

1. 可读流 (Readable Stream)：用于读取数据的流，例如 `fs.createReadStream()` 可以从文件读取内容。
2. 可写流 (Writable Stream)：用于写入数据的流，例如 `fs.createWriteStream()` 可以将数据写入文件。
3. 双工流 (Duplex Stream)：可读又可写的流，例如 `net.Socket`。
4. 转换流 (Transform Stream)：可以在数据写入和读取时修改或转换数据的流，例如文件压缩操作中的压缩和解压缩过程。

这些流都是单向的，流的操作通常通过 pipe (管道) 来连接。`source` 流通过 pipe 将数据流向 `dest` 流，实现数据传输。

二、应用场景

流 (Stream) 在 Node.js 中应用广泛，特别适用于处理 I/O 操作。以下是一些常见的应用场景：

1. HTTP 请求和响应

在 Node.js 的 HTTP 服务器模块中，`request` 是可读流，`response` 是可写流。可以通过流的方式来处理 HTTP 请求和响应，特别适用于大文件的传输。

```
const http = require('http');
const fs = require('fs');

const server = http.createServer((req, res) => {
  const readStream = fs.createReadStream('largeFile.txt');
  readStream.pipe(res);
});

server.listen(3000);
```

2. 文件操作

在处理文件读写时，流可以将数据分段处理，适用于大文件的读写操作，避免一次性将整个文件读入内存。

```
const fs = require('fs');

const readStream = fs.createReadStream('source.txt');
const writeStream = fs.createWriteStream('destination.txt');

readStream.pipe(writeStream);
```

3. 数据转换

转换流（Transform Stream）是一种特殊的流，可以在数据写入和读取时对数据进行修改或转换。常见的应用包括数据压缩、加密解密等操作。

```
const { Transform } = require('stream');

const myTransform = new Transform({
  transform(chunk, encoding, callback) {
    // 对数据进行转换处理
    const transformedData = doSomething(chunk);
    // 将转换后的数据传给下一个流程
    callback(null, transformedData);
  }
});
```

4. 打包工具构建过程

一些前端打包构建工具（如Gulp）通常会频繁地进行文件操作，使用流可以提高效率和降低内存占用。

以上只是流的一些应用场景，实际上流在Node.js中几乎无处不在，通过流的处理，可以更好地控制和处理数据，提高性能和效率。

五、参考文献

- <https://xie.infoq.cn/article/1a9695020828460eb3c4ff1fa>
(<https://xie.infoq.cn/article/1a9695020828460eb3c4ff1fa>)
- <https://juejin.cn/post/6844903891083984910>
(<https://juejin.cn/post/6844903891083984910>)

面试官：说说对 Node 中的 fs 模块的理解？有哪些常用方法

一、什么是 fs 模块？

在 Node.js 中，fs (filesystem) 模块提供了本地文件的读写能力，它基本上是对 POSIX 文件操作命令的简单包装。通过 fs 模块，我们可以对文件进行读取、写入、追加写入、拷贝等操作。

要使用 fs 模块，我们首先需要导入该模块，如下：

```
const fs = require('fs');
```

该模块对所有文件系统操作提供了异步（不具有 sync 后缀）和同步（具有 sync 后缀）两种操作方式，供开发者根据需求进行选择。

二、文件相关知识

在使用 fs 模块前，有几个文件相关的概念需要了解：

1. 「权限位 mode」

权限位用于对文件的所有者、文件所属组和其他用户进行权限分配。其中，类型又分成读、写和执行，具备权限位 4、2、1，不具备权限为 0。

如在 Linux 系统中，我们可以通过 `ls -l` 命令查看文件的权限位：

```
drwxr-xr-x 1 PandaShen 197121 0 Jun 28 14:41 core
-rw-r--r-- 1 PandaShen 197121 293 Jun 28 17:44 index.md
```

在开头的十位中，`d` 表示文件夹，`-` 表示文件，后九位表示当前用户、用户所属组和其他用户的权限位，按每三位划分，分别代表读（r）、写（w）和执行（x），`-` 表示没有当前位对应的权限。

2. 「标识位 flag」

标识位代表对文件的操作方式，如可读、可写、即可读又可写等等，常用的标识位如下：

符号 含义

- r 读取文件，如果文件不存在则抛出异常。
- r+ 读取并写入文件，如果文件不存在则抛出异常。
- rs 读取并写入文件，指示操作系统绕开本地文件系统缓存。
- w 写入文件，文件不存在会被创建，存在则清空后写入。
- wx 写入文件，排它方式打开。
- w+ 读取并写入文件，文件不存在则创建文件，存在则清空后写入。
- wx+ 和 w+ 类似，排他方式打开。
- a 追加写入，文件不存在则创建文件。
- ax 与 a 类似，排他方式打开。
- a+ 读取并追加写入，不存在则创建。
- ax+ 与 a+ 类似，排他方式打开。

3. 「文件描述符 fd」

操作系统为每个打开的文件分配一个名为文件描述符的数值标识，文件操作使用这些文件描述符来识别和追踪每个特定的文件。在 Node.js 中，每操作一个文件，文件描述符是递增的，文件描述符一般从 3 开始，因为前面有 0、1、2 三个比较特殊的描述符，分别代表 `process.stdin`（标准输入）、`process.stdout`（标准输出）和 `process.stderr`（错误输出）。

三、常用方法

下面介绍fs模块的常用方法，并给出相应的代码示例。

文件读取

1. fs.readFileSync

同步读取文件内容，返回文件内容的Buffer或指定编码的字符串。

```
const fs = require('fs');

// 同步读取文件，返回 Buffer
const buf = fs.readFileSync('file.txt');

// 同步读取文件，返回指定编码的字符串
const data = fs.readFileSync('file.txt', 'utf8');
```

2. fs.readFile

异步读取文件内容
，通过回调函数获取结果。

```
const fs = require('fs');

fs.readFile('file.txt', 'utf8', (err, data) => {
  if (!err) {
    console.log(data); // 打印文件内容
  }
});
```

文件写入

1. fs.writeFileSync

同步写入文件内容。

```
const fs = require('fs');

fs.writeFileSync('file.txt', 'Hello, world!');
```

2. fs.writeFile

异步写入文件内容，通过回调函数处理写入结果。

```
const fs = require('fs');

fs.writeFile('file.txt', 'Hello, world!', err => {
  if (!err) {
    console.log('写入成功');
  }
});
```

文件追加写入

1. fs.appendFileSync

同步追加写入文件内容。

```
const fs = require('fs');

fs.appendFileSync('file.txt', 'Hello, world!');
```

2. fs.appendFile

异步追加写入文件内容，通过回调函数处理写入结果。

```
const fs = require('fs');

fs.appendFile('file.txt', 'Hello, world!', err => {
  if (!err) {
    console.log('追加写入成功');
  }
});
```

文件拷贝

1. fs.copyFileSync

同步拷贝文件。

```
const fs = require('fs');

fs.copyFileSync('file.txt', 'file-copy.txt');
```

2. fs.copyFile

异步拷贝文件，通过回调函数处理拷贝结果。


```
const fs = require('fs');

fs.copyFile('file.txt', 'file-copy.txt', err => {
  if (!err) {
    console.log('拷贝成功');
  }
});
```

创建目录

1. fs.mkdirSync

同步创建目录。

```
const fs = require('fs');

fs.mkdirSync('new-folder');
```

2. fs.mkdir

异步创建目录，通过回调函数处理创建结果。

```
const fs = require('fs');

fs.mkdir('new-folder', err => {
  if (!err) {
    console.log('目录创建成功');
  }
});
```

参考文献

- [Node.js文档：fs模块 \(http://nodejs.cn/api/fs.html\)](http://nodejs.cn/api/fs.html)
- [Node.js中文网：fs模块介绍 \(https://www.nodejs.cn/api/fs.html\)](https://www.nodejs.cn/api/fs.html)

面试官：说说对 Node 中的 process 的理解？有哪些常用方法？

1. 简介

`process` 对象是 Node.js 中的全局变量，它提供了有关当前 Node.js 进程的信息并允许对其进行控制。通过 `process` 对象，我们可以获取进程的环境变量、命令行参数，控制进程的行为以及与其他进程进行通信。

2. 常用属性

`process.env`

`process.env` 是一个包含用户环境信息的对象。它可以用于获取环境变量的值，例如获取不同环境下的配置信息。比如我们经常使用 `process.env.NODE_ENV` 来区分开发环境和生产环境。

`process.argv`

`process.argv` 是一个包含命令行参数的数组。当通过命令行执行Node脚本时，`process.argv` 会获取命令行传入的参数。数组的第一个元素是Node的路径，第二个元素是脚本文件的路径，后续元素是真正的命令行参数。我们可以通过解析 `process.argv` 来获取传入的命令行参数。

```
// 示例：获取命令行传入的参数
const args = process.argv.slice(2);
console.log(args);
```

`process.cwd()`

`process.cwd()` 方法返回当前Node进程执行的目录。在不同的操作中执行Node脚本时，其工作目录可能会不同。通过 `process.cwd()` 可以获取当前脚本执行的目录，便于处理文件路径。

```
// 示例：获取当前进程工作目录
console.log(process.cwd());
```

`process.pid`

`process.pid` 属性返回当前进程的ID（PID）。PID是操作系统中用于标识进程的唯一值。

```
// 示例：获取当前进程ID
console.log(process.pid);
```

`process.platform`

`process.platform` 属性返回当前进程运行的操作系统平台，如 `'win32'`、`'linux'` 等。

```
// 示例：获取当前操作系统平台
console.log(process.platform);
```

`process.uptime()`

`process.uptime()` 方法返回当前进程已运行的时间，单位为秒。例如，可以使用 `pm2` 来守护进程并获取其运行时间。

```
// 示例：获取当前进程已运行的时间
console.log(process.uptime());
```

3. 常用方法

`process.nextTick()`

`process.nextTick()` 方法将回调函数放在当前执行栈的底部，以便在下一个事件循环中执行。这个方法的优先级比 `setTimeout()` 和 `setImmediate()` 更高。它通常用于确保回调函数在当前操作完成后尽快执行，例如在 I/O 操作之后处理数据。

```
// 示例：使用process.nextTick()确保回调函数尽快执行
function foo() {
  console.log('foo');
}

process.nextTick(foo);
console.log('bar');
```

输出结果为：`bar`、`foo`。

process.on(event, callback)

`process.on()` 方法用于监听 Node 进程上的不同事件。常见的事件包括 `uncaughtException`（捕获未捕获的异常）和 `exit`（进程退出）等。我们可以通过监听这些事件来处理异常情况或在进程退出时执行一些清理操作。

```
// 示例：捕获未捕获的异常并处理
process.on('uncaughtException', (err) => {
  console.error('Caught exception:', err);
});
```

上述代码会捕获未捕获的异常，并打印错误信息。

4. 总结

`process` 对象是 Node.js 中一个重要的全局变量，提供了有关当前进程的信息和控制功能。它的常用属性包括 `process.env`、`process.argv`、`process.cwd()`、`process.pid` 和 `process.platform` 等。常用方法包括 `process.nextTick()` 和 `process.on()` 等。通过使用 `process` 对象，我们可以更好地管理和控制 Node.js 进程的行为。

参考文献

- [Node.js Documentation: Process \(http://nodejs.cn/api/process.html\)](http://nodejs.cn/api/process.html)

面试官：说说对Nodejs中的事件循环机制理解？

一、什么是事件循环？

在 Node.js 中，事件循环是基于 libuv 实现的。libuv 是一个多平台的专注于异步 I/O 的库，负责处理底层的事件循环和 I/O 操作，使得 Node.js 能够实现非阻塞的 I/O。

事件循环主要是为了处理异步操作。在 Node.js 中，几乎所有的 I/O 操作都是异步的，包括文件读写、网络请求等。为了能够在异步操作完成后及时执行相应的回调函数，Node.js 引入了事件循环机制。

二、事件循环的流程

Node.js的事件循环是基于事件触发器的概念。事件触发器可以添加监听器，当特定事件发生时，触发相应的回调函数。事件循环不断地检查事件触发器是否有待处理的事件，如果有，则执行相应的回调函数。

事件循环的流程可以简化为以下几个步骤：

1. 执行当前的同步代码。
2. 检查是否有待处理的微任务（即process.nextTick和Promise的then回调），如果有，则依次执行微任务。
3. 检查是否有待处理的宏任务（即定时器、IO事件和setImmediate回调），如果有，则执行宏任务队列中最早的一个任务。
4. 重复上述步骤，直到所有的任务队列为空。

三、事件循环的阶段

Node.js的事件循环分为6个阶段，每个阶段都有对应的任务队列。事件循环每次遍历一个阶段，都会依次执行该阶段的任务队列中的任务。下面是Node.js事件循环的6个阶段：

1. timers阶段：执行定时器回调。
2. I/O事件回调阶段：执行I/O相关的回调，例如网络请求、文件读写等。
3. idle, prepare阶段：内部使用，不常用。
4. poll阶段：检索新的I/O事件并执行相关回调。
5. check阶段：执行setImmediate回调。
6. close阶段：执行关闭事件的回调。

四、题目分析

题目代码如下：

```

async function async1() {
  console.log('async1 start')
  await async2()
  console.log('async1 end')
}

async function async2() {
  console.log('async2')
}

console.log('script start')

setTimeout(function () {
  console.log('setTimeout0')
}, 0)

setTimeout(function () {
  console.log('setTimeout2')
}, 300)

setImmediate(() => console.log('setImmediate'));

process.nextTick(() => console.log('nextTick1'));

async1();

process.nextTick(() => console.log('nextTick2'));

new Promise(function (resolve) {
  console.log('promise1')
  resolve();
  console.log('promise2')
}).then(function () {
  console.log('promise3')
})

console.log('script end')

```

分析过程如下：

1. 执行同步代码，输出 `script start`。
2. 遇到第一个 `setTimeout`，将其回调函数放入 `timer` 阶段。
3. 遇到第二个 `setTimeout`，设置了300ms后放入 `timer` 阶段。
4. 遇到 `setImmediate`，放入 `check` 阶段。
5. 遇到第一个 `process.nextTick`，放入微任务队列。
6. 执行 `async1()`，输出 `async1 start`，然后执行 `async2()`，输出 `async2`，`async2` 后面的代码进入微任务队列，等待下一轮事件循环。
7. 遇到第二个 `process.nextTick`，放入微任务队列。
8. 遇到 `new Promise`，输出 `promise1`，然后执行立即执行函数，输出 `promise2`，接着 `then` 的回调进入微任务队列。
9. 执行 `console.log('script end')`，输出 `script end`。
10. 执行微任务队列中的回调，依次输出 `nextTick1`、`nextTick2`、`async1`

`end`、`promise3`。

11. 执行 `timer` 阶段，输出 `setTimeout0`。
12. 执行 `check` 阶段，输出 `setImmediate`。
13. 300ms后，`timer` 阶段有任务，输出 `setTimeout2`。

最终输出结果为：

```
script start
async1 start
async2
promise1
promise2
script end
nextTick1
nextTick2
async1 end
promise3
setTimeout0
setImmediate
setTimeout2
```

五、setTimeout与setImmediate的输出顺序

最后还有一个关于 `setTimeout` 与 `setImmediate` 的输出顺序的问题。

```
setTimeout(() => {
  console.log("setTimeout");
}, 0);

setImmediate(() => {
  console.log("setImmediate");
});
```

可能输出情况有两种：

情况一

:

```
setTimeout
setImmediate
```

情况二：

```
setImmediate
setTimeout
```

这是因为 `setTimeout` 的回调函数虽然设置了0毫秒的延迟，但是实际上会被强制改成1ms，而 `setImmediate` 会在检查阶段（`check` 阶段）立即执行。因此，如果同步代码执行时间较长，可能会导致 `setTimeout` 的回调在 `setImmediate` 之后执行。

面试官：说说对中间件概念的理解，如何封装 node 中间件？

一、中间件概念

中间件（Middleware）是介于应用系统和系统软件之间的一类软件，它使用系统软件所提供的基础服务（功能），衔接网络上应用系统的各个部分或不同的应用，能够达到资源共享、功能共享的目的。

在Node.js中，中间件主要是指封装HTTP请求细节处理的方法。例如在Express、Koa等Web框架中，中间件的本质为一个回调函数，参数包含请求对象、响应对象和执行下一个中间件的函数。在这些中间件函数中，我们可以执行业务逻辑代码，修改请求和响应对象，返回响应数据等操作。

二、封装中间件

1. Token校验中间件

```
// token校验中间件
module.exports = (options) => async (ctx, next) {
  try {
    // 获取token
    const token = ctx.header.authorization;
    if (token) {
      try {
        // 使用verify函数验证token，并获取用户相关信息
        await verify(token);
      } catch (err) {
        console.log(err);
      }
    }
    // 进入下一个中间件
    await next();
  } catch (err) {
    console.log(err);
  }
}
```

2. 日志中间件

```
const fs = require('fs');

// 日志中间件
module.exports = (options) => async (ctx, next) => {
  const startTime = Date.now();
  const requestTime = new Date();
  await next();
  const ms = Date.now() - startTime;
  let logEntry = `${ctx.request.ip} -- ${requestTime} -- ${ctx.method} -- ${ctx.url} -- ${ms}ms`;
  // 输出日志到文件
  fs.appendFileSync('./log.txt', logEntry + '\n');
}
```

3. Koa-bodyparser中间件简单实现

```
// 自定义koa-bodyparser中间件
const querystring = require("querystring");

module.exports = function bodyParser() {
  return async (ctx, next) => {
    await new Promise((resolve, reject) => {
      // 存储数据的数组
      let dataArr = [];

      // 接收数据
      ctx.req.on("data", data => dataArr.push(data));

      // 整合数据并使用 Promise 成功
      ctx.req.on("end", () => {
        // 获取请求数据的类型 json 或表单
        let contentType = ctx.get("Content-Type");

        // 获取数据 Buffer 格式
        let data = Buffer.concat(dataArr).toString();

        if (contentType === "application/x-www-form-urlencoded") {
          // 如果是表单提交, 则将查询字符串转换成对象赋值给 ctx.request.body
          ctx.request.body = querystring.parse(data);
        } else if (contentType === "application/json") {
          // 如果是json, 则将字符串格式的对象转换成对象赋值给 ctx.request.body
          ctx.request.body = JSON.parse(data);
        }

        // 执行成功的回调
        resolve();
      });
    });

    // 继续向下执行
    await next();
  };
};
```

4. Koa-static中间件简单实现

```
const fs = require("fs");
const path = require("path");
const mime = require("mime");
const { promisify } = require("util");

// 将 stat 和 access 转换成 Promise
const stat = promisify(fs.stat);
const access = promisify(fs.access);

module.exports = function (dir) {
  return async (ctx, next) => {
    // 将访问的路由处理成绝对路径，这里要使用 join 因为有可能是 /
    let realPath = path.join(dir, ctx.path);

    try {
      // 获取 stat 对象
      let statObj = await stat(realPath);

      // 如果是文件，则设置文件类型并直接响应内容，否则当作文件夹寻找 index.html
      if (statObj.isFile()) {
        ctx.set("Content-Type", `${mime.getType()};charset=utf8`);
        ctx.body = fs.createReadStream(realPath);
      } else {
        let filename = path.join(realPath, "index.html");

        // 如果不存在该文件则执行 catch 中的 next 交给其他中间件处理
        await access(filename);

        // 存在设置文件类型并响应内容
        ctx.set("Content-Type", "text/html; charset=utf8");
        ctx.body = fs.createReadStream(filename);
      }
    } catch (e) {
      await next();
    }
  }
}
```

三、总结

在封装中间件时，每个中间件应足够简单，职责单一。中间件的代码编写应高效，必要时通过缓存重复获取数据。

Node.js中的Web框架如Koa本身比较简洁，但通过中间件的机制能实现各种需要的功能，使得Web应用具备良好的可拓展性和组合性。

通过将公共逻辑处理编写在中间件中，可以避免在每个接口回调中重复编写相同的代码，减少冗余代码，达到类似装饰者模式的效果。

参考文献

- [Middleware - 百度百科](https://baike.baidu.com/item/%E4%B8%AD%E9%97%B4%E4%BB%B6)
(<https://baike.baidu.com/item/%E4%B8%AD%E9%97%B4%E4%BB%B6>)

- [从 Koa 中间件原理浅谈中间件的封装](https://segmentfault.com/a/1190000017897279) (<https://segmentfault.com/a/1190000017897279>)
- [自定义 Koa-bodyparser 中间件](https://www.jianshu.com/p/81b6ebc0dd85) (<https://www.jianshu.com/p/81b6ebc0dd85>)

面试官：React中的key有什么作用？

一、key

首先，先给出 `react` 组件中进行列表渲染的一个示例：

```
const data = [  
  { id: 0, name: 'abc' },  
  { id: 1, name: 'def' },  
  { id: 2, name: 'ghi' },  
  { id: 3, name: 'jkl' }  
];  
  
const ListItem = (props) => {  
  return <li>{props.name}</li>;  
};  
  
const List = () => {  
  return (  
    <ul>  
      {data.map((item) => (  
        <ListItem name={item.name}></ListItem>  
      ))}  
    </ul>  
  );  
};
```

然后在输出就可以看到 `react` 所提示的警告信息：

```
Each child in a list should have a unique "key" prop.
```

根据意思就可以得到渲染列表的每一个子元素都应该需要一个唯一的 `key` 值

在这里可以使用列表的 `id` 属性作为 `key` 值以解决上面这个警告

```
const List = () => {  
  return (  
    <ul>  
      {data.map((item) => (  
        <ListItem name={item.name} key={item.id}></ListItem>  
      ))}  
    </ul>  
  );  
};
```

二、作用

跟 `Vue` 一样，`React` 也存在 `Diff` 算法，而元素 `key` 属性的作用是用于判断元素是新创建的还是被移动的元素，从而减少不必要的元素渲染

因此 `key` 的值需要为每一个元素赋予一个确定的标识

如果列表数据渲染中，在数据后面插入一条数据，`key` 作用并不大，如下：

```
this.state = {
  numbers:[111,222,333]
}

insertMovie() {
  const newMovies = [...this.state.numbers, 444];
  this.setState({
    movies: newMovies
  })
}

<ul>
  {
    this.state.movies.map((item, index) => {
      return <li>{item}</li>
    })
  }
</ul>
```

前面的元素在 `diff` 算法中，前面的元素由于是完全相同的，并不会产生删除创建操作，在最后一个比较的时候，则需要插入到新的 `DOM` 树中

因此，在这种情况下，元素有无 `key` 属性意义并不大

下面再来看看在前面插入数据时，使用 `key` 与不使用 `key` 的区别：

```
insertMovie() {
  const newMovies = [000,...this.state.numbers];
  this.setState({
    movies: newMovies
  })
}
```

当拥有 `key` 的时候，`react` 根据 `key` 属性匹配原有树上的子元素以及最新树上的子元素，像上述情况只需要将 `000` 元素插入到最前面位置

当没有 `key` 的时候，所有的 `li` 标签都需要进行修改

同样，并不是拥有 `key` 值代表性能越高，如果说只是文本内容改变了，不写 `key` 反而性能和效率更高

主要是因为不写 `key` 是将所有的文本内容替换一下，节点不会发生变化

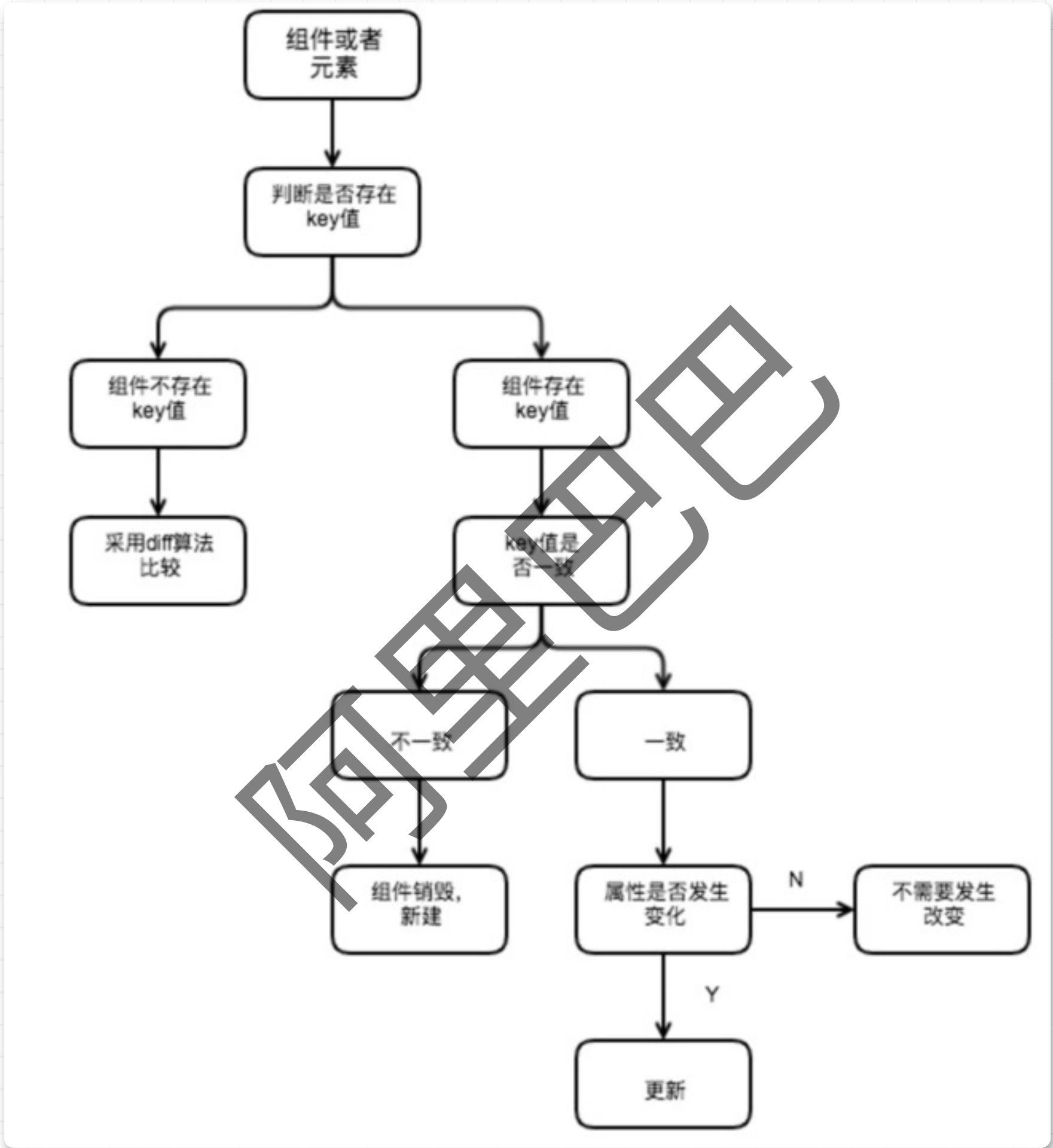
而写 `key` 则涉及到了节点的增和删，发现旧 `key` 不存在了，则将其删除，新 `key` 在之前没有，则插入，这就增加性能的开销

三、总结

良好使用 `key` 属性是性能优化的非常关键的一步，注意事项为：

- `key` 应该是唯一的
- `key` 不要使用随机值（随机数在下次 `render` 时，会重新生成一个数字）
- 使用 `index` 作为 `key` 值，对性能没有优化

`react` 判断 `key` 的流程具体如下图：



参考文献

- <https://zh-hans.reactjs.org/docs/lists-and-keys.html#gatsby-focus-wrapper>
- <https://segmentfault.com/a/1190000017511836>

面试官：React中组件之间如何通信？

一、组件通信

我们将组件间通信可以拆分为两个词：

- 组件
- 通信

组件是 `vue` 中最强大的功能之一，同样组件化是 `React` 的核心思想

相比 `vue`，`React` 的组件更加灵活和多样，按照不同的方式可以分成很多类型的组件

而通信指的是发送者通过某种媒体以某种格式来传递信息到受信者以达到某个目的，广义上，任何信息的交通都是通信

组件间通信即指组件通过某种方式来传递信息以达到某个目的

二、如何通信

组件传递的方式有很多种，根据传送者和接收者可以分为如下：

- 父组件向子组件传递
- 子组件向父组件传递
- 兄弟组件之间的通信
- 父组件向后代组件传递
- 非关系组件传递

父组件向子组件传递

由于 `React` 的数据流动为单向的，父组件向子组件传递是最常见的方式

父组件在调用子组件的时候，只需要在子组件标签内传递参数，子组件通过 `props` 属性就能接收父组件传递过来的参数

```
function EmailInput(props) {
  return (
    <label>
      Email: <input value={props.email} />
    </label>
  );
}

const element = <EmailInput email="123124132@163.com" />;
```

子组件向父组件传递

子组件向父组件通信的基本思路是，父组件向子组件传一个函数，然后通过这个函数的回调，拿到子组件传过来的值

父组件对应代码如下：

```

class Parents extends Component {
  constructor() {
    super();
    this.state = {
      price: 0
    };
  }

  getItemPrice(e) {
    this.setState({
      price: e
    });
  }

  render() {
    return (
      <div>
        <div>price: {this.state.price}</div>
        /* 向子组件中传入一个函数 */
        <Child getPrice={this.getItemPrice.bind(this)} />
      </div>
    );
  }
}

```

子组件对应代码如下：

```

class Child extends Component {
  clickGoods(e) {
    // 在此函数中传入值
    this.props.getPrice(e);
  }

  render() {
    return (
      <div>
        <button onClick={this.clickGoods.bind(this, 100)}>goods1</button>
        <button onClick={this.clickGoods.bind(this, 1000)}>goods2</button>
      </div>
    );
  }
}

```

兄弟组件之间的通信

如果是兄弟组件之间的传递，则父组件作为中间层来实现数据的互通，通过使用父组件传递

```
class Parent extends React.Component {
  constructor(props) {
    super(props)
    this.state = {count: 0}
  }
  setCount = () => {
    this.setState({count: this.state.count + 1})
  }
  render() {
    return (
      <div>
        <SiblingA
          count={this.state.count}
        />
        <SiblingB
          onClick={this.setCount}
        />
      </div>
    );
  }
}
```

父组件向后代组件传递

父组件向后代组件传递数据是一件最普通的事情，就像全局数据一样

使用 `context` 提供了组件之间通讯的一种方式，可以共享数据，其他数据都能读取对应的数据

通过使用 `React.createContext` 创建一个 `context`

```
const PriceContext = React.createContext('price')
```

`context` 创建成功后，其下存在 `Provider` 组件用于创建数据源，`Consumer` 组件用于接收数据，使用实例如下：

`Provider` 组件通过 `value` 属性用于给后代组件传递数据：

```
<PriceContext.Provider value={100}>
</PriceContext.Provider>
```

如果想要获取 `Provider` 传递的数据，可以通过 `Consumer` 组件或者或者使用 `contextType` 属性接收，对应分别如下：

```
class MyClass extends React.Component {
  static contextType = PriceContext;
  render() {
    let price = this.context;
    /* 基于这个值进行渲染工作 */
  }
}
```


Consumer 组件：

```
<PriceContext.Consumer>
  { /*这里是一个函数*/ }
  {
    price => <div>price: {price}</div>
  }
</PriceContext.Consumer>
```

非关系组件传递

如果组件之间关系类型比较复杂的情况，建议将数据进行一个全局资源管理，从而实现通信，例如 `redux`。关于 `redux` 的使用后续再详细介绍

三、总结

由于 `React` 是单向数据流，主要思想是组件不会改变接收的数据，只会监听数据的变化，当数据发生变化时它们会使用接收到的新值，而不是去修改已有的值
因此，可以看到通信过程中，数据的存储位置都是存放在上级位置中

参考文献

- <https://react.docschina.org/docs/context.html>

面试官：React事件绑定的方式有哪些？区别？

一、React事件

在 `react` 应用中，事件名都是用小驼峰格式进行书写，例如 `onclick` 要改写成 `onClick`
最简单的事件绑定如下：

```
class ShowAlert extends React.Component {
  showAlert() {
    console.log("Hi");
  }

  render() {
    return <button onClick={this.showAlert}>show</button>;
  }
}
```

从上面可以看到，事件绑定的方法需要使用 `{}` 包住
上述的代码看似没有问题，但是当将处理函数输出代码换成 `console.log(this)` 的时候，点击按钮，则会发现控制台输出 `undefined`

二、如何绑定

为了解决上面正确输出 `this` 的问题，常见的绑定方式有如下：

- `render`方法中使用`bind`
- `render`方法中使用箭头函数
- `constructor`中`bind`
- 定义阶段使用箭头函数绑定

render方法中使用bind

如果使用一个类组件，在其中给某个组件/元素一个 `onClick` 属性，它现在并会自定绑定其 `this` 到当前组件，解决这个问题方法是在事件函数后使用 `.bind(this)` 将 `this` 绑定到当前组件中

```
class App extends React.Component {
  handleClick() {
    console.log('this > ', this);
  }
  render() {
    return (
      <div onClick={this.handleClick.bind(this)}>test</div>
    )
  }
}
```

这种方式在组件每次 `render` 渲染的时候，都会重新进行 `bind` 的操作，影响性能

render方法中使用箭头函数

通过 `ES6` 的上下文来将 `this` 的指向绑定给当前组件，同样再每一次 `render` 的时候都会生成新的方法，影响性能

```
class App extends React.Component {
  handleClick() {
    console.log('this > ', this);
  }
  render() {
    return (
      <div onClick={e => this.handleClick(e)}>test</div>
    )
  }
}
```

constructor中bind

在 `constructor` 中预先 `bind` 当前组件，可以避免在 `render` 操作中重复绑定

```
class App extends React.Component {
  constructor(props) {
    super(props);
    this.handleClick = this.handleClick.bind(this);
  }
  handleClick() {
    console.log('this >', this);
  }
  render() {
    return (
      <div onClick={this.handleClick}>test</div>
    )
  }
}
```

定义阶段使用箭头函数绑定

跟上述方式三一样，能够在 `render` 操作中重复绑定，实现也非常的简单，如下：

```
class App extends React.Component {
  constructor(props) {
    super(props);
  }
  handleClick = () => {
    console.log('this >', this);
  }
  render() {
    return (
      <div onClick={this.handleClick}>test</div>
    )
  }
}
```

三、区别

上述四种方法的方式，区别主要如下：

- 编写方面：方式一、方式二写法简单，方式三的编写过于冗杂
- 性能方面：方式一和方式二在每次组件render的时候都会生成新的方法实例，性能问题欠缺。若该函数作为属性值传给子组件的时候，都会导致额外的渲染。而方式三、方式四只会生成一个方法实例

综合上述，方式四是最优的事件绑定方式

参考文献

- <https://segmentfault.com/a/1190000011317515>

面试官：React构建组件的方式有哪些？区别？

一、组件

组件就是把图形、非图形的各种逻辑均抽象为一个统一的概念（组件）来实现开发的模式

在 [React](#) 中，一个类、一个函数都可以视为一个组件

- 降低整个系统的耦合度，在保持接口不变的情况下，我们可以替换不同的组件快速完成需求，例如输入框，可以替换为日历、时间、范围等组件作具体的实现
- 调试方便，由于整个系统是通过组件组合起来的，在出现问题的时候，可以用排除法直接移除组件，或者根据报错的组件快速定位问题，之所以能够快速定位，是因为每个组件之间低耦合，职责单一，所以逻辑会比分析整个系统要简单
- 提高可维护性，由于每个组件的职责单一，并且组件在系统中是被复用的，所以对代码进行优化可获得系统的整体升级

二、如何构建

在 [React](#) 目前来讲，组件的创建主要分成了三种方式：

- 函数式创建
- 通过 `React.createClass` 方法创建
- 继承 `React.Component` 创建

函数式创建

在 [React Hooks](#) 出来之前，函数式组件可以视为无状态组件，只负责根据传入的 [props](#) 来展示视图，不涉及对 [state](#) 状态的操作

大多数组件可以写为无状态组件，通过简单组合构建其他组件

在 [React](#) 中，通过函数简单创建组件的示例如下：

```
function HelloComponent(props, /* context */) {  
  return <div>Hello {props.name}</div>  
}
```

通过 `React.createClass` 方法创建

[React.createClass](#) 是 react 刚开始推荐的创建组件的方式，目前这种创建方式已经不怎么用了

像上述通过函数式创建的组件的方式，最终会通过 [babel](#) 转化成 [React.createClass](#) 这种形式，转化成如下：

```
function HelloComponent(props) /* context */{  
  return React.createElement(  
    "div",  
    null,  
    "Hello ",  
    props.name  
  );  
}
```

由于上述的编写方式过于冗杂，目前基本上不使用上

继承 `React.Component` 创建

同样在 `react hooks` 出来之前，有状态的组件只能通过继承 `React.Component` 这种形式进行创建

有状态的组件也就是组件内部存在维护的数据，在类创建的方式中通过 `this.state` 进行访问

当调用 `this.setState` 修改组件的状态时，组件会再次调用 `render()` 方法进行重新渲染

通过继承 `React.Component` 创建一个时钟示例如下：

```
class Timer extends React.Component {
  constructor(props) {
    super(props);
    this.state = { seconds: 0 };
  }

  tick() {
    this.setState(state => ({
      seconds: state.seconds + 1
    }));
  }

  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }

  componentWillUnmount() {
    clearInterval(this.interval);
  }

  render() {
    return (
      <div>
        Seconds: {this.state.seconds}
      </div>
    );
  }
}
```

三、区别

由于 `React.createClass` 创建的方式过于冗杂，并不建议使用

而像函数式创建和类组件创建的区别主要在于需要创建的组件是否需要为有状态组件：

- 对于一些无状态的组件创建，建议使用函数式创建的方式
- 由于 `react hooks` 的出现，函数式组件创建的组件通过使用 `hooks` 方法也能使之成为有状态组件，再加上目前推崇函数式编程，所以这里建议都使用函数式的方式来创建组件

在考虑组件的选择原则上，能用无状态组件则用无状态组件

参考文献

- <https://react.docschina.org/>

面试官：state 和 props 有什么区别？

一、state

一个组件的显示形态可以由数据状态和外部参数所决定，而数据状态就是 `state`，一般在 `constructor` 中初始化

当需要修改里面的值的状态需要通过调用 `setState` 来改变，从而达到更新组件内部数据的作用，并且重新调用组件 `render` 方法，如下面的例子：

```
class Button extends React.Component {
  constructor() {
    super();
    this.state = {
      count: 0,
    };
  }

  updateCount() {
    this.setState((prevState, props) => {
      return { count: prevState.count + 1 };
    });
  }

  render() {
    return (
      <button onClick={() => this.updateCount()}>
        Clicked {this.state.count} times
      </button>
    );
  }
}
```

`setState` 还可以接受第二个参数，它是一个函数，会在 `setState` 调用完成并且组件开始重新渲染时被调用，可以用来监听渲染是否完成

```
this.setState(
  {
    name: "JS每日一题",
  },
  () => console.log("setState finished")
);
```

二、props

`React` 的核心思想就是组件化思想，页面会被切分成一些独立的、可复用的组件

组件从概念上看就是一个函数，可以接受一个参数作为输入值，这个参数就是 `props`，所以可以把 `props` 理解为从外部传入组件内部的数据

`react` 具有单向数据流的特性，所以他的主要作用是从父组件向子组件中传递数据

`props` 除了可以传字符串，数字，还可以传递对象，数组甚至是回调函数，如下：

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello {this.props.name}</h1>;
  }
}

const element = <Welcome name="Sara" onChange={this.handleClick} />;
```

上述 `name` 属性与 `onChange` 方法都能在子组件的 `props` 变量中访问

在子组件中，`props` 在内部不可变的，如果想要改变它看，只能通过外部组件传入新的 `props` 来重新渲染子组件，否则子组件的 `props` 和展示形式不会改变

三、区别

相同点：

- 两者都是 JavaScript 对象
- 两者都是用于保存信息
- `props` 和 `state` 都能触发渲染更新

区别：

- `props` 是外部传递给组件的，而 `state` 是在组件内被组件自己管理的，一般在 `constructor` 中初始化
- `props` 在组件内部是不可修改的，但 `state` 在组件内部可以进行修改
- `state` 是多变的、可以修改

参考文献

- <https://lucybain.com/blog/2016/react-state-vs-props/>
(<https://lucybain.com/blog/2016/react-state-vs-props/>)
- <https://juejin.cn/post/6844904009203974158>
(<https://juejin.cn/post/6844904009203974158>)

面试官：`super()` 和 `super(props)` 有什么区别？

一、ES6 类

在 `ES6` 中，通过 `extends` 关键字实现类的继承，方式如下：

```

class sup {
  constructor(name) {
    this.name = name;
  }

  printName() {
    console.log(this.name);
  }
}

class sub extends sup {
  constructor(name, age) {
    super(name); // super代表的事父类的构造函数
    this.age = age;
  }

  printAge() {
    console.log(this.age);
  }
}

let jack = new sub("jack", 20);
jack.printName(); //输出: jack
jack.printAge(); //输出: 20

```

在上面的例子中，可以看到通过 `super` 关键字实现调用父类，`super` 代替的是父类的构造函数，使用 `super(name)` 相当于调用 `sup.prototype.constructor.call(this, name)`

如果在子类中不使用 `super`，关键字，则会引发报错，如下：

```
ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from derived constructor
```

报错的原因是 子类是没有自己的 `this` 对象的，它只能继承父类的 `this` 对象，然后对其进行加工而 `super()` 就是将父类中的 `this` 对象继承给子类的，没有 `super()` 子类就得不到 `this` 对象如果先调用 `this`，再初始化 `super()`，同样是禁止的行为

```

class sub extends sup {
  constructor(name, age) {
    this.age = age;
    super(name); // super代表的事父类的构造函数
  }
}

```

所以在子类 `constructor` 中，必须先代用 `super` 才能引用 `this`

二、类组件

在 `React` 中，类组件是基于 `ES6` 的规范实现的，继承 `React.Component`，因此如果用到 `constructor` 就必须写 `super()` 才初始化 `this`

这时候，在调用 `super()` 的时候，我们一般都需要传入 `props` 作为参数，如果不传进去，`React` 内部也会将其定义在组件实例中

```
// React 内部
const instance = new YourComponent(props);
instance.props = props;
```

所以无论有没有 `constructor`，在 `render` 中 `this.props` 都是可以使用的，这是 `React` 自动附带的，是可以不写的：

```
class HelloMessage extends React.Component {
  render() {
    return <div>nice to meet you! {this.props.name}</div>;
  }
}
```

但是也不建议使用 `super()` 代替 `super(props)`

因为在 `React` 会在类组件构造函数生成实例后再给 `this.props` 赋值，所以在不传递 `props` 在 `super` 的情况下，调用 `this.props` 为 `undefined`，如下情况：

```
class Button extends React.Component {
  constructor(props) {
    super(); // 没传入 props
    console.log(props); // {}
    console.log(this.props); // undefined
    // ...
  }
}
```

而传入 `props` 的则都能正常访问，确保了 `this.props` 在构造函数执行完毕之前已被赋值，更符合逻辑，如下：

```
class Button extends React.Component {
  constructor(props) {
    super(props); // 没传入 props
    console.log(props); // {}
    console.log(this.props); // {}
    // ...
  }
}
```

三、总结

在 `React` 中，类组件基于 `ES6`，所以在 `constructor` 中必须使用 `super`

在调用 `super` 过程，无论是否传入 `props`，`React` 内部都会将 `props` 赋值给组件实例 `props` 属性中

如果只调用了 `super()`，那么 `this.props` 在 `super()` 和构造函数结束之间仍是 `undefined`

参考文献

- <https://overreacted.io/zh-hans/why-do-we-write-super-props/>
(<https://overreacted.io/zh-hans/why-do-we-write-super-props/>)
- <https://segmentfault.com/q/1010000008340434>
(<https://segmentfault.com/q/1010000008340434>)

面试官：你在React项目中是如何使用Redux的？项目结构是如何划分的？

一、背景

在前面文章了解中，我们了解到 `redux` 是用于数据状态管理，而 `react` 是一个视图层面的库。如果将两者连接在一起，可以使用官方推荐 `react-redux` 库，其具有高效且灵活的特性。

`react-redux` 将组件分成：

- 容器组件：存在逻辑处理
- UI 组件：只负责显示和交互，内部不处理逻辑，状态由外部控制

通过 `redux` 将整个应用状态存储到 `store` 中，组件可以派发 `dispatch` 行为 `action` 给 `store`。

其他组件通过订阅 `store` 中的状态 `state` 来更新自身的视图。

二、如何做

使用 `react-redux` 分成了两大核心：

- `Provider`
- `connection`

Provider

在 `redux` 中存在一个 `store` 用于存储 `state`，如果将这个 `store` 存放在顶层元素中，其他组件都被包裹在顶层元素之上。

那么所有的组件都能够受到 `redux` 的控制，都能够获取到 `redux` 中的数据。

使用方式如下：

```
<Provider store = {store}>  
  <App />  
</Provider>
```

connection

`connect` 方法将 `store` 上的 `getState` 和 `dispatch` 包装成组件的 `props`。

导入 `connect` 如下：

```
import { connect } from "react-redux";
```


用法如下：

```
connect(mapStateToProps, mapDispatchToProps)(MyComponent)
```

可以传递两个参数：

- mapStateToProps
- mapDispatchToProps

mapStateToProps

把 `redux` 中的数据映射到 `react` 中的 `props` 中去

如下：

```
const mapStateToProps = (state) => {  
  return {  
    // prop : state.xxx | 意思是将state中的某个数据映射到props中  
    foo: state.bar  
  }  
}
```

组件内部就能够通过 `props` 获取到 `store` 中的数据

```
class Foo extends Component {  
  constructor(props){  
    super(props);  
  }  
  render(){  
    return(  
      // 这样子渲染的其实就是state.bar的数据了  
      <div>this.props.foo</div>  
    )  
  }  
}  
Foo = connect()(Foo)  
export default Foo
```

mapDispatchToProps

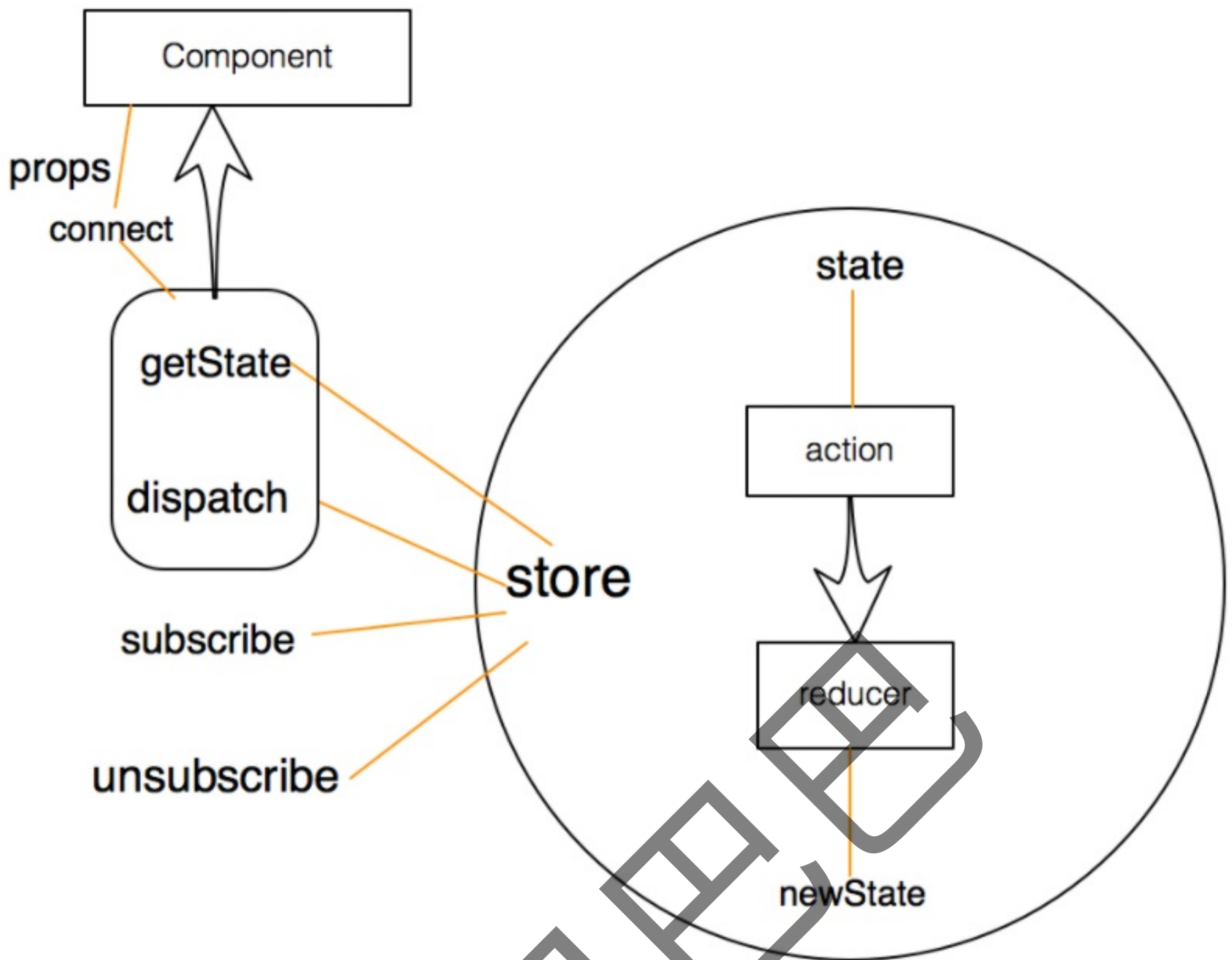
将 `redux` 中的 `dispatch` 映射到组件内部的 `props` 中

```
const mapDispatchToProps = (dispatch) => { // 默认传递参数就是dispatch
  return {
    onClick: () => {
      dispatch({
        type: 'increatment'
      });
    }
  };
}
```

```
class Foo extends Component {
  constructor(props){
    super(props);
  }
  render(){
    return(
      <button onClick = {this.props.onClick}>点击increase</button>
    )
  }
}
Foo = connect()(Foo);
export default Foo;
```

小结

整体流程图大致如下所示：



三、项目结构

可以根据项目具体情况进行选择，以下列出两种常见的组织结构

按角色组织（MVC）

角色如下：

- reducers
- actions
- components
- containers

参考如下：

```
reducers/  
  todoReducer.js  
  filterReducer.js  
actions/  
  todoAction.js  
  filterActions.js  
components/  
  todoList.js  
  todoItem.js  
  filter.js  
containers/  
  todoListContainer.js  
  todoItemContainer.js  
  filterContainer.js
```

按功能组织

使用 `redux` 使用功能组织项目，也就是把完成同一应用功能的代码放在一个目录下，一个应用功能包含多个角色的代码

`Redux` 中，不同的角色就是 `reducer`、`actions` 和视图，而应用功能对应的就是用户界面的交互模块参考如下：

```
todoList/  
  actions.js  
  actionTypes.js  
  index.js  
  reducer.js  
  views/  
    components.js  
    containers.js  
filter/  
  actions.js  
  actionTypes.js  
  index.js  
  reducer.js  
  views/  
    components.js  
    container.js
```

每个功能模块对应一个目录，每个目录下包含同样的角色文件：

- `actionTypes.js` 定义 `action` 类型
- `actions.js` 定义 `action` 构造函数
- `reducer.js` 定义这个功能模块如果响应 `actions.js` 定义的动作
- `views` 包含功能模块中所有的 `React` 组件，包括展示组件和容器组件
- `index.js` 把所有的角色导入，统一导出

其中 `index` 模块用于导出对外的接口

```
import * as actions from './actions.js';
import reducer from './reducer.js';
import view from './views/container.js';

export { actions, reducer, view };
```

导入方法如下：

```
import { actions, reducer, view as TodoList } from './xxxx'
```

参考文献

- <https://www.redux.org.cn/docs/basics/UsageWithReact.html>
- <https://segmentfault.com/a/1190000010384268>

面试官：在react中组件间过渡动画如何实现？

一、动画

在日常开发中，页面切换时的转场动画是比较基础的一个场景

当一个组件在显示与消失过程中存在过渡动画，可以很好的增加用户的体验

在 `react` 中实现过渡动画效果会有很多种选择，如 `react-transition-group`，`react-motion`，`Animated`，以及原生的 `CSS` 都能完成切换动画

二、如何实现

在 `react` 中，`react-transition-group` 是一种很好的解决方案，其为元素添加 `enter`，`enter-active`，`exit`，`exit-active` 这一系列勾子

可以帮助我们方便的实现组件的入场和离场动画

其主要提供了三个主要的组件：

- `CSSTransition`：在前端开发中，结合 `CSS` 来完成过渡动画效果
- `SwitchTransition`：两个组件显示和隐藏切换时，使用该组件
- `TransitionGroup`：将多个动画组件包裹在其中，一般用于列表中元素的动画

CSSTransition

其实现动画的原理在于，当 `CSSTransition` 的 `in` 属性置为 `true` 时，`CSSTransition` 首先会给予子组件加上 `xxx-enter`、`xxx-enter-active` 的 `class` 执行动画

当动画执行结束后，会移除两个 `class`，并且添加 `-enter-done` 的 `class`

所以可以利用这一点，通过 `css` 的 `transition` 属性，让元素在两个状态之间平滑过渡，从而得到相应的动画效果

当 `in` 属性置为 `false` 时，`CSSTransition` 会给予子组件加上 `xxx-exit` 和 `xxx-exit-active` 的 `class`，然后开始执行动画，当动画结束后，移除两个 `class`，然后添加 `-enter-done` 的 `class`

如下例子：


```
export default class App2 extends React.PureComponent {

  state = {show: true};

  onToggle = () => this.setState({show: !this.state.show});

  render() {
    const {show} = this.state;
    return (
      <div className='container'>
        <div className='square-wrapper'>
          <CSSTransition
            in={show}
            timeout={500}
            classNames='fade'
            unmountOnExit={true}
          >
            <div className='square' />
          </CSSTransition>
        </div>
        <Button onClick={this.onToggle}>toggle</Button>
      </div>
    );
  }
}
```

对应 `css` 样式如下：

```
.fade-enter {
  opacity: 0;
  transform: translateX(100%);
}

.fade-enter-active {
  opacity: 1;
  transform: translateX(0);
  transition: all 500ms;
}

.fade-exit {
  opacity: 1;
  transform: translateX(0);
}

.fade-exit-active {
  opacity: 0;
  transform: translateX(-100%);
  transition: all 500ms;
}
```

SwitchTransition

`SwitchTransition` 可以完成两个组件之间切换的炫酷动画

比如有一个按钮需要在 `on` 和 `off` 之间切换，我们希望看到 `on` 先从左侧退出，`off` 再从右侧进入

`SwitchTransition` 中主要有一个属性 `mode`，对应两个值：

- `in-out`：表示新组件先进入，旧组件再移除；
- `out-in`：表示旧组件先移除，新组件再进入

`SwitchTransition` 组件里面要有 `CSSTransition`，不能直接包裹你想要切换的组件

里面的 `CSSTransition` 组件不再像以前那样接受 `in` 属性来判断元素是何种状态，取而代之的是 `key` 属性

下面给出一个按钮入场和出场的示例，如下：

```
import { SwitchTransition, CSSTransition } from "react-transition-group";

export default class SwitchAnimation extends PureComponent {
  constructor(props) {
    super(props);

    this.state = {
      isOn: true
    }
  }

  render() {
    const {isOn} = this.state;

    return (
      <SwitchTransition mode="out-in">
        <CSSTransition classNames="btn"
          timeout={500}
          key={isOn ? "on" : "off"}>
          {
            <button onClick={this.btnClick.bind(this)}>
              {isOn ? "on": "off"}
            </button>
          }
        </CSSTransition>
      </SwitchTransition>
    )
  }

  btnClick() {
    this.setState({isOn: !this.state.isOn})
  }
}
```

`css` 文件对应如下：

```
.btn-enter {
  transform: translate(100%, 0);
  opacity: 0;
}

.btn-enter-active {
  transform: translate(0, 0);
  opacity: 1;
  transition: all 500ms;
}

.btn-exit {
  transform: translate(0, 0);
  opacity: 1;
}

.btn-exit-active {
  transform: translate(-100%, 0);
  opacity: 0;
  transition: all 500ms;
}
```

TransitionGroup

当有一组动画的时候，就可将这些 `CSSTransition` 放入到一个 `TransitionGroup` 中来完成动画

同样 `CSSTransition` 里面没有 `in` 属性，用到了 `key` 属性

`TransitionGroup` 在感知 `children` 发生变化时，先保存移除的节点，当动画结束后才真正移除其处理方式如下：

- 插入的节点，先渲染 dom，然后再做动画
- 删除的节点，先做动画，然后再删除 dom

如下：

```
import React, { PureComponent } from 'react'
import { CSSTransition, TransitionGroup } from 'react-transition-group';

export default class GroupAnimation extends PureComponent {
  constructor(props) {
    super(props);

    this.state = {
      friends: []
    }
  }

  render() {
    return (
      <div>
        <TransitionGroup>
          {
            this.state.friends.map((item, index) => {
              return (
                <CSSTransition classNames="friend" timeout={300} key={index}>
                  <div>{item}</div>
                </CSSTransition>
              )
            })
          }
        </TransitionGroup>
        <button onClick={e => this.addFriend()}>+friend</button>
      </div>
    )
  }

  addFriend() {
    this.setState({
      friends: [...this.state.friends, "coderwhy"]
    })
  }
}
```

对应 [css](#) 如下：

```
.friend-enter {
  transform: translate(100%, 0);
  opacity: 0;
}

.friend-enter-active {
  transform: translate(0, 0);
  opacity: 1;
  transition: all 500ms;
}

.friend-exit {
  transform: translate(0, 0);
  opacity: 1;
}

.friend-exit-active {
  transform: translate(-100%, 0);
  opacity: 0;
  transition: all 500ms;
}
```

参考文献

◦ <https://segmentfault.com/a/11900000018861018>

面试官：说说 React 性能优化的手段有哪些？

一、性能优化

React 凭借 virtual DOM 和 diff 算法拥有高效的性能，但是某些情况下，性能明显可以进一步提高

在前面文章中，我们了解到类组件通过调用 `setState` 方法，就会导致 `render`，父组件一旦发生 `render` 渲染，子组件一定也会执行 `render` 渲染

当我们想要更新一个子组件的时候，如下图绿色部分：

理想状态只调用该路径下的组件 `render`：

但是 `react` 的默认做法是调用所有组件的 `render`，再对生成的虚拟 DOM 进行对比（黄色部分），如不变则不进行更新

从上图可见，黄色部分 `diff` 算法对比是明显的性能浪费的情况

二、如何做

在[React中如何避免不必要的render \(https://mp.weixin.qq.com/s/h4NX4Plr6TCj0IhlawiJTg\)](https://mp.weixin.qq.com/s/h4NX4Plr6TCj0IhlawiJTg)中，我们了解到如何避免不必要的 `render` 来应付上面的问题，主要手段是通过 `shouldComponentUpdate`、`PureComponent`、`React.memo`，这三种形式这里就不再复述除此之外， 常见性能优化常见的手段有如下：

- 避免使用内联函数
- 使用 `React Fragments` 避免额外标记
- 使用 `Immutable`
- 懒加载组件
- 事件绑定方式
- 服务端渲染

避免使用内联函数

如果我们使用内联函数，则每次调用 `render` 函数时都会创建一个新的函数实例，如下：

```
import React from "react";

export default class InlineFunctionComponent extends React.Component {
  render() {
    return (
      <div>
        <h1>Welcome Guest</h1>
        <input type="button" onClick={(e) => { this.setState({inputValue: e.target.value}) }} value="Click For Inline Function" />
      </div>
    )
  }
}
```

我们应该在组件内部创建一个函数，并将事件绑定到该函数本身。这样每次调用 `render` 时就不会创建单独的函数实例，如下：

```
import React from "react";

export default class InlineFunctionComponent extends React.Component {

  setNewStateData = (event) => {
    this.setState({
      inputValue: e.target.value
    })
  }

  render() {
    return (
      <div>
        <h1>Welcome Guest</h1>
        <input type="button" onClick={this.setNewStateData} value="Click For Inline Function" />
      </div>
    )
  }
}
```

使用 React Fragments 避免额外标记

用户创建新组件时，每个组件应具有单个父标签。父级不能有两个标签，所以顶部要有一个公共标签，所以我们经常在组件顶部添加额外标签 `div`

这个额外标签除了充当父标签之外，并没有其他作用，这时候则可以使用 `fragment`

其不会向组件引入任何额外标记，但它可以作为父级标签的作用，如下所示：

```
export default class NestedRoutingComponent extends React.Component {
  render() {
    return (
      <>
        <h1>This is the Header Component</h1>
        <h2>Welcome To Demo Page</h2>
      </>
    )
  }
}
```

事件绑定方式

在[事件绑定方式 \(https://mp.weixin.qq.com/s/VfQ34ZEPXUXsimzMaj_41A\)](https://mp.weixin.qq.com/s/VfQ34ZEPXUXsimzMaj_41A)中，我们了解到四种事件绑定的方式

从性能方面考虑，在 `render` 方法中使用 `bind` 和 `render` 方法中使用箭头函数这两种形式在每次组件 `render` 的时候都会生成新的方法实例，性能欠缺

而 `constructor` 中 `bind` 事件与定义阶段使用箭头函数绑定这两种形式只会生成一个方法实例，性能方面会有所改善

使用 Immutable

在[理解Immutable中](https://mp.weixin.qq.com/s/laYJ_KNa8M5JNBnIoIMDAA) (https://mp.weixin.qq.com/s/laYJ_KNa8M5JNBnIoIMDAA), 我们了解到使用 `Immutable` 可以给 `React` 应用带来性能的优化, 主要体现在减少渲染的次数

在做 `react` 性能优化的时候, 为了避免重复渲染, 我们会在 `shouldComponentUpdate()` 中做对比, 当返回 `true` 执行 `render` 方法

`Immutable` 通过 `is` 方法则可以完成对比, 而无需像一样通过深度比较的方式比较

懒加载组件

从工程方面考虑, `webpack` 存在代码拆分能力, 可以为应用创建多个包, 并在运行时动态加载, 减少初始包的大小

而在 `react` 中使用到了 `Suspense` 和 `lazy` 组件实现代码拆分功能, 基本使用如下:

```
const johanComponent = React.lazy(() => import(/* webpackChunkName: "johanComponent" */
  './myAwesome.component'));

export const johanAsyncComponent = props => (
  <React.Suspense fallback=<Spinner />>
    <johanComponent {...props} />
  </React.Suspense>
);
```

服务端渲染

采用服务端渲染端方式, 可以使用户更快的看到渲染完成的页面

服务端渲染, 需要起一个 `node` 服务, 可以使用 `express`、`koa` 等, 调用 `react` 的 `renderToString` 方法, 将根组件渲染成字符串, 再输出到响应中

例如:

```
import { renderToString } from "react-dom/server";
import MyPage from "./MyPage";
app.get("/", (req, res) => {
  res.write("<!DOCTYPE html><html><head><title>My Page</title></head><body>");
  res.write("<div id='content'>");
  res.write(renderToString(<MyPage />));
  res.write("</div></body></html>");
  res.end();
});
```

客户端使用 `render` 方法来生成 HTML

```
import ReactDOM from 'react-dom';
import MyPage from './MyPage';
ReactDOM.render(<MyPage />, document.getElementById('app'));
```

其他

除此之外, 还存在的优化手段有组件拆分、合理使用 `hooks` 等性能优化手段...

三、总结

通过上面初步学习，我们了解到 `react` 常见的性能优化可以分成三个层面：

- 代码层面
- 工程层面
- 框架机制层面

通过这三个层面的优化结合，能够使基于 `react` 项目的性能更上一层楼

参考文献

- <https://zhuanlan.zhihu.com/p/108666350>
- <https://segmentfault.com/a/1190000007811296>

面试官：说说 React 生命周期有哪些不同阶段？每个阶段对应的方法是什么？

一、生命周期

生命周期 (`Life Cycle`) 的概念应用很广泛，特别是在经济、环境、技术、社会等诸多领域经常出现，其基本涵义可以通俗地理解为“从摇篮到坟墓” (`Cradle-to-Grave`) 的整个过程

跟 `Vue` 一样，`React` 整个组件生命周期包括从创建、初始化数据、编译模板、挂载 Dom → 渲染、更新 → 渲染、卸载等一系列过程

二、流程

这里主要讲述 `react16.4` 之后的生命周期，可以分成三个阶段：

- 创建阶段
- 更新阶段
- 卸载阶段

创建阶段

创建阶段主要分成了以下几个生命周期方法：

- `constructor`
- `getDerivedStateFromProps`
- `render`
- `componentDidMount`

`constructor`

实例过程中自动调用的方法，在方法内部通过 `super` 关键字获取来自父组件的 `props`

在该方法中，通常的操作为初始化 `state` 状态或者在 `this` 上挂载方法

`getDerivedStateFromProps`

该方法是新增的生命周期方法，是一个静态的方法，因此不能访问到组件的实例

执行时机：组件创建和更新阶段，不论是 `props` 变化还是 `state` 变化，也会调用

在每次 `render` 方法前调用，第一个参数为即将更新的 `props`，第二个参数为上一个状态的 `state`，可以比较 `props` 和 `state` 来加一些限制条件，防止无用的 `state` 更新

该方法需要返回一个新的对象作为新的 `state` 或者返回 `null` 表示 `state` 状态不需要更新

render

类组件必须实现的方法，用于渲染 `DOM` 结构，可以访问组件 `state` 与 `prop` 属性

注意：不要在 `render` 里面 `setState`，否则会触发死循环导致内存崩溃

componentDidMount

组件挂载到真实 `DOM` 节点后执行，其在 `render` 方法之后执行

此方法多用于执行一些数据获取，事件监听等操作

更新阶段

该阶段的函数主要为如下方法：

- `getDerivedStateFromProps`
- `shouldComponentUpdate`
- `render`
- `getSnapshotBeforeUpdate`
- `componentDidUpdate`

getDerivedStateFromProps

该方法介绍同上

shouldComponentUpdate

用于告知组件本身基于当前的 `props` 和 `state` 是否需要重新渲染组件，默认情况返回 `true`

执行时机：到新的 `props` 或者 `state` 时都会调用，通过返回 `true` 或者 `false` 告知组件更新与否

一般情况，不建议在该周期方法中进行深层比较，会影响效率

同时也不能调用 `setState`，否则会导致无限循环调用更新

render

介绍如上

getSnapshotBeforeUpdate

该周期函数在 `render` 后执行，执行之时 `DOM` 元素还没有被更新

该方法返回的一个 `Snapshot` 值，作为 `componentDidUpdate` 第三个参数传入


```
getSnapshotBeforeUpdate(prevProps, prevState) {
  console.log( '#enter getSnapshotBeforeUpdate' );
  return 'foo';
}

componentDidUpdate(prevProps, prevState, snapshot) {
  console.log( '#enter componentDidUpdate snapshot = ', snapshot );
}
```

此方法的目的在于获取组件更新前的一些信息，比如组件的滚动位置之类的，在组件更新后可以根据这些信息恢复一些UI视觉上的状态

componentDidUpdate

执行时机：组件更新结束后触发

在该方法中，可以根据前后的 `props` 和 `state` 的变化做相应的操作，如获取数据，修改 `DOM` 样式等

卸载阶段

componentWillUnmount

此方法用于组件卸载前，清理一些注册是监听事件，或者取消订阅的网络请求等
一旦一个组件实例被卸载，其不会被再次挂载，而只可能是被重新创建

三、总结

新版生命周期整体流程如下图所示：

旧的生命周期流程图如下：

Uploading image.png...

通过两个图的对比，可以发现新版的生命周期减少了以下三种方法：

- `componentWillMount`
- `componentWillReceiveProps`
- `componentWillUpdate`

其实这三个方法仍然存在，只是在前者加上了 `UNSAFE_` 前缀，如 `UNSAFE_componentWillMount`，并不像字面意思那样表示不安全，而是表示这些生命周期的代码可能在未来的 `react` 版本可能废除

同时也新增了两个生命周期函数：

- `getDerivedStateFromProps`
- `getSnapshotBeforeUpdate`

参考文献

- <https://github.com/pomelovico/keep/issues/23>
- <https://segmentfault.com/a/1190000020268993>

面试官：说说 React 中的 setState 执行机制

state

一个组件的显示形态可以由数据状态和外部参数所决定，而数据状态就是 `state`

当需要修改里面的值的状态需要通过调用 `setState` 来改变，从而达到更新组件内部数据的作用

如下例子：

```
import React, { Component } from 'react'

export default class App extends Component {
  constructor(props) {
    super(props);

    this.state = {
      message: "Hello World"
    }
  }

  render() {
    return (
      <div>
        <h2>{this.state.message}</h2>
        <button onClick={e => this.changeText()}>面试官系列</button>
      </div>
    )
  }

  changeText() {
    this.setState({
      message: "JS每日一题"
    })
  }
}
```

通过点击按钮触发 `onclick` 事件，执行 `this.setState` 方法更新 `state` 状态，然后重新执行 `render` 函数，从而导致页面的视图更新

如果直接修改 `state` 的状态，如下：

```
changeText() {
  this.state.message = "你好啊,李银河";
}
```

我们会发现页面并不会有任何反应，但是 `state` 的状态是已经发生了改变

这是因为 `React` 并不像 `vue2` 中调用 `Object.defineProperty` 数据响应式或者 `Vue3` 调用 `Proxy` 监听数据的变化

必须通过 `setState` 方法来告知 `react` 组件 `state` 已经发生了改变

关于 `state` 方法的定义是从 `React.Component` 中继承，定义的源码如下：

```
Component.prototype.setState = function(partialState, callback) {
  invariant(
    typeof partialState === 'object' ||
    typeof partialState === 'function' ||
    partialState == null,
    'setState(...): takes an object of state variables to update or a ' +
    'function which returns an object of state variables.',
  );
  this.updater.enqueueSetState(this, partialState, callback, 'setState');
};
```

从上面可以看到 `setState` 第一个参数可以是一个对象，或者是一个函数，而第二个参数是一个回调函数，用于可以实时的获取到更新之后的数据

更新类型

在使用 `setState` 更新数据的时候，`setState` 的更新类型分成：

- 异步更新
- 同步更新

异步更新

先举出一个例子：

```
changeText() {
  this.setState({
    message: "你好啊"
  })
  console.log(this.state.message); // Hello World
}
```

从上面可以看到，最终打印结果为 `Hello world`，并不能在执行完 `setState` 之后立马拿到最新的 `state` 的结果

如果想要立刻获取更新后的值，在第二个参数的回调中更新后会执行

```
changeText() {
  this.setState({
    message: "你好啊"
  }, () => {
    console.log(this.state.message); // 你好啊
  });
}
```

同步更新

同样先给出一个在 `setTimeout` 中更新的例子：

```
changeText() {
  setTimeout(() => {
    this.setState({
      message: "你好啊"
    });
    console.log(this.state.message); // 你好啊
  }, 0);
}
```

上面的例子中，可以看到更新是同步

再来举一个原生 **DOM** 事件的例子：

```
componentDidMount() {
  const btnEl = document.getElementById("btn");
  btnEl.addEventListener('click', () => {
    this.setState({
      message: "你好啊,李银河"
    });
    console.log(this.state.message); // 你好啊,李银河
  })
}
```

小结

- 在组件生命周期或React合成事件中，setState是异步
- 在setTimeout或者原生dom事件中，setState是同步

三、批量更新

同样先给出一个例子：

```
handleClick = () => {
  this.setState({
    count: this.state.count + 1,
  })
  console.log(this.state.count) // 1

  this.setState({
    count: this.state.count + 1,
  })
  console.log(this.state.count) // 1

  this.setState({
    count: this.state.count + 1,
  })
  console.log(this.state.count) // 1
}
```

点击按钮触发事件，打印的都是 1，页面显示 **count** 的值为 2

对同一个值进行多次 `setState`，`setState` 的批量更新策略会对其进行覆盖，取最后一次的执行结果

上述的例子，实际等价于如下：

```
Object.assign(
  previousState,
  {index: state.count+ 1},
  {index: state.count+ 1},
  ...
)
```

由于后面的数据会覆盖前面的更改，所以最终只加了一次

如果是下一个 `state` 依赖前一个 `state` 的话，推荐给 `setState` 一个参数传入一个 `function`，如下：

```
onClick = () => {
  this.setState((prevState, props) => {
    return {count: prevState.count + 1};
  });
  this.setState((prevState, props) => {
    return {count: prevState.count + 1};
  });
}
```

而在 `setTimeout` 或者原生 `dom` 事件中，由于是同步的操作，所以并不会进行覆盖现象

参考文献

- <https://juejin.cn/post/6844903667426918408>
- <https://juejin.cn/post/6844903636749778958>
- <https://segmentfault.com/a/11900000039077904>

面试官：说说 Real DOM 和 Virtual DOM 的区别？优缺点？

一、DOM

Real DOM，真实 `DOM`，意思为文档对象模型，是一个结构化文本的抽象，在页面渲染出的每一个结点都是一个真实 `DOM` 结构，如下：

`Virtual Dom`，本质上是以 `JavaScript` 对象形式存在的对 `DOM` 的描述

创建虚拟 `DOM` 目的就是为了更好将虚拟的节点渲染到页面视图中，虚拟 `DOM` 对象的节点与真实 `DOM` 的属性一一照应

在 `React` 中，`JSX` 是其一大特性，可以让你在 `JS` 中通过使用 `XML` 的方式去直接声明界面的 `DOM` 结构


```
// 创建 h1 标签, 右边千万不能加引号
const vDom = <h1>Hello World</h1>;
// 找到 <div id="root"></div> 节点
const root = document.getElementById("root");
// 把创建的 h1 标签渲染到 root 节点上
ReactDOM.render(vDom, root);
```

上述中, `ReactDOM.render()` 用于将你创建好的虚拟 `DOM` 节点插入到某个真实节点上, 并渲染到页面上

`JSX` 实际是一种语法糖, 在使用过程中会被 `babel` 进行编译转化成 `JS` 代码, 上述 `VDOM` 转化为如下:

```
const vDom = React.createElement(
  'h1',
  { className: 'hClass', id: 'hId' },
  'hello world'
)
```

可以看到, `JSX` 就是为了简化直接调用 `React.createElement()` 方法:

- 第一个参数是标签名, 例如 `h1`、`span`、`table`...
- 第二个参数是个对象, 里面存着标签的一些属性, 例如 `id`、`class` 等
- 第三个参数是节点中的文本

通过 `console.log(VDOM)`, 则能够得到虚拟 `VDOM` 消息

所以可以得到, `JSX` 通过 `babel` 的方式转化成 `React.createElement` 执行, 返回值是一个对象, 也就是虚拟 `DOM`

二、区别

两者的区别如下:

- 虚拟 `DOM` 不会进行排版与重绘操作, 而真实 `DOM` 会频繁重排与重绘
- 虚拟 `DOM` 的总损耗是“虚拟 `DOM` 增删改+真实 `DOM` 差异增删改+排版与重绘”, 真实 `DOM` 的总损耗是“真实 `DOM` 完全增删改+排版与重绘”

拿[以前文章](https://mp.weixin.qq.com/s?biz=MzU1OTgxNDQ1Nw==&mid=2247484516&idx=1&sn=965a4ce32bf93adb9ed112922c) (<https://mp.weixin.qq.com/s?biz=MzU1OTgxNDQ1Nw==&mid=2247484516&idx=1&sn=965a4ce32bf93adb9ed112922c>)

举过的例子:

传统的原生 `api` 或 `jQuery` 去操作 `DOM` 时, 浏览器会从构建 `DOM` 树开始从头到尾执行一遍流程

当你在一次操作时, 需要更新 10 个 `DOM` 节点, 浏览器没这么智能, 收到第一个更新 `DOM` 请求后, 并不知道后续还有 9 次更新操作, 因此会马上执行流程, 最终执行 10 次流程

而通过 `VNode`, 同样更新 10 个 `DOM` 节点, 虚拟 `DOM` 不会立即操作 `DOM`, 而是将这 10 次更新的 `diff` 内容保存到本地的一个 `js` 对象中, 最终将这个 `js` 对象一次性 `attach` 到 `DOM` 树上, 避免大量的无谓计算

三、优缺点

真实 **DOM** 的优势：

- 易用

缺点：

- 效率低，解析速度慢，内存占用量过高
- 性能差：频繁操作真实 DOM，易于导致重绘与回流

使用虚拟 **DOM** 的优势如下：

- 简单方便：如果使用手动操作真实 **DOM** 来完成页面，繁琐又容易出错，在大规模应用下维护起来也很困难
- 性能方面：使用 Virtual DOM，能够有效避免真实 DOM 数频繁更新，减少多次引起重绘与回流，提高性能
- 跨平台：React 借助虚拟 DOM，带来了跨平台的能力，一套代码多端运行

缺点：

- 在一些性能要求极高的应用中虚拟 DOM 无法进行针对性的极致优化
- 首次渲染大量 DOM 时，由于多了一层虚拟 DOM 的计算，速度比正常稍慢

参考文献

- <https://juejin.cn/post/6844904052971536391>
(<https://juejin.cn/post/6844904052971536391>)
- <https://www.html.cn/qa/other/22832.html>
(<https://www.html.cn/qa/other/22832.html>)

面试官：说说React JSX转换成真实DOM过程？

一、React JSX

react 通过将组件编写的 **JSX** 映射到屏幕，以及组件中的状态发生了变化之后 **React** 会将这些「变化」更新到屏幕上

在前面文章了解中，**JSX** 通过 **babel** 最终转化成 **React.createElement** 这种形式，例如：

```
<div>
  
  <Hello />
</div>
```

会被 **babel** 转化成如下：

```
React.createElement(  
  "div",  
  null,  
  React.createElement("img", {  
    src: "avatar.png",  
    className: "profile"  
  }),  
  React.createElement(Hello, null)  
);
```

在转化过程中，`babel` 在编译时会判断 JSX 中组件的首字母：

- 当首字母为小写时，其被认定为原生 `DOM` 标签，`createElement` 的第一个变量被编译为字符串
- 当首字母为大写时，其被认定为自定义组件，`createElement` 的第一个变量被编译为对象

最终都会通过 `ReactDOM.render(...)` 方法进行挂载，如下：

```
ReactDOM.render(<App />, document.getElementById("root"));
```

二、过程

在 `react` 中，节点大致可以分成四个类别：

- 原生标签节点
- 文本节点
- 函数组件
- 类组件

如下所示：

```
class ClassComponent extends Component {
  static defaultProps = {
    color: "pink"
  };
  render() {
    return (
      <div className="border">
        <h3>ClassComponent</h3>
        <p className={this.props.color}>{this.props.name}</p >
      </div>
    );
  }
}

function FunctionComponent(props) {
  return (
    <div className="border">
      FunctionComponent
      <p>{props.name}</p >
    </div>
  );
}

const jsx = (
  <div className="border">
    <p>xx</p >
    <a href=" ">xxx</ a>
    <FunctionComponent name="函数组件" />
    <ClassComponent name="类组件" color="red" />
  </div>
);
```

这些类别最终都会被转化成 `React.createElement` 这种形式

`React.createElement` 其被调用时会传入标签类型 `type`，标签属性 `props` 及若干子元素 `children`，作用是生成一个虚拟 `Dom` 对象，如下所示：

```
function createElement(type, config, ...children) {
  if (config) {
    delete config.__self;
    delete config.__source;
  }
  // ! 源码中做了详细处理, 比如过滤掉key、ref等
  const props = {
    ...config,
    children: children.map(child =>
      typeof child === "object" ? child : createTextNode(child)
    )
  };
  return {
    type,
    props
  };
}

function createTextNode(text) {
  return {
    type: TEXT,
    props: {
      children: [],
      nodeValue: text
    }
  };
}

export default {
  createElement
};
```

`createElement` 会根据传入的节点信息进行一个判断：

- 如果是原生标签节点，`type` 是字符串，如 `div`、`span`
- 如果是文本节点，`type` 就没有，这里是 `TEXT`
- 如果是函数组件，`type` 是函数名
- 如果是类组件，`type` 是类名

虚拟 `DOM` 会通过 `ReactDOM.render` 进行渲染成真实 `DOM`，使用方法如下：

```
ReactDOM.render(element, container[, callback])
```

当首次调用时，容器节点里的所有 `DOM` 元素都会被替换，后续的调用则会使用 `React` 的 `diff` 算法进行高效的更新

如果提供了可选的回调函数 `callback`，该回调将在组件被渲染或更新之后被执行

`render` 大致实现方法如下：

```
function render(vnode, container) {
  console.log("vnode", vnode); // 虚拟DOM对象
  // vnode _> node
  const node = createNode(vnode, container);
  container.appendChild(node);
}
```



```

}

// 创建真实DOM节点
function createNode(vnode, parentNode) {
  let node = null;
  const {type, props} = vnode;
  if (type === TEXT) {
    node = document.createTextNode("");
  } else if (typeof type === "string") {
    node = document.createElement(type);
  } else if (typeof type === "function") {
    node = type.isReactComponent
      ? updateClassComponent(vnode, parentNode)
      : updateFunctionComponent(vnode, parentNode);
  } else {
    node = document.createDocumentFragment();
  }
  reconcileChildren(props.children, node);
  updateNode(node, props);
  return node;
}

// 遍历下子vnode, 然后把子vnode->真实DOM节点, 再插入父node中
function reconcileChildren(children, node) {
  for (let i = 0; i < children.length; i++) {
    let child = children[i];
    if (Array.isArray(child)) {
      for (let j = 0; j < child.length; j++) {
        render(child[j], node);
      }
    } else {
      render(child, node);
    }
  }
}

function updateNode(node, nextVal) {
  Object.keys(nextVal)
    .filter(k => k !== "children")
    .forEach(k => {
      if (k.slice(0, 2) === "on") {
        let eventName = k.slice(2).toLocaleLowerCase();
        node.addEventListener(eventName, nextVal[k]);
      } else {
        node[k] = nextVal[k];
      }
    });
}

// 返回真实dom节点
// 执行函数
function updateFunctionComponent(vnode, parentNode) {
  const {type, props} = vnode;
  let vvnode = type(props);

```

```
const node = createNode(vvnode, parentNode);
return node;
}

// 返回真实dom节点
// 先实例化, 再执行render函数
function updateClassComponent(vnode, parentNode) {
  const {type, props} = vnode;
  let cmp = new type(props);
  const vvnode = cmp.render();
  const node = createNode(vvnode, parentNode);
  return node;
}
export default {
  render
};
```

三、总结

在 `react` 源码中, 虚拟 `Dom` 转化成真实 `Dom` 整体流程如下图所示:

其渲染流程如下所示:

- 使用 `React.createElement` 或 `JSX` 编写 `React` 组件, 实际上所有的 `JSX` 代码最后都会转换成 `React.createElement(...)`, `Babel` 帮助我们完成了这个转换的过程。
- `createElement` 函数对 `key` 和 `ref` 等特殊的 `props` 进行处理, 并获取 `defaultProps` 对默认 `props` 进行赋值, 并且对传入的孩子节点进行处理, 最终构造成一个虚拟 `DOM` 对象
- `ReactDOM.render` 将生成好的虚拟 `DOM` 渲染到指定容器上, 其中采用了批处理、事务等机制并且对特定浏览器进行了性能优化, 最终转换为真实 `DOM`

参考文献

- <https://bbs.huaweicloud.com/blogs/265503>)
- <https://huang-qing.github.io/react/2019/05/29/React-VirDom/>
- <https://segmentfault.com/a/1190000018891454>

面试官：说说 `React Router` 有几种模式？实现原理？

一、`React Router`

在单页应用中, 一个 `web` 项目只有一个 `html` 页面, 一旦页面加载完成之后, 就不用因为用户的操作而进行页面的重新加载或者跳转, 其特性如下:

- 改变 `url` 且不让浏览器像服务器发送请求
- 在不刷新页面的前提下动态改变浏览器地址栏中的 `URL` 地址

其中主要分成了两种模式:

- `hash` 模式: 在 `url` 后面加上 `#`, 如 `http://127.0.0.1:5500/home/#/page1`
- `history` 模式: 允许操作浏览器的曾经在标签页或者框架里访问的会话历史记录

二、使用

`React Router`对应的 `hash` 模式和 `history` 模式对应的组件为：

- `HashRouter`
- `BrowserRouter`

这两个组件的使用都十分的简单，作为最顶层组件包裹其他组件，如下所示

```
// 1.import { BrowserRouter as Router } from "react-router-dom";  
// 2.import { HashRouter as Router } from "react-router-dom";
```

```
import React from 'react';  
import {  
  BrowserRouter as Router,  
  // HashRouter as Router  
  Switch,  
  Route,  
} from "react-router-dom";  
import Home from './pages/Home';  
import Login from './pages/Login';  
import Backend from './pages/Backend';  
import Admin from './pages/Admin';
```

```
function App() {  
  return (  
    <Router>  
      <Route path="/login" component={Login}/>  
      <Route path="/backend" component={Backend}/>  
      <Route path="/admin" component={Admin}/>  
      <Route path="/" component={Home}/>  
    </Router>  
  );  
}  
  
export default App;
```

三、实现原理

路由描述了 `URL` 与 `UI` 之间的映射关系，这种映射是单向的，即 `URL` 变化引起 `UI` 更新（无需刷新页面）

下面以 `hash` 模式为例子，改变 `hash` 值并不会导致浏览器向服务器发送请求，浏览器不发出请求，也就不会刷新页面

`hash` 值改变，触发全局 `window` 对象上的 `hashchange` 事件。所以 `hash` 模式路由就是利用 `hashchange` 事件监听 `URL` 的变化，从而进行 `DOM` 操作来模拟页面跳转

`react-router` 也是基于这个特性实现路由的跳转

下面以 `HashRouter` 组件分析进行展开：

HashRouter

HashRouter 包裹了整应用，

通过 `window.addEventListener('hashChange',callback)` 监听 `hash` 值的变化，并传递给其嵌套的组件

然后通过 `context` 将 `location` 数据往后代组件传递，如下：

```
import React, { Component } from 'react';
import { Provider } from './context'
// 该组件下Api提供给子组件使用
class HashRouter extends Component {
  constructor() {
    super()
    this.state = {
      location: {
        pathname: window.location.hash.slice(1) || '/'
      }
    }
  }
  // url路径变化 改变location
  componentDidMount() {
    window.location.hash = window.location.hash || '/'
    window.addEventListener('hashchange', () => {
      this.setState({
        location: {
          ...this.state.location,
          pathname: window.location.hash.slice(1) || '/'
        }
      }, () => console.log(this.state.location))
    })
  }
  render() {
    let value = {
      location: this.state.location
    }
    return (
      <Provider value={value}>
        {
          this.props.children
        }
      </Provider>
    );
  }
}

export default HashRouter;
```

`Router` 组件主要做的是通过 `BrowserRouter` 传过来的当前值，通过 `props` 传进来的 `path` 与 `context` 传进来的 `pathname` 进行匹配，然后决定是否执行渲染组件

```
import React, { Component } from 'react';
import { Consumer } from './context'
const { pathToRegexp } = require("path-to-regexp");
class Route extends Component {
  render() {
    return (
      <Consumer>
      {
        state => {
          console.log(state)
          let {path, component: Component} = this.props
          let pathname = state.location.pathname
          let reg = pathToRegexp(path, [], {end: false})
          // 判断当前path是否包含pathname
          if(pathname.match(reg)) {
            return <Component></Component>
          }
          return null
        }
      }
    </Consumer>
    );
  }
}
export default Route;
```

参考文献

- <https://juejin.cn/post/6870376090297171975#heading-9>
- <https://segmentfault.com/a/1190000023560665>

面试官：说说React diff的原理是什么？

一、Virtual DOM

跟 `Vue` 一致，`React` 通过引入 `Virtual DOM` 的概念，极大地避免无效的 `Dom` 操作，使我们的页面的构建效率提到了极大的提升

而 `diff` 算法就是更高效地通过对比新旧 `Virtual DOM` 来找出真正的 `Dom` 变化之处

传统 `diff` 算法通过循环递归对节点进行依次对比，效率低下，算法复杂度达到 $O(n^3)$ ，`react` 将算法进行一个优化，复杂度姜维 $O(n)$ ，两者效率差距如下图：

二、原理

react 中 diff 算法主要遵循三个层级的策略：

- tree 层级
- component 层级
- element 层级

tree 层级

DOM 节点跨层级的操作不做优化，只会对相同层级的节点进行比较

只有删除、创建操作，没有移动操作，如下图：

react 发现新树中，R 节点下没有了 A，那么直接删除 A，在 D 节点下创建 A 以及下属节点

上述操作中，只有删除和创建操作

component 层级

如果是同一个类的组件，则会继续往下 diff 运算，如果不是一个类的组件，那么直接删除这个组件下的所有子节点，创建新的

当 component D 换成了 component G 后，即使两者的结构非常类似，也会将 D 删除再重新创建 G

element 层级

对于比较同一层级的节点们，每个节点在对应的层级用唯一的 key 作为标识

提供了 3 种节点操作，分别为 INSERT_MARKUP (插入)、MOVE_EXISTING (移动)和 REMOVE_NODE (删除)

如下场景：

通过 key 可以准确地发现新旧集合中的节点都是相同的节点，因此无需进行节点删除和创建，只需要将旧集合中节点的位置进行移动，更新为新集合中节点的位置

流程如下表：

- index：新集合的遍历下标。
- oldIndex：当前节点在老集合中的下标
- maxIndex：在新集合访问过的节点中，其在老集合的最大下标

如果当前节点在新集合中的位置比老集合中的位置靠前的话，是不会影响后续节点操作的，这里这时候被动字节不用动

操作过程中只比较 oldIndex 和 maxIndex，规则如下：

- 当 oldIndex > maxIndex 时，将 oldIndex 的值赋值给 maxIndex
- 当 oldIndex = maxIndex 时，不操作
- 当 oldIndex < maxIndex 时，将当前节点移动到 index 的位置

diff 过程如下：

- 节点B：此时 `maxIndex=0`，`oldIndex=1`；满足 `maxIndex < oldIndex`，因此B节点不动，此时`maxIndex= Math.max(oldIndex, maxIndex)`，就是1
- 节点A：此时`maxIndex=1`，`oldIndex=0`；不满足`maxIndex < oldIndex`，因此A节点进行移动操作，此时`maxIndex= Math.max(oldIndex, maxIndex)`，还是1
- 节点D：此时`maxIndex=1`，`oldIndex=3`；满足`maxIndex < oldIndex`，因此D节点不动，此时`maxIndex= Math.max(oldIndex, maxIndex)`，就是3
- 节点C：此时`maxIndex=3`，`oldIndex=2`；不满足`maxIndex < oldIndex`，因此C节点进行移动操作，当前已经比较完了

当ABCD节点比较完成后，`diff` 过程还没完，还会整体遍历老集合中节点，看有没有没用到的节点，有的话，就删除

三、注意事项

对于简单列表渲染而言，不使用 `key` 比使用 `key` 的性能，例如：

将一个`[1,2,3,4,5]`，渲染成如下的样子：

```
<div>1</div>
<div>2</div>
<div>3</div>
<div>4</div>
<div>5</div>
```

后续更改成`[1,3,2,5,4]`，使用 `key` 与不使用 `key` 作用如下：

```
1.加key
<div key='1'>1</div>      <div key='1'>1</div>
<div key='2'>2</div>      <div key='3'>3</div>
<div key='3'>3</div> =====> <div key='2'>2</div>
<div key='4'>4</div>      <div key='5'>5</div>
<div key='5'>5</div>      <div key='4'>4</div>
操作：节点2移动至下标为2的位置，节点4移动至下标为4的位置。

2.不加key
<div>1</div>      <div>1</div>
<div>2</div>      <div>3</div>
<div>3</div> =====> <div>2</div>
<div>4</div>      <div>5</div>
<div>5</div>      <div>4</div>
操作：修改第1个到第5个节点的innerText
```

如果我们对这个集合进行增删的操作改成`[1,3,2,5,6]`

1.加key

```
<div key='1'>1</div>      <div key='1'>1</div>
<div key='2'>2</div>      <div key='3'>3</div>
<div key='3'>3</div> =====> <div key='2'>2</div>
<div key='4'>4</div>      <div key='5'>5</div>
<div key='5'>5</div>      <div key='6'>6</div>
```

操作：节点2移动至下标为2的位置，新增节点6至下标为4的位置，删除节点4。

2.不加key

```
<div>1</div>      <div>1</div>
<div>2</div>      <div>3</div>
<div>3</div> =====> <div>2</div>
<div>4</div>      <div>5</div>
<div>5</div>      <div>6</div>
```

操作：修改第1个到第5个节点的innerText

由于 `dom` 节点的移动操作开销是比较昂贵的，没有 `key` 的情况下要比有 `key` 的性能更好

参考文献

- <https://zhuanlan.zhihu.com/p/140489744>
- <https://zhuanlan.zhihu.com/p/20346379>

面试官：说说React render方法的原理？在什么时候会被触发？

一、原理

首先，`render` 函数在 `react` 中有两种形式：

在类组件中，指的是 `render` 方法：

```
class Foo extends React.Component {
  render() {
    return <h1> Foo </h1>;
  }
}
```

在函数组件中，指的是函数组件本身：

```
function Foo() {
  return <h1> Foo </h1>;
}
```

在 `render` 中，我们会编写 `jsx`，`jsx` 通过 `babel` 编译后就会转化成我们熟悉的 `js` 格式，如下：

```
return (  
  <div className='cn'>  
    <Header> hello </Header>  
    <div> start </div>  
    Right Reserve  
  </div>  
)
```

babel 编译后：

```
return (  
  React.createElement(  
    'div',  
    {  
      className : 'cn'  
    },  
    React.createElement(  
      Header,  
      null,  
      'hello'  
    ),  
    React.createElement(  
      'div',  
      null,  
      'start'  
    ),  
    'Right Reserve'  
  )  
)
```

从名字上来看，createElement 方法用来元素的

在react中，这个元素就是虚拟DOM树的节点，接收三个参数：

- type：标签
- attributes：标签属性，若无则为null
- children：标签的子节点

这些虚拟DOM树最终会渲染成真实DOM

在render过程中，React 将新调用的render函数返回的树与旧版本的树进行比较，这一步是决定如何更新DOM的必要步骤，然后进行diff比较，更新DOM树

二、触发时机

render的执行时机主要分成了两部分：

- 类组件调用 setState 修改状态

```

class Foo extends React.Component {
  state = { count: 0 };

  increment = () => {
    const { count } = this.state;

    const newCount = count < 10 ? count + 1 : count;

    this.setState({ count: newCount });
  };

  render() {
    const { count } = this.state;
    console.log("Foo render");

    return (
      <div>
        <h1> {count} </h1>
        <button onClick={this.increment}>Increment</button>
      </div>
    );
  }
}

```

点击按钮，则调用 `setState` 方法，无论 `count` 发生变化辩护，控制台都会输出 `Foo render`，证明 `render` 执行了

- 函数组件通过 `useState hook` 修改状态

```

function Foo() {
  const [count, setCount] = useState(3);

  function increment() {
    const newCount = count < 10 ? count + 1 : count;
    setCount(newCount);
  }

  console.log("Foo render");

  return (
    <div>
      <h1> {count} </h1>
      <button onClick={increment}>Increment</button>
    </div>
  );
}

```

函数组件通过 `useState` 这种形式更新数据，当数组的值不发生改变，就不会触发 `render`

- 类组件重新渲染


```

class App extends React.Component {
  state = { name: "App" };
  render() {
    return (
      <div className="App">
        <Foo />
        <button onClick={() => this.setState({ name: "App" })}>
          Change name
        </button>
      </div>
    );
  }
}

function Foo() {
  console.log("Foo render");

  return (
    <div>
      <h1> Foo </h1>
    </div>
  );
}

```

只要点击了 `App` 组件内的 `Change name` 按钮，不管 `Foo` 具体实现是什么，都会被重新 `render` 渲染

- 函数组件重新渲染

```

function App(){
  const [name,setName] = useState('App')

  return (
    <div className="App">
      <Foo />
      <button onClick={() => setName("aaa")}>
        { name }
      </button>
    </div>
  )
}

function Foo() {
  console.log("Foo render");

  return (
    <div>
      <h1> Foo </h1>
    </div>
  );
}

```

可以发现，使用 `useState` 来更新状态的时候，只有首次会触发 `Foo render`，后面并不会导致 `Foo render`

三、总结

`render` 函数里面可以编写 `JSX`，转化成 `createElement` 这种形式，用于生成虚拟 `DOM`，最终转化成真实 `DOM`

在 `React` 中，类组件只要执行了 `setState` 方法，就一定会触发 `render` 函数执行，函数组件使用 `useState` 更改状态不一定导致重新 `render`

组件的 `props` 改变了，不一定触发 `render` 函数的执行，但是如果 `props` 的值来自于父组件或者祖先组件的 `state`

在这种情况下，父组件或者祖先组件的 `state` 发生了改变，就会导致子组件的重新渲染

所以，一旦执行了 `setState` 就会执行 `render` 方法，`useState` 会判断当前值有无发生改变确定是否执行 `render` 方法，一旦父组件发生渲染，子组件也会渲染

参考文献

- <https://zhuanlan.zhihu.com/p/45091185>
- <https://juejin.cn/post/6844904181493415950>

面试官：说说React服务端渲染怎么做？原理是什么？

一、SSR

在SSR中，我们了解到 `Server-Side Rendering`，简称 `SSR`，意为服务端渲染

指由服务侧完成页面的 `HTML` 结构拼接的页面处理技术，发送到浏览器，然后为其绑定状态与事件，成为完全可交互页面的过程

其解决的问题主要有两个：

- SEO，由于搜索引擎爬虫抓取工具可以直接查看完全渲染的页面
- 加速首屏加载，解决首屏白屏问题

二、如何做

在 `react` 中，实现 `SSR` 主要有两种形式：

- 手动搭建一个 `SSR` 框架
- 使用成熟的 `SSR` 框架，如 `Next.js`

这里主要以手动搭建一个 `SSR` 框架进行实现

首先通过 `express` 启动一个 `app.js` 文件，用于监听3000端口的请求，当请求根目录时，返回 `HTML`，如下：

```
const express = require('express')
const app = express()
app.get('/', (req,res) => res.send(`
<html>
  <head>
    <title>ssr demo</title>
  </head>
  <body>
    Hello world
  </body>
</html>
`))

app.listen(3000, () => console.log('Exampleapp listening on port 3000!'))
```

然后再服务器中编写 `react` 代码，在 `app.js` 中进行应引用

```
import React from 'react'

const Home = () =>{

  return <div>home</div>

}

export default Home
```

为了让服务器能够识别 `JSX`，这里需要使用 `webpalc` 对项目进行打包转换，创建一个配置文件 `webpack.server.js` 并进行相关配置，如下：

```

const path = require('path') //node的path模块
const nodeExternals = require('webpack-node-externals')

module.exports = {
  target: 'node',
  mode: 'development', //开发模式
  entry: './app.js', //入口
  output: { //打包出口
    filename: 'bundle.js', //打包后的文件名
    path: path.resolve(__dirname, 'build') //存放到根目录的build文件夹
  },
  externals: [nodeExternals()], //保持node中require的引用方式
  module: {
    rules: [{ //打包规则
      test: /\.js?$/, //对所有js文件进行打包
      loader: 'babel-loader', //使用babel-loader进行打包
      exclude: /node_modules/, //不打包node_modules中的js文件
      options: {
        presets: ['react', 'stage-0', 'env', {
          //loader时额外的打包规则,对react,JSX, ES6进行转换
          targets: {
            browsers: ['last 2 versions'] //对主流浏览器最近两个版本进行兼容
          }
        }
      }
    }
  ]
}

```

接着借助 `react-dom` 提供了服务端渲染的 `renderToString` 方法，负责把 `React` 组件解析成 `html`

```

import express from 'express'
import React from 'react' //引入React以支持JSX的语法
import { renderToString } from 'react-dom/server' //引入renderToString方法
import Home from './src/containers/Home'

const app = express()
const content = renderToString(<Home/>)
app.get('/', (req, res) => res.send(
  <html>
    <head>
      <title>ssr demo</title>
    </head>
    <body>
      ${content}
    </body>
  </html>
))

app.listen(3001, () => console.log('Example app listening on port 3001!'))

```

上面的过程中，已经能够成功将组件渲染到了页面上

但是像一些事件处理的方法，是无法在服务端完成，因此需要将组件代码在浏览器中再执行一遍，这种服务器端和客户端共用一套代码的方式就称之为「同构」

重构通俗讲就是一套React代码在服务器上运行一遍，到达浏览器又运行一遍：

- 服务端渲染完成页面结构
- 浏览器端渲染完成事件绑定

浏览器实现事件绑定的方式为让浏览器去拉取JS文件执行，让JS代码来控制，因此需要引入script标签

通过script标签为页面引入客户端执行的react代码，并通过express的static中间件为js文件配置路由，修改如下：

```
import express from 'express'
import React from 'react'//引入React以支持JSX的语法
import { renderToString } from 'react-dom/server'//引入renderToString方法
import Home from './src/containers/Home'

const app = express()
app.use(express.static('public'));
//使用express提供的static中间件,中间件会将所有静态文件的路由指向public文件夹
const content = renderToString(<Home/>)

app.get('/',(req,res)=>res.send(`
<html>
  <head>
    <title>ssr demo</title>
  </head>
  <body>
    ${content}
    <script src="/index.js"></script>
  </body>
</html>
`))

app.listen(3001, () =>console.log('Example app listening on port 3001!'))
```

然后再客户端执行以下react代码，新建webpack.client.js作为客户端React代码的webpack配置文件如下：


```

const path = require('path') //node的path模块

module.exports = {
  mode: 'development', //开发模式
  entry: './src/client/index.js', //入口
  output: { //打包出口
    filename: 'index.js', //打包后的文件名
    path: path.resolve(__dirname, 'public') //存放到根目录的build文件夹
  },
  module: {
    rules: [ //打包规则
      {
        test: /\.js?$/, //对所有js文件进行打包
        loader: 'babel-loader', //使用babel-loader进行打包
        exclude: /node_modules/, //不打包node_modules中的js文件
        options: {
          presets: ['react', 'stage-0'], ['env', {
            //loader时额外的打包规则,这里对react,JSX进行转换
            targets: {
              browsers: ['last 2 versions'] //对主流浏览器最近两个版本进行兼容
            }
          }
        ]
      }
    ]
  }
}

```

这种方法就能够简单实现首页的 `react` 服务端渲染，过程对应如下图：

在做完初始渲染的时候，一个应用会存在路由的情况，配置信息如下：

```

import React from 'react' //引入React以支持JSX
import { Route } from 'react-router-dom' //引入路由
import Home from './containers/Home' //引入Home组件

export default (
  <div>
    <Route path="/" exact component={Home}></Route>
  </div>
)

```

然后通过 `index.js` 引用路由信息，如下：

```
import React from 'react'
import ReactDOM from 'react-dom'
import { BrowserRouter } from 'react-router-dom'
import Router from './Routers'

const App = () => {
  return (
    <BrowserRouter>
      {Router}
    </BrowserRouter>
  )
}

ReactDOM.hydrate(<App />, document.getElementById('root'))
```

这时候控制台会存在报错信息，原因在于每个 `Route` 组件外面包裹着一层 `div`，但服务端返回的代码中并没有这个 `div`

解决方法只需要将路由信息在服务端执行一遍，使用 `StaticRouter` 来替代 `BrowserRouter`，通过 `context` 进行参数传递

```
import express from 'express'
import React from 'react'//引入React以支持JSX的语法
import { renderToString } from 'react-dom/server'//引入renderToString方法
import { StaticRouter } from 'react-router-dom'
import Router from './Routers'

const app = express()
app.use(express.static('public'));
//使用express提供的static中间件,中间件会将所有静态文件的路由指向public文件夹

app.get('/',(req,res)=>{
  const content = renderToString((
    //传入当前path
    //context为必填参数,用于服务端渲染参数传递
    <StaticRouter location={req.path} context={}>
      {Router}
    </StaticRouter>
  ))
  res.send(`
    <html>
      <head>
        <title>ssr demo</title>
      </head>
      <body>
        <div id="root">${content}</div>
        <script src="/index.js"></script>
      </body>
    </html>
  `)
})

app.listen(3001, () => console.log('Exampleapp listening on port 3001!'))
```

这样也就完成了路由的服务端渲染

三、原理

整体 `react` 服务端渲染原理并不复杂，具体如下：

`node server` 接收客户端请求，得到当前的请求 `url` 路径，然后在已有的路由表内查找到对应的组件，拿到需要请求的数据，将数据作为 `props`、`context` 或者 `store` 形式传入组件

然后基于 `react` 内置的服务端渲染方法 `renderToString()` 把组件渲染为 `html` 字符串在把最终的 `html` 进行输出前需要将数据注入到浏览器端

浏览器开始进行渲染和节点对比，然后执行完成组件内事件绑定和一些交互，浏览器重用了服务端输出的 `html` 节点，整个流程结束

参考文献

- <https://zhuanlan.zhihu.com/p/52693113>

- <https://segmentfault.com/a/1190000020417285>
- <https://juejin.cn/post/6844904000387563533#heading-14>

面试官：说说React的事件机制？

一、事件

React 基于浏览器的事件机制自身实现了一套事件机制，包括事件注册、事件的合成、事件冒泡、事件派发等

在 **React** 中这套事件机制被称之为合成事件

合成事件（SyntheticEvent）

合成事件是 **React** 模拟原生 **DOM** 事件所有能力的一个事件对象，即浏览器原生事件的跨浏览器包装器

根据 **W3C** 规范来定义合成事件，兼容所有浏览器，拥有与浏览器原生事件相同的接口，例如：

```
const button = <button onClick={handleClick}>按钮</button>
```

如果想要获得原生 **DOM** 事件，可以通过 `e.nativeEvent` 属性获取

```
const handleClick = (e) => console.log(e.nativeEvent);  
const button = <button onClick={handleClick}>按钮</button>
```

从上面可以看到 **React** 事件和原生事件也非常的相似，但也有一定的区别：

- 事件名称命名方式不同

```
// 原生事件绑定方式  
<button onclick="handleClick()">按钮命名</button>  
  
// React 合成事件绑定方式  
const button = <button onClick={handleClick}>按钮命名</button>
```

- 事件处理函数书写不同

```
// 原生事件 事件处理函数写法  
<button onclick="handleClick()">按钮命名</button>  
  
// React 合成事件 事件处理函数写法  
const button = <button onClick={handleClick}>按钮命名</button>
```

虽然 `onclick` 看似绑定到 **DOM** 元素上，但实际并不会把事件代理函数直接绑定到真实的节点上，而是把所有的事件绑定到结构的最外层，使用一个统一的事件去监听

这个事件监听器上维持了一个映射来保存所有组件内部的事件监听和处理函数。当组件挂载或卸载时，只是在这个统一的事件监听器上插入或删除一些对象

当事件发生时，首先被这个统一的事件监听器处理，然后在映射里找到真正的事件处理函数并调用。这样做简化了事件处理和回收机制，效率也有很大提升

二、执行顺序

关于 `React` 合成事件与原生事件执行顺序，可以看看下面一个例子：

```
import React from 'react';
class App extends React.Component{

  constructor(props) {
    super(props);
    this.parentRef = React.createRef();
    this.childRef = React.createRef();
  }
  componentDidMount() {
    console.log("React componentDidMount! ");
    this.parentRef.current?.addEventListener("click", () => {
      console.log("原生事件：父元素 DOM 事件监听！");
    });
    this.childRef.current?.addEventListener("click", () => {
      console.log("原生事件：子元素 DOM 事件监听！");
    });
    document.addEventListener("click", (e) => {
      console.log("原生事件：document DOM 事件监听！");
    });
  }
  parentClickFun = () => {
    console.log("React 事件：父元素事件监听！");
  };
  childClickFun = () => {
    console.log("React 事件：子元素事件监听！");
  };
  render() {
    return (
      <div ref={this.parentRef} onClick={this.parentClickFun}>
        <div ref={this.childRef} onClick={this.childClickFun}>
          分析事件执行顺序
        </div>
      </div>
    );
  }
}
export default App;
```

输出顺序为：

```
原生事件：子元素 DOM 事件监听！
原生事件：父元素 DOM 事件监听！
React 事件：子元素事件监听！
React 事件：父元素事件监听！
原生事件：document DOM 事件监听！
```

可以得出以下结论：

- React 所有事件都挂载在 document 对象上
- 当真实 DOM 元素触发事件，会冒泡到 document 对象后，再处理 React 事件
- 所以会先执行原生事件，然后处理 React 事件
- 最后真正执行 document 上挂载的事件

对应过程如图所示：

所以想要阻止不同时间段的冒泡行为，对应使用不同的方法，对应如下：

- 阻止合成事件间的冒泡，用 `e.stopPropagation()`
- 阻止合成事件与最外层 document 上的事件间的冒泡，用 `e.nativeEvent.stopImmediatePropagation()`
- 阻止合成事件与除最外层 document 上的原生事件上的冒泡，通过判断 `e.target` 来避免

```
document.body.addEventListener('click', e => {
  if (e.target && e.target.matches('div.code')) {
    return;
  }
  this.setState({ active: false, }); });
}
```

三、总结

React 事件机制总结如下：

- React 上注册的事件最终会绑定在 document 这个 DOM 上，而不是 React 组件对应的 DOM (减少内存开销就是因为所有的事件都绑定在 document 上，其他节点没有绑定事件)
- React 自身实现了一套事件冒泡机制，所以这也就是为什么我们 `event.stopPropagation()` 无效的原因。
- React 通过队列的形式，从触发的组件向父组件回溯，然后调用他们 JSX 中定义的 callback
- React 有一套自己的合成事件 SyntheticEvent

参考文献

- <https://zh-hans.reactjs.org/docs/events.html>
- https://segmentfault.com/a/1190000015725214?utm_source=sf-similar-article
- <https://segmentfault.com/a/1190000038251163>

面试官：说说react中引入css的方式有哪几种？区别？

一、css方案

组件式开发选择合适的 `css` 解决方案尤为重要

通常会遵循以下规则：

- 可以编写局部 `css`，不会随意污染其他组件内的原生；
- 可以编写动态的 `css`，可以获取当前组件的一些状态，根据状态的变化生成不同的 `css` 样式；
- 支持所有的 `css` 特性：伪类、动画、媒体查询等；
- 编写起来简洁方便、最好符合一贯的 `css` 风格特点

在这一方面，`vue`使用`css`起来更为简洁：

- 通过 `style` 标签编写样式
- `scoped` 属性决定编写的样式是否局部有效
- `lang` 属性设置预处理器
- 内联样式风格的方式来根据最新状态设置和改变`css`

而在`react`中，引入`CSS`就不如`Vue`方便简洁，其引入`css`的方式有很多种，各有利弊

二、方式

常见的`CSS`引入方式有以下：

- 在组件内直接使用
- 组件中引入 `.css` 文件
- 组件中引入 `.module.css` 文件
- `CSS in JS`

在组件内直接使用

直接在组件中书写`css`样式，通过`style`属性直接引入，如下：

```
import React, { Component } from "react";

const div1 = {
  width: "300px",
  margin: "30px auto",
  backgroundColor: "#44014C", //驼峰法
  minHeight: "200px",
  boxSizing: "border-box"
};

class Test extends Component {
  constructor(props, context) {
    super(props);
  }

  render() {
    return (
      <div>
        <div style={div1}>123</div>
        <div style={{backgroundColor:"red"}}>
        </div>
      </div>
    );
  }
}

export default Test;
```

上面可以看到，`css`属性需要转换成驼峰写法

这种方式优点：

- 内联样式，样式之间不会有冲突

- 可以动态获取当前state中的状态

缺点：

- 写法上都需要使用驼峰标识
- 某些样式没有提示
- 大量的样式，代码混乱
- 某些样式无法编写(比如伪类/伪元素)

组件中引入css文件

将 `css` 单独写在一个 `css` 文件中，然后在组件中直接引入

`App.css` 文件：

```
.title {
  color: red;
  font-size: 20px;
}

.desc {
  color: green;
  text-decoration: underline;
}
```

组件中引入：

```
import React, { PureComponent } from 'react';

import Home from './Home';

import './App.css';

export default class App extends PureComponent {
  render() {
    return (
      <div className="app">
        <h2 className="title">我是App的标题</h2>
        <p className="desc">我是App中的一段文字描述</p>
        <Home/>
      </div>
    )
  }
}
```

这种方式存在不好的地方在于样式是全局生效，样式之间会互相影响

组件中引入 `.module.css` 文件

将 `css` 文件作为一个模块引入，这个模块中的所有 `css`，只作用于当前组件。不会影响当前组件的后代组件

这种方式是 `webpack` 特工的方案，只需要配置 `webpack` 配置文件中 `modules:true` 即可

```
import React, { PureComponent } from 'react';

import Home from './Home';

import './App.module.css';

export default class App extends PureComponent {
  render() {
    return (
      <div className="app">
        <h2 className="title">我是App的标题</h2>
        <p className="desc">我是App中的一段文字描述</p>
        <Home/>
      </div>
    )
  }
}
```

这种方式能够解决局部作用域问题，但也有一定的缺陷：

- 引用的类名，不能使用连接符(.xxx-xx)，在 JavaScript 中是不识别的
- 所有的 className 都必须使用 {style.className} 的形式来编写
- 不方便动态来修改某些样式，依然需要使用内联样式的方式；

CSS in JS

CSS-in-JS，是指一种模式，其中 CSS 由 JavaScript 生成而不是在外部文件中定义

此功能并不是 React 的一部分，而是由第三方库提供，例如：

- styled-components
- emotion
- glamorous

下面主要看看 styled-components 的基本使用

本质是通过函数的调用，最终创建一个组件：

- 这个组件会被自动添加上一个不重复的 class
- styled-components 会给该 class 添加相关的样式

基本使用如下：

创建一个 style.js 文件用于存放样式组件：

```
export const SelfLink = styled.div`
  height: 50px;
  border: 1px solid red;
  color: yellow;
`;

export const SelfButton = styled.div`
  height: 150px;
  width: 150px;
  color: ${props => props.color};
  background-image: url(${props => props.src});
  background-size: 150px 150px;
`;
```

引入样式组件也很简单：

```
import React, { Component } from "react";

import { SelfLink, SelfButton } from "../style";

class Test extends Component {
  constructor(props, context) {
    super(props);
  }

  render() {
    return (
      <div>
        <SelfLink title="People's Republic of China">app.js</SelfLink>
        <SelfButton color="palevioletred" style={{ color: "pink" }} src={list}>
          SelfButton
        </SelfButton>
      </div>
    );
  }
}

export default Test;
```

三、区别

通过上面四种样式的引入，可以看到：

- 在组件内直接使用 `css` 该方式编写方便，容易能够根据状态修改样式属性，但是大量的演示编写容易导致代码混乱
- 组件中引入 `.css` 文件符合我们日常的编写习惯，但是作用域是全局的，样式之间会层叠
- 引入 `.module.css` 文件能够解决局部作用域问题，但是不方便动态修改样式，需要使用内联的方式进行样式的编写
- 通过 `css in js` 这种方法，可以满足大部分场景的应用，可以类似于预处理器一样样式嵌套、定义、修改状态等

至于使用 `react` 用哪种方案引入 `css`，并没有一个绝对的答案，可以根据各自情况选择合适的方案

参考文献

- <https://zh-hans.reactjs.org/docs/faq-styling.html#gatsby-focus-wrapper>
- <https://mp.weixin.qq.com/s/oywTpNKEikMXn8QTBglTow>

面试官：说说你在React项目是如何捕获错误的？

一、React错误

错误在我们日常编写代码是非常常见的

举个例子，在 `react` 项目中去编写组件内 `JavaScript` 代码错误会导致 `React` 的内部状态被破坏，导致整个应用崩溃，这是不应该出现的现象

作为一个框架，`react` 也有自身对于错误的处理的解决方案

二、如何做

为了解决出现的错误导致整个应用崩溃的问题，`react16` 引入了「错误边界」新的概念

错误边界是一种 `React` 组件，这种组件可以捕获发生在其子组件树任何位置的 `JavaScript` 错误，并打印这些错误，同时展示降级 `UI`，而并不会渲染那些发生崩溃的子组件树

错误边界在渲染期间、生命周期方法和整个组件树的构造函数中捕获错误

形成错误边界组件的两个条件：

- 使用了 `static getDerivedStateFromError()`
- 使用了 `componentDidCatch()`

抛出错误后，请使用 `static getDerivedStateFromError()` 渲染备用 `UI`，使用 `componentDidCatch()` 打印错误信息，如下：

```

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    // 更新 state 使下一次渲染能够显示降级后的 UI
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    // 你同样可以将错误日志上报给服务器
    logErrorToMyService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      // 你可以自定义降级后的 UI 并渲染
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}

```

然后就可以把自身组件的作为错误边界的子组件，如下：

```

<ErrorBoundary>
  <MyWidget />
</ErrorBoundary>

```

下面这些情况无法捕获到异常：

- 事件处理
- 异步代码
- 服务端渲染
- 自身抛出来的错误

在 [react 16](#) 版本之后，会把渲染期间发生的所有错误打印到控制台

除了错误信息和 JavaScript 栈外，React 16 还提供了组件栈追踪。现在你可以准确地查看发生在组件树内的错误信息：

可以看到在错误信息下方文字中存在一个组件栈，便于我们追踪错误

对于错误边界无法捕获的异常，如事件处理过程中发生问题并不会捕获到，是因为其不会在渲染期间触发，并不会导致渲染时候问题

这种情况可以使用 [js](#) 的 [try...catch...](#) 语法，如下：

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = { error: null };
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    try {
      // 执行操作，如有错误则会抛出
    } catch (error) {
      this.setState({ error });
    }
  }

  render() {
    if (this.state.error) {
      return <h1>Caught an error.</h1>
    }
    return <button onClick={this.handleClick}>Click Me</button>
  }
}
```

除此之外还可以通过监听 `onerror` 事件

```
window.addEventListener('error', function(event) { ... })
```

参考文献

- <https://zh-hans.reactjs.org/docs/error-boundaries.html>

面试官：说说你在使用React 过程中遇到的常见问题？如何解决？

一、前言

在使用 `react` 开发项目过程中，每个人或多或少都会遇到一些"奇怪"的问题，本质上都是我们对其理解的不够透彻

`react` 系列，33个工作日，33次凌晨还在亮起的台灯，到今天就圆满画上句号了，比心

在系列中我们列出了很多比较经典的考题，工作中遇到的问题也往往就藏中其中，只是以不同的表现形式存在罢了

今天的题解不算题解，准确来说是对整个系列的一次贯穿，总结

目录：

- react 有什么特性
- 生命周期有哪些不同阶段？每个阶段对应的方法是？
- state 和 props有什么区别？
- `super()`和`super(props)`有什么区别？
- `setState`执行机制？

- React的事件机制?
- 事件绑定的方式有哪些?
- 构建组件的方式有哪些? 区别?
- 组件之间如何通信?
- key有什么作用?
- refs 的理解? 应用场景?
- Hooks的理解? 解决了什么问题?
- 如何引入css?
- redux工作原理?
- redux中间件有哪些?
- react-router组件有哪些?
- render触发时机?
- 如何减少render?
- JSX转化DOM过程?
- 性能优化手段有哪些
- 如何做服务端渲染?

react 有什么特性

主要的特性分为:

- JSX语法
- 单向数据绑定
- 虚拟DOM
- 声明式编程
- Component

借助这些特性, **react** 整体使用起来更加简单高效, 组件式开发提高了代码的复用率

生命周期有哪些不同阶段? 每个阶段对应的方法是?

主要分成了新的生命周期和旧的生命周期:

- 新版生命周期整体流程如下图所示:

旧的生命周期流程图如下:

state 和 props有什么区别?

两者相同点:

- 两者都是 JavaScript 对象
- 两者都是用于保存信息
- props 和 state 都能触发渲染更新

区别:

- props 是外部传递给组件的, 而 state 是在组件内被组件自己管理的, 一般在 constructor 中初始化
- props 在组件内部是不可修改的, 但 state 在组件内部可以进行修改
- state 是多变的、可以修改

super()和super(props)有什么区别?

在 **React** 中, 类组件基于 **ES6**, 所以在 **constructor** 中必须使用 **super**

在调用 **super** 过程, 无论是否传入 **props**, **React** 内部都会将 **props** 赋值给组件实例 **props** 属性中

如果只调用了 `super()`，那么 `this.props` 在 `super()` 和构造函数结束之间仍是 `undefined`

setState执行机制？

在 `react` 类组件的状态需要通过 `setState` 进行更改，在不同场景下对应不同的执行顺序：

- 在组件生命周期或React合成事件中，`setState`是异步
- 在`setTimeout`或者原生dom事件中，`setState`是同步

当我们批量更改 `state` 的值的时候，`react` 内部会将其进行覆盖，只取最后一次的执行结果

当需要下一个 `state` 依赖当前 `state` 的时候，则可以在 `setState` 中传递一个回调函数进行下次更新

React的事件机制？

`React` 基于浏览器的事件机制自身实现了一套事件机制，包括事件注册、事件的合成、事件冒泡、事件派发等

组件注册的事件最终会绑定在 `document` 这个 `DOM` 上，而不是 `React` 组件对应的 `DOM`，从而节省内存开销

自身实现了一套事件冒泡机制，阻止不同时间段的冒泡行为，需要对应使用不同的方法

事件绑定的方式有哪些？

`react` 常见的绑定方式有如下：

- `render`方法中使用`bind`
- `render`方法中使用箭头函数
- `constructor`中`bind`
- 定义阶段使用箭头函数绑定

前两种方式在每次组件 `render` 的时候都会生成新的方法实例，性能问题欠缺

构建组件的方式有哪些？区别？

组件的创建主要分成了三种方式：

- 函数式创建
- 继承 `React.Component` 创建
- 通过 `React.createClass` 方法创建

如今一般都是前两种方式，对于一些无状态的组件创建，建议使用函数式创建的方式，再比如 `hooks` 的机制下，函数式组件能做类组件对应的的事情，所以建议都使用函数式的方式来创建组件

组件之间如何通信？

组件间通信可以通过 `props`、传递回调函数、`context`、`redux` 等形式进行组件之间通讯

key有什么作用？

使用 `key` 是 `react` 性能优化的手段，在一系列数据最前面插入元素，如果没有 `key` 的值，则所有的元素都需要进行更换，而有 `key` 的情况只需要将最新元素插入到前面，不涉及删除操作

在使用 `key` 的时候应保证：

- `key` 应该是唯一的

- key不要使用随机值（随机数在下一次 render 时，会重新生成一个数字）
- 避免使用 index 作为 key

refs 的理解？应用场景？

Refs 允许我们访问 DOM 节点或在 render 方法中创建的 React 元素

下面的场景使用 refs 非常有用：

- 对Dom元素的焦点控制、内容选择、控制
- 对Dom元素的内容设置及媒体播放
- 对Dom元素的操作和对组件实例的操作
- 集成第三方 DOM 库

Hooks的理解？解决了什么问题？

Hook 是 React 16.8 的新增特性。它可以让你在不编写 class 的情况下使用 state 以及其他的 React 特性

解决问题如下：

- 难以重用和共享组件中的与状态相关的逻辑
- 逻辑复杂的组件难以开发与维护，当我们的组件需要处理多个互不相关的 local state 时，每个生命周期函数中可能会包含着各种互不相关的逻辑在里面
- 类组件中的this增加学习成本，类组件在基于现有工具的优化上存在些许问题
- 由于业务变动，函数组件不得不改为类组件等等

如何引入css？

常见的 CSS 引入方式有以下：

- 在组件内直接使用
- 组件中引入 .css 文件
- 组件中引入 .module.css 文件
- CSS in JS

组件内直接使用 css 会导致大量的代码，而文件中直接引入 css 文件是全局作用域，发生层叠

引入 .module.css 文件能够解决局部作用域问题，但是不方便动态修改样式，需要使用内联的方式进行样式的编写

css in js 这种方法，可以满足大部分场景的应用，可以类似于预处理器一样样式嵌套、定义、修改状态等

redux工作原理？

redux 要求我们把数据都放在 store 公共存储空间

一个组件改变了 store 里的数据内容，其他组件就能感知到 store 的变化，再来取数据，从而间接的实现了这些数据传递的功能

工作流程图如下所示：

redux中间件有哪些？

市面上有很多优秀的 redux 中间件，如：

- redux-thunk：用于异步操作

- `redux-logger`: 用于日志记录

react-router组件有哪些?

常见的组件有:

- `BrowserRouter`、`HashRouter`
- `Route`
- `Link`、`NavLink`
- `switch`
- `redirect`

render触发时机?

在 `React` 中, 类组件只要执行了 `setState` 方法, 就一定会触发 `render` 函数执行

函数组件 `useState` 会判断当前值有无发生改变确定是否执行 `render` 方法, 一旦父组件发生渲染, 子组件也会渲染

如何减少render?

父组件渲染导致子组件渲染, 子组件并没有发生任何改变, 这时候就可以从避免无谓的渲染, 具体实现的方式有如下:

- `shouldComponentUpdate`
- `PureComponent`
- `React.memo`

JSX转化DOM过程?

`jsx` 首先会转化成 `React.createElement` 这种形式, `React.createElement` 作用是生成一个虚拟 `Dom` 对象, 然后通过 `ReactDOM.render` 进行渲染成真实 `DOM`

性能优化手段有哪些

除了减少 `render` 的渲染之外, 还可以通过以下手段进行优化:

除此之外, 常见性能优化常见的手段有如下:

- 避免使用内联函数
- 使用 `React Fragments` 避免额外标记
- 使用 `Immutable`
- 懒加载组件
- 事件绑定方式
- 服务端渲染

如何做服务端渲染?

`node server` 接收客户端请求, 得到当前的请求 `url` 路径, 然后在已有的路由表内查找到对应的组件, 拿到需要请求的数据, 将数据作为 `props`、`context` 或者 `store` 形式传入组件

然后基于 `react` 内置的服务端渲染方法 `renderToString()` 把组件渲染为 `html` 字符串在把最终的 `html` 进行输出前需要将数据注入到浏览器端

浏览器开始进行渲染和节点对比, 然后执行完成组件内事件绑定和一些交互, 浏览器重用了服务端输出的 `html` 节点, 整个流程结束

面试官：说说你对React Router的理解？常用的Router组件有哪些？

一、react-router

`react-router` 等前端路由的原理大致相同，可以实现无刷新的条件下切换显示不同的页面

路由的本质就是页面的 `URL` 发生改变时，页面的显示结果可以根据 `URL` 的变化而变化，但是页面不会刷新

因此，可以通过前端路由可以实现单页(SPA)应用

`react-router` 主要分成了几个不同的包：

- `react-router`：实现了路由的核心功能
- `react-router-dom`：基于 `react-router`，加入了在浏览器运行环境下的一些功能
- `react-router-native`：基于 `react-router`，加入了 `react-native` 运行环境下的一些功能
- `react-router-config`：用于配置静态路由的工具库

二、有哪些

这里主要讲述的是 `react-router-dom` 的常用 `API`，主要是提供了一些组件：

- `BrowserRouter`、`HashRouter`
- `Route`
- `Link`、`NavLink`
- `switch`
- `redirect`

`BrowserRouter`、`HashRouter`

`Router` 中包含了对路径改变的监听，并且会将相应的路径传递给子组件

`BrowserRouter` 是 `history` 模式，`HashRouter` 模式

使用两者作为最顶层组件包裹其他组件

```
import { BrowserRouter as Router } from "react-router-dom";

export default function App() {
  return (
    <Router>
      <main>
        <nav>
          <ul>
            <li>
              <a href="/">Home</a>
            </li>
            <li>
              <a href="/about">About</a>
            </li>
            <li>
              <a href="/contact">Contact</a>
            </li>
          </ul>
        </nav>
      </main>
    </Router>
  );
}
```

Route

Route 用于路径的匹配，然后进行组件的渲染，对应的属性如下：

- path 属性：用于设置匹配到的路径
- component 属性：设置匹配到路径后，渲染的组件
- render 属性：设置匹配到路径后，渲染的内容
- exact 属性：开启精准匹配，只有精准匹配到完全一致的路径，才会渲染对应的组件

```
import { BrowserRouter as Router, Route } from "react-router-dom";

export default function App() {
  return (
    <Router>
      <main>
        <nav>
          <ul>
            <li>
              <a href="/">Home</a>
            </li>
            <li>
              <a href="/about">About</a>
            </li>
            <li>
              <a href="/contact">Contact</a>
            </li>
          </ul>
        </nav>
        <Route path="/" render={() => <h1>Welcome!</h1>} />
      </main>
    </Router>
  );
}
```

Link、NavLink

通常路径的跳转是使用 `Link` 组件，最终会被渲染成 `a` 元素，其中属性 `to` 代替 `a` 标题的 `href` 属性

`NavLink` 是在 `Link` 基础之上增加了一些样式属性，例如组件被选中时，发生样式变化，则可以设置 `NavLink` 的以下属性：

- `activeStyle`：活跃时（匹配时）的样式
- `activeClassName`：活跃时添加的 class

如下：

```
<NavLink to="/" exact activeStyle={{color: "red"}}>首页</NavLink>
<NavLink to="/about" activeStyle={{color: "red"}}>关于</NavLink>
<NavLink to="/profile" activeStyle={{color: "red"}}>我的</NavLink>
```

如果需要实现 `js` 实现页面的跳转，那么可以通过下面的形式：

通过 `Route` 作为顶层组件包裹其他组件后，页面组件就可以接收到一些路由相关的东西，比如 `props.history`


```
const Contact = ({ history }) => (
  <Fragment>
    <h1>Contact</h1>
    <button onClick={() => history.push("/")}>Go to home</button>
    <FakeText />
  </Fragment>
);
```

`props` 中接收到的 `history` 对象具有一些方便的方法，如 `goBack`，`goForward`，`push`

redirect

用于路由的重定向，当这个组件出现时，就会执行跳转到对应的 `to` 路径中，如下例子：

```
const About = ({
  match: {
    params: { name },
  },
}) => (
  // props.match.params.name
  <Fragment>
    {name !== "tom" ? <Redirect to="/" /> : null}
    <h1>About {name}</h1>
    <FakeText />
  </Fragment>
)
```

上述组件当接收到的路由参数 `name` 不等于 `tom` 的时候，将会自动重定向到首页

switch

`switch` 组件的作用适用于当匹配到第一个组件的时候，后面的组件就不应该继续匹配

如下例子：

```
<Switch>
  <Route exact path="/" component={Home} />
  <Route path="/about" component={About} />
  <Route path="/profile" component={Profile} />
  <Route path="/:userid" component={User} />
  <Route component={NoMatch} />
</Switch>
```

如果不使用 `switch` 组件进行包裹

除了一些路由相关的组件之外，`react-router` 还提供一些 `hooks`，如下：

- `useHistory`
- `useParams`
- `useLocation`

useHistory

`useHistory` 可以让组件内部直接访问 `history`，无须通过 `props` 获取

```
import { useHistory } from "react-router-dom";

const Contact = () => {
  const history = useHistory();
  return (
    <Fragment>
      <h1>Contact</h1>
      <button onClick={() => history.push("/")}>Go to home</button>
    </Fragment>
  );
};
```

useParams

```
const About = () => {
  const { name } = useParams();
  return (
    // props.match.params.name
    <Fragment>
      {name !== "John Doe" ? <Redirect to="/" /> : null}
      <h1>About {name}</h1>
      <Route component={Contact} />
    </Fragment>
  );
};
```

useLocation

`useLocation` 会返回当前 URL 的 `location` 对象

```
import { useLocation } from "react-router-dom";

const Contact = () => {
  const { pathname } = useLocation();

  return (
    <Fragment>
      <h1>Contact</h1>
      <p>Current URL: {pathname}</p>
    </Fragment>
  );
};
```

三、参数传递

这些路由传递参数主要分成了三种形式：

- 动态路由的方式

- search 传递参数
- to 传入对象

动态路由

动态路由的概念指的是路由中的路径并不会固定

例如将 `path` 在 `Route` 匹配时写成 `/detail/:id`，那么 `/detail/abc`、`/detail/123` 都可以匹配到该 `Route`

```
<NavLink to="/detail/abc123">详情</NavLink>

<Switch>
  ... 其他Route
  <Route path="/detail/:id" component={Detail}/>
  <Route component={NoMatch} />
</Switch>
```

获取参数方式如下：

```
console.log(props.match.params.xxx)
```

search 传递参数

在跳转的路径中添加了一些 query 参数；

```
<NavLink to="/detail2?name=why&age=13">详情2</NavLink>

<Switch>
  <Route path="/detail2" component={Detail2}/>
</Switch>
```

获取形式如下：

```
console.log(props.location.search)
```

to 传入对象

传递方式如下：

```
<NavLink to={{
  pathname: "/detail2",
  query: {name: "kobe", age: 30},
  state: {height: 1.98, address: "洛杉矶"},
  search: "?apikey=123"
}}>
  详情2
</NavLink>
```

获取参数的形式如下：

```
console.log(props.location)
```

参考文献

- <http://react-guide.github.io/react-router-cn/docs/API.html#route>

说说你对Redux的理解？其工作原理？

一、Redux

React是用于构建用户界面的，帮助我们解决渲染 **DOM** 的过程

而在整个应用中会存在很多个组件，每个组件的 **state** 是由自身进行管理，包括组件定义自身的 **state**、组件之间的通信通过 **props** 传递、使用 **Context** 实现数据共享

如果让每个组件都存储自身相关的状态，理论上来讲不会影响应用的运行，但在开发及后续维护阶段，我们将花费大量精力去查询状态的变化过程

这种情况下，如果将所有的状态进行集中管理，当需要更新状态的时候，仅需要对这个管理集中处理，而不用去关心状态是如何分发到每一个组件内部的

redux 就是一个实现上述集中管理的容器，遵循三大基本原则：

- 单一数据源
- **state** 是只读的
- 使用纯函数来执行修改

注意的是，**redux** 并不是只应用在 **react** 中，还与其他界面库一起使用，如 **Vue**

二、工作原理

redux 要求我们把数据都放在 **store** 公共存储空间

一个组件改变了 **store** 里的数据内容，其他组件就能感知到 **store** 的变化，再来取数据，从而间接的实现了这些数据传递的功能

工作流程图如下所示：

根据流程图，可以想象，**React Components** 是借书的用户，**Action Creator** 是借书时说的话（借什么书），**Store** 是图书馆管理员，**Reducer** 是记录本（借什么书，还什么书，在哪儿，需要查一下），**state** 是书籍信息

整个流程就是借书的用户需要先存在，然后需要借书，需要一句话来描述借什么书，图书馆管理员听到后需要查一下记录本，了解图书的位置，最后图书馆管理员会把这本书给到这个借书人

转换为代码是，**React Components** 需要获取一些数据，然后它就告知 **Store** 需要获取数据，这就是就是 **Action Creator**，**Store** 接收到之后去 **Reducer** 查一下，**Reducer** 会告诉 **Store** 应该给这个组件什么数据

三、如何使用

创建一个 **store** 的公共数据区域

```
import { createStore } from 'redux' // 引入一个第三方的方法
const store = createStore() // 创建数据的公共存储区域 (管理员)
```

还需要创建一个记录本去辅助管理数据，也就是 `reducer`，本质就是一个函数，接收两个参数 `state`，`action`，返回 `state`

```
// 设置默认值
const initialState = {
  counter: 0
}

const reducer = (state = initialState, action) => {
}
```

然后就可以将记录本传递给 `store`，两者建立连接。如下：

```
const store = createStore(reducer)
```

如果想要获取 `store` 里面的数据，则通过 `store.getState()` 来获取当前 `state`

```
console.log(store.getState());
```

下面再看看如何更改 `store` 里面数据，是通过 `dispatch` 来派发 `action`，通常 `action` 中都会有 `type` 属性，也可以携带其他的数据

```
store.dispatch({
  type: "INCREMENT"
})

store.dispatch({
  type: "DECREMENT"
})

store.dispatch({
  type: "ADD_NUMBER",
  number: 5
})
```

下面再来看看修改 `reducer` 中的处理逻辑：


```
const reducer = (state = initialState, action) => {
  switch (action.type) {
    case "INCREMENT":
      return {...state, counter: state.counter + 1};
    case "DECREMENT":
      return {...state, counter: state.counter - 1};
    case "ADD_NUMBER":
      return {...state, counter: state.counter + action.number}
    default:
      return state;
  }
}
```

注意，`reducer` 是一个纯函数，不需要直接修改 `state`

这样派发 `action` 之后，既可以通过 `store.subscribe` 监听 `store` 的变化，如下：

```
store.subscribe(() => {
  console.log(store.getState());
})
```

在 `React` 项目中，会搭配 `react-redux` 进行使用

完整代码如下：

```
const redux = require('redux');

const initialState = {
  counter: 0
}

// 创建reducer
const reducer = (state = initialState, action) => {
  switch (action.type) {
    case "INCREMENT":
      return {...state, counter: state.counter + 1};
    case "DECREMENT":
      return {...state, counter: state.counter - 1};
    case "ADD_NUMBER":
      return {...state, counter: state.counter + action.number}
    default:
      return state;
  }
}

// 根据reducer创建store
const store = redux.createStore(reducer);

store.subscribe(() => {
  console.log(store.getState());
})

// 修改store中的state
store.dispatch({
  type: "INCREMENT"
})
// console.log(store.getState());

store.dispatch({
  type: "DECREMENT"
})
// console.log(store.getState());

store.dispatch({
  type: "ADD_NUMBER",
  number: 5
})
// console.log(store.getState());
```

小结

- createStore可以帮助创建 store
- store.dispatch 帮助派发 action , action 会传递给 store
- store.getState 这个方法可以帮助获取 store 里边所有的数据内容
- store.subscribe 方法订阅 store 的改变, 只要 store 发生改变, store.subscribe 这个函数接收的这个回调函数就会被执行

参考文献

- <https://cn.redux.js.org/docs/introduction/>
- <https://www.redux.org.cn/docs/basics/Actions.html>
- [https://lulujianglab.com/posts/大白话解析 Redux 、 redux-thunk 、 redux-saga 和 react-redux](https://lulujianglab.com/posts/大白话解析-Redux-、redux-thunk-、redux-saga-和-react-redux-)

面试官：说说你对immutable的理解？如何应用在react项目中？

一、Immutable

Immutable，不可改变的，在计算机中，即指一旦创建，就不能再被更改的数据

对 `Immutable` 对象的任何修改或添加删除操作都会返回一个新的 `Immutable` 对象

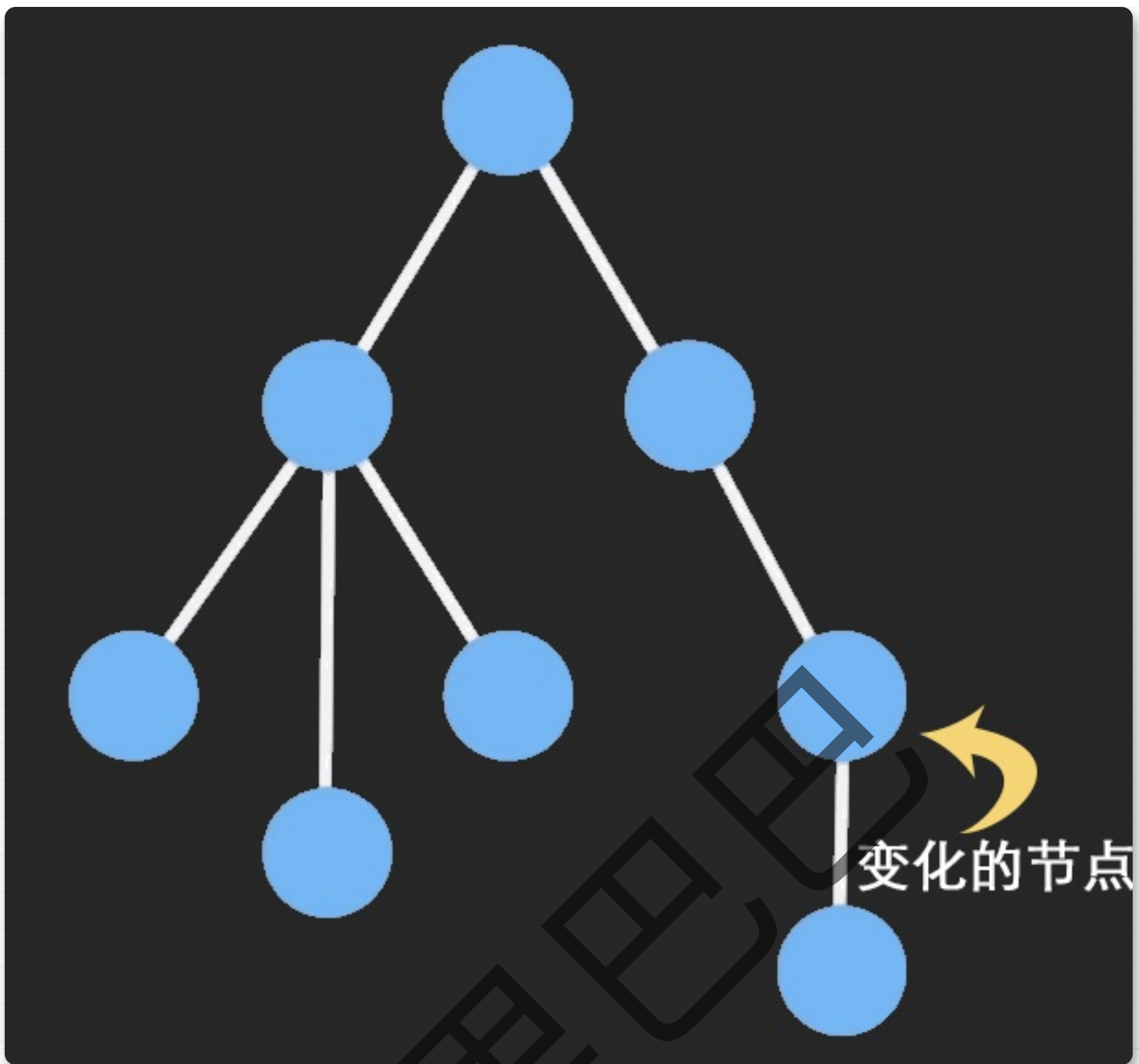
`Immutable` 实现的原理是 `Persistent Data Structure`（持久化数据结构）：

- 用一种数据结构来保存数据
- 当数据被修改时，会返回一个对象，但是新的对象会尽可能的利用之前的数据结构而不会对内存造成浪费

也就是使用旧数据创建新数据时，要保证旧数据同时可用且不变，同时为了避免 `deepCopy` 把所有节点都复制一遍带来的性能损耗，`Immutable` 使用了 `Structural Sharing`（结构共享）

如果对象树中一个节点发生变化，只修改这个节点和受它影响的父节点，其它节点则进行共享

如下图所示：



二、如何使用

使用 [Immutable](#) 对象最主要的库是 [immutable.js](#)

[immutable.js](#) 是一个完全独立的库，无论基于什么框架都可以用它

其出现场景在于弥补 Javascript 没有不可变数据结构的问题，通过 structural sharing来解决的性能问题

内部提供了一套完整的 Persistent Data Structure，还有很多易用的数据类型，如 [Collection](#)、[List](#)、[Map](#)、[Set](#)、[Record](#)、[Seq](#)，其中：

- List: 有序索引集，类似 JavaScript 中的 Array
- Map: 无序索引集，类似 JavaScript 中的 Object
- Set: 没有重复值的集合

主要的方法如下：

- `fromJS()`：将一个js数据转换为Immutable类型的数据

```
const obj = Immutable.fromJS({a:'123',b:'234'})
```

- `toJS()`：将一个Immutable数据转换为JS类型的数据
- `is()`：对两个对象进行比较

```
import { Map, is } from 'immutable'
const map1 = Map({ a: 1, b: 1, c: 1 })
const map2 = Map({ a: 1, b: 1, c: 1 })
map1 === map2 //false
Object.is(map1, map2) // false
is(map1, map2) // true
```

- `get(key)`: 对数据或对象取值
- `getIn([])`: 对嵌套对象或数组取值，传参为数组，表示位置

```
let abs = Immutable.fromJS({a: {b:2}});
abs.getIn(['a', 'b']) // 2
abs.getIn(['a', 'c']) // 子级没有值

let arr = Immutable.fromJS([1, 2, 3, {a: 5}]);
arr.getIn([3, 'a']); // 5
arr.getIn([3, 'c']); // 子级没有值
```

◦

如下例子：使用方法如下：

```
import Immutable from 'immutable';
foo = Immutable.fromJS({a: {b: 1}});
bar = foo.setIn(['a', 'b'], 2); // 使用 setIn 赋值
console.log(foo.getIn(['a', 'b'])); // 使用 getIn 取值，打印 1
console.log(foo === bar); // 打印 false
```

如果换到原生的 `js`，则对应如下：

```
let foo = {a: {b: 1}};
let bar = foo;
bar.a.b = 2;
console.log(foo.a.b); // 打印 2
console.log(foo === bar); // 打印 true
```

三、在React中应用

使用 `Immutable` 可以给 `React` 应用带来性能的优化，主要体现在减少渲染的次数

在做 `react` 性能优化的时候，为了避免重复渲染，我们会在 `shouldComponentUpdate()` 中做对比，当返回 `true` 执行 `render` 方法

`Immutable` 通过 `is` 方法则可以完成对比，而无需像一样通过深度比较的方式比较

在使用 `redux` 过程中也可以结合 `Immutable`，不使用 `Immutable` 前修改一个数据需要做一个深拷贝


```
import '_' from 'lodash';

const Component = React.createClass({
  getInitialState() {
    return {
      data: { times: 0 }
    }
  },
  handleAdd() {
    let data = _.cloneDeep(this.state.data);
    data.times = data.times + 1;
    this.setState({ data: data });
  }
});
```

使用 Immutable 后：

```
getInitialState() {
  return {
    data: Map({ times: 0 })
  }
},
handleAdd() {
  this.setState({ data: this.state.data.update('times', v => v + 1) });
  // 这时的 times 并不会改变
  console.log(this.state.data.get('times'));
}
```

同理，在 `redux` 中也可以将数据进行 `fromJS` 处理

```
import * as constants from './constants'
import {fromJS} from 'immutable'
const defaultState = fromJS({ //将数据转化成immutable数据
  home:true,
  focused:false,
  mouseIn:false,
  list:[],
  page:1,
  totalPage:1
})
export default(state=defaultState,action)=>{
  switch(action.type){
    case constants.SEARCH_FOCUS:
      return state.set('focused',true) //更改immutable数据
    case constants.CHANGE_HOME_ACTIVE:
      return state.set('home',action.value)
    case constants.SEARCH_BLUR:
      return state.set('focused',false)
    case constants.CHANGE_LIST:
      // return state.set('list',action.data).set('totalPage',action.totalPage)
      //merge效率更高，执行一次改变多个数据
      return state.merge({
        list:action.data,
        totalPage:action.totalPage
      })
    case constants.MOUSE_ENTER:
      return state.set('mouseIn',true)
    case constants.MOUSE_LEAVE:
      return state.set('mouseIn',false)
    case constants.CHANGE_PAGE:
      return state.set('page',action.page)
    default:
      return state
  }
}
```

参考文献

- <https://zhuanlan.zhihu.com/p/20295971?spm=a2c4e.11153940.blogcont69516.18.4f275a00EzBHjr&columnSlug=purerender>
- <https://www.jianshu.com/p/7bf04638e82a>

面试官：说说你是如何提高组件的渲染效率的？在React中如何避免不必要的render？

一、渲染

react 基于虚拟 DOM 和高效 Diff 算法的完美配合，实现了对 DOM 最小粒度的更新，大多数情况下，React 对 DOM 的渲染效率足以我们的业务日常

复杂业务场景下，性能问题依然会困扰我们。此时需要采取一些措施来提升运行性能，避免不必要的渲染则是业务中常见的优化手段之一

二、如何做

在之前文章中，我们了解到 `render` 的触发时机，简单来讲就是类组件通过调用 `setState` 方法，就会导致 `render`，父组件一旦发生 `render` 渲染，子组件一定也会执行 `render` 渲染

从上面可以看到，父组件渲染导致子组件渲染，子组件并没有发生任何改变，这时候就可以从避免无谓的渲染，具体实现的方式有如下：

- `shouldComponentUpdate`
- `PureComponent`
- `React.memo`

`shouldComponentUpdate`

通过 `shouldComponentUpdate` 生命周期函数来比对 `state` 和 `props`，确定是否要重新渲染

默认情况下返回 `true` 表示重新渲染，如果不希望组件重新渲染，返回 `false` 即可

`PureComponent`

跟 `shouldComponentUpdate` 原理基本一致，通过对 `props` 和 `state` 的浅比较结果来实现 `shouldComponentUpdate`，源码大致如下：

```
if (this._compositeType === CompositeTypes.PureClass) {
  shouldUpdate = !shallowEqual(prevProps, nextProps) || !shallowEqual(inst.state, nextState);
}
```

`shallowEqual` 对应方法大致如下：

```

const hasOwnProperty = Object.prototype.hasOwnProperty;

/**
 * is 方法来判断两个值是否是相等的值，为何这么写可以移步 MDN 的文档
 * https://developer.mozilla.org/zh-CN/docs/Web/JavaScript/Reference/Global_Objects/Object/is
 */
function is(x: mixed, y: mixed): boolean {
  if (x === y) {
    return x !== 0 || y !== 0 || 1 / x === 1 / y;
  } else {
    return x !== x && y !== y;
  }
}

function shallowEqual(objA: mixed, objB: mixed): boolean {
  // 首先对基本类型进行比较
  if (is(objA, objB)) {
    return true;
  }

  if (typeof objA !== 'object' || objA === null ||
    typeof objB !== 'object' || objB === null) {
    return false;
  }

  const keysA = Object.keys(objA);
  const keysB = Object.keys(objB);

  // 长度不相等直接返回false
  if (keysA.length !== keysB.length) {
    return false;
  }

  // key相等的情况下，再去循环比较
  for (let i = 0; i < keysA.length; i++) {
    if (
      !hasOwnProperty.call(objB, keysA[i]) ||
      !is(objA[keysA[i]], objB[keysA[i]])
    ) {
      return false;
    }
  }

  return true;
}

```

当对象包含复杂的数据结构时，对象深层的数据已改变却没有触发 `render`

注意：在 `react` 中，是不建议使用深层次结构的数据

React.memo

`React.memo` 用来缓存组件的渲染，避免不必要的更新，其实也是一个高阶组件，与 `PureComponent` 十分类似。但不同的是，`React.memo` 只能用于函数组件

```
import { memo } from 'react';

function Button(props) {
  // Component code
}

export default memo(Button);
```

如果需要深层次比较，这时候可以给 `memo` 第二个参数传递比较函数

```
function arePropsEqual(prevProps, nextProps) {
  // your code
  return prevProps === nextProps;
}

export default memo(Button, arePropsEqual);
```

三、总结

在实际开发过程中，前端性能问题是一个必须考虑的问题，随着业务的复杂，遇到性能问题的概率也在增高

除此之外，建议将页面进行更小的颗粒化，如果一个过大，当状态发生修改的时候，就会导致整个大组件的渲染，而对组件进行拆分后，粒度变小了，也能够减少子组件不必要的渲染

参考文献

◦ <https://juejin.cn/post/6844903781679759367#heading-12>

面试官：说说对 React 的理解？有哪些特性？

React

React，用于构建用户界面的 JavaScript 库，只提供了 UI 层面的解决方案

遵循组件设计模式、声明式编程范式和函数式编程概念，以使前端应用程序更高效

使用虚拟 `DOM` 来有效地操作 `DOM`，遵循从高阶组件到低阶组件的单向数据流

帮助我们将界面成了各个独立的小块，每一个块就是组件，这些组件之间可以组合、嵌套，构成整体页面

`react` 类组件使用一个名为 `render()` 的方法或者函数组件 `return`，接收输入的数据并返回需要展示的内容


```
class HelloMessage extends React.Component {
  render() {
    return <div>Hello {this.props.name}</div>;
  }
}

ReactDOM.render(
  <HelloMessage name="Taylor" />,
  document.getElementById("hello-example")
);
```

上述这种类似 XML 形式就是 JSX，最终会被 babel 编译为合法的 JS 语句调用

被传入的数据可在组件中通过 this.props 在 render() 访问

二、特性

React 特性有很多，如：

- JSX 语法
- 单向数据绑定
- 虚拟 DOM
- 声明式编程
- Component

着重介绍下声明式编程及 Component

声明式编程

声明式编程是一种编程范式，它关注的是你要做什么，而不是如何做

它表达逻辑而不显式地定义步骤。这意味着我们需要根据逻辑的计算来声明要显示的组件

如实现一个标记的地图：

通过命令式创建地图、创建标记、以及在地图上添加的标记的步骤如下：

```
// 创建地图
const map = new Map.map(document.getElementById("map"), {
  zoom: 4,
  center: { lat, lng },
});

// 创建标记
const marker = new Map.marker({
  position: { lat, lng },
  title: "Hello Marker",
});

// 地图上添加标记
marker.setMap(map);
```

而用 React 实现上述功能则如下：

```
<Map zoom={4} center={{lat, lng}}>
  <Marker position={{lat, lng}} title={"Hello Marker"} />
</Map>
```

声明式编程方式使得 `React` 组件很容易使用，最终的代码简单易于维护

Component

在 `React` 中，一切皆为组件。通常将应用程序的整个逻辑分解为小的单个部分。 我们将每个单独的部分称为组件

组件可以是一个函数或者是一个类，接受数据输入，处理它并返回在 `UI` 中呈现的 `React` 元素

函数式组件如下：

```
const Header = () => {
  return (
    <Jumbotron style={{ backgroundColor: "orange" }}>
      <h1>TODO App</h1>
    </Jumbotron>
  );
};
```

类组件（有状态组件）如下：

```
class Dashboard extends React.Component {
  constructor(props) {
    super(props);

    this.state = {};
  }
  render() {
    return (
      <div className="dashboard">
        <ToDoForm />
        <ToDoList />
      </div>
    );
  }
}
```

一个组件该有的特点如下：

- 可组合：每个组件易于和其它组件一起使用，或者嵌套在另一个组件内部
- 可重用：每个组件都是具有独立功能的，它可以被使用在多个 `UI` 场景
- 可维护：每个小的组件仅仅包含自身的逻辑，更容易被理解和维护

三、优势

通过上面的初步了解，可以感受到 `React` 存在的优势：

- 高效灵活
- 声明式的设计，简单使用

- 组件式开发，提高代码复用率
- 单向响应的数据流会比双向绑定的更安全，速度更快

参考文献

- <https://segmentfault.com/a/1190000015924762>
(<https://segmentfault.com/a/1190000015924762>)
- <https://react.docschina.org/> (<https://react.docschina.org/>)

面试官：说说对Fiber架构的理解？解决了什么问题？

一、问题

JavaScript引擎和页面渲染引擎两个线程是互斥的，当其中一个线程执行时，另一个线程只能挂起等待

如果 JavaScript 线程长时间地占用了主线程，那么渲染层面的更新就不得不长时间地等待，界面长时间不更新，会导致页面响应度变差，用户可能会感觉到卡顿

而这也正是 React 15 的 Stack Reconciler所面临的问题，当 React在渲染组件时，从开始到渲染完成整个过程是一气呵成的，无法中断

如果组件较大，那么 js 线程会一直执行，然后等到整棵 VDOM 树计算完成后，才会交给渲染的线程

这就会导致一些用户交互、动画等任务无法立即得到处理，导致卡顿的情况

二、是什么

React Fiber 是 Facebook 花费两年余时间对 React 做出的一个重大改变与优化，是对 React 核心算法的一次重新实现。从Facebook在 React Conf 2017 会议上确认，React Fiber 在React 16 版本发布

在 react 中，主要做了以下的操作：

- 为每个增加了优先级，优先级高的任务可以中断低优先级的任务。然后再重新，注意是重新执行优先级低的任务
- 增加了异步任务，调用requestIdleCallback api，浏览器空闲的时候执行
- dom diff树变成了链表，一个dom对应两个fiber（一个链表），对应两个队列，这都是为找到被中断的任务，重新执行

从架构角度来看，Fiber 是对 React核心算法（即调和过程）的重写

从编码角度来看，Fiber是 React内部所定义的一种数据结构，它是 Fiber树结构的节点单位，也就是 React 16 新架构下的虚拟 DOM

一个 fiber 就是一个 JavaScript对象，包含了元素的信息、该元素的更新操作队列、类型，其数据结构如下：

```
type Fiber = {  
  // 用于标记fiber的WorkTag类型，主要表示当前fiber代表的组件类型如FunctionComponent、  
  // ClassComponent等  
  tag: WorkTag,  
  // ReactElement里面的key  
  key: null | string,  
  // ReactElement.type，调用`createElement`的第一个参数
```

```
elementType: any,  
// The resolved function/class/ associated with this fiber.  
// 表示当前代表的节点类型  
type: any,  
// 表示当前FiberNode对应的element组件实例  
stateNode: any,  
  
// 指向他在Fiber节点树中的`parent`，用来在处理完这个节点之后向上返回  
return: Fiber | null,  
// 指向自己的第一个子节点  
child: Fiber | null,  
// 指向自己的兄弟结构，兄弟节点的return指向同一个父节点  
sibling: Fiber | null,  
index: number,  
  
ref: null | (((handle: mixed) => void) & { _stringRef: ?string }) | RefObject,  
  
// 当前处理过程中的组件props对象  
pendingProps: any,  
// 上一次渲染完成之后的props  
memoizedProps: any,  
  
// 该Fiber对应的组件产生的Update会存放在这个队列里面  
updateQueue: UpdateQueue<any> | null,  
  
// 上一次渲染的时候的state  
memoizedState: any,  
  
// 一个列表，存放这个Fiber依赖的context  
firstContextDependency: ContextDependency<mixed> | null,  
  
mode: TypeOfMode,  
  
// Effect  
// 用来记录Side Effect  
effectTag: SideEffectTag,  
  
// 单链表用来快速查找下一个side effect  
nextEffect: Fiber | null,  
  
// 子树中第一个side effect  
firstEffect: Fiber | null,  
// 子树中最后一个side effect  
lastEffect: Fiber | null,  
  
// 代表任务在未来的哪个时间点应该被完成，之后版本改名为 lanes  
expirationTime: ExpirationTime,  
  
// 快速确定子树中是否有不在等待的变化  
childExpirationTime: ExpirationTime,  
  
// fiber的版本池，即记录fiber更新过程，便于恢复  
alternate: Fiber | null,
```

```
}
```

三、如何解决

Fiber 把渲染更新过程拆分成多个子任务，每次只做一小部分，做完看是否还有剩余时间，如果有继续下一个任务；如果没有，挂起当前任务，将时间控制权交给主线程，等主线程不忙的时候再继续执行。即可以中断与恢复，恢复后也可以复用之前的中间状态，并给不同的任务赋予不同的优先级，其中每个任务更新单元为 **React Element** 对应的 **Fiber** 节点。

实现的上述方式的是 **requestIdleCallback** 方法。

window.requestIdleCallback() 方法将在浏览器的空闲时段内调用的函数排队。这使开发者能够在主事件循环上执行后台和低优先级工作，而不会影响延迟关键事件，如动画和输入响应。

首先 **React** 中任务切割为多个步骤，分批完成。在完成一部分任务之后，将控制权交回给浏览器，让浏览器有时间再进行页面的渲染。等浏览器忙完之后有剩余时间，再继续之前 **React** 未完成的任务，是一种合作式调度。

该实现过程是基于 **Fiber** 节点实现，作为静态的数据结构来说，每个 **Fiber** 节点对应一个 **React element**，保存了该组件的类型（函数组件/类组件/原生组件等等）、对应的 **DOM** 节点等信息。

作为动态的工作单元来说，每个 **Fiber** 节点保存了本次更新中该组件改变的状态、要执行的工作。

每个 **Fiber** 节点有个对应的 **React element**，多个 **Fiber** 节点根据如下三个属性构建一颗树：

```
// 指向父级Fiber节点
this.return = null
// 指向子Fiber节点
this.child = null
// 指向右边第一个兄弟Fiber节点
this.sibling = null
```

通过这些属性就能找到下一个执行目标。

参考文献

- <https://juejin.cn/post/6926432527980691470>
- <https://zhuanlan.zhihu.com/p/137234573>

面试官：说说对React Hooks的理解？解决了什么问题？

一、React Hooks

Hook 是 **React 16.8** 的新增特性。它可以让你在不编写 **class** 的情况下使用 **state** 以及其他的 **React** 特性。

至于为什么引入 **hook**，官方给出的动机是解决长时间使用和维护 **react** 过程中常遇到的问题，例如：

- 难以重用和共享组件中的与状态相关的逻辑
- 逻辑复杂的组件难以开发与维护，当我们的组件需要处理多个互不相关的 **local state** 时，每个生命周期函数中可能会包含着各种互不相关的逻辑在里面
- 类组件中的 **this** 增加学习成本，类组件在基于现有工具的优化上存在些许问题
- 由于业务变动，函数组件不得不改为类组件等等

在以前，函数组件也被称为无状态的组件，只负责渲染的一些工作
因此，现在的函数组件也可以是有状态的组件，内部也可以维护自身的状态以及做一些逻辑方面的处理

二、有哪些

上面讲到， Hooks 让我们的函数组件拥有了类组件的特性，例如组件内的状态、生命周期

最常见的 hooks 有如下：

- useState
- useEffect
- 其他

useState

首先给出一个例子，如下：

```
import React, { useState } from 'react';

function Example() {
  // 声明一个叫 "count" 的 state 变量
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

在函数组件中通过 useState 实现函数内部维护 state，参数为 state 默认的值，返回值是一个数组，第一个值为当前的 state，第二个值为更新 state 的函数

该函数组件等价于的类组件如下：

```

class Example extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  render() {
    return (
      <div>
        <p>You clicked {this.state.count} times</p>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Click me
        </button>
      </div>
    );
  }
}

```

从上述两种代码分析，可以看出两者区别：

- state声明方式：在函数组件中通过 `useState` 直接获取，类组件通过 `constructor` 构造函数中设置
- state读取方式：在函数组件中直接使用变量，类组件通过 `this.state.count` 的方式获取
- state更新方式：在函数组件中通过 `setCount` 更新，类组件通过 `this.setState()`

总的来讲，`useState` 使用起来更为简洁，减少了 `this` 指向不明确的情况

useEffect

`useEffect` 可以让我们在函数组件中进行一些带有副作用的操作

同样给出一个计时器示例：

```

class Example extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  componentDidMount() {
    document.title = `You clicked ${this.state.count} times`;
  }

  componentDidUpdate() {
    document.title = `You clicked ${this.state.count} times`;
  }

  render() {
    return (
      <div>
        <p>You clicked {this.state.count} times</p>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Click me
        </button>
      </div>
    );
  }
}

```

从上面可以看见，组件在加载和更新阶段都执行同样操作
而如果使用 `useEffect` 后，则能够将相同的逻辑抽离出来，这是类组件不具备的方法
对应的 `useEffect` 示例如下：

```

import React, { useState, useEffect } from 'react';
function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => { document.title = `You clicked ${count} times`; });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

```

`useEffect` 第一个参数接受一个回调函数，默认情况下，`useEffect` 会在第一次渲染和更新之后都会执行，相当于在 `componentDidMount` 和 `componentDidUpdate` 两个生命周期函数中执行回调

如果某些特定值在两次重渲染之间没有发生变化，你可以跳过对 `effect` 的调用，这时候只需要传入第二个参数，如下：

```
useEffect(() => {  
  document.title = `You clicked ${count} times`;  
}, [count]); // 仅在 count 更改时更新
```

上述传入第二个参数后，如果 `count` 的值是 `5`，而且我们的组件重渲染的时候 `count` 还是等于 `5`，React 将对前一次渲染的 `[5]` 和后一次渲染的 `[5]` 进行比较，如果是相等则跳过 `effects` 执行。回调函数中可以返回一个清除函数，这是 `effect` 可选的清除机制，相当于类组件中 `componentWillUnmount` 生命周期函数，可做一些清除副作用的操作，如下：

```
useEffect(() => {  
  function handleStatusChange(status) {  
    setIsOnline(status.isOnline);  
  }  
  
  ChatAPI.subscribeToFriendStatus(props.friend.id, handleStatusChange);  
  return () => {  
    ChatAPI.unsubscribeFromFriendStatus(props.friend.id, handleStatusChange);  
  };  
});
```

所以，`useEffect` 相当于 `componentDidMount`，`componentDidUpdate` 和 `componentWillUnmount` 这三个生命周期函数的组合。

其它 hooks

在组件通信过程中可以使用 `useContext`，`refs` 学习中我们也用到了 `useRef` 获取 `DOM` 结构..... 还有很多额外的 `hooks`，如：

- `useReducer`
- `useCallback`
- `useMemo`
- `useRef`

三、解决什么

通过对上面的初步认识，可以看到 `hooks` 能够更容易解决状态相关的重用的问题：

- 每调用 `useHook` 一次都会生成一份独立的状态
- 通过自定义 `hook` 能够更好的封装我们的功能

编写 `hooks` 为函数式编程，每个功能都包裹在函数中，整体风格更清爽，更优雅。

`hooks` 的出现，使函数组件的功能得到了扩充，拥有了类组件相似的功能，在我们日常使用中，使用 `hooks` 能够解决大多数问题，并且还拥有代码复用机制，因此优先考虑 `hooks`。

参考文献

- <https://zh-hans.reactjs.org/docs/hooks-state.html>
- <https://zh-hans.reactjs.org/docs/hooks-effect.html>
- <https://www.cnblogs.com/lalalagq/p/9898531.html>

面试官：说说对React refs 的理解？应用场景？

一、Refs

Refs 在计算机中称为弹性文件系统（英语：Resilient File System，简称ReFS）

React 中的 Refs 提供了一种方式，允许我们访问 DOM 节点或在 render 方法中创建的 React 元素

本质为 ReactDOM.render() 返回的组件实例，如果是渲染组件则返回的是组件实例，如果渲染 dom 则返回的是具体的 dom 节点

二、如何使用

创建 ref 的形式有三种：

- 传入字符串，使用时通过 this.refs.传入的字符串的格式获取对应的元素
- 传入对象，对象是通过 React.createRef() 方式创建出来，使用时获取到创建的对象中存在 current 属性就是对应的元素
- 传入函数，该函数会在 DOM 被挂载时进行回调，这个函数会传入一个 元素对象，可以自己保存，使用时，直接拿到之前保存的元素对象即可
- 传入hook，hook是通过 useRef() 方式创建，使用时通过生成hook对象的 current 属性就是对应的元素

传入字符串

只需要在对应元素或组件中 ref 属性

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.myRef = React.createRef();
  }
  render() {
    return <div ref="myref" />;
  }
}
```

访问当前节点的方式如下：

```
this.refs.myref.innerHTML = "hello";
```

传入对象

refs 通过 React.createRef() 创建，然后将 ref 属性添加到 React 元素中，如下：


```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.myRef = React.createRef();
  }
  render() {
    return <div ref={this.myRef} />;
  }
}
```

当 `ref` 被传递给 `render` 中的元素时，对该节点的引用可以在 `ref` 的 `current` 属性中访问

```
const node = this.myRef.current;
```

传入函数

当 `ref` 传入为一个函数的时候，在渲染过程中，回调函数参数会传入一个元素对象，然后通过实例将对象进行保存

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.myRef = React.createRef();
  }
  render() {
    return <div ref={element => this.myref = element} />;
  }
}
```

获取 `ref` 对象只需要通过先前存储的对象即可

```
const node = this.myref
```

传入hook

通过 `useRef` 创建一个 `ref`，整体使用方式与 `React.createRef` 一致

```
function App(props) {
  const myref = useRef()
  return (
    <div ref={myref}></div>
  )
}
```

获取 `ref` 属性也是通过 `hook` 对象的 `current` 属性

```
const node = myref.current;
```

上述三种情况都是 `ref` 属性用于原生 `HTML` 元素上，如果 `ref` 设置的组件为一个类组件的时候，`ref` 对象接收到的是组件的挂载实例

注意的是，不能在函数组件上使用 `ref` 属性，因为他们并没有实例

三、应用场景

在某些情况下，我们会通过使用 `refs` 来更新组件，但这种方式并不推荐，更多情况我们是通过 `props` 与 `state` 的方式进行去重新渲染子元素

过多使用 `refs`，会使组件的实例或者是 `DOM` 结构暴露，违反组件封装的原则

例如，避免在 `Dialog` 组件里暴露 `open()` 和 `close()` 方法，最好传递 `isOpen` 属性

但下面的场景使用 `refs` 非常有用：

- 对 `Dom` 元素的焦点控制、内容选择、控制
- 对 `Dom` 元素的内容设置及媒体播放
- 对 `Dom` 元素的操作和对组件实例的操作
- 集成第三方 `DOM` 库

参考文献

- <https://zh-hans.reactjs.org/docs/refs-and-the-dom.html>
- <https://segmentfault.com/a/1190000020842342>

面试官：说说对React中类组件和函数组件的理解？有什么区别？

一、类组件

类组件，顾名思义，也就是通过使用 `ES6` 类的编写形式去编写组件，该类必须继承 `React.Component`

如果想要访问父组件传递过来的参数，可通过 `this.props` 的方式去访问

在组件中必须实现 `render` 方法，在 `return` 中返回 `React` 对象，如下：

```
class Welcome extends React.Component {
  constructor(props) {
    super(props)
  }
  render() {
    return <h1>Hello, {this.props.name}</h1>
  }
}
```

二、函数组件

函数组件，顾名思义，就是通过函数编写的形式去实现一个 `React` 组件，是 `React` 中定义组件最简单的方式

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

函数第一个参数为 `props` 用于接收父组件传递过来的参数

三、区别

针对两种 `React` 组件，其区别主要分成以下几大方向：

- 编写形式
- 状态管理
- 生命周期
- 调用方式
- 获取渲染的值

编写形式

两者最明显的区别在于编写形式的不同，同一种功能的实现可以分别对应类组件和函数组件的编写形式

函数组件：

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

类组件：

```
class Welcome extends React.Component {  
  constructor(props) {  
    super(props)  
  }  
  render() {  
    return <h1>Hello, {this.props.name}</h1>  
  }  
}
```

状态管理

在 `hooks` 出来之前，函数组件就是无状态组件，不能保管组件的状态，不像类组件中调用 `setState`

如果想要管理 `state` 状态，可以使用 `useState`，如下：

```
const FunctionalComponent = () => {
  const [count, setCount] = React.useState(0);

  return (
    <div>
      <p>count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Click</button>
    </div>
  );
};
```

在使用 `hooks` 情况下，一般如果函数组件调用 `state`，则需要创建一个类组件或者 `state` 提升到你的父组件中，然后通过 `props` 对象传递到子组件

生命周期

在函数组件中，并不存在生命周期，这是因为这些生命周期钩子都来自于继承的 `React.Component`

所以，如果用到生命周期，就只能使用类组件

但是函数组件使用 `useEffect` 也能够完成替代生命周期的作用，这里给出一个简单的例子：

```
const FunctionalComponent = () => {
  useEffect(() => {
    console.log("Hello");
  }, []);
  return <h1>Hello, World</h1>;
};
```

上述简单的例子对应类组件中的 `componentDidMount` 生命周期

如果在 `useEffect` 回调函数中 `return` 一个函数，则 `return` 函数会在组件卸载的时候执行，正如 `componentWillUnmount`

```
const FunctionalComponent = () => {
  React.useEffect(() => {
    return () => {
      console.log("Bye");
    };
  }, []);
  return <h1>Bye, World</h1>;
};
```

调用方式

如果是一个函数组件，调用则是执行函数即可：

```
// 你的代码
function SayHi() {
  return <p>Hello, React</p>
}
// React内部
const result = SayHi(props) // » <p>Hello, React</p>
```

如果是一个类组件，则需要将组件进行实例化，然后调用实例对象的 `render` 方法：

```
// 你的代码
class SayHi extends React.Component {
  render() {
    return <p>Hello, React</p>
  }
}
// React内部
const instance = new SayHi(props) // » SayHi {}
const result = instance.render() // » <p>Hello, React</p>
```

获取渲染的值

首先给出一个示例

函数组件对应如下：

```
function ProfilePage(props) {
  const showMessage = () => {
    alert('Followed ' + props.user);
  }

  const handleClick = () => {
    setTimeout(showMessage, 3000);
  }

  return (
    <button onClick={handleClick}>Follow</button>
  )
}
```

类组件对应如下：


```
class ProfilePage extends React.Component {
  showMessage() {
    alert('Followed ' + this.props.user);
  }

  handleClick() {
    setTimeout(this.showMessage.bind(this), 3000);
  }

  render() {
    return <button onClick={this.handleClick.bind(this)}>Follow</button>
  }
}
```

两者看起来实现功能是一致的，但是在类组件中，输出 `this.props.user`，`Props` 在 `React` 中是不可变的所以它永远不会改变，但是 `this` 总是可变的，以便您可以在 `render` 和生命周期函数中读取新版本

因此，如果我们的组件在请求运行时更新。`this.props` 将会改变。`showMessage` 方法从“最新”的 `props` 中读取 `user`

而函数组件，本身就不存在 `this`，`props` 并不发生改变，因此同样是点击，`alert` 的内容仍旧是之前的内容

小结

两种组件都有各自的优缺点

函数组件语法更短、更简单，这使得它更容易开发、理解和测试

而类组件也会因大量使用 `this` 而让人感到困惑

参考文献

- <https://zh-hans.reactjs.org/docs/components-and-props.html#function-and-class-components>
- <https://juejin.cn/post/6844903806140973069>

面试官：说说对Redux中间件的理解？常用的中间件有哪些？实现原理？

一、中间件

中间件（Middleware）是介于应用系统和系统软件之间的一类软件，它使用系统软件所提供的基础服务（功能），衔接网络上应用系统的各个部分或不同的应用，能够达到资源共享、功能共享的目的

在上篇文章中，了解到了 `Redux` 整个工作流程，当 `action` 发出之后，`reducer` 立即算出 `state`，整个过程是一个同步的操作

那么如果需要在支持异步操作，或者支持错误处理、日志监控，这个过程就可以用上中间件

`Redux` 中，中间件就是放在就是在 `dispatch` 过程，在分发 `action` 进行拦截处理，如下图：

其本质上一个函数，对 `store.dispatch` 方法进行了改造，在发出 `Action` 和执行 `Reducer` 这两步之间，添加了其他功能

二、常用的中间件

有很多优秀的 `redux` 中间件，如：

- `redux-thunk`：用于异步操作
- `redux-logger`：用于日志记录

上述的中间件都需要通过 `applyMiddlewares` 进行注册，作用是将所有的中间件组成一个数组，依次执行

然后作为第二个参数传入到 `createStore` 中

```
const store = createStore(  
  reducer,  
  applyMiddleware(thunk, logger)  
);
```

redux-thunk

`redux-thunk` 是官网推荐的异步处理中间件

默认情况下的 `dispatch(action)`，`action` 需要是一个 `JavaScript` 的对象

`redux-thunk` 中间件会判断你当前传进来的数据类型，如果是一个函数，将会给函数传入参数值 (`dispatch`, `getState`)

- `dispatch` 函数用于我们之后再次派发 `action`
- `getState` 函数考虑到我们之后的一些操作需要依赖原来的状态，用于让我们可以获取之前的一些状态

所以 `dispatch` 可以写成下述函数的形式：

```
const getHomeMultidataAction = () => {  
  return (dispatch) => {  
    axios.get("http://xxx.xx.xx.xx/test").then(res => {  
      const data = res.data.data;  
      dispatch(changeBannersAction(data.banner.list));  
      dispatch(changeRecommendsAction(data.recommend.list));  
    })  
  }  
}
```

redux-logger

如果想要实现一个日志功能，则可以使用现成的 `redux-logger`

```
import { applyMiddleware, createStore } from 'redux';
import createLogger from 'redux-logger';
const logger = createLogger();

const store = createStore(
  reducer,
  applyMiddleware(logger)
);
```

这样我们就能简单通过中间件函数实现日志记录的信息

三、实现原理

首先看看 `applyMiddlewares` 的源码

```
export default function applyMiddleware(...middlewares) {
  return (createStore) => (reducer, preloadedState, enhancer) => {
    var store = createStore(reducer, preloadedState, enhancer);
    var dispatch = store.dispatch;
    var chain = [];

    var middlewareAPI = {
      getState: store.getState,
      dispatch: (action) => dispatch(action)
    };
    chain = middlewares.map(middleware => middleware(middlewareAPI));
    dispatch = compose(...chain)(store.dispatch);

    return {...store, dispatch}
  }
}
```

所有中间件被放进了一个数组 `chain`，然后嵌套执行，最后执行 `store.dispatch`。可以看到，中间件内部（`middlewareAPI`）可以拿到 `getState` 和 `dispatch` 这两个方法

在上面的学习中，我们了解到了 `redux-thunk` 的基本使用

内部会将 `dispatch` 进行一个判断，然后执行对应操作，原理如下：

```
function patchThunk(store) {
  let next = store.dispatch;

  function dispatchAndThunk(action) {
    if (typeof action === "function") {
      action(store.dispatch, store.getState);
    } else {
      next(action);
    }
  }

  store.dispatch = dispatchAndThunk;
}
```

实现一个日志输出的原理也非常简单，如下：

```
let next = store.dispatch;

function dispatchAndLog(action) {
  console.log("dispatching:", addAction(10));
  next(addAction(5));
  console.log("新的state:", store.getState());
}

store.dispatch = dispatchAndLog;
```

参考文献

- http://www.ruanyifeng.com/blog/2016/09/redux_tutorial_part_two_async_operatior

面试官：说说对受控组件和非受控组件的理解？应用场景？

一、受控组件

受控组件，简单来讲，就是受我们控制的组件，组件的状态全程响应外部数据

举个简单的例子：

```
class TestComponent extends React.Component {
  constructor (props) {
    super(props);
    this.state = { username: 'lindaIdai' };
  }
  render () {
    return <input name="username" value={this.state.username} />
  }
}
```

这时候当我们在输入框输入内容的时候，会发现输入的内容并无法显示出来，也就是 `input` 标签是一个可读的状态

这是因为 `value` 被 `this.state.username` 所控制住。当用户输入新的内容时，`this.state.username` 并不会自动更新，这样的话 `input` 内的内容也就不会变了

如果想要解除被控制，可以为 `input` 标签设置 `onChange` 事件，输入的时候触发事件函数，在函数内部实现 `state` 的更新，从而导致 `input` 框的内容页发现改变

因此，受控组件我们一般需要初始状态和一个状态更新事件函数

二、非受控组件

非受控组件，简单来讲，就是不受我们控制的组件

一般情况是在初始化的时候接受外部数据，然后自己在内部存储其自身状态

当需要时，可以使用 `ref` 查询 `DOM` 并查找其当前值，如下：

```
import React, { Component } from 'react';

export class UnControll extends Component {
  constructor (props) {
    super(props);
    this.inputRef = React.createRef();
  }
  handleSubmit = (e) => {
    console.log('我们可以获得input内的值为', this.inputRef.current.value);
    e.preventDefault();
  }
  render () {
    return (
      <form onSubmit={e => this.handleSubmit(e)}>
        <input defaultValue="lindaidal" ref={this.inputRef} />
        <input type="submit" value="提交" />
      </form>
    )
  }
}
```

关于 `refs` 的详情使用可以参考[之前文章](#)

<https://mp.weixin.qq.com/s/ZBKWcsIVBi0IKQgz7lYzbA>

三、应用场景

大部分时候推荐使用受控组件来实现表单，因为在受控组件中，表单数据由 `React` 组件负责处理

如果选择非受控组件的话，控制能力较弱，表单数据就由 `DOM` 本身处理，但更加方便快捷，代码量少

针对两者的区别，其应用场景如下图所示：

参考文献

- <http://meloguo.com/2018/10/08/受控与非受控组件/>
- <https://zhuanlan.zhihu.com/p/37579677>

面试官：说说对高阶组件的理解？应用场景？

一、高阶组件

高阶函数（Higher-order function），至少满足下列一个条件的函数

- 接受一个或多个函数作为输入
- 输出一个函数

在 `React` 中，高阶组件即接受一个或多个组件作为参数并且返回一个组件，本质也就是一个函数，并不是一个组件

```
const EnhancedComponent = highOrderComponent(WrappedComponent);
```

上述代码中，该函数接受一个组件 `WrappedComponent` 作为参数，返回加工过的新组件 `EnhancedComponent`

高阶组件的这种实现方式，本质上是一个装饰者设计模式

二、如何编写

最基本的高阶组件的编写模板如下：

```
import React, { Component } from 'react';

export default (WrappedComponent) => {
  return class EnhancedComponent extends Component {
    // do something
    render() {
      return <WrappedComponent />;
    }
  }
}
```

通过对传入的原始组件 `WrappedComponent` 做一些你想要的操作（比如操作 `props`，提取 `state`，给原始组件包裹其他元素等），从而加工出想要的组件 `EnhancedComponent`

把通用的逻辑放在高阶组件中，对组件实现一致的处理，从而实现代码的复用

所以，高阶组件的主要功能是封装并分离组件的通用逻辑，让通用逻辑在组件间更好地被复用

但在使用高阶组件的同时，一般遵循一些约定，如下：

- `props` 保持一致
- 你不能在函数式（无状态）组件上使用 `ref` 属性，因为它没有实例
- 不要以任何方式改变原始组件 `WrappedComponent`
- 透传不相关 `props` 属性给被包裹的组件 `WrappedComponent`
- 不要再 `render()` 方法中使用高阶组件
- 使用 `compose` 组合高阶组件
- 包装显示名字以便于调试

这里需要注意的是，高阶组件可以传递所有的 `props`，但是不能传递 `ref`

如果向一个高阶组件添加 `ref` 引用，那么 `ref` 指向的是最外层容器组件实例的，而不是被包裹的组件，如果需要传递 `refs` 的话，则使用 `React.forwardRef`，如下：

```
function withLogging(WrappedComponent) {
  class Enhance extends WrappedComponent {
    componentWillMount() {
      console.log('Current props', this.props);
      console.log('Next props', nextProps);
    }
    render() {
      const {forwardedRef, ...rest} = this.props;
      // 把 forwardedRef 赋值给 ref
      return <WrappedComponent {...rest} ref={forwardedRef} />;
    }
  };

  // React.forwardRef 方法会传入 props 和 ref 两个参数给其回调函数
  // 所以这边的 ref 是由 React.forwardRef 提供的
  function forwardRef(props, ref) {
    return <Enhance {...props} forwardedRef={ref} />
  }

  return React.forwardRef(forwardRef);
}
const EnhancedComponent = withLogging(SomeComponent);
```

三、应用场景

通过上面的了解，高阶组件能够提高代码的复用性和灵活性，在实际应用中，常常用于与核心业务无关但在多个模块使用的功能，如权限控制、日志记录、数据校验、异常处理、统计上报等

举个例子，存在一个组件，需要从缓存中获取数据，然后渲染。一般情况，我们会如下编写：

```
import React, { Component } from 'react'

class MyComponent extends Component {

  componentWillMount() {
    let data = localStorage.getItem('data');
    this.setState({data});
  }

  render() {
    return <div>{this.state.data}</div>
  }
}
```

上述代码当然可以实现该功能，但是如果还有其他组件也有类似功能的时候，每个组件都需要重复写 `componentWillMount` 中的代码，这明显是冗杂的

下面就可以通过高价组件来进行改写，如下：

```
import React, { Component } from 'react'

function withPersistentData(WrappedComponent) {
  return class extends Component {
    componentWillMount() {
      let data = localStorage.getItem('data');
      this.setState({data});
    }

    render() {
      // 通过{...this.props} 把传递给当前组件的属性继续传递给被包装的组件WrappedComponent
      return <WrappedComponent data={this.state.data} {...this.props} />
    }
  }
}

class MyComponent2 extends Component {
  render() {
    return <div>{this.props.data}</div>
  }
}

const MyComponentWithPersistentData = withPersistentData(MyComponent2)
```

再比如组件渲染性能监控，如下：

```
class Home extends React.Component {
  render() {
    return (<h1>Hello World.</h1>);
  }
}

function withTiming(WrappedComponent) {
  return class extends WrappedComponent {
    constructor(props) {
      super(props);
      this.start = 0;
      this.end = 0;
    }
    componentWillMount() {
      super.componentWillMount && super.componentWillMount();
      this.start = Date.now();
    }
    componentDidMount() {
      super.componentDidMount && super.componentDidMount();
      this.end = Date.now();
      console.log(`${WrappedComponent.name} 组件渲染时间为 ${this.end - this.start} ms`);
    }
    render() {
      return super.render();
    }
  };
}

export default withTiming(Home);
```

参考文献

- <https://zh-hans.reactjs.org/docs/higher-order-components.html#gatsby-focus-wrapper>
- <https://zh.wikipedia.org/wiki/%E9%AB%98%E9%98%B6%E5%87%BD%E6%95%B0>
- <https://segmentfault.com/a/1190000010307650>
- <https://zhuanlan.zhihu.com/p/61711492>

面试官：SPA首屏加载速度慢的怎么解决？

一、什么是首屏加载

首屏时间（First Contentful Paint），指的是浏览器从响应用户输入网址地址，到首屏内容渲染完成的时间，此时整个网页不一定要全部渲染完成，但需要展示当前视窗需要的内容

首屏加载可以说是用户体验中「最重要」的环节

关于计算首屏时间

利用 `performance.timing` 提供的数据：

通过 `DOMContentLoaded` 或者 `performance` 来计算出首屏时间

```
// 方案一：
document.addEventListener('DOMContentLoaded', (event) => {
  console.log('first contentful painting');
});
// 方案二：
performance.getEntriesByName("first-contentful-paint")[0].startTime

// performance.getEntriesByName("first-contentful-paint")[0]
// 会返回一个 PerformancePaintTiming的实例，结构如下：
{
  name: "first-contentful-paint",
  entryType: "paint",
  startTime: 507.80000002123415,
  duration: 0,
};
```

二、加载慢的原因

在页面渲染的过程，导致加载速度慢的因素可能如下：

- 网络延时问题
- 资源文件体积是否过大
- 资源是否重复发送请求去加载了
- 加载脚本的时候，渲染内容堵塞了

三、解决方案

常见的几种SPA首屏优化方式

- 减小入口文件体积
- 静态资源本地缓存
- UI框架按需加载
- 图片资源的压缩
- 组件重复打包
- 开启GZip压缩
- 使用SSR

减小入口文件体积

常用的手段是路由懒加载，把不同路由对应的组件分割成不同的代码块，待路由被请求的时候会单独打包路由，使得入口文件变小，加载速度大大增加

在 `vue-router` 配置路由的时候，采用动态加载路由的形式

```
routes:[
  path: 'Blogs',
  name: 'ShowBlogs',
  component: () => import('./components/ShowBlogs.vue')
]
```

以函数的形式加载路由，这样就可以把各自的路由文件分别打包，只有在解析给定的路由时，才会加载路由组件

静态资源本地缓存

后端返回资源问题：

- 采用 HTTP 缓存，设置 Cache-Control，Last-Modified，Etag 等响应头
- 采用 Service Worker 离线缓存

前端合理利用 localStorage

UI 框架按需加载

在日常使用 UI 框架，例如 element-UI、或者 antd，我们经常直接引用整个 UI 库

```
import ElementUI from 'element-ui'  
Vue.use(ElementUI)
```

但实际上我用到的组件只有按钮，分页，表格，输入与警告 所以我们要按需引用

```
import { Button, Input, Pagination, Table, TableColumn, MessageBox } from 'element-ui';  
Vue.use(Button)  
Vue.use(Input)  
Vue.use(Pagination)
```

组件重复打包

假设 A.js 文件是一个常用的库，现在有多个路由使用了 A.js 文件，这就造成了重复下载

解决方案：在 webpack 的 config 文件中，修改 CommonsChunkPlugin 的配置

```
minChunks: 3
```

minChunks 为 3 表示会把使用 3 次及以上的包抽离出来，放进公共依赖文件，避免了重复加载组件

图片资源的压缩

图片资源虽然不在编码过程中，但它却是对页面性能影响最大的因素

对于所有的图片资源，我们可以进行适当的压缩

对页面上使用到的 icon，可以使用在线字体图标，或者雪碧图，将众多小图标合并到同一张图上，用以减轻 http 请求压力。

开启 GZip 压缩

拆完包之后，我们再用 gzip 做一下压缩 安装 compression-webpack-plugin

```
cnmp i compression-webpack-plugin -D
```

在 vue.config.js 中引入并修改 webpack 配置

```
const CompressionPlugin = require('compression-webpack-plugin')

configureWebpack: (config) => {
  if (process.env.NODE_ENV === 'production') {
    // 为生产环境修改配置...
    config.mode = 'production'
    return {
      plugins: [new CompressionPlugin({
        test: /\.js$|\.html$|\.css/, //匹配文件名
        threshold: 10240, //对超过10k的数据进行压缩
        deleteOriginalAssets: false //是否删除原文件
      })]
    }
  }
}
```

在服务器我们也要做相应的配置 如果发送请求的浏览器支持 `gzip`，就发送给它 `gzip` 格式的文件 我的服务器是用 `express` 框架搭建的 只要安装一下 `compression` 就能使用

```
const compression = require('compression')
app.use(compression()) // 在其他中间件使用之前调用
```

使用SSR

SSR (Server side)，也就是服务端渲染，组件或页面通过服务器生成html字符串，再发送到浏览器 从头搭建一个服务端渲染是很复杂的，`vue` 应用建议使用 `Nuxt.js` 实现服务端渲染

小结：

减少首屏渲染时间的方法有很多，总的来讲可以分成两大部分：资源加载优化 和 页面渲染优化 下图是更为全面的首屏优化的方案

大家可以根据自己项目的情况选择各种方式进行首屏渲染的优化

参考文献

- https://zhuanlan.zhihu.com/p/88639980?utm_source=wechat_session
- <https://www.chengrang.com/how-browsers-work.html>
- <https://juejin.cn/post/6844904185264095246>

面试官：SSR解决了什么问题？有做过SSR吗？你是怎么做的？

一、SSR

`Server-Side Rendering` 我们称其为 `SSR`，意为服务端渲染

指由服务侧完成页面的 `HTML` 结构拼接的页面处理技术，发送到浏览器，然后为其绑定状态与事件，成为完全可交互页面的过程

先来看看 Web 3 个阶段的发展史：

- 传统服务端渲染 SSR
- 单页面应用 SPA
- 服务端渲染 SSR

「传统web开发」

网页内容在服务端渲染完成，一次性传输到浏览器

打开页面查看源码，浏览器拿到的是全部的 dom 结构

「单页应用SPA」

单页应用优秀的用户体验，使其逐渐成为主流，页面内容由 JS 渲染出来，这种方式称为客户端渲染

打开页面查看源码，浏览器拿到的仅有宿主元素 #app，并没有内容

服务端渲染SSR

SSR 解决方案，后端渲染出完整的首屏的 dom 结构返回，前端拿到的内容包括首屏及完整 spa 结构，应用激活后依然按照 spa 方式运行

看完前端发展，我们再看看 Vue 官方对 SSR 的解释：

“

Vue.js 是构建客户端应用程序的框架。默认情况下，可以在浏览器中输出 Vue 组件，进行生成 DOM 和操作 DOM。然而，也可以将同一个组件渲染为服务器端的 HTML 字符串，将它们直接发送到浏览器，最后将这些静态标记“激活”为客户端上完全可交互的应用程序

服务器渲染的 Vue.js 应用程序也可以被认为是“同构”或“通用”，因为应用程序的大部分代码都可以在服务器和客户端上运行

”

我们从上解释得到以下结论：

- Vue SSR 是一个在 SPA 上进行改良的服务端渲染
- 通过 Vue SSR 渲染的页面，需要在客户端激活才能实现交互
- Vue SSR 将包含两部分：服务端渲染的首屏，包含交互的 SPA

二、解决了什么

SSR 主要解决了以下两种问题：

- seo：搜索引擎优先爬取页面 HTML 结构，使用 SSR 时，服务端已经生成了和业务想关联的 HTML，有利于 seo
- 首屏呈现渲染：用户无需等待页面所有 js 加载完成就可以看到页面视图（压力来到了服务器，所以需要权衡哪些用服务端渲染，哪些交给客户端）

但是使用 SSR 同样存在以下的缺点：

- 复杂度：整个项目的复杂度
- 库的支持性，代码兼容
- 性能问题
 - 每个请求都是 n 个实例的创建，不然会污染，消耗会变得很大
 - 缓存 node serve、nginx 判断当前用户有没有过期，如果没过期的话就缓存，用刚刚的结果。
 - 降级：监控 cpu、内存占用过多，就 spa，返回单个的壳
- 服务器负载变大，相对于前后端分离服务器只需要提供静态资源来说，服务器负载更大，所以要慎重使用

所以在我们选择是否使用 SSR 前，我们需要慎重问问自己这些问题：

1. 需要 SEO 的页面是否只是少数几个，这些是否可以使用预渲染（Prerender SPA Plugin）实现
2. 首屏的请求响应逻辑是否复杂，数据返回是否大量且缓慢

三、如何实现

对于同构开发，我们依然使用 webpack 打包，我们要解决两个问题：服务端首屏渲染和客户端激活
这里需要生成一个服务器 bundle 文件用于服务端首屏渲染和一个客户端 bundle 文件用于客户端激活

代码结构 除了两个不同入口之外，其他结构和之前 vue 应用完全相同

```
src
├── router
├── index.js # 路由声明
├── store
├── index.js # 全局状态
├── main.js # 用于创建vue实例
├── entry-client.js # 客户端入口，用于静态内容“激活”
└── entry-server.js # 服务端入口，用于首屏内容渲染
```

路由配置

```

import Vue from "vue";
import Router from "vue-router";

Vue.use(Router);
//导出工厂函数

export function createRouter() {
  return new Router({
    mode: 'history',
    routes: [
      // 客户端没有编译器，这里要写成渲染函数
      { path: "/", component: { render: h => h('div', 'index page') } },
      { path: "/detail", component: { render: h => h('div', 'detail page') } }
    ]
  });
}

```

主文件main.js

跟之前不同，主文件是负责创建 `vue` 实例的工厂，每次请求均会有独立的 `vue` 实例创建

```

import Vue from "vue";
import App from "./App.vue";
import { createRouter } from "./router";
// 导出Vue实例工厂函数，为每次请求创建独立实例
// 上下文用于给vue实例传递参数
export function createApp(context) {
  const router = createRouter();
  const app = new Vue({
    router,
    context,
    render: h => h(App)
  });
  return { app, router };
}

```

编写服务端入口 `src/entry-server.js`

它的任务是创建 `Vue` 实例并根据传入 `url` 指定首屏


```
import { createApp } from './main';
// 返回一个函数，接收请求上下文，返回创建的vue实例
export default context => {
  // 这里返回一个Promise，确保路由或组件准备就绪
  return new Promise((resolve, reject) => {
    const { app, router } = createApp(context);
    // 跳转到首屏的地址
    router.push(context.url);
    // 路由就绪，返回结果
    router.onReady(() => {
      resolve(app);
    }, reject);
  });
};
```

编写客户端入口 `entry-client.js`

客户端入口只需创建 `vue` 实例并执行挂载，这一步称为激活

```
import { createApp } from './main';
// 创建vue、router实例
const { app, router } = createApp();
// 路由就绪，执行挂载
router.onReady(() => {
  app.$mount("#app");
});
```

对 `webpack` 进行配置

安装依赖

```
npm install webpack-node-externals lodash.merge -D
```

对 `vue.config.js` 进行配置

```
// 两个插件分别负责打包客户端和服务端
const VueSSRServerPlugin = require("vue-server-renderer/server-plugin");
const VueSSRClientPlugin = require("vue-server-renderer/client-plugin");
const nodeExternals = require("webpack-node-externals");
const merge = require("lodash.merge");
// 根据传入环境变量决定入口文件和相应配置项
const TARGET_NODE = process.env.WEBPACK_TARGET === "node";
const target = TARGET_NODE ? "server" : "client";
module.exports = {
  css: {
    extract: false
  },
  outputDir: './dist'+target,
  configureWebpack: () => ({
    // 将 entry 指向应用程序的 server / client 文件
    entry: `./src/entry-${target}.js`,
```

```

// 对 bundle renderer 提供 source map 支持
devtool: 'source-map',
// target设置为node使webpack以Node适用的方式处理动态导入,
// 并且还会在编译Vue组件时告知`vue-loader`输出面向服务器代码。
target: TARGET_NODE ? "node" : "web",
// 是否模拟node全局变量
node: TARGET_NODE ? undefined : false,
output: {
  // 此处使用Node风格导出模块
  libraryTarget: TARGET_NODE ? "commonjs2" : undefined
},
// https://webpack.js.org/configuration/externals/#function
// https://github.com/liady/webpack-node-externals
// 外置化应用程序依赖模块。可以使服务器构建速度更快, 并生成较小的打包文件。
externals: TARGET_NODE
? nodeExternals({
  // 不要外置化webpack需要处理的依赖模块。
  // 可以在这里添加更多的文件类型。例如, 未处理 *.vue 原始文件,
  // 还应该将修改`global` (例如polyfill) 的依赖模块列入白名单
  whitelist: [/\.css$/]
})
: undefined,
optimization: {
  splitChunks: undefined
},
// 这是将服务器的整个输出构建为单个 JSON 文件的插件。
// 服务端默认文件名为 `vue-ssr-server-bundle.json`
// 客户端默认文件名为 `vue-ssr-client-manifest.json`。
plugins: [TARGET_NODE ? new VueSSRServerPlugin() : new
  VueSSRClientPlugin()]
),
chainWebpack: config => {
  // cli4项目添加
  if (TARGET_NODE) {
    config.optimization.delete('splitChunks')
  }

  config.module
    .rule("vue")
    .use("vue-loader")
    .tap(options => {
      merge(options, {
        optimizeSSR: false
      });
    });
});
}
};

```

对脚本进行配置, 安装依赖

```
npm i cross-env -D
```

定义创建脚本 `package.json`

```
"scripts": {  
  "build:client": "vue-cli-service build",  
  "build:server": "cross-env WEBPACK_TARGET=node vue-cli-service build",  
  "build": "npm run build:server && npm run build:client"  
}
```

“

执行打包：npm run build

”

最后修改宿主文件 `/public/index.html`

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="utf-8">  
    <meta http-equiv="X-UA-Compatible" content="IE=edge">  
    <meta name="viewport" content="width=device-width,initial-scale=1.0">  
    <title>Document</title>  
  </head>  
  <body>  
    <!--vue-ssr-outlet-->  
  </body>  
</html>
```

“

`<!--vue-ssr-outlet-->` 是服务端渲染入口位置，注意不能为了好看而在前后加空格

”

安装 `vuex`

```
npm install -S vuex
```

创建 `vuex` 工厂函数

```
import Vue from 'vue'
import Vuex from 'vuex'
Vue.use(Vuex)
export function createStore () {
  return new Vuex.Store({
    state: {
      count:108
    },
    mutations: {
      add(state){
        state.count += 1;
      }
    }
  })
}
```

在 `main.js` 文件中挂载 `store`

```
import { createStore } from './store'
export function createApp (context) {
  // 创建实例
  const store = createStore()
  const app = new Vue({
    store, // 挂载
    render: h => h(App)
  })
  return { app, router, store }
}
```

服务器端渲染的是应用程序的"快照"，如果应用依赖于一些异步数据，那么在开始渲染之前，需要先预取和解析好这些数据

在 `store` 进行一步数据获取

```

export function createStore() {
  return new Vuex.Store({
    mutations: {
      // 加一个初始化
      init(state, count) {
        state.count = count;
      },
    },
    actions: {
      // 加一个异步请求count的action
      getCount({ commit }) {
        return new Promise(resolve => {
          setTimeout(() => {
            commit("init", Math.random() * 100);
            resolve();
          }, 1000);
        });
      },
    },
  });
}

```

组件中的数据预取逻辑

```

export default {
  asyncData({ store, route }) { // 约定预取逻辑编写在预取钩子asyncData中
    // 触发 action 后, 返回 Promise 以便确定请求结果
    return store.dispatch("getCount");
  }
};

```

服务端数据预取, [entry-server.js](#)


```

import { createApp } from './app';
export default context => {
  return new Promise((resolve, reject) => {
    // 拿出store和router实例
    const { app, router, store } = createApp(context);
    router.push(context.url);
    router.onReady(() => {
      // 获取匹配的路由组件数组
      const matchedComponents = router.getMatchedComponents();

      // 若无匹配则抛出异常
      if (!matchedComponents.length) {
        return reject({ code: 404 });
      }

      // 对所有匹配的路由组件调用可能存在的`asyncData()`
      Promise.all(
        matchedComponents.map(Component => {
          if (Component.asyncData) {
            return Component.asyncData({
              store,
              route: router.currentRoute,
            });
          }
        })
      )
        .then(() => {
          // 所有预取钩子 resolve 后,
          // store 已经填充入渲染应用所需状态
          // 将状态附加到上下文, 且 `template` 选项用于 renderer 时,
          // 状态将自动序列化为 `window.__INITIAL_STATE__`, 并注入 HTML
          context.state = store.state;

          resolve(app);
        })
        .catch(reject);
    }, reject);
  });
};

```

客户端在挂载到应用程序之前，`store` 就应该获取到状态，`entry-client.js`

```

// 导出store
const { app, router, store } = createApp();
// 当使用 template 时, context.state 将作为 window.__INITIAL_STATE__ 状态自动嵌入到最终的 HTML
// 在客户端挂载到应用程序之前, store 就应该获取到状态:
if (window.__INITIAL_STATE__) {
  store.replaceState(window.__INITIAL_STATE__);
}

```

客户端数据预取处理，`main.js`

```
Vue.mixin({
  beforeMount() {
    const { asyncData } = this.$options;
    if (asyncData) {
      // 将获取数据操作分配给 promise
      // 以便在组件中，我们可以在数据准备就绪后
      // 通过运行 `this.dataPromise.then(...)` 来执行其他任务
      this.dataPromise = asyncData({
        store: this.$store,
        route: this.$route,
      });
    }
  },
});
```

修改服务器启动文件

```
// 获取文件路径
const resolve = dir => require('path').resolve(__dirname, dir)
// 第 1 步：开放dist/client目录，关闭默认下载index页的选项，不然到不了后面路由
app.use(express.static(resolve('../dist/client'), {index: false}))
// 第 2 步：获得一个createBundleRenderer
const { createBundleRenderer } = require('vue-server-renderer');
// 第 3 步：服务端打包文件地址
const bundle = resolve("../dist/server/vue-ssr-server-bundle.json");
// 第 4 步：创建渲染器
const renderer = createBundleRenderer(bundle, {
  runInNewContext: false, // https://ssr.vuejs.org/zh/api/#runinnewcontext
  template: require('fs').readFileSync(resolve("../public/index.html"), "utf8"), // 宿主文件
  clientManifest: require(resolve("../dist/client/vue-ssr-clientmanifest.json")) // 客户端清单
});
app.get("", async (req, res) => {
  // 设置url和title两个重要参数
  const context = {
    title: 'ssr test',
    url: req.url
  }
  const html = await renderer.renderToString(context);
  res.send(html)
})
```

小结

- 使用 `ssr` 不存在单例模式，每次用户请求都会创建一个新的 `vue` 实例
- 实现 `ssr` 需要实现服务端首屏渲染和客户端激活
- 服务端异步获取数据 `asyncData` 可以分为首屏异步获取和切换组件获取
 - 首屏异步获取数据，在服务端预渲染的时候就应该已经完成
 - 切换组件通过 `mixin` 混入，在 `beforeMount` 钩子完成数据获取

参考文献

面试官：Vue.observable你有了解过吗？说说看

一、Observable 是什么

Observable 翻译过来我们可以理解成「可观察的」

我们先来看一下其在 **Vue** 中的定义

“

Vue.observable，让一个对象变成响应式数据。**Vue** 内部会用它来处理 **data** 函数返回的对象

”

返回的对象可以直接用于渲染函数和计算属性内，并且会在发生变更时触发相应的更新。也可以作为最小化的跨组件状态存储器

```
Vue.observable({ count : 1 })
```

其作用等同于

```
new vue({ count : 1 })
```

在 **Vue 2.x** 中，被传入的对象会直接被 **Vue.observable** 变更，它和被返回的对象是同一个对象

在 **Vue 3.x** 中，则会返回一个可响应的代理，而对源对象直接进行变更仍然是不可响应的

二、使用场景

在非父子组件通信时，可以使用通常的 **bus** 或者使用 **vuex**，但是实现的功能不是太复杂，而使用上面两个又有点繁琐。这时，**observable** 就是一个很好的选择

创建一个 **js** 文件

```
// 引入vue
import Vue from 'vue'
// 创建state对象，使用observable让state对象可响应
export let state = Vue.observable({
  name: '张三',
  'age': 38
})
// 创建对应的方法
export let mutations = {
  changeName(name) {
    state.name = name
  },
  setAge(age) {
    state.age = age
  }
}
```

在 `.vue` 文件中直接使用即可

```
<template>
  <div>
    姓名: {{ name }}
    年龄: {{ age }}
    <button @click="changeName('李四')">改变姓名</button>
    <button @click="setAge(18)">改变年龄</button>
  </div>
</template>
import { state, mutations } from '@store'
export default {
  // 在计算属性中拿到值
  computed: {
    name() {
      return state.name
    },
    age() {
      return state.age
    }
  },
  // 调用mutations里面的方法，更新数据
  methods: {
    changeName: mutations.changeName,
    setAge: mutations.setAge
  }
}
```

三、原理分析

源码位置：`src\core\observer\index.js`

```

export function observe (value: any, asRootData: ?boolean): Observer | void {
  if (!isObject(value) || value instanceof VNode) {
    return
  }
  let ob: Observer | void
  // 判断是否存在__ob__响应式属性
  if (hasOwn(value, '__ob__') && value.__ob__ instanceof Observer) {
    ob = value.__ob__
  } else if (
    shouldObserve &&
    !isServerRendering() &&
    (Array.isArray(value) || isPlainObject(value)) &&
    Object.isExtensible(value) &&
    !value._isVue
  ) {
    // 实例化Observer响应式对象
    ob = new Observer(value)
  }
  if (asRootData && ob) {
    ob.vmCount++
  }
  return ob
}

```

Observer 类

```

export class Observer {
  value: any;
  dep: Dep;
  vmCount: number; // number of vms that have this object as root $data

  constructor (value: any) {
    this.value = value
    this.dep = new Dep()
    this.vmCount = 0
    def(value, '__ob__', this)
    if (Array.isArray(value)) {
      if (hasProto) {
        protoAugment(value, arrayMethods)
      } else {
        copyAugment(value, arrayMethods, arrayKeys)
      }
      this.observeArray(value)
    } else {
      // 实例化对象是一个对象，进入walk方法
      this.walk(value)
    }
  }
}

```

walk 函数


```

walk (obj: Object) {
  const keys = Object.keys(obj)
  // 遍历key, 通过defineReactive创建响应式对象
  for (let i = 0; i < keys.length; i++) {
    defineReactive(obj, keys[i])
  }
}

```

defineReactive 方法

```

export function defineReactive (
  obj: Object,
  key: string,
  val: any,
  customSetter?: ?Function,
  shallow?: boolean
) {
  const dep = new Dep()

  const property = Object.getOwnPropertyDescriptor(obj, key)
  if (property && property.configurable === false) {
    return
  }

  // cater for pre-defined getter/setters
  const getter = property && property.get
  const setter = property && property.set
  if ((!getter || setter) && arguments.length === 2) {
    val = obj[key]
  }

  let childOb = !shallow && observe(val)
  // 接下来调用Object.defineProperty()给对象定义响应式属性
  Object.defineProperty(obj, key, {
    enumerable: true,
    configurable: true,
    get: function reactiveGetter () {
      const value = getter ? getter.call(obj) : val
      if (Dep.target) {
        dep.depend()
        if (childOb) {
          childOb.dep.depend()
          if (Array.isArray(value)) {
            dependArray(value)
          }
        }
      }
    },
    set: function reactiveSetter (newVal) {
      const value = getter ? getter.call(obj) : val
      /* eslint-disable no-self-compare */

```

```
if (newVal === value || (newVal !== newVal && value !== value)) {
  return
}
/* eslint-enable no-self-compare */
if (process.env.NODE_ENV !== 'production' && customSetter) {
  customSetter()
}
// #7981: for accessor properties without setter
if (getter && !setter) return
if (setter) {
  setter.call(obj, newVal)
} else {
  val = newVal
}
childOb = !shallow && observe(newVal)
// 对观察者watchers进行通知,state就成了全局响应式对象
dep.notify()
}
})
}
```

参考文献

- https://blog.csdn.net/qq_32682301/article/details/105419673
- <https://wbbyouzi.com/archives/343>

面试官：Vue中的\$nextTick有什么作用？

一、NextTick是什么

官方对其的定义

“

在下次 DOM 更新循环结束之后执行延迟回调。在修改数据之后立即使用这个方法，获取更新后的 DOM

”

什么意思呢？

我们可以理解成，Vue 在更新 DOM 时是异步执行的。当数据发生变化，Vue 将开启一个异步更新队列，视图需要等队列中所有数据变化完成之后，再统一进行更新

举例一下

Html 结构

```
<div id="app">{{ message }}</div>
```

构建一个 vue 实例

```
const vm = new Vue({
  el: '#app',
  data: {
    message: '原始值'
  }
})
```

修改 `message`

```
this.message = '修改后的值1'
this.message = '修改后的值2'
this.message = '修改后的值3'
```

这时候想获取页面最新的 `DOM` 节点，却发现获取到的是旧值

```
console.log(vm.$el.textContent) // 原始值
```

这是因为 `message` 数据在发现变化的时候，`vue` 并不会立刻去更新 `Dom`，而是将修改数据的操作放在了一个异步操作队列中

如果我们一直修改相同数据，异步操作队列还会进行去重

等待同一事件循环中的所有数据变化完成之后，会将队列中的事件拿来进行处理，进行 `DOM` 的更新

为什么要有nexttick

举个例子

```
{{num}}
for(let i=0; i<100000; i++){
  num = i
}
```

如果没有 `nextTick` 更新机制，那么 `num` 每次更新值都会触发视图更新(上面这段代码也就是会更新10万次视图)，有了 `nextTick` 机制，只需要更新一次，所以 `nextTick` 本质是一种优化策略

二、使用场景

如果想要在修改数据后立刻得到更新后的 `DOM` 结构，可以使用 `Vue.nextTick()`

第一个参数为：回调函数（可以获取最近的 `DOM` 结构）

第二个参数为：执行函数上下文

```
// 修改数据
vm.message = '修改后的值'
// DOM 还没有更新
console.log(vm.$el.textContent) // 原始的值
Vue.nextTick(function () {
  // DOM 更新了
  console.log(vm.$el.textContent) // 修改后的值
})
```

组件内使用 `vm.$nextTick()` 实例方法只需要通过 `this.$nextTick()`，并且回调函数中的 `this` 将自动绑定到当前的 `Vue` 实例上

```
this.message = '修改后的值'
console.log(this.$el.textContent) // => '原始的值'
this.$nextTick(function () {
  console.log(this.$el.textContent) // => '修改后的值'
})
```

`$nextTick()` 会返回一个 `Promise` 对象，可以用 `async/await` 完成相同作用的事情

```
this.message = '修改后的值'
console.log(this.$el.textContent) // => '原始的值'
await this.$nextTick()
console.log(this.$el.textContent) // => '修改后的值'
```

三、实现原理

源码位置： [/src/core/util/next-tick.js](#)

`callbacks` 也就是异步操作队列

`callbacks` 新增回调函数后又执行了 `timerFunc` 函数，`pending` 是用来标识同一个时间只能执行一次

```

export function nextTick(cb?: Function, ctx?: Object) {
  let _resolve;

  // cb 回调函数会经统一处理压入 callbacks 数组
  callbacks.push(() => {
    if (cb) {
      // 给 cb 回调函数执行加上了 try-catch 错误处理
      try {
        cb.call(ctx);
      } catch (e) {
        handleError(e, ctx, 'nextTick');
      }
    } else if (_resolve) {
      _resolve(ctx);
    }
  });

  // 执行异步延迟函数 timerFunc
  if (!pending) {
    pending = true;
    timerFunc();
  }

  // 当 nextTick 没有传入函数参数的时候，返回一个 Promise 化的调用
  if (!cb && typeof Promise !== 'undefined') {
    return new Promise(resolve => {
      _resolve = resolve;
    });
  }
}

```

`timerFunc` 函数定义，这里是根据当前环境支持什么方法则确定调用哪个，分别有：

`Promise.then`、`MutationObserver`、`setImmediate`、`setTimeout`

通过上面任意一种方法，进行降级操作


```

export let isUsingMicroTask = false
if (typeof Promise !== 'undefined' && isNative(Promise)) {
  //判断1: 是否原生支持Promise
  const p = Promise.resolve()
  timerFunc = () => {
    p.then(flushCallbacks)
    if (isIOS) setTimeout(noop)
  }
  isUsingMicroTask = true
} else if (!isIE && typeof MutationObserver !== 'undefined' && (
  isNative(MutationObserver) ||
  MutationObserver.toString() === '[object MutationObserverConstructor]'
)) {
  //判断2: 是否原生支持MutationObserver
  let counter = 1
  const observer = new MutationObserver(flushCallbacks)
  const textNode = document.createTextNode(String(counter))
  observer.observe(textNode, {
    characterData: true
  })
  timerFunc = () => {
    counter = (counter + 1) % 2
    textNode.data = String(counter)
  }
  isUsingMicroTask = true
} else if (typeof setImmediate !== 'undefined' && isNative(setImmediate)) {
  //判断3: 是否原生支持setImmediate
  timerFunc = () => {
    setImmediate(flushCallbacks)
  }
} else {
  //判断4: 上面都不行, 直接用setTimeout
  timerFunc = () => {
    setTimeout(flushCallbacks, 0)
  }
}

```

无论是微任务还是宏任务，都会放到 `flushCallbacks` 使用

这里将 `callbacks` 里面的函数复制一份，同时 `callbacks` 置空

依次执行 `callbacks` 里面的函数

```

function flushCallbacks () {
  pending = false
  const copies = callbacks.slice(0)
  callbacks.length = 0
  for (let i = 0; i < copies.length; i++) {
    copies[i]()
  }
}

```

「小结：」

1. 把回调函数放入callbacks等待执行
2. 将执行函数放到微任务或者宏任务中
3. 事件循环到了微任务或者宏任务，执行函数依次执行callbacks中的回调

参考文献

◦ <https://juejin.cn/post/6844904147804749832>

面试官：Vue中的过滤器了解吗？过滤器的应用场景有哪些？

一、filter

过滤器（[filter](#)）是输送介质管道上不可缺少的一种装置

大白话，就是把一些不必要的东西过滤掉

过滤器实质不改变原始数据，只是对数据进行加工处理后返回过滤后的数据再进行调用处理，我们也可以理解其为一个纯函数

[Vue](#) 允许你自定义过滤器，可被用于一些常见的文本格式化

ps: [Vue3](#) 中已废弃 [filter](#)

二、如何用

[vue](#) 中的过滤器可以用在两个地方：双花括号插值和 [v-bind](#) 表达式，过滤器应该被添加在 [JavaScript](#) 表达式的尾部，由“管道”符号指示：

```
<!-- 在双花括号中 -->
{{ message | capitalize }}

<!-- 在 `v-bind` 中 -->
<div v-bind:id="rawId | formatId"></div>
```

定义filter

在组件的选项中定义本地的过滤器

```
filters: {
  capitalize: function (value) {
    if (!value) return ''
    value = value.toString()
    return value.charAt(0).toUpperCase() + value.slice(1)
  }
}
```

定义全局过滤器：

```
Vue.filter('capitalize', function (value) {
  if (!value) return ""
  value = value.toString()
  return value.charAt(0).toUpperCase() + value.slice(1)
})

new Vue({
  // ...
})
```

注意：当全局过滤器和局部过滤器重名时，会采用局部过滤器

过滤器函数总接收表达式的值（之前的操作链的结果）作为第一个参数。在上述例子中，`capitalize` 过滤器函数将会收到 `message` 的值作为第一个参数

过滤器可以串联：

```
{{ message | filterA | filterB }}
```

在这个例子中，`filterA` 被定义为接收单个参数的过滤器函数，表达式 `message` 的值将作为参数传入到函数中。然后继续调用同样被定义为接收单个参数的过滤器函数 `filterB`，将 `filterA` 的结果传递到 `filterB` 中。

过滤器是 `JavaScript` 函数，因此可以接收参数：

```
{{ message | filterA('arg1', arg2) }}
```

这里，`filterA` 被定义为接收三个参数的过滤器函数。

其中 `message` 的值作为第一个参数，普通字符串 `'arg1'` 作为第二个参数，表达式 `arg2` 的值作为第三个参数

举个例子：

```
<div id="app">
  <p>{{ msg | msgFormat('疯狂', '--)}}</p>
</div>

<script>
  // 定义一个 Vue 全局的过滤器，名字叫做 msgFormat
  Vue.filter('msgFormat', function(msg, arg, arg2) {
    // 字符串的 replace 方法，第一个参数，除了可写一个字符串之外，还可以定义一个正则
    return msg.replace(/单纯/g, arg+arg2)
  })
</script>
```

小结：

- 局部过滤器优先于全局过滤器被调用
- 一个表达式可以使用多个过滤器。过滤器之间需要用管道符“|”隔开。其执行顺序从左往右

三、应用场景

平时开发中，需要用到过滤器的地方有很多，比如单位转换、数字打点、文本格式化、时间格式化之类的等

比如我们要实现将30000 => 30,000，这时候我们就需要使用过滤器

```
Vue.filter('toThousandFilter', function (value) {  
  if (!value) return ''  
  value = value.toString()  
  return .replace(str.indexOf('.') > -1 ? /(\d)(?=(\d{3})+\.)/g : /(\d)(?=(?!\d{3})+$)/g, '$1,')  
})
```

四、原理分析

使用过滤器

```
{{ message | capitalize }}
```

在模板编译阶段过滤器表达式将会被编译为过滤器函数，主要是用过 `parseFilters`，我们放到最后讲

```
_s(_f('filterFormat')(message))
```

首先分析一下 `_f`：

`_f` 函数全名是：`resolveFilter`，这个函数的作用是从 `this.$options.filters` 中找出注册的过滤器并返回

```
// 变为  
this.$options.filters['filterFormat'](message) // message为参数
```

关于 `resolveFilter`

```
import { identity, resolveAsset } from 'core/util/index'  
  
export function resolveFilter(id){  
  return resolveAsset(this.$options, 'filters', id, true) || identity  
}
```

内部直接调用 `resolveAsset`，将 `option` 对象，类型，过滤器 `id`，以及一个触发警告的标志作为参数传递，如果找到，则返回过滤器；

`resolveAsset` 的代码如下：

```

export function resolveAsset(options,type,id,warnMissing){ // 因为我们找的是过滤器，所以在
resolveFilter函数中调用时 type 的值直接给的 'filters',实际这个函数还可以拿到其他很多东西
  if(typeof id !== 'string'){ // 判断传递的过滤器id 是不是字符串， 不是则直接返回
    return
  }
  const assets = options[type] // 将我们注册的所有过滤器保存在变量中
  // 接下来的逻辑便是判断id是否在assets中存在， 即进行匹配
  if(hasOwn(assets,id)) return assets[id] // 如找到， 直接返回过滤器
  // 没有找到， 代码继续执行
  const camelizedId = camelize(id) // 万一你是驼峰的呢
  if(hasOwn(assets,camelizedId)) return assets[camelizedId]
  // 没找到， 继续执行
  const PascalCaseId = capitalize(camelizedId) // 万一你是首字母大写的驼峰呢
  if(hasOwn(assets,PascalCaseId)) return assets[PascalCaseId]
  // 如果还是没找到， 则检查原型链(即访问属性)
  const result = assets[id] || assets[camelizedId] || assets[PascalCaseId]
  // 如果依然没找到， 则在非生产环境的控制台打印警告
  if(process.env.NODE_ENV !== 'production' && warnMissing && !result){
    warn('Failed to resolve ' + type.slice(0,-1) + ':' + id, options)
  }
  // 无论是否找到， 都返回查找结果
  return result
}

```

下面再来分析一下 `_s`：

`_s` 函数的全称是 `toString`，过滤器处理后的结果会当作参数传递给 `toString` 函数，最终 `toString` 函数执行后的结果会保存到 `Vnode` 中的 `text` 属性中，渲染到视图中

```

function toString(value){
  return value == null
  ? ''
  : typeof value === 'object'
  ? JSON.stringify(value,null,2) // JSON.stringify() 第三个参数可用来控制字符串里面的间距
  : String(value)
}

```

最后，在分析下 `parseFilters`，在模板编译阶段使用该函数阶段将模板过滤器解析为过滤器函数调用表达式


```

function parseFilters (filter) {
  let filters = filter.split('|')
  let expression = filters.shift().trim() // shift()删除数组第一个元素并将其返回，该方法会更改原数组
  let i
  if (filters) {
    for(i = 0; i < filters.length; i++){
      experssion = warpFilter(expression, filters[i].trim()) // 这里传进去的expression实际上是管道
      // 符号前面的字符串，即过滤器的第一个参数
    }
  }
  return expression
}
// warpFilter函数实现
function warpFilter(exp, filter){
  // 首先判断过滤器是否有其他参数
  const i = filter.indexOf('(')
  if(i < 0){ // 不含其他参数，直接进行过滤器表达式字符串的拼接
    return `_f("${filter}")(${exp})`
  }else{
    const name = filter.slice(0, i) // 过滤器名称
    const args = filter.slice(i+1) // 参数，但还多了 ')'
    return `_f("${name}")(${exp}, ${args})` // 注意这一步少给了一个 ')'
  }
}

```

小结：

- 在编译阶段通过 `parseFilters` 将过滤器编译成函数调用（串联过滤器则是一个嵌套的函数调用，前一个过滤器执行的结果是后一个过滤器函数的参数）
- 编译后通过调用 `resolveFilter` 函数找到对应过滤器并返回结果
- 执行结果作为参数传递给 `toString` 函数，而 `toString` 执行后，其结果会保存在 `Vnode` 的 `text` 属性中，渲染到视图

参考文献

- https://blog.csdn.net/weixin_42724176/article/details/105546684

面试官：Vue中组件和插件有什么区别？

一、组件是什么

回顾以前对组件的定义：

组件就是把图形、非图形的各种逻辑均抽象为一个统一的概念（组件）来实现开发的模式，在 `Vue` 中每一个 `.vue` 文件都可以视为一个组件

组件的优势

- 降低整个系统的耦合度，在保持接口不变的情况下，我们可以替换不同的组件快速完成需求，例如输入框，可以替换为日历、时间、范围等组件作具体的实现

- 调试方便，由于整个系统是通过组件组合起来的，在出现问题的时候，可以用排除法直接移除组件，或者根据报错的组件快速定位问题，之所以能够快速定位，是因为每个组件之间低耦合，职责单一，所以逻辑会比分析整个系统要简单
- 提高可维护性，由于每个组件的职责单一，并且组件在系统中是被复用的，所以对代码进行优化可获得系统的整体升级

二、插件是什么

插件通常用来为 `Vue` 添加全局功能。插件的功能范围没有严格的限制——一般有下面几种：

- 添加全局方法或者属性。如： `vue-custom-element`
- 添加全局资源：指令/过滤器/过渡等。如 `vue-touch`
- 通过全局混入来添加一些组件选项。如 `vue-router`
- 添加 `Vue` 实例方法，通过把它们添加到 `Vue.prototype` 上实现。
- 一个库，提供自己的 `API`，同时提供上面提到的一个或多个功能。如 `vue-router`

三、两者的区别

两者的区别主要表现在以下几个方面：

- 编写形式
- 注册形式
- 使用场景

编写形式

编写组件

编写一个组件，可以有很多方式，我们最常见的就是 `vue` 单文件的这种格式，每一个 `.vue` 文件我们都可以看成是一个组件

`vue` 文件标准格式

```
<template>
</template>
<script>
export default{
  ...
}
</script>
<style>
</style>
```

我们还可以通过 `template` 属性来编写一个组件，如果组件内容多，我们可以在外部定义 `template` 组件内容，如果组件内容并不多，我们可直接写在 `template` 属性上

```
<template id="testComponent">  // 组件显示的内容
  <div>component!</div>
</template>

Vue.component('componentA',{
  template: '#testComponent'
  template: '<div>component</div>' // 组件内容少可以通过这种形式
})
```

编写插件

vue 插件的实现应该暴露一个 `install` 方法。这个方法的第一个参数是 `Vue` 构造器，第二个参数是一个可选的选项对象

```
MyPlugin.install = function (Vue, options) {
  // 1. 添加全局方法或 property
  Vue.myGlobalMethod = function () {
    // 逻辑...
  }

  // 2. 添加全局资源
  Vue.directive('my-directive', {
    bind (el, binding, vnode, oldVnode) {
      // 逻辑...
    }
    ...
  })

  // 3. 注入组件选项
  Vue.mixin({
    created: function () {
      // 逻辑...
    }
    ...
  })

  // 4. 添加实例方法
  Vue.prototype.$myMethod = function (methodOptions) {
    // 逻辑...
  }
}
```

注册形式

组件注册

vue 组件注册主要分为全局注册与局部注册

全局注册通过 `Vue.component` 方法，第一个参数为组件的名称，第二个参数为传入的配置项

```
Vue.component('my-component-name',{ /* ... */})
```

局部注册只需在用到的地方通过 `components` 属性注册一个组件

```
const component1 = {...} // 定义一个组件

export default {
  components:{
    component1 // 局部注册
  }
}
```

插件注册

插件的注册通过 `Vue.use()` 的方式进行注册（安装），第一个参数为插件的名字，第二个参数是可选择的配置项

```
Vue.use(插件名字,{ /* ... */})
```

注意的是：

注册插件的时候，需要在调用 `new Vue()` 启动应用之前完成

`Vue.use` 会自动阻止多次注册相同插件，只会注册一次

使用场景

具体的其实在插件是什么章节已经表述了，这里在总结一下

组件 (`Component`) 是用来构成你的 `App` 的业务模块，它的目标是 `App.vue`

插件 (`Plugin`) 是用来增强你的技术栈的功能模块，它的目标是 `Vue` 本身

简单来说，插件就是指对 `Vue` 的功能的增强或补充

面试官：Vue实例挂载的过程

一、思考

我们都听过知其然知其所以然这句话

那么不知道大家是否思考过 `new Vue()` 这个过程中究竟做了些什么？

过程中是如何完成数据的绑定，又是如何将数据渲染到视图的等等

一、分析

首先找到 `vue` 的构造函数

源码位置：`src\core\instance\index.js`

```
function Vue (options) {
  if (process.env.NODE_ENV !== 'production' &&
    !(this instanceof Vue)
  ) {
    warn('Vue is a constructor and should be called with the `new` keyword')
  }
  this._init(options)
}
```

`options` 是用户传递过来的配置项，如 `data`、`methods` 等常用的方法

`vue` 构造函数调用 `_init` 方法，但我们发现本文件中并没有此方法，但仔细可以看到文件下方定义了很多初始化方法

```
initMixin(Vue); // 定义 _init
stateMixin(Vue); // 定义 $set $get $delete $watch 等
eventsMixin(Vue); // 定义事件 $on $once $off $emit
lifecycleMixin(Vue); // 定义 _update $forceUpdate $destroy
renderMixin(Vue); // 定义 _render 返回虚拟dom
```

首先可以看 `initMixin` 方法，发现该方法在 `Vue` 原型上定义了 `_init` 方法

源码位置：src\core\instance\init.js

```
Vue.prototype._init = function (options?: Object) {
  const vm: Component = this
  // a uid
  vm._uid = uid++
  let startTag, endTag
  /* istanbul ignore if */
  if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
    startTag = `vue-perf-start:${vm._uid}`
    endTag = `vue-perf-end:${vm._uid}`
    mark(startTag)
  }

  // a flag to avoid this being observed
  vm._isVue = true
  // merge options
  // 合并属性，判断初始化的是否是组件，这里合并主要是 mixins 或 extends 的方法
  if (options && options._isComponent) {
    // optimize internal component instantiation
    // since dynamic options merging is pretty slow, and none of the
    // internal component options needs special treatment.
    initInternalComponent(vm, options)
  } else { // 合并vue属性
    vm.$options = mergeOptions(
      resolveConstructorOptions(vm.constructor),
      options || {},
      vm
    )
  }
}
```



```

/* istanbul ignore else */
if (process.env.NODE_ENV !== 'production') {
  // 初始化proxy拦截器
  initProxy(vm)
} else {
  vm._renderProxy = vm
}
// expose real self
vm._self = vm
// 初始化组件生命周期标志位
initLifecycle(vm)
// 初始化组件事件侦听
initEvents(vm)
// 初始化渲染方法
initRender(vm)
callHook(vm, 'beforeCreate')
// 初始化依赖注入内容，在初始化data、props之前
initInjections(vm) // resolve injections before data/props
// 初始化props/data/method/watch/methods
initState(vm)
initProvide(vm) // resolve provide after data/props
callHook(vm, 'created')

/* istanbul ignore if */
if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
  vm._name = formatComponentName(vm, false)
  mark(endTag)
  measure(`vue ${vm._name} init`, startTag, endTag)
}
// 挂载元素
if (vm.$options.el) {
  vm.$mount(vm.$options.el)
}
}

```

仔细阅读上面的代码，我们得到以下结论：

- 在调用 `beforeCreate` 之前，数据初始化并未完成，像 `data`、`props` 这些属性无法访问到
- 到了 `created` 的时候，数据已经初始化完成，能够访问 `data`、`props` 这些属性，但这时候并未完成 `dom` 的挂载，因此无法访问到 `dom` 元素
- 挂载方法是调用 `vm.$mount` 方法

`initState` 方法是完成 `props/data/method/watch/methods` 的初始化

源码位置：`src\core\instance\state.js`

```
export function initState (vm: Component) {  
  // 初始化组件的watcher列表  
  vm._watchers = []  
  const opts = vm.$options  
  // 初始化props  
  if (opts.props) initProps(vm, opts.props)  
  // 初始化methods方法  
  if (opts.methods) initMethods(vm, opts.methods)  
  if (opts.data) {  
    // 初始化data  
    initData(vm)  
  } else {  
    observe(vm._data = {}, true /* asRootData */)  
  }  
  if (opts.computed) initComputed(vm, opts.computed)  
  if (opts.watch && opts.watch !== nativeWatch) {  
    initWatch(vm, opts.watch)  
  }  
}
```

我们和这里主要看初始化 `data` 的方法为 `initData`，它与 `initState` 在同一文件上

```

function initData (vm: Component) {
  let data = vm.$options.data
  // 获取到组件上的data
  data = vm._data = typeof data === 'function'
    ? getData(data, vm)
    : data || {}
  if (!isPlainObject(data)) {
    data = {}
    process.env.NODE_ENV !== 'production' && warn(
      'data functions should return an object:\n' +
      'https://vuejs.org/v2/guide/components.html#data-Must-Be-a-Function',
      vm
    )
  }
  // proxy data on instance
  const keys = Object.keys(data)
  const props = vm.$options.props
  const methods = vm.$options.methods
  let i = keys.length
  while (i--) {
    const key = keys[i]
    if (process.env.NODE_ENV !== 'production') {
      // 属性名不能与方法名重复
      if (methods && hasOwn(methods, key)) {
        warn(
          `Method "${key}" has already been defined as a data property.`,
          vm
        )
      }
    }
    // 属性名不能与state名称重复
    if (props && hasOwn(props, key)) {
      process.env.NODE_ENV !== 'production' && warn(
        `The data property "${key}" is already declared as a prop. ` +
        `Use prop default value instead.`,
        vm
      )
    } else if (isReserved(key)) { // 验证key值的合法性
      // 将_data中的数据挂载到组件vm上,这样就可以通过this.xxx访问到组件上的数据
      proxy(vm, `_${key}`, data[key])
    }
  }
  // observe data
  // 响应式监听data是数据的变化
  observe(data, true /* asRootData */)
}

```

仔细阅读上面的代码，我们可以得到以下结论：

- 初始化顺序： `props`、`methods`、`data`
- `data` 定义的时候可选择函数形式或者对象形式（组件只能为函数形式）

关于数据响应式在这就不展开详细说明

上文提到挂载方法是调用 `vm.$mount` 方法

源码位置：

```
Vue.prototype.$mount = function (
  el?: string | Element,
  hydrating?: boolean
): Component {
  // 获取或查询元素
  el = el && query(el)

  /* istanbul ignore if */
  // vue 不允许直接挂载到body或页面文档上
  if (el === document.body || el === document.documentElement) {
    process.env.NODE_ENV !== 'production' && warn(
      'Do not mount Vue to <html> or <body> - mount to normal elements instead.'
    )
    return this
  }

  const options = this.$options
  // resolve template/el and convert to render function
  if (!options.render) {
    let template = options.template
    // 存在template模板, 解析vue模板文件
    if (template) {
      if (typeof template === 'string') {
        if (template.charAt(0) === '#') {
          template = idToTemplate(template)
          /* istanbul ignore if */
          if (process.env.NODE_ENV !== 'production' && !template) {
            warn(
              'Template element not found or is empty: ${options.template}',
              this
            )
          }
        }
      } else if (template.nodeType) {
        template = template.innerHTML
      } else {
        if (process.env.NODE_ENV !== 'production') {
          warn('invalid template option:' + template, this)
        }
        return this
      }
    } else if (el) {
      // 通过选择器获取元素内容
      template = getOuterHTML(el)
    }
    if (template) {
      /* istanbul ignore if */
      if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
        mark('compile')
      }
    }
  }
}
```

```

/**
 * 1. 将temmplate解析ast tree
 * 2. 将ast tree转换成render语法字符串
 * 3. 生成render方法
 */
const { render, staticRenderFns } = compileToFunctions(template, {
  outputSourceRange: process.env.NODE_ENV !== 'production',
  shouldDecodeNewlines,
  shouldDecodeNewlinesForHref,
  delimiters: options.delimiters,
  comments: options.comments
}, this)
options.render = render
options.staticRenderFns = staticRenderFns

/* istanbul ignore if */
if (process.env.NODE_ENV !== 'production' && config.performance && mark) {
  mark('compile end')
  measure(`vue ${this._name} compile`, 'compile', 'compile end')
}
}
}
return mount.call(this, el, hydrating)
}

```

阅读上面代码，我们能得到以下结论：

- 不要将根元素放到 `body` 或者 `html` 上
- 可以在对象中定义 `template/render` 或者直接使用 `template`、`el` 表示元素选择器
- 最终都会解析成 `render` 函数，调用 `compileToFunctions`，会将 `template` 解析成 `render` 函数

对 `template` 的解析步骤大致分为以下几步：

- 将 `html` 文档片段解析成 `ast` 描述符
- 将 `ast` 描述符解析成字符串
- 生成 `render` 函数

生成 `render` 函数，挂载到 `vm` 上后，会再次调用 `mount` 方法

源码位置：src\platforms\web\runtime\index.js

```

// public mount method
Vue.prototype.$mount = function (
  el?: string | Element,
  hydrating?: boolean
): Component {
  el = el && inBrowser ? query(el) : undefined
  // 渲染组件
  return mountComponent(this, el, hydrating)
}

```

调用 `mountComponent` 渲染组件

```
export function mountComponent (  
  vm: Component,  
  el: ?Element,  
  hydrating?: boolean  
) : Component {  
  vm.$el = el  
  // 如果没有获取解析的render函数, 则会抛出警告  
  // render是解析模板文件生成的  
  if (!vm.$options.render) {  
    vm.$options.render = createEmptyVNode  
    if (process.env.NODE_ENV !== 'production') {  
      /* istanbul ignore if */  
      if ((vm.$options.template && vm.$options.template.charAt(0) !== '#') ||  
        vm.$options.el || el) {  
        warn(  
          'You are using the runtime-only build of Vue where the template ' +  
          'compiler is not available. Either pre-compile the templates into ' +  
          'render functions, or use the compiler-included build.',  
          vm  
        )  
      } else {  
        // 没有获取到vue的模板文件  
        warn(  
          'Failed to mount component: template or render function not defined.',  
          vm  
        )  
      }  
    }  
  }  
  // 执行beforeMount钩子  
  callHook(vm, 'beforeMount')  
  
  let updateComponent  
  /* istanbul ignore if */  
  if (process.env.NODE_ENV !== 'production' && config.performance && mark) {  
    updateComponent = () => {  
      const name = vm._name  
      const id = vm._uid  
      const startTag = `vue-perf-start:${id}`  
      const endTag = `vue-perf-end:${id}`  
  
      mark(startTag)  
      const vnode = vm._render()  
      mark(endTag)  
      measure(`vue ${name} render`, startTag, endTag)  
  
      mark(startTag)  
      vm._update(vnode, hydrating)  
      mark(endTag)  
      measure(`vue ${name} patch`, startTag, endTag)  
    }  
  }
```



```

} else {
  // 定义更新函数
  updateComponent = () => {
    // 实际调用是在lifeCycleMixin中定义的_update和renderMixin中定义的_render
    vm._update(vm._render(), hydrating)
  }
}
// we set this to vm._watcher inside the watcher's constructor
// since the watcher's initial patch may call $forceUpdate (e.g. inside child
// component's mounted hook), which relies on vm._watcher being already defined
// 监听当前组件状态，当有数据变化时，更新组件
new Watcher(vm, updateComponent, noop, {
  before () {
    if (vm._isMounted && !vm._isDestroyed) {
      // 数据更新引发的组件更新
      callHook(vm, 'beforeUpdate')
    }
  }
}, true /* isRenderWatcher */)
hydrating = false

// manually mounted instance, call mounted on self
// mounted is called for render-created child components in its inserted hook
if (vm.$vnode == null) {
  vm._isMounted = true
  callHook(vm, 'mounted')
}
return vm
}

```

阅读上面代码，我们得到以下结论：

- 会触发 `beforeCreate` 钩子
- 定义 `updateComponent` 渲染页面视图的方法
- 监听组件数据，一旦发生变化，触发 `beforeUpdate` 生命钩子

`updateComponent` 方法主要执行在 `vue` 初始化时声明的 `render`，`update` 方法

`render` 的作用主要是生成 `vnode`

源码位置：`src\core\instance\render.js`

```

// 定义vue 原型上的render方法
Vue.prototype._render = function (): VNode {
  const vm: Component = this
  // render函数来自于组件的option
  const { render, _parentVnode } = vm.$options

  if (_parentVnode) {
    vm.$scopedSlots = normalizeScopedSlots(
      _parentVnode.data.scopedSlots,
      vm.$slots,
      vm.$scopedSlots
    )
  }
}

```

```

}

// set parent vnode. this allows render functions to have access
// to the data on the placeholder node.
vm.$vnode = _parentVnode
// render self
let vnode
try {
  // There's no need to maintain a stack because all render fns are called
  // separately from one another. Nested component's render fns are called
  // when parent component is patched.
  currentRenderingInstance = vm
  // 调用render方法, 自己独特的render方法, 传入createElement参数, 生成vNode
  vnode = render.call(vm._renderProxy, vm.$createElement)
} catch (e) {
  handleError(e, vm, `render`)
  // return error render result,
  // or previous vnode to prevent render error causing blank component
  /* istanbul ignore else */
  if (process.env.NODE_ENV !== 'production' && vm.$options.renderError) {
    try {
      vnode = vm.$options.renderError.call(vm._renderProxy, vm.$createElement, e)
    } catch (e) {
      handleError(e, vm, `renderError`)
      vnode = vm._vnode
    }
  } else {
    vnode = vm._vnode
  }
} finally {
  currentRenderingInstance = null
}
// if the returned array contains only a single node, allow it
if (Array.isArray(vnode) && vnode.length === 1) {
  vnode = vnode[0]
}
// return empty vnode in case the render function errored out
if (!(vnode instanceof VNode)) {
  if (process.env.NODE_ENV !== 'production' && Array.isArray(vnode)) {
    warn(
      'Multiple root nodes returned from render function. Render function ' +
      'should return a single root node.',
      vm
    )
  }
  vnode = createEmptyVNode()
}
// set parent
vnode.parent = _parentVnode
return vnode
}

```

`_update` 主要功能是调用 `patch`，将 `vnode` 转换为真实 `DOM`，并且更新到页面中

源码位置：src\core\instance\lifecycle.js

```
Vue.prototype._update = function (vnode: VNode, hydrating?: boolean) {
  const vm: Component = this
  const prevEl = vm.$el
  const prevVnode = vm._vnode
  // 设置当前激活的作用域
  const restoreActiveInstance = setActiveInstance(vm)
  vm._vnode = vnode
  // Vue.prototype.__patch__ is injected in entry points
  // based on the rendering backend used.
  if (!prevVnode) {
    // initial render
    // 执行具体的挂载逻辑
    vm.$el = vm.__patch__(vm.$el, vnode, hydrating, false /* removeOnly */)
  } else {
    // updates
    vm.$el = vm.__patch__(prevVnode, vnode)
  }
  restoreActiveInstance()
  // update __vue__ reference
  if (prevEl) {
    prevEl.__vue__ = null
  }
  if (vm.$el) {
    vm.$el.__vue__ = vm
  }
  // if parent is an HOC, update its $el as well
  if (vm.$vnode && vm.$parent && vm.$vnode === vm.$parent._vnode) {
    vm.$parent.$el = vm.$el
  }
  // updated hook is called by the scheduler to ensure that children are
  // updated in a parent's updated hook.
}
```

三、结论

- new Vue的时候调用会调用 _init 方法
 - 定义 \$set、\$get、\$delete、\$watch 等方法
 - 定义 \$on、\$off、\$emit、\$off 等事件
 - 定义 _update、\$forceUpdate、\$destroy 生命周期
- 调用 \$mount 进行页面的挂载
- 挂载的时候主要是通过 mountComponent 方法
- 定义 updateComponent 更新函数
- 执行 render 生成虚拟 DOM
- _update 将虚拟 DOM 生成真实 DOM 结构，并且渲染到页面中

参考文献

- <https://www.cnblogs.com/gerry2019/p/12001661.html>
- <https://github.com/vuejs/vue/tree/dev/src/core/instance>

面试官：Vue常用的修饰符有哪些有什么应用场景

一、修饰符是什么

在程序世界里，修饰符是用于限定类型以及类型成员的声明的一种符号

在 **Vue** 中，修饰符处理了许多 **DOM** 事件的细节，让我们不再需要花大量的时间去处理这些烦恼的事情，而能有更多的精力专注于程序的逻辑处理

vue 中修饰符分为以下五种：

- 表单修饰符
- 事件修饰符
- 鼠标按键修饰符
- 键值修饰符
- v-bind修饰符

二、修饰符的作用

表单修饰符

在我们填写表单的时候用得最多的是 **input** 标签，指令用得最多的是 **v-model**

关于表单的修饰符有如下：

- lazy
- trim
- number

lazy

在我们填完信息，光标离开标签的时候，才会将值赋予给 **value**，也就是在 **change** 事件之后再进行信息同步

```
<input type="text" v-model.lazy="value">
<p>{{value}}</p>
```

trim

自动过滤用户输入的首空格字符，而中间的空格不会过滤

```
<input type="text" v-model.trim="value">
```

number

自动将用户的输入值转为数值类型，但如果这个值无法被 **parseFloat** 解析，则会返回原来的值

```
<input v-model.number="age" type="number">
```

事件修饰符

事件修饰符是对事件捕获以及目标进行了处理，有如下修饰符：

- stop
- prevent
- self
- once
- capture
- passive
- native

stop

阻止了事件冒泡，相当于调用了 `event.stopPropagation` 方法

```
<div @click="shout(2)">  
  <button @click.stop="shout(1)">ok</button>  
</div>  
//只输出1
```

prevent

阻止了事件的默认行为，相当于调用了 `event.preventDefault` 方法

```
<form v-on:submit.prevent="onSubmit"></form>
```

self

只当在 `event.target` 是当前元素自身时触发处理函数

```
<div v-on:click.self="doThat">...</div>
```

“

使用修饰符时，顺序很重要；相应的代码会以同样的顺序产生。因此，用 `v-on:click.prevent.self` 会阻止「所有的点击」，而 `v-on:click.self.prevent` 只会阻止对元素自身的点击

”

once

绑定了事件以后只能触发一次，第二次就不会触发

```
<button @click.once="shout(1)">ok</button>
```

capture

使事件触发从包含这个元素的顶层开始往下触发

```
<div @click.capture="shout(1)">
  obj1
<div @click.capture="shout(2)">
  obj2
<div @click="shout(3)">
  obj3
<div @click="shout(4)">
  obj4
</div>
</div>
</div>
</div>
// 输出结构: 1 2 4 3
```

passive

在移动端，当我们在监听元素滚动事件的时候，会一直触发 `onscroll` 事件会让我们的网页变卡，因此我们使用这个修饰符的时候，相当于给 `onscroll` 事件整了一个 `.lazy` 修饰符

```
<!-- 滚动事件的默认行为 (即滚动行为) 将会立即触发 -->
<!-- 而不会等待 `onScroll` 完成 -->
<!-- 这其中包含 `event.preventDefault()` 的情况 -->
<div v-on:scroll.passive="onScroll">...</div>
```

“

不要把 `.passive` 和 `.prevent` 一起使用, 因为 `.prevent` 将会被忽略, 同时浏览器可能会向你展示一个警告。

`passive` 会告诉浏览器你不想阻止事件的默认行为

”

native

让组件变成像 `html` 内置标签那样监听根元素的原生事件，否则组件上使用 `v-on` 只会监听自定义事件

```
<my-component v-on:click.native="doSomething"></my-component>
```

“

使用 `.native` 修饰符来操作普通 HTML 标签是会令事件失效的

”

鼠标按钮修饰符

鼠标按钮修饰符针对的就是左键、右键、中键点击，有如下：

- left 左键点击
- right 右键点击

- middle 中键点击

```
<button @click.left="shout(1)">ok</button>
<button @click.right="shout(1)">ok</button>
<button @click.middle="shout(1)">ok</button>
```

键盘修饰符

键盘修饰符是用来修饰键盘事件（`onkeyup`，`onkeydown`）的，有如下：

`keyCode` 存在很多，但 `vue` 为我们提供了别名，分为以下两种：

- 普通键（enter、tab、delete、space、esc、up...）
- 系统修饰键（ctrl、alt、meta、shift...）

```
// 只有按键为keyCode的时候才触发
<input type="text" @keyup.keyCode="shout()">
```

还可以通过以下方式自定义一些全局的键盘码别名

```
Vue.config.keyCodes.f2 = 113
```

v-bind修饰符

v-bind修饰符主要是为属性进行操作，用来分别有如下：

- async
- prop
- camel

async

能对 `props` 进行一个双向绑定

```
//父组件
<comp :myMessage.sync="bar"></comp>
//子组件
this.$emit('update:myMessage',params);
```

以上这种方法相当于以下的简写

```
//父亲组件
<comp :myMessage="bar" @update:myMessage="func"></comp>
func(e){
  this.bar = e;
}
//子组件js
func2(){
  this.$emit('update:myMessage',params);
}
```

使用 `async` 需要注意以下两点：

- 使用 `sync` 的时候，子组件传递的事件名格式必须为 `update:value`，其中 `value` 必须与子组件中 `props` 中声明的名称完全一致
- 注意带有 `.sync` 修饰符的 `v-bind` 不能和表达式一起使用
- 将 `v-bind.sync` 用在一个字面量的对象上，例如 `v-bind.sync="{ title: doc.title }"`，是无法正常工作的

props

设置自定义标签属性，避免暴露数据，防止污染HTML结构

```
<input id="uid" title="title1" value="1" :index.prop="index">
```

camel

将命名变为驼峰命名法，如将 `view-Box` 属性名转换为 `viewBox`

```
<svg :viewBox="viewBox"></svg>
```

三、应用场景

根据每一个修饰符的功能，我们可以得到以下修饰符的应用场景：

- `.stop`：阻止事件冒泡
- `.native`：绑定原生事件
- `.once`：事件只执行一次
- `.self`：将事件绑定在自身身上，相当于阻止事件冒泡
- `.prevent`：阻止默认事件
- `.capture`：用于事件捕获
- `.once`：只触发一次
- `.keyCode`：监听特定键盘按下
- `.right`：右键

参考文献

- <https://segmentfault.com/a/1190000016786254>

面试官：Vue组件之间的通信方式都有哪些？

一、组件间通信的概念

开始之前，我们把「组件间通信」这个词进行拆分

- 组件
- 通信

都知道组件是 `vue` 最强大的功能之一，`vue` 中每一个 `.vue` 我们都可以视之为一个组件通信指的是发送者通过某种媒体以某种格式来传递信息到受信者以达到某个目的。广义上，任何信息的交通都是通信「组件间通信」即指组件(`.vue`)通过某种方式来传递信息以达到某个目的举个栗子我们在使用 `UI` 框架中的

table 组件，可能会往 table 组件中传入某些数据，这个本质就形成了组件之间的通信

二、组件间通信解决了什么

在古代，人们通过驿站、飞鸽传书、烽火报警、符号、语言、眼神、触碰等方式进行信息传递，到了今天，随着科技水平的飞速发展，通信基本完全利用有线或无线电完成，相继出现了有线电话、固定电话、无线电话、手机、互联网甚至视频电话等各种通信方式从上面这段话，我们可以看到通信的本质是信息同步，共享回到 vue 中，每个组件之间的都有独自的作用域，组件间的数据是无法共享的但实际开发工作中我们常常需要让组件之间共享数据，这也是组件通信的目的要让它们互相之间能进行通讯，这样才能构成一个有机的完整系统

二、组件间通信的分类

组件间通信的分类可以分成以下

- 父子组件之间的通信
- 兄弟组件之间的通信
- 祖孙与后代组件之间的通信
- 非关系组件间之间的通信

关系图：

三、组件间通信的方案

整理 vue 中8种常规的通信方案

1. 通过 props 传递
2. 通过 \$emit 触发自定义事件
3. 使用 ref
4. EventBus
5. \$parent 或 \$root
6. attrs 与 listeners
7. Provide 与 Inject
8. Vuex

props传递数据

- 适用场景：父组件传递数据给子组件
- 子组件设置 props 属性，定义接收父组件传递过来的参数
- 父组件在使用子组件标签中通过字面量来传递值

Children.vue

```
props:{
  // 字符串形式
  name:String // 接收的类型参数
  // 对象形式
  age:{
    type:Number, // 接收的类型为数值
    default:18, // 默认值为18
    require:true // age属性必须传递
  }
}
```

Father.vue 组件

```
<Children name="jack" age=18 />
```

\$emit 触发自定义事件

- 适用场景：子组件传递数据给父组件
- 子组件通过 `$emit` 触发自定义事件，`$emit` 第二个参数为传递的数值
- 父组件绑定监听器获取到子组件传递过来的参数

Childen.vue

```
this.$emit('add', good)
```

Father.vue

```
<Children @add="cartAdd($event)" />
```

ref

- 父组件在使用子组件的时候设置 `ref`
- 父组件通过设置子组件 `ref` 来获取数据

父组件

```
<Children ref="foo" />
```

```
this.$refs.foo // 获取子组件实例，通过子组件实例我们就能拿到对应的数据
```

EventBus

- 使用场景：兄弟组件传值
- 创建一个中央事件总线 `EventBus`
- 兄弟组件通过 `$emit` 触发自定义事件，`$emit` 第二个参数为传递的数值
- 另一个兄弟组件通过 `$on` 监听自定义事件

Bus.js

// 创建一个中央时间总线类

```
class Bus {
  constructor() {
    this.callbacks = {}; // 存放事件的名字
  }
  $on(name, fn) {
    this.callbacks[name] = this.callbacks[name] || [];
    this.callbacks[name].push(fn);
  }
  $emit(name, args) {
    if (this.callbacks[name]) {
      this.callbacks[name].forEach((cb) => cb(args));
    }
  }
}
```

// main.js

Vue.prototype.\$bus = new Bus() // 将\$bus挂载到vue实例的原型上

// 另一种方式

Vue.prototype.\$bus = new Vue() // Vue已经实现了Bus的功能

Children1.vue

```
this.$bus.$emit('foo')
```

Children2.vue

```
this.$bus.$on('foo', this.handle)
```

\$parent 或 \$root

- 通过共同祖辈 `$parent` 或者 `$root` 搭建通信桥连

兄弟组件

```
this.$parent.on('add', this.add)
```

另一个兄弟组件

```
this.$parent.emit('add')
```

\$attrs 与 \$listeners

- 适用场景：祖先传递数据给子孙
- 设置批量向下传属性 `$attrs` 和 `$listeners`
- 包含了父级作用域中不作为 `prop` 被识别（且获取）的特性绑定（`class` 和 `style` 除外）。
- 可以通过 `v-bind="$attrs"` 传入内部组件

```
// child: 并未在props中声明foo
<p>{{ $attrs.foo }}</p>

// parent
<HelloWorld foo="foo"/>
```

```
// 给Grandson隔代传值, communication/index.vue
<Child2 msg="lalala" @some-event="onSomeEvent"></Child2>

// Child2做展开
<Grandson v-bind="$attrs" v-on="$listeners"></Grandson>

// Grandson使用
<div @click="$emit('some-event', 'msg from grandson')">
  {{msg}}
</div>
```

provide 与 inject

- 在祖先组件定义 `provide` 属性，返回传递的值
- 在后代组件通过 `inject` 接收组件传递过来的值

祖先组件

```
provide(){
  return {
    foo:'foo'
  }
}
```

后代组件

```
inject:['foo'] // 获取到祖先组件传递过来的值
```

vuex

- 适用场景：复杂关系的组件数据传递
- `Vuex` 作用相当于一个用来存储共享变量的容器
- `state` 用来存放共享变量的地方
- `getter`，可以增加一个 `getter` 派生状态，（相当于 `store` 中的计算属性），用来获得共享变量的值
- `mutations` 用来存放修改 `state` 的方法。
- `actions` 也是用来存放修改 `state` 的方法，不过 `action` 是在 `mutations` 的基础上进行。常用来做一些异步操作

小结

- 父子关系的组件数据传递选择 `props` 与 `$emit` 进行传递，也可选择 `ref`
- 兄弟关系的组件数据传递可选择 `$bus`，其次可以选择 `$parent` 进行传递
- 祖先与后代组件数据传递可选择 `attrs` 与 `listeners` 或者 `Provide` 与 `Inject`
- 复杂关系的组件数据传递可以通过 `vuex` 存放共享的变量

参考文献

- <https://juejin.cn/post/6844903990052782094#heading-0>
- <https://zh.wikipedia.org/wiki/%E9%80%9A%E4%BF%A1>

面试官：Vue项目中你是如何解决跨域的呢？

一、跨域是什么

跨域本质是浏览器基于「**同源策略**」的一种安全手段

同源策略（Sameoriginpolicy），是一种约定，它是浏览器最核心也最基本的安全功能

所谓同源（即指在同一个域）具有以下三个相同点

- 协议相同（protocol）
- 主机相同（host）
- 端口相同（port）

反之非同源请求，也就是协议、端口、主机其中一项不相同的时候，这时候就会产生跨域

“

一定要注意跨域是浏览器的限制，你用抓包工具抓取接口数据，是可以看到接口已经把数据返回回来了，只是浏览器的限制，你获取不到数据。用postman请求接口能够请求到数据。这些再次印证了跨域是浏览器的限制。

”

二、如何解决

解决跨域的方法有很多，下面列举了三种：

- JSONP
- CORS
- Proxy

而在 `vue` 项目中，我们主要针对 `CORS` 或 `Proxy` 这两种方案进行展开

CORS

CORS（Cross-Origin Resource Sharing，跨域资源共享）是一个系统，它由一系列传输的HTTP头组成，这些HTTP头决定浏览器是否阻止前端 JavaScript 代码获取跨域请求的响应

`CORS` 实现起来非常方便，只需要增加一些 `HTTP` 头，让服务器能声明允许的访问来源

只要后端实现了 `CORS`，就实现了跨域

以 `koa` 框架举例

添加中间件，直接设置 `Access-Control-Allow-Origin` 响应头

```
app.use(async (ctx, next)=> {
  ctx.set('Access-Control-Allow-Origin', '*');
  ctx.set('Access-Control-Allow-Headers', 'Content-Type, Content-Length, Authorization, Accept, X-Requested-With , yourHeaderFeild');
  ctx.set('Access-Control-Allow-Methods', 'PUT, POST, GET, DELETE, OPTIONS');
  if (ctx.method === 'OPTIONS') {
    ctx.body = 200;
  } else {
    await next();
  }
})
```

ps: `Access-Control-Allow-Origin` 设置为*其实意义不大，可以说是形同虚设，实际应用中，上线前我们会将 `Access-Control-Allow-Origin` 值设为我们目标 `host`

Proxy

代理（Proxy）也称网络代理，是一种特殊的网络服务，允许一个（一般为客户端）通过这个服务与另一个网络终端（一般为服务器）进行非直接的连接。一些网关、路由器等网络设备具备网络代理功能。一般认为代理服务有利于保障网络终端的隐私或安全，防止攻击

「方案一」

如果是通过 `vue-cli` 脚手架工具搭建项目，我们可以通过 `webpack` 为我们起一个本地服务器作为请求的代理对象

通过该服务器转发请求至目标服务器，得到结果再转发给前端，但是最终发布上线时如果web应用和接口服务器不在一起仍会跨域

在 `vue.config.js` 文件，新增以下代码

```
module.exports = {
  devServer: {
    host: '127.0.0.1',
    port: 8084,
    open: true, // vue项目启动时自动打开浏览器
    proxy: {
      '/api': { // '/api'是代理标识，用于告诉node，url前面是/api的就是使用代理的
        target: 'http://xxx.xxx.xx.xx:8080', //目标地址，一般是指后台服务器地址
        changeOrigin: true, //是否跨域
        pathRewrite: { // pathRewrite 的作用是把实际Request Url中的'/api'用''代替
          '^/api': ''
        }
      }
    }
  }
}
```

通过 `axios` 发送请求中，配置请求的根路径

```
axios.defaults.baseURL = '/api'
```

「方案二」

此外，还可通过服务端实现代理请求转发

以 `express` 框架为例

```
var express = require('express');
const proxy = require('http-proxy-middleware')
const app = express()
app.use(express.static(__dirname + '/'))
app.use('/api', proxy({ target: 'http://localhost:4000', changeOrigin: false
                        }));
module.exports = app
```

「方案三」

通过配置 `nginx` 实现代理

```
server {
    listen 80;
    # server_name www.josephxia.com;
    location / {
        root /var/www/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }
    location /api {
        proxy_pass http://127.0.0.1:3000;
        proxy_redirect off;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

面试官：Vue项目中有封装过axios吗？主要是封装哪方面的？

一、axios是什么

`axios` 是一个轻量的 `HTTP` 客户端

基于 `XMLHttpRequest` 服务来执行 `HTTP` 请求，支持丰富的配置，支持 `Promise`，支持浏览器端和 `Node.js` 端。自 `Vue 2.0`起，尤大宣布取消对 `vue-resource` 的官方推荐，转而推荐 `axios`。现在 `axios` 已经成为大部分 `Vue` 开发者的首选

特性

- 从浏览器中创建 `XMLHttpRequests`
- 从 `node.js` 创建 `http` 请求
- 支持 `Promise` API
- 拦截请求和响应

- 转换请求数据和响应数据
- 取消请求
- 自动转换 **JSON** 数据
- 客户端支持防御 **XSRF**

基本使用

安装

```
// 项目中安装
npm install axios --S
// cdn 引入
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
```

导入

```
import axios from 'axios'
```

发送请求

```
axios({
  url: 'xxx', // 设置请求的地址
  method: 'GET', // 设置请求方法
  params: { // get请求使用params进行参数凭借,如果是post请求用data
    type: '',
    page: 1
  }
}).then(res => {
  // res为后端返回的数据
  console.log(res);
})
```

并发请求 `axios.all([])`

```
function getUserAccount() {
  return axios.get('/user/12345');
}

function getUserPermissions() {
  return axios.get('/user/12345/permissions');
}

axios.all([getUserAccount(), getUserPermissions()])
  .then(axios.spread(function (res1, res2) {
    // res1第一个请求的返回的内容, res2第二个请求返回的内容
    // 两个请求都执行完成才会执行
  })));
```

二、为什么要封装

`axios` 的 API 很友好，你完全可以很轻松地在项目中直接使用。

不过随着项目规模增大，如果每发起一次 `HTTP` 请求，就要把这些比如设置超时时间、设置请求头、根据项目环境判断使用哪个请求地址、错误处理等等操作，都需要写一遍

这种重复劳动不仅浪费时间，而且让代码变得冗余不堪，难以维护。为了提高我们的代码质量，我们应该在项目中二次封装一下 `axios` 再使用

举个例子：

```
axios('http://localhost:3000/data', {
  // 配置代码
  method: 'GET',
  timeout: 1000,
  withCredentials: true,
  headers: {
    'Content-Type': 'application/json',
    Authorization: 'xxx',
  },
  transformRequest: [function (data, headers) {
    return data;
  }],
  // 其他请求配置...
})
.then((data) => {
  // todo: 真正业务逻辑代码
  console.log(data);
}, (err) => {
  // 错误处理代码
  if (err.response.status === 401) {
    // handle authorization error
  }
  if (err.response.status === 403) {
    // handle server forbidden error
  }
  // 其他错误处理.....
  console.log(err);
});
```

如果每个页面都发送类似的请求，都要写一堆的配置与错误处理，就显得过于繁琐了

这时候我们就需要对 `axios` 进行二次封装，让使用更为便利

三、如何封装

封装的同时，你需要和 后端协商好一些约定，请求头，状态码，请求超时时间.....

设置接口请求前缀：根据开发、测试、生产环境的不同，前缀需要加以区分

请求头：来实现一些具体的业务，必须携带一些参数才可以请求(例如：会员业务)

状态码：根据接口返回的不同 `status`，来执行不同的业务，这块需要和后端约定好

请求方法：根据 `get`、`post` 等方法进行一个再次封装，使用起来更为方便

请求拦截器：根据请求的请求头设定，来决定哪些请求可以访问

响应拦截器：这块就是根据 后端`返回来的状态码判定执行不同业务

设置接口请求前缀

利用 `node` 环境变量来作判断，用来区分开发、测试、生产环境

```
if (process.env.NODE_ENV === 'development') {
  axios.defaults.baseURL = 'http://dev.xxx.com'
} else if (process.env.NODE_ENV === 'production') {
  axios.defaults.baseURL = 'http://prod.xxx.com'
}
```

在本地调试的时候，还需要在 `vue.config.js` 文件中配置 `devServer` 实现代理转发，从而实现跨域

```
devServer: {
  proxy: {
    '/proxyApi': {
      target: 'http://dev.xxx.com',
      changeOrigin: true,
      pathRewrite: {
        '/proxyApi': ''
      }
    }
  }
}
```

设置请求头与超时时间

大部分情况下，请求头都是固定的，只有少部分情况下，会需要一些特殊的请求头，这里将普适性的请求头作为基础配置。当需要特殊请求头时，将特殊请求头作为参数传入，覆盖基础配置

```
const service = axios.create({
  ...
  timeout: 30000, // 请求 30s 超时
  headers: {
    get: {
      'Content-Type': 'application/x-www-form-urlencoded;charset=utf-8'
      // 在开发中，一般还需要单点登录或者其他功能的通用请求头，可以一并配置进来
    },
    post: {
      'Content-Type': 'application/json;charset=utf-8'
      // 在开发中，一般还需要单点登录或者其他功能的通用请求头，可以一并配置进来
    }
  },
})
```

封装请求方法

先引入封装好的方法，在要调用的接口重新封装成一个方法暴露出去


```

// get 请求
export function httpGet({
  url,
  params = {}
}) {
  return new Promise((resolve, reject) => {
    axios.get(url, {
      params
    }).then((res) => {
      resolve(res.data)
    }).catch(err => {
      reject(err)
    })
  })
}

// post
// post请求
export function httpPost({
  url,
  data = {},
  params = {}
}) {
  return new Promise((resolve, reject) => {
    axios({
      url,
      method: 'post',
      transformRequest: [function (data) {
        let ret = ''
        for (let it in data) {
          ret += encodeURIComponent(it) + '=' + encodeURIComponent(data[it]) + '&'
        }
        return ret
      }],
      // 发送的数据
      data,
      // url参数
      params
    }).then(res => {
      resolve(res.data)
    })
  })
}

```

把封装的方法放在一个 `api.js` 文件中

```

import { httpGet, httpPost } from './http'
export const getorglist = (params = {}) => httpGet({ url: 'apps/api/org/list', params })

```

页面中就能直接调用

```
// .vue
import { getorglist } from '@assets/js/api'

getorglist({ id: 200 }).then(res => {
  console.log(res)
})
```

这样可以把 `api` 统一管理起来，以后维护修改只需要在 `api.js` 文件操作即可

请求拦截器

请求拦截器可以在每个请求里加上 `token`，做了统一处理后维护起来也方便

```
// 请求拦截器
axios.interceptors.request.use(
  config => {
    // 每次发送请求之前判断是否存在token
    // 如果存在，则统一在http请求的header都加上token，这样后台根据token判断你的登录情况，此处token一般是用户完成登录后储存在localStorage里的
    token && (config.headers.Authorization = token)
    return config
  },
  error => {
    return Promise.error(error)
  })
```

响应拦截器

响应拦截器可以在接收到响应后先做一层操作，如根据状态码判断登录状态、授权

```
// 响应拦截器
axios.interceptors.response.use(response => {
  // 如果返回的状态码为200，说明接口请求成功，可以正常拿到数据
  // 否则的话抛出错误
  if (response.status === 200) {
    if (response.data.code === 511) {
      // 未授权调取授权接口
    } else if (response.data.code === 510) {
      // 未登录跳转登录页
    } else {
      return Promise.resolve(response)
    }
  } else {
    return Promise.reject(response)
  }
}, error => {
  // 我们可以在这里对异常状态作统一处理
  if (error.response.status) {
    // 处理请求失败的情况
    // 对不同返回码对相应处理
    return Promise.reject(error.response)
  }
})
```

小结

- 封装是编程中很有意义的手段，简单的 `axios` 封装，就可以让我们领略到它的魅力
- 封装 `axios` 没有一个绝对的标准，只要你的封装可以满足你的项目需求，并且用起来方便，那就是一个好的封装方案

参考文献

- <https://www.html.cn/qa/vue-js/20544.html>
- <https://juejin.cn/post/6844904033782611976>
- <https://juejin.cn/post/6844903801451708429>

面试官：v-if和v-for的优先级是什么？

一、作用

`v-if` 指令用于条件性地渲染一块内容。这块内容只会在指令的表达式返回 `true` 值的时候被渲染

`v-for` 指令基于一个数组来渲染一个列表。`v-for` 指令需要使用 `item in items` 形式的特殊语法，其中 `items` 是源数据数组或者对象，而 `item` 则是被迭代的数组元素的别名

在 `v-for` 的时候，建议设置 `key` 值，并且保证每个 `key` 值是独一无二的，这便于 `diff` 算法进行优化

两者在用法上

```
<Modal v-if="isShow" />

<li v-for="item in items" :key="item.id">
  {{ item.label }}
</li>
```

二、优先级

`v-if` 与 `v-for` 都是 `vue` 模板系统中的指令

在 `vue` 模板编译的时候，会将指令系统转化成可执行的 `render` 函数

示例

编写一个 `p` 标签，同时使用 `v-if` 与 `v-for`

```
<div id="app">
  <p v-if="isShow" v-for="item in items">
    {{ item.title }}
  </p>
</div>
```

创建 `vue` 实例，存放 `isShow` 与 `items` 数据

```
const app = new Vue({
  el: "#app",
  data() {
    return {
      items: [
        { title: "foo" },
        { title: "baz" }
      ]
    }
  },
  computed: {
    isShow() {
      return this.items && this.items.length > 0
    }
  }
})
```

模板指令的代码都会生成在 `render` 函数中，通过 `app.$options.render` 就能得到渲染函数

```
f anonymous() {
  with (this) { return
    _c('div', { attrs: { "id": "app" } },
    _l((items), function (item)
      { return (isShow) ? _c('p', [_v("\n" + _s(item.title) + "\n")) : _e() }, 0) }
    )
  }
```

`_l` 是 `vue` 的列表渲染函数，函数内部都会进行一次 `if` 判断

初步得到结论：`v-for` 优先级是比 `v-if` 高

再将 `v-for` 与 `v-if` 置于不同标签

```
<div id="app">
  <template v-if="isShow">
    <p v-for="item in items">{{item.title}}</p>
  </template>
</div>
```

再输出下 `render` 函数

```
f anonymous() {
  with(this){return
    _c('div',{attrs:{"id":"app"}},
    [(isShow)?[_v("\n"),
      _l((items),function(item){return _c('p',[_v(_s(item.title))])}):_e()],2)}
}
```

这时候我们可以看到，`v-for` 与 `v-if` 作用在不同标签时候，是先进行判断，再进行列表的渲染

我们再查看下 `vue` 源码

源码位置：`\vue-dev\src\compiler\codegen\index.js`

```
export function genElement (el: ASTElement, state: CodegenState): string {
  if (el.parent) {
    el.pre = el.pre || el.parent.pre
  }
  if (el.staticRoot && !el.staticProcessed) {
    return genStatic(el, state)
  } else if (el.once && !el.onceProcessed) {
    return genOnce(el, state)
  } else if (el.for && !el.forProcessed) {
    return genFor(el, state)
  } else if (el.if && !el.ifProcessed) {
    return genIf(el, state)
  } else if (el.tag === 'template' && !el.slotTarget && !state.pre) {
    return genChildren(el, state) || 'void 0'
  } else if (el.tag === 'slot') {
    return genSlot(el, state)
  } else {
    // component or element
    ...
  }
}
```

在进行 `if` 判断的时候，`v-for` 是比 `v-if` 先进行判断

最终结论：`v-for` 优先级比 `v-if` 高

三、注意事项

1. 永远不要把 `v-if` 和 `v-for` 同时用在同一个元素上，带来性能方面的浪费（每次渲染都会先循环再进行条件判断）
2. 如果避免出现这种情况，则在外层嵌套 `template`（页面渲染不生成 `dom` 节点），在这一层进行 `v-if` 判断，然后在内部进行 `v-for` 循环

```
<template v-if="isShow">
  <p v-for="item in items">
</template>
```

3. 如果条件出现在循环内部，可通过计算属性 `computed` 提前过滤掉那些不需要显示的项

```
computed: {
  items: function() {
    return this.list.filter(function (item) {
      return item.isShow
    })
  }
}
```

面试官：v-show和v-if有什么区别？使用场景分别是什么？

一、v-show与v-if的共同点

我们都知道在 `vue` 中 `v-show` 与 `v-if` 的作用效果是相同的(不含 `v-else`)，都能控制元素在页面是否显示

在用法上也是相同的

```
<Model v-show="isShow" />
<Model v-if="isShow" />
```

- 当表达式为 `true` 的时候，都会占据页面的位置
- 当表达式都为 `false` 时，都不会占据页面位置

二、v-show与v-if的区别

- 控制手段不同
- 编译过程不同
- 编译条件不同

控制手段：`v-show` 隐藏则是为该元素添加 `css--display:none`，`dom` 元素依旧还在。`v-if` 显示隐藏是将 `dom` 元素整个添加或删除

编译过程：`v-if` 切换有一个局部编译/卸载的过程，切换过程中合适地销毁和重建内部的事件监听和子组件；`v-show` 只是简单的基于 `css` 切换

编译条件：`v-if` 是真正的条件渲染，它会确保在切换过程中条件块内的事件监听器和子组件适当地被销毁和重建。只有渲染条件为假时，并不做操作，直到为真才渲染

- `v-show` 由 `false` 变为 `true` 的时候不会触发组件的生命周期
- `v-if` 由 `false` 变为 `true` 的时候，触发组件的 `beforeCreate`、`create`、`beforeMount`、`mounted` 钩子，由 `true` 变为 `false` 的时候触发组件的 `beforeDestroy`、`destroyed` 方法

性能消耗：`v-if` 有更高的切换消耗；`v-show` 有更高的初始渲染消耗；

三、v-show与v-if原理分析

具体解析流程这里不展开讲，大致流程如下

- 将模板 `template` 转为 `ast` 结构的 `JS` 对象
- 用 `ast` 得到的 `JS` 对象拼装 `render` 和 `staticRenderFns` 函数
- `render` 和 `staticRenderFns` 函数被调用后生成虚拟 `VNODE` 节点，该节点包含创建 `DOM` 节点所需信息
- `vm.patch` 函数通过虚拟 `DOM` 算法利用 `VNODE` 节点创建真实 `DOM` 节点

v-show原理

不管初始条件是什么，元素总是会被渲染

我们看一下在 `vue` 中是如何实现的

代码很好理解，有 `transition` 就执行 `transition`，没有就直接设置 `display` 属性

```
// https://github.com/vuejs/vue-next/blob/3cd30c5245da0733f9eb6f29d220f39c46518162/packages/runtime-dom/src/directives/vShow.ts
export const vShow: ObjectDirective<VShowElement> = {
  beforeMount(el, { value }, { transition }) {
    el._vod = el.style.display === 'none' ? '' : el.style.display
    if (transition && value) {
      transition.beforeEnter(el)
    } else {
      setDisplay(el, value)
    }
  },
  mounted(el, { value }, { transition }) {
    if (transition && value) {
      transition.enter(el)
    }
  },
  updated(el, { value, oldValue }, { transition }) {
    // ...
  },
  beforeUnmount(el, { value }) {
    setDisplay(el, value)
  }
}
```

v-if原理

`v-if` 在实现上比 `v-show` 要复杂的多，因为还有 `else` `else-if` 等条件需要处理，这里我们也只摘抄源码中处理 `v-if` 的一小部分

返回一个 `node` 节点，`render` 函数通过表达式的值来决定是否生成 `DOM`

```
// https://github.com/vuejs/vue-next/blob/cdc9f336fd/packages/compiler-core/src/transforms/vIf.ts
export const transformIf = createStructuralDirectiveTransform(
  /^(if|else|else-if)$/,
  (node, dir, context) => {
    return processIf(node, dir, context, (ifNode, branch, isRoot) => {
      // ...
      return () => {
        if (isRoot) {
          ifNode.codegenNode = createCodegenNodeForBranch(
            branch,
            key,
            context
          ) as IfConditionalExpression
        } else {
          // attach this branch's codegen node to the v-if root.
          const parentCondition = getParentCondition(ifNode.codegenNode!)
          parentCondition.alternate = createCodegenNodeForBranch(
            branch,
            key + ifNode.branches.length - 1,
            context
          )
        }
      }
    })
  }
)
```

四、v-show与v-if的使用场景

`v-if` 与 `v-show` 都能控制 `dom` 元素在页面的显示

`v-if` 相比 `v-show` 开销更大的（直接操作 `dom` 节点增加与删除）

如果需要非常频繁地切换，则使用 `v-show` 较好

如果在运行时条件很少改变，则使用 `v-if` 较好

参考文献

- <https://www.jianshu.com/p/7af8554d8f08>
- <https://juejin.cn/post/6897948855904501768>

面试官：vue3有了解过吗？能说说跟vue2的区别吗？

一、Vue3介绍

关于 `vue3` 的重构背景，尤大是这样说的：

「Vue 新版本的理念成型于 2018 年末，当时 Vue 2 的代码库已经有两岁半了。比起通用软件的生命周期来这好像也没那么久，但在这段时期，前端世界已经今昔非比了

在我们更新（和重写）Vue 的主要版本时，主要考虑两点因素：首先是新的 JavaScript 语言特性在主流浏览器中的受支持水平；其次是当前代码库中随时间推移而逐渐暴露出来的一些设计和架构问题」

简要就是：

- 利用新的语言特性(es6)
- 解决架构问题

哪些变化

从上图中，我们可以概览 Vue3 的新特性，如下：

- 速度更快
- 体积减少
- 更易维护
- 更接近原生
- 更易使用

速度更快

vue3 相比 vue2

- 重写了虚拟 Dom 实现
- 编译模板的优化
- 更高效的组件初始化
- `undate` 性能提高 1.3~2 倍
- `SSR` 速度提高了 2~3 倍

体积更小

通过 `webpack` 的 `tree-shaking` 功能，可以将无用模块“剪辑”，仅打包需要的能够 `tree-shaking`，有两大好处：

- 对开发人员，能够对 `vue` 实现更多其他的功能，而不必担忧整体体积过大
- 对使用者，打包出来的包体积变小了

`vue` 可以开发出更多其他的功能，而不必担忧 `vue` 打包出来的整体体积过多

更易维护

compositon Api

- 可与现有的 `Options API` 一起使用
- 灵活的逻辑组合与复用
- `Vue3` 模块可以和其他框架搭配使用

更好的Typescript支持

VUE3 是基于 typescript 编写的，可以享受到自动的类型定义提示

编译器重写

更接近原生

可以自定义渲染 API

更易使用

响应式 Api 暴露出来

轻松识别组件重新渲染原因

二、Vue3新增特性

Vue 3 中需要关注的一些新功能包括：

- fragments
- Teleport
- composition Api
- createRenderer

fragments

在 Vue3.x 中，组件现在支持有多个根节点

```
<!-- Layout.vue -->
<template>
  <header>...</header>
  <main v-bind="$attrs">...</main>
  <footer>...</footer>
</template>
```

Teleport

Teleport 是一种能够将我们的模板移动到 DOM 中 Vue app 之外的其他位置的技术，就有点像哆啦A梦的“任意门”

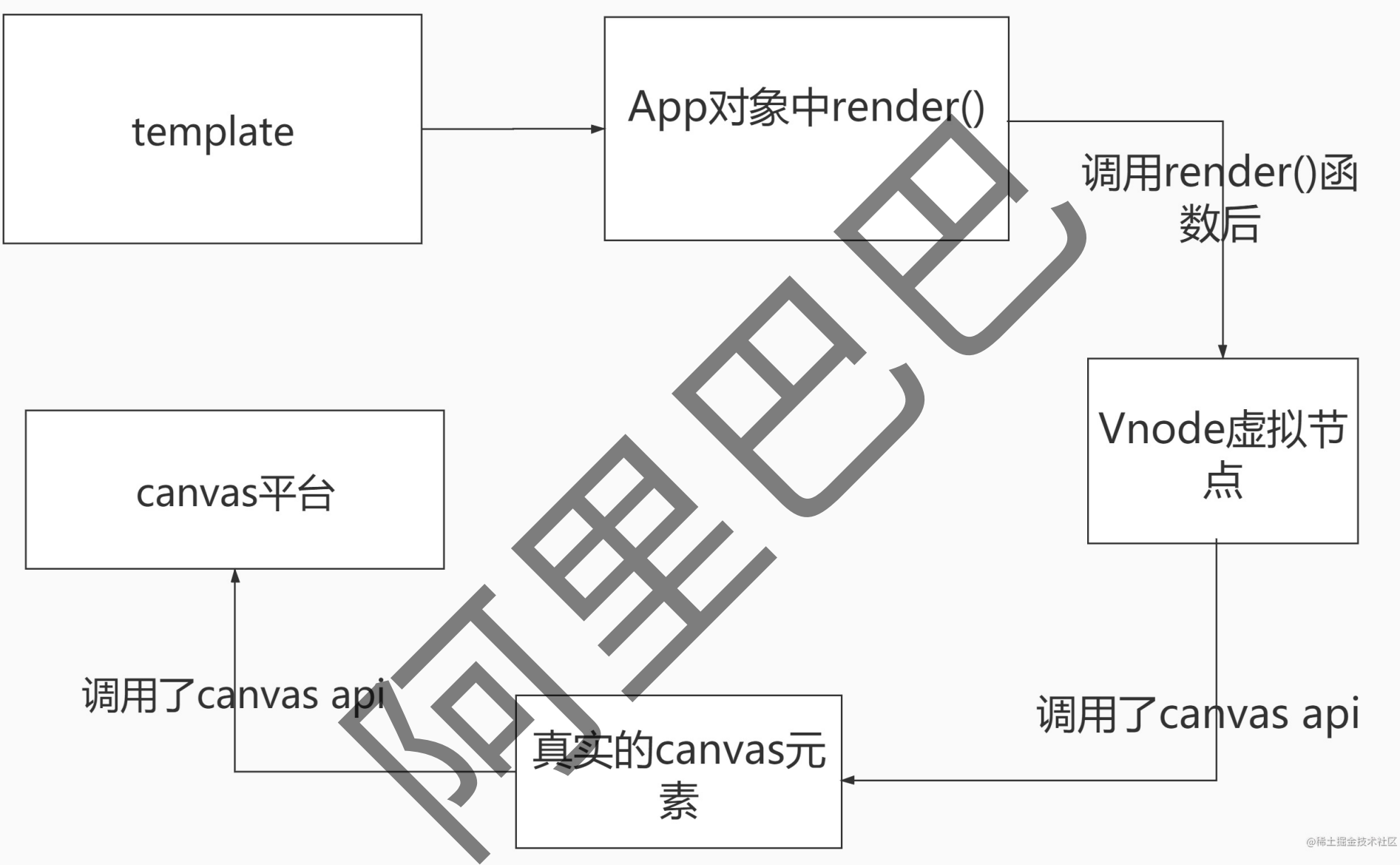
在 vue2 中，像 modals, toast 等这样的元素，如果我们嵌套在 Vue 的某个组件内部，那么处理嵌套组件的定位、z-index 和样式就会变得很困难

通过 Teleport，我们可以在组件的逻辑位置写模板代码，然后在 Vue 应用范围之外渲染它

```
<button @click="showToast" class="btn">打开 toast</button>
<!-- to 属性就是目标位置 -->
<teleport to="#teleport-target">
  <div v-if="visible" class="toast-wrap">
    <div class="toast-msg">我是一个 Toast 文案</div>
  </div>
</teleport>
```

createRenderer

通过 createRenderer，我们能够构建自定义渲染器，我们能够将 vue 的开发模型扩展到其他平台
我们可以将其生成在 canvas 画布上



关于 createRenderer，我们了解下基本使用，就不展开讲述了

```
import { createRenderer } from '@vue/runtime-core'

const { render, createApp } = createRenderer({
  patchProp,
  insert,
  remove,
  createElement,
  // ...
})

export { render, createApp }

export * from '@vue/runtime-core'
```

composition Api

composition Api，也就是组合式 [api](#)，通过这种形式，我们能够更加容易维护我们的代码，将相同功能的变量进行一个集中式的管理

关于 [compositon api](#) 的使用，这里以下图展开

简单使用：

```
export default {
  setup() {
    const count = ref(0)
    const double = computed(() => count.value * 2)
    function increment() {
      count.value++
    }
    onMounted(() => console.log('component mounted!'))
    return {
      count,
      double,
      increment
    }
  }
}
```

三、非兼容变更

Global API

- 全局 [Vue API](#) 已更改为使用应用程序实例
- 全局和内部 [API](#) 已经被重构为可 [tree-shakable](#)

模板指令

- 组件上 [v-model](#) 用法已更改

- `<template v-for>` 和 非 `v-for` 节点上 `key` 用法已更改
- 在同一元素上使用的 `v-if` 和 `v-for` 优先级已更改
- `v-bind="object"` 现在排序敏感
- `v-for` 中的 `ref` 不再注册 `ref` 数组

组件

- 只能使用普通函数创建功能组件
- `functional` 属性在单文件组件 (SFC)
- 异步组件现在需要 `defineAsyncComponent` 方法来创建

渲染函数

- 渲染函数 API 改变
- `$scopedSlots` property 已删除，所有插槽都通过 `$slots` 作为函数暴露
- 自定义指令 API 已更改为与组件生命周期一致
- 一些转换 `class` 被重命名了：
 - `v-enter` -> `v-enter-from`
 - `v-leave` -> `v-leave-from`
- 组件 `watch` 选项和实例方法 `$watch` 不再支持点分隔字符串路径，请改用计算函数作为参数
- 在 `Vue 2.x` 中，应用根容器的 `outerHTML` 将替换为根组件模板（如果根组件没有模板/渲染选项，则最终编译为模板）。`VUE3.x` 现在使用应用程序容器的 `innerHTML`。

其他小改变

- `destroyed` 生命周期选项被重命名为 `unmounted`
- `beforeDestroy` 生命周期选项被重命名为 `beforeUnmount`
- `[prop default]` 工厂函数不再有权访问 `this` 是上下文
- 自定义指令 API 已更改为与组件生命周期一致
- `data` 应始终声明为函数
- 来自 `mixin` 的 `data` 选项现在可简单地合并
- `attribute` 强制策略已更改
- 一些过渡 `class` 被重命名
- 组建 `watch` 选项和实例方法 `$watch` 不再支持以点分隔的字符串路径。请改用计算属性函数作为参数。
- `<template>` 没有特殊指令的标记 (`v-if/else-if/else`、`v-for` 或 `v-slot`) 现在被视为普通元素，并将生成原生的 `<template>` 元素，而不是渲染其内部内容。
- 在 `Vue 2.x` 中，应用根容器的 `outerHTML` 将替换为根组件模板（如果根组件没有模板/渲染选项，则最终编译为模板）。`Vue 3.x` 现在使用应用容器的 `innerHTML`，这意味着容器本身不再被视为模板的一部分。

移除 API

- `keyCode` 支持作为 `v-on` 的修饰符
- `$on`，`$off` 和 `$once` 实例方法
- 过滤 `filter`
- 内联模板 `attribute`
- `$destroy` 实例方法。用户不应再手动管理单个 `Vue` 组件的生命周期。

参考文献

- <https://composition-api.vuejs.org/zh/#api-%E4%BB%8B%E7%BB%8D>

面试官：vue要做权限管理该怎么做？如果控制到按钮级别的权限怎么做？

一、权限控制

权限是对特定资源的访问许可，所谓权限控制，也就是确保用户只能访问到被分配的资源而前端权限归根结底是请求的发起权，请求的发起可能有下面两种形式触发

- 页面加载触发
- 页面上的按钮点击触发

总的来说，所有的请求发起都触发自前端路由或视图

所以我们可以从这两方面入手，对触发权限的源头进行控制，最终要实现的目标是：

- 路由方面，用户登录后只能看到自己有权访问的导航菜单，也只能访问自己有权访问的路由地址，否则将跳转 `4xx` 提示页
- 视图方面，用户只能看到自己有权浏览的内容和有权操作的控件
- 最后再加上请求控制作为最后一道防线，路由可能配置失误，按钮可能忘了加权限，这种时候请求控制可以用来兜底，越权请求将在前端被拦截

二、如何做

前端权限控制可以分为四个方面：

- 接口权限
- 按钮权限
- 菜单权限
- 路由权限

接口权限

接口权限目前一般采用 `jwt` 的形式来验证，没有通过的话一般返回 `401`，跳转到登录页面重新进行登录登录完拿到 `token`，将 `token` 存起来，通过 `axios` 请求拦截器进行拦截，每次请求的时候头部携带 `token`

```
axios.interceptors.request.use(config => {
  config.headers['token'] = cookie.get('token')
  return config
})
axios.interceptors.response.use(res=>{},{response}=>{
  if (response.data.code === 40099 || response.data.code === 40098) { //token过期或者错误
    router.push('/login')
  }
})
```

路由权限控制

「方案一」

初始化即挂载全部路由，并且在路由上标记相应的权限信息，每次路由跳转前做校验

```
const routerMap = [
  {
    path: '/permission',
    component: Layout,
    redirect: '/permission/index',
    alwaysShow: true, // will always show the root menu
    meta: {
      title: 'permission',
      icon: 'lock',
      roles: ['admin', 'editor'] // you can set roles in root nav
    },
    children: [{
      path: 'page',
      component: () => import('@/views/permission/page'),
      name: 'pagePermission',
      meta: {
        title: 'pagePermission',
        roles: ['admin'] // or you can only set roles in sub nav
      }
    }, {
      path: 'directive',
      component: () => import('@/views/permission/directive'),
      name: 'directivePermission',
      meta: {
        title: 'directivePermission'
        // if do not set roles, means: this page does not require permission
      }
    }
  ]
}]
```

这种方式存在以下四种缺点：

- 加载所有的路由，如果路由很多，而用户并不是所有的路由都有权限访问，对性能会有影响。
- 全局路由守卫里，每次路由跳转都要做权限判断。
- 菜单信息写死在前端，要改个显示文字或权限信息，需要重新编译
- 菜单跟路由耦合在一起，定义路由的时候还有添加菜单显示标题，图标之类的信息，而且路由不一定作为菜单显示，还要多加字段进行标识

「方案二」

初始化的时候先挂载不需要权限控制的路由，比如登录页，404等错误页。如果用户通过URL进行强制访问，则会直接进入404，相当于从源头上做了控制

登录后，获取用户的权限信息，然后筛选有权限访问的路由，在全局路由守卫里进行调用 `addRoutes` 添加路由

```
import router from './router'
```

```
import store from './store'
import { Message } from 'element-ui'
import NProgress from 'nprogress' // progress bar
import 'nprogress/nprogress.css' // progress bar style
import { getToken } from '@/utils/auth' // getToken from cookie

NProgress.configure({ showSpinner: false }) // NProgress Configuration

// permission judge function
function hasPermission(roles, permissionRoles) {
  if (roles.indexOf('admin') >= 0) return true // admin permission passed directly
  if (!permissionRoles) return true
  return roles.some(role => permissionRoles.indexOf(role) >= 0)
}

const whiteList = ['/login', '/authredirect'] // no redirect whitelist

router.beforeEach((to, from, next) => {
  NProgress.start() // start progress bar
  if (getToken()) { // determine if there has token
    /* has token */
    if (to.path === '/login') {
      next({ path: '/' })
      NProgress.done() // if current page is dashboard will not trigger afterEach hook, so manually handle it
    } else {
      if (store.getters.roles.length === 0) { // 判断当前用户是否已拉取完user_info信息
        store.dispatch('GetUserInfo').then(res => { // 拉取user_info
          const roles = res.data.roles // note: roles must be a array! such as: ['editor', 'develop']
          store.dispatch('GenerateRoutes', { roles }).then(() => { // 根据roles权限生成可访问的路由表
            router.addRoutes(store.getters.addRoutes) // 动态添加可访问路由表
            next({ ...to, replace: true }) // hack方法 确保addRoutes已完成, set the replace: true so the navigation will not leave a history record
          })
        }).catch((err) => {
          store.dispatch('FedLogOut').then(() => {
            Message.error(err || 'Verification failed, please login again')
            next({ path: '/' })
          })
        })
      } else {
        // 没有动态改变权限的需求可直接next() 删除下方权限判断 ↓
        if (hasPermission(store.getters.roles, to.meta.roles)) {
          next() //
        } else {
          next({ path: '/401', replace: true, query: { noGoBack: true } })
        }
        // 可删 ↑
      }
    }
  } else {
    /* has no token */
    if (whiteList.indexOf(to.path) !== -1) { // 在免登录白名单, 直接进入
```

```

    next()
  } else {
    next('/login') // 否则全部重定向到登录页
    NProgress.done() // if current page is login will not trigger afterEach hook, so manually handle it
  }
}
})

router.afterEach(() => {
  NProgress.done() // finish progress bar
})

```

按需挂载，路由就需要知道用户的路由权限，也就是在用户登录进来的时候就要知道当前用户拥有哪些路由权限

这种方式也存在了以下的缺点：

- 全局路由守卫里，每次路由跳转都要做判断
- 菜单信息写死在前端，要改个显示文字或权限信息，需要重新编译
- 菜单跟路由耦合在一起，定义路由的时候还有添加菜单显示标题，图标之类的信息，而且路由不一定作为菜单显示，还要多加字段进行标识

菜单权限

菜单权限可以理解成将页面与理由进行解耦

方案一

菜单与路由分离，菜单由后端返回
前端定义路由信息

```

{
  name: "login",
  path: "/login",
  component: () => import("@/pages/Login.vue")
}

```

`name` 字段都不为空，需要根据此字段与后端返回菜单做关联，后端返回的菜单信息中必须要有 `name` 对应的字段，并且做唯一性校验

全局路由守卫里做判断

```

function hasPermission(router, accessMenu) {
  if (whiteList.indexOf(router.path) !== -1) {
    return true;
  }
  let menu = Util.getMenuByName(router.name, accessMenu);
  if (menu.name) {
    return true;
  }
  return false;
}

```



```

Router.beforeEach(async (to, from, next) => {
  if (getToken()) {
    let userInfo = store.state.user.userInfo;
    if (!userInfo.name) {
      try {
        await store.dispatch("GetUserInfo")
        await store.dispatch('updateAccessMenu')
        if (to.path === '/login') {
          next({ name: 'home_index' })
        } else {
          //Util.toDefaultPage([...routers], to.name, router, next);
          next({ ...to, replace: true })//菜单权限更新完成,重新进一次当前路由
        }
      }
    }
    catch (e) {
      if (whiteList.indexOf(to.path) !== -1) { // 在免登录白名单, 直接进入
        next()
      } else {
        next('/login')
      }
    }
  } else {
    if (to.path === '/login') {
      next({ name: 'home_index' })
    } else {
      if (hasPermission(to, store.getters.accessMenu)) {
        Util.toDefaultPage(store.getters.accessMenu,to, routes, next);
      } else {
        next({ path: '/403',replace:true })
      }
    }
  }
} else {
  if (whiteList.indexOf(to.path) !== -1) { // 在免登录白名单, 直接进入
    next()
  } else {
    next('/login')
  }
}
let menu = Util.getMenuByName(to.name, store.getters.accessMenu);
Util.title(menu.title);
});

Router.afterEach((to) => {
  window.scrollTo(0, 0);
});

```

每次路由跳转的时候都要判断权限，这里的判断也很简单，因为菜单的 `name` 与路由的 `name` 是一一对应的，而后端返回的菜单就已经是经过权限过滤的

如果根据路由 `name` 找不到对应的菜单，就表示用户有没有权限访问

如果路由很多，可以在应用初始化的时候，只挂载不需要权限控制的路由。取得后端返回的菜单后，根据菜单与路由的对应关系，筛选出可访问的路由，通过 `addRoutes` 动态挂载

这种方式的缺点：

- 菜单需要与路由做一一对应，前端添加了新功能，需要通过菜单管理功能添加新的菜单，如果菜单配置的不对会导致应用不能正常使用
- 全局路由守卫里，每次路由跳转都要做判断

方案二

菜单和路由都由后端返回

前端统一定义路由组件

```
const Home = () => import("../pages/Home.vue");
const UserInfo = () => import("../pages/UserInfo.vue");
export default {
  home: Home,
  userInfo: UserInfo
};
```

后端路由组件返回以下格式

```
[
  {
    name: "home",
    path: "/",
    component: "home"
  },
  {
    name: "home",
    path: "/userinfo",
    component: "userInfo"
  }
]
```

在将后端返回路由通过 `addRoutes` 动态挂载之间，需要将数据处理一下，将 `component` 字段换为真正的组件

如果有嵌套路由，后端功能设计的时候，要注意添加相应的字段，前端拿到数据也要做相应的处理

这种方法也会存在缺点：

- 全局路由守卫里，每次路由跳转都要做判断
- 前后端的配合要求更高

按钮权限

方案一

按钮权限也可以用 `v-if` 判断

但是如果页面过多，每个页面都要获取用户权限 `role` 和路由表里的 `meta.btnPermissions`，然后再做判断

这种方式就不展开举例了

方案二

通过自定义指令进行按钮权限的判断

首先配置路由

```
{
  path: '/permission',
  component: Layout,
  name: '权限测试',
  meta: {
    btnPermissions: ['admin', 'supper', 'normal']
  },
  //页面需要的权限
  children: [{
    path: 'supper',
    component: _import('system/supper'),
    name: '权限测试页',
    meta: {
      btnPermissions: ['admin', 'supper']
    } //页面需要的权限
  }],
  {
    path: 'normal',
    component: _import('system/normal'),
    name: '权限测试页',
    meta: {
      btnPermissions: ['admin']
    } //页面需要的权限
  }
}
```

自定义权限鉴定指令

```

import Vue from 'vue'
/**权限指令**/
const has = Vue.directive('has', {
  bind: function (el, binding, vnode) {
    // 获取页面按钮权限
    let btnPermissionsArr = [];
    if(binding.value){
      // 如果指令传值，获取指令参数，根据指令参数和当前登录人按钮权限做比较。
      btnPermissionsArr = Array.of(binding.value);
    }else{
      // 否则获取路由中的参数，根据路由的btnPermissionsArr和当前登录人按钮权限做比较。
      btnPermissionsArr = vnode.context.$route.meta.btnPermissions;
    }
    if (!Vue.prototype.$_has(btnPermissionsArr)) {
      el.parentNode.removeChild(el);
    }
  }
});
// 权限检查方法
Vue.prototype.$_has = function (value) {
  let isExist = false;
  // 获取用户按钮权限
  let btnPermissionsStr = sessionStorage.getItem("btnPermissions");
  if (btnPermissionsStr == undefined || btnPermissionsStr == null) {
    return false;
  }
  if (value.indexOf(btnPermissionsStr) > -1) {
    isExist = true;
  }
  return isExist;
};
export {has}

```

在使用的按钮中只需要引用 `v-has` 指令

```
<el-button @click='editClick' type='primary' v-has>编辑</el-button>
```

小结

关于权限如何选择哪种合适的方案，可以根据自己项目的方案项目，如考虑路由与菜单是否分离
权限需要前后端结合，前端尽可能的去控制，更多的需要后台判断

参考文献

- <https://segmentfault.com/a/1190000020887109>
- <https://juejin.cn/post/6844903648057622536#heading-6>

面试官：vue项目本地开发完成后部署到服务器后报404是什么原因呢？

一、如何部署

前后端分离开发模式下，前后端是独立部署的，前端只需要将最后的构建物上传至目标服务器的 web 容器指定的静态目录下即可

我们知道 vue 项目在构建后，是生成一系列的静态文件

常规部署我们只需要将这个目录上传至目标服务器即可

```
// scp 上传 user为主机登录用户，host为主机外网ip, xx为web容器静态资源路径
scp dist.zip user@host:/xx/xx/xx
```

让 web 容器跑起来，以 nginx 为例

```
server {
  listen 80;
  server_name www.xxx.com;

  location / {
    index /data/dist/index.html;
  }
}
```

配置完成记得重启 nginx

```
// 检查配置是否正确
nginx -t

// 平滑重启
nginx -s reload
```

操作完后就可以在浏览器输入域名进行访问了

当然上面只是提到最简单也是最直接的一种部署方式

什么自动化，镜像，容器，流水线部署，本质也是将这套逻辑抽象，隔离，用程序来代替重复性的劳动，本文不展开

二、404问题

这是一个经典的问题，相信很多同学都有遇到过，那么你知道其真正的原因吗？

我们先还原一下场景：

- vue 项目在本地时运行正常，但部署到服务器中，刷新页面，出现了404错误

先定位一下，HTTP 404 错误意味着链接指向的资源不存在

问题在于为什么不存在？且为什么只有 history 模式下会出现这个问题？

为什么history模式下有问题

Vue 是属于单页应用（single-page application）

而 SPA 是一种网络应用程序或网站的模型，所有用户交互是通过动态重写当前页面，前面我们也看到了，不管我们应用有多少页面，构建物都只会产生出一个 `index.html`

现在，我们回头来看一下我们的 `nginx` 配置

```
server {  
  listen 80;  
  server_name www.xxx.com;  
  
  location / {  
    index /data/dist/index.html;  
  }  
}
```

可以根据 `nginx` 配置得出，当我们在地址栏输入 `www.xxx.com` 时，这时会打开我们 `dist` 目录下的 `index.html` 文件，然后我们在跳转路由进入到 `www.xxx.com/login`

关键在这里，当我们在 `website.com/login` 页执行刷新操作，`nginx location` 是没有相关配置的，所以就会出现 404 的情况

为什么hash模式下没有问题

`router hash` 模式我们都知道是用符号 `#` 表示的，如 `website.com/#/login`，`hash` 的值为 `#/login`

它的特点在于：`hash` 虽然出现在 `URL` 中，但不会被包括在 `HTTP` 请求中，对服务端完全没有影响，因此改变 `hash` 不会重新加载页面

`hash` 模式下，仅 `hash` 符号之前的内容会被包含在请求中，如 `website.com/#/login` 只有 `website.com` 会被包含在请求中，因此对于服务端来说，即使没有配置 `location`，也不会返回 404 错误

解决方案

看到这里我相信大部分同学都能想到怎么解决问题了，

产生问题的本质是因为我们的路由是通过 JS 来执行视图切换的，

当我们进入到子路由时刷新页面，`web` 容器没有相对应的页面此时会出现 404

所以我们只需要配置将任意页面都重定向到 `index.html`，把路由交由前端处理

对 `nginx` 配置文件 `.conf` 修改，添加 `try_files $uri $uri/ /index.html;`

```
server {  
  listen 80;  
  server_name www.xxx.com;  
  
  location / {  
    index /data/dist/index.html;  
    try_files $uri $uri/ /index.html;  
  }  
}
```

修改完配置文件后记得配置的更新

```
nginx -s reload
```

这么做以后，你的服务器就不再返回 404 错误页面，因为对于所有路径都会返回 `index.html` 文件
为了避免这种情况，你应该在 `Vue` 应用里面覆盖所有的路由情况，然后在给出一个 404 页面

```
const router = new VueRouter({
  mode: 'history',
  routes: [
    { path: '', component: NotFoundComponent }
  ]
})
```

关于后端配置方案还有：`Apache`、`nodejs` 等，思想是一致的，这里就不展开述说了

参考文献

- <https://juejin.cn/post/6844903872637632525>
- <https://vue-js.com/topic/5f8cf91d96b2cb0032c385c0>

面试官：为什么data属性是一个函数而不是一个对象？

一、实例和组件定义data的区别

`vue` 实例的时候定义 `data` 属性既可以是一个对象，也可以是一个函数

```
const app = new Vue({
  el: "#app",
  // 对象格式
  data: {
    foo: "foo"
  },
  // 函数格式
  data() {
    return {
      foo: "foo"
    }
  }
})
```

组件中定义 `data` 属性，只能是一个函数

如果为组件 `data` 直接定义为一个对象


```
Vue.component('component1',{
  template: `<div>组件</div>`,
  data:{
    foo:"foo"
  }
})
```

则会得到警告信息

警告说明：返回的 `data` 应该是一个函数在每一个组件实例中

二、组件data定义函数与对象的区别

上面讲到组件 `data` 必须是一个函数，不知道大家有没有思考过这是为什么呢？

在我们定义好一个组件的时候，`vue` 最终都会通过 `Vue.extend()` 构成组件实例

这里我们模仿组件构造函数，定义 `data` 属性，采用对象的形式

```
function Component(){
}
Component.prototype.data = {
  count : 0
}
```

创建两个组件实例

```
const componentA = new Component()
const componentB = new Component()
```

修改 `componentA` 组件 `data` 属性的值，`componentB` 中的值也发生了改变

```
console.log(componentB.data.count) // 0
componentA.data.count = 1
console.log(componentB.data.count) // 1
```

产生这样的原因这是两者共用了同一个内存地址，`componentA` 修改的内容，同样对 `componentB` 产生了影响

如果我们采用函数的形式，则不会出现这种情况（函数返回的对象内存地址并不相同）

```
function Component(){
  this.data = this.data()
}
Component.prototype.data = function (){
  return {
    count : 0
  }
}
```

修改 `componentA` 组件 `data` 属性的值，`componentB` 中的值不受影响

```
console.log(componentB.data.count) // 0
componentA.data.count = 1
console.log(componentB.data.count) // 0
```

`vue` 组件可能会有很多个实例，采用函数返回一个全新 `data` 形式，使每个实例对象的数据不会受到其他实例对象数据的污染

三、原理分析

首先可以看看 `vue` 初始化 `data` 的代码，`data` 的定义可以是函数也可以是对象

源码位置：[/vue-dev/src/core/instance/state.js](#)

```
function initData (vm: Component) {
  let data = vm.$options.data
  data = vm._data = typeof data === 'function'
    ? getData(data, vm)
    : data || {}
  ...
}
```

`data` 既能是 `object` 也能是 `function`，那为什么还会出现上文警告呢？

别急，继续看下文

组件在创建的时候，会进行选项的合并

源码位置：[/vue-dev/src/core/util/options.js](#)

自定义组件会进入 `mergeOptions` 进行选项合并

```
Vue.prototype._init = function (options?: Object) {
  ...
  // merge options
  if (options && options._isComponent) {
    // optimize internal component instantiation
    // since dynamic options merging is pretty slow, and none of the
    // internal component options needs special treatment.
    initInternalComponent(vm, options)
  } else {
    vm.$options = mergeOptions(
      resolveConstructorOptions(vm.constructor),
      options || {},
      vm
    )
  }
  ...
}
```

定义 `data` 会进行数据校验

源码位置： </vue-dev/src/core/instance/init.js>

这时候 `vm` 实例为 `undefined`，进入 `if` 判断，若 `data` 类型不是 `function`，则出现警告提示

```
strats.data = function (
  parentVal: any,
  childVal: any,
  vm?: Component
): ?Function {
  if (!vm) {
    if (childVal && typeof childVal !== "function") {
      process.env.NODE_ENV !== "production" &&
        warn(
          'The "data" option should be a function ' +
            "that returns a per-instance value in component " +
            "definitions.",
          vm
        );

      return parentVal;
    }
    return mergeDataOrFn(parentVal, childVal);
  }
  return mergeDataOrFn(parentVal, childVal, vm);
};
```

四、结论

- 根实例对象 `data` 可以是对象也可以是函数（根实例是单例），不会产生数据污染情况
- 组件实例对象 `data` 必须为函数，目的是为了防止多个组件实例对象之间共用一个 `data`，产生数据污染。采用函数的形式，`initData` 时会将其作为工厂函数都会返回全新 `data` 对象

面试官：什么是虚拟DOM？如何实现一个虚拟DOM？说说你的思路

一、什么是虚拟DOM

虚拟 DOM（[Virtual DOM](#)）这个概念相信大家都不陌生，从 [React](#) 到 [Vue](#)，虚拟 DOM 为这两个框架都带来了跨平台的能力（[React-Native](#) 和 [Weex](#)）

实际上它只是一层对真实 DOM 的抽象，以 [JavaScript](#) 对象（[VNode](#) 节点）作为基础的树，用对象的属性来描述节点，最终可以通过一系列操作使这棵树映射到真实环境上

在 [Javascript](#) 对象中，虚拟 DOM 表现为一个 [Object](#) 对象。并且最少包含标签名（[tag](#)）、属性（[attrs](#)）和子元素对象（[children](#)）三个属性，不同框架对这三个属性的命名可能会有差别

创建虚拟 DOM 就是为了更好将虚拟的节点渲染到页面视图中，所以虚拟 DOM 对象的节点与真实 DOM 的属性一一照应

在 [vue](#) 中同样使用到了虚拟 DOM 技术

定义真实 DOM

```
<div id="app">
  <p class="p">节点内容</p>
  <h3>{{ foo }}</h3>
</div>
```

实例化 [vue](#)

```
const app = new Vue({
  el:"#app",
  data:{
    foo:"foo"
  }
})
```

观察 [render](#) 的 [render](#)，我们能得到虚拟 DOM

```
(function anonymous(
){
  with(this){return _c('div',{attrs:{"id":"app"}},[_c('p',{staticClass:"p"},
    [_v("节点内容")]),_v(" "),_c('h3',[_v(_s(foo))])])})})
```

通过 [VNode](#)，[vue](#) 可以对这颗抽象树进行创建节点、删除节点以及修改节点的操作，经过 [diff](#) 算法得出一些需要修改的最小单位，再更新视图，减少了 [dom](#) 操作，提高了性能

二、为什么需要虚拟DOM

DOM 是很慢的，其元素非常庞大，页面的性能问题，大部分都是由 DOM 操作引起的

真实的 DOM 节点，哪怕一个最简单的 [div](#) 也包含着很多属性，可以打印出来直观感受一下：

由此可见，操作 **DOM** 的代价仍旧是昂贵的，频繁操作还是会出现页面卡顿，影响用户的体验

「举个例子：」

你用传统的原生 **api** 或 **jQuery** 去操作 **DOM** 时，浏览器会从构建 **DOM** 树开始从头到尾执行一遍流程

当你在一次操作时，需要更新10个 **DOM** 节点，浏览器没这么智能，收到第一个更新 **DOM** 请求后，并不知道后续还有9次更新操作，因此会马上执行流程，最终执行10次流程

而通过 **VNode**，同样更新10个 **DOM** 节点，虚拟 **DOM** 不会立即操作 **DOM**，而是将这10次更新的 **diff** 内容保存到本地的一个 **js** 对象中，最终将这个 **js** 对象一次性 **attach** 到 **DOM** 树上，避免大量的无谓计算

“

很多人认为虚拟 DOM 最大的优势是 diff 算法，减少 JavaScript 操作真实 DOM 的带来的性能消耗。虽然这一个虚拟 DOM 带来的一个优势，但并不是全部。虚拟 DOM 最大的优势在于抽象了原本的渲染过程，实现了跨平台的能力，而不仅仅局限于浏览器的 DOM，可以是安卓和 IOS 的原生组件，可以是近期很火热的小程序，也可以是各种 GUI

”

三、如何实现虚拟DOM

首先可以看看 **vue** 中 **VNode** 的结构

源码位置：src/core/vdom/vnode.js

```
export default class VNode {
  tag: string | void;
  data: VNodeData | void;
  children: ?Array<VNode>;
  text: string | void;
  elm: Node | void;
  ns: string | void;
  context: Component | void; // rendered in this component's scope
  functionalContext: Component | void; // only for functional component root nodes
  key: string | number | void;
  componentOptions: VNodeComponentOptions | void;
  componentInstance: Component | void; // component instance
  parent: VNode | void; // component placeholder node
  raw: boolean; // contains raw HTML? (server only)
  isStatic: boolean; // hoisted static node
  isRootInsert: boolean; // necessary for enter transition check
  isComment: boolean; // empty comment placeholder?
  isCloned: boolean; // is a cloned node?
  isOnce: boolean; // is a v-once node?

  constructor (
    tag?: string,
    data?: VNodeData,
    children?: ?Array<VNode>,
    text?: string,
    elm?: Node,
    context?: Component,
```

```

componentOptions?: VNodeComponentOptions
){
  /*当前节点的标签名*/
  this.tag = tag
  /*当前节点对应的对象，包含了具体的一些数据信息，是一个VNodeData类型，可以参考
  VNodeData类型中的数据信息*/
  this.data = data
  /*当前节点的子节点，是一个数组*/
  this.children = children
  /*当前节点的文本*/
  this.text = text
  /*当前虚拟节点对应的真实dom节点*/
  this.elm = elm
  /*当前节点的名字空间*/
  this.ns = undefined
  /*编译作用域*/
  this.context = context
  /*函数化组件作用域*/
  this.functionalContext = undefined
  /*节点的key属性，被当作节点的标志，用以优化*/
  this.key = data && data.key
  /*组件的option选项*/
  this.componentOptions = componentOptions
  /*当前节点对应的组件的实例*/
  this.componentInstance = undefined
  /*当前节点的父节点*/
  this.parent = undefined
  /*简而言之就是是否为原生HTML或只是普通文本，innerHTML的时候为true，textContent的时候
  为false*/
  this.raw = false
  /*静态节点标志*/
  this.isStatic = false
  /*是否作为跟节点插入*/
  this.isRootInsert = true
  /*是否为注释节点*/
  this.isComment = false
  /*是否为克隆节点*/
  this.isCloned = false
  /*是否有v-once指令*/
  this.isOnce = false
}

// DEPRECATED: alias for componentInstance for backwards compat.
/* istanbul ignore next https://github.com/answershuto/learnVue*/
get child (): Component | void {
  return this.componentInstance
}
}

```

这里对 `VNode` 进行稍微的说明：

- 所有对象的 `context` 选项都指向了 `Vue` 实例
- `elm` 属性则指向了其相对应的真实 `DOM` 节点

vue 是通过 `createElement` 生成 `VNode`

源码位置: `src/core/vdom/create-element.js`

```
export function createElement (
  context: Component,
  tag: any,
  data: any,
  children: any,
  normalizationType: any,
  alwaysNormalize: boolean
): VNode | Array<VNode> {
  if (Array.isArray(data) || isPrimitive(data)) {
    normalizationType = children
    children = data
    data = undefined
  }
  if (isTrue(alwaysNormalize)) {
    normalizationType = ALWAYS_NORMALIZE
  }
  return _createElement(context, tag, data, children, normalizationType)
}
```

上面可以看到 `createElement` 方法实际上是对 `_createElement` 方法的封装，对参数的传入进行了判断

```

export function _createElement(
  context: Component,
  tag?: string | Class<Component> | Function | Object,
  data?: VNodeData,
  children?: any,
  normalizationType?: number
): VNode | Array<VNode> {
  if (isDef(data) && isDef((data: any).__ob__)) {
    process.env.NODE_ENV !== 'production' && warn(
      `Avoid using observed data object as vnode data: ${JSON.stringify(data)}\n` +
      'Always create fresh vnode data objects in each render!',
      context
    )
    return createEmptyVNode()
  }
  // object syntax in v-bind
  if (isDef(data) && isDef(data.is)) {
    tag = data.is
  }
  if (!tag) {
    // in case of component :is set to falsy value
    return createEmptyVNode()
  }
  ...
  // support single function children as default scoped slot
  if (Array.isArray(children) &&
    typeof children[0] === 'function'
  ){
    data = data || {}
    data.scopedSlots = { default: children[0] }
    children.length = 0
  }
  if (normalizationType === ALWAYS_NORMALIZE) {
    children = normalizeChildren(children)
  } else if ( === SIMPLE_NORMALIZE) {
    children = simpleNormalizeChildren(children)
  }
  // 创建VNode
  ...
}

```

可以看到 `_createElement` 接收 5 个参数：

- `context` 表示 `VNode` 的上下文环境，是 `Component` 类型
- `tag` 表示标签，它可以是一个字符串，也可以是一个 `Component`
- `data` 表示 `VNode` 的数据，它是一个 `VNodeData` 类型
- `children` 表示当前 `VNode` 的子节点，它是任意类型的
- `normalizationType` 表示子节点规范的类型，类型不同规范的方法也就不一样，主要是参考 `render` 函数是编译生成的还是用户手写的

根据 `normalizationType` 的类型，`children` 会有不同的定义

```
if (normalizationType === ALWAYS_NORMALIZE) {
  children = normalizeChildren(children)
} else if ( === SIMPLE_NORMALIZE) {
  children = simpleNormalizeChildren(children)
}
```

`simpleNormalizeChildren` 方法调用场景是 `render` 函数是编译生成的

`normalizeChildren` 方法调用场景分为下面两种：

- `render` 函数是用户手写的
- 编译 `slot`、`v-for` 的时候会产生嵌套数组

无论是 `simpleNormalizeChildren` 还是 `normalizeChildren` 都是对 `children` 进行规范（使 `children` 变成了一个类型为 `VNode` 的 `Array`），这里就不展开说了

规范化 `children` 的源码位置在：`src/core/vdom/helpers/normalize-children.js`

在规范化 `children` 后，就去创建 `VNode`

```
let vnode, ns
// 对tag进行判断
if (typeof tag === 'string') {
  let Ctor
  ns = (context.$vnode && context.$vnode.ns) || config.getTagNamespace(tag)
  if (config.isReservedTag(tag)) {
    // 如果是内置的节点，则直接创建一个普通VNode
    vnode = new VNode(
      config.parsePlatformTagName(tag), data, children,
      undefined, undefined, context
    )
  } else if (isDef(Ctor = resolveAsset(context.$options, 'components', tag))) {
    // component
    // 如果是component类型，则会通过createComponent创建VNode节点
    vnode = createComponent(Ctor, data, context, children, tag)
  } else {
    vnode = new VNode(
      tag, data, children,
      undefined, undefined, context
    )
  }
} else {
  // direct component options / constructor
  vnode = createComponent(tag, data, context, children)
}
```

`createComponent` 同样是创建 `VNode`

源码位置：`src/core/vdom/create-component.js`

```
export function createComponent (
```

```

Ctor: Class<Component> | Function | Object | void,
data: ?VNodeData,
context: Component,
children: ?Array<VNode>,
tag?: string
): VNode | Array<VNode> | void {
  if (isUndef(Ctor)) {
    return
  }
  // 构建子类构造函数
  const baseCtor = context.$options._base

  // plain options object: turn it into a constructor
  if (isObject(Ctor)) {
    Ctor = baseCtor.extend(Ctor)
  }

  // if at this stage it's not a constructor or an async component factory,
  // reject.
  if (typeof Ctor !== 'function') {
    if (process.env.NODE_ENV !== 'production') {
      warn(`Invalid Component definition: ${String(Ctor)}`, context)
    }
    return
  }

  // async component
  let asyncFactory
  if (isUndef(Ctor.cid)) {
    asyncFactory = Ctor
    Ctor = resolveAsyncComponent(asyncFactory, baseCtor, context)
    if (Ctor === undefined) {
      return createAsyncPlaceholder(
        asyncFactory,
        data,
        context,
        children,
        tag
      )
    }
  }

  data = data || {}

  // resolve constructor options in case global mixins are applied after
  // component constructor creation
  resolveConstructorOptions(Ctor)

  // transform component v-model data into props & events
  if (isDef(data.model)) {
    transformModel(Ctor.options, data)
  }

```

```

// extract props
const propsData = extractPropsFromVNodeData(data, Ctor, tag)

// functional component
if (isTrue(Ctor.options.functional)) {
  return createFunctionalComponent(Ctor, propsData, data, context, children)
}

// extract listeners, since these needs to be treated as
// child component listeners instead of DOM listeners
const listeners = data.on
// replace with listeners with .native modifier
// so it gets processed during parent component patch.
data.on = data.nativeOn

if (isTrue(Ctor.options.abstract)) {
  const slot = data.slot
  data = {}
  if (slot) {
    data.slot = slot
  }
}

// 安装组件钩子函数，把钩子函数合并到data.hook中
installComponentHooks(data)

//实例化一个VNode返回。组件的VNode是没有children的
const name = Ctor.options.name || tag
const vnode = new VNode(
  `vue-component-${Ctor.cid}${name ? `-${name}` : ''}`,
  data, undefined, undefined, undefined, context,
  { Ctor, propsData, listeners, tag, children },
  asyncFactory
)
if (__WEEX__ && isRecyclableComponent(vnode)) {
  return renderRecyclableComponentTemplate(vnode)
}

return vnode
}

```

稍微提下 `createElement` 生成 `VNode` 的三个关键流程：

- 构造子类构造函数 `Ctor`
- `installComponentHooks` 安装组件钩子函数
- 实例化 `vnode`

小结

`createElement` 创建 `VNode` 的过程，每个 `VNode` 有 `children`，`children` 每个元素也是一个 `VNode`，这样就形成了一个虚拟树结构，用于描述真实的 `DOM` 树结构

参考文献

- <https://ustbhuangyi.github.io/vue-analysis/v2/data-driven/create-element.html#children-%E7%9A%84%E8%A7%84%E8%8C%83%E5%8C%96>
- <https://juejin.cn/post/6876711874050818061>

面试官：你了解axios的原理吗？有看过它的源码吗？

一、axios的使用

关于 `axios` 的基本使用，上篇文章已经有所涉及，这里再稍微回顾下：

「发送请求」

```
import axios from 'axios';

axios(config) // 直接传入配置
axios(url[, config]) // 传入url和配置
axios[method](url[, option]) // 直接调用请求方式方法，传入url和配置
axios[method](url[, data[, option]]) // 直接调用请求方式方法，传入data、url和配置
axios.request(option) // 调用 request 方法

const axiosInstance = axios.create(config)
// axiosInstance 也具有以上 axios 的能力

axios.all([axiosInstance1, axiosInstance2]).then(axios.spread(response1, response2))
// 调用 all 和传入 spread 回调
```

「请求拦截器」

```
axios.interceptors.request.use(function (config) {
  // 这里写发送请求前处理的代码
  return config;
}, function (error) {
  // 这里写发送请求错误相关的代码
  return Promise.reject(error);
});
```

「响应拦截器」

```
axios.interceptors.response.use(function (response) {
  // 这里写得到响应数据后处理的代码
  return response;
}, function (error) {
  // 这里写得到错误响应处理的代码
  return Promise.reject(error);
});
```

「取消请求」


```
// 方式一
const CancelToken = axios.CancelToken;
const source = CancelToken.source();

axios.get('xxxx', {
  cancelToken: source.token
})
// 取消请求 (请求原因是可选的)
source.cancel('主动取消请求');

// 方式二
const CancelToken = axios.CancelToken;
let cancel;

axios.get('xxxx', {
  cancelToken: new CancelToken(function executor(c) {
    cancel = c;
  })
});
cancel('主动取消请求');
```

二、实现一个简易版axios

构建一个 `Axios` 构造函数，核心代码为 `request`

```
class Axios {
  constructor() {

  }

  request(config) {
    return new Promise(resolve => {
      const {url = '', method = 'get', data = {}} = config;
      // 发送ajax请求
      const xhr = new XMLHttpRequest();
      xhr.open(method, url, true);
      xhr.onload = function() {
        console.log(xhr.responseText)
        resolve(xhr.responseText);
      }
      xhr.send(data);
    })
  }
}
```

导出 `axios` 实例

// 最终导出axios的方法，即实例的request方法

```
function CreateAxiosFn() {  
  let axios = new Axios();  
  let req = axios.request.bind(axios);  
  return req;  
}
```

// 得到最后的全局变量axios

```
let axios = CreateAxiosFn();
```

上述就已经能够实现 `axios({ })` 这种方式的请求

下面是来实现下 `axios.method()` 这种形式的请求

// 定义get,post...方法，挂在到Axios原型上

```
const methodsArr = ['get', 'delete', 'head', 'options', 'put', 'patch', 'post'];  
methodsArr.forEach(met => {  
  Axios.prototype[met] = function() {  
    console.log('执行'+met+'方法');  
    // 处理单个方法  
    if (['get', 'delete', 'head', 'options'].includes(met)) { // 2个参数(url[, config])  
      return this.request({  
        method: met,  
        url: arguments[0],  
        ...arguments[1] || {}  
      })  
    } else { // 3个参数(url[, data[, config]])  
      return this.request({  
        method: met,  
        url: arguments[0],  
        data: arguments[1] || {},  
        ...arguments[2] || {}  
      })  
    }  
  }  
})
```

将 `Axios.prototype` 上的方法搬运到 `request` 上

首先实现个工具类，实现将 `b` 方法混入到 `a`，并且修改 `this` 指向

```
const utils = {
  extend(a,b, context) {
    for(let key in b) {
      if (b.hasOwnProperty(key)) {
        if (typeof b[key] === 'function') {
          a[key] = b[key].bind(context);
        } else {
          a[key] = b[key]
        }
      }
    }
  }
}
```

修改导出的方法

```
function CreateAxiosFn() {
  let axios = new Axios();

  let req = axios.request.bind(axios);
  // 增加代码
  utils.extend(req, Axios.prototype, axios)

  return req;
}
```

构建拦截器的构造函数

```
class InterceptorsManage {
  constructor() {
    this.handlers = [];
  }

  use(fullfield, rejected) {
    this.handlers.push({
      fullfield,
      rejected
    })
  }
}
```

实现 `axios.interceptors.response.use` 和 `axios.interceptors.request.use`

```

class Axios {
  constructor() {
    // 新增代码
    this.interceptors = {
      request: new InterceptorsManage,
      response: new InterceptorsManage
    }
  }

  request(config) {
    ...
  }
}

```

执行语句 `axios.interceptors.response.use` 和 `axios.interceptors.request.use` 的时候，实现获取 `axios` 实例上的 `interceptors` 对象，然后再获取 `response` 或 `request` 拦截器，再执行对应的拦截器的 `use` 方法

把 `Axios` 上的方法和属性搬到 `request` 过去

```

function CreateAxiosFn() {
  let axios = new Axios();

  let req = axios.request.bind(axios);
  // 混入方法，处理axios的request方法，使之拥有get,post...方法
  utils.extend(req, Axios.prototype, axios)
  // 新增代码
  utils.extend(req, axios)
  return req;
}

```

现在 `request` 也有了 `interceptors` 对象，在发送请求的时候，会先获取 `request` 拦截器的 `handlers` 的方法来执行

首先将执行 `ajax` 的请求封装成一个方法

```

request(config) {
  this.sendAjax(config)
}
sendAjax(config){
  return new Promise(resolve => {
    const {url = '', method = 'get', data = {}} = config;
    // 发送ajax请求
    console.log(config);
    const xhr = new XMLHttpRequest();
    xhr.open(method, url, true);
    xhr.onload = function() {
      console.log(xhr.responseText)
      resolve(xhr.responseText);
    };
    xhr.send(data);
  })
}

```

获得 `handlers` 中的回调

```

request(config) {
  // 拦截器和请求组装队列
  let chain = [this.sendAjax.bind(this), undefined] // 成对出现的，失败回调暂时不处理

  // 请求拦截
  this.interceptors.request.handlers.forEach(interceptor => {
    chain.unshift(interceptor.fullfield, interceptor.rejected)
  })

  // 响应拦截
  this.interceptors.response.handlers.forEach(interceptor => {
    chain.push(interceptor.fullfield, interceptor.rejected)
  })

  // 执行队列，每次执行一对，并给promise赋最新的值
  let promise = Promise.resolve(config);
  while(chain.length > 0) {
    promise = promise.then(chain.shift(), chain.shift())
  }
  return promise;
}

```

`chains` 大概

是 `['fulfilled1', 'reject1', 'fulfilled2', 'reject2', 'this.sendAjax', 'undefined', 'fulfilled2', 'reject2', 'fu`
这种形式

这样就能够成功实现一个简易版 `axios`

三、源码分析

首先看看目录结构

```
|— /lib/                                # 项目源码目
| |— /adapters/                         # 定义发送请求的适配器
| | |— http.js                         # node环境http对象
| | |— xhr.js                          # 浏览器环境XML对象
| |— /cancel/                          # 定义取消功能
| |— /helpers/                         # 一些辅助方法
| |— /core/                           # 一些核心功能
| | |— Axios.js                       # axios实例构造函数
| | |— createError.js                # 抛出错误
| | |— dispatchRequest.js           # 用来调用http请求适配器方法发送请求
| | |— InterceptorManager.js        # 拦截器管理器
| | |— mergeConfig.js               # 合并参数
| | |— settle.js                    # 根据http响应状态, 改变Promise的状态
| | |— transformData.js             # 改变数据格式
| |— axios.js                        # 入口, 创建构造函数
| |— defaults.js                    # 默认配置
| |— utils.js                       # 公用工具
```

[axios](#) 发送请求有很多实现的方法, 实现入口文件为 [axios.js](#)


```

function createInstance(defaultConfig) {
  var context = new Axios(defaultConfig);

  // instance指向了request方法，且上下文指向context，所以可以直接以 instance(option) 方式调用
  // Axios.prototype.request 内对第一个参数的数据类型判断，使我们能够以 instance(url, option) 方式调用
  var instance = bind(Axios.prototype.request, context);

  // 把Axios.prototype上的方法扩展到instance对象上，
  // 并指定上下文为context，这样执行Axios原型链上的方法时，this会指向context
  utils.extend(instance, Axios.prototype, context);

  // Copy context to instance
  // 把context对象上的自身属性和方法扩展到instance上
  // 注：因为extend内部使用的forEach方法对对象做for in 遍历时，只遍历对象本身的属性，而不会遍历原型链上的属性
  // 这样，instance就有了 defaults、interceptors 属性。
  utils.extend(instance, context);
  return instance;
}

// Create the default instance to be exported 创建一个由默认配置生成的axios实例
var axios = createInstance(defaults);

// Factory for creating new instances 扩展axios.create工厂函数，内部也是 createInstance
axios.create = function create(instanceConfig) {
  return createInstance(mergeConfig(axios.defaults, instanceConfig));
};

// Expose all/spread
axios.all = function all(promises) {
  return Promise.all(promises);
};

axios.spread = function spread(callback) {
  return function wrap(arr) {
    return callback.apply(null, arr);
  };
};

module.exports = axios;

```

主要核心是 `Axios.prototype.request`，各种请求方式的调用实现都是在 `request` 内部实现的，简单看下 `request` 的逻辑

```

Axios.prototype.request = function request(config) {
  // Allow for axios('example/url'[, config]) a la fetch API
  // 判断 config 参数是否是 字符串，如果是则认为第一个参数是 URL，第二个参数是真正的config
  if (typeof config === 'string') {
    config = arguments[1] || {};
    // 把 url 放置到 config 对象中，便于之后的 mergeConfig
    config.url = arguments[0];
  } else {
    // 如果 config 参数是否是 字符串，则整体都当做config
    config = config || {};
  }
  // 合并默认配置和传入的配置
  config = mergeConfig(this.defaults, config);
  // 设置请求方法
  config.method = config.method ? config.method.toLowerCase() : 'get';
  /*
    something... 此部分会在后续拦截器单独讲述
  */
};

// 在 Axios 原型上挂载 'delete', 'get', 'head', 'options' 且不传参的请求方法，实现内部也是 request
utils.forEach(['delete', 'get', 'head', 'options'], function forEachMethodNoData(method) {
  Axios.prototype[method] = function(url, config) {
    return this.request(utils.merge(config || {}, {
      method: method,
      url: url
    }));
  };
});

// 在 Axios 原型上挂载 'post', 'put', 'patch' 且传参的请求方法，实现内部同样也是 request
utils.forEach(['post', 'put', 'patch'], function forEachMethodWithData(method) {
  Axios.prototype[method] = function(url, data, config) {
    return this.request(utils.merge(config || {}, {
      method: method,
      url: url,
      data: data
    }));
  };
});

```

`request` 入口参数为 `config`，可以说 `config` 贯彻了 `axios` 的一生

`axios` 中的 `config` 主要分布在这几个地方：

- 默认配置 `defaults.js`
- `config.method` 默认为 `get`
- 调用 `createInstance` 方法创建 `axios` 实例，传入的 `config`
- 直接或间接调用 `request` 方法，传入的 `config`

```

// axios.js
// 创建一个由默认配置生成的axios实例
var axios = createInstance(defaults);

// 扩展axios.create工厂函数，内部也是 createInstance
axios.create = function create(instanceConfig) {
  return createInstance(mergeConfig(axios.defaults, instanceConfig));
};

// Axios.js
// 合并默认配置和传入的配置
config = mergeConfig(this.defaults, config);
// 设置请求方法
config.method = config.method ? config.method.toLowerCase() : 'get';

```

从源码中，可以看到优先级：默认配置对象 `default` < `method:get` < `Axios` 的实例属性 `this.default` < `request` 参数

下面重点看看 `request` 方法

```

Axios.prototype.request = function request(config) {
  /*
    先是 mergeConfig ... 等，不再阐述
  */
  // Hook up interceptors middleware 创建拦截器链. dispatchRequest 是重中之重，后续重点
  var chain = [dispatchRequest, undefined];

  // push各个拦截器方法 注意： interceptor.fulfilled 或 interceptor.rejected 是可能为undefined
  this.interceptors.request.forEach(function unshiftRequestInterceptors(interceptor) {
    // 请求拦截器逆序 注意此处的 forEach 是自定义的拦截器的forEach方法
    chain.unshift(interceptor.fulfilled, interceptor.rejected);
  });

  this.interceptors.response.forEach(function pushResponseInterceptors(interceptor) {
    // 响应拦截器顺序 注意此处的 forEach 是自定义的拦截器的forEach方法
    chain.push(interceptor.fulfilled, interceptor.rejected);
  });

  // 初始化一个promise对象，状态为resolved，接收到的参数为已经处理合并过的config对象
  var promise = Promise.resolve(config);

  // 循环拦截器的链
  while (chain.length) {
    promise = promise.then(chain.shift(), chain.shift()); // 每一次向外弹出拦截器
  }
  // 返回 promise
  return promise;
};

```

拦截器 `interceptors` 是在构建 `axios` 实例化的属性

```
function Axios(instanceConfig) {
  this.defaults = instanceConfig;
  this.interceptors = {
    request: new InterceptorManager(), // 请求拦截
    response: new InterceptorManager() // 响应拦截
  };
}
```

InterceptorManager 构造函数

```
// 拦截器的初始化 其实就是一组钩子函数
function InterceptorManager() {
  this.handlers = [];
}

// 调用拦截器实例的use时就是往钩子函数中push方法
InterceptorManager.prototype.use = function use(fulfilled, rejected) {
  this.handlers.push({
    fulfilled: fulfilled,
    rejected: rejected
  });
  return this.handlers.length - 1;
};

// 拦截器是可以取消的，根据use的时候返回的ID，把某一个拦截器方法置为null
// 不能用 splice 或者 slice 的原因是 删除之后id 就会变化，导致之后的顺序或者是操作不可控
InterceptorManager.prototype.eject = function eject(id) {
  if (this.handlers[id]) {
    this.handlers[id] = null;
  }
};

// 这就是在 Axios的request方法中 中循环拦截器的方法 forEach 循环执行钩子函数
InterceptorManager.prototype.forEach = function forEach(fn) {
  utils.forEach(this.handlers, function forEachHandler(h) {
    if (h !== null) {
      fn(h);
    }
  });
}
```

请求拦截器方法是被 `unshift` 到拦截器中，响应拦截器是被 `push` 到拦截器中的。最终它们会拼接上一个叫 `dispatchRequest` 的方法被后续的 `promise` 顺序执行

```
var utils = require('./utils');
var transformData = require('./transformData');
var isCancel = require('./cancel/isCancel');
var defaults = require('./defaults');
var isAbsoluteURL = require('./helpers/isAbsoluteURL');
var combineURLs = require('./helpers/combineURLs');
```

```
// 判断请求是否已被取消, 如果已经被取消, 抛出已取消
function throwIfCancellationRequested(config) {
  if (config.cancelToken) {
    config.cancelToken.throwIfRequested();
  }
}

module.exports = function dispatchRequest(config) {
  throwIfCancellationRequested(config);

  // 如果包含baseUrl, 并且不是config.url绝对路径, 组合baseUrl以及config.url
  if (config.baseURL && !isAbsoluteURL(config.url)) {
    // 组合baseURL与url形成完整的请求路径
    config.url = combineURLs(config.baseURL, config.url);
  }

  config.headers = config.headers || {};

  // 使用lib/defaults.js中的transformRequest方法, 对config.headers和config.data进行格式化
  // 比如将headers中的Accept, Content-Type统一处理成大写
  // 比如如果请求正文是一个Object会格式化为JSON字符串, 并添加application/json;charset=utf-8的
  // Content-Type
  // 等一系列操作
  config.data = transformData(
    config.data,
    config.headers,
    config.transformRequest
  );

  // 合并不同配置的headers, config.headers的配置优先级更高
  config.headers = utils.merge(
    config.headers.common || {},
    config.headers[config.method] || {},
    config.headers || {}
  );

  // 删除headers中的method属性
  utils.forEach(
    ['delete', 'get', 'head', 'post', 'put', 'patch', 'common'],
    function cleanHeaderConfig(method) {
      delete config.headers[method];
    }
  );

  // 如果config配置了adapter, 使用config中配置adapter的替代默认的请求方法
  var adapter = config.adapter || defaults.adapter;

  // 使用adapter方法发起请求 (adapter根据浏览器环境或者Node环境会有不同)
  return adapter(config).then(
    // 请求正确返回的回调
    function onAdapterResolution(response) {
      // 判断是否以及取消了请求, 如果取消了请求抛出以取消
      throwIfCancellationRequested(config);
    }
  );
}
```



```
// 使用lib/defaults.js中的transformResponse方法，对服务器返回的数据进行格式化
// 例如，使用JSON.parse对响应正文进行解析
response.data = transformData(
  response.data,
  response.headers,
  config.transformResponse
);

return response;
},
// 请求失败的回调
function onAdapterRejection(reason) {
  if (!isCancel(reason)) {
    throwIfCancellationRequested(config);

    if (reason && reason.response) {
      reason.response.data = transformData(
        reason.response.data,
        reason.response.headers,
        config.transformResponse
      );
    }
  }
  return Promise.reject(reason);
}
);
};
```

再来看看 [axios](#) 是如何实现取消请求的，实现文件在 [CancelToken.js](#)


```

function CancelToken(executor) {
  if (typeof executor !== 'function') {
    throw new TypeError('executor must be a function.');
```

// 在 CancelToken 上定义一个 pending 状态的 promise，将 resolve 回调赋值给外部变量

```

  resolvePromise
  var resolvePromise;
  this.promise = new Promise(function promiseExecutor(resolve) {
    resolvePromise = resolve;
  });

  var token = this;
  // 立即执行传入的 executor 函数，将真实的 cancel 方法通过参数传递出去。
  // 一旦调用就执行 resolvePromise 即前面的 promise 的 resolve，就更改变 promise 的状态为 resolve。
  // 那么 xhr 中定义的 CancelToken.promise.then 方法就会执行，从而 xhr 内部会取消请求
  executor(function cancel(message) {
    // 判断请求是否已经取消过，避免多次执行
    if (token.reason) {
      return;
    }
    token.reason = new Cancel(message);
    resolvePromise(token.reason);
  });
}

CancelToken.source = function source() {
  // source 方法就是返回了一个 CancelToken 实例，与直接使用 new CancelToken 是一样的操作
  var cancel;
  var token = new CancelToken(function executor(c) {
    cancel = c;
  });
  // 返回创建的 CancelToken 实例以及取消方法
  return {
    token: token,
    cancel: cancel
  };
};

```

实际上取消请求的操作是在 `xhr.js` 中也有响应的配合的

```

if (config.cancelToken) {
  config.cancelToken.promise.then(function onCanceled(cancel) {
    if (!request) {
      return;
    }
    // 取消请求
    request.abort();
    reject(cancel);
  });
}

```

巧妙的地方在 `CancelToken` 中 `executor` 函数，通过 `resolve` 函数的传递与执行，控制 `promise` 的状态

小结

参考文献

- <https://juejin.cn/post/6856706569263677447#heading-4>
- <https://juejin.cn/post/6844903907500490766>
- <https://github.com/axios/axios>

面试官：你了解vue的diff算法吗？说说看

一、dom diff

`diff` 算法是一种通过同层的树节点进行比较的高效算法

其有两个特点：

- 比较只会同层级进行，不会跨层级比较
- 在 `diff` 比较的过程中，循环从两边向中间比较

`diff` 算法在很多场景下都有应用，在 `vue` 中，作用于虚拟 `dom` 渲染成真实 `dom` 的新旧 `VNode` 节点比较

二、比较方式

`diff` 整体策略为：深度优先，同层比较

1. 比较只会同层级进行，不会跨层级比较

```

```

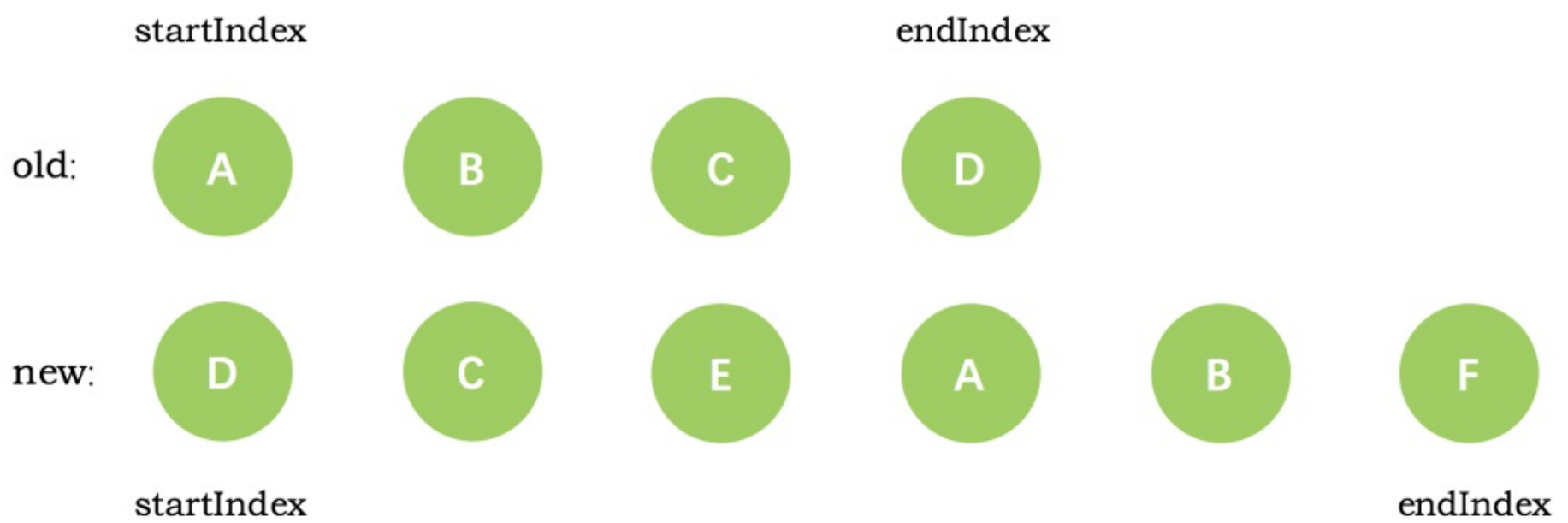
2. 比较的过程中，循环从两边向中间收拢

```

```

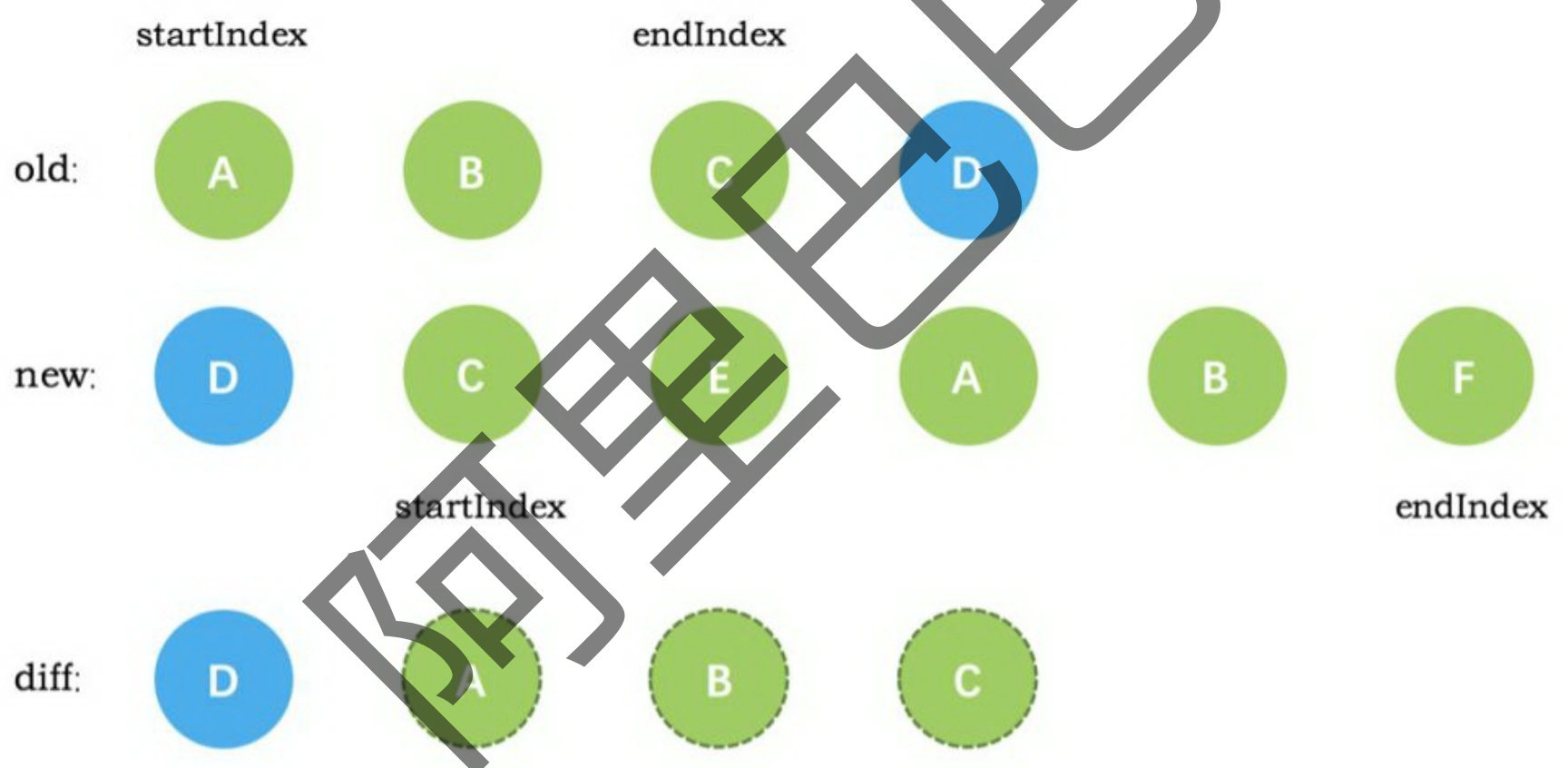
下面举个 `vue` 通过 `diff` 算法更新的例子：

新旧 `VNode` 节点如下图所示：



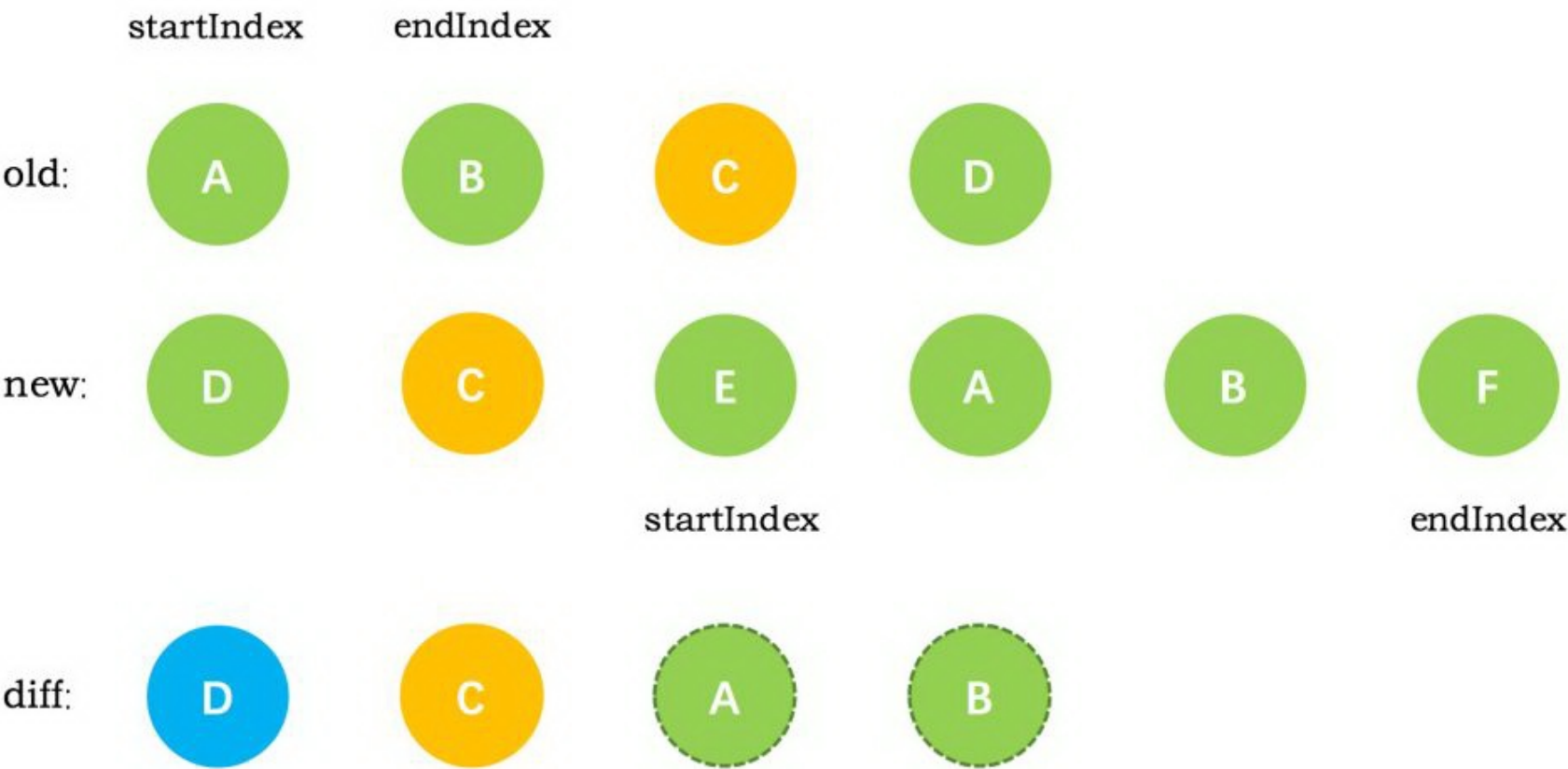
第一次循环后，发现旧节点D与新节点D相同，直接复用旧节点D作为 `diff` 后的第一个真实节点，同时旧节点 `endIndex` 移动到C，新节点的 `startIndex` 移动到了 C

第一次:



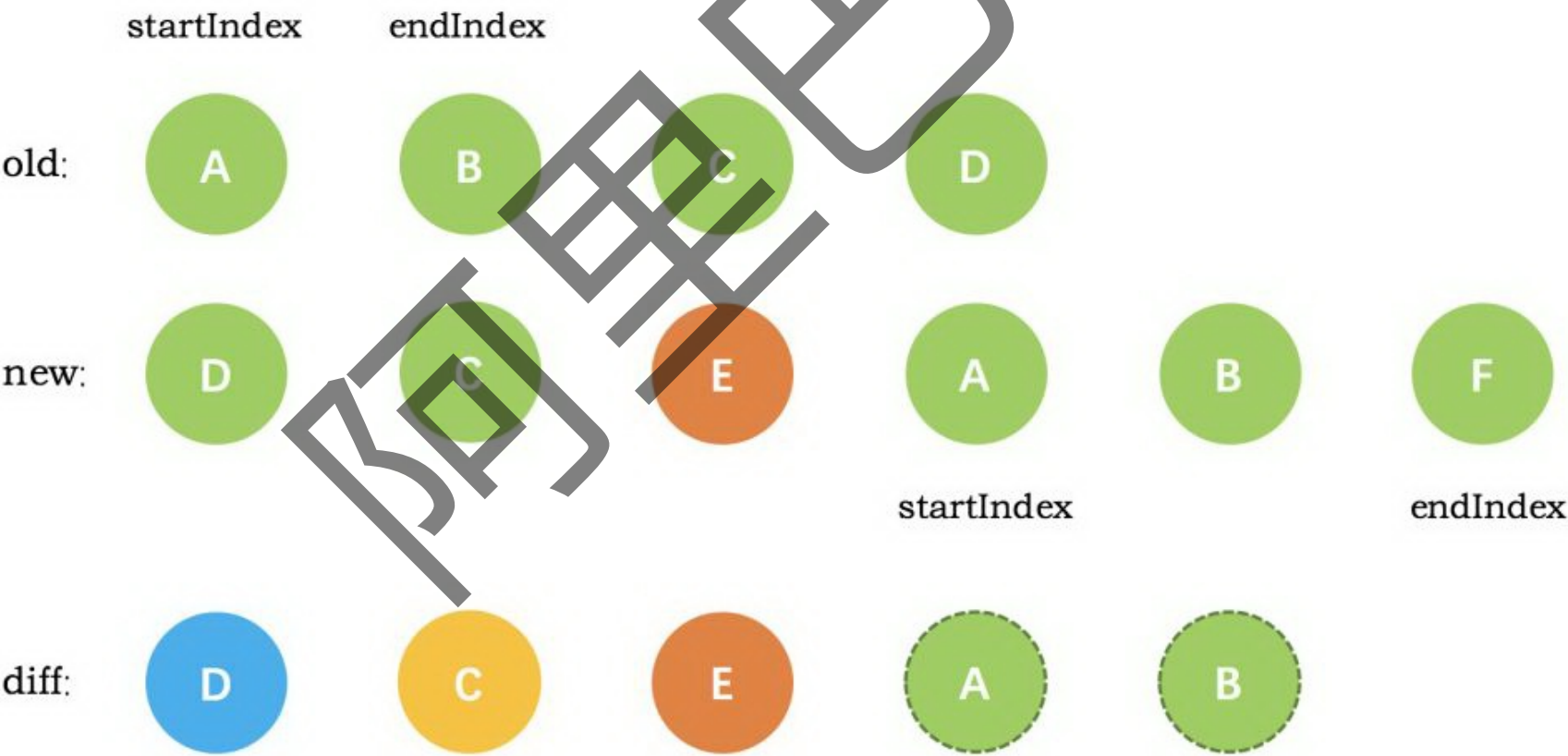
第二次循环后，同样是旧节点的末尾和新节点的开头(都是 C)相同，同理，`diff` 后创建了 C 的真实节点插入到第一次创建的 D 节点后面。同时旧节点的 `endIndex` 移动到了 B，新节点的 `startIndex` 移动到了 E

第二次:



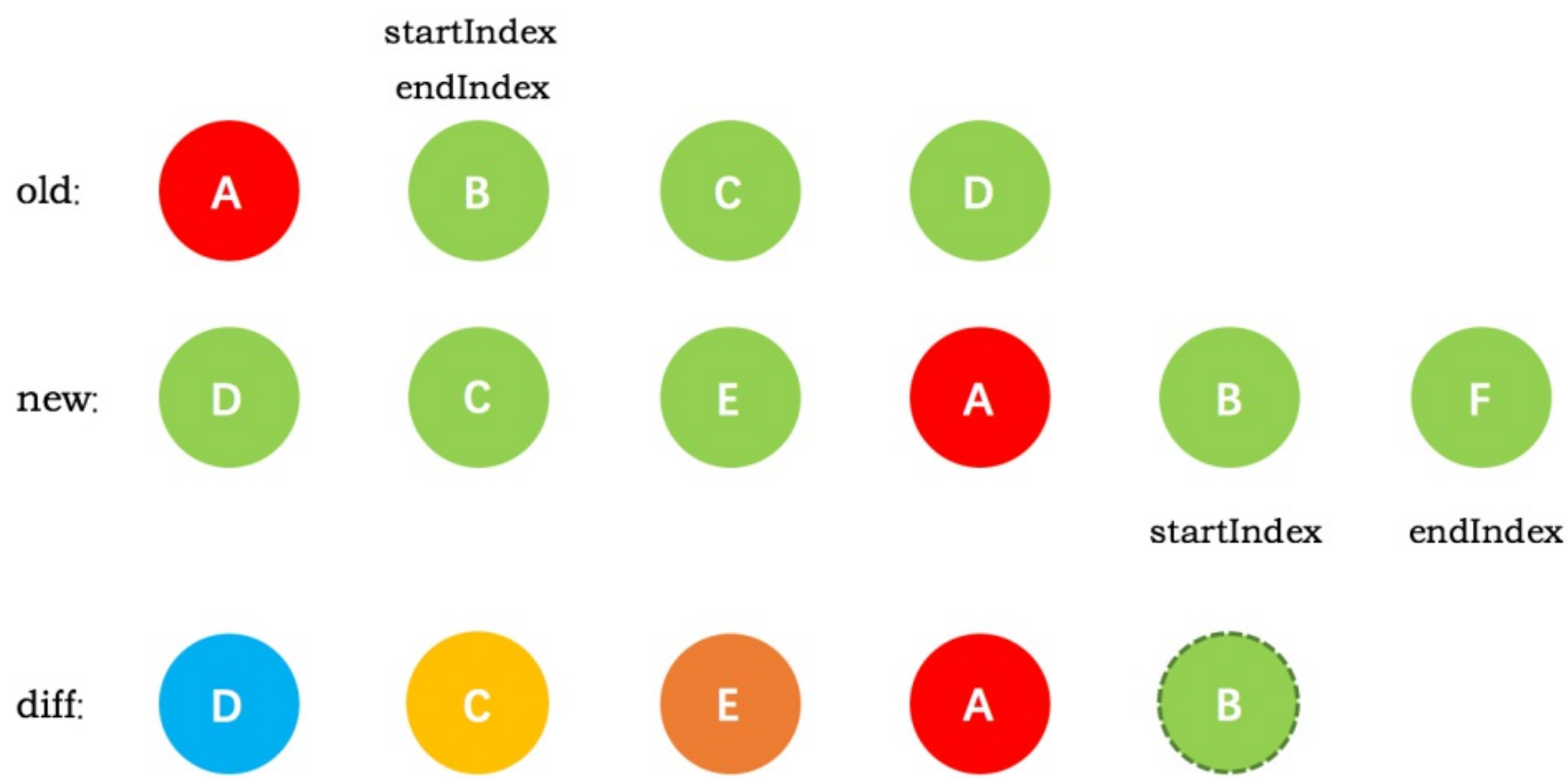
第三次循环中，发现E没有找到，这时候只能直接创建新的真实节点 E，插入到第二次创建的 C 节点之后。同时新节点的 `startIndex` 移动到了 A。旧节点的 `startIndex` 和 `endIndex` 都保持不动

第三次:



第四次循环中，发现了新旧节点的开头(都是 A)相同，于是 `diff` 后创建了 A 的真实节点，插入到前一次创建的 E 节点后面。同时旧节点的 `startIndex` 移动到了 B，新节点的 `startIndex` 移动到了 B

第四次:



第五次循环中，情形同第四次循环一样，因此 `diff` 后创建了 `B` 真实节点 插入到前一次创建的 `A` 节点后面。同时旧节点的 `startIndex` 移动到了 `C`，新节点的 `startIndex` 移动到了 `F`

第五次:



新节点的 `startIndex` 已经大于 `endIndex` 了，需要创建 `newStartIdx` 和 `newEndIdx` 之间的所有节点，也就是节点 `F`，直接创建 `F` 节点对应的真实节点放到 `B` 节点后面

第六次:

因 $\text{oldStartIndex} > \text{oldEndIndex}$, 退出循环, 补充new增加的节点F

diff:



三、原理分析

当数据发生改变时, `set` 方法会调用 `Dep.notify` 通知所有订阅者 `Watcher`, 订阅者就会调用 `patch` 给真实的 `DOM` 打补丁, 更新相应的视图

源码位置: `src/core/vdom/patch.js`


```

function patch(oldVnode, vnode, hydrating, removeOnly) {
  if (isUndef(vnode)) { // 没有新节点, 直接执行destory钩子函数
    if (isDef(oldVnode)) invokeDestroyHook(oldVnode)
    return
  }

  let isInitialPatch = false
  const insertedVnodeQueue = []

  if (isUndef(oldVnode)) {
    isInitialPatch = true
    createElm(vnode, insertedVnodeQueue) // 没有旧节点, 直接用新节点生成dom元素
  } else {
    const isRealElement = isDef(oldVnode.nodeType)
    if (!isRealElement && sameVnode(oldVnode, vnode)) {
      // 判断旧节点和新节点自身一样, 一致执行patchVnode
      patchVnode(oldVnode, vnode, insertedVnodeQueue, null, null, removeOnly)
    } else {
      // 否则直接销毁及旧节点, 根据新节点生成dom元素
      if (isRealElement) {

        if (oldVnode.nodeType === 1 && oldVnode.hasAttribute(SSR_ATTR)) {
          oldVnode.removeAttribute(SSR_ATTR)
          hydrating = true
        }
        if (isTrue(hydrating)) {
          if (hydrate(oldVnode, vnode, insertedVnodeQueue)) {
            invokeInsertHook(vnode, insertedVnodeQueue, true)
            return oldVnode
          }
        }
        oldVnode = emptyNodeAt(oldVnode)
      }
    }
    return vnode.elm
  }
}

```

`patch` 函数前两个参数位为 `oldVnode` 和 `Vnode` , 分别代表新的节点和之前的旧节点, 主要做了四个判断:

- 没有新节点, 直接触发旧节点的 `destory` 钩子
- 没有旧节点, 说明是页面刚开始初始化的时候, 此时, 根本不需要比较了, 直接全是新建, 所以只调用 `createElm`
- 旧节点和新节点自身一样, 通过 `sameVnode` 判断节点是否一样, 一样时, 直接调用 `patchVnode` 去处理这两个节点
- 旧节点和新节点自身不一样, 当两个节点不一样的时候, 直接创建新节点, 删除旧节点

下面主要讲的是 `patchVnode` 部分

```

function patchVnode (oldVnode, vnode, insertedVnodeQueue, removeOnly) {
  // 如果新旧节点一致, 什么都不做

```

```
if (oldVnode === vnode) {
  return
}

// 让vnode.el引用到现在的真实dom, 当el修改时, vnode.el会同步变化
const elm = vnode.elm = oldVnode.elm

// 异步占位符
if (isTrue(oldVnode.isAsyncPlaceholder)) {
  if (isDef(vnode.asyncFactory.resolved)) {
    hydrate(oldVnode.elm, vnode, insertedVnodeQueue)
  } else {
    vnode.isAsyncPlaceholder = true
  }
  return
}
// 如果新旧都是静态节点, 并且具有相同的key
// 当vnode是克隆节点或是v-once指令控制的节点时, 只需要把oldVnode.elm和oldVnode.child都
复制到vnode上
// 也不用再有其他操作
if (isTrue(vnode.isStatic) &&
  isTrue(oldVnode.isStatic) &&
  vnode.key === oldVnode.key &&
  (isTrue(vnode.isCloned) || isTrue(vnode.isOnce))) {
  vnode.componentInstance = oldVnode.componentInstance
  return
}

let i
const data = vnode.data
if (isDef(data) && isDef(i = data.hook) && isDef(i = i.prepatch)) {
  i(oldVnode, vnode)
}

const oldCh = oldVnode.children
const ch = vnode.children
if (isDef(data) && isPatchable(vnode)) {
  for (i = 0; i < cbs.update.length; ++i) cbs.update[i](oldVnode, vnode)
  if (isDef(i = data.hook) && isDef(i = i.update)) i(oldVnode, vnode)
}
// 如果vnode不是文本节点或者注释节点
if (isUndef(vnode.text)) {
  // 并且都有子节点
  if (isDef(oldCh) && isDef(ch)) {
    // 并且子节点不完全一致, 则调用updateChildren
    if (oldCh !== ch) updateChildren(elm, oldCh, ch, insertedVnodeQueue, removeOnly)

    // 如果只有新的vnode有子节点
  } else if (isDef(ch)) {
    if (isDef(oldVnode.text)) nodeOps.setTextContent(elm, '')
    // elm已经引用了老的dom节点, 在老的dom节点上添加子节点
    addVnodes(elm, null, ch, 0, ch.length - 1, insertedVnodeQueue)
  }
}
```

```

    // 如果新vnode没有子节点，而vnode有子节点，直接删除老的oldCh
  } else if (isDef(oldCh)) {
    removeVnodes(elm, oldCh, 0, oldCh.length - 1)

    // 如果老节点是文本节点
  } else if (isDef(oldVnode.text)) {
    nodeOps.setTextContent(elm, "")
  }

  // 如果新vnode和老vnode是文本节点或注释节点
  // 但是vnode.text !== oldVnode.text时，只需要更新vnode.elm的文本内容就可以
} else if (oldVnode.text !== vnode.text) {
  nodeOps.setTextContent(elm, vnode.text)
}
if (isDef(data)) {
  if (isDef(i = data.hook) && isDef(i = i.postpatch)) i(oldVnode, vnode)
}
}
}

```

`patchVnode` 主要做了几个判断：

- 新节点是否是文本节点，如果是，则直接更新 `dom` 的文本内容为新节点的文本内容
- 新节点和旧节点如果都有子节点，则处理比较更新子节点
- 只有新节点有子节点，旧节点没有，那么不用比较了，所有节点都是全新的，所以直接全部新建就好了，新建是指创建出所有新 `DOM`，并且添加进父节点
- 只有旧节点有子节点而新节点没有，说明更新后的页面，旧节点全部都不见了，那么要做的，就是把所有的旧节点删除，也就是直接把 `DOM` 删除

子节点不完全一致，则调用 `updateChildren`

```

function updateChildren (parentElm, oldCh, newCh, insertedVnodeQueue, removeOnly) {
  let oldStartIdx = 0 // 旧头索引
  let newStartIdx = 0 // 新头索引
  let oldEndIdx = oldCh.length - 1 // 旧尾索引
  let newEndIdx = newCh.length - 1 // 新尾索引
  let oldStartVnode = oldCh[0] // oldVnode的第一个child
  let oldEndVnode = oldCh[oldEndIdx] // oldVnode的最后一个child
  let newStartVnode = newCh[0] // newVnode的第一个child
  let newEndVnode = newCh[newEndIdx] // newVnode的最后一个child
  let oldKeyToIdx, idxInOld, vnodeToMove, refElm

  // removeOnly is a special flag used only by <transition-group>
  // to ensure removed elements stay in correct relative positions
  // during leaving transitions
  const canMove = !removeOnly

  // 如果oldStartVnode和oldEndVnode重合，并且新的也都重合了，证明diff完了，循环结束
  while (oldStartIdx <= oldEndIdx && newStartIdx <= newEndIdx) {
    // 如果oldVnode的第一个child不存在
    if (isUndef(oldStartVnode)) {
      // oldStart索引右移
      oldStartVnode = oldCh[++oldStartIdx] // Vnode has been moved left
    }
  }
}

```

```

// 如果oldVnode的最后一个child不存在
} else if (isUndef(oldEndVnode)) {
  // oldEnd索引左移
  oldEndVnode = oldCh[--oldEndIdx]

// oldStartVnode和newStartVnode是同一个节点
} else if (sameVnode(oldStartVnode, newStartVnode)) {
  // patch oldStartVnode和newStartVnode, 索引左移, 继续循环
  patchVnode(oldStartVnode, newStartVnode, insertedVnodeQueue)
  oldStartVnode = oldCh[++oldStartIdx]
  newStartVnode = newCh[++newStartIdx]

// oldEndVnode和newEndVnode是同一个节点
} else if (sameVnode(oldEndVnode, newEndVnode)) {
  // patch oldEndVnode和newEndVnode, 索引右移, 继续循环
  patchVnode(oldEndVnode, newEndVnode, insertedVnodeQueue)
  oldEndVnode = oldCh[--oldEndIdx]
  newEndVnode = newCh[--newEndIdx]

// oldStartVnode和newEndVnode是同一个节点
} else if (sameVnode(oldStartVnode, newEndVnode)) { // Vnode moved right
  // patch oldStartVnode和newEndVnode
  patchVnode(oldStartVnode, newEndVnode, insertedVnodeQueue)
  // 如果removeOnly是false, 则将oldStartVnode.elm移动到oldEndVnode.elm之后
  canMove && nodeOps.insertBefore(parentElm, oldStartVnode.elm,
nodeOps.nextSibling(oldEndVnode.elm))
  // oldStart索引右移, newEnd索引左移
  oldStartVnode = oldCh[++oldStartIdx]
  newEndVnode = newCh[--newEndIdx]

// 如果oldEndVnode和newStartVnode是同一个节点
} else if (sameVnode(oldEndVnode, newStartVnode)) { // Vnode moved left
  // patch oldEndVnode和newStartVnode
  patchVnode(oldEndVnode, newStartVnode, insertedVnodeQueue)
  // 如果removeOnly是false, 则将oldEndVnode.elm移动到oldStartVnode.elm之前
  canMove && nodeOps.insertBefore(parentElm, oldEndVnode.elm, oldStartVnode.elm)
  // oldEnd索引左移, newStart索引右移
  oldEndVnode = oldCh[--oldEndIdx]
  newStartVnode = newCh[++newStartIdx]

// 如果都不匹配
} else {
  if (isUndef(oldKeyToIdx)) oldKeyToIdx = createKeyToOldIdx(oldCh, oldStartIdx, oldEndIdx)

  // 尝试在oldChildren中寻找和newStartVnode的具有相同的key的Vnode
  idxInOld = isDef(newStartVnode.key)
    ? oldKeyToIdx[newStartVnode.key]
    : findIdxInOld(newStartVnode, oldCh, oldStartIdx, oldEndIdx)

  // 如果未找到, 说明newStartVnode是一个新的节点
  if (isUndef(idxInOld)) { // New element
    // 创建一个新Vnode

```



```

createElm(newStartVnode, insertedVnodeQueue, parentElm, oldStartVnode.elm)

// 如果找到了和newStartVnodej具有相同的key的Vnode, 叫vnodeToMove
} else {
  vnodeToMove = oldCh[idxInOld]
  /* istanbul ignore if */
  if (process.env.NODE_ENV !== 'production' && !vnodeToMove) {
    warn(
      'It seems there are duplicate keys that is causing an update error. ' +
      'Make sure each v-for item has a unique key.'
    )
  }

  // 比较两个具有相同的key的新节点是否是同一个节点
  // 不设key, newCh和oldCh只会进行头尾两端的相互比较, 设key后, 除了头尾两端的比较
  // 外, 还会从用key生成的对象oldKeyToIdx中查找匹配的节点, 所以为节点设置key可以更高效率的利用
  // dom。
  if (sameVnode(vnodeToMove, newStartVnode)) {
    // patch vnodeToMove和newStartVnode
    patchVnode(vnodeToMove, newStartVnode, insertedVnodeQueue)
    // 清除
    oldCh[idxInOld] = undefined
    // 如果removeOnly是false, 则将找到的和newStartVnodej具有相同的key的Vnode, 叫
    // vnodeToMove.elm
    // 移动到oldStartVnode.elm之前
    canMove && nodeOps.insertBefore(parentElm, vnodeToMove.elm, oldStartVnode.elm)

    // 如果key相同, 但是节点不相同, 则创建一个新的节点
  } else {
    // same key but different element. treat as new element
    createElm(newStartVnode, insertedVnodeQueue, parentElm, oldStartVnode.elm)
  }
}

// 右移
newStartVnode = newCh[++newStartIdx]
}
}

```

`while` 循环主要处理了以下五种情景：

- 当新老 `VNode` 节点的 `start` 相同时，直接 `patchVnode`，同时新老 `VNode` 节点的开始索引都加 1
- 当新老 `VNode` 节点的 `end` 相同时，同样直接 `patchVnode`，同时新老 `VNode` 节点的结束索引都减 1
- 当老 `VNode` 节点的 `start` 和新 `VNode` 节点的 `end` 相同时，这时候在 `patchVnode` 后，还需要将当前真实 `dom` 节点移动到 `oldEndVnode` 的后面，同时老 `VNode` 节点开始索引加 1，新 `VNode` 节点的结束索引减 1
- 当老 `VNode` 节点的 `end` 和新 `VNode` 节点的 `start` 相同时，这时候在 `patchVnode` 后，还需要将当前真实 `dom` 节点移动到 `oldStartVnode` 的前面，同时老 `VNode` 节点结束索引减 1，新 `VNode` 节点的开始索引加 1
- 如果都不满足以上四种情形，那说明没有相同的节点可以复用，则会分为以下两种情况：

- 从旧的 `VNode` 为 `key` 值，对应 `index` 序列为 `value` 值的哈希表中找到与 `newStartVnode` 一致 `key` 的旧的 `VNode` 节点，再进行 `patchVnode`，同时将这个真实 `dom` 移动到 `oldStartVnode` 对应的真实 `dom` 的前面
- 调用 `createElm` 创建一个新的 `dom` 节点放到当前 `newStartIdx` 的位置

小结

- 当数据发生改变时，订阅者 `watcher` 就会调用 `patch` 给真实的 `DOM` 打补丁
- 通过 `isSameVnode` 进行判断，相同则调用 `patchVnode` 方法
- `patchVnode` 做了以下操作：
 - 找到对应的真实 `dom`，称为 `el`
 - 如果都有都有文本节点且不相等，将 `el` 文本节点设置为 `Vnode` 的文本节点
 - 如果 `oldVnode` 有子节点而 `VNode` 没有，则删除 `el` 子节点
 - 如果 `oldVnode` 没有子节点而 `VNode` 有，则将 `VNode` 的子节点真实化后添加到 `el`
 - 如果两者都有子节点，则执行 `updateChildren` 函数比较子节点
- `updateChildren` 主要做了以下操作：
 - 设置新旧 `VNode` 的头尾指针
 - 新旧头尾指针进行比较，循环向中间靠拢，根据情况调用 `patchVnode` 进行 `patch` 重复流程、调用 `createElem` 创建一个新节点，从哈希表寻找 `key` 一致的 `VNode` 节点再分情况操作

参考文献

- <https://juejin.cn/post/6881907432541552648#heading-1>
- <https://www.infoq.cn/article/udlcpkh4iqb0cr5wgy7f>

面试官：你对SPA单页面的理解，它的优缺点分别是什么？如何实现SPA应用呢

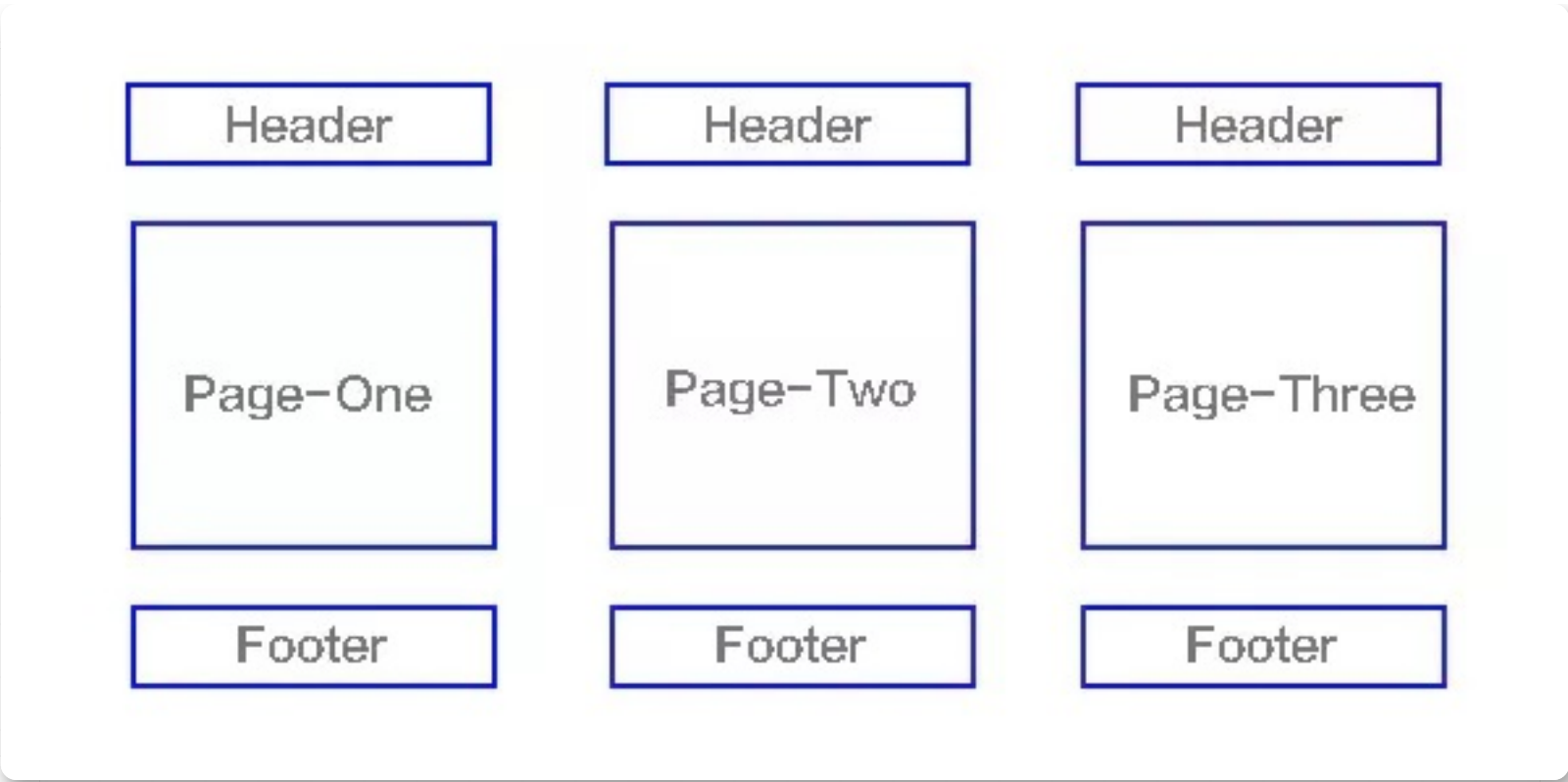
一、什么是SPA

SPA (single-page application)，翻译过来就是单页应用 `SPA` 是一种网络应用程序或网站的模型，它通过动态重写当前页面来与用户交互，这种方法避免了页面之间切换打断用户体验在单页应用中，所有必要的代码（`HTML`、`JavaScript` 和 `CSS`）都通过单个页面的加载而检索，或者根据需要（通常是响应用户操作）动态装载适当的资源并添加到页面页面在任何时间点都不会重新加载，也不会将控制转移到其他页面举个例子来讲就是一个杯子，早上装的牛奶，中午装的是开水，晚上装的是茶，我们发现，变的始终是杯子里的内容，而杯子始终是那个杯子结构如下图

我们熟知的JS框架如 `react`、`vue`、`angular`、`ember` 都属于 `SPA`

二、SPA和MPA的区别

上面大家已经对单页面有所了解了，下面来讲讲多页应用MPA (MultiPage-page application)，翻译过来就是多页应用在 `MPA` 中，每个页面都是一个主页面，都是独立的当我们在访问另一个页面的时候，都需要重新加载 `html`、`css`、`js` 文件，公共文件则根据需求按需加载如下图



单页应用与多页应用的区别

	单页面应用（SPA）	多页面应用（MPA）
组成	一个主页面和多个页面片段	多个主页面
刷新方式	局部刷新	整页刷新
url模式	哈希模式	历史模式
SEO搜索引擎优化	难实现，可使用SSR方式改善	容易实现
数据传递	容易	通过url、cookie、localStorage等传递
页面切换	速度快，用户体验良好	切换加载资源，速度慢，用户体验差
维护成本	相对容易	相对复杂

单页应用优缺点

优点：

- 具有桌面应用的即时性、网站的可移植性和可访问性
- 用户体验好、快，内容的改变不需要重新加载整个页面
- 良好的前后端分离，分工更明确

缺点：

- 不利于搜索引擎的抓取
- 首次渲染速度相对较慢
-

三、实现一个SPA

原理

- 监听地址栏中 hash 变化驱动界面变化
- 用 pushstate 记录浏览器的历史，驱动界面发送变化

实现

hash 模式

核心通过监听 url 中的 hash 来进行路由跳转

```
// 定义 Router
class Router {
  constructor () {
    this.routes = {}; // 存放路由path及callback
    this.currentUrl = "";

    // 监听路由change调用相对应的路由回调
    window.addEventListener('load', this.refresh, false);
    window.addEventListener('hashchange', this.refresh, false);
  }

  route(path, callback){
    this.routes[path] = callback;
  }

  push(path) {
    this.routes[path] && this.routes[path]()
  }
}

// 使用 router
window.miniRouter = new Router();
miniRouter.route('/', () => console.log('page1'))
miniRouter.route('/page2', () => console.log('page2'))

miniRouter.push('/') // page1
miniRouter.push('/page2') // page2
```

history 模式

history 模式核心借用 [HTML5 history api](#)，[api](#) 提供了丰富的 [router](#) 相关属性先了解一个几个相关的api

- [history.pushState](#) 浏览器历史纪录添加记录
- [history.replaceState](#) 修改浏览器历史纪录中当前纪录
- [history.popState](#) 当 [history](#) 发生变化时触发

```

// 定义 Router
class Router {
  constructor () {
    this.routes = {};
    this.listerPopState()
  }

  init(path) {
    history.replaceState({path: path}, null, path);
    this.routes[path] && this.routes[path]();
  }

  route(path, callback){
    this.routes[path] = callback;
  }

  push(path) {
    history.pushState({path: path}, null, path);
    this.routes[path] && this.routes[path]();
  }

  listerPopState () {
    window.addEventListener('popstate', e => {
      const path = e.state && e.state.path;
      this.routes[path] && this.routes[path]()
    })
  }
}

// 使用 Router

window.miniRouter = new Router();
miniRouter.route('/', ()=> console.log('page1'))
miniRouter.route('/page2', ()=> console.log('page2'))

// 跳转
miniRouter.push('/page2') // page2

```

四、题外话：如何给SPA做SEO

下面给出基于 Vue 的 SPA 如何实现 SEO 的三种方式

1. 「SSR服务端渲染」

将组件或页面通过服务器生成html，再返回给浏览器，如 [nuxt.js](#)

2. 「静态化」

目前主流的静态化主要有两种：（1）一种是通过程序将动态页面抓取并保存为静态页面，这样的页面的实际存在于服务器的硬盘中（2）另外一种是通过WEB服务器的 [URL Rewrite](#)的方式，它的原理是通过web服务器内部模块按一定规则将外部的URL请求转化为内部的文件地址，一句话来说就是把外部请求的静态地址转化为实际的动态页面地址，而静态页面实际是不存在的。这两种方法都达到了实现URL静态化的效果

3. 「使用 Phantomjs 针对爬虫处理」

原理是通过 [Nginx](#) 配置，判断访问来源是否为爬虫，如果是则搜索引擎的爬虫请求会转发到一个 [node server](#)，再通过 [PhantomJS](#) 来解析完整的 [HTML](#)，返回给爬虫。下面是大致流程图

参考文献

- <https://segmentfault.com/a/1190000019623624>
- <https://juejin.cn/post/6844903512107663368>
- <https://www.cnblogs.com/constantince/p/5586851.html>

面试官：你是怎么处理vue项目中的错误的？

一、错误类型

任何一个框架，对于错误的处理都是一种必备的能力

在 [Vue](#) 中，则是定义了一套对应的错误处理规则给到使用者，且在源代码级别，对部分必要的过程做了一定的错误处理。

主要的错误来源包括：

- 后端接口错误
- 代码中本身逻辑错误

二、如何处理

后端接口错误

通过 [axios](#) 的 [interceptor](#) 实现网络请求的 [response](#) 先进行一层拦截

```
apiClient.interceptors.response.use(
  response => {
    return response;
  },
  error => {
    if (error.response.status === 401) {
      router.push({ name: "Login" });
    } else {
      message.error("出错了");
      return Promise.reject(error);
    }
  }
);
```

代码逻辑问题

全局设置错误处理

设置全局错误处理函数

```
Vue.config.errorHandler = function (err, vm, info) {  
  // handle error  
  // `info` 是 Vue 特定的错误信息，比如错误所在的生命周期钩子  
  // 只在 2.2.0+ 可用  
}
```

`errorHandler` 指定组件的渲染和观察期间未捕获错误的处理函数。这个处理函数被调用时，可获取错误信息和 `Vue` 实例

不过值得注意的是，在不同 `Vue` 版本中，该全局 `API` 作用的范围会有所不同：

“
从 2.2.0 起，这个钩子也会捕获组件生命周期钩子里的错误。同样的，当这个钩子是 `undefined` 时，被捕获的错误会通过 `console.error` 输出而避免应用崩

“
从 2.4.0 起，这个钩子也会捕获 `Vue` 自定义事件处理函数内部的错误了

“
从 2.6.0 起，这个钩子也会捕获 `v-on` DOM 监听器内部抛出的错误。另外，如果任何被覆盖的钩子或处理函数返回一个 `Promise` 链（例如 `async` 函数），则来自其 `Promise` 链的错误也会被处理

生命周期钩子

`errorCaptured` 是 2.5.0 新增的一个生命钩子函数，当捕获到一个来自子孙组件的错误时被调用

基本类型

```
(err: Error, vm: Component, info: string) => ?boolean
```

此钩子会收到三个参数：错误对象、发生错误的组件实例以及一个包含错误来源信息的字符串。此钩子可以返回 `false` 以阻止该错误继续向上传播

参考官网，错误传播规则如下：

- 默认情况下，如果全局的 `config.errorHandler` 被定义，所有的错误仍会发送它，因此这些错误仍然会向单一的分析服务的地方进行汇报
- 如果一个组件的继承或父级从属链路中存在多个 `errorCaptured` 钩子，则它们将会被相同的错误逐个唤起。
- 如果此 `errorCaptured` 钩子自身抛出了一个错误，则这个新错误和原本被捕获的错误都会发送给全局的 `config.errorHandler`
- 一个 `errorCaptured` 钩子能够返回 `false` 以阻止错误继续向上传播。本质上是说“这个错误已经被搞定了且应该被忽略”。它会阻止其它任何会被这个错误唤起的 `errorCaptured` 钩子和全局的 `config.errorHandler`

下面来看个例子

定义一个父组件 `cat`

```
Vue.component('cat', {
  template: `
    <div>
      <h1>Cat: </h1>
      <slot></slot>
    </div>`,
  props: {
    name: {
      required: true,
      type: String
    }
  },
  errorCaptured(err, vm, info) {
    console.log(`cat EC: ${err.toString()} \ninfo: ${info}`);
    return false;
  }
});
```

定义一个子组件 `kitten`，其中 `dontexist()` 并没有定义，存在错误

```
Vue.component('kitten', {
  template: `<div><h1>Kitten: {{ dontexist() }}</h1></div>`,
  props: {
    name: {
      required: true,
      type: String
    }
  }
});
```

页面中使用组件

```
<div id="app" v-cloak>
  <cat name="my cat">
    <kitten></kitten>
  </cat>
</div>
```

在父组件的 `errorCaptured` 则能够捕获到信息

```
cat EC: TypeError: dontexist is not a function
info: render
```

三、源码分析

异常处理源码

源码位置： `/src/core/util/error.js`


```
// Vue 全局配置,也就是上面的Vue.config
import config from './config'
import { warn } from './debug'
// 判断环境
import { inBrowser, inWeex } from './env'
// 判断是否是Promise, 通过val.then === 'function' && val.catch === 'function', val ! === null && val
!== undefined
import { isPromise } from 'shared/util'
// 当错误函数处理错误时, 停用deps跟踪以避免可能出现的infinite rendering
// 解决以下出现的问题https://github.com/vuejs/vuex/issues/1505的问题
import { pushTarget, popTarget } from './observer/dep'

export function handleError (err: Error, vm: any, info: string) {
  // Deactivate deps tracking while processing error handler to avoid possible infinite rendering.
  pushTarget()
  try {
    // vm指当前报错的组件实例
    if (vm) {
      let cur = vm
      // 首先获取到报错的组件, 之后递归查找当前组件的父组件, 依次调用errorCaptured 方法。
      // 在遍历调用完所有 errorCaptured 方法、或 errorCaptured 方法有报错时, 调用
      globalHandleError 方法
      while ((cur = cur.$parent)) {
        const hooks = cur.$options.errorCaptured
        // 判断是否存在errorCaptured钩子函数
        if (hooks) {
          // 选项合并的策略, 钩子函数会被保存在一个数组中
          for (let i = 0; i < hooks.length; i++) {
            // 如果errorCaptured 钩子执行自身抛出了错误,
            // 则用try{}catch{}捕获错误, 将这个新错误和原本被捕获的错误都会发送给全局的
            config.errorHandler
            // 调用globalHandleError方法
            try {
              // 当前errorCaptured执行, 根据返回是否是false值
              // 是false, capture = true, 阻止其它任何会被这个错误唤起的 errorCaptured 钩子
              // 和全局的 config.errorHandler
              // 是true capture = false, 组件的继承或父级从属链路中存在的多个 errorCaptured
              // 钩子, 会被相同的错误逐个唤起
              // 调用对应的钩子函数, 处理错误
              const capture = hooks[i].call(cur, err, vm, info) === false
              if (capture) return
            } catch (e) {
              globalHandleError(e, cur, 'errorCaptured hook')
            }
          }
        }
      }
    }
    // 除非禁止错误向上传播, 否则都会调用全局的错误处理函数
    globalHandleError(err, vm, info)
  } finally {
    popTarget()
  }
}
```

```

}
// 异步错误处理函数
export function invokeWithErrorHandling (
  handler: Function,
  context: any,
  args: null | any[],
  vm: any,
  info: string
) {
  let res
  try {
    // 根据参数选择不同的handle执行方式
    res = args ? handler.apply(context, args) : handler.call(context)
    // handle返回结果存在
    // res._isVue an flag to avoid this being observed, 如果传入值的_isVue为ture时(即传入的
    // 值是Vue实例本身)不会新建observer实例
    // isPromise(res) 判断val.then === 'function' && val.catch === 'function', val ! === null &&
    // val !== undefined
    // !res._handled _handle是Promise 实例的内部变量之一, 默认是false, 代表
    // onFulfilled,onRejected是否被处理
    if (res && !res._isVue && isPromise(res) && !res._handled) {
      res.catch(e => handleError(e, vm, info + ` (Promiss/async)`))
      // avoid catch triggering multiple times when nested calls
      // 避免嵌套调用时catch多次的触发
      res._handled = true
    }
  } catch (e) {
    // 处理执行错误
    handleError(e, vm, info)
  }
  return res
}

//全局错误处理
function globalHandleError (err, vm, info) {
  // 获取全局配置, 判断是否设置处理函数, 默认undefined
  // 已配置
  if (config.errorHandler) {
    // try{}catch{} 住全局错误处理函数
    try {
      // 执行设置的全局错误处理函数, handle error 想干啥就干啥
      return config.errorHandler.call(null, err, vm, info)
    } catch (e) {
      // 如果开发者在errorHandler函数中手动抛出同样错误信息throw err
      // 判断err信息是否相等, 避免log两次
      // 如果抛出新的错误信息throw err Error('你好毒'), 将会一起log输出
      if (e !== err) {
        logError(e, null, 'config errorHandler')
      }
    }
  }
}
// 未配置常规log输出
logError(err, vm, info)

```

```
}

// 错误输出函数
function logError (err, vm, info) {
  if (process.env.NODE_ENV !== 'production') {
    warn( 'Error in ${info}: "${err.toString()}"', vm)
  }
  /* istanbul ignore else */
  if ((inBrowser || inWeex) && typeof console !== 'undefined') {
    console.error(err)
  } else {
    throw err
  }
}
}
```

小结

- `handleError` 在需要捕获异常的地方调用，首先获取到报错的组件，之后递归查找当前组件的父组件，依次调用 `errorCaptured` 方法，在遍历调用完所有 `errorCaptured` 方法或 `errorCaptured` 方法有报错时，调用 `globalHandleError` 方法
- `globalHandleError` 调用全局的 `errorHandler` 方法，再通过 `logError` 判断环境输出错误信息
- `invokeWithErrorHandling` 更好的处理异步错误信息
- `logError` 判断环境，选择不同的抛错方式。非生产环境下，调用 `warn` 方法处理错误

参考文献

- <https://juejin.cn/post/6844904096936230925>
- <https://segmentfault.com/a/1190000018606181>

面试官：你有写过自定义指令吗？自定义指令的应用场景有哪些？

一、什么是指令

开始之前我们先学习一下指令系统这个词

「**指令系统**」是计算机硬件的语言系统，也叫机器语言，它是系统程序员看到的计算机的主要属性。因此指令系统表征了计算机的基本功能决定了机器所要求的能力

在 `vue` 中提供了一套为数据驱动视图更为方便的操作，这些操作被称为指令系统

我们看到的 `v-` 开头的行内属性，都是指令，不同的指令可以完成或实现不同的功能

除了核心功能默认内置的指令 (`v-model` 和 `v-show`)，`Vue` 也允许注册自定义指令

指令使用的几种方式：

```
//会实例化一个指令，但这个指令没有参数
`v-xxx`

// -- 将值传到指令中
`v-xxx="value"`

// -- 将字符串传入到指令中，如`v-html="<p>内容</p>"`
`v-xxx="string"`

// -- 传参数（`arg`），如`v-bind:class="className"`
`v-xxx:arg="value"`

// -- 使用修饰符（`modifier`）
`v-xxx:arg.modifier="value"`
```

二、如何实现

注册一个自定义指令有全局注册与局部注册

全局注册主要是通过 `Vue.directive` 方法进行注册

`Vue.directive` 第一个参数是指令的名字（不需要写上 `v-` 前缀），第二个参数可以是对象数据，也可以是一个指令函数

```
// 注册一个全局自定义指令 `v-focus`
Vue.directive('focus', {
  // 当被绑定的元素插入到 DOM 中时.....
  inserted: function (el) {
    // 聚焦元素
    el.focus() // 页面加载完成之后自动让输入框获取到焦点的小功能
  }
})
```

局部注册通过在组件 `options` 选项中设置 `directive` 属性

```
directives: {
  focus: {
    // 指令的定义
    inserted: function (el) {
      el.focus() // 页面加载完成之后自动让输入框获取到焦点的小功能
    }
  }
}
```

然后你可以在模板中任何元素上使用新的 `v-focus` property，如下：

```
<input v-focus />
```

自定义指令也像组件那样存在钩子函数：

- `bind`：只调用一次，指令第一次绑定到元素时调用。在这里可以进行一次性的初始化设置

- `inserted`：被绑定元素插入父节点时调用（仅保证父节点存在，但不一定已被插入文档中）
- `update`：所在组件的 `VNode` 更新时调用，但是可能发生在其子 `VNode` 更新之前。指令的值可能发生了改变，也可能没有。但是你可以通过比较更新前后的值来忽略不必要的模板更新
- `componentUpdated`：指令所在组件的 `VNode` 及其子 `VNode` 全部更新后调用
- `unbind`：只调用一次，指令与元素解绑时调用

所有的钩子函数的参数都有以下：

- `el`：指令所绑定的元素，可以用来直接操作 `DOM`
- `binding`：一个对象，包含以下 `property`：
 - `name`：指令名，不包括 `v-` 前缀。
 - `value`：指令的绑定值，例如：`v-my-directive="1 + 1"` 中，绑定值为 `2`。
 - `oldValue`：指令绑定的前一个值，仅在 `update` 和 `componentUpdated` 钩子中可用。无论值是否改变都可用。
 - `expression`：字符串形式的指令表达式。例如 `v-my-directive="1 + 1"` 中，表达式为 `"1 + 1"`。
 - `arg`：传给指令的参数，可选。例如 `v-my-directive:foo` 中，参数为 `"foo"`。
 - `modifiers`：一个包含修饰符的对象。例如：`v-my-directive.foo.bar` 中，修饰符对象为 `{ foo: true, bar: true }`
- `vnode`：Vue 编译生成的虚拟节点
- `oldVnode`：上一个虚拟节点，仅在 `update` 和 `componentUpdated` 钩子中可用

除了 `el` 之外，其它参数都应该是只读的，切勿进行修改。如果需要在钩子之间共享数据，建议通过元素的 `dataset` 来进行

举个例子：

```
<div v-demo="{ color: 'white', text: 'hello!' }"></div>
<script>
  Vue.directive('demo', function (el, binding) {
    console.log(binding.value.color) // "white"
    console.log(binding.value.text) // "hello!"
  })
</script>
```

三、应用场景

使用自定义指令可以满足我们日常一些场景，这里给出几个自定义指令的案例：

- 表单防止重复提交
- 图片懒加载
- 一键 Copy 的功能

表单防止重复提交

表单防止重复提交这种情况设置一个 `v-throttle` 自定义指令来实现

举个例子：

```
// 1.设置v-throttle自定义指令
Vue.directive('throttle', {
  bind: (el, binding) => {
    let throttleTime = binding.value; // 节流时间
    if (!throttleTime) { // 用户若不设置节流时间，则默认2s
      throttleTime = 2000;
    }
    let cbFun;
    el.addEventListener('click', event => {
      if (!cbFun) { // 第一次执行
        cbFun = setTimeout(() => {
          cbFun = null;
        }, throttleTime);
      } else {
        event && event.stopImmediatePropagation();
      }
    }, true);
  },
});
// 2.为button标签设置v-throttle自定义指令
<button @click="sayHello" v-throttle>提交</button>
```

图片懒加载

设置一个 `v-lazy` 自定义指令完成图片懒加载

```
const LazyLoad = {
  // install方法
  install(Vue, options) {
    // 代替图片的loading图
    let defaultSrc = options.default;
    Vue.directive('lazy', {
      bind(el, binding) {
        LazyLoad.init(el, binding.value, defaultSrc);
      },
      inserted(el) {
        // 兼容处理
        if ('IntersectionObserver' in window) {
          LazyLoad.observe(el);
        } else {
          LazyLoad.listenerScroll(el);
        }
      }
    });
  },
  // 初始化
  init(el, val, def) {
    // data-src 储存真实src
    el.setAttribute('data-src', val);
  }
}
```



```
// 设置src为loading图
el.setAttribute('src',def);
},
// 利用IntersectionObserver监听el
observe(el){
  let io = new IntersectionObserver(entries => {
    let realSrc = el.dataset.src;
    if(entries[0].isIntersecting){
      if(realSrc){
        el.src = realSrc;
        el.removeAttribute('data-src');
      }
    }
  });
  io.observe(el);
},
// 监听scroll事件
listenerScroll(el){
  let handler = LazyLoad.throttle(LazyLoad.load,300);
  LazyLoad.load(el);
  window.addEventListener('scroll',() => {
    handler(el);
  });
},
// 加载真实图片
load(el){
  let windowHeight = document.documentElement.clientHeight
  let elTop = el.getBoundingClientRect().top;
  let elBtm = el.getBoundingClientRect().bottom;
  let realSrc = el.dataset.src;
  if(elTop - windowHeight < 0 && elBtm > 0){
    if(realSrc){
      el.src = realSrc;
      el.removeAttribute('data-src');
    }
  }
},
// 节流
throttle(fn,delay){
  let timer;
  let prevTime;
  return function(...args){
    let currTime = Date.now();
    let context = this;
    if(!prevTime) prevTime = currTime;
    clearTimeout(timer);

    if(currTime - prevTime > delay){
      prevTime = currTime;
      fn.apply(context,args);
      clearTimeout(timer);
      return;
    }
  }
}
```

```
timer = setTimeout(function(){  
  prevTime = Date.now();  
  timer = null;  
  fn.apply(context,args);  
},delay);  
}  
}  
  
}  
export default LazyLoad;
```

一键 Copy 的功能

阿里巴巴

```
import { Message } from 'ant-design-vue';

const vCopy = { //
  /*
    bind 钩子函数，第一次绑定时调用，可以在这里做初始化设置
    el: 作用的 dom 对象
    value: 传给指令的值，也就是我们要 copy 的值
  */
  bind(el, { value }) {
    el.$value = value; // 用一个全局属性来存传进来的值，因为这个值在别的钩子函数里还会用到
    el.handler = () => {
      if (!el.$value) {
        // 值为空的时候，给出提示，我这里的提示是用的 ant-design-vue 的提示，你们随意
        Message.warning('无复制内容');
        return;
      }
      // 动态创建 textarea 标签
      const textarea = document.createElement('textarea');
      // 将该 textarea 设为 readonly 防止 iOS 下自动唤起键盘，同时将 textarea 移出可视区域
      textarea.readOnly = 'readonly';
      textarea.style.position = 'absolute';
      textarea.style.left = '-9999px';
      // 将要 copy 的值赋给 textarea 标签的 value 属性
      textarea.value = el.$value;
      // 将 textarea 插入到 body 中
      document.body.appendChild(textarea);
      // 选中值并复制
      textarea.select();
      // textarea.setSelectionRange(0, textarea.value.length);
      const result = document.execCommand('Copy');
      if (result) {
        Message.success('复制成功');
      }
      document.body.removeChild(textarea);
    };
    // 绑定点击事件，就是所谓的一键 copy 啦
    el.addEventListener('click', el.handler);
  },
  // 当传进来的值更新的时候触发
  componentUpdated(el, { value }) {
    el.$value = value;
  },
  // 指令与元素解绑的时候，移除事件绑定
  unbind(el) {
    el.removeEventListener('click', el.handler);
  },
};

export default vCopy;
```

关于自定义指令还有很多应用场景，如：拖拽指令、页面水印、权限校验等等应用场景

参考文献

- <https://juejin.cn/post/6844904197448531975#heading-5>
- https://www.cnblogs.com/chenwenhao/p/11924161.html#_label2

面试官：你知道vue中key的原理吗？说说你对它的理解

一、Key是什么

开始之前，我们先还原两个实际工作场景

1. 当我们在使用 `v-for` 时，需要给单元加上 `key`

```
<ul>
  <li v-for="item in items" :key="item.id">...</li>
</ul>
```

2. 用 `+new Date()` 生成的时间戳作为 `key`，手动强制触发重新渲染

```
<Comp :key="+new Date()" />
```

那么这背后的逻辑是什么，`key` 的作用又是什么？

一句话来讲

“

key是给每一个vnode的唯一id，也是diff的一种优化策略，可以根据key，更准确，更快的找到对应的vnode节点

”

场景背后的逻辑

当我们在使用 `v-for` 时，需要给单元加上 `key`

- 如果不用key，Vue会采用就地复地原则：最小化element的移动，并且会尝试尽最大程度在同适当的地方对相同类型的element，做patch或者reuse。
- 如果使用了key，Vue会根据keys的顺序记录element，曾经拥有了key的element如果不再出现的话，会被直接remove或者destoryed

用 `+new Date()` 生成的时间戳作为 `key`，手动强制触发重新渲染

- 当拥有新值的rerender作为key时，拥有了新key的Comp出现了，那么旧key Comp会被移除，新key Comp触发渲染

二、设置key与不设置key区别

举个例子：

创建一个实例，2秒后往 `items` 数组插入数据

```
<body>
  <div id="demo">
    <p v-for="item in items" :key="item">{{item}}</p>
  </div>
  <script src="../../dist/vue.js"></script>
  <script>
    // 创建实例
    const app = new Vue({
      el: '#demo',
      data: { items: ['a', 'b', 'c', 'd', 'e'] },
      mounted () {
        setTimeout(() => {
          this.items.splice(2, 0, 'f') //
        }, 2000);
      },
    });
  </script>
</body>
```

在不使用 `key` 的情况，`vue` 会进行这样的操作：

分析下整体流程：

- 比较A，A，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 比较B，B，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 比较C，F，相同类型的节点，进行 `patch`，数据不同，发生 `dom` 操作
- 比较D，C，相同类型的节点，进行 `patch`，数据不同，发生 `dom` 操作
- 比较E，D，相同类型的节点，进行 `patch`，数据不同，发生 `dom` 操作
- 循环结束，将E插入到 `DOM` 中

一共发生了3次更新，1次插入操作

在使用 `key` 的情况：`vue` 会进行这样的操作：

- 比较A，A，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 比较B，B，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 比较C，F，不相同类型的节点
 - 比较E、E，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 比较D、D，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 比较C、C，相同类型的节点，进行 `patch`，但数据相同，不发生 `dom` 操作
- 循环结束，将F插入到C之前

一共发生了0次更新，1次插入操作

通过上面两个小例子，可见设置 `key` 能够大大减少对页面的 `DOM` 操作，提高了 `diff` 效率

设置key值一定能提高diff效率吗？

其实不然，文档中也明确表示

“

当 Vue.js 用 v-for 正在更新已渲染过的元素列表时，它默认用“就地复用”策略。如果数据项的顺序被改变，Vue 将不会移动 DOM 元素来匹配数据项的顺序，而是简单复用此处每个元素，并且确保它在特定索引下显示已被渲染过的每个元素

”

这个默认的模式是高效的，但是只适用于不依赖子组件状态或临时 DOM 状态（例如：表单输入值）的列表渲染输出

建议尽可能在使用 v-for 时提供 key，除非遍历输出的 DOM 内容非常简单，或者是刻意依赖默认行为以获取性能上的提升

三、原理分析

源码位置：core/vdom/patch.js

这里判断是否为同一个 key，首先判断的是 key 值是否相等如果没有设置 key，那么 key 为 undefined，这时候 undefined 是恒等于 undefined

```
function sameVnode (a, b) {
  return (
    a.key === b.key && (
      (
        a.tag === b.tag &&
        a.isComment === b.isComment &&
        isDef(a.data) === isDef(b.data) &&
        sameInputType(a, b)
      ) || (
        isTrue(a.isAsyncPlaceholder) &&
        a.asyncFactory === b.asyncFactory &&
        isUndef(b.asyncFactory.error)
      )
    )
  )
}
```

updateChildren 方法中会对新旧 vnode 进行 diff，然后将比对出的结果用来更新真实的 DOM


```

function updateChildren (parentElm, oldCh, newCh, insertedVnodeQueue, removeOnly) {
  ...
  while (oldStartIdx <= oldEndIdx && newStartIdx <= newEndIdx) {
    if (isUndef(oldStartVnode)) {
      ...
    } else if (isUndef(oldEndVnode)) {
      ...
    } else if (sameVnode(oldStartVnode, newStartVnode)) {
      ...
    } else if (sameVnode(oldEndVnode, newEndVnode)) {
      ...
    } else if (sameVnode(oldStartVnode, newEndVnode)) { // Vnode moved right
      ...
    } else if (sameVnode(oldEndVnode, newStartVnode)) { // Vnode moved left
      ...
    } else {
      if (isUndef(oldKeyToIdx)) oldKeyToIdx = createKeyToOldIdx(oldCh, oldStartIdx, oldEndIdx)
      idxInOld = isDef(newStartVnode.key)
        ? oldKeyToIdx[newStartVnode.key]
        : findIdxInOld(newStartVnode, oldCh, oldStartIdx, oldEndIdx)
      if (isUndef(idxInOld)) { // New element
        createElm(newStartVnode, insertedVnodeQueue, parentElm, oldStartVnode.elm, false,
newCh, newStartIdx)
      } else {
        vnodeToMove = oldCh[idxInOld]
        if (sameVnode(vnodeToMove, newStartVnode)) {
          patchVnode(vnodeToMove, newStartVnode, insertedVnodeQueue, newCh,
newStartIdx)
          oldCh[idxInOld] = undefined
          canMove && nodeOps.insertBefore(parentElm, vnodeToMove.elm, oldStartVnode.elm)
        } else {
          // same key but different element. treat as new element
          createElm(newStartVnode, insertedVnodeQueue, parentElm, oldStartVnode.elm, false,
newCh, newStartIdx)
        }
      }
      newStartVnode = newCh[++newStartIdx]
    }
  }
  ...
}

```

参考文献

- <https://juejin.cn/post/6844903826693029895>
- <https://juejin.cn/post/6844903985397104648>

面试官：动态给vue的data添加一个新的属性时会发生什么？怎样解决？

一、直接添加属性的问题

我们从一个例子开始

定义一个 `p` 标签，通过 `v-for` 指令进行遍历

然后给 `button` 标签绑定点击事件，我们预期点击按钮时，数据新增一个属性，界面也 新增一行

```
<p v-for="(value,key) in item" :key="key">
  {{ value }}
</p>
<button @click="addProperty">动态添加新属性</button>
```

实例化一个 `vue` 实例，定义 `data` 属性和 `methods` 方法

```
const app = new Vue({
  el:"#app",
  data:()=>{
    item:{
      oldProperty:"旧属性"
    }
  },
  methods:{
    addProperty(){
      this.items.newProperty = "新属性" // 为items添加新属性
      console.log(this.items) // 输出带有newProperty的items
    }
  }
})
```

点击按钮，发现结果不及预期，数据虽然更新了（`console` 打印出了新属性），但页面并没有更新

二、原理分析

为什么产生上面的情况呢？

下面来分析一下

`vue2` 是用过 `Object.defineProperty` 实现数据响应式

```
const obj = {}
Object.defineProperty(obj, 'foo', {
  get() {
    console.log(`get foo:${val}`);
    return val
  },
  set(newVal) {
    if (newVal !== val) {
      console.log(`set foo:${newVal}`);
      val = newVal
    }
  }
})
}
```

当我们访问 `foo` 属性或者设置 `foo` 值的时候都能够触发 `setter` 与 `getter`

```
obj.foo
obj.foo = 'new'
```

但是我们为 `obj` 添加新属性时，却无法触发事件属性的拦截

```
obj.bar = '新属性'
```

原因是一开始 `obj` 的 `foo` 属性被设成了响应式数据，而 `bar` 是后面新增的属性，并没有通过 `Object.defineProperty` 设置成响应式数据

三、解决方案

`Vue` 不允许在已经创建的实例上动态添加新的响应式属性

若想实现数据与视图同步更新，可采取下面三种解决方案：

- `Vue.set()`
- `Object.assign()`
- `$forceUpdated()`

Vue.set()

`Vue.set( target, propertyName/index, value )`

参数

- `{Object | Array} target`
- `{string | number} propertyName/index`
- `{any} value`

返回值：设置的值

通过 `Vue.set` 向响应式对象中添加一个 `property`，并确保这个新 `property` 同样是响应式的，且触发视图更新

关于 `Vue.set` 源码（省略了很多与本节不相关的代码）

源码位置： [src\core\observer\index.js](#)

```
function set (target: Array<any> | Object, key: any, val: any): any {  
  ...  
  defineReactive(ob.value, key, val)  
  ob.dep.notify()  
  return val  
}
```

这里无非再次调用 [defineReactive](#) 方法，实现新增属性的响应式

关于 [defineReactive](#) 方法，内部还是通过 [Object.defineProperty](#) 实现属性拦截

大致代码如下：

```
function defineReactive(obj, key, val) {  
  Object.defineProperty(obj, key, {  
    get() {  
      console.log(`get ${key}:${val}`);  
      return val  
    },  
    set(newVal) {  
      if (newVal !== val) {  
        console.log(`set ${key}:${newVal}`);  
        val = newVal  
      }  
    }  
  })  
}
```

Object.assign()

直接使用 [Object.assign\(\)](#) 添加到对象的新属性不会触发更新

应创建一个新的对象，合并原对象和混入对象的属性

```
this.someObject = Object.assign({}, this.someObject, {newProperty1:1, newProperty2:2 ...})
```

\$forceUpdate

如果你发现你自己需要在 [Vue](#) 中做一次强制更新，99.9% 的情况，是你在某个地方做错了事

[\\$forceUpdate](#) 迫使 [Vue](#) 实例重新渲染

PS：仅仅影响实例本身和插入插槽内容的子组件，而不是所有子组件。

小结

- 如果为对象添加少量的新属性，可以直接采用 [Vue.set\(\)](#)
- 如果需要为新对象添加大量的新属性，则通过 [Object.assign\(\)](#) 创建新对象
- 如果你实在不知道怎么操作时，可采取 [\\$forceUpdate\(\)](#) 进行强制刷新（不建议）

PS: `vue3` 是用过 `proxy` 实现数据响应式的, 直接动态添加新属性仍可以实现数据响应式

参考文献

- <https://cn.vuejs.org/v2/api/#Vue-set>

面试官：双向数据绑定是什么

一、什么是双向绑定

我们先从单向绑定切入单向绑定非常简单, 就是把 `Model` 绑定到 `View`, 当我们用 `JavaScript` 代码更新 `Model` 时, `View` 就会自动更新双向绑定就很容易联想到了, 在单向绑定的基础上, 用户更新了 `View`, `Model` 的数据也自动被更新了, 这种情况就是双向绑定举个栗子

当用户填写表单时, `View` 的状态就被更新了, 如果此时可以自动更新 `Model` 的状态, 那就相当于我们把 `Model` 和 `View` 做

二、双向绑定的原理是什么

我们都知道 `Vue` 是数据双向绑定的框架, 双向绑定由三个重要部分构成

- 数据层 (`Model`): 应用的数据及业务逻辑
- 视图层 (`View`): 应用的展示效果, 各类UI组件
- 业务逻辑层 (`ViewModel`): 框架封装的核心, 它负责将数据与视图关联起来

而上面的这个分层的架构方案, 可以用一个专业术语进行称呼: `MVVM` 这里的控制层的核心功能便是“数据双向绑定”。自然, 我们只需弄懂它是什么, 便可以进一步了解数据绑定的原理

理解ViewModel

它的主要职责就是:

- 数据变化后更新视图
- 视图变化后更新数据

当然, 它还有两个主要部分组成

- 监听器 (`Observer`): 对所有数据的属性进行监听
- 解析器 (`Compiler`): 对每个元素节点的指令进行扫描跟解析, 根据指令模板替换数据, 以及绑定相应的更新函数

三、实现双向绑定

我们还是以 `Vue` 为例, 先来看看 `Vue` 中的双向绑定流程是什么的

1. `new Vue()` 首先执行初始化, 对 `data` 执行响应化处理, 这个过程发生 `Observe` 中
2. 同时对模板执行编译, 找到其中动态绑定的数据, 从 `data` 中获取并初始化视图, 这个过程发生在 `Compile` 中
3. 同时定义一个更新函数和 `Watcher`, 将来对应数据变化时 `Watcher` 会调用更新函数
4. 由于 `data` 的某个 `key` 在一个视图中可能出现多次, 所以每个 `key` 都需要一个管家 `Dep` 来管理多个 `Watcher`
5. 将来 `data` 中数据一旦发生变化, 会首先找到对应的 `Dep`, 通知所有 `Watcher` 执行更新函数

流程图如下：

实现

先来一个构造函数：执行初始化，对 `data` 执行响应化处理

```
class Vue {
  constructor(options) {
    this.$options = options;
    this.$data = options.data;

    // 对data选项做响应式处理
    observe(this.$data);

    // 代理data到vm上
    proxy(this);

    // 执行编译
    new Compile(options.el, this);
  }
}
```

对 `data` 选项执行响应化具体操作

```
function observe(obj) {
  if (typeof obj !== "object" || obj == null) {
    return;
  }
  new Observer(obj);
}

class Observer {
  constructor(value) {
    this.value = value;
    this.walk(value);
  }
  walk(obj) {
    Object.keys(obj).forEach((key) => {
      defineReactive(obj, key, obj[key]);
    });
  }
}
```

编译Compile

对每个元素节点的指令进行扫描跟解析,根据指令模板替换数据,以及绑定相应的更新函数


```

class Compile {
  constructor(el, vm) {
    this.$vm = vm;
    this.$el = document.querySelector(el); // 获取dom
    if (this.$el) {
      this.compile(this.$el);
    }
  }
  compile(el) {
    const childNodes = el.childNodes;
    Array.from(childNodes).forEach((node) => { // 遍历子元素
      if (this.isElement(node)) { // 判断是否为节点
        console.log("编译元素" + node.nodeName);
      } else if (this.isInterpolation(node)) {
        console.log("编译插值文本" + node.textContent); // 判断是否为插值文本 {{}}
      }
      if (node.childNodes && node.childNodes.length > 0) { // 判断是否有子元素
        this.compile(node); // 对子元素进行递归遍历
      }
    });
  }
  isElement(node) {
    return node.nodeType == 1;
  }
  isInterpolation(node) {
    return node.nodeType == 3 && /\{\}/.test(node.textContent);
  }
}

```

依赖收集

视图中会用到 `data` 中某 `key`，这称为依赖。同一个 `key` 可能出现多次，每次都需要收集出来用一个 `Watcher` 来维护它们，此过程称为依赖收集。多个 `Watcher` 需要一个 `Dep` 来管理，需要更新时由 `Dep` 统一通知。

实现思路

1. `defineReactive` 时为每一个 `key` 创建一个 `Dep` 实例
2. 初始化视图时读取某个 `key`，例如 `name1`，创建一个 `watcher1`
3. 由于触发 `name1` 的 `getter` 方法，便将 `watcher1` 添加到 `name1` 对应的 `Dep` 中
4. 当 `name1` 更新，`setter` 触发时，便可通过对应 `Dep` 通知其管理所有 `Watcher` 更新

```

// 负责更新视图
class Watcher {
  constructor(vm, key, updater) {
    this.vm = vm
    this.key = key
    this.updaterFn = updater

    // 创建实例时，把当前实例指定到Dep.target静态属性上
    Dep.target = this
    // 读一下key，触发get
    vm[key]
    // 置空
    Dep.target = null
  }

  // 未来执行dom更新函数，由dep调用的
  update() {
    this.updaterFn.call(this.vm, this.vm[this.key])
  }
}

```

声明 `Dep`

```

class Dep {
  constructor() {
    this.deps = []; // 依赖管理
  }
  addDep(dep) {
    this.deps.push(dep);
  }
  notify() {
    this.deps.forEach((dep) => dep.update());
  }
}

```

创建 `watcher` 时触发 `getter`

```

class Watcher {
  constructor(vm, key, updateFn) {
    Dep.target = this;
    this.vm[this.key];
    Dep.target = null;
  }
}

```

依赖收集，创建 `Dep` 实例

```
function defineReactive(obj, key, val) {
  this.observe(val);
  const dep = new Dep();
  Object.defineProperty(obj, key, {
    get() {
      Dep.target && dep.addDep(Dep.target); // Dep.target也就是Watcher实例
      return val;
    },
    set(newVal) {
      if (newVal === val) return;
      dep.notify(); // 通知dep执行更新方法
    },
  });
}
```

参考文献

- <https://www.liaoxuefeng.com/wiki/1022910821149312/1109527162256416>
- <https://juejin.cn/post/6844903942254510087#heading-9>

面试官：有使用过vue吗？说说你对vue的理解

一、从历史说起

Web是World Wide Web的简称，中文译为万维网我们可以将它规划成如下的几个时代来进行理解

- 石器时代
- 文明时代
- 工业革命时代
- 百花齐放时代

石器时代

石器时代指的就是我们的静态网页，可以欣赏一下1997的Apple官网

最早的网页是没有数据库的，可以理解成就是一张可以在网络上浏览的报纸，直到CGI技术的出现通过CGI Perl 运行一小段代码与数据库或文件系统进行交互，如当时的Google（1998年）

文明时代

ASP，JSP大家应该都不会太陌生，最早出现于 2005 年左右，先后出现了微软的 ASP 和 Java Server Pages [JSP] 等技术,取代了 CGI ，增强了 WEB 与服务端的交互的安全性，类似于下面这样，其实就是Java + HTML

```
<%@ page language="java" contentType="text/html; charset=utf-8"
pageEncoding="utf-8"%> <!DOCTYPE html> <html> <head> <meta charset="utf-8">
<title>JSP demo</title> </head> <body>  </body>
</html>
```

JSP有一个很大的缺点，就是不太灵活，因为JSP是在服务器端执行的，通常返回该客户端的就是一个HTML文本。我们每次的请求：获取的数据、内容的加载，都是服务器为我们返回渲染完成之后的DOM，这也就使得我们开发网站的灵活度大打折扣在这种情况下，同年：Ajax火了(小细节，这里为什么说火了，因为 Ajax 技术并不是 2005 年出现的，他的雏形是 1999 年)，现在看来很常见的技术手段，在当时可是珍贵无比

工业革命时代

到这里大家就更熟悉了，移动设备的普及，Jquery的出现，以及SPA (Single Page Application 单页面应用) 的雏形，Backbone EmberJS AngularJS 这样一批前端框架随之出现，但当时SPA的路不好走，例如SEO问题，SPA 过多的页面、复杂场景下 View 的绑定等，都没有很好的处理经过这几年的飞速发展，节约了开发人员大量的精力、降低了开发者和开发过程的门槛，极大提升了开发效率和迭代速度，我们可以称之其为工业时代

百花齐放时代

这里没有文字，放一张图感受一下

PS：这里为什么要说这么多Web的历史，我们可以看到Web技术的变化之大与快，每一种新的技术出现都是一些特定场景的解决方案，那我们今天的主角Vue又是为了解决什么呢？我们接着往下看

二、vue是什么

Vue.js (/vjuːz/, 或简称为Vue) 是一个用于创建用户界面的开源JavaScript框架，也是一个创建单页应用的Web应用框架。2016年一项针对JavaScript的调查表明，Vue有着89%的开发者的满意度。在GitHub上，该项目平均每天能收获95颗星，为Github有史以来星标数第3多的项目同时也是一款流行的JavaScript前端框架，旨在更好地组织与简化Web开发。Vue所关注的核心是MVC模式中的视图层，同时，它也能方便地获取数据更新，并通过组件内部特定的方法实现视图与模型的交互PS: Vue作者尤雨溪是在为AngularJS工作之后开发出了这一框架。他声称自己的思路是提取Angular中为自己所喜欢的部分，构建出一款相当轻量的框架最早发布于2014年2月

三、Vue核心特性

数据驱动 (MVVM)

MVVM 表示的是 **Model-View-ViewModel**

- Model：模型层，负责处理业务逻辑以及和服务端进行交互
- View：视图层：负责将数据模型转化为UI展示出来，可以简单的理解为HTML页面
- ViewModel：视图模型层，用来连接Model和View，是Model和View之间的通信桥梁

这时候需要一张直观的关系图，如下

组件化

1.什么是组件化一句话来说就是把图形、非图形的各种逻辑均抽象为一个统一的概念（组件）来实现开发的模式，在 **Vue** 中每一个 **.vue** 文件都可以视为一个组件2.组件化的优势

- 降低整个系统的耦合度，在保持接口不变的情况下，我们可以替换不同的组件快速完成需求，例如输入框，可以替换为日历、时间、范围等组件作具体的实现
- 调试方便，由于整个系统是通过组件组合起来的，在出现问题的时候，可以用排除法直接移除组件，或者根据报错的组件快速定位问题，之所以能够快速定位，是因为每个组件之间低耦合，职责单一，所以逻辑会比分析整个系统要简单
- 提高可维护性，由于每个组件的职责单一，并且组件在系统中是被复用的，所以对代码进行优化可获得系统的整体升级

指令系统

解释：指令 (Directives) 是带有 v- 前缀的特殊属性作用：当表达式的值改变时，将其产生的连带影响，响应式地作用于 DOM

- 常用的指令
 - 条件渲染指令 `v-if`
 - 列表渲染指令 `v-for`
 - 属性绑定指令 `v-bind`
 - 事件绑定指令 `v-on`
 - 双向数据绑定指令 `v-model`

没有指令之前我们是怎么做的？是不是先要获取到DOM然后在....干点啥

四、Vue跟传统开发的区别

没有落地使用场景的革命不是好革命，就以一个高频的应用场景来示意吧注册账号这个需求大家应该很熟悉了，如下

用 `jquery` 来实现大概的思路就是选择流程dom对象，点击按钮隐藏当前活动流程dom对象，显示下一流程dom对象如下图(代码就不上了，上了就篇文章就没了..)

用 `vue` 来实现，我们知道 `vue` 基本不操作 `dom` 节点，双向绑定使 `dom` 节点跟视图绑定后，通过修改变量的值控制 `dom` 节点的各类属性。所以其实现思路为：视图层使用一变量控制dom节点显示与否，点击按钮则改变该变量，如下图

总结就是：

- Vue所有的界面事件，都是只去操作数据的，Jquery操作DOM
- Vue所有界面的变动，都是根据数据自动绑定出来的，Jquery操作DOM

五、Vue和React对比

这里就做几个简单的类比吧，当然没有好坏之分，只是使用场景不同

相同点

- 都有组件化思想
- 都支持服务器端渲染
- 都有Virtual DOM (虚拟dom)
- 数据驱动视图
- 都有支持native的方案：`Vue`的`weex`、`React`的`React native`
- 都有自己的构建工具：`Vue`的`vue-cli`、`React`的`Create React App`

区别

- 数据流向的不同。`react`从诞生开始就推崇单向数据流，而`Vue`是双向数据流
- 数据变化的实现原理不同。`react`使用的是不可变数据，而`Vue`使用的是可变的数据
- 组件化通信的不同。`react`中我们通过使用回调函数来进行通信的，而`Vue`中子组件向父组件传递消息有两种方式：事件和回调函数
- diff算法不同。`react`主要使用diff队列保存需要更新哪些DOM，得到patch树，再统一操作批量更新DOM。`Vue`使用双向指针，边对比，边更新DOM

参考文献

- <https://segmentfault.com/a/1190000016269636>
- <https://zh.wikipedia.org/zh-cn/Vue.js>
- <https://zhuanlan.zhihu.com/p/20197803>
- <https://zhuanlan.zhihu.com/p/38296857>

面试官：说下你的vue项目的目录结构，如果是大型项目你该怎么划分结构和划分组件呢？

一、为什么要划分

使用 `vue` 构建项目，项目结构清晰会提高开发效率，熟悉项目的各种配置同样会让开发效率更高

在划分项目结构的时候，需要遵循一些基本的原则：

- 文件夹和文件夹内部文件的语义一致性
- 单一入口/出口
- 就近原则，紧耦合的文件应该放到一起，且应以相对路径引用
- 公共的文件应该以绝对路径的方式从根目录引用
- `/src` 外的文件不应该被引入

文件夹和文件夹内部文件的语义一致性

我们的目录结构都会有一个文件夹是按照路由模块来划分的，如 `pages` 文件夹，这个文件夹里面应该包含我们项目所有的路由模块，并且仅应该包含路由模块，而不应该有别的其他的非路由模块的文件夹

这样做的好处在于一眼就从 `pages` 文件夹看出这个项目的路由有哪些

单一入口/出口

举个例子，在 `pages` 文件夹里面存在一个 `seller` 文件夹，这时候 `seller` 文件夹应该作为一个独立的模块由外部引入，并且 `seller/index.js` 应该作为外部引入 `seller` 模块的唯一入口

// 错误用法

```
import sellerReducer from 'src/pages/seller/reducer'
```

// 正确用法

```
import { reducer as sellerReducer } from 'src/pages/seller'
```

这样做的好处在于，无论你的模块文件夹内部有多乱，外部引用的时候，都是从一个入口文件引入，这样就很好的实现了隔离，如果后续有重构需求，你就会发现这种方式的优点

就近原则，紧耦合的文件应该放到一起，且应以相对路径引用

使用相对路径可以保证模块内部的独立性

// 正确用法

```
import styles from './index.module.scss'
```

// 错误用法

```
import styles from 'src/pages/seller/index.module.scss'
```


举个例子

假设我们现在的 `seller` 目录是在 `src/pages/seller`，如果我们后续发生了路由变更，需要加一个层级，变成 `src/pages/user/seller`。

如果我们采用第一种相对路径的方式，那就可以直接将整个文件夹拖过去就好，`seller` 文件夹内部不需要做任何变更。

但是如果采用第二种绝对路径的方式，移动文件夹的同时，还需要对每个 `import` 的路径做修改

公共的文件应该以绝对路径的方式从根目录引用

公共指的是多个路由模块共用，如一些公共的组件，我们可以放在 `src/components` 下
在使用到的页面中，采用绝对路径的形式引用

```
// 错误用法
import Input from '../components/input'
// 正确用法
import Input from 'src/components/input'
```

同样的，如果我们需要对文件夹结构进行调整。将 `/src/components/input` 变成 `/src/components/new/input`，如果使用绝对路径，只需要全局搜索替换
再加上绝对路径有全局的语义，相对路径有独立模块的语义

/src 外的文件不应该被引入

`vue-cli` 脚手架已经帮我们做了相关的约束了，正常我们的前端项目都会有个 `src` 文件夹，里面放着所有的项目需要的资源，`js`，`css`，`png`，`svg` 等等。`src` 外会放一些项目配置，依赖，环境等文件

这样的好处是方便划分项目代码文件和配置文件

二、目录结构

单页面目录结构

```
project
|
| .browserslistrc
| .env.production
| .eslintrc.js
| .gitignore
| babel.config.js
| package-lock.json
| package.json
| README.md
| vue.config.js
| yarn-error.log
| yarn.lock
|
|—public
|   favicon.ico
|   index.html
|
|-- src
|   |-- components
|   |   |-- input
|   |   |   |-- index.js
|   |   |   |-- index.module.scss
|   |-- pages
|   |   |-- seller
|   |   |   |-- components
|   |   |   |   |-- input
|   |   |   |   |   |-- index.js
|   |   |   |   |   |-- index.module.scss
|   |   |-- reducer.js
|   |   |-- saga.js
|   |   |-- index.js
|   |   |-- index.module.scss
|   |-- buyer
|   |   |-- index.js
|   |-- index.js
```

多页面目录结构

```
my-vue-test:
|
| .browserslistrc
| .env.production
| .eslintrc.js
| .gitignore
| babel.config.js
| package-lock.json
| package.json
| README.md
| vue.config.js
| yarn-error.log
| yarn.lock
|
|—public
```

favicon.ico

index.html

src

apis //接口文件根据页面或实例模块化

index.js

login.js

components //全局公共组件

header

index.less

index.vue

config //配置 (环境变量配置不同passid等)

env.js

index.js

contant //常量

index.js

images //图片

logo.png

pages //多页面vue项目, 不同的实例

index //主实例

index.js

index.vue

main.js

router.js

store.js

components //业务组件

pages //此实例中的各个路由

amenu

index.vue

bmenu

index.vue

login //另一个实例

index.js

index.vue

main.js

scripts //包含各种常用配置, 工具函数

map.js

utils

helper.js

store //vuex仓库

index.js

```
├── index
│   ├── actions.js
│   ├── getters.js
│   ├── index.js
│   ├── mutation-types.js
│   ├── mutations.js
│   └── state.js
├── user
│   ├── actions.js
│   ├── getters.js
│   ├── index.js
│   ├── mutation-types.js
│   ├── mutations.js
│   └── state.js
├── styles //样式统一配置
│   └── components.less
├── animation
│   ├── index.less
│   └── slide.less
├── base
│   ├── index.less
│   ├── style.less
│   ├── var.less
│   └── widget.less
└── common
    ├── index.less
    ├── reset.less
    ├── style.less
    └── transition.less
```

小结

项目的目录结构很重要，因为目录结构能体现很多东西，怎么规划目录结构可能每个人有自己的理解，但是按照一定的规范去进行目录的设计，能让项目整个架构看起来更为简洁，更加易用

参考文献

- <https://juejin.cn/post/6844904129186234381#heading-0>
- <https://zhuanlan.zhihu.com/p/89693668>

面试官：说说你对keep-alive的理解是什么？

一、Keep-alive 是什么

`keep-alive` 是 `vue` 中的内置组件，能在组件切换过程中将状态保留在内存中，防止重复渲染 `DOM`

`keep-alive` 包裹动态组件时，会缓存不活动的组件实例，而不是销毁它们

`keep-alive` 可以设置以下 `props` 属性：

- `include` - 字符串或正则表达式。只有名称匹配的组件会被缓存
- `exclude` - 字符串或正则表达式。任何名称匹配的组件都不会被缓存
- `max` - 数字。最多可以缓存多少组件实例

关于 `keep-alive` 的基本用法：

```
<keep-alive>
  <component :is="view"></component>
</keep-alive>
```

使用 `includes` 和 `exclude`：

```
<keep-alive include="a,b">
  <component :is="view"></component>
</keep-alive>

<!-- 正则表达式 (使用 `v-bind`) -->
<keep-alive :include="/alb/">
  <component :is="view"></component>
</keep-alive>

<!-- 数组 (使用 `v-bind`) -->
<keep-alive :include="['a', 'b']">
  <component :is="view"></component>
</keep-alive>
```

匹配首先检查组件自身的 `name` 选项，如果 `name` 选项不可用，则匹配它的局部注册名称（父组件 `components` 选项的键值），匿名组件不能被匹配

设置了 `keep-alive` 缓存的组件，会多出两个生命周期钩子（`activated` 与 `deactivated`）：

- 首次进入组件时：`beforeRouteEnter` > `beforeCreate` > `created` > `mounted` > `activated` > ... > `beforeRouteLeave` > `deactivated`
- 再次进入组件时：`beforeRouteEnter` > `activated` > ... > `beforeRouteLeave` > `deactivated`

二、使用场景

使用原则：当我们在某些场景下不需要让页面重新加载时我们可以使用 `keepalive`

举个栗子：

当我们从 `首页` -> `列表页` -> `商详页` -> `再返回`，这时候列表页应该是需要 `keep-alive`

从 `首页` -> `列表页` -> `商详页` -> `返回到列表页(需要缓存)` -> `返回到首页(需要缓存)` -> `再次进入列表页(不需要缓存)`，这时候可以按需来控制页面的 `keep-alive`

在路由中设置 `keepAlive` 属性判断是否需要缓存

```
{
  path: 'list',
  name: 'itemList', // 列表页
  component (resolve) {
    require(['@/pages/item/list'], resolve)
  },
  meta: {
    keepAlive: true,
    title: '列表页'
  }
}
```

使用 `<keep-alive>`

```
<div id="app" class="wrapper">
  <keep-alive>
    <!-- 需要缓存的视图组件 -->
    <router-view v-if="$route.meta.keepAlive"></router-view>
  </keep-alive>
  <!-- 不需要缓存的视图组件 -->
  <router-view v-if="!$route.meta.keepAlive"></router-view>
</div>
```

三、原理分析

`keep-alive` 是 `vue` 中内置的一个组件

源码位置：`src/core/components/keep-alive.js`

```
export default {
  name: 'keep-alive',
  abstract: true,

  props: {
    include: [String, RegExp, Array],
    exclude: [String, RegExp, Array],
    max: [String, Number]
  },

  created () {
    this.cache = Object.create(null)
    this.keys = []
  },

  destroyed () {
    for (const key in this.cache) {
      pruneCacheEntry(this.cache, key, this.keys)
    }
  },

  mounted () {
```



```

this.$watch('include', val => {
  pruneCache(this, name => matches(val, name))
})
this.$watch('exclude', val => {
  pruneCache(this, name => !matches(val, name))
})
},

render() {
  /* 获取默认插槽中的第一个组件节点 */
  const slot = this.$slots.default
  const vnode = getFirstComponentChild(slot)
  /* 获取该组件节点的componentOptions */
  const componentOptions = vnode && vnode.componentOptions

  if (componentOptions) {
    /* 获取该组件节点的名称，优先获取组件的name字段，如果name不存在则获取组件的tag */
    const name = getComponentName(componentOptions)

    const { include, exclude } = this
    /* 如果name不在include中或者存在于exlude中则表示不缓存，直接返回vnode */
    if (
      (include && (!name || !matches(include, name))) ||
      // excluded
      (exclude && name && matches(exclude, name))
    ) {
      return vnode
    }

    const { cache, keys } = this
    /* 获取组件的key值 */
    const key = vnode.key == null
      /* same constructor may get registered as different local components
      // so cid alone is not enough (#3269)
      ? componentOptions.Ctor.cid + (componentOptions.tag ? `::${componentOptions.tag}` : ")
      : vnode.key
    /* 拿到key值后去this.cache对象中去寻找是否有该值，如果有则表示该组件有缓存，即命中缓存 */
    if (cache[key]) {
      vnode.componentInstance = cache[key].componentInstance
      // make current key freshest
      remove(keys, key)
      keys.push(key)
    }
    /* 如果没有命中缓存，则将其设置进缓存 */
    else {
      cache[key] = vnode
      keys.push(key)
      // prune oldest entry
      /* 如果配置了max并且缓存的长度超过了this.max，则从缓存中删除第一个 */
      if (this.max && keys.length > parseInt(this.max)) {
        pruneCacheEntry(cache, keys[0], keys, this._vnode)
      }
    }
  }
}

```

```

    }

    vnode.data.keepAlive = true
  }
  return vnode || (slot && slot[0])
}
}

```

可以看到该组件没有 `template`，而是用了 `render`，在组件渲染的时候会自动执行 `render` 函数
`this.cache` 是一个对象，用来存储需要缓存的组件，它将以如下形式存储：

```

this.cache = {
  'key1': '组件1',
  'key2': '组件2',
  // ...
}

```

在组件销毁的时候执行 `pruneCacheEntry` 函数

```

function pruneCacheEntry (
  cache: VNodeCache,
  key: string,
  keys: Array<string>,
  current?: VNode
){
  const cached = cache[key]
  /* 判断当前没有处于被渲染状态的组件，将其销毁 */
  if (cached && (!current || cached.tag !== current.tag)) {
    cached.componentInstance.$destroy()
  }
  cache[key] = null
  remove(keys, key)
}

```

在 `mounted` 钩子函数中观测 `include` 和 `exclude` 的变化，如下：

```

mounted () {
  this.$watch('include', val => {
    pruneCache(this, name => matches(val, name))
  })
  this.$watch('exclude', val => {
    pruneCache(this, name => !matches(val, name))
  })
}

```

如果 `include` 或 `exclude` 发生了变化，即表示定义需要缓存的组件的规则或者不需要缓存的组件的规则发生了变化，那么就执行 `pruneCache` 函数，函数如下：

```
function pruneCache (keepAliveInstance, filter) {
  const { cache, keys, _vnode } = keepAliveInstance
  for (const key in cache) {
    const cachedNode = cache[key]
    if (cachedNode) {
      const name = getComponentName(cachedNode.componentOptions)
      if (name && !filter(name)) {
        pruneCacheEntry(cache, key, keys, _vnode)
      }
    }
  }
}

```

在该函数内对 `this.cache` 对象进行遍历，取出每一项的 `name` 值，用其与新的缓存规则进行匹配，如果匹配不上，则表示在新的缓存规则下该组件已经不需要被缓存，则调用 `pruneCacheEntry` 函数将其从 `this.cache` 对象剔除即可

关于 `keep-alive` 的最强大缓存功能是在 `render` 函数中实现

首先获取组件的 `key` 值：

```
const key = vnode.key == null?
componentOptions.Ctor.cid + (componentOptions.tag ? `:${componentOptions.tag}` : "")
: vnode.key

```

拿到 `key` 值后去 `this.cache` 对象中去找是否有该值，如果有则表示该组件有缓存，即命中缓存，如下：

```
/* 如果命中缓存，则直接从缓存中拿 vnode 的组件实例 */
if (cache[key]) {
  vnode.componentInstance = cache[key].componentInstance
  /* 调整该组件key的顺序，将其从原来的地方删掉并重新放在最后一个 */
  remove(keys, key)
  keys.push(key)
}

```

直接从缓存中拿 `vnode` 的组件实例，此时重新调整该组件 `key` 的顺序，将其从原来的地方删掉并重新放在 `this.keys` 中最后一个

`this.cache` 对象中没有该 `key` 值的情况，如下：

```
/* 如果没有命中缓存，则将其设置进缓存 */
else {
  cache[key] = vnode
  keys.push(key)
  /* 如果配置了max并且缓存的长度超过了this.max，则从缓存中删除第一个 */
  if (this.max && keys.length > parseInt(this.max)) {
    pruneCacheEntry(cache, keys[0], keys, this._vnode)
  }
}

```

表明该组件还没有被缓存过，则以该组件的 `key` 为键，组件 `vnode` 为值，将其存入 `this.cache` 中，并且把 `key` 存入 `this.keys` 中

此时再判断 `this.keys` 中缓存组件的数量是否超过了设置的最大缓存数量值 `this.max`，如果超过了，则把第一个缓存组件删掉

四、思考题：缓存后如何获取数据

解决方案可以有以下两种：

- `beforeRouteEnter`
- `activated`

`beforeRouteEnter`

每次组件渲染的时候，都会执行 `beforeRouteEnter`

```
beforeRouteEnter(to, from, next){
  next(vm=>{
    console.log(vm)
    // 每次进入路由执行
    vm.getData() // 获取数据
  })
},
```

`activated`

在 `keep-alive` 缓存的组件被激活的时候，都会执行 `activated` 钩子

```
activated(){
  this.getData() // 获取数据
},
```

注意：服务器端渲染期间 `activated` 不被调用

参考文献

- <https://www.cnblogs.com/dhui/p/13589401.html>
- <https://www.cnblogs.com/wangjiachen666/p/11497200.html>

面试官：说说你对slot的理解？slot使用场景有哪些？

一、slot是什么

在HTML中 `slot` 元素，作为 `Web Components` 技术套件的一部分，是Web组件内的一个占位符

该占位符可以在后期使用自己的标记语言填充

举个栗子

```
<template id="element-details-template">
  <slot name="element-name">Slot template</slot>
</template>
<element-details>
  <span slot="element-name">1</span>
</element-details>
<element-details>
  <span slot="element-name">2</span>
</element-details>
```

`template` 不会展示到页面中，需要用先获取它的引用，然后添加到 `DOM` 中，

```
customElements.define('element-details',
  class extends HTMLElement {
    constructor() {
      super();
      const template = document
        .getElementById('element-details-template')
        .content;
      const shadowRoot = this.attachShadow({mode: 'open'})
        .appendChild(template.cloneNode(true));
    }
  })
```

在 `Vue` 中的概念也是如此

`Slot` 又名插槽，花名“占坑”，我们可以理解为 `slot` 在组件模板中占好了位置，当使用该组件标签时候，组件标签里面的内容就会自动填坑（替换组件模板中 `slot` 位置），作为承载分发内容的出口

可以将其类比为插卡式的FC游戏机，游戏机暴露卡槽（插槽）让用户插入不同的游戏磁条（自定义内容）

二、使用场景

通过插槽可以让用户可以拓展组件，去更好地复用组件和对其做定制化处理

如果父组件在使用到一个复用组件的时候，获取这个组件在不同的地方有少量的更改，如果去重写组件是一件不明智的事情

通过 `slot` 插槽向组件内部指定位置传递内容，完成这个复用组件在不同场景的应用

比如布局组件、表格列、下拉选、弹框显示内容等

三、分类

`slot` 可以分来以下三种：

- 默认插槽
- 具名插槽
- 作用域插槽

默认插槽

子组件用 `<slot>` 标签来确定渲染的位置，标签里面可以放 `DOM` 结构，当父组件使用的时候没有往插槽传入内容，标签内 `DOM` 结构就会显示在页面

父组件在使用的时候，直接在子组件的标签内写入内容即可

子组件 `Child.vue`

```
<template>
  <slot>
    <p>插槽后备的内容</p>
  </slot>
</template>
```

父组件

```
<Child>
  <div>默认插槽</div>
</Child>
```

具名插槽

子组件用 `name` 属性来表示插槽的名字，不传为默认插槽

父组件中在使用时在默认插槽的基础上加上 `slot` 属性，值为子组件插槽 `name` 属性值

子组件 `Child.vue`

```
<template>
  <slot>插槽后备的内容</slot>
  <slot name="content">插槽后备的内容</slot>
</template>
```

父组件

```
<child>
  <template v-slot:default>具名插槽</template>
  <!-- 具名插槽用插槽名做参数 -->
  <template v-slot:content>内容...</template>
</child>
```

作用域插槽

子组件在作用域上绑定属性来将子组件的信息传给父组件使用，这些属性会被挂在父组件 `v-slot` 接受的对象上

父组件中在使用时通过 `v-slot:`（简写：`#`）获取子组件的信息，在内容中使用

子组件 `Child.vue`


```
<template>
  <slot name="footer" testProps="子组件的值">
    <h3>没传footer插槽</h3>
  </slot>
</template>
```

父组件

```
<child>
  <!-- 把v-slot的值指定为作用域上下文对象 -->
  <template v-slot:default="slotProps">
    来自子组件数据: {{slotProps.testProps}}
  </template>
  <template #default="slotProps">
    来自子组件数据: {{slotProps.testProps}}
  </template>
</child>
```

小结：

- `v-slot` 属性只能在 `<template>` 上使用，但在只有默认插槽时可以在组件标签上使用
- 默认插槽名为 `default`，可以省略 `default` 直接写 `v-slot`
- 缩写为 `#` 时不能不写参数，写成 `#default`
- 可以通过解构获取 `v-slot={user}`，还可以重命名 `v-slot="{user: newName}"` 和定义默认值 `v-slot="{user = '默认值'}"`

四、原理分析

`slot` 本质上是返回 `VNode` 的函数，一般情况下，`Vue` 中的组件要渲染到页面上需要经过 `template -> render function -> VNode -> DOM` 过程，这里看看 `slot` 如何实现：

编写一个 `buttonCounter` 组件，使用匿名插槽

```
Vue.component('button-counter', {
  template: '<div> <slot>我是默认内容</slot></div>'
})
```

使用该组件

```
new Vue({
  el: '#app',
  template: '<button-counter><span>我是slot传入内容</span></button-counter>',
  components: {buttonCounter}
})
```

获取 `buttonCounter` 组件渲染函数

```
(function anonymous(
){
  with(this){return _c('div',[_t("default",[_v("我是默认内容")])],2)}
})
```

`_v` 表示穿件普通文本节点，`_t` 表示渲染插槽的函数

渲染插槽函数 `renderSlot`（做了简化）

```
function renderSlot (
  name,
  fallback,
  props,
  bindObject
){
  // 得到渲染插槽内容的函数
  var scopedSlotFn = this.$scopedSlots[name];
  var nodes;
  // 如果存在插槽渲染函数，则执行插槽渲染函数，生成nodes节点返回
  // 否则使用默认值
  nodes = scopedSlotFn(props) || fallback;
  return nodes;
}
```

`name` 属性表示定义插槽的名字，默认值为 `default`，`fallback` 表示子组件中的 `slot` 节点的默认值

关于 `this.$scopedSlots` 是什么，我们可以先看看 `vm.$slots`

```
function initRender (vm) {
  ...
  vm.$slots = resolveSlots(options._renderChildren, renderContext);
  ...
}
```

`resolveSlots` 函数会对 `children` 节点做归类 and 过滤处理，返回 `slots`

```

function resolveSlots (
  children,
  context
) {
  if (!children || !children.length) {
    return {}
  }
  var slots = {};
  for (var i = 0, l = children.length; i < l; i++) {
    var child = children[i];
    var data = child.data;
    // remove slot attribute if the node is resolved as a Vue slot node
    if (data && data.attrs && data.attrs.slot) {
      delete data.attrs.slot;
    }
    // named slots should only be respected if the vnode was rendered in the
    // same context.
    if ((child.context === context || child.fnContext === context) &&
      data && data.slot !== null
    ) {
      // 如果slot存在(slot="header") 则拿对应的值作为key
      var name = data.slot;
      var slot = (slots[name] || (slots[name] = []));
      // 如果是tempalte元素 则把template的children添加进数组中，这也就是为什么你写的template
      // 标签并不会渲染成另一个标签到页面
      if (child.tag === 'template') {
        slot.push.apply(slot, child.children || []);
      } else {
        slot.push(child);
      }
    } else {
      // 如果没有就默认是default
      (slots.default || (slots.default = [])).push(child);
    }
  }
  // ignore slots that contains only whitespace
  for (var name$1 in slots) {
    if (slots[name$1].every(isWhitespace)) {
      delete slots[name$1];
    }
  }
  return slots
}

```

`_render` 渲染函数通过 `normalizeScopedSlots` 得到 `vm.$scopedSlots`

```

vm.$scopedSlots = normalizeScopedSlots(
  _parentVnode.data.scopedSlots,
  vm.$slots,
  vm.$scopedSlots
);

```

作用域插槽中父组件能够得到子组件的值是因为在 `renderSlot` 的时候执行会传入 `props`，也就是上述 `_t` 第三个参数，父组件则能够得到子组件传递过来的值

参考文献

- <https://juejin.cn/post/6844903817746628615#heading-4>
- https://developer.mozilla.org/en-US/docs/Web/Web_Components/Using_templates_and_slots
- https://segmentfault.com/a/1190000019492734?utm_source=tag-newest

面试官：说说你对vue的mixin的理解，有什么应用场景？

一、mixin是什么

`Mixin` 是面向对象程序设计语言中的类，提供了方法的实现。其他类可以访问 `mixin` 类的方法而不必成为其子类

`Mixin` 类通常作为功能模块使用，在需要该功能时“混入”，有利于代码复用又避免了多继承的复杂

Vue中的mixin

先来看一下官方定义

“
`mixin`（混入），提供了一种非常灵活的方式，来分发 `Vue` 组件中的可复用功能。

本质其实就是一个 `js` 对象，它可以包含我们组件中任意功能选项，如 `data`、`components`、`methods`、`created`、`computed` 等等

我们只要将共用的功能以对象的方式传入 `mixins` 选项中，当组件使用 `mixins` 对象时所有 `mixins` 对象的选项都将被混入该组件本身的选项中来

在 `Vue` 中我们可以「局部混入」跟「全局混入」

局部混入

定义一个 `mixin` 对象，有组件 `options` 的 `data`、`methods` 属性

```
var myMixin = {
  created: function () {
    this.hello()
  },
  methods: {
    hello: function () {
      console.log('hello from mixin!')
    }
  }
}
```

组件通过 `mixins` 属性调用 `mixin` 对象

```
Vue.component('componentA',{
  mixins: [myMixin]
})
```

该组件在使用的时候，混合了 `mixin` 里面的方法，在自动执行 `created` 生命钩子，执行 `hello` 方法

全局混入

通过 `Vue.mixin()` 进行全局的混入

```
Vue.mixin({
  created: function () {
    console.log("全局混入")
  }
})
```

使用全局混入需要特别注意，因为它会影响到每一个组件实例（包括第三方组件）

PS：全局混入常用于插件的编写

注意事项：

当组件存在与 `mixin` 对象相同的选项的时候，进行递归合并的时候组件的选项会覆盖 `mixin` 的选项

但是如果相同选项为生命周期钩子的时候，会合并成一个数组，先执行 `mixin` 的钩子，再执行组件的钩子

二、使用场景

在日常的开发中，我们经常会遇到在不同的组件中经常会需要用到一些相同或者相似的代码，这些代码的功能相对独立

这时，可以通过 `Vue` 的 `mixin` 功能将相同或者相似的代码提出来

举个例子

定义一个 `modal` 弹窗组件，内部通过 `isShowing` 来控制显示

```
const Modal = {
  template: '#modal',
  data() {
    return {
      isShowing: false
    }
  },
  methods: {
    toggleShow() {
      this.isShowing = !this.isShowing;
    }
  }
}
```

定义一个 `tooltip` 提示框，内部通过 `isShowing` 来控制显示

```
const Tooltip = {
  template: '#tooltip',
  data() {
    return {
      isShowing: false
    }
  },
  methods: {
    toggleShow() {
      this.isShowing = !this.isShowing;
    }
  }
}
```

通过观察上面两个组件，发现两者的逻辑是相同，代码控制显示也是相同的，这时候 `mixin` 就派上用场了

首先抽出共同代码，编写一个 `mixin`

```
const toggle = {
  data() {
    return {
      isShowing: false
    }
  },
  methods: {
    toggleShow() {
      this.isShowing = !this.isShowing;
    }
  }
}
```

两个组件在使用上，只需要引入 `mixin`

```
const Modal = {
  template: '#modal',
  mixins: [toggle]
};

const Tooltip = {
  template: '#tooltip',
  mixins: [toggle]
}
```

通过上面小小的例子，让我们知道了 `Mixin` 对于封装一些可复用的功能如此有趣、方便、实用

三、源码分析

首先从 `Vue.mixin` 入手

源码位置：`/src/core/global-api/mixin.js`


```
export function initMixin (Vue: GlobalAPI) {
  Vue.mixin = function (mixin: Object) {
    this.options = mergeOptions(this.options, mixin)
    return this
  }
}
```

主要是调用 `mergeOptions` 方法

源码位置： `/src/core/util/options.js`

```
export function mergeOptions (
  parent: Object,
  child: Object,
  vm?: Component
): Object {
  if (child.mixins) { // 判断有没有mixin 也就是mixin里面挂mixin的情况 有的话递归进行合并
    for (let i = 0, l = child.mixins.length; i < l; i++) {
      parent = mergeOptions(parent, child.mixins[i], vm)
    }
  }

  const options = {}
  let key
  for (key in parent) {
    mergeField(key) // 先遍历parent的key 调对应的strats[XXX]方法进行合并
  }
  for (key in child) {
    if (!hasOwn(parent, key)) { // 如果parent已经处理过某个key 就不处理了
      mergeField(key) // 处理child中的key 也就parent中没有处理过的key
    }
  }
  function mergeField (key) {
    const strat = strats[key] || defaultStrat
    options[key] = strat(parent[key], child[key], vm, key) // 根据不同类型的options调用strats中不同的方法进行合并
  }
  return options
}
```

从上面的源码，我们得到以下几点：

- 优先递归处理 `mixins`
- 先遍历合并 `parent` 中的 `key`，调用 `mergeField` 方法进行合并，然后保存在变量 `options`
- 再遍历 `child`，合并补上 `parent` 中没有的 `key`，调用 `mergeField` 方法进行合并，保存在变量 `options`
- 通过 `mergeField` 函数进行了合并

下面是关于 `Vue` 的几种类型的合并策略

- 替换型
- 合并型

- 队列型
- 叠加型

替换型

替换型合并有 `props`、`methods`、`inject`、`computed`

```
strats.props =
strats.methods =
strats.inject =
strats.computed = function (
  parentVal: ?Object,
  childVal: ?Object,
  vm?: Component,
  key: string
): ?Object {
  if (!parentVal) return childVal // 如果parentVal没有值, 直接返回childVal
  const ret = Object.create(null) // 创建一个第三方对象 ret
  extend(ret, parentVal) // extend方法实际是把parentVal的属性复制到ret中
  if (childVal) extend(ret, childVal) // 把childVal的属性复制到ret中
  return ret
}
strats.provide = mergeDataOrFn
```

同名的 `props`、`methods`、`inject`、`computed` 会被后来者代替

合并型

和并型合并有: `data`

```

strats.data = function(parentVal, childVal, vm) {
  return mergeDataOrFn(
    parentVal, childVal, vm
  )
};

function mergeDataOrFn(parentVal, childVal, vm) {
  return function mergedInstanceDataFn() {
    var childData = childVal.call(vm, vm) // 执行data挂的函数得到对象
    var parentData = parentVal.call(vm, vm)
    if (childData) {
      return mergeData(childData, parentData) // 将2个对象进行合并
    } else {
      return parentData // 如果没有childData 直接返回parentData
    }
  }
}

function mergeData(to, from) {
  if (!from) return to
  var key, toVal, fromVal;
  var keys = Object.keys(from);
  for (var i = 0; i < keys.length; i++) {
    key = keys[i];
    toVal = to[key];
    fromVal = from[key];
    // 如果不存在这个属性，就重新设置
    if (!to.hasOwnProperty(key)) {
      set(to, key, fromVal);
    }
    // 存在相同属性，合并对象
    else if (typeof toVal == "object" && typeof fromVal == "object") {
      mergeData(toVal, fromVal);
    }
  }
  return to
}

```

`mergeData` 函数遍历了要合并的 `data` 的所有属性，然后根据不同情况进行合并：

- 当目标 `data` 对象不包含当前属性时，调用 `set` 方法进行合并（`set`方法其实就是一些合并重新赋值的方法）
- 当目标 `data` 对象包含当前属性并且当前值为纯对象时，递归合并当前对象值，这样做是为了防止对象存在新增属性

队列性

队列性合并有：全部生命周期和 `watch`

```

function mergeHook (
  parentVal: ?Array<Function>,
  childVal: ?Function | ?Array<Function>
): ?Array<Function> {
  return childVal
    ? parentVal
      ? parentVal.concat(childVal)
        : Array.isArray(childVal)
          ? childVal
            : [childVal]
        : parentVal
    }

LIFECYCLE_HOOKS.forEach(hook => {
  strats[hook] = mergeHook
})

// watch
strats.watch = function (
  parentVal,
  childVal,
  vm,
  key
) {
  // work around Firefox's Object.prototype.watch...
  if (parentVal === nativeWatch) { parentVal = undefined; }
  if (childVal === nativeWatch) { childVal = undefined; }
  /* istanbul ignore if */
  if (!childVal) { return Object.create(parentVal || null); }
  {
    assertObjectType(key, childVal, vm);
  }
  if (!parentVal) { return childVal; }
  var ret = {};
  extend(ret, parentVal);
  for (var key$1 in childVal) {
    var parent = ret[key$1];
    var child = childVal[key$1];
    if (parent && !Array.isArray(parent)) {
      parent = [parent];
    }
    ret[key$1] = parent
      ? parent.concat(child)
        : Array.isArray(child) ? child : [child];
  }
  return ret
};

```

生命周期钩子和 `watch` 被合并为一个数组，然后正序遍历一次执行

叠加型

叠加型合并有：`component`、`directives`、`filters`

```
strats.components=
strats.directives=

strats.filters = function mergeAssets(
  parentVal, childVal, vm, key
){
  var res = Object.create(parentVal || null);
  if (childVal) {
    for (var key in childVal) {
      res[key] = childVal[key];
    }
  }
  return res
}
```

叠加型主要是通过原型链进行层层叠加

小结：

- 替换型策略有 `props`、`methods`、`inject`、`computed`，就是将新的同名参数替代旧的参数
- 合并型策略是 `data`，通过 `set` 方法进行合并和重新赋值
- 队列型策略有生命周期函数和 `watch`，原理是将函数存入一个数组，然后正序遍历依次执行
- 叠加型有 `component`、`directives`、`filters`，通过原型链进行层层叠加

参考文献

- <https://zhuanlan.zhihu.com/p/31018570>
- <https://juejin.cn/post/6844904015495446536#heading-1>
- <https://juejin.cn/post/6844903846775357453>

面试官：请描述下你对vue生命周期的理解？在created和mounted这两个生命周期中请求数据有什么区别呢？

一、生命周期是什么

生命周期 (`Life Cycle`) 的概念应用很广泛，特别是在政治、经济、环境、技术、社会等诸多领域经常出现，其基本涵义可以通俗地理解为“从摇篮到坟墓” (`Cradle-to-Grave`) 的整个过程在 `Vue` 中实例从创建到销毁的过程就是生命周期，即指从创建、初始化数据、编译模板、挂载Dom→渲染、更新→渲染、卸载等一系列过程我们可以把组件比喻成工厂里面的一条流水线，每个工人（生命周期）站在各自的岗位，当任务流转到工人身边的时候，工人就开始工作PS：在 `Vue` 生命周期钩子会自动绑定 `this` 上下文到实例中，因此你可以访问数据，对 `property` 和方法进行运算这意味着「**你不能使用箭头函数来定义一个生命周期方法**」（例如 `created: () => this.fetchTodos()`）

二、生命周期有哪些

`Vue` 生命周期总共可以分为8个阶段：创建前后，载入前后，更新前后，销毁前销毁后，以及一些特殊场景的生命周期

生命周期	描述
beforeCreate	组件实例被创建之初
created	组件实例已经完全创建
beforeMount	组件挂载之前
mounted	组件挂载到实例上去之后
beforeUpdate	组件数据发生变化，更新之前
updated	组件数据更新之后
beforeDestroy	组件实例销毁之前
destroyed	组件实例销毁之后
activated	keep-alive 缓存的组件激活时
deactivated	keep-alive 缓存的组件停用时调用
errorCaptured	捕获一个来自子孙组件的错误时被调用

三、生命周期整体流程

Vue 生命周期流程图

具体分析

「beforeCreate -> created」

- 初始化 vue 实例，进行数据观测

「created」

- 完成数据观测，属性与方法的运算，watch、event 事件回调的配置
- 可调用 methods 中的方法，访问和修改 data 数据触发响应式渲染 dom，可通过 computed 和 watch 完成数据计算
- 此时 vm.\$el 并没有被创建

「created -> beforeMount」

- 判断是否存在 el 选项，若不存在则停止编译，直到调用 vm.\$mount(el) 才会继续编译
- 优先级：render > template > outerHTML
- vm.el 获取到的是挂载 DOM 的

「beforeMount」

- 在此阶段可获取到 vm.el
- 此阶段 vm.el 虽已完成 DOM 初始化，但并未挂载在 el 选项上

「beforeMount -> mounted」

- 此阶段 vm.el 完成挂载，vm.\$el 生成的 DOM 替换了 el 选项所对应的 DOM

「mounted」

- vm.el 已完成 DOM 的挂载与渲染，此刻打印 vm.\$el，发现之前的挂载点及内容已被替换成新的 DOM

「beforeUpdate」

- 更新的数据必须是被渲染在模板上的（el、template、render 之一）
- 此时 view 层还未更新
- 若在 beforeUpdate 中再次修改数据，不会再次触发更新方法

「updated」

- 完成 view 层的更新
- 若在 updated 中再次修改数据，会再次触发更新方法（beforeUpdate、updated）

「beforeDestroy」

- 实例被销毁前调用，此时实例属性与方法仍可访问

「destroyed」

- 完全销毁一个实例。可清理它与其它实例的连接，解绑它的全部指令及事件监听器
- 并不能清除 DOM，仅仅销毁实例

「使用场景分析」

生命周期	描述
beforeCreate	执行时组件实例还未创建，通常用于插件开发中执行一些初始化任务
created	组件初始化完毕，各种数据可以使用，常用于异步数据获取
beforeMount	未执行渲染、更新，dom未创建
mounted	初始化结束，dom已创建，可用于获取访问数据和dom元素
beforeUpdate	更新前，可用于获取更新前各种状态
updated	更新后，所有状态已是最新
beforeDestroy	销毁前，可用于一些定时器或订阅的取消
destroyed	组件已销毁，作用同上

四、数据请求在created和mounted的区别

created 是在组件实例一旦创建完成的时候立刻调用，这时候页面 dom 节点并未生成；mounted 是在页面 dom 节点渲染完毕之后就立刻执行的。触发时机上 created 是比 mounted 要更早的，两者的相同点：都能拿到实例对象的属性和方法。讨论这个问题本质就是触发的时机，放在 mounted 中的请求有可能导致页面闪动（因为此时页面 dom 结构已经生成），但如果在页面加载前完成请求，则不会出现此情况。建议对页面内容的改动放在 created 生命周期当中。

参考文献

- <https://juejin.cn/post/6844903811094413320>
- <https://baike.baidu.com/>

面试官：Vue3.0 所采用的 Composition Api 与 Vue2.x 使用的 Options Api 有什么不同？

开始之前

Composition API 可以说是 Vue3 的最大特点，那么为什么要推出 Composition Api，解决了什么问题？

通常使用 Vue2 开发的项目，普遍会存在以下问题：

- 代码的可读性随着组件变大而变差
- 每一种代码复用的方式，都存在缺点
- TypeScript 支持有限

以上通过使用 Composition Api 都能迎刃而解

正文

一、Options Api

Options API，即大家常说的选项API，即以 **vue** 为后缀的文件，通过定义 **methods**，**computed**，**watch**，**data** 等属性与方法，共同处理页面逻辑

如下图：

可以看到 **Options** 代码编写方式，如果是组件状态，则写在 **data** 属性上，如果是方法，则写在 **methods** 属性上...

用组件的选项 (**data**、**computed**、**methods**、**watch**) 组织逻辑在大多数情况下都有效
然而，当组件变得复杂，导致对应属性的列表也会增长，这可能会导致组件难以阅读和理解

二、Composition Api

在 Vue3 Composition API 中，组件根据逻辑功能来组织的，一个功能所定义的所有 API 会放在一起（更加的高内聚，低耦合）

即使项目很大，功能很多，我们都能快速的定位到这个功能所用到的所有 API

三、对比

下面对 **Composition Api** 与 **Options Api** 进行两大方面的比较

- 逻辑组织
- 逻辑复用

逻辑组织

Options API

假设一个组件是一个大型组件，其内部有很多处理逻辑关注点（对应下图不用颜色）

可以看到，这种碎片化使得理解和维护复杂组件变得困难

选项的分离掩盖了潜在的逻辑问题。此外，在处理单个逻辑关注点时，我们必须不断地“跳转”相关代码的选项块

Compostion API

而 **Compositon API** 正是解决上述问题，将某个逻辑关注点相关的代码全都放在一个函数里，这样当需要修改一个功能时，就不再需要在文件中跳来跳去

下面举个简单例子，将处理 **count** 属性相关的代码放在同一个函数了

```
function useCount() {
  let count = ref(10);
  let double = computed(() => {
    return count.value * 2;
  });

  const handleConut = () => {
    count.value = count.value * 2;
  };

  console.log(count);

  return {
    count,
    double,
    handleConut,
  };
}
```

组件上中使用 `count`

```
export default defineComponent({
  setup() {
    const { count, double, handleConut } = useCount();
    return {
      count,
      double,
      handleConut
    }
  },
});
```

再来一张图进行对比，可以很直观地感受到 `Composition API`在逻辑组织方面的优势，以后修改一个属性功能的时候，只需要跳到控制该属性的方法中即可

Options API

```
export default {
  data() {
    return {
      // ...
    }
  },
  computed: {
    // ...
  },
  methods: {
    // ...
  },
  watch: {
    // ...
  },
  created() {
    // ...
  },
  mounted() {
    // ...
  },
  beforeDestroy() {
    // ...
  },
  destroyed() {
    // ...
  },
  render(h) {
    // ...
  }
}
```

Composition API

```
import { ref, computed, watch, onCreated, onMounted, onBeforeDestroy, onDestroyed, render } from 'vue'

export default {
  setup() {
    // ...
  },
  render(h) {
    // ...
  }
}
```

逻辑复用

在 [Vue2](#) 中，我们是用过 [mixin](#) 去复用相同的逻辑

下面举个例子，我们会另起一个 [mixin.js](#) 文件

```

export const MoveMixin = {
  data() {
    return {
      x: 0,
      y: 0,
    };
  },

  methods: {
    handleKeyup(e) {
      console.log(e.code);
      // 上下左右 x y
      switch (e.code) {
        case "ArrowUp":
          this.y--;
          break;
        case "ArrowDown":
          this.y++;
          break;
        case "ArrowLeft":
          this.x--;
          break;
        case "ArrowRight":
          this.x++;
          break;
      }
    },
  },

  mounted() {
    window.addEventListener("keyup", this.handleKeyup);
  },

  unmounted() {
    window.removeEventListener("keyup", this.handleKeyup);
  },
};

```

然后在组件中使用

```

<template>
  <div>
    Mouse position: x {{ x }} / y {{ y }}
  </div>
</template>
<script>
import mousePositionMixin from './mouse'
export default {
  mixins: [mousePositionMixin]
}
</script>

```

使用单个 `mixin` 似乎问题不大，但是当我们一个组件混入大量不同的 `mixins` 的时候

```
mixins: [mousePositionMixin, fooMixin, barMixin, otherMixin]
```

会存在两个非常明显的问题：

- 命名冲突
- 数据来源不清晰

现在通过 `Composition API` 这种方式改写上面的代码

```
import { onMounted, onUnmounted, reactive } from "vue";
export function useMove() {
  const position = reactive({
    x: 0,
    y: 0,
  });

  const handleKeyup = (e) => {
    console.log(e.code);
    // 上下左右 x y
    switch (e.code) {
      case "ArrowUp":
        // y.value--;
        position.y--;
        break;
      case "ArrowDown":
        // y.value++;
        position.y++;
        break;
      case "ArrowLeft":
        // x.value--;
        position.x--;
        break;
      case "ArrowRight":
        // x.value++;
        position.x++;
        break;
    }
  };

  onMounted(() => {
    window.addEventListener("keyup", handleKeyup);
  });

  onUnmounted(() => {
    window.removeEventListener("keyup", handleKeyup);
  });

  return { position };
}
```

在组件中使用


```
<template>
  <div>
    Mouse position: x {{ x }} / y {{ y }}
  </div>
</template>

<script>
import { useMove } from './useMove';
import { toRefs } from 'vue';
export default {
  setup() {
    const { position } = useMove();
    const { x, y } = toRefs(position);
    return {
      x,
      y,
    };
  },
};
</script>
```

可以看到，整个数据来源清晰了，即使去编写更多的 hook 函数，也不会出现命名冲突的问题

小结

- 在逻辑组织和逻辑复用方面，Composition API 是优于 Options API
- 因为 Composition API 几乎是函数，会有更好的类型推断。
- Composition API 对 tree-shaking 友好，代码也更容易压缩
- Composition API 中见不到 this 的使用，减少了 this 指向不明的情况
- 如果是小型组件，可以继续使用 Options API，也是十分友好的

面试官：Vue3.0 性能提升主要是通过哪几方面体现的？

一、编译阶段

回顾 Vue2，我们知道每个组件实例都对应一个 watcher 实例，它会在组件渲染的过程中把用到的数据 property 记录为依赖，当依赖发生改变，触发 setter，则会通知 watcher，从而使关联的组件重新渲染

试想一下，一个组件结构如下图

```
<template>
  <div id="content">
    <p class="text">静态文本</p>
    <p class="text">静态文本</p>
    <p class="text">{{ message }}</p>
    <p class="text">静态文本</p>
    ...
    <p class="text">静态文本</p>
  </div>
</template>
```

可以看到，组件内部只有一个动态节点，剩余一堆都是静态节点，所以这里很多 `diff` 和遍历其实都是不需要的，造成性能浪费

因此，`Vue3` 在编译阶段，做了进一步优化。主要有如下：

- `diff` 算法优化
- 静态提升
- 事件监听缓存
- `SSR` 优化

diff 算法优化

`vue3` 在 `diff` 算法中相比 `vue2` 增加了静态标记

关于这个静态标记，其作用是为了会发生变化的地方添加一个 `flag` 标记，下次发生变化的时候直接找该地方进行比较

下图这里，已经标记静态节点的 `p` 标签在 `diff` 过程中则不会比较，把性能进一步提高

关于静态类型枚举如下

```
export const enum PatchFlags {
  TEXT = 1, // 动态的文本节点
  CLASS = 1 << 1, // 2 动态的 class
  STYLE = 1 << 2, // 4 动态的 style
  PROPS = 1 << 3, // 8 动态属性，不包括类名和样式
  FULL_PROPS = 1 << 4, // 16 动态 key，当 key 变化时需要完整的 diff 算法做比较
  HYDRATE_EVENTS = 1 << 5, // 32 表示带有事件监听器的节点
  STABLE_FRAGMENT = 1 << 6, // 64 一个不会改变子节点顺序的 Fragment
  KEYED_FRAGMENT = 1 << 7, // 128 带有 key 属性的 Fragment
  UNKEYED_FRAGMENT = 1 << 8, // 256 子节点没有 key 的 Fragment
  NEED_PATCH = 1 << 9, // 512
  DYNAMIC_SLOTS = 1 << 10, // 动态 slot
  HOISTED = -1, // 特殊标志是负整数表示永远不会用作 diff
  BAIL = -2 // 一个特殊的标志，指代差异算法
}
```

静态提升

`Vue3` 中对不参与更新的元素，会做静态提升，只会被创建一次，在渲染时直接复用

这样就免去了重复的创建节点，大型应用会受益于这个改动，免去了重复的创建操作，优化了运行时候的内存占用

```
<span>你好</span>

<div>{{ message }}</div>
```

没有做静态提升之前

```
export function render(_ctx, _cache, $props, $setup, $data, $options) {
  return (_openBlock(), _createBlock(_Fragment, null, [
    _createVNode("span", null, "你好"),
    _createVNode("div", null, _toDisplayString(_ctx.message), 1 /* TEXT */)
  ], 64 /* STABLE_FRAGMENT */))
}
```

做了静态提升之后

```
const _hoisted_1 = /*#__PURE__*/_createVNode("span", null, "你好", -1 /* HOISTED */)

export function render(_ctx, _cache, $props, $setup, $data, $options) {
  return (_openBlock(), _createBlock(_Fragment, null, [
    _hoisted_1,
    _createVNode("div", null, _toDisplayString(_ctx.message), 1 /* TEXT */)
  ], 64 /* STABLE_FRAGMENT */))
}

// Check the console for the AST
```

静态内容 `_hoisted_1` 被放置在 `render` 函数外，每次渲染的时候只要取 `_hoisted_1` 即可

同时 `_hoisted_1` 被打上了 `PatchFlag`，静态标记值为 `-1`，特殊标志是负整数表示永远不会用于 Diff

事件监听缓存

默认情况下绑定事件行为会被视为动态绑定，所以每次都会去追踪它的变化

```
<div>
  <button @click = 'onClick'>点我</button>
</div>
```

没开启事件监听器缓存

```
export const render = /*#__PURE__*/_withId(function render(_ctx, _cache, $props, $setup,
$data, $options) {
  return (_openBlock(), _createBlock("div", null, [
    _createVNode("button", { onClick: _ctx.onClick }, "点我", 8 /* PROPS */, ["onClick"])
    // PROPS=1<<3, 8 //动态属性，但不包含类名和样式
  ]))
})
```

开启事件侦听器缓存后

```
export function render(_ctx, _cache, $props, $setup, $data, $options) {
  return (_openBlock(), _createBlock("div", null, [
    _createVNode("button", {
      onClick: _cache[1] || (_cache[1] = (...args) => (_ctx.onClick(...args)))
    }, "点我")
  ]))
}
```

上述发现开启了缓存后，没有了静态标记。也就是说下次 diff 算法的时候直接使用

SSR 优化

当静态内容大到一定量级时候，会用 createStaticVNode 方法在客户端去生成一个 static node，这些静态 node，会被直接 innerHtml，就不需要创建对象，然后根据对象渲染

```
div>
  <div>
    <span>你好</span>
  </div>
  ... // 很多个静态属性
  <div>
    <span>{{ message }}</span>
  </div>
</div>
```

编译后

```
import { mergeProps as _mergeProps } from "vue"
import { ssrRenderAttrs as _ssrRenderAttrs, ssrInterpolate as _ssrInterpolate } from
"@vue/server-renderer"

export function ssrRender(_ctx, _push, _parent, _attrs, $props, $setup, $data, $options) {
  const _cssVars = { style: { color: _ctx.color } }
  _push(`<div${
    _ssrRenderAttrs(_mergeProps(_attrs, _cssVars))
  }><div><span>你好</span>...<div><span>你好</span><div><span>${
    _ssrInterpolate(_ctx.message)
  }</span></div></div>`)
}
```

二、源码体积

相比 Vue2，Vue3 整体体积变小了，除了移出一些不常用的 API，再重要的是 Tree shanking

任何一个函数，如 ref、reavtived、computed 等，仅仅在用到的时候才打包，没用到的模块都被摇掉，打包的整体体积变小

```
import { computed, defineComponent, ref } from 'vue';
export default defineComponent({
  setup(props, context) {
    const age = ref(18)

    let state = reactive({
      name: 'test'
    })

    const readOnlyAge = computed(() => age.value++) // 19

    return {
      age,
      state,
      readOnlyAge
    }
  }
});
```

三、响应式系统

vue2 中采用 `defineProperty` 来劫持整个对象，然后进行深度遍历所有属性，给每个属性添加 `getter` 和 `setter`，实现响应式

vue3 采用 `proxy` 重写了响应式系统，因为 `proxy` 可以对整个对象进行监听，所以不需要深度遍历

- 可以监听动态属性的添加
- 可以监听到数组的索引和数组 `length` 属性
- 可以监听删除属性

关于这两个 API 具体的不同，我们下篇文章会进行一个更加详细的介绍

参考文献

- <https://juejin.cn/post/6903171037211557895>

面试官：Vue3.0的设计目标是什么？做了哪些优化

一、设计目标

不以解决实际业务痛点的更新都是耍流氓，下面我们来列举一下 `Vue3` 之前我们或许会面临的问题

- 随着功能的生长，复杂组件的代码变得越来越难以维护
- 缺少一种比较「干净」的在多个组件之间提取和复用逻辑的机制
- 类型推断不够友好
- `bundle` 的时间太久了

而 `Vue3` 经过长达两三年时间的筹备，做了哪些事情？

我们从结果反推

- 更小
- 更快
- TypeScript支持
- API设计一致性
- 提高自身可维护性
- 开放更多底层功能

一句话概述，就是更小更快更友好了

更小

Vue3 移除一些不常用的 API

引入 `tree-shaking`，可以将无用模块“剪辑”，仅打包需要的，使打包的整体体积变小了

更快

主要体现在编译方面：

- diff算法优化
- 静态提升
- 事件监听缓存
- SSR优化

下篇文章我们会进一步介绍

更友好

vue3 在兼顾 vue2 的 `options API`的同时还推出了 `composition API`，大大增加了代码的逻辑组织和代码复用能力

这里代码简单演示下：

存在一个获取鼠标位置的函数

```
import { toRefs, reactive } from 'vue';
function useMouse(){
  const state = reactive({x:0,y:0});
  const update = e=>{
    state.x = e.pageX;
    state.y = e.pageY;
  }
  onMounted(()=>{
    window.addEventListener('mousemove',update);
  })
  onUnmounted(()=>{
    window.removeEventListener('mousemove',update);
  })

  return toRefs(state);
}
```

我们只需要调用这个函数，即可获取 `x`、`y` 的坐标，完全不用关注实现过程

试想一下，如果很多类似的第三方库，我们只需要调用即可，不必关注实现过程，开发效率大大提高

同时，VUE3 是基于 `typescript` 编写的，可以享受到自动的类型定义提示

三、优化方案

vue3 从很多层面都做了优化，可以分成三个方面：

- 源码
- 性能
- 语法 API

源码

源码可以从两个层面展开：

- 源码管理
- TypeScript

源码管理

vue3 整个源码是通过 monorepo 的方式维护的，根据功能将不同的模块拆分到 packages 目录下面不同的子目录中

这样使得模块拆分更细化，职责划分更明确，模块之间的依赖关系也更加明确，开发人员也更容易阅读、理解和更改所有模块源码，提高代码的可维护性

另外一些 package（比如 reactivity 响应式库）是可以独立于 Vue 使用的，这样用户如果只想使用 Vue3 的响应式能力，可以单独依赖这个响应式库而不用去依赖整个 Vue

TypeScript

Vue3 是基于 TypeScript 编写的，提供了更好的类型检查，能支持复杂的类型推导

性能

vue3 是从哪些方面对性能进行进一步优化呢？

- 体积优化
- 编译优化
- 数据劫持优化

这里讲述数据劫持：

在 vue2 中，数据劫持是通过 Object.defineProperty，这个 API 有一些缺陷，并不能检测对象属性的添加和删除

```
Object.defineProperty(data, 'a',{
  get(){
    // track
  },
  set(){
    // trigger
  }
})
```

尽管 Vue 为了解决这个问题提供了 set 和 delete 实例方法，但是对于用户来说，还是增加了一定的心智负担

同时在面对嵌套层级比较深的情况下，就存在性能问题

```
default {  
  data: {  
    a: {  
      b: {  
        c: {  
          d: 1  
        }  
      }  
    }  
  }  
}
```

相比之下，`vue3` 是通过 `proxy` 监听整个对象，那么对于删除还是监听当然也能监听到

同时 `Proxy` 并不能监听到内部深层次的对象变化，而 `Vue3` 的处理方式是在 `getter` 中去递归响应式，这样的好处是真正访问到的内部对象才会变成响应式，而不是无脑递归

语法 API

这里当然说的就是 `composition API`，其两大显著的优化：

- 优化逻辑组织
- 优化逻辑复用

逻辑组织

一张图，我们可以很直观地感受到 `Composition API` 在逻辑组织方面的优势

相同功能的代码编写在一块，而不像 `options API` 那样，各个功能的代码混成一块

逻辑复用

在 `vue2` 中，我们是通过 `mixin` 实现功能混合，如果多个 `mixin` 混合，会存在两个非常明显的问题：命名冲突和数据来源不清晰

而通过 `composition` 这种形式，可以将一些复用的代码抽离出来作为一个函数，只要使用的地方直接进行调用即可

同样是上文的获取鼠标位置的例子

```
import { toRefs, reactive, onUnmounted, onMounted } from 'vue';
function useMouse(){
  const state = reactive({x:0,y:0});
  const update = e=>{
    state.x = e.pageX;
    state.y = e.pageY;
  }
  onMounted(()=>{
    window.addEventListener('mousemove',update);
  })
  onUnmounted(()=>{
    window.removeEventListener('mousemove',update);
  })

  return toRefs(state);
}
```

组件使用

```
import useMousePosition from './mouse'
export default {
  setup() {
    const { x, y } = useMousePosition()
    return { x, y }
  }
}
```

可以看到，整个数据来源清晰了，即使去编写更多的 `hook` 函数，也不会出现命名冲突的问题

参考文献

- <https://juejin.cn/post/6850418112878575629#heading-5>

面试官：Vue3.0里为什么要用 Proxy API 替代 defineProperty API ？

一、Object.defineProperty

定义：`Object.defineProperty()` 方法会直接在一个对象上定义一个新属性，或者修改一个对象的现有属性，并返回此对象

为什么能实现响应式

通过 `defineProperty` 两个属性，`get` 及 `set`

- `get`

属性的 `getter` 函数，当访问该属性时，会调用此函数。执行时不传入任何参数，但是会传入 `this` 对象（由于继承关系，这里的 `this` 并不一定是定义该属性的对象）。该函数的返回值会被用作属性的值

- `set`

属性的 setter 函数，当属性值被修改时，会调用此函数。该方法接受一个参数（也就是被赋予的新值），会传入赋值时的 this 对象。默认为 undefined

下面通过代码展示：

定义一个响应式函数 `defineReactive`

```
function update() {
  app.innerText = obj.foo
}

function defineReactive(obj, key, val) {
  Object.defineProperty(obj, key, {
    get() {
      console.log(`get ${key}:${val}`);
      return val
    },
    set(newVal) {
      if (newVal !== val) {
        val = newVal
        update()
      }
    }
  })
}
```

调用 `defineReactive`，数据发生变化触发 `update` 方法，实现数据响应式

```
const obj = {}
defineReactive(obj, 'foo', '')
setTimeout(()=>{
  obj.foo = new Date().toLocaleTimeString()
},1000)
```

在对象存在多个 `key` 情况下，需要进行遍历

```
function observe(obj) {
  if (typeof obj !== 'object' || obj == null) {
    return
  }
  Object.keys(obj).forEach(key => {
    defineReactive(obj, key, obj[key])
  })
}
```

如果存在嵌套对象的情况，还需要在 `defineReactive` 中进行递归

```
function defineReactive(obj, key, val) {
  observe(val)
  Object.defineProperty(obj, key, {
    get() {
      console.log(`get ${key}.${val}`);
      return val
    },
    set(newVal) {
      if (newVal !== val) {
        val = newVal
        update()
      }
    }
  })
}
```

当给 `key` 赋值为对象的时候，还需要在 `set` 属性中进行递归

```
set(newVal) {
  if (newVal !== val) {
    observe(newVal) // 新值是对象的情况
    notifyUpdate()
  }
}
```

上述例子能够实现对一个对象的基本响应式，但仍然存在诸多问题
现在对一个对象进行删除与添加属性操作，无法劫持到

```
const obj = {
  foo: "foo",
  bar: "bar"
}
observe(obj)
delete obj.foo // no ok
obj.jar = 'xxx' // no ok
```

当我们对一个数组进行监听的时候，并不那么好使了

```
const arrData = [1,2,3,4,5];
arrData.forEach((val,index)=>{
  defineProperty(arrData,index,val)
})
arrData.push() // no ok
arrData.pop() // no ok
arrDate[0] = 99 // ok
```

可以看到数据的 `api` 无法劫持到，从而无法实现数据响应式，

所以在 `Vue2` 中，增加了 `set`、`delete` API，并且对数组 `api` 方法进行一个重写

还有一个问题则是，如果存在深层的嵌套对象关系，需要深层的进行监听，造成了性能的极大问题

小结

- 检测不到对象属性的添加和删除
- 数组 [API](#) 方法无法监听到
- 需要对每个属性进行遍历监听，如果嵌套对象，需要深层监听，造成性能问题

二、proxy

[Proxy](#) 的监听是针对一个对象的，那么对这个对象的所有操作会进入监听操作，这就完全可以代理所有属性了

在 [ES6](#) 系列中，我们详细讲解过 [Proxy](#) 的使用，就不再述说了

下面通过代码进行展示：

定义一个响应式方法 [reactive](#)

```
function reactive(obj) {
  if (typeof obj !== 'object' && obj !== null) {
    return obj
  }
  // Proxy相当于在对象外层加拦截
  const observed = new Proxy(obj, {
    get(target, key, receiver) {
      const res = Reflect.get(target, key, receiver)
      console.log(`获取${key}:${res}`)
      return res
    },
    set(target, key, value, receiver) {
      const res = Reflect.set(target, key, value, receiver)
      console.log(`设置${key}:${value}`)
      return res
    },
    deleteProperty(target, key) {
      const res = Reflect.deleteProperty(target, key)
      console.log(`删除${key}:${res}`)
      return res
    }
  })
  return observed
}
```

测试一下简单数据的操作，发现都能劫持


```
const state = reactive({
  foo: 'foo'
})
// 1. 获取
state.foo // ok
// 2. 设置已存在属性
state.foo = 'fooooooo' // ok
// 3. 设置不存在属性
state.dong = 'dong' // ok
// 4. 删除属性
delete state.dong // ok
```

再测试嵌套对象情况，这时候发现就不那么 OK 了

```
const state = reactive({
  bar: { a: 1 }
})

// 设置嵌套对象属性
state.bar.a = 10 // no ok
```

如果要解决，需要在 `get` 之上再进行一层代理

```
function reactive(obj) {
  if (typeof obj !== 'object' && obj !== null) {
    return obj
  }
  // Proxy 相当于在对象外层加拦截
  const observed = new Proxy(obj, {
    get(target, key, receiver) {
      const res = Reflect.get(target, key, receiver)
      console.log(`获取${key}:${res}`)
      return isObject(res) ? reactive(res) : res
    },
  })
  return observed
}
```

三、总结

`Object.defineProperty` 只能遍历对象属性进行劫持

```
function observe(obj) {
  if (typeof obj !== 'object' || obj === null) {
    return
  }
  Object.keys(obj).forEach(key => {
    defineReactive(obj, key, obj[key])
  })
}
```

`Proxy` 直接可以劫持整个对象，并返回一个新对象，我们可以只操作新的对象达到响应式目的

```
function reactive(obj) {
  if (typeof obj !== 'object' && obj !== null) {
    return obj
  }
  // Proxy相当于在对象外层加拦截
  const observed = new Proxy(obj, {
    get(target, key, receiver) {
      const res = Reflect.get(target, key, receiver)
      console.log(`获取${key}:${res}`)
      return res
    },
    set(target, key, value, receiver) {
      const res = Reflect.set(target, key, value, receiver)
      console.log(`设置${key}:${value}`)
      return res
    },
    deleteProperty(target, key) {
      const res = Reflect.deleteProperty(target, key)
      console.log(`删除${key}:${res}`)
      return res
    }
  })
  return observed
}
```

`Proxy` 可以直接监听数组的变化（`push`、`shift`、`splice`）

```
const obj = [1,2,3]
const proxObj = reactive(obj)
obj.push(4) // ok
```

`Proxy` 有多达13种拦截方法,不限于 `apply`、`ownKeys`、`deleteProperty`、`has` 等等，这是 `Object.defineProperty` 不具备的

正因为 `defineProperty` 自身的缺陷，导致 `Vue2` 在实现响应式过程需要实现其他的方法辅助（如重写数组方法、增加额外 `set`、`delete` 方法）

```
// 数组重写
const originalProto = Array.prototype
const arrayProto = Object.create(originalProto)
['push', 'pop', 'shift', 'unshift', 'splice', 'reverse', 'sort'].forEach(method => {
  arrayProto[method] = function () {
    originalProto[method].apply(this.arguments)
    dep.notice()
  }
});

// set、delete
Vue.set(obj, 'bar', 'newbar')
Vue.delete(obj, 'bar')
```

`Proxy` 不兼容IE，也没有 `polyfill`，`defineProperty` 能支持到IE9

参考文献

- https://developer.mozilla.org/zh-CN/docs/Web/JavaScript/Reference/Global_Objects/Object/defineProperty

面试官：用Vue3.0 写过组件吗？如果想实现一个 Modal你会怎么设计？

一、组件设计

组件就是把图形、非图形的各种逻辑均抽象为一个统一的概念（组件）来实现开发的模式

现在有一个场景，点击新增与编辑都弹框出来进行填写，功能上大同小异，可能只是标题内容或者是显示的主体内容稍微不同

这时候就没必要写两个组件，只需要根据传入的参数不同，组件显示不同内容即可

这样，下次开发相同界面程序时就可以写更少的代码，意味着更高的开发效率，更少的 `Bug` 和更少的程序体积

二、需求分析

实现一个 `Modal` 组件，首先确定需要完成的内容：

- 遮罩层
- 标题内容
- 主体内容
- 确定和取消按钮

主体内容需要灵活，所以可以是字符串，也可以是一段 `html` 代码

特点是它们在当前 `vue` 实例之外独立存在，通常挂载于 `body` 之上

除了通过引入 `import` 的形式，我们还可通过 `API` 的形式进行组件的调用

还可以包括配置全局样式、国际化、与 `typescript` 结合

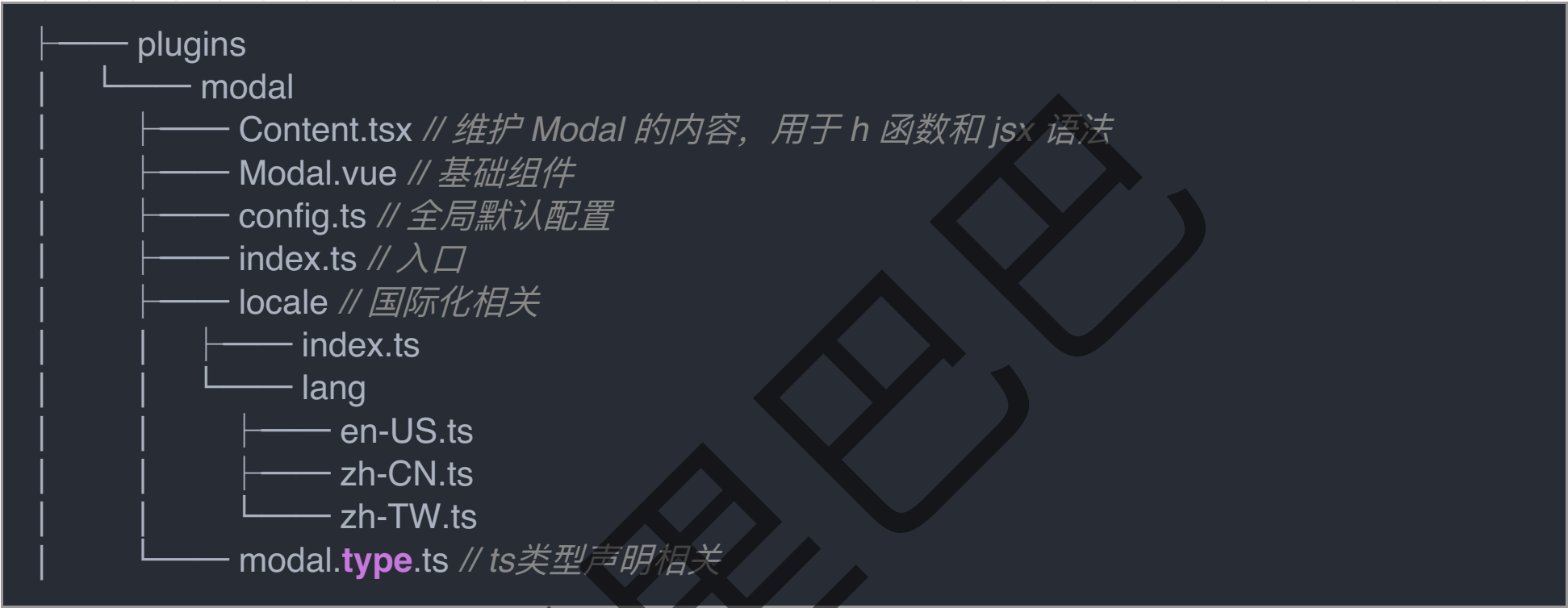
三、实现流程

首先看看大致流程：

- 目录结构
- 组件内容
- 实现 API 形式
- 事件处理
- 其他完善

目录结构

Modal 组件相关的目录结构



因为 Modal 会被 `app.use(Modal)` 调用作为一个插件，所以都放在 `plugins` 目录下

组件内容

首先实现 `modal.vue` 的主体显示内容大致如下

```

<Teleport to="body" :disabled="!isTeleport">
  <div v-if="modelValue" class="modal">
    <div
      class="mask"
      :style="style"
      @click="maskClose && !loading && handleCancel()"
    ></div>
    <div class="modal__main">
      <div class="modal__title line line--b">
        <span>{{ title || t('r.title') }}</span>
        <span
          v-if="close"
          :title="t('r.close')"
          class="close"
          @click="!loading && handleCancel()"
        >x</span>
      </div>
      <div class="modal__content">
        <Content v-if="typeof content === 'function'" :render="content" />
        <slot v-else>
          {{ content }}
        </slot>
      </div>
      <div class="modal__btns line line--t">
        <button :disabled="loading" @click="handleConfirm">
          <span class="loading" v-if="loading">○</span>{{ t('r.confirm') }}
        </button>
        <button @click="!loading && handleCancel()">
          {{ t('r.cancel') }}
        </button>
      </div>
    </div>
  </div>
</Teleport>

```

最外层上通过Vue3 `Teleport` 内置组件进行包裹，其相当于传送门，将里面的内容传送至 `body` 之上并且从 `DOM` 结构上来看，把 `modal` 该有的内容（遮罩层、标题、内容、底部按钮）都实现了关于主体内容

```

<div class="modal__content">
  <Content v-if="typeof content==='function'"
    :render="content" />
  <slot v-else>
    {{content}}
  </slot>
</div>

```

可以看到根据传入 `content` 的类型不同，对应显示不同得到内容
最常见的则是通过调用字符串和默认插槽的形式

```
// 默认插槽
<Modal v-model="show"
  title="演示 slot">
  <div>hello world~</div>
</Modal>

// 字符串
<Modal v-model="show"
  title="演示 content"
  content="hello world~" />
```

通过 API 形式调用 `Modal` 组件的时候，`content` 可以使用下面两种

- h 函数

```
$modal.show({
  title: '演示 h 函数',
  content(h) {
    return h(
      'div',
      {
        style: 'color:red;',
        onClick: ($event: Event) => console.log('clicked', $event.target)
      },
      'hello world ~'
    );
  }
});
```

- JSX

```
$modal.show({
  title: '演示 jsx 语法',
  content() {
    return (
      <div
        onClick=({$event: Event}) => console.log('clicked', $event.target)}
      >
        hello world ~
      </div>
    );
  }
});
```

实现 API 形式

那么组件如何实现 API 形式调用 `Modal` 组件呢？

在 `Vue2` 中，我们可以借助 `Vue` 实例以及 `Vue.extend` 的方式获得组件实例，然后挂载到 `body` 上


```
import Modal from './Modal.vue';
const ComponentClass = Vue.extend(Modal);
const instance = new ComponentClass({ el: document.createElement("div") });
document.body.appendChild(instance.$el);
```

虽然 `Vue3` 移除了 `Vue.extend` 方法，但可以通过 `createVNode` 实现

```
import Modal from './Modal.vue';
const container = document.createElement('div');
const vnode = createVNode(Modal);
render(vnode, container);
const instance = vnode.component;
document.body.appendChild(container);
```

在 `Vue2` 中，可以通过 `this` 的形式调用全局 API

```
export default {
  install(vue) {
    vue.prototype.$create = create
  }
}
```

而在 `Vue3` 的 `setup` 中已经没有 `this` 概念了，需要调用 `app.config.globalProperties` 挂载到全局

```
export default {
  install(app) {
    app.config.globalProperties.$create = create
  }
}
```

事件处理

下面再看看 `Modal` 组件内部是如何处理「确定」「取消」事件的，既然是 `Vue3`，当然采用 `Compositon API` 形式

```
// Modal.vue
setup(props, ctx) {
  let instance = getCurrentInstance(); // 获得当前组件实例
  onBeforeMount(() => {
    instance._hub = {
      'on-cancel': () => {},
      'on-confirm': () => {}
    };
  });

  const handleConfirm = () => {
    ctx.emit('on-confirm');
    instance._hub['on-confirm']();
  };
  const handleCancel = () => {
    ctx.emit('on-cancel');
    ctx.emit('update:modelValue', false);
    instance._hub['on-cancel']();
  };

  return {
    handleConfirm,
    handleCancel
  };
}
```

在上面代码中，可以看得到除了使用传统 `emit` 的形式使父组件监听，还可通过 `_hub` 属性中添加 `on-cancel`，`on-confirm` 方法实现在 `API` 中进行监听

```
app.config.globalProperties.$modal = {
  show({}) {
    /* 监听 确定、取消 事件 */
  }
}
```

下面再来目睹下 `_hub` 是如何实现

```

// index.ts
app.config.globalProperties.$modal = {
  show({
    /* 其他选项 */
    onConfirm,
    onCancel
  }) {
    /* ... */

    const { props, _hub } = instance;

    const _closeModal = () => {
      props.modelValue = false;
      container.parentNode!.removeChild(container);
    };
    // 往 _hub 新增事件的具体实现
    Object.assign(_hub, {
      async 'on-confirm'() {
        if (onConfirm) {
          const fn = onConfirm();
          // 当方法返回为 Promise
          if (fn && fn.then) {
            try {
              props.loading = true;
              await fn;
              props.loading = false;
              _closeModal();
            } catch (err) {
              // 发生错误时，不关闭弹框
              console.error(err);
              props.loading = false;
            }
          } else {
            _closeModal();
          }
        } else {
          _closeModal();
        }
      },
      'on-cancel'() {
        onCancel && onCancel();
        _closeModal();
      }
    });
  }
};

```

其他完善

关于组件实现国际化、与 [typescript](#) 结合，大家可以根据自身情况在此基础上进行更改

参考文献

◦ <https://segmentfault.com/a/11900000038928664>

面试官：说说Vue 3.0中Treeshaking特性？举例说明一下？

一、Tree shaking

Tree shaking 是一种通过清除多余代码方式来优化项目打包体积的技术，专业术语叫 **Dead code elimination**

简单来讲，就是在保持代码运行结果不变的前提下，去除无用的代码

如果把代码打包比作制作蛋糕，传统的方式是把鸡蛋（带壳）全部丢进去搅拌，然后放入烤箱，最后把（没有用的）蛋壳全部挑选并剔除出去

而 **treeshaking** 则是一开始就把有用的蛋白蛋黄（import）放入搅拌，最后直接作出蛋糕

也就是说，**tree shaking** 其实是找出使用的代码

在 **Vue2** 中，无论我们使用什么功能，它们最终都会出现在生产代码中。主要原因是 **Vue** 实例在项目中是单例的，捆绑程序无法检测到该对象的哪些属性在代码中被使用到

```
import Vue from 'vue'

Vue.nextTick(() => {})
```

而 **Vue3** 源码引入 **tree shaking** 特性，将全局 API 进行分块。如果您不使用其某些功能，它们将不会包含在您的基础包中

```
import { nextTick, observable } from 'vue'

nextTick(() => {})
```

二、如何做

Tree shaking 是基于 **ES6** 模板语法（**import** 与 **exports**），主要是借助 **ES6** 模块的静态编译思想，在编译时就能确定模块的依赖关系，以及输入和输出的变量

Tree shaking 无非就是做了两件事：

- 编译阶段利用 **ES6 Module** 判断哪些模块已经加载
- 判断那些模块和变量未被使用或者引用，进而删除对应代码

下面就来举个例子：

通过脚手架 **vue-cli** 安装 **Vue2** 与 **Vue3** 项目

```
vue create vue-demo
```

Vue2 项目

组件中使用 **data** 属性

```
<script>
  export default {
    data: () => ({
      count: 1,
    }),
  };
</script>
```

对项目进行打包，体积如下图

DONE Compiled successfully in 18393ms

File	Size	Gzipped
dist\js\chunk-vendors.28d0d835.js	89.59 KiB	32.11 KiB
dist\js\app.3f482fc1.js	2.01 KiB	1.01 KiB

Images and other types of assets omitted.

为组件设置其他属性（`computed`、`watch`）

```
export default {
  data: () => ({
    question: '',
    count: 1,
  }),
  computed: {
    double: function () {
      return this.count * 2;
    },
  },
  watch: {
    question: function (newQuestion, oldQuestion) {
      this.answer = 'xxx';
    }
  }
};
```

再一次打包，发现打包出来的体积并没有变化

Vue3 项目

组件中简单使用

```
import { reactive, defineComponent } from "vue";
export default defineComponent({
  setup() {
    const state = reactive({
      count: 1,
    });
    return {
      state,
    };
  },
});
```

将项目进行打包

在组件中引入 `computed` 和 `watch`

```
import { reactive, defineComponent, computed, watch } from "vue";
export default defineComponent({
  setup() {
    const state = reactive({
      count: 1,
    });
    const double = computed(() => {
      return state.count * 2;
    });

    watch(
      () => state.count,
      (count, preCount) => {
        console.log(count);
        console.log(preCount);
      }
    );
    return {
      state,
      double,
    };
  },
});
```

再次对项目进行打包，可以看到在引入 `computer` 和 `watch` 之后，项目整体体积变大了

三、作用

通过 `Tree shaking`，`Vue3` 给我们带来的好处是：

- 减少程序体积（更小）
- 减少程序执行时间（更快）
- 便于将来对程序架构进行优化（更友好）

参考文献

- <https://segmentfault.com/a/11900000038962700>

面试官：与webpack类似的工具还有哪些？区别？

一、模块化工具

模块化是一种处理复杂系统分解为更好的可管理模块的方式

可以用来分割，组织和打包应用。每个模块完成一个特定的子功能，所有的模块按某种方法组装起来，成为一个整体([bundle](#))

在前端领域中，并非只有[webpack](#)这一款优秀的模块打包工具，还有其他类似的工具，例如[Rollup](#)、[Parcel](#)、[snowpack](#)，以及最近风头无两的[Vite](#)

通过这些模块打包工具，能够提高我们的开发效率，减少开发成本

这里没有提及[gulp](#)、[grunt](#)是因为它们只是定义为构建工具，不能类比

Rollup

[Rollup](#) 是一款 [ES Modules](#) 打包器，从作用上来看，[Rollup](#) 与 [Webpack](#) 非常类似。不过相比于 [Webpack](#)，[Rollup](#) 要小巧的多

现在很多我们熟知的库都使用它进行打包，比如：[Vue](#)、[React](#) 和 [three.js](#) 等

举个例子：

```
// ./src/messages.js
export default {
  hi: 'Hey Guys, I am zce~'
}

// ./src/logger.js
export const log = msg => {
  console.log('----- INFO -----')
  console.log(msg)
  console.log('-----')
}

export const error = msg => {
  console.error('----- ERROR -----')
  console.error(msg)
  console.error('-----')
}

// ./src/index.js
import { log } from './logger'
import messages from './messages'
log(messages.hi)
```

然后通过 [rollup](#) 进行打包

```
$ npx rollup ./src/index.js --file ./dist/bundle.js
```

打包结果如下图

可以看到，代码非常简洁，完成不像 `webpack` 那样存在大量引导代码和模块函数

并且 `error` 方法由于没有被使用，输出的结果中并无 `error` 方法，可以看到，`rollup` 默认开始 `Tree-shaking` 优化输出结果

因此，可以看到 `Rollup` 的优点：

- 代码效率更简洁、效率更高
- 默认支持 `Tree-shaking`

但缺点也十分明显，加载其他类型的资源文件或者支持导入 `CommonJS` 模块，又或是编译 `ES` 新特性，这些额外的需求 `Rollup` 需要使用插件去完成

综合来看，`rollup` 并不适合开发应用使用，因为需要使用第三方模块，而目前第三方模块大多数使用 `CommonJs` 方式导出成员，并且 `rollup` 不支持 `HMR`，使开发效率降低

但是在用于打包 `JavaScript` 库时，`rollup` 比 `webpack` 更有优势，因为其打包出来的代码更小、更快，其存在的缺点可以忽略

Parcel

`Parcel`，是一款完全零配置的前端打包器，它提供了“傻瓜式”的使用体验，只需了解简单的命令，就能构建前端应用程序

`Parcel` 跟 `Webpack` 一样都支持以任意类型文件作为打包入口，但建议使用 `HTML` 文件作为入口，该 `HTML` 文件像平时一样正常编写代码、引用资源。如下所示：

```
<!-- ./src/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Parcel Tutorials</title>
</head>
<body>
  <script src="main.js"></script>
</body>
</html>
```

`main.js` 文件通过 `ES Module` 方法导入其他模块成员

```
// ./src/main.js
import { log } from './logger'
log('hello parcel')
// ./src/logger.js
export const log = msg => {
  console.log('----- INFO -----')
  console.log(msg)
}
```

运行之后，使用命令打包

```
npx parcel src/index.html
```

执行命令后，[Parcel](#) 不仅打包了应用，同时也启动了一个开发服务器，跟 [webpack Dev Server](#) 一样

跟 [webpack](#) 类似，也支持模块热替换，但用法更简单

同时，[Parcel](#) 有个十分好用的功能：支持自动安装依赖，像 [webpack](#) 开发阶段突然使用安装某个第三方依赖，必然会终止 [dev server](#) 然后安装再启动。而 [Parcel](#) 则免了这繁琐的工作流程

同时，[Parcel](#) 能够零配置加载其他类型的资源文件，无须像 [webpack](#) 那样配置对应的 [loader](#)

打包命令如下：

```
npx parcel src/index.html
```

由于打包过程是多进程同时工作，构建速度会比 [Webpack](#) 快，输出文件也会被压缩，并且样式代码也会被单独提取到单个文件中

可以感受到，[Parcel](#) 给开发者一种很大的自由度，只管去实现业务代码，其他事情用 [Parcel](#) 解决

Snowpack

Snowpack，是一种闪电般快速的前端构建工具，专为现代 [Web](#) 设计，较复杂的打包工具（如 [Webpack](#) 或 [Parcel](#)）的替代方案，利用 [JavaScript](#) 的本机模块系统，避免不必要的工作并保持流畅的开发体验

开发阶段，每次保存单个文件时，[Webpack](#) 和 [Parcel](#) 都需要重新构建和重新打包应用程序的整个 [bundle](#)。而 [Snowpack](#) 为你的应用程序每个文件构建一次，就可以永久缓存，文件更改时，[Snowpack](#) 会重新构建该单个文件

下图给出 [webpack](#) 与 [snowpack](#) 打包区别：

在重新构建每次变更时没有任何的时间浪费，只需要在浏览器中进行 HMR 更新

Vite

vite，是一种新型前端构建工具，能够显著提升前端开发体验

它主要由两部分组成：

- 一个开发服务器，它基于原生 ES 模块 提供了丰富的内建功能，如速度快到惊人的 [模块热

更新HMR

- 一套构建指令，它使用 Rollup打包你的代码，并且它是预配置的，可以输出用于生产环境的优化过的静态资源

其作用类似 `webpack` + `webpack-dev-server`，其特点如下：

- 快速的冷启动
- 即时的模块热更新
- 真正的按需编译

`vite` 会直接启动开发服务器，不需要进行打包操作，也就意味着不需要分析模块的依赖、不需要编译，因此启动速度非常快

利用现代浏览器支持 `ES Module` 的特性，当浏览器请求某个模块的时候，再根据需要对模块的内容进行编译，这种方式大大缩短了编译时间

原理图如下所示：

在热模块 `HMR` 方面，当修改一个模块的时候，仅需让浏览器重新请求该模块即可，无须像 `webpack` 那样需要把该模块的相关依赖模块全部编译一次，效率更高

webpack

相比上述的模块化工具，`webpack` 大而全，很多常用的功能做到开箱即用。有两大最核心的特点：
「一切皆模块」和「按需加载」

与其他构建工具相比，有如下优势：

- 智能解析：对 CommonJS、AMD、ES6 的语法做了兼容
- 万物模块：对 js、css、图片等资源文件都支持打包
- 开箱即用：HRM、Tree-shaking 等功能
- 代码分割：可以将代码切割成不同的 chunk，实现按需加载，降低了初始化时间
- 插件系统，具有强大的 Plugin 接口，具有更好的灵活性和扩展性
- 易于调试：支持 SourceUrls 和 SourceMaps
- 快速运行：`webpack` 使用异步 IO 并具有多级缓存，这使得 `webpack` 很快且在增量编译上更加快
- 生态环境好：社区更丰富，出现的问题更容易解决

参考文献

- <https://zhuanlan.zhihu.com/p/95684686>
- <https://cn.vitejs.dev/guide/>
- <https://segmentfault.com/a/1190000039370642>

面试官：如何提高webpack的构建速度？

一、背景

随着我们的项目涉及到页面越来越多，功能和业务代码也会随着越多，相应的 `webpack` 的构建时间也会越来越久

构建时间与我们日常开发效率密切相关，当我们本地开发启动 `devServer` 或者 `build` 的时候，如果时间过长，会大大降低我们的工作效率

所以，优化 `webpack` 构建速度是十分重要的环节

二、如何优化

常见的提升构建速度的手段有如下：

- 优化 loader 配置
- 合理使用 resolve.extensions
- 优化 resolve.modules
- 优化 resolve.alias
- 使用 DLLPlugin 插件
- 使用 cache-loader
- terser 启动多线程
- 合理使用 sourceMap

优化loader配置

在使用 loader 时，可以通过配置 `include`、`exclude`、`test` 属性来匹配文件，接触 `include`、`exclude` 规定哪些匹配应用 loader

如采用 ES6 的项目为例，在配置 `babel-loader` 时，可以这样：

```
module.exports = {
  module: {
    rules: [
      {
        // 如果项目源码中只有 js 文件就不要写成 /\.jsx?$/, 提升正则表达式性能
        test: /\.js$/,
        // babel-loader 支持缓存转换出的结果，通过 cacheDirectory 选项开启
        use: ['babel-loader?cacheDirectory'],
        // 只对项目根目录下的 src 目录中的文件采用 babel-loader
        include: path.resolve(__dirname, 'src'),
      },
    ],
  },
};
```

合理使用 resolve.extensions

在开发中我们会有各种各样的模块依赖，这些模块可能来自于自己编写的代码，也可能来自第三方库，`resolve` 可以帮助 `webpack` 从每个 `require/import` 语句中，找到需要引入到合适的模块代码

通过 `resolve.extensions` 是解析到文件时自动添加拓展名，默认情况如下：

```
module.exports = {
  ...
  extensions:['.warm','.mjs','.js','.json']
}
```

当我们引入文件的时候，若没有文件后缀名，则会根据数组内的值依次查找

当我们配置的时候，则不要随便把所有后缀都写在里面，这会调用多次文件的查找，这样就会减慢打包速度

优化 resolve.modules

`resolve.modules` 用于配置 `webpack` 去哪些目录下寻找第三方模块。默认值为 `['node_modules']`，所以默认会从 `node_modules` 中查找文件

当安装的第三方模块都放在项目根目录下的 `./node_modules` 目录下时，所以可以指明存放第三方模块的绝对路径，以减少寻找，配置如下：

```
module.exports = {
  resolve: {
    // 使用绝对路径指明第三方模块存放的位置，以减少搜索步骤
    // 其中 __dirname 表示当前工作目录，也就是项目根目录
    modules: [path.resolve(__dirname, 'node_modules')]
  },
};
```

优化 `resolve.alias`

`alias` 给一些常用的路径起一个别名，特别当我们的项目目录结构比较深的时候，一个文件的路径可能是 `./.././` 的形式

通过配置 `alias` 以减少查找过程

```
module.exports = {
  ...
  resolve: {
    alias: {
      "@": path.resolve(__dirname, './src')
    }
  }
}
```

使用 `DLLPlugin` 插件

`DLL` 全称是 动态链接库，是为软件在 windows 种实现共享函数库的一种实现方式，而 `Webpack` 也内置了 `DLL` 的功能，为的就是可以共享，不经常改变的代码，抽成一个共享的库。这个库在之后的编译过程中，会被引入到其他项目的代码中

使用步骤分成两部分：

- 打包一个 `DLL` 库
- 引入 `DLL` 库

打包一个 `DLL` 库

`webpack` 内置了一个 `DllPlugin` 可以帮助我们打包一个 `DLL` 的库文件


```

module.exports = {
  ...
  plugins:[
    new webpack.DllPlugin({
      name:'dll_[name]',
      path:path.resolve(__dirname,"./dll/[name].manifest.json")
    })
  ]
}

```

引入 DLL 库

使用 `webpack` 自带的 `DllReferencePlugin` 插件对 `manifest.json` 映射文件进行分析，获取要使用的 `DLL` 库

然后再通过 `AddAssetHtmlPlugin` 插件，将我们打包的 `DLL` 库引入到 `Html` 模块中

```

module.exports = {
  ...
  new webpack.DllReferencePlugin({
    context:path.resolve(__dirname,"./dll/dll_react.js"),
    manifest:path.resolve(__dirname,"./dll/react.manifest.json")
  }),
  new AddAssetHtmlPlugin({
    outputPath:"./auto",
    filepath:path.resolve(__dirname,"./dll/dll_react.js")
  })
}

```

使用 cache-loader

在一些性能开销较大的 `loader` 之前添加 `cache-loader`，以将结果缓存到磁盘里，显著提升二次构建速度

保存和读取这些缓存文件会有一些时间开销，所以请只对性能开销较大的 `loader` 使用此 `loader`

```

module.exports = {
  module: {
    rules: [
      {
        test: /\.ext$/,
        use: ['cache-loader', ...loaders],
        include: path.resolve('src'),
      },
    ],
  },
};

```

terser 启动多线程

使用多进程并行运行来提高构建速度

```
module.exports = {
  optimization: {
    minimizer: [
      new TerserPlugin({
        parallel: true,
      }),
    ],
  },
};
```

合理使用 sourceMap

打包生成 `sourceMap` 的时候，如果信息越详细，打包速度就会越慢。对应属性取值如下所示：

devtool	构建速度	重新构建速度	生产环境	品质(quality)
(none)	+++	+++	yes	打包后的代码
eval	+++	+++	no	生成后的代码
cheap-eval-source-map	+	++	no	转换过的代码（仅限行）
cheap-module-eval-source-map	o	++	no	原始源代码（仅限行）
eval-source-map	--	+	no	原始源代码
cheap-source-map	+	o	yes	转换过的代码（仅限行）
cheap-module-source-map	o	-	yes	原始源代码（仅限行）
inline-cheap-source-map	+	o	no	转换过的代码（仅限行）
inline-cheap-module-source-map	o	-	no	原始源代码（仅限行）
source-map	--	--	yes	原始源代码
inline-source-map	--	--	no	原始源代码
hidden-source-map	--	--	yes	原始源代码
nosources-source-map	--	--	yes	无源代码内容

+++ 非常快速 ++ 快速 + 比较快 o 中等 - 比较慢 -- 慢

三、总结

可以看到，优化 `webpack` 构建的方式有很多，主要可以从优化搜索时间、缩小文件搜索范围、减少不必要的编译等方面入手

参考文献

- <https://github.com/ly2011/blog/issues/44>
- <https://xie.infoq.cn/article/541418eb82a674741a0ad8865>
- <https://zhuanlan.zhihu.com/p/139498741>

面试官：说说Loader和Plugin的区别？编写Loader，Plugin的思路？

一、区别

前面两节我们有提到 `Loader` 与 `Plugin` 对应的概念，先回顾下

- `loader` 是文件加载器，能够加载资源文件，并对这些文件进行一些处理，诸如编译、压缩等，最终一起打包到指定的文件中
- `plugin` 赋予了 `webpack` 各种灵活的功能，例如打包优化、资源管理、环境变量注入等，目的是解决 `loader` 无法实现的其他事

从整个运行时机上来看，如下图所示：

可以看到，两者在运行时机上的区别：

- `loader` 运行在打包文件之前
- `plugins` 在整个编译周期都起作用

在 `Webpack` 运行的生命周期中会广播出许多事件，`Plugin` 可以监听这些事件，在合适的时机通过 `Webpack` 提供的 `API` 改变输出结果

对于 `loader`，实质是一个转换器，将A文件进行编译形成B文件，操作的是文件，比如将 `A.scss` 或 `A.less` 转变为 `B.css`，单纯的文件转换过程

二、编写loader

在编写 `loader` 前，我们首先需要了解 `loader` 的本质

其本质为函数，函数中的 `this` 作为上下文会被 `webpack` 填充，因此我们不能将 `loader` 设为一个箭头函数

函数接受一个参数，为 `webpack` 传递给 `loader` 的文件源内容

函数中 `this` 是由 `webpack` 提供的对象，能够获取当前 `loader` 所需要的各种信息

函数中有异步操作或同步操作，异步操作通过 `this.callback` 返回，返回值要求为 `string` 或者 `Buffer`

代码如下所示：

```
// 导出一个函数，source为webpack传递给loader的文件源内容
module.exports = function(source) {
  const content = doSomething2JsString(source);

  // 如果 loader 配置了 options 对象，那么this.query将指向 options
  const options = this.query;

  // 可以用作解析其他模块路径的上下文
  console.log(this.context);

  /**
   * this.callback 参数:
   * error: Error | null, 当 loader 出错时向外抛出一个 error
   * content: String | Buffer, 经过 loader 编译后需要导出的内容
   * sourceMap: 为方便调试生成的编译后内容的 source map
   * ast: 本次编译生成的 AST 静态语法树，之后执行的 loader 可以直接使用这个 AST，进而省去
   重复生成 AST 的过程
   */
  this.callback(null, content); // 异步
  return content; // 同步
}
```

一般在编写 loader 的过程中，保持功能单一，避免做多种功能

如 less 文件转换成 css 文件也不是一步到位，而是 less-loader、css-loader、style-loader 几个 loader 的链式调用才能完成转换

三、编写 plugin

由于 webpack 基于发布订阅模式，在运行的生命周期中会广播出许多事件，插件通过监听这些事件，就可以在特定的阶段执行自己的插件任务

在之前也了解过，webpack 编译会创建两个核心对象：

- compiler：包含了 webpack 环境的所有的配置信息，包括 options，loader 和 plugin，和 webpack 整个生命周期相关的钩子
- compilation：作为 plugin 内置事件回调函数的参数，包含了当前的模块资源、编译生成资源、变化的文件以及被跟踪依赖的状态信息。当检测到一个文件变化，一次新的 Compilation 将被创建

如果自己要实现 plugin，也需要遵循一定的规范：

- 插件必须是一个函数或者是一个包含 apply 方法的对象，这样才能访问 compiler 实例
- 传给每个插件的 compiler 和 compilation 对象都是同一个引用，因此不建议修改
- 异步的事件需要在插件处理完任务时调用回调函数通知 Webpack 进入下一个流程，不然会卡住

实现 plugin 的模板如下：


```
class MyPlugin {
  // Webpack 会调用 MyPlugin 实例的 apply 方法给插件实例传入 compiler 对象
  apply (compiler) {
    // 找到合适的事件钩子，实现自己的插件功能
    compiler.hooks.emit.tap('MyPlugin', compilation => {
      // compilation: 当前打包构建流程的上下文
      console.log(compilation);

      // do something...
    })
  }
}
```

在 `emit` 事件发生时，代表源文件的转换和组装已经完成，可以读取到最终将输出的资源、代码块、模块及其依赖，并且可以修改输出资源的内容

参考文献

- <https://webpack.docschina.org/api/loaders/>
- <https://webpack.docschina.org/api/compiler-hooks/>
- <https://segmentfault.com/a/1190000039877943>

面试官：说说webpack proxy工作原理？为什么能解决跨域？

一、什么是webpack proxy

`webpack proxy`，即 `webpack` 提供的代理服务

基本行为就是接收客户端发送的请求后转发给其他服务器

其目的是为了便于开发者在开发模式下解决跨域问题（浏览器安全策略限制）

想要实现代理首先需要一个中间服务器，`webpack` 中提供服务器的工具为 `webpack-dev-server`

webpack-dev-server

`webpack-dev-server` 是 `webpack` 官方推出的一款开发工具，将自动编译和自动刷新浏览器等一系列对开发友好的功能全部集成在了一起

目的是为了提高开发者日常的开发效率，「只适用在开发阶段」

关于配置方面，在 `webpack` 配置对象属性中通过 `devServer` 属性提供，如下：

```
// ./webpack.config.js
const path = require('path')

module.exports = {
  // ...
  devServer: {
    contentBase: path.join(__dirname, 'dist'),
    compress: true,
    port: 9000,
    proxy: {
      '/api': {
        target: 'https://api.github.com'
      }
    }
  },
  // ...
}
```

`devServer` 里面 `proxy` 则是关于代理的配置，该属性为对象的形式，对象中每一个属性就是一个代理的规则匹配

属性的名称是需要被代理的请求路径前缀，一般为了辨别都会设置前缀为 `/api`，值为对应的代理匹配规则，对应如下：

- `target`：表示的是代理到的目标地址
- `pathRewrite`：默认情况下，我们的 `/api-hy` 也会被写入到URL中，如果希望删除，可以使用 `pathRewrite`
- `secure`：默认情况下不接收转发到https的服务器上，如果希望支持，可以设置为 `false`
- `changeOrigin`：它表示是否更新代理后请求的 `headers` 中 `host` 地址

二、工作原理

`proxy` 工作原理实质上是利用 `http-proxy-middleware` 这个 `http` 代理中间件，实现请求转发给其他服务器

举个例子：

在开发阶段，本地地址为 `http://localhost:3000`，该浏览器发送一个前缀带有 `/api` 标识的请求到服务端获取数据，但响应这个请求的服务器只是将请求转发到另一台服务器中

```
const express = require('express');
const proxy = require('http-proxy-middleware');

const app = express();

app.use('/api', proxy({target: 'http://www.example.org', changeOrigin: true}));
app.listen(3000);

// http://localhost:3000/api/foo/bar -> http://www.example.org/api/foo/bar
```

三、跨域

在开发阶段，`webpack-dev-server` 会启动一个本地开发服务器，所以我们的应用在开发阶段是独立运行在 `localhost` 的一个端口上，而后端服务又是运行在另外一个地址上

所以在开发阶段中，由于浏览器同源策略的原因，当本地访问后端就会出现跨域请求的问题

通过设置 `webpack proxy` 实现代理请求后，相当于浏览器与服务端中添加一个代理者

当本地发送请求的时候，代理服务器响应该请求，并将请求转发到目标服务器，目标服务器响应数据后再将数据返回给代理服务器，最终再由代理服务器将数据响应给本地

在代理服务器传递数据给本地浏览器的过程中，两者同源，并不存在跨域行为，这时候浏览器就能正常接收数据

注意：「服务器与服务器之间请求数据并不会存在跨域行为，跨域行为是浏览器安全策略限制」

参考文献

- <https://webpack.docschina.org/configuration/dev-server/#devserverproxy>

面试官：说说webpack中常见的Loader？解决了什么问题？

一、是什么

`loader` 用于对模块的"源代码"进行转换，在 `import` 或"加载"模块时预处理文件

`webpack` 做的事情，仅仅是分析出各种模块的依赖关系，然后形成资源列表，最终打包生成到指定的文件中。如下图所示：

在 `webpack` 内部中，任何文件都是模块，不仅仅只是 `js` 文件

默认情况下，在遇到 `import` 或者 `require` 加载模块的时候，`webpack` 只支持对 `js` 和 `json` 文件打包

像 `css`、`sass`、`png` 等这些类型的文件的时候，`webpack` 则无能为力，这时候就需要配置对应的 `loader` 进行文件内容的解析

在加载模块的时候，执行顺序如下：

当 `webpack` 碰到不识别的模块的时候，`webpack` 会在配置的中查找该文件解析规则

关于配置 `loader` 的方式有三种：

- 配置方式（推荐）：在 `webpack.config.js` 文件中指定 `loader`
- 内联方式：在每个 `import` 语句中显式指定 `loader`
- CLI 方式：在 `shell` 命令中指定它们

配置方式

关于 `loader` 的配置，我们是写在 `module.rules` 属性中，属性介绍如下：

- `rules` 是一个数组的形式，因此我们可以配置很多个 `loader`
- 每一个 `loader` 对应一个对象的形式，对象属性 `test` 为匹配的规则，一般情况为正则表达式
- 属性 `use` 针对匹配到文件类型，调用对应的 `loader` 进行处理

代码编写，如下形式：

```
module.exports = {
  module: {
    rules: [
      {
        test: /\.css$/,
        use: [
          { loader: 'style-loader' },
          {
            loader: 'css-loader',
            options: {
              modules: true
            }
          },
          { loader: 'sass-loader' }
        ]
      }
    ]
  }
};
```

二、特性

这里继续拿上述代码，来讲讲 `loader` 的特性

从上述代码可以看到，在处理 `css` 模块的时候，`use` 属性中配置了三个 `loader` 分别处理 `css` 文件

因为 `loader` 支持链式调用，链中的每个 `loader` 会处理之前已处理过的资源，最终变为 `js` 代码。顺序为相反的顺序执行，即上述执行方式为 `sass-loader`、`css-loader`、`style-loader`

除此之外，`loader` 的特性还有如下：

- `loader` 可以是同步的，也可以是异步的
- `loader` 运行在 `Node.js` 中，并且能够执行任何操作
- 除了常见的通过 `package.json` 的 `main` 来将一个 `npm` 模块导出为 `loader`，还可以在 `module.rules` 中使用 `loader` 字段直接引用一个模块
- 插件(plugin)可以为 `loader` 带来更多特性
- `loader` 能够产生额外的任意文件

可以通过 `loader` 的预处理函数，为 `JavaScript` 生态系统提供更多能力。用户现在可以更加灵活地引入细粒度逻辑，例如：压缩、打包、语言翻译和更多其他特性

三、常见的loader

在页面开发过程中，我们经常加载除了 `js` 文件以外的内容，这时候我们就需要配置响应的 `loader` 进行加载

常见的 `loader` 如下：

- `style-loader`: 将 `css` 添加到 `DOM` 的内联样式标签 `style` 里
- `css-loader`: 允许将 `css` 文件通过 `require` 的方式引入，并返回 `css` 代码
- `less-loader`: 处理 `less`
- `sass-loader`: 处理 `sass`

- postcss-loader: 用postcss来处理CSS
- autoprefixer-loader: 处理CSS3属性前缀，已被弃用，建议直接使用postcss
- file-loader: 分发文件到output目录并返回相对路径
- url-loader: 和file-loader类似，但是当文件小于设定的limit时可以返回一个Data Url
- html-minify-loader: 压缩HTML
- babel-loader :用babel来转换ES6文件到ES

下面给出一些常见的 loader 的使用：

css-loader

分析 `css` 模块之间的关系，并合成一个 `css`

```
npm install --save-dev css-loader
```

```
rules: [  
  ...,  
  {  
    test: /\.css$/,  
    use: {  
      loader: "css-loader",  
      options: {  
        // 启用/禁用 url() 处理  
        url: true,  
        // 启用/禁用 @import 处理  
        import: true,  
        // 启用/禁用 Sourcemap  
        sourceMap: false  
      }  
    }  
  }  
]
```

如果只通过 `css-loader` 加载文件，这时候页面代码设置的样式并没有生效

原因在于，`css-loader` 只是负责将 `.css` 文件进行一个解析，而并不会将解析后的 `css` 插入到页面中

如果我们希望再完成插入 `style` 的操作，那么我们还需要另外一个 loader，就是 `style-loader`

style-loader

把 `css-loader` 生成的内容，用 `style` 标签挂载到页面的 `head` 中

```
npm install --save-dev style-loader
```

```
rules: [  
  ...,  
  {  
    test: /\.css$/,  
    use: ["style-loader", "css-loader"]  
  }  
]
```

同一个任务的 `loader` 可以同时挂载多个，处理顺序为：从右到左，从下往上

less-loader

开发中，我们也常常会使用 `less`、`sass`、`stylus` 预处理器编写 `css` 样式，使开发效率提高，这里需要使用 `less-loader`

```
npm install less-loader -D
```

```
rules: [  
  ...,  
  {  
    test: /\.css$/,  
    use: ["style-loader", "css-loader", "less-loader"]  
  }  
]
```

raw-loader

在 `webpack` 中通过 `import` 方式导入文件内容，该 `loader` 并不是内置的，所以首先要安装

```
npm install --save-dev raw-loader
```

然后在 `webpack.config.js` 中进行配置

```
module.exports = {  
  ...,  
  module: {  
    rules: [  
      {  
        test: /\.txt$/,  
        use: 'raw-loader'  
      }  
    ]  
  }  
}
```

file-loader

把识别出的资源模块，移动到指定的输出目录，并且返回这个资源在输出目录的地址(字符串)

```
npm install --save-dev file-loader
```

```
rules: [  
  ...,  
  {  
    test: /\.?(png|jpe?g|gif)$/ ,  
    use: {  
      loader: "file-loader",  
      options: {  
        // placeholder 占位符 [name] 源资源模块的名称  
        // [ext] 源资源模块的后缀  
        name: "[name]_[hash].[ext]",  
        //打包后的存放位置  
        outputPath: "./images",  
        // 打包后文件的 url  
        publicPath: './images',  
      }  
    }  
  }  
]
```

url-loader

可以处理 `file-loader` 所有的事情，但是遇到图片格式的模块，可以选择性的把图片转成 `base64` 格式的字符串，并打包到 `js` 中，对小体积的图片比较合适，大图片不合适。

```
npm install --save-dev url-loader
```

```
rules: [  
  ...,  
  {  
    test: /\.?(png|jpe?g|gif)$/ ,  
    use: {  
      loader: "url-loader",  
      options: {  
        // placeholder 占位符 [name] 源资源模块的名称  
        // [ext] 源资源模块的后缀  
        name: "[name]_[hash].[ext]",  
        //打包后的存放位置  
        outputPath: "./images"  
        // 打包后文件的 url  
        publicPath: './images',  
        // 小于 100 字节转成 base64 格式  
        limit: 100  
      }  
    }  
  }  
]
```

参考文献

- <https://webpack.docschina.org/concepts/loaders/>
- <https://segmentfault.com/a/1190000018680530>

面试官：说说webpack中常见的Plugin？解决了什么问题？

一、是什么

Plugin (Plug-in) 是一种计算机应用程序，它和主应用程序互相交互，以提供特定的功能

是一种遵循一定规范的应用程序接口编写出来的程序，只能运行在程序规定的系统下，因为其需要调用原纯净系统提供的函数库或者数据

webpack 中的 **plugin** 也是如此，**plugin** 赋予其各种灵活的功能，例如打包优化、资源管理、环境变量注入等，它们会运行在 **webpack** 的不同阶段（钩子 / 生命周期），贯穿了 **webpack** 整个编译周期

目的在于解决 **loader** 无法实现的其他事

配置方式

这里讲述文件的配置方式，一般情况，通过配置文件导出对象中 **plugins** 属性传入 **new** 实例对象。如下所示：

```
const HtmlWebpackPlugin = require('html-webpack-plugin'); // 通过 npm 安装
const webpack = require('webpack'); // 访问内置的插件
module.exports = {
  ...
  plugins: [
    new webpack.ProgressPlugin(),
    new HtmlWebpackPlugin({ template: './src/index.html' }),
  ],
};
```

二、特性

其本质是一个具有 **apply** 方法 **javascript** 对象

apply 方法会被 **webpack compiler** 调用，并且在整个编译生命周期都可以访问 **compiler** 对象


```
const pluginName = 'ConsoleLogOnBuildWebpackPlugin';

class ConsoleLogOnBuildWebpackPlugin {
  apply(compiler) {
    compiler.hooks.run.tap(pluginName, (compilation) => {
      console.log('webpack 构建过程开始! ');
    });
  }
}

module.exports = ConsoleLogOnBuildWebpackPlugin;
```

`compiler hook` 的 `tap` 方法的第一个参数，应是驼峰式命名的插件名称

关于整个编译生命周期钩子，有如下：

- `entry-option`：初始化 `option`
- `run`
- `compile`：真正开始的编译，在创建 `compilation` 对象之前
- `compilation`：生成好了 `compilation` 对象
- `make` 从 `entry` 开始递归分析依赖，准备对每个模块进行 `build`
- `after-compile`：编译 `build` 过程结束
- `emit`：在将内存中 `assets` 内容写到磁盘文件夹之前
- `after-emit`：在将内存中 `assets` 内容写到磁盘文件夹之后
- `done`：完成所有的编译过程
- `failed`：编译失败的时候

三、常见的Plugin

常见的 `plugin` 有如图所示：

下面介绍几个常用的插件用法：

HtmlWebpackPlugin

在打包结束后，自动生成一个 `html` 文文件，并把打包生成的 `js` 模块引入到该 `html` 中

```
npm install --save-dev html-webpack-plugin
```

```
// webpack.config.js
const HtmlWebpackPlugin = require("html-webpack-plugin");
module.exports = {
  ...
  plugins: [
    new HtmlWebpackPlugin({
      title: "My App",
      filename: "app.html",
      template: "./src/html/index.html"
    })
  ]
};
```

```
<!--./src/html/index.html-->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title><%=htmlWebpackPlugin.options.title%></title>
</head>
<body>
  <h1>html-webpack-plugin</h1>
</body>
</html>
```

在 `html` 模板中，可以通过 `<%=htmlWebpackPlugin.options.XXX%>` 的方式获取配置的值
更多的配置可以自行查找

clean-webpack-plugin

删除（清理）构建目录

```
npm install --save-dev clean-webpack-plugin
```

```
const {CleanWebpackPlugin} = require('clean-webpack-plugin');
module.exports = {
  ...
  plugins: [
    ...,
    new CleanWebpackPlugin(),
    ...
  ]
}
```

mini-css-extract-plugin

提取 `CSS` 到一个单独的文件中

```
npm install --save-dev mini-css-extract-plugin
```

```
const MiniCssExtractPlugin = require('mini-css-extract-plugin');
module.exports = {
  ...,
  module: {
    rules: [
      {
        test: /\.s[ac]ss$/,
        use: [
          {
            loader: MiniCssExtractPlugin.loader
          },
          'css-loader',
          'sass-loader'
        ]
      }
    ]
  },
  plugins: [
    ...,
    new MiniCssExtractPlugin({
      filename: '[name].css'
    }),
    ...
  ]
}
```

DefinePlugin

允许在编译时创建配置的全局对象，是一个 [webpack](#) 内置的插件，不需要安装

```
const { DefinePlugin } = require('webpack')

module.exports = {
  ...
  plugins:[
    new DefinePlugin({
      BASE_URL:"/"
    })
  ]
}
```

这时候编译 [template](#) 模块的时候，就能通过下述形式获取全局对象

```
<link rel="icon" href="<%= BASE_URL%>favicon.ico">
```

copy-webpack-plugin

复制文件或目录到执行区域，如 [vue](#) 的打包过程中，如果我们将一些文件放到 [public](#) 的目录下，那么这个目录会被复制到 [dist](#) 文件夹中

```
npm install copy-webpack-plugin -D
```

```
new CopyWebpackPlugin({
  patterns:[
    {
      from:"public",
      globOptions:{
        ignore:[
          '**/index.html'
        ]
      }
    }
  ]
})
```

复制的规则在 `patterns` 属性中设置：

- `from`：设置从哪一个源中开始复制
- `to`：复制到的位置，可以省略，会默认复制到打包的目录下
- `globOptions`：设置一些额外的选项，其中可以编写需要忽略的文件

参考文献

- <https://webpack.docschina.org/concepts/plugins/>
- <https://baike.baidu.com/item/Plugin>
- <https://segmentfault.com/a/11900000018695134>

面试官：说说webpack的构建流程？

一、运行流程

`webpack` 的运行流程是一个串行的过程，它的工作流程就是将各个插件串联起来

在运行过程中会广播事件，插件只需要监听它所关心的事件，就能加入到这条 `webpack` 机制中，去改变 `webpack` 的运作，使得整个系统扩展性良好

从启动到结束会依次执行以下三大步骤：

- 初始化流程：从配置文件和 `Shell` 语句中读取与合并参数，并初始化需要使用的插件和配置插件等执行环境所需要的参数
- 编译构建流程：从 `Entry` 发出，针对每个 `Module` 串行调用对应的 `Loader` 去翻译文件内容，再找到该 `Module` 依赖的 `Module`，递归地进行编译处理
- 输出流程：对编译后的 `Module` 组合成 `Chunk`，把 `Chunk` 转换成文件，输出到文件系统

初始化流程

从配置文件和 `Shell` 语句中读取与合并参数，得出最终的参数

配置文件默认下为 `webpack.config.js`，也或者通过命令的形式指定配置文件，主要作用是用于激活 `webpack` 的加载项和插件

关于文件配置内容分析，如下注释：

```
var path = require('path');
var node_modules = path.resolve(__dirname, 'node_modules');
var pathToReact = path.resolve(node_modules, 'react/dist/react.min.js');

module.exports = {
  // 入口文件，是模块构建的起点，同时每一个入口文件对应最后生成的一个 chunk。
  entry: './path/to/my/entry/file.js',
  // 文件路径指向(可加快打包过程)。
  resolve: {
    alias: {
      'react': pathToReact
    }
  },
  // 生成文件，是模块构建的终点，包括输出文件与输出路径。
  output: {
    path: path.resolve(__dirname, 'build'),
    filename: '[name].js'
  },
  // 这里配置了处理各模块的 loader，包括 css 预处理 loader，es6 编译 loader，图片处理 loader。
  module: {
    loaders: [
      {
        test: /\.js$/,
        loader: 'babel',
        query: {
          presets: ['es2015', 'react']
        }
      }
    ],
    noParse: [pathToReact]
  },
  // webpack 各插件对象，在 webpack 的事件流中执行对应的方法。
  plugins: [
    new webpack.HotModuleReplacementPlugin()
  ]
};
```

webpack 将 webpack.config.js 中的各个配置项拷贝到 options 对象中，并加载用户配置的 plugins

完成上述步骤之后，则开始初始化 Compiler 编译对象，该对象掌控者 webpack 声明周期，不执行具体的任务，只是进行一些调度工作

```

class Compiler extends Tapable {
  constructor(context) {
    super();
    this.hooks = {
      beforeCompile: new AsyncSeriesHook(["params"]),
      compile: new SyncHook(["params"]),
      afterCompile: new AsyncSeriesHook(["compilation"]),
      make: new AsyncParallelHook(["compilation"]),
      entryOption: new SyncBailHook(["context", "entry"])
      // 定义了很多不同类型的钩子
    };
    // ...
  }
}

function webpack(options) {
  var compiler = new Compiler();
  ...// 检查options,若watch字段为true,则开启watch线程
  return compiler;
}
...

```

`Compiler` 对象继承自 `Tapable`，初始化时定义了很多钩子函数

编译构建流程

根据配置中的 `entry` 找出所有的入口文件

```

module.exports = {
  entry: './src/file.js'
}

```

初始化完成后会调用 `Compiler` 的 `run` 来真正启动 `webpack` 编译构建流程，主要流程如下：

- `compile` 开始编译
- `make` 从入口点分析模块及其依赖的模块，创建这些模块对象
- `build-module` 构建模块
- `seal` 封装构建结果
- `emit` 把各个chunk输出到结果文件

compile 编译

执行了 `run` 方法后，首先会触发 `compile`，主要是构建一个 `Compilation` 对象

该对象是编译阶段的主要执行者，主要会依次下述流程：执行模块创建、依赖收集、分块、打包等主要任务的对象

make 编译模块

当完成了上述的 `compilation` 对象后，就开始从 `Entry` 入口文件开始读取，主要执行 `_addModuleChain()` 函数，如下：


```

_addModuleChain(context, dependency, onModule, callback) {
  ...
  // 根据依赖查找对应的工厂函数
  const Dep = /** @type {DepConstructor} */(dependency.constructor);
  const moduleFactory = this.dependencyFactories.get(Dep);

  // 调用工厂函数NormalModuleFactory的create来生成一个空的NormalModule对象
  moduleFactory.create({
    dependencies: [dependency]
    ...
  }, (err, module) => {
    ...
    const afterBuild = () => {
      this.processModuleDependencies(module, err => {
        if (err) return callback(err);
        callback(null, module);
      });
    };

    this.buildModule(module, false, null, null, err => {
      ...
      afterBuild();
    })
  })
}

```

过程如下：

`_addModuleChain` 中接收参数 `dependency` 传入的入口依赖，使用对应的工厂函数 `NormalModuleFactory.create` 方法生成一个空的 `module` 对象

回调中会把此 `module` 存入 `compilation.modules` 对象和 `dependencies.module` 对象中，由于是入口文件，也会存入 `compilation.entries` 中

随后执行 `buildModule` 进入真正的构建模块 `module` 内容的过程

build module 完成模块编译

这里主要调用配置的 `loaders`，将我们的模块转成标准的 `JS` 模块

在用 `Loader` 对一个模块转换完后，使用 `acorn` 解析转换后的内容，输出对应的抽象语法树（`AST`），以方便 `Webpack` 后面对代码的分析

从配置的入口模块开始，分析其 `AST`，当遇到 `require` 等导入其它模块语句时，便将其加入到依赖的模块列表，同时对新找出的依赖模块递归分析，最终搞清所有模块的依赖关系

输出流程

seal 输出资源

`seal` 方法主要是要生成 `chunks`，对 `chunks` 进行一系列的优化操作，并生成要输出的代码

`webpack` 中的 `chunk`，可以理解为配置在 `entry` 中的模块，或者是动态引入的模块

根据入口和模块之间的依赖关系，组装成一个个包含多个模块的 `Chunk`，再把每个 `Chunk` 转换成一个单独的文件加入到输出列表

emit 输出完成

在确定好输出内容后，根据配置确定输出的路径和文件名

```
output: {
  path: path.resolve(__dirname, 'build'),
  filename: '[name].js'
}
```

在 `Compiler` 开始生成文件前，钩子 `emit` 会被执行，这是我们修改最终文件的最后一个机会
从而 `webpack` 整个打包过程则结束了

小结

参考文献

- <https://github.com/Cosen95/blog/issues/48>
- <https://developer.aliyun.com/article/61047>

面试官：说说webpack的热更新是如何做到的？原理是什么？

一、是什么

`HMR` 全称 `Hot Module Replacement`，可以理解为模块热替换，指在应用程序运行过程中，替换、添加、删除模块，而无需重新刷新整个应用

例如，我们在应用运行过程中修改了某个模块，通过自动刷新会导致整个应用的整体刷新，那页面中的状态信息都会丢失

如果使用的是 `HMR`，就可以实现只将修改的模块实时替换至应用中，不必完全刷新整个应用

在 `webpack` 中配置开启热模块也非常的简单，如下代码：

```
const webpack = require('webpack')
module.exports = {
  // ...
  devServer: {
    // 开启 HMR 特性
    hot: true
    // hotOnly: true
  }
}
```

通过上述这种配置，如果我们修改并保存 `css` 文件，确实能够以不刷新的形式更新到页面中
但是，当我们修改并保存 `js` 文件之后，页面依旧自动刷新了，这里并没有触发热模块

所以，HMR 并不像 Webpack 的其他特性一样可以开箱即用，需要有一些额外的操作
我们需要去指定哪些模块发生更新时进行 HRM，如下代码：

```
if(module.hot){
  module.hot.accept('./util.js',()=>{
    console.log("util.js更新了")
  })
}
```

二、实现原理

首先来看看一张图，如下：

- Webpack Compile：将 JS 源代码编译成 bundle.js
- HMR Server：用来将热更新的文件输出给 HMR Runtime
- Bundle Server：静态资源文件服务器，提供文件访问路径
- HMR Runtime：socket服务器，会被注入到浏览器，更新文件的变化
- bundle.js：构建输出的文件
- 在HMR Runtime 和 HMR Server之间建立 websocket，即图上4号线，用于实时更新文件变化

上面图中，可以分成两个阶段：

- 启动阶段为上图 1 - 2 - A - B

在编写未经过 webpack 打包的源代码后，Webpack Compile 将源代码和 HMR Runtime 一起编译成 bundle 文件，传输给 Bundle Server 静态资源服务器

- 更新阶段为上图 1 - 2 - 3 - 4

当某一个文件或者模块发生变化时，webpack 监听到文件变化对文件重新编译打包，编译生成唯一的 hash 值，这个 hash 值用来作为下一次热更新的标识

根据变化的内容生成两个补丁文件：manifest（包含了 hash 和 chunkId，用来说明变化的内容）和 chunk.js 模块

由于 socket 服务器在 HMR Runtime 和 HMR Server之间建立 websocket 链接，当文件发生改动的时候，服务端会向浏览器推送一条消息，消息包含文件改动后生成的 hash 值，如下图的 h 属性，作为下一次热更细的标识

在浏览器接受到这条消息之前，浏览器已经在上一次 socket 消息中已经记住了此时的 hash 标识，这时候我们会创建一个 ajax 去服务端请求获取到变化内容的 manifest 文件

mainfest 文件包含重新 build 生成的 hash 值，以及变化的模块，对应上图的 c 属性

浏览器根据 manifest 文件获取模块变化的内容，从而触发 render 流程，实现局部模块更新

三、总结

关于 webpack 热模块更新的总结如下：

- 通过 `webpack-dev-server` 创建两个服务器：提供静态资源的服务（express）和Socket服务
- express server 负责直接提供静态资源的服务（打包后的资源直接被浏览器请求和解析）
- socket server 是一个 websocket 的长连接，双方可以通信
- 当 socket server 监听到对应的模块发生变化时，会生成两个文件.json（manifest文件）和.js文件（update chunk）
- 通过长连接，socket server 可以直接将这两个文件主动发送给客户端（浏览器）
- 浏览器拿到两个新的文件后，通过HMR runtime机制，加载这两个文件，并且针对修改的模块进行更新

参考文献

- <https://zhuanlan.zhihu.com/p/138446061>
- <https://github.com/Jocs/jocs.github.io/issues/15>
- <https://juejin.cn/post/6844904134697549832>

面试官：说说你对Webpack的理解？解决了什么问题？

一、背景与模块化

Webpack是一个用于现代JavaScript应用程序的静态模块打包工具。在前端开发的历程中，从最早的文件划分到命名空间方式，再到立即执行函数提供私有空间，都是为了实现模块化。然而，这些方式都存在一些问题，例如全局污染、模块间依赖关系难以维护、加载过程不可控等。

理想的解决方式是，引入一个入口文件，在其中通过代码控制，按需加载其它模块，且模块化规范明确。这时，Webpack应运而生，它的目标是更高效地管理和维护项目中的每一个资源。

二、解决的问题

Webpack解决了现代前端开发中遇到的一系列问题：

1. 模块化开发

Webpack引入了模块化开发，支持各种前端模块类型，如CommonJS、ES Modules等。通过模块化的开发方式，可以将代码拆分成相互依赖的模块，提高代码的可维护性和复用性。

2. 高级特性支持

Webpack支持使用高级特性来加快开发效率和提高安全性。例如，可以使用ES6+和TypeScript来开发脚本逻辑，使用Sass、Less等来编写CSS样式代码，充分发挥现代JavaScript和CSS的优势。

3. 自动监听与热更新

Webpack可以监听文件的变化，并实时将变化反映到浏览器中，提高开发的效率。它支持热模块替换（Hot Module Replacement），在开发过程中无需手动刷新页面，即可看到修改的效果。

4. 代码优化与压缩

Webpack可以对代码进行优化和压缩，减小文件体积，提高加载速度，优化用户体验。它支持合并、分割代码，将公共代码提取为单独的文件，充分利用浏览器的缓存机制。

5. 资源加载控制

Webpack使得所有资源文件的加载都可以通过代码控制。在入口文件中，可以按需加载其他模块，减少不必要的资源加载，降低网络请求，提高性能。

综上所述，Webpack在前端开发中起到了模块整合、编译代码、优化性能、资源加载控制等重要作用，使得前端开发更加高效、便捷、可维护。

三、Webpack的能力

Webpack具备以下几方面的能力：

1. 编译代码能力

Webpack可以编译各种类型的资源文件，如JavaScript、CSS、图片等。它支持对ES6+和TypeScript的编译，将高级语法转换为浏览器可识别的代码，解决了浏览器兼容性问题。

2. 模块整合能力

Webpack可以将项目中的所有模块整合为一个或多个bundle。它会在内部构建一个依赖图，映射项目所需的每个模块，将它们整合在一起。这样可以减少浏览器频繁请求文件的问题，提高页面加载速度和性能。

3. 万物皆可模块能力

Webpack支持将所有资源文件都视为模块。不仅可以处理JavaScript模块，还可以处理HTML、CSS、图片等资源模块。这样使得项目维护更加方便，统一了模块化方案，所有资源文件的加载都可以通过代码控制。

综上所述，Webpack是一个强大的前端构建工具，通过它，我们可以轻松地实现模块化开发、优化代码和性能，提高开发效率和用户体验。

参考文献

- [Webpack官方文档 \(https://webpack.docschina.org/concepts/\)](https://webpack.docschina.org/concepts/)
- [掘金-Webpack深入浅出 \(https://juejin.cn/post/267875652\)](https://juejin.cn/post/267875652)

面试官：说说如何借助webpack来优化前端性能？

一、背景

随着前端的项目逐渐扩大，必然会带来的一个问题就是性能
尤其在大型复杂的项目中，前端业务可能因为一个小小的数据依赖，导致整个页面卡顿甚至奔溃
一般项目在完成后，会通过 `webpack` 进行打包，利用 `webpack` 对前端项目性能优化是一个十分重要的环节

二、如何优化

通过 `webpack` 优化前端的手段有：

- JS代码压缩
- CSS代码压缩
- Html文件代码压缩
- 文件大小压缩
- 图片压缩
- Tree Shaking
- 代码分离
- 内联 chunk

JS代码压缩

[terser](#) 是一个 [JavaScript](#) 的解释、绞肉机、压缩机的工具集，可以帮助我们压缩、丑化我们的代码，让 [bundle](#) 更小

在 [production](#) 模式下，[webpack](#) 默认就是使用 [TerserPlugin](#) 来处理我们的代码的。如果想要自定义配置它，配置方法如下：

```
const TerserPlugin = require('terser-webpack-plugin')
module.exports = {
  ...
  optimization: {
    minimize: true,
    minimizer: [
      new TerserPlugin({
        parallel: true // 电脑cpu核数-1
      })
    ]
  }
}
```

属性介绍如下：

- `extractComments`：默认值为`true`，表示会将注释抽取到一个单独的文件中，开发阶段，我们可设置为 `false`，不保留注释
- `parallel`：使用多进程并发运行提高构建的速度，默认值是`true`，并发运行的默认数量：`os.cpus().length - 1`
- `terserOptions`：设置我们的`terser`相关的配置：
- `compress`：设置压缩相关的选项，`mangle`：设置丑化相关的选项，可以直接设置为`true`
- `mangle`：设置丑化相关的选项，可以直接设置为`true`
- `toplevel`：底层变量是否进行转换
- `keep_classnames`：保留类的名称
- `keep_fnames`：保留函数的名称

CSS代码压缩

[CSS](#) 压缩通常是去除无用的空格等，因为很难去修改选择器、属性的名称、值等

CSS的压缩我们可以使用另外一个插件：[css-minimizer-webpack-plugin](#)

```
npm install css-minimizer-webpack-plugin -D
```

配置方法如下：


```
const CssMinimizerPlugin = require('css-minimizer-webpack-plugin')
module.exports = {
  // ...
  optimization: {
    minimize: true,
    minimizer: [
      new CssMinimizerPlugin({
        parallel: true
      })
    ]
  }
}
```

Html文件代码压缩

使用 `HtmlWebpackPlugin` 插件来生成 `HTML` 的模板时候，通过配置属性 `minify` 进行 `html` 优化

```
module.exports = {
  ...
  plugin:[
    new HtmlWebpackPlugin({
      ...
      minify:{
        minifyCSS:false, // 是否压缩css
        collapseWhitespace:false, // 是否折叠空格
        removeComments:true // 是否移除注释
      }
    })
  ]
}
```

设置了 `minify`，实际会使用另一个插件 `html-minifier-terser`

文件大小压缩

对文件的大小进行压缩，减少 `http` 传输过程中宽带的损耗

```
npm install compression-webpack-plugin -D
```

```
new CompressionPlugin({
  test:/\.css|js$/, // 哪些文件需要压缩
  threshold:500, // 设置文件多大开始压缩
  minRatio:0.7, // 至少压缩的比例
  algorithm:"gzip", // 采用的压缩算法
})
```

图片压缩

一般来说在打包之后，一些图片文件的大小是远远要比 `js` 或者 `css` 文件要来的大，所以图片压缩较为重要

配置方法如下：

```
module: {
  rules: [
    {
      test: /\.png|jpg|gif$/,
      use: [
        {
          loader: 'file-loader',
          options: {
            name: '[name]_[hash].[ext]',
            outputPath: 'images/',
          }
        },
        {
          loader: 'image-webpack-loader',
          options: {
            // 压缩 jpeg 的配置
            mozjpeg: {
              progressive: true,
              quality: 65
            },
            // 使用 imagemin**-optipng 压缩 png, enable: false 为关闭
            optipng: {
              enabled: false,
            },
            // 使用 imagemin-pngquant 压缩 png
            pngquant: {
              quality: '65-90',
              speed: 4
            },
            // 压缩 gif 的配置
            gifsicle: {
              interlaced: false,
            },
            // 开启 webp, 会把 jpg 和 png 图片压缩为 webp 格式
            webp: {
              quality: 75
            }
          }
        }
      ]
    }
  ]
}
```

Tree Shaking

`Tree Shaking` 是一个术语，在计算机中表示消除死代码，依赖于 `ES Module` 的静态语法分析（不执行任何的代码，可以明确知道模块的依赖关系）

在 `webpack` 实现 `Tree shaking` 有两种不同的方案：

- `usedExports`：通过标记某些函数是否被使用，之后通过 `Terser` 来进行优化的
- `sideEffects`：跳过整个模块/文件，直接查看该文件是否有副作用

两种不同的配置方案，有不同的效果

`usedExports`

配置方法也很简单，只需要将 `usedExports` 设为 `true`

```
module.exports = {  
  ...  
  optimization:{  
    usedExports  
  }  
}
```

使用之后，没被用上的代码在 `webpack` 打包中会加入 `unused harmony export mul` 注释，用来告知 `Terser` 在优化时，可以删除掉这段代码

如下面 `sum` 函数没被用到，`webpack` 打包会添加注释，`terser` 在优化时，则将该函数去掉

`sideEffects`

`sideEffects` 用于告知 `webpack compiler` 哪些模块时有副作用，配置方法是在 `package.json` 中设置 `sideEffects` 属性

如果 `sideEffects` 设置为 `false`，就是告知 `webpack` 可以安全的删除未用到的 `exports`

如果有些文件需要保留，可以设置为数组的形式

```
"sideEffects":[  
  "./src/util/format.js",  
  "*.css" // 所有的css文件  
]
```

上述都是关于 `javascript` 的 `tree shaking`，`css` 同样也能够实现 `tree shaking`

`css tree shaking`

`css` 进行 `tree shaking` 优化可以安装 `PurgeCss` 插件

```
npm install purgecss-plugin-webpack -D
```

```
const PurgeCssPlugin = require('purgecss-webpack-plugin')
module.exports = {
  ...
  plugins:[
    new PurgeCssPlugin({
      path:glob.sync(`${path.resolve('./src')}/**/*`), {nodir:true} // src里面的所有文件
      safelist:function(){
        return {
          standard:["html"]
        }
      }
    })
  ]
}
```

- paths：表示要检测哪些目录下的内容需要被分析，配合使用glob
- 默认情况下，Purgecss会将我们的html标签的样式移除掉，如果我们希望保留，可以添加一个safelist的属性

代码分离

将代码分离到不同的 **bundle** 中，之后我们可以按需加载，或者并行加载这些文件

默认情况下，所有的 **JavaScript** 代码（业务代码、第三方依赖、暂时没有用到的模块）在首页全部都加载，就会影响首页的加载速度

代码分离可以分出出更小的 **bundle**，以及控制资源加载优先级，提供代码的加载性能

这里通过 **splitChunksPlugin** 来实现，该插件 **webpack** 已经默认安装和集成，只需要配置即可

默认配置中，chunks仅仅针对于异步（async）请求，我们可以设置为initial或者all

```
module.exports = {
  ...
  optimization:{
    splitChunks:{
      chunks:"all"
    }
  }
}
```

splitChunks 主要属性有如下：

- Chunks，对同步代码还是异步代码进行处理
- minSize： 拆分包的大小，至少为minSize，如何包的大小不超过minSize，这个包不会拆分
- maxSize： 将大于maxSize的包，拆分为不小于minSize的包
- minChunks： 被引入的次数，默认是1

内联chunk

可以通过 **InlineChunkHtmlPlugin** 插件将一些 **chunk** 的模块内联到 **html**，如 **runtime** 的代码（对模块进行解析、加载、模块信息相关的代码），代码量并不大，但是必须加载的

```
const InlineChunkHtmlPlugin = require('react-dev-utils/InlineChunkHtmlPlugin')
const HtmlWebpackPlugin = require('html-webpack-plugin')
module.exports = {
  ...
  plugin:[
    new InlineChunkHtmlPlugin(HtmlWebpackPlugin,['runtime.+\.js/'])
  ]
}
```

三、总结

关于 **webpack** 对前端性能的优化，可以通过文件体积大小入手，其次还可通过分包的形式、减少http请求次数等方式，实现对前端性能的优化

参考文献

- <https://zhuanlan.zhihu.com/p/139498741>

面试官：请你谈谈微信小程序的实现原理？

一、背景

在传统的网页开发中，渲染线程和脚本是互斥的，这也是为什么长时间的脚本运行可能会导致页面失去响应的原因，因为我们常说的 **JS** 是单线程的。而微信小程序则选用了 **Hybrid** 的渲染方式，将视图层和逻辑层分开，实现双线程同时运行。在此模式下，视图层的界面使用 **WebView** 进行渲染，而逻辑层则在 **JSCore** 中运行。

在小程序中，渲染层主要负责界面渲染相关的任务，并在 **WebView** 线程里执行。一个小程序可能存在多个界面，所以渲染层存在多个 **WebView** 线程。而逻辑层则采用 **JsCore** 线程运行 **JS** 脚本，执行的都是与小程序业务逻辑有关的代码。

二、通信

在小程序的渲染层，宿主环境会将 **wxml** 转化成对应的 **JS** 对象。当逻辑层的数据发生变更时，通过宿主环境提供的 **setData** 方法，数据就可以从逻辑层传递到渲染层。然后通过对比数据变化前后的差异，将差异应用在原来的 **Dom** 树上，从而渲染出正确的视图。

例如，用户点击界面上某个按钮，这类反馈需要通知给开发者的逻辑层，以便将对应的处理状态呈现给用户。对于事件的分发处理，微信进行了特殊的处理，将所有的事件拦截后，丢到逻辑层，交给 **JavaScript** 进行处理。

由于小程序是基于双线程的，因此在视图层和逻辑层之间的任何数据传递都是线程间的通信，会有一定的延时。所以在小程序中，页面更新成了异步操作。异步会使得各部分的运行时序变得复杂一些，逻辑层与渲染层需要有一定的机制保证时序正确，在每个小程序页面的生命周期中，存在着若干次页面数据通信。

三、运行机制

小程序启动运行主要有两种情况：

- 冷启动（重新开始）：用户首次打开或者小程序被微信主动销毁后再次打开的情况，此时小程序需要重新加载启动，即为冷启动
- 热启动：用户已经打开过小程序，然后在一定时间内再次打开该小程序，此时无需重新启动，只需要将后台态的小程序切换到前台，这个过程就是热启动

需要注意的是：

1. 小程序没有重启的概念
2. 当小程序进入后台，客户端会维持一段时间的运行状态，超过一定时间后会被微信主动销毁
3. 短时间内收到系统两次以上内存警告，也会对小程序进行销毁，这也就为什么一旦页面内存溢出，页面会奔溃的本质原因了

当开发者在后台发布新版本之后，无法立刻影响到所有现网用户，但最差情况下，也在发布之后 24 小时之内下发新版本信息到用户。每次冷启动时，都会检查是否有更新版本，如果发现有新版本，将会异步下载新版本的代码包，并同时用客户端本地的包进行启动，即新版本的小程序需要等下一次冷启动才会应用上。

四、举例说明

以下是一个简单的示例来展示小程序的数据通信过程：

```
Page({
  data: {
    text: 'Hello, World!'
  },

  changeText: function () {
    this.setData({
      text: 'Hello, OpenAI!'
    });
  },

  onLoad: function () {
    // 页面加载时，打印初始数据
    console.log(this.data.text);
  },

  onReady: function () {
    // 页面初次渲染完成后，改变数据
    this.changeText();
  },

  onShow: function () {
    // 页面显示后，打印改变后的数据
    console.log(this.data.text);
  }
});
```

在这个示例中，我们首先在 `Page` 对象的 `data` 属性中定义了初始数据 `text`。当页面加载完成后，我们在 `onLoad` 生命周期函数中打印出初始数据。然后在页面初次渲染完成后，我们通过 `this.setData` 方法改变数据。最后在页面显示后，我们打印出改变后的数据。

参考资料

◦ <https://zhuanlan.zhihu.com/p/510792920>

面试官：说说微信小程序的生命周期函数有哪些？

微信小程序的生命周期可以分为三部分:应用生命周期、页面生命周期和组件生命周期。下面我会详细介绍每个生命周期的函数及其触发时机。

一、应用生命周期

应用生命周期函数定义在app.js中。

函数名	触发时机
onLaunch	小程序初始化完成时触发(全局只触发一次)
onShow	小程序启动,或从后台进入前台显示时触发
onHide	小程序从前台进入后台时触发
onError	小程序发生脚本错误或API调用报错时触发
onPageNotFound	小程序要打开的页面不存在时触发
onUnhandledRejection	小程序有未处理的Promise拒绝时触发
onThemeChange	系统切换主题时触发

二、页面生命周期

页面生命周期函数在page.js中定义。

函数名	触发时机	常见作用
onLoad	页面加载时触发	用于加载页面数据
onShow	页面显示时触发	页面重新显示时刷新数据
onReady	页面初次渲染完成时触发	获取页面 DOM 节点
onHide	页面隐藏时触发	停止页面动画或计时器等
onUnload	页面卸载时触发	停止网络请求或清理订阅等

三、组件生命周期

组件生命周期函数在component.js中定义。

函数名	触发时机
created	组件实例刚创建时触发
attached	组件进入页面节点树时触发
ready	组件首次渲染完成时触发
moved	组件位置改变时触发
detached	组件离开页面节点树时触发
error	组件方法错误时触发

四、执行顺序

一个典型的完整生命周期顺序为：

1. 小程序打开: onLaunch -> onShow -> onLoad -> onShow -> onReady
2. 进入新页面: onHide -> onLoad -> onShow -> onReady
3. 返回上一页: onUnload -> onShow
4. 小程序切后台: onHide

5. 小程序重启: onLaunch -> onShow

面试官：说说你对微信小程序的理解？优缺点？

什么是微信小程序

微信小程序是一种不需要下载安装即可使用的应用,它实现了应用“触手可及”的梦想,用户扫一扫或者搜一下即可打开应用。

小程序也体现了“用完即走”的理念,用户不用关心是否安装太多应用的问题。应用将无处不在,随时可用,但又无需安装卸载。

小程序的由来

当微信中的WebView逐渐成为移动Web的一个重要入口时,微信就有相关的JS-SDK。JS-SDK解决了移动网页能力不足的问题,但没有解决体验不良的问题。

为此,小程序应运而生,具有:

- 快速加载
- 更强大的能力
- 原生体验
- 易用的微信数据开放
- 高效简单的开发

小程序页面本质上是网页,技术模型与H5类似,只是自定义了WXML标签。

小程序的优缺点

「优点:」

- 随用随走,代替许多APP
- 借助平台流量,易被接受
- 相对安全
- 开发门槛低
- 降低兼容性限制

「缺点:」

- 用户留存率不高
- 体积限制,无法开发大型小程序
- 受限于微信管控,自由度较低

总结

微信小程序结合了APP和网页的优点,为用户和开发者带来便利,但也存在自身的限制。以其便捷性为先,逐步优化体验,或可成为移动端一个重要的形式。

面试官：说说微信小程序中路由跳转的方式有哪些？区别？

一、wx.navigateTo

保留当前页面,跳转到应用内的某个页面。可以使用wx.navigateBack回到原页面。

「特点」: 页面栈增多,占用内存。

「参数」：

```
{  
  url: 'test?id=1' // 需要跳转的应用内非 tabBar 的页面的路径  
}
```

二、wx.redirectTo

关闭当前页面,跳转到应用内的某个页面。

「特点」:不能回到原页面,页面栈不增加。

「参数」：

```
{  
  url: 'test?id=1' // 需要跳转的应用内非 tabBar 的页面的路径  
}
```

三、wx.switchTab

跳转到 tabBar 页面,关闭其他所有非 tabBar 页面。

「特点」:页面栈清空只留下新的 tabBar 页面。

「参数」：

```
{  
  url: 'test' // 需要跳转的 tabBar 页面的路径  
}
```

四、wx.navigateBack

关闭当前页面,返回上一页面或多级页面。

「特点」:页面栈逐级退出,减少内存占用。

「参数」：

```
{  
  delta: 1 // 返回的页面数  
}
```

五、wx.reLaunch

关闭所有页面,打开到应用内的某个页面。

「特点」:页面栈清空只留下新的页面。

「参数」：

```
{
  url: 'test' // 需要跳转的页面路径
}
```

总结

根据需要选择合适的页面跳转方式,控制页面栈数量,合理管理小程序内存占用。

面试官：说说微信小程序的支付流程？

一、前言

微信小程序作为一种新型的应用形态，为电商类小程序提供了非常完善、优秀、安全的支付功能。通过调用微信的API，可以实现快捷便捷的支付体验。

场景流程

流程大致如下：

1. 用户通过分享或扫描二维码进入商户小程序，用户选择购买，完成选购流程。
2. 调起微信支付控件，用户开始输入支付密码。
3. 密码验证通过，支付成功。商户后台得到支付成功的通知。
4. 返回商户小程序，显示购买成功。
5. 微信支付公众号下发支付凭证。

二、详细流程

以电商小程序为例，支付流程图如下：

具体操作流程如下：

1. 打开小程序，点击直接下单。此时，小程序会调用 `wx.login` 获取用户临时登录凭证 `code`，然后发送到后端服务器换取 `openId`。
2. 用户下单，小程序需要将购买的商品Id，商品数量，以及用户的openId传送到服务器。
3. 服务器在接收到商品Id、商品数量、openId后，生成订单数据。同时，通过签名算法，向微信支付发送请求，获取预付单信息（`prepay_id`）。然后，服务器会将获取的数据再次进行签名，向小程序端响应必要的信息。
4. 小程序端在获取对应的参数后，调用 `wx.requestPayment()` 发起微信支付，唤醒支付工作台，进行支付。
5. 接下来，用户会进行微信支付密码、指纹等验证。确认支付之后，微信后台进行鉴权，直接返回给前端支付的结果。前端收到返回数据后对支付结果进行展示。
6. 最后，微信后台在给前端返回支付的结果后，也会向后台返回一个支付结果。后台通过这个支付结果来更新订单的状态。

以下是后端响应数据和 `wx.requestPayment` 方法所需要的参数示例：

```
wx.requestPayment({
  // 时间戳
  timeStamp: "",
  // 随机字符串
  nonceStr: "",
  // 统一下单接口返回的 prepay_id 参数值
  package: "",
  // 签名类型
  signType: "",
  // 签名
  paySign: "",
  // 调用成功回调
  success () {},
  // 失败回调
  fail () {},
  // 接口调用结束回调
  complete () {}
})
```

参数表如下所示：

属性	类型	默认值	必填	说明
timeStamp	string		是	时间戳，从 1970 年 1 月 1 日 00:00:00 至今的秒数，即当前的时间
nonceStr	string		是	随机字符串，长度为32个字符以下
package	string		是	统一下单接口返回的 prepay_id 参数值，提交格式如：prepay_id=***
signType	string	MD5	否	签名算法，应与后台下单时的值一致
paySign	string		是	签名，具体见微信支付文档
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

三、支付安全

对于支付的安全，微信小程序采用了多重安全措施。首先，微信支付是基于HTTPS传输加密数据，采用安全协议来保证信息传输的安全性。其次，微信支付服务提供了商户证书下发、报文签名以及商户数据包（MD5）校验等多种安全机制来防止信息被篡改。此外，用户的支付信息如银行卡号、密码、验证码

等信息均不会保存在商户系统或者微信系统，即使在微信服务器被攻破，也无法获取用户的真实支付信息。最后，微信支付还为用户提供了交易过程中的安全保障，通过多种风险控制系统为用户拦截可疑交易。

四、结束

小程序支付和以往的网页、APP微信支付大同小异，可以说小程序的支付变得更加简洁，不需要设置支付目录、域名授权等操作。其优化了支付流程，同时在安全方面也做出了保障，从而为用户提供了更好的支付体验。

参考文献

- https://pay.weixin.qq.com/wiki/doc/apiv3/open/pay/chapter2_8_0.shtml
- <https://juejin.cn/post/6844903895970349064>

面试官：说说微信小程序的登录流程？

微信小程序的登录流程主要可以概括为以下步骤：

一、获取code

1. 用户在小程序内点击登录按钮
2. 调用wx.login()获取code

二、获取openid

1. 将code发送到开发者服务器
2. 开发者服务器使用code向微信服务器获取openid和session_key

三、校验openid

1. 开发者服务器拿到openid,去数据库查询是否已存在
2. 若不存在,表示新用户,创建新记录
3. 若存在,校验用户信息,生成自定义登录态

四、返回登录态

1. 开发者服务器将登录态返回给小程序
2. 小程序存储登录态到本地

五、校验登录态

1. 小程序重新启动时可校验登录态
2. 调用wx.checkSessionAPI验证登录态是否过期
3. 过期则重新执行登录流程,未过期则继续使用

综上,微信小程序的登录流程主要分为获取openid、生成和校验登录态几个步骤。具体实现还需要开发者服务器配合,参考微信官方文档。

面试官：说说提高微信小程序的应用速度的手段有哪些？

您好,提高微信小程序应用速度的主要手段可以概括如下：

一、加载优化

- 控制代码包体积,及时清理无用代码
- 使用子包按需加载,减少主包体积
- 预加载子包,避免首次点击卡顿

二、渲染优化

- onLoad阶段发请求,不等待onReady
- 缓存本地数据,减少不必要的网络请求
- 合并setData调用,避免频繁刷新界面
- 抽离自定义组件,减少主界面渲染压力

三、首屏优化

- 预请求数据,利用缓存数据构建首屏
- 采用骨架屏 placeholders 占位
- 提前预加载资源包

四、启动优化

- 显示加载图标避免白屏
- 合理制定启动图片大小

五、长列表优化

- 虚拟列表,减少DOM节点
- 预加载可视区域外图片
- 图片懒加载,滚动到可视区再加载

六、打包策略

- 压缩代码,上传时启用代码压缩
- 按功能拆分子包
- 删除无用文件,精简资源体积

综上,通过加载、渲染、启动各阶段的优化,可以有效提升小程序的整体运行速度和流畅度。

作为面试官，为什么我推荐组件库作为前端面试的亮点？

图解算法小册



最近整理了一本算法小册，感兴趣的同学可以加我微信 [linwu-hi](#) 进行获取



目录页

数据结构	9
一、链表	9
二、双向链表	15
三、队列	27
四、栈	30
五、优先队列	33
六、哈希表	36
七、图	40
八、堆	63
九、LRU	70
十、不相交集	76
十一、布隆过滤器	83
十二、树	90
十三、字典树	135
算法	142
十四、排序	142
十五、搜索	189
十六、链表	196
十七、栈	199
十八、队列	226
十九、双指针	234

@稀土掘金技术社区
www.coding-time.cn

前言

在上一篇[作为面试官，为什么我推荐微前端作为前端面试的亮点？](https://juejin.cn/post/7252342216843296828) (https://juejin.cn/post/7252342216843296828)反馈效果不错，我接着出第二篇 [组件库专题](#)，主要是我选择的方向，前端同学都可以很轻易尝试，这样项目上就增加很多亮点了

“

大厂面试的时候，我也看到很多候选人写了 [xx组件的封装](#)，很少见过 [二次组件库的封装](#) 或者 [维护开源组件库](#)，其实这些都是项目上的亮点，一般面试官如果看到，都会详细考察

”

本文将会以 [antd](#) [Element](#) [vant](#)等等 组件库为例子，会进行分析对比

为什么需要二次封装组件库？

“

实际工作中，我们在项目中需要自定义 [主题色](#)，[更改按钮样式](#)，[自定义图标](#)，自定义 [table组件](#) 等等，这些都可以基于 [antd](#) 组件库进行二次封装，减少重复工作，提升开发效率。

”

所以我们在封装的时候按照下面这四个原则进行思考就行了，另外本身封装组件库对于项目来说也是没有任何风险，因为一开始我们把 [PropsType](#) 直接进行 [转发](#)，内部再进行增加业务的功能，这样就是达到完全的 [解耦](#)

- **「统一风格」**：在一个大的项目或者多个相关的项目中，保持一致的界面风格和交互方式是非常重要的。通过二次封装，我们可以定义统一的样式和行为，减少不一致性。
- **「降低维护成本」**：当底层的组件库更新时，我们可能需要在项目的多个地方进行修改。但是如果有了自己的封装，只需要在封装层面进行更新即可，这大大降低了维护成本。
- **「增加定制功能」**：有些时候，我们需要在原有组件库的基础上增加一些特定的功能，如特定的验证、错误处理等。二次封装提供了这样的可能。
- **「提高开发效率」**：在一些常用的功能（如表单验证、全局提示等）上，二次封装可以提供更方便的API，提高开发效率。

请结合一个组件库设计的过程,谈谈前端工程化的思想

当我们结合一个组件库设计的过程来谈论前端工程化的思想时，需要理清这些要点：

1. 使用 [Lerna](#) 进行多包管理：通过 [Lerna](#) 来管理多个包（组件），实现组件级别的解耦、独立版本控制、按需加载等特性。

```
# 安装 Lerna
npm install -g lerna

# 初始化一个 Lerna 仓库
lerna init

# 创建 "Button" 组件包
lerna create button --yes
```

2. 规范化提交：使用规范化的提交信息可以提高 Git 日志的可读性，并且可以通过 conventional commits 自动生成 CHANGELOG。可以使用 commitizen、commitlint 等工具来配置。

安装相关工具

```
npm install commitizen cz-conventional-changelog --save-dev
```

```
// package.json
{
  "scripts": {
    "commit": "git-cz"
  },
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

3. 代码规范化：通过 ESLint、Prettier 等工具实现代码规范化和格式化，并封装为自己的规范预设。

安装相关工具

```
npm install eslint prettier eslint-plugin-prettier eslint-config-prettier --save-dev
```

```
// .eslintrc.js
module.exports = {
  extends: ['eslint:recommended', 'plugin:prettier/recommended'],
};

// .prettierrc.js
module.exports = {
  singleQuote: true,
  trailingComma: 'es5',
};
```

4. 组件开发调试：需要考虑热更新编译、软链接引用等问题，以方便在开发过程中进行组件的调试。

```
// packages/button/src/Button.js
import React from 'react';

const Button = ({ type = 'primary', onClick, children }) => {
  return (
    <button className={`button ${type}`} onClick={onClick}>
      {children}
    </button>
  );
};

export default Button;
```

5. 文档站点：可以基于 dumi 搭建文档站点，并实现 CDN 加速、增量发布等优化。可以使用 surge 实现 PR 预览。

```
<!-- packages/button/docs/index.md -->
# Button

A simple button component.

## Usage

import { Button } from 'button-library';

const MyComponent = () => {
  return <Button onClick={() => alert('Button clicked!')}>Click Me</Button>;
};

### Props

| Name   | Type           | Default | Description           |
| ----- | -
| type   | `primary` \| `secondary` | `primary` | The type of the button. |
| onClick | `function`      |          | Event handler for click event. |
```

6. 单元测试：需要考虑 jest、enzyme 等工具的配合使用，生成测试覆盖率报告。

```
# 安装相关工具
npm install jest enzyme enzyme-adapter-react-16 react-test-renderer --save-dev
```

```
// packages/button/src/Button.test.js
import React from 'react';
import { mount } from 'enzyme';
import Button from './Button';

describe('Button', () => {
  it('renders without crashing', () => {
    const wrapper = mount(<Button>Click Me</Button>);
    expect(wrapper.exists()).toBe(true);
  });

  it('calls onClick function when clicked', () => {
    const onClickMock = jest.fn();
    const wrapper = mount(<Button onClick={onClickMock}>Click Me</Button>);

    wrapper.find('button').simulate('click');
    expect(onClickMock).toHaveBeenCalledTimes(1);
  });
});
```

7. 按需加载：需要配合 `babel-plugin-import` 实现按需加载，即在编译时修改导入路径来实现组件的按需加载。

```
# 安装相关工具
npm install babel-plugin-import --save-dev
```

```
// .babelrc
{
  "plugins": [
    [
      "import",
      {
        "libraryName": "button-library",
        "style": "css"
      }
    ]
  ]
}
```

8. 组件设计：需要考虑响应式、主题、国际化、TypeScript 支持等问题，以保证组件的灵活性和可扩展性。


```
// packages/button/src/Button.js
import React from 'react';
import PropTypes from 'prop-types';

const Button = ({ type = 'primary', onClick, children }) => {
  return (
    <button className={`button ${type}`} onClick={onClick}>
      {children}
    </button>
  );
};

Button.propTypes = {
  type: PropTypes.oneOf(['primary', 'secondary']),
  onClick: PropTypes.func,
  children: PropTypes.node.isRequired,
};

export default Button;
```

9. 发布前的自动化脚本：需要编写自动化脚本来规范发布流程，确保发布的一致性和可靠性。

```
// package.json
{
  "scripts": {
    "prepublish": "npm run lint && npm run test",
    "lint": "eslint .",
    "test": "jest"
  }
}
```

10. 发布后的处理：考虑补丁升级、文档站点同步发布等问题，以便及时修复问题并提供最新的文档。

11. 制定 Contributing 文档：制定 Contributing 文档可以降低开源社区贡献的门槛，并确保社区成员了解如何参与项目。处理 issues 和 PR 需要有专人负责。

如何对一个组件库进行测试？

“

首先需要明确，组件库的测试大致可以分为两类：一类是针对组件本身的功能和性能的测试（例如，单元测试、性能测试），另一类是针对组件在集成环境下的行为和性能的测试（例如，集成测试、系统测试）。

”

「1. 功能测试（单元测试）」

通常来说，组件的功能测试可以通过单元测试来完成。单元测试的目的是验证组件的单个功能是否按照预期工作。这通常可以通过编写测试用例来完成，每个测试用例针对一个特定的功能。

```
import { Button } from '../src/Button';

test('Button should do something', () => {
  const component = new YourComponent();
  // your test logic here
  expect(component.doSomething()).toBe('expected result');
});
```

「2. 边界测试」

边界测试是一种特殊的功能测试，用于检查组件在输入或输出达到极限或边界条件时的行为。

```
test('Button should handle boundary condition', () => {
  const component = new YourComponent();
  // test with boundary value
  expect(component.handleBoundaryCondition('boundary value')).toBe('expected result');
});
```

「3. 响应测试」

响应测试通常涉及到 UI 组件在不同的设备或屏幕尺寸下的行为。这可能需要使用端到端（E2E）测试工具，如 Puppeteer、Cypress 等。

```
import { test } from '@playwright/test';

test('Button should be responsive', async ({ page }) => {
  await page.goto('http://localhost:3000/your-component');
  const component = await page.$('#your-component-id');
  expect(await component.isVisible()).toBe(true);

  // Simulate a mobile device
  await page.setViewportSize({ width: 375, height: 812 });
  // Check the component under this condition
  // your test logic here
});
```

「4. 交互测试」

交互测试也可以通过端到端（E2E）测试工具来完成。

```
test('Button should handle interactions', async ({ page }) => {
  await page.goto('http://localhost:3000/your-component');
  const component = await page.$('#your-component-id');

  // Simulate a click event
  await component.click();
  // Check the result of the interaction
  // your test logic here
});
```

「5. 异常测试」

异常测试用于验证组件在遇到错误或非法输入时能否正确处理。这通常可以通过在测试用例中模拟错误条件来完成。

```
test('Button should handle errors', () => {
  const component = new YourComponent();
  // Test with illegal argument
  expect(() => {
    component.doSomething('illegal argument');
  }).toThrow('Expected error message');
});
```

「6. 性能测试」

性能测试用于验证组件的性能，例如，加载速度、内存消耗等。

```
import { performance } from 'perf_hooks';

test('Button should have good performance', () => {
  const start = performance.now();
  const component = new YourComponent();
  component.doSomething();
  const end = performance.now();
  const duration = end - start;
  expect(duration).toBeLessThan(50); // Expect the operation to finish within 50 ms
});
```

「7. 自动化测试」

单元测试、集成测试和系统测试都可以通过自动化测试工具进行。例如，Jest 和 Mocha 可以用于自动化运行 JavaScript 单元测试，Puppeteer 和 Selenium 可以用于自动化运行端到端测试。

```
module.exports = {
  roots: ['<rootDir>/src'],
  testMatch: ['**/__tests__/**/*.+(ts|tsx|js)', '**/?(*.)+(spec|test).+(ts|tsx|js)'],
  transform: {
    '^.+\\.?(ts|tsx)$': 'ts-jest'
  }
};
```

Element-UI 的多语言方案是怎么设计的？

“

Element UI 使用了 Vue 的插件 `vue-i18n` 实现多语言支持，具体的设计和实现过程如下：

”

「1. 定义语言包」

首先，Element UI 定义了一个 JavaScript 对象作为语言包。每种语言都有一个对应的语言包，例如：

```
export default {
  el: {
    colorpicker: {
      confirm: 'OK',
      clear: 'Clear'
    },
    // ...other components
  }
};
```

「2. 加载语言包」

Element UI 提供了一个 `i18n` 方法用于加载语言包。

```
import ElementUI from 'element-ui';
import locale from 'element-ui/lib/locale/lang/en';

Vue.use(ElementUI, { locale });
```

「3. 使用语言包」

Element UI 的组件会使用 `$t` 方法获取语言包中的文本。例如：

```
<template>
  <el-button>{{ $t('el.button.confirm') }}</el-button>
</template>
```

在这个例子中，按钮的文本会根据当前的语言包来显示。

「4. 集成 vue-i18n」

如果你的项目中已经使用了 `vue-i18n`，Element UI 会优先使用 `vue-i18n` 提供的 `$t` 方法。你可以这样配置：

```
import Vue from 'vue';
import VueI18n from 'vue-i18n';
import ElementUI from 'element-ui';
import enLocale from 'element-ui/lib/locale/lang/en';
import zhLocale from 'element-ui/lib/locale/lang/zh-CN';

Vue.use(VueI18n);

const messages = {
  en: {
    message: 'hello',
    ...enLocale // 或者用 Object.assign({ message: 'hello' }, enLocale)
  },
  zh: {
    message: '你好',
    ...zhLocale // 或者用 Object.assign({ message: '你好' }, zhLocale)
  }
};

const i18n = new VueI18n({
  locale: 'zh', // set locale
  messages, // set locale messages
});

Vue.use(ElementUI, {
  i18n: (key, value) => i18n.t(key, value)
});
```

在这个例子中，我们先加载了 `vue-i18n`，然后定义了两种语言的语言包（英文和中文）。最后，我们配置了 Element UI 使用 `vue-i18n` 的 `$t` 方法。

这样，Element UI 的组件就能够根据 `vue-i18n` 的语言设置显示对应的文本。

组件库如何实现在线主题定制的？

1. 使用 CSS 变量定义样式

将组件的样式使用 CSS 变量定义,这样可以通过改变 CSS 变量的值来修改样式。

```
:root {
  --primary-color: #1890ff;
}

.btn {
  background: var(--primary-color);
}
```

2. 提供主题文件进行配置

让用户可以通过导入自定义的主题文件来覆盖默认样式。

```
// theme.js
export default {
  '--primary-color': '#409eff'
}
```

3. 在线主题编辑器

提供一个在线工具,用户可以在工具中配置主题,生成主题文件。

工具会提交主题配置,服务器端接收后动态编译生成新的样式,并返回给前端。

4. 前端应用新样式

前端通过加载服务器返回的 CSS 文件来应用新的主题样式,实现样式更新而无需重新打包。

```
// 请求主题文件
fetchTheme(theme).then(css => {
  // 动态创建style标签,插入css
  const style = document.createElement('style');
  style.innerHTML = css;
  document.head.appendChild(style);
})
```

5. 持久化主题配置

将用户主题配置持久化本地存储,这样每次访问都可以应用上次选定的主题。

组件库的类型定义应该怎样设计?

“

组件库的类型定义设计取决于很多因素,包括库的大小、复杂度、可能的使用场景等。

”

「1. 定义全局类型 versus 定义组件Props类型」

在组件库中,我们经常需要定义一些可以在多个组件之间共享的全局类型,以及针对特定组件的props类型。例如:

```
// 全局类型
export interface Size {
  width: number;
  height: number;
}

// 组件Props类型
export interface ButtonProps {
  size?: Size;
  label: string;
  onClick?: () => void;
}
```


「2. 类型导出应该集中还是分散?」

是否集中导出类型取决于组件库的大小和复杂度。对于小型库，可以在一个单独的文件中集中导出所有类型；对于大型库，可能需要将类型定义分散在各个组件文件中，然后在一个单独的文件中重新导出它们。例如：

```
// 在各个组件文件中定义和导出类型
// button.ts
export interface ButtonProps { /*...*/ }

// 在一个单独的文件中重新导出所有类型
// types.ts
export type { ButtonProps } from './button';
```

「3. 如何设计类型层级关系?类型复用?」

在设计类型时，应尽可能地利用 TypeScript 的类型系统来构建类型层级关系，并复用类型。例如，你可以使用类型交叉（&）和类型联合（|）来复用类型：

```
type SmallSize = { width: number; height: number };
type LargeSize = SmallSize & { depth: number };

type Size = SmallSize | LargeSize;
```

「4. 类型定义要充分还是精简?」

类型定义应尽可能精简，同时提供足够的信息来描述类型的形状和行为。避免使用 `any` 或 `unknown` 类型，除非有特别的理由。例如：

```
// 不好的类型定义
interface ButtonProps {
  [key: string]: any; // 这不提供任何有关props的信息
}

// 好的类型定义
interface ButtonProps {
  size?: Size;
  label: string;
  onClick?: () => void;
}
```

总的来说，设计好的类型定义可以提高代码的可读性和可维护性，同时减少运行时错误。

组件库的渐进升级策略应该怎么设计?

组件库的渐进升级策略通常会涉及到版本控制、向下兼容性、废弃通知以及旧版本的兼容性等多个方面。这种策略的主要目的是在保持库的稳定性和功能性的同时，尽可能地减少对用户的影响。

「1. 版本控制策略」

组件库通常遵循[语义化版本 \(https://semver.org/lang/zh-CN/\)](https://semver.org/lang/zh-CN/) (SemVer) 规范进行版本控制。在语义化版本中，每个版本号都由三部分组成：主版本号、次版本号和补丁版本号。

例如，版本号为 1.2.3 表示主版本号为 1，次版本号为 2，补丁版本号为 3。

- 主版本号 (Major)：当你做了不兼容的 API 修改
- 次版本号 (Minor)：当你做了向下兼容的功能性新增
- 补丁版本号 (Patch)：当你做了向下兼容的问题修复

「2. 向下兼容处理」

向下兼容性是指在升级组件库时，保证新版本不会破坏旧版本的功能。例如，如果新版本的一个组件删除了一个属性，而这个属性在旧版本中是必需的，那么这个变化就不是向下兼容的。

在进行不向下兼容的变化时，应在主版本号上进行增加，以警告用户可能需要修改他们的代码。

「3. 功能被废弃怎么通知用户升级?」

当一个功能或者组件被废弃时，应在库的文档、更新日志以及相关的 API 文档中明确注明。在代码中，可以通过添加警告或者错误信息来提醒用户：

```
function deprecatedFunction() {  
  console.warn('Warning: deprecatedFunction is deprecated and will be removed in the next major version.');
```

// 功能的原始实现

```
}
```

「4. 兼容旧版本的方案」

兼容旧版本的策略取决于特定的需求和资源。一种常见的策略是在主版本升级后，继续维护旧版本的一个分支，以便在必要时进行修复和改进。例如，如果当前版本是 2.x.x，那么可以维护一个 1.x.x 的分支。

在实践中，以上的策略和方法可能需要根据具体的情况进行调整。一个好的渐进升级策略应能够平衡新功能的引入、旧功能的废弃以及向下兼容性的维护。

组件库的按需加载实现中存在哪些潜在问题,如何解决?

“

按需加载（也称为代码拆分）是现代前端开发中常见的一种优化手段，可以有效地减少应用的初始加载时间。对于组件库来说，它使用户只加载和使用他们真正需要的组件，而不是加载整个库。

”

babel-plugin-import

“

「Babel 插件」：使用如 babel-plugin-import 的 Babel 插件可以在编译时将导入整个库的语句转换为仅导入使用的组件。

”

```
``javascript
import { Button } from 'your-ui-lib';
// 在编译时，babel-plugin-import 将上面的语句转换为以下语句：
// import Button from 'your-ui-lib/button';
``
```

tree-shaking

Webpack、Rollup 等工具都已经支持了 Tree shaking。在项目的配置中开启 Tree shaking，然后使用 ES Modules 的导入导出语法，即可实现按需加载。

但是在使用 Tree shaking 的时候，有一个需要特别注意的地方，就是“副作用（side effects）”。

有些模块的代码可能会在导入时执行一些副作用，例如改变全局变量、改变导入模块的状态等。这种情况下，即使模块中的部分导出没有被使用，由于其副作用，也不能被 Tree shaking 移除。否则，可能会导致程序运行出错。

例如，在 CSS in JS 的库中，可能存在这样的代码：

```
import './styles.css'; // 有副作用，改变了全局的样式
```

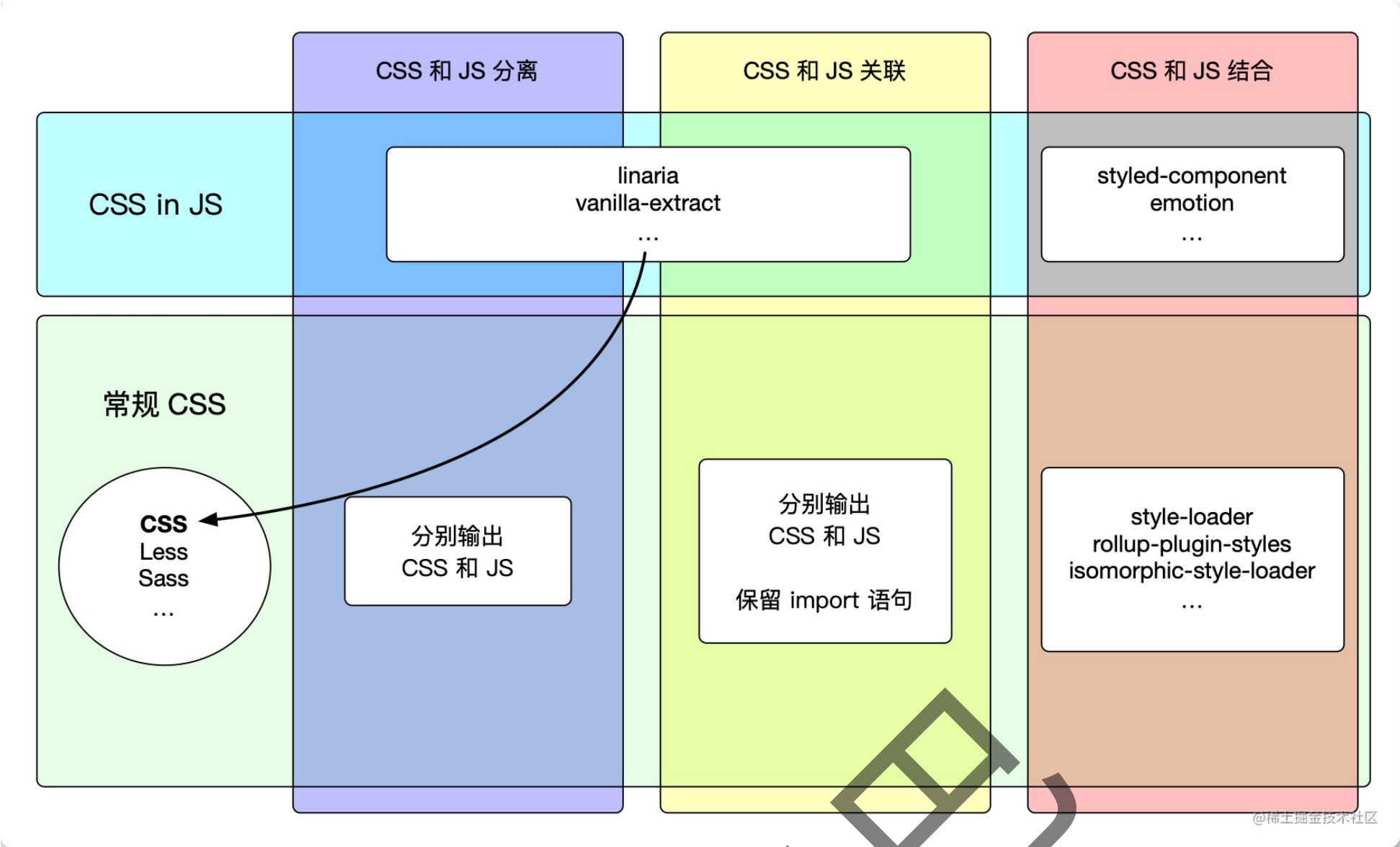
在这种情况下，你需要在 package.json 中显式地指定模块的副作用，以防止它们被错误地移除：

```
{
  "name": "your-library",
  "sideEffects": [
    "./src/styles.css"
  ]
}
```

如果你的库没有任何副作用，你可以将 sideEffects 设置为 false：

```
{
  "name": "your-library",
  "sideEffects": false
}
```

样式如何实现真正的按需加载?避免样式重复打包?



	样式和逻辑分离	样式和逻辑结合	样式和逻辑关联
开发打包流程	中等	简单	复杂
输出文件	JS 文件和 CSS 文件	JS 文件	JS 文件和 CSS 文件
使用方法	分别引入 JS 和 CSS	只引入 JS	只引入 JS
按需加载	需要额外支持	支持	支持
性能影响	无	带额外 runtime，可能有影响	无
SSR	支持	需要额外支持（部分方案不支持）	支持（可能需要使用者调整配置）
支持写法	常规 CSS / 零运行时 CSS in JS	常规 CSS / CSS in JS	常规 CSS / 零运行时 CSS in JS
关键样式提取	自行处理	支持	自行处理

这种方案中,组件的CSS和JS在代码层面上是分离的,开发时写在不同的文件里。在打包时生成独立的逻辑文件和样式文件。

优点:

- 适用面广,可以支持不同的框架和技术栈。
- 支持SSR,样式处理留给使用者。
- 可以直接提供源码,便于主题定制。

缺点:

- 使用时需要分别引入逻辑和样式,按需加载实现复杂, 需要借助 [babel-plugin-import](https://link.juejin.cn/?target=https%3A%2F%2Fgithub.com%2Fumijs%2Fbabel-plugin-import) (<https://link.juejin.cn/?target=https%3A%2F%2Fgithub.com%2Fumijs%2Fbabel-plugin-import>)、[unplugin-vue-components](https://link.juejin.cn/?target=https%3A%2F%2Fgithub.com%2Fantfu%2Funplugin-vue-components) (<https://link.juejin.cn/?target=https%3A%2F%2Fgithub.com%2Fantfu%2Funplugin-vue-components>)等。
- 样式文件打包可能存在冗余。

适合需要高适用性和灵活性的组件库。

样式和逻辑结合

这种方案将CSS和JS打包在一起,输出单一的JS文件。主要有两种实现形式:

1. CSS in JS:样式以对象或字符串形式存在在JS中。
2. 将CSS打包进JS:通过构建工具,将CSS文件内容注入到JS中。

优点:

- 使用简单,只需要引入JS即可。
- 天然支持按需加载。

缺点:

- 需要额外的runtime,可能影响性能。
- 难以利用浏览器缓存。
- SSR需要框架额外支持。

样式和逻辑关联

这种方案下,虽然CSS和JS在源码层分离,但组件内会直接引用样式,且输出文件中保留import语句。

优点:

- 使用简单,只引入JS即可。
- 支持按需加载。

缺点:

- 对构建和SSR都有一定要求。
- 样式编译复杂。

设计一个组件库的 CI/CD 和发布流程。

“
可以参考antd

当你设计一个组件库的 CI/CD 和发布流程时,可以考虑以下步骤:

「1. 分支管理:」

开发者在开发新特性或修复 bug 时,应该在新的分支(通常称为 `feature` 分支)上进行开发。完成开发后,提交一个 pull request 到 `main` 或 `master` 分支,并进行代码审查。

```
git checkout -b feature/new-component
# 开发过程...
git add .
git commit -m "Add new component"
git push origin feature/new-component
```

「2. 代码检查:」

使用如 ESLint、Stylelint 等工具进行代码检查,使用 Jest 等工具进行单元测试和覆盖率检查。这些步骤可以在提交代码时或者 pull request 的过程中自动进行。

例如，可以在 `package.json` 中添加如下 `scripts`：

```
{
  "scripts": {
    "lint": "eslint --ext .js,.jsx,.ts,.tsx src",
    "test": "jest"
  }
}
```

并在 CI/CD 工具中（如 GitHub Actions、Jenkins 等）配置相应的任务：

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v2
      - name: Use Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '14'
      - name: Install dependencies
        run: npm ci
      - name: Run lint
        run: npm run lint
      - name: Run tests
        run: npm run test
```

「3. 版本管理：」

在合并代码并发布新版本前，需要确认新的版本号，并生成相应的 `changelog`。可以使用如 `standard-version` 这样的工具自动化这个过程。

```
npx standard-version
```

「4. 构建：」

使用如 Webpack、Rollup 等工具进行构建，生成可以在不同环境（如浏览器、Node.js）下使用的代码。

```
npm run build
```

「5. 发布：」

将构建好的代码发布到 `npm`，同时更新文档网站。

「6. 部署：」

部署到github pages或者自建服务

如何实现button按钮

[jcode \(https://code.juejin.cn/pen/7261131681648607288\)](https://code.juejin.cn/pen/7261131681648607288)

```
import React, { CSSProperties, FC, MouseEvent, ReactNode } from 'react';

interface ButtonProps {
  lock?: boolean;
  classNames?: Record<string, string>;
  danger?: boolean;
  disabled?: boolean;
  ghost?: boolean;
  href?: string;
  htmlType?: 'button' | 'submit' | 'reset';
  icon?: ReactNode;
  loading?: boolean | { delay: number };
  shape?: 'default' | 'circle' | 'round';
  size?: 'large' | 'middle' | 'small';
  styles?: Record<string, CSSProperties>;
  target?: string;
  type?: 'primary' | 'dashed' | 'link' | 'text' | 'default';
  onClick?: (event: MouseEvent<HTMLButtonElement | HTMLAnchorElement>) => void;
  children?: ReactNode;
}

const Button: FC<ButtonProps> = ({
  lock,
  classNames,
  danger,
  disabled,
  ghost,
  href,
  htmlType = 'button',
  icon,
  loading,
  shape,
  size,
  styles,
  target,
  type = 'default',
  onClick,
  children
}) => {
  const baseClassName = 'button';
```

```

const className = [
  baseClassName,
  type && `${baseClassName}--${type}`,
  size && `${baseClassName}--${size}`,
  shape && `${baseClassName}--${shape}`,
  disabled && `${baseClassName}--disabled`,
  danger && `${baseClassName}--danger`,
  ghost && `${baseClassName}--ghost`,
  loading && `${baseClassName}--loading`,
  lock && `${baseClassName}--lock`,
].filter(Boolean).join(' ');

const handleClick = (e: MouseEvent<HTMLButtonElement | HTMLAnchorElement>) => {
  if (disabled) {
    e.preventDefault();
  } else if (onClick) {
    onClick(e);
  }
};

return href ? (
  <a
    className={className}
    href={href}
    target={target}
    onClick={handleClick}
  >
    {children}
  </a>
) : (
  <button
    className={className}
    type={htmlType}
    disabled={disabled}
    onClick={handleClick}
  >
    {children}
  </button>
);
};

export default Button;

```

如何实现modal组件

[jcode \(https://code.juejin.cn/pen/7261132915558645819\)](https://code.juejin.cn/pen/7261132915558645819)

```

interface IModalProps {
  afterClose?: () => void;
  bodyStyle?: CSSProperties;
  cancelButtonProps?: React.ButtonHTMLAttributes<HTMLButtonElement>;
  cancelText?: ReactNode;

```

```

centered?: boolean;
closelcon?: boolean | ReactNode;
confirmLoading?: boolean;
destroyOnClose?: boolean;
focusTriggerAfterClose?: boolean;
footer?: ReactNode;
forceRender?: boolean;
getContainer?: HTMLElement | (() => HTMLElement) | string | false;
keyboard?: boolean;
mask?: boolean;
maskClosable?: boolean;
maskStyle?: CSSProperties;
modalRender?: (node: ReactNode) => ReactNode;
okButtonProps?: React.ButtonHTMLAttributes<HTMLButtonElement>;
okText?: ReactNode;
okType?: string;
style?: CSSProperties;
title?: ReactNode;
open?: boolean;
width?: string | number;
wrapClassName?: string;
zIndex?: number;
onCancel?: (e: React.MouseEvent<HTMLButtonElement>) => void;
onOk?: (e: React.MouseEvent<HTMLButtonElement>) => void;
afterOpenChange?: (open: boolean) => void;
}

```

```

const Modal: React.FC<IModalProps> = ({
  children,
  title = "",
  onCancel,
  onOk,
  open = false,
  mask = true,
}) => {
  return (
    <
      {mask && <div className="modal-mask" style={{display: open ? 'block' : 'none'}}></div>}
      {open && (
        <div className="modal" style={{display: 'block'}}>
          <h2 className="modal-title">{title}</h2>
          <div className="modal-body">{children}</div>
          <div className="modal-footer">
            <button className="modal-footer-cancel" onClick={onCancel}>
              Cancel
            </button>
            <button className="modal-footer-ok" onClick={onOk}>
              OK
            </button>
          </div>
        </div>
      )}
    </div>
  )}
</>

```

```
);  
};  
  
Modal.info = function(props: IModalProps) {  
  const div = document.createElement('div');  
  document.body.appendChild(div);  
  
  function remove() {  
    ReactDOM.unmountComponentAtNode(div);  
    document.body.removeChild(div);  
  }  
  
  function onCancel(e: React.MouseEvent<HTMLButtonElement>) {  
    if (props.onCancel) {  
      props.onCancel(e);  
    }  
    remove();  
  }  
  
  function onOk(e: React.MouseEvent<HTMLButtonElement>) {  
    if (props.onOk) {  
      props.onOk(e);  
    }  
    remove();  
  }  
  
  ReactDOM.render(  
    <Modal {...props} onCancel={onCancel} onOk={onOk} open={true} />,  
    div  
  );  
};
```

如何实现高性能Tree组件

实现Tree组件的核心思路是什么？

Tree组件的核心思路是将原始的嵌套children数据结构平铺成一维数组,然后通过计算每个节点的深度(deep)、层级关系等信息,在渲染时动态计算缩进宽度、连接线等,从而实现树形结构的可视化。

「Tree组件如何实现高性能大数据渲染？」

- 将原始树形数据平铺为一维数组,便于后续计算
- 计算出实际需要渲染的节点数据,过滤隐藏的节点
- 利用虚拟列表技术只渲染可视区域的数据,实现大数据量的高效渲染

```
function flattenTreeData(treeData = [], parent = null) {
  const nodes = [];

  treeData.forEach((node) => {
    const newNode = {
      ...node,
      parent,
    };

    nodes.push(newNode);

    if (newNode.children) {
      nodes.push(...flattenTreeData(newNode.children, newNode));
    }
  });

  return nodes;
}
```

「如何计算Tree组件中节点的各种状态(展开/折叠、选中等)?」

- 展开/折叠状态根据ExpandedKeys计算
- 复选框选中状态需要考虑受控/非受控,严格受控模式,及父子节点关联
- 需要递归计算父节点和子节点的状态
- 利用平铺后的索引进行相关节点查询

```
function flattenTreeData(treeData = [], parent = null) {
  const nodes = [];

  treeData.forEach((node) => {
    const newNode = {
      ...node,
      parent,
    };

    nodes.push(newNode);

    if (newNode.children) {
      nodes.push(...flattenTreeData(newNode.children, newNode));
    }
  });

  return nodes;
}
```

「Tree组件的交互如何实现?点击节点展开折叠,复选框状态切换等」

- 点击展开折叠通过更新节点自身状态、可视状态及ExpandedKeys实现
- 点击复选框需要递归更新父子节点的状态,及相关keys
- 计算并保存实时状态,通过回调函数通知外部

```
function toggleExpanded(nodes, node) {
  return nodes.map((currentNode) => {
    if (currentNode === node) {
      return {
        ...currentNode,
        expanded: !currentNode.expanded,
      };
    }
    return currentNode;
  });
}

// 在渲染时计算缩进:
function renderNode(node) {
  const indentLevel = getIndentLevel(node);
  const style = {
    paddingLeft: `${indentLevel * 16}px`,
  };

  return (
    <div style={style} onClick={() => handleNodeClick(node)}>
      {node.label}
    </div>
  );
}
```

如何实现高性能表格Table组件？

可参考[ali-react-table：高性能 React 表格组件](https://ali-react-table.js.org/blog/2020/04/15/ali-react-table-introduction/) (<https://ali-react-table.js.org/blog/2020/04/15/ali-react-table-introduction/>)

表格组件的性能瓶颈主要在哪里？

- 渲染大量 DOM;
- 频繁的更新渲染,如选中行状态改变引起整个表格重新渲染。

如何优化表格组件的渲染性能？

- 只渲染必要的列：


```

const columnsToRender = columns.filter(column => column.shouldRender);

return (
  <table>
    <thead>
      <tr>
        {columnsToRender.map(column => (
          <th key={column.key}>{column.title}</th>
        ))}
      </tr>
    </thead>
    <tbody>
      {data.map(row => (
        <tr key={row.id}>
          {columnsToRender.map(column => (
            <td key={column.key}>{row[column.key]}</td>
          ))}
        </tr>
      ))}
    </tbody>
  </table>
);

```

2. 细粒度更新，只更新变化行/列。在React中，可以使用 [React.memo](#) 或者 [shouldComponentUpdate](#) 来避免不必要的重渲染：

```

function Row({ data, columns }) {
  return (
    <tr>
      {columns.map(column => (
        <Cell key={column.key} data={data[column.key]} />
      ))}
    </tr>
  );
}

const areEqual = (prevProps, nextProps) => {
  return prevProps.data === nextProps.data && prevProps.columns === nextProps.columns;
};

export default React.memo(Row, areEqual);

```

3. 采用虚拟化技术，只渲染可视区的行。可以使用第三方库如 [react-window](#) 或者 [react-virtualized](#) 来实现：

```
import { FixedSizeList as List } from "react-window";

function Table({ data, columns }) {
  const Row = ({ index, style }) => (
    <div style={style}>
      {columns.map(column => (
        <Cell key={column.key} data={data[index][column.key]} />
      ))}
    </div>
  );

  return (
    <List
      height={500}
      itemCount={data.length}
      itemSize={35}
    >
      {Row}
    </List>
  );
}
```

4. 使用Web Workers来处理数据处理或计算密集型任务：

```
// 创建一个新的 worker
const worker = new Worker('worker.js');

// 向 worker 发送数据
worker.postMessage(data);

// 监听 worker 的消息
worker.addEventListener('message', (event) => {
  // 更新表格数据
  updateTable(event.data);
});
```

在 `worker.js` 中：

```
self.addEventListener('message', (event) => {
  // 处理数据
  const processedData = processData(event.data);

  // 发送处理后的数据
  self.postMessage(processedData);
});
```

基于「Web Components」封装组件库

“

这个可以当做拓展了解一下，目前有越来越多的开源组件库往这个方向发展，可以参考这篇文章[如何基于 WebComponents 封装 UI 组件库](https://juejin.cn/post/7096265630466670606)
(<https://juejin.cn/post/7096265630466670606>)

”

阿里巴巴